

REPARTO DE PROBLEMAS

Universidad de Ibagu  - XXXIX MARATON NACIONAL DE PROGRAMACION ACIS/REDIS 2026

Marca quien ya lo leyo. En Tematica anota de que se trata; en Dificultad, que tan duro se ve y si vale la pena ahora.

Problema	Juan	Felipe	Reyes	Tematica	Dificultad
A	☐	☐	☐		
B	☐	☐	☐		
C	☐	☐	☐		
D	☐	☐	☐		
E	☐	☐	☐		
F	☐	☐	☐		
G	☐	☐	☐		
H	☐	☐	☐		
I	☐	☐	☐		
J	☐	☐	☐		
K	☐	☐	☐		
L	☐	☐	☐		
M	☐	☐	☐		
N	☐	☐	☐		

GATILLOS - del enunciado a la tecnica

Para cuando el problema no te suena a nada. Busca la senal, quedate con el nombre de la tecnica y de ahi usa el indice. El numero de la derecha es la seccion.

Cuanto aguanta cada complejidad (~10^8 operaciones en 1 s)

n <= 11	O(n!) - permutaciones, next_permutation	
n <= 20	O(2^n * n) - bitmask DP	2
n <= 25	O(2^n) - si no cabe, meet in the middle	
n <= 40	meet in the middle, O(2^(n/2))	
n <= 450	O(n^3) - Floyd, DP de intervalos	3
n <= 5.000	O(n^2 log n)	
n <= 10.000	O(n^2) - DP de dos indices	2
n <= 100.000	O(n sqrt n) - Mo, bloques	1
n <= 200.000	O(n log^2 n) - binaria + estructura	
n <= 1.000.000	O(n log n) - sort, Fenwick, Dijkstra	1
n <= 100.000.000	O(n) - dos punteros, criba, prefijos	
n hasta 10^18	formula cerrada, log, o binaria	5

Lo que dice el enunciado -> la tecnica

maximizar el minimo / minimizar el maximo	binaria sobre la respuesta	11
cuantas formas hay, modulo 10^9+7	DP, o combinatoria con inverso modular	7
el k-esimo mas grande / mas pequeno	binaria + contar, o heap de tamaño k	11
minimo numero de movimientos o pasos	BFS (todas las aristas pesan 1)	3
los pesos son solo 0 y 1	BFS 0-1 con deque	3
pesos cualesquiera, ninguno negativo	Dijkstra con heap	3
puede haber pesos negativos	Bellman-Ford	3
distancia entre TODOS los pares	Floyd-Warshall (n <= 450)	3
conectar todo al menor costo	MST: Kruskal + DSU	3
quedaron en el mismo grupo?	DSU (union-find)	3
hay ciclo en un grafo dirigido?	Kahn, o DFS de tres colores	3
dependencias, en que orden hacerlo	orden topologico	3
dos nodos se alcanzan mutuamente	SCC (Kosaraju o Tarjan)	3
si quito esta arista se desconecta?	puentes y puntos de articulacion	3
emparejar dos grupos, uno a uno	Kuhn, o flujo maximo	3
separar / cortar al menor costo	min cut = max flow (Dinic)	3
recorrer cada arista exactamente una vez	camino o circuito euleriano	3
ancestro comun, o distancia en un arbol	LCA con binary lifting	3
aplicar el mismo salto k veces	binary lifting en grafo funcional	3
a lo sumo k de algo, en un tramo	ventana deslizante	13
subarreglo de suma maxima	Kadane	2
consultas de rango y el arreglo NO cambia	prefijos, o sparse table para min/max	1
consultas de rango y SI cambia	Fenwick si es suma, segtree si no	1
sumar un valor a todo un rango	segment tree con lazy propagation	1
contar pares desordenados	inversiones: merge sort o Fenwick	1
el primer mayor (o menor) a la derecha	pila monotona	1
existe una asignacion verdadero/falso?	2-SAT	3
cuantos numeros entre A y B cumplen X	digit DP	2
subsecuencia comun pero por pedazos seguidos	LCS con bloques de K: dos tablas	2
palindromos, todos los centros	Manacher	9
donde aparece este patron	KMP, o funcion Z	9
muchas comparaciones de subcadenas	hashing mod 2^61-1, y LCP por binaria	9
prefijos de un diccionario	trie	9
MUCHOS patrones contra un mismo texto	Aho-Corasick (trie + suffix links)	9
preguntas sobre TODAS las subcadenas	suffix automaton	9
la k-esima subcadena en orden alfabetico	bajar por el suffix automaton con un dp	9
valores gigantes pero pocos distintos	compresion de coordenadas	1
fechas dadas como dia, mes y ano	calendario a mano; OJO el ano bisiesto	13
partir un total en partes exactamente iguales	el tamaño tiene que DIVIDIR el total	5
n entre 30 y 45: 2^n no cabe pero 2^(n/2) si	meet in the middle, partir y cruzar	13
maximo o suma en el camino de un arbol, con cambios	heavy-light decomposition	3
contar pares coprimos o filtrar por divisor comun	inversion de Mobius	6
llenar una cuadrícula con dominos	broken profile DP; n es el lado CORTO	2
escoger intervalos sin solapar pero CON pesos	DP ordenando por fin, no el greedy	2

Lo que dice el enunciado -> la tecnica (cont.)

cuantos menores llevo hasta ahora	Fenwick indexado por VALOR comprimido	1
subsecuencia creciente mas larga	LIS en O(n log n) con lower_bound	2
mochila, escoger con un tope	DP de mochila	2
dos juegan optimo, quien gana	Grundy / Nim, o DP de estados	8
area, contener, o cruzarse en el plano	primitivas: cross, dot, orientacion	4
el poligono mas chico que cubre los puntos	convex hull	4
donde se cortan rectas, segmentos, circulos	intersecciones y distancias	4
los dos puntos mas cercanos	closest pair por barrido	4
cuantos divisores, o factorizar muchos	criba con menor factor primo	6
dividir en modulo	inverso modular (Fermat)	6
congruencias simultaneas	CRT, aunque no sean coprimos	6
inverso o ecuacion con modulo NO primo	Euclides extendido	6
cuantos de 1..n son coprimos con n	phi de Euler	6
es primo? factorizar un numero de 10^18	Miller-Rabin + Pollard rho	6
recurrencia lineal con n hasta 10^18	exponenciacion de matrices	5
multiplicar polinomios, sumas de pares	NTT mod 998244353	5
la funcion sube y despues baja	busqueda ternaria	11
numero de cientos de cifras, sin modulo	Python: el int no tiene tope	13

Trampas que cuestan el problema entero

dos numeros de 10^9 multiplicados	se sale de int: long long siempre	14
tres numeros de 10^9 multiplicados	se sale de long long: __int128	14
Fenwick con indice 0	es 1-indexado: i & -i con i=0 no avanza	1
el enunciado pide el orden de ENTRADA	no ordenes ni uses set: guarda el indice	
hay empates	ordena por (clave, indice) y define el criterio	
aparecen decimales	nunca ==, usa eps; mejor aun, pasa a enteros	14
el grafo puede venir desconectado	el DFS va en un for por TODOS los nodos	3
n = 0 y n = 1	el borde que siempre se olvida	
la respuesta cabe pero el intermedio no	reordena la formula o divide antes	14
recursion profunda en Python	memo con dict, no @lru_cache (revienta)	13
da WA y no se ve por que	stress test contra fuerza bruta	13

Secciones del notebook

Data Structures	25 entradas	1
Dynamic Programming	18 entradas	2
Graph	41 entradas	3
Geometry	16 entradas	4
Math	26 entradas	5
Number Theory	25 entradas	6
Combinatorics	16 entradas	7
Game Theory	4 entradas	8
String	19 entradas	9
Greedy	22 entradas	10
Binary Search	4 entradas	11
Constructive Algorithms	2 entradas	12
Implementation	30 entradas	13
Formulario	18 entradas	14
Divide and Conquer	1 entrada	15
CSES: Advanced Techniques	1 entrada	16
CSES: Dynamic Programming	5 entradas	17
CSES: Graph Algorithms	21 entradas	18
CSES: Introductory Problems	24 entradas	19
CSES: Mathematics	1 entrada	20
CSES: Range Queries	25 entradas	21
CSES: Sorting and Searching	9 entradas	22
CSES: String Algorithms	21 entradas	23
CSES: Tree Algorithms	2 entradas	24
Hub LeetCode (referencia, del equipo)	188 entradas, secciones 25-46	

XXXIX MARATON NACIONAL DE PROGRAMACION ACIS/REDIS 2026

----- NOTEBOOK -----

Universidad de Ibague

Participantes:

Juan David Reyes Cure
Juan Felipe Rodriguez Barbosa
Juan Esteban Rodriguez Castellanos

October 02, 2026

Entrenador:

Stiven Correa

Contents

1 Data Structures

1.1 Brute Force - Count Majority Subarrays	11
1.2 Compresion de Coordenadas	11
1.3 Conteo de Inversiones con Fenwick - Frosh	11
1.4 Conteo de Inversiones y Redondeo Exacto - Sky Line	12
1.5 DSU, Fenwick y Sparse Table	12
1.6 Fenwick Tree (Consultas offline) - D Query	13
1.7 Fenwick Tree (Offline Queries) - K Query	14
1.8 Fenwick Tree (Prefix Minimum, Dominancia 3D) - The Day Of The Competitors	14
1.9 Hash Map y Ordenamiento	15
1.10 Hash Set - Dangerous Dive	15
1.11 Heap con Borrado Perezoso - Aquarium Bubble Lights	15
1.12 Mo's Algorithm - Wine Production	16
1.13 Ordered Set (PBDS)	17
1.14 Persistencia (DFS + Rollback) - The Declaration Of Independence	17
1.15 Prefix Sums - Count Majority Subarrays2	17
1.16 Prefix Sums con Primera Aparicion - Accounting Qualifying Con Prefix	18
1.17 Segment Tree - Block Placement Queries	18
1.18 Segment Tree - Monarchy In Vertigo	19
1.19 Segment Tree - Running A Penitentiary	20
1.20 Segment Tree con Lazy (Sumar a un Rango)	20
1.21 Segment Tree sobre Coordenadas Comprimidas - Emergency Rations	21
1.22 Small-to-Large Merging (DSU on Tree) - Distinct Colors	22
1.23 Sparse Table sobre Bloques Iguales - Frequent	23
1.24 Sqrt Decomposition - Holes	23
1.25 Tree Construction - Binary Tree	24

2 Dynamic Programming

2.1 Binary Search - Patches	25
2.2 Bitmask DP - Music Festival	25
2.3 Bitmask SOS DP - El Juego Del Calamar	25

2.4 Camino en Grilla con Balance de Parentesis - Has Valid Path	26
2.5 Convex Hull Trick	27
2.6 Digit DP	27
2.7 Distancia de Damerau-Levenshtein	27
2.8 Divide and Conquer Optimization	28
2.9 DP con Arreglo de Diferencias - Balanced Balloons	28
2.10 DP sobre Intervalos con Bandera de Inversion - Artic Virus	29
2.11 Interval Scheduling - Yuyuan Market	29
2.12 LCS con Bloques de K Caracteres	30
2.13 LIS en $O(n \log n)$ con Reconstruccion	30
2.14 Prefix Sums - Zig Zag Array	31
2.15 SOS (Sum over Subsets) + Inclusion-Exclusion - How Many Teams	31
2.16 Steiner Tree - Joining Capitals	32
2.17 Subset Sum (Knapsack) - Karamell	32
2.18 Valor Esperado - Revenge Of The Salary Of At Coder Inc	33

3 Graph

3.1 2-SAT (Kosaraju) - Signal Coverage	33
3.2 Articulation Points (Tarjan) - Submerging Islands	35
3.3 BFS - Ladder Length	35
3.4 BFS - Boss	35
3.5 BFS - Bacon Number	36
3.6 BFS Backtracking - Find Ladders	38
3.7 BFS sobre Grafo Producto (dos fichas a la vez) - Infiltration Route	38
3.8 Biconnected Components (Block-Cut Tree) + LCA - Hedge Mazes	40
3.9 Bipartite Matching - Attacking Rooks	42
3.10 Bipartite Matching - Knights In The Board	42
3.11 Bipartite Matching (Algoritmo de Kuhn) - School Dance	43
3.12 Circuito Euleriano (Algoritmo de Hierholzer) - Mail Delivery	43
3.13 Clique Maximo (Bron-Kerbosch con Pivote) - Holy Network	44
3.14 Clique Maximo con Backtracking Iterativo - Holy Network (clique Iterativo)	45

Contents (cont.)

3.15 Componentes Fuente con Kosaraju - Lumina Juan	45	4.16 Triangle Classification - AROD	80
3.16 Componentes Fuente de la Condensacion (Tarjan) - Lumina	46	5 Math	
3.17 Componentes Fuertemente Conexas (Kosaraju) - Planets And Kingdoms	47	5.1 2-Coloreo por Suma Potencia de 2 - Drug Test	82
3.18 DFS - Artwork	48	5.2 Ad-hoc - Attention To Meeting	82
3.19 DFS - Denouncing Mafia	48	5.3 Algebra - The Wisdom Of Master Wei	82
3.20 DFS - A Mazing!	49	5.4 Bit Manipulation - Meme Wars	82
3.21 Dijkstra - Fantastic Robot	49	5.5 Brute Force - Antiarithmetic	82
3.22 Dijkstra con Nodos Virtuales - Chinese Innovation	51	5.6 Construction - Ezraft And Array	83
3.23 Dijkstra con Reconstruccion de Camino	51	5.7 Divisores de la Suma Total - Puntos Equiespaciados En Un Circulo	83
3.24 DSU (enfoque INCORRECTO para este problema) - Holy Network (DSU, Incorrecta)	52	5.8 Eliminacion Gaussiana (mod p)	84
3.25 Existencia de Camino Euleriano con Aristas Ocultas - Apple Pie	52	5.9 Exponenciacion de Matrices (Recurrencias Lineales con n hasta 10^{18})	85
3.26 Flujo Maximo (Dinic), Corte Minimo y Matching Bipartito (Kuhn) - Flujo Maximo Y Matc...	54	5.10 Fibonacci (Fast Doubling) - Fibonacci Sum	85
3.27 Heavy-Light Decomposition - Los Ratones III	55	5.11 Fibonacci (Zeckendorf-like Greedy) - Getting In Shape	86
3.28 Kruskal Reconstruction Tree - Trucks	57	5.12 Fibonacci Fast Doubling - Pou	86
3.29 LCA (Binary Lifting) - Subway Lines	57	5.13 Greedy - A Healthy Menu	87
3.30 LCA con Binary Lifting	58	5.14 Implementation - Hour For A Run	87
3.31 Max Flow - Download Speed	59	5.15 Logaritmos - Gatuno's Descent Into Psychopathy	87
3.32 Max Flow - March Of The Penguins	59	5.16 Matrix Exponentiation - Buses	87
3.33 Max Flow (Binary Search + Dinic) - Gasoline	60	5.17 NTT (Multiplicar Polinomios mod 998244353)	87
3.34 MST con Prim (Grafo Denso) - Brute	61	5.18 Numeros Romanos	88
3.35 Puentes y Puntos de Articulacion	62	5.19 Optimization - Satoru Gojo	88
3.36 Quitar el DAG de Rutas Minimias - Almost Shortest Path	63	5.20 Paridad - Call	89
3.37 SCC (Tarjan) y 2-SAT - SCC Y 2 SAT	64	5.21 Polynomial Composition - Coach	89
3.38 State-Space BFS - Folding Machine	64	5.22 Prefix Sums - Triangles	90
3.39 Strongly Connected Components (Tarjan) - Proving Equivalences	65	5.23 Recurrencia (Fibonacci) - Fractions Are Better When Continued	90
3.40 Tree Degree Analysis - Colmillos	66	5.24 Recurrencias y busqueda de intervalo factible - Investigating Quadradômeda	90
3.41 Tree Reconstruction (BFS por capas) - Expansion Of The Road Network	66	5.25 Recursion (Simulacion tipo Josephus) - Metal Detector	91
4 Geometry		5.26 Simulacion - Cahokia Ruins	91
4.1 2D Difference Array (Coordinate Compression) - Office Space	67	6 Number Theory	
4.2 Area Circulo-Poligono	68	6.1 Aritmetica Modular (Euclides Extendido, Phi, CRT General)	91
4.3 Bounding Box de una Elipse Rotada - Eclipse	69	6.2 BFS sobre Residuos - Only1s0s	92
4.4 Contar Cruces de Cuerdas con Fenwick - Lonely Creatures	69	6.3 Big Exponentiation - Uj (cpp)	93
4.5 Convex Hull y Barrido Angular - Guard Deployment	71	6.4 Big Exponentiation - Uj (java)	93
4.6 Coordenadas Polares	73	6.5 Criba de Eratostenes (Solo Impares)	93
4.7 Geometria 1D - Kilometric Intersection	74	6.6 Criba de Factor Primo Minimo + Inclusion-Exclusion - Ingredients That May Harm You	93
4.8 Intersecciones, Distancias y Par Mas Cercano - Intersecciones Y Distancias	74	6.7 Diferencia de Cuadrados - Square Diff	94
4.9 K Vecinos Mutuos y Clasificacion de Componentes - Celestial Veins	75	6.8 Divisibilidad - Archmage's Crystals	94
4.10 K-Means (Clustering)	76	6.9 Divisors - Aesthetics In Poetry	94
4.11 Primitivas y Convex Hull	78	6.10 Exponenciacion Modular (Torre de Exponentes) - Exponentiation II	95
4.12 Probabilidad de Circulo Contenido - Omens	78	6.11 Factorizacion de un Numero Gigante Redondeado - Fuzzy Factorization	95
4.13 Punto en Poligono Convexo ($O(\log n)$)	78	6.12 Factorizacion en Digitos	95
4.14 Rotating Calipers (Par Mas Lejano)	79	6.13 GCD and LCM - Journeygrapes	96
4.15 Shoelace (Area de Poligono)	80		

Contents (cont.)

6.14 Miller-Rabin y Pollard Rho (Factorizar hasta 10^{18})	96	9.8 Monotonic Stack - Remove Duplicates1	114
6.15 Modular Arithmetic - Broul	97	9.9 Palindromo Ignorando Simbolos - Just Palindromes	114
6.16 Modular Arithmetic - Creating Multiples	97	9.10 Palindromos No Solapados (Greedy) - Max Palindromes	115
6.17 Modulo de Entero Gigante	98	9.11 Parentesis Balanceados (Todas las Variantes)	115
6.18 Modulo de un Binario Gigante - Binary Dozens	98	9.12 Partition and Permutation - Policianacional	117
6.19 Modulo de un Binario Gigante (Python) - Binary Dozens	98	9.13 Remove Duplicate Letters - Remove Duplicates	117
6.20 Number Theory (Criba y Factorizacion) - Counting Divisors	98	9.14 Rotacion Minima (Dos Punteros) - Fingerprints	118
6.21 Number Theory (GCD y Divisores) - The Seals Of Shanghai	99	9.15 Simulation - Process Str	119
6.22 Number Theory (LCM via Factorizacion de Ciclos) - Permutation Rounds	100	9.16 Suffix Array + LCP - Couple Of Bip Bop	119
6.23 Reconstruccion con Divisores - Remainders	100	9.17 Suffix Automaton - Beautiful Words	120
6.24 Sieve of Eratosthenes - Prime Sieve	100	9.18 Trie + DP de Segmentacion (Maximo) - Spacebar Tokenizer	121
6.25 Ternas Pitagoricas - Pythagorean	101	9.19 Z-Function - A String Problem	122
7 Combinatorics		10 Greedy	
7.1 Combinatoria (Factoriales Modulares) - Binomial Coefficients	101	10.1 Binary Search - Maratona Brasileira De Popcorn	122
7.2 Combinatoria Avanzada (Lucas, Catalan, Stirling)	102	10.2 Bit Manipulation - Lecographically Maximum	123
7.3 Construction - Farmpiggy And Subset Sum	102	10.3 Budget Optimization - Posters	123
7.4 Conteo de Distintos con Inventario de Frecuencias - Total Numbers	103	10.4 Buy Low Sell High (Comprar y Vender Acciones)	123
7.5 Conteo de Inversiones (cadena repetida) - Inversions	103	10.5 Constructive Algorithms - Nomad	124
7.6 Euler's Formula - Pizza Cutter	103	10.6 DP - Exhibition Of Clownfish	124
7.7 Formula Cerrada con Inverso Modular - Number Of Sets	104	10.7 Emparejar Extremos - Axles	125
7.8 Paridad de Binomiales (Kummer) - Less Coint Tosses	105	10.8 Huffman	125
7.9 Permutation Cycles - Lines Of Containers	105	10.9 Huffman sobre Frecuencias Variables en el Tiempo - Efficient Encoding	125
7.10 Probabilidad (Paradoja del cumpleaños) - Mount Rushmore And Birthdays	106	10.10 Interval Scheduling - Interval Scheduling Greedy	126
7.11 Probabilidad (Valor esperado con inverso modular) - Frangolino Ali Na Mesa	106	10.11 Maximo Conjunto a Distancia d en Arbol - Oficinas A Distancia D En Arbol	126
7.12 Ranking - Palindromic Lexi Small	107	10.12 Prefijos sobre Secuencias Balanceadas - Incompatible Pairs	127
7.13 Ranking de Combinaciones - Discrete Catalog	107	10.13 Prefix Sums - Maximizing Advertising	127
7.14 Suma de Subconjuntos (2^n) - Group Work	108	10.14 Puente y Linterna - The Bridge At Night	128
7.15 Tecnica de Contribucion - Bytelandia's Stones	108	10.15 Puente y Linterna (version con indices) - Bridge Juan	128
7.16 Triangulos en Poligono Convexo	108	10.16 Segment Counting - Cerros	129
8 Game Theory		10.17 Simulation - Kathmandu	129
8.1 Misere Nim y Grundy Acotado	108	10.18 Sliding Window	129
8.2 Nim con Segment Tree - Juan Vs Frank	109	10.19 Sorted Pairing - Balatro	129
8.3 Posiciones Perdedoras (potencias de 2) - Digitsduel	109	10.20 Sorting - Finding Forks	130
8.4 Sprague-Grundy - Marbles	109	10.21 Sorting (Prefix Sums) - Coven Complications	130
9 String		10.22 Two Pointers (Container With Most Water)	130
9.1 Brute Force Matching - Enigma	110	11 Binary Search	
9.2 Chequeo de Palindromo	110	11.1 Busqueda sobre la Respuesta con Fracciones Exactas - Clean Streets	130
9.3 Hashing de Cadenas (mod $2^{61}-1$)	111	11.2 Busqueda Ternaria (Real y Entera)	132
9.4 KMP para Contar Ocurrencias Solapadas - Judgmental Crowd	111	11.3 Greedy (busqueda binaria sobre la respuesta) - Minas Gerais' Walls	132
9.5 KMP sobre Cadena Recursiva (Duplicacion) - Song Of The Sirens	112	11.4 Two Pointers - Black Hills Golden Jewels	133
9.6 KMP y Funcion Z	113	12 Constructive Algorithms	
9.7 Maximo Solape Sufijo-Prefijo (KMP) - Sign	114	12.1 Hashing - Handling The Blocks	133

Contents (cont.)

12.2	Secuencias de Langford - Doubled Sequence II	133	14.10	Rectas y Conicas	152
13	Implementation		14.11	Secuencias Notables	153
13.1	Ad Hoc - Toilet Seat	134	14.12	STL Indispensable	153
13.2	Ad Hoc - Rock Paper Scissors Tournament	134	14.13	Sumas y Series	154
13.3	Ad Hoc - Zero Or One	135	14.14	Tabla ASCII	154
13.4	Ad Hoc - Collatz Polynomial	135	14.15	Teoria de Numeros	155
13.5	Ad Hoc - Unraveling Monty Hall	135	14.16	Trampas de C++	155
13.6	Arreglos - Buffoon	135	14.17	Trigonometria	156
13.7	Backtracking (Enumeracion de bits desconocidos) - Harmonics With Interference	136	14.18	Trucos de Bits	156
13.8	Brute Force - Cake Hater	136	15	Divide and Conquer	
13.9	Brute Force - LLMS	136	15.1	Conteo de Inversiones con Merge Sort - Frosh Merge Sort	157
13.10	Circular Queue - Eeny Meeny	137	16	CSES: Advanced Techniques	
13.11	Clasificacion de Cadenas por Prefijo - Joker Or Judger	137	16.1	Meet in the Middle	157
13.12	Conjuntos - João João	138	17	CSES: Dynamic Programming	
13.13	Constructive - A Story About Tea	138	17.1	Counting Numbers - Digit DP con Estado 'ya Empezo'	158
13.14	Conteo de Records - Harder Horizons	138	17.2	Counting Tilings - Broken Profile DP (Dominos)	158
13.15	Cycle Detection - Switches	138	17.3	Elevator Rides - Bitmask DP con Estado de Dos Partes	159
13.16	Date Parsing - Crochet	139	17.4	Increasing Subsequence II - Contar Subsecuencias Crecientes con Fenwick	159
13.17	Dia Siguiente y Anos Bisiestos	139	17.5	Projects - Weighted Interval Scheduling	160
13.18	Fuerza Bruta $O(n^2)$ sobre Subperiodos - Accounting Qualifying	140	18	CSES: Graph Algorithms	
13.19	Generacion de Numeros con Digitos Distintos - Division Digitos Distintos	140	18.1	Building Roads - DSU (Union-Find)	160
13.20	Greedy - Escalator	141	18.2	Building Teams - BFS de Bicoloreo (Bipartito)	161
13.21	Lectura de Entrada	141	18.3	Counting Rooms - Flood Fill en Grilla (BFS)	162
13.22	Lights Out (Primera Fila Forzada) - Turnswitch	143	18.4	Course Schedule - Orden Topologico (Kahn)	162
13.23	Maquina de Estados por Objeto - Fraud	143	18.5	Cycle Finding - Bellman-Ford (Ciclo Negativo)	163
13.24	Maximizar Area del Rectangulo Envolvente - Kings Conquest	144	18.6	Flight Discount - Dijkstra sobre Grafo de Capas	163
13.25	Python para Competencias (Enteros Gigantes y Atajos)	144	18.7	Flight Routes - K Caminos Mas Cortos	164
13.26	Simulacion con Parsing deCodigo - Impossible Primebox	145	18.8	Game Routes - DP sobre DAG en Orden Topologico	165
13.27	Simulation - Enigma Of The Jewelry Case	146	18.9	High Score - Bellman-Ford (Camino Maximo)	165
13.28	Stress Test (Encontrar el Caso que da WA)	146	18.10	Investigation - Dijkstra con Cuatro Cantidades a la Vez	166
13.29	Template Base	147	18.11	Labyrinth - BFS con Reconstruccion de Camino	167
13.30	Umbral de Frecuencia - Lottery	147	18.12	Longest Flight Route - Camino Mas Largo en DAG (DFS con Memo)	168
14	Formulario		18.13	Message Route - BFS de Camino Minimo con Reconstruccion	169
14.1	Algebra y Ecuaciones	147	18.14	Monsters - BFS Multifuente + BFS de Escape	169
14.2	Areas y Volumenes	148	18.15	Planetqueriesi - Binary Lifting sobre Grafo Funcional	171
14.3	Checklist de Contest	148	18.16	Round Trip - Deteccion de Ciclo con BFS	171
14.4	Combinatoria	149	18.17	Round Trip (Stack) - Deteccion de Ciclo con Pila (DFS Iterativo)	172
14.5	Cotas y Limites	149	18.18	Round Trip II - Ciclo en Grafo Dirigido (DFS de Tres Colores)	173
14.6	Datos que no se Deducen	150	18.19	Shortest Routes I - Dijkstra con Priority Queue	174
14.7	Geometria	150	18.20	Shortest Routes II - Dijkstra desde Cada Nodo	174
14.8	Grafos, Esperanza y Calendario	151	18.21	Shortest Routes II (Floyd Warshall) - Floyd-Warshall	175
14.9	Probabilidad Geometrica y Areas Raras	151			

Contents (cont.)

19 CSES: Introductory Problems

19.1 Apple Divisions - Fuerza Bruta 2^n	176
19.2 Bit Strings - Exponenciacion Modular Rapida	176
19.3 Chessboard And Queens - Backtracking con Poda por Diagonales	176
19.4 Coin Piles - Invariantes Aritmeticos	177
19.5 Creating Strings - Backtracking sobre Frecuencias	177
19.6 Digit Queries - Busqueda por Bloques de Longitud	177
19.7 Gray Code - Identidad i XOR ($i >> 1$)	177
19.8 Grid Coloring I - Greedy Local con 4 Colores	178
19.9 Grid Path Description - Backtracking con Poda de Zona Encerrada	178
19.10 Increasing Array - Greedy de Maximo Prefijo	178
19.11 Knight Moves Grid - BFS en Grilla	179
19.12 Mex Grid Construction - Identidad MEX = XOR	179
19.13 Missing Number - Suma de Gauss	179
19.14 Number Spiral - Formula por Capas	179
19.15 Palindrome Reorder - Conteo de Frecuencias	179
19.16 Permutations - Construccion Directa	180
19.17 Rab Game I - Construccion por Rotacion de Bloque	180
19.18 Repetitions - Barrido Lineal de Rachas	180
19.19 String Reorder - Greedy Lexicografico con Cota de Factibilidad	181
19.20 Tower Of Hanoi - Recursion Clasica	181
19.21 Trailing Zeros - Formula de Legendre	181
19.22 Two Knights - Conteo por Complemento	181
19.23 Two Sets - Construccion por Bloques de 4	181
19.24 Weird Algorithm - Simulacion Directa	182

20 CSES: Mathematics

20.1 Counting Coprimes Pairs - Mobius (Inversion para Contar Coprimos)	182
--	-----

21 CSES: Range Queries

21.1 Distinct Value Queries II - Segment Tree sobre Ocurrencia Anterior	183
21.2 Distinct Values Queries - Mo's Algorithm	184
21.3 Dynamic Range Minimum Queries - Segment Tree (Minimo)	185
21.4 Dynamic Range Sum Queries - Fenwick Tree (BIT)	185
21.5 Forest Queries - Prefix Sums 2D	186
21.6 Forest Queries II - Fenwick 2D	186
21.7 Hotel Queries - Segment Tree con Descenso	187
21.8 Increasing Array Queries - Monotonic Stack + Binary Lifting con Costos	187
21.9 List Removals - Fenwick con Descenso Binario (k-esimo)	188
21.10 Missing Coin Sum Queries - Cubetas por Potencia de 2 + Segment Tree	188
21.11 Movie Festival Queries - Greedy + Binary Lifting sobre el Tiempo	189
21.12 Pizzeria Queries - Truco de p[i] mas y menos i	189
21.13 Polynomial Queries - Lazy Propagation de Progresiones Aritmeticas	190
21.14 Prefix Sum Queries - Segment Tree con Nodo Compuesto	191

21.15 Range Interval Queries - Barrido Offline con Fenwick	192
21.16 Range Queries And Copies - Segment Tree Persistente	193
21.17 Range Update Queries - Fenwick de Diferencias	193
21.18 Range Updates And Sums - Lazy Propagation con Suma y Asignacion	194
21.19 Range XOR Queries - Prefix XOR	195
21.20 Salary Queries - Fenwick con Compresion Offline	195
21.21 Static Range Minimum Queries - Sparse Table	196
21.22 Static Range Sum Queries - Prefix Sums	196
21.23 Sub Array Sum Queries - Segment Tree de Maximo Subarreglo	197
21.24 Sub Array Sum Queries II - Maximo Subarreglo en un Rango	197
21.25 Visible Building Queries - Monotonic Stack + Binary Lifting	198

22 CSES: Sorting and Searching

22.1 Apartments - Dos Punteros sobre Dos Listas Ordenadas	199
22.2 Array Division - Binaria sobre la Respuesta	199
22.3 Collecting Numbers - Contar Descensos en las Posiciones	199
22.4 Collecting Numbers II - Recalcular Solo lo que Cambia	200
22.5 Ferris Wheel - Dos Punteros desde los Extremos	200
22.6 Movie Festival - Interval Scheduling (Ordenar por FIN)	201
22.7 Nested Ranges Check - Dos Barridos sobre Rangos Ordenados	201
22.8 Nested Ranges Count - Contar Rangos Anidados con Fenwick	201
22.9 Sub Array Sums II - Prefijos con Map (Valores Negativos)	202

23 CSES: String Algorithms

23.1 All Palindromes - Manacher (Palindromo Mas Largo que Termina en Cada Posicion)	203
23.2 Counting Patterns - Aho-Corasick (Conteo de Ocurrencias)	203
23.3 Distinct Subsequences - DP con Resta de la Aparicion Anterior	205
23.4 Distinct Substrings - Suffix Automaton (Contar Distintas)	205
23.5 Finding Borders - Funcion de Prefijos (Cadena de Bordes)	206
23.6 Finding Patterns - Aho-Corasick (Aparece o No)	206
23.7 Finding Periods - Periodos = n - Bordes	207
23.8 Inverse Suffix Array - Reconstruir la Cadena desde el Suffix Array	208
23.9 Longest Palindrome - Manacher	209
23.10 Minimal Rotation - Rotacion Minima (Dos Punteros)	209
23.11 Palindrome Queries - Hashing Doble con Segment Tree	209
23.12 Pattern Positions - Aho-Corasick (Primera Aparicion)	210
23.13 Repeating Substring - Suffix Automaton (Ocurrencias por Estado)	212
23.14 Required Substring - DP sobre el Automata de KMP	213
23.15 String Functions - Funcion Z y Funcion de Prefijos	214
23.16 String Matching - KMP (Busqueda de Patron)	214
23.17 String Transformation - Transformada de Burrows-Wheeler Inversa	215
23.18 Substring Distribution - Suffix Automaton + Arreglo de Diferencias	215
23.19 Substring Order I - Descenso por el Suffix Automaton (Distintas)	216
23.20 Substring Order II - Descenso por el Suffix Automaton (con Repetidas)	217

Contents (cont.)

23.21 Word Combinations - Trie + DP de Segmentacion (Conteo)	219	25.38 Minimum Element After Replacement With Digit Sum - Suma de Digitos	228
24 CSES: Tree Algorithms		25.39 Missing Number - Numero Faltante	228
24.1 Company Queries II - LCA con Binary Lifting	219	25.40 Pascal's Triangle - Triangulo de Pascal	229
24.2 Path Queries II - Heavy-Light Decomposition	220	25.41 Plus One - Sumar Uno a un Numero por Digitos	229
25 Hub LeetCode: Arrays		25.42 Rank Transform Of An Array - Compresion de Coordenadas	229
25.1 3sum Closest - Ordenar y Dos Punteros	222	25.43 Reduction Operations To Make The Array Elements Equal - Contar por Escalones	229
25.2 Beautiful Array - Construcccion Divide y Conquista	222	25.44 Remove Covered Intervals - Intervalos Cubiertos	229
25.3 Buy Two Chocolates - Los Dos mas Baratos	222	25.45 Remove Duplicates From Sorted Array - Quitar Repetidos en Ordenado	229
25.4 Check If Array Is Sorted And Rotated - Ordenado y Rotado	222	25.46 Remove Element - Filtrar en el Sitio	229
25.5 Cinema Seat Allocation - Grupos de Sillas por Fila	223	25.47 Search Insert Position - Búsqueda Lineal	230
25.6 Concatenation Of Array - Concatenacion	223	25.48 Set Mismatch - Repetido y Faltante	230
25.7 Construct Uniform Parity Array I - Paridad por Fuerza Bruta	223	25.49 Shuffle The Array - Intercalar Dos Mitades	230
25.8 Construct Uniform Parity Array II - Paridad con el Menor Impar	223	25.50 Single Number - El que Aparece Una Vez	230
25.9 Contains Duplicate - Buscar Repetidos	223	25.51 Smallest Index With Digit Sum Equal To Index - Suma de Digitos	230
25.10 Count Subarrays With Majority Element I - Contar por Bloques Iguales	223	25.52 Smallest Missing Integer Greater Than Sequential Prefix Sum - Prefijo Secuencial	230
25.11 Daily Temperatures - Siguiente Dia mas Caliente	224	25.53 Smallest Missing Multiple Of K - Buscar el Primer Multiplo Ausente	230
25.12 Destroying Asteroids - Greedy Creciente	224	25.54 Smallest Stable Index I - Maximo por Prefijo y Minimo por Sufijo	230
25.13 Digit Frequency Score - Frecuencia de Digitos	224	25.55 Smallest Stable Index II - Maximo por Prefijo y Minimo por Sufijo	231
25.14 Earliest Finish Time For Land And Water Rides I - Probar Todos los Pares	224	25.56 Summary Ranges - Agrupar Rangos Consecutivos	231
25.15 Earliest Finish Time For Land And Water Rides II - Fijar el Mejor Primero	225	25.57 Two Sum - Fuerza Bruta $O(n^2)$	231
25.16 Final Prices With A Special Discount In A Shop - Siguiente Menor o Igual	225	25.58 Valid Mountain Array - Verificar Sube y Baja	231
25.17 Find All Numbers Disappeared In An Array - Faltantes de 1 a n	225	26 Hub LeetCode: Binary Search	
25.18 Find Missing Elements - Faltantes entre el Minimo y el Maximo	225	26.1 Binary Search - Búsqueda Binaria	231
25.19 Find Right Interval - Buscar el Intervalo de la Derecha	225	26.2 Count Elements With At Least K Greater Values - Ordenar y Buscar	232
25.20 Find The Highest Altitude - Suma Acumulada	225	26.3 Find Minimum In Rotated Sorted Array - Búsqueda en Arreglo Rotado	232
25.21 Find The Prefix Common Array Of Two Arrays - Interseccion de Prefijos	226	26.4 Peak Index In A Mountain Array - Binaria sobre Monotonia	232
25.22 First Missing Positive - Primer Positivo Faltante	226	26.5 Search In Rotated Sorted Array - Búsqueda en Arreglo Rotado	232
25.23 Hoy Many Numbers Are Smaller Than The Current Number - Fuerza Bruta $O(n^2)$	226	27 Hub LeetCode: Binary Tree	
25.24 Keep Multiplying Found Values By Two - Duplicar mientras Exista	226	27.1 Construct Binary Tree From Inorder And Postorder Traversal - Divide y Conquista sob...	233
25.25 Kth Largest Element In An Array - Counting Sort Parcial	226	27.2 Counts Nodes Equal To Average Of Subtree - DFS que Sube Informacion	233
25.26 Last Stone Weight - Simulacion con los Dos Mayores	226	27.3 Maximum Depth Of Binary Tree - DFS en Arbol Binario	233
25.27 Left And Right Sum Differences - Sumas a Izquierda y Derecha	226	28 Hub LeetCode: Bit Manipulation	
25.28 Magical String - Secuencia Autodescriptiva	227	28.1 Minimum Number Of Flips To Reverse Binary String - Comparar un Binario con su Re...	233
25.29 Majority Element - Elemento Mayoritario	227	29 Hub LeetCode: Combinatorics	
25.30 Max Consecutive Ones - Racha Maxima	227	29.1 Count The Number Of Computer Unlocking Permutations - Factorial con un Filtro	234
25.31 Maximum Ice Cream Bars - Greedy del mas Barato Primero	227	29.2 Distribute Candies Among Children I - Fuerza Bruta con Poda	234
25.32 Maximum Product Of Two Elements In An Array - Los Dos Mayores	227	29.3 Number Of Sets Of K Non Overlapping Line Segments - Stars and Bars	234
25.33 Maximum Total Subarray Value I - Observacion que Colapsa el Problema	227	29.4 Unique 3 Digit Event Numbers - Conteo con Frecuencias	234
25.34 Median Of Two Sorted Arrays - Avanzar hasta la Mitad	227	30 Hub LeetCode: DP	
25.35 Merge Intervals - Fusionar Intervalos	228	30.1 Climbing Stairs - Memoizacion	234
25.36 Minimum Absolute Difference - Diferencia Minima en Ordenado	228	30.2 Distinct Subsequences I - DP en Dos Cadenas	234
25.37 Minimum Cost Of Buying Candies With Discount - Greedy de Tercias	228		

Contents (cont.)

30.3 Distinct Subsequences II - DP con Resta de Repetidos	234	35.1 Construct Target Array With Multiple Sums - Deshacer el Proceso al Reves	245
30.4 Find X Value Of Array I - DP sobre Residuos	235	35.2 Find K Pairs With Smallest Sums - Heap Maximo de Tamano k	246
30.5 Maximum Score Of Non Overlapping Intervals - DP de Intervalos con Binaria	235	35.3 Maximum Number Of Eaten Apples - Heap por Fecha de Vencimiento	246
31 Hub LeetCode: Game Theory		36 Hub LeetCode: Linked List	
31.1 Divisor Game - Paridad	235	36.1 Add Two Numbers - Suma con Acarreo en Lista Ligada	246
31.2 Find The Winning Player In Coin Game - Simulacion de Turnos	235	36.2 Copy List With Random Pointer - Atajo de Libreria	247
31.3 Nim Game - Posiciones Perdedoras	235	36.3 Insertion Sort List - Insertion Sort en Lista Ligada	247
32 Hub LeetCode: Geometry		36.4 Linked List Cycle - Liebre y Tortuga	247
32.1 Angle Between Hands Of A Clock - Angulos de un Reloj	235	36.5 Merge Two Sorted Lists - Fusion de Listas Ligadas	247
32.2 Circle And Rectangle Overlapping - Punto mas Cercano en un Rectangulo	235	36.6 Odd Even Linked List - Separar en Dos Listas	248
32.3 Minimum Cuts To Divide A Circle - Casos por Paridad	236	36.7 Palindrome Linked List - Palindromo en Lista Ligada	248
32.4 Rectangle Overlap - Interseccion de Rectangulos	236	36.8 Remove Duplicates From Sorted List - Recorrido de Lista Ligada	248
33 Hub LeetCode: Graphs		36.9 Reverse Linked List - Invertir Lista Ligada	248
33.1 All Paths From Source To Target - Backtracking de Caminos	236	37 Hub LeetCode: Math	
33.2 Cheapest Flights Within K Stops - DP con Estado (nodo, saltos)	236	37.1 Counts Commas In Range I - Conteo por Magnitud	249
33.3 Check If There Is A Valid Path In A Grid - Compatibilidad de Tubos en Matriz	236	37.2 Counts Commas In Range II - Conteo por Bandas de Magnitud	249
33.4 Clone Graph - Clonar Grafo con Diccionario	237	37.3 Palindrome Number - Palindromo con Dos Punteros	249
33.5 Course Schedule - Orden Topologico de Kahn	237	37.4 Roman To Integer - Numeros Romanos	249
33.6 Course Schedule II - Orden Topologico de Kahn	238	37.5 Smallest Divisible Digit Product I - Fuerza Bruta	249
33.7 Evaluate Division - Grafo con Pesos Multiplicativos	238	37.6 Sqrt(x) - Raiz Entera	249
33.8 Find A Safe Walk Through A Grid - Busqueda con Vida Restante	238	37.7 Sum Of Square Numbers - Dos Punteros sobre Cuadrados	250
33.9 Find Eventual Safe States - Nodos que No Llegan a un Ciclo	239	37.8 Total Waviness Of Numbers In Range I - Fuerza Bruta sobre Digitos	250
33.10 Holding Hands - DSU sobre Parejas	239	38 Hub LeetCode: Matrix	
33.11 Is Graph Bipartite - Coloreo a Dos Colores	239	38.1 Image Overlap - Conteo de Desplazamientos	250
33.12 Jump Game III - DFS con Visitados	239	38.2 Island Perimeter - Recorrido de Matriz	250
33.13 Longest Increasing Path In A Matrix - DFS con Memoizacion en Matriz	240	39 Hub LeetCode: Prefix Sum	
33.14 Minimum Height Trees - Pelar Hojas (Centro del Arbol)	240	39.1 Continuous Subarray Sum - Residuos de Prefijos	250
33.15 Minimum Moves To Clean The Classroom - Dijkstra sobre (nodo, bitmask) con Poda... ..	240	39.2 Make Sum Divisible By P - Residuos de Prefijos	250
33.16 Network Delay Time - Dijkstra	242	39.3 Minimum Operations To Reduce X To Zero - Prefijos y Sufijos	251
33.17 Network Recovery Pathways - Binaria sobre la Respuesta en un DAG	243	39.4 Ways To Make A Fair Array - Prefijos Pares e Impares por Separado	251
33.18 Number Of Provinces - Componentes Conexas	243	40 Hub LeetCode: Queue	
33.19 Reconstruct Itinerary - Camino Euleriano (Hierholzer)	243	40.1 Design Circular Queue - Cola Circular con Arreglo Fijo	251
33.20 Redundant Connection - DSU (Union-Find)	244	40.2 Implement Queue Using Stacks - Cola	252
33.21 Redundant Connection II - Ciclo en Grafo Dirigido por Casos	244	40.3 Number Of Students Unable To Eat Lunch - Simulacion de Cola	252
33.22 Remove Methods From Project - Alcanzabilidad y Contaminacion	244	40.4 Time Needed To Buy Tickets - Simulacion de Cola	252
33.23 Sum Of Distances In Tree - Rerooting en Arbol	245	41 Hub LeetCode: Segment Tree	
34 Hub LeetCode: Greedy		41.1 Find X Value Of Array II - Segment Tree con Nodo Compuesto	252
34.1 Maximum Number Of Coins You Can Get - Greedy sobre Ordenado	245	42 Hub LeetCode: Sliding Window	
34.2 Maximum Number Of Non Overlapping Palindrome Substrings - Greedy de Izquierda... ..	245	42.1 Find The Largest Almost Missing Integer - Contar en Cuantas Ventanas Aparece	253
34.3 Stone Game - Greedy que Acierta por Casualidad	245	42.2 Find Two Non Overlapping Subarrays Each With Target Sum - Dos Ventanas sin Trasl...	253
35 Hub LeetCode: Heap			

Contents (cont.)

42.3	Length Of Longest Subarray With At Most K Frequency - Ventana con Frecuencias	253	45.26	Reverse Substrings Between Each Pair Of Parentheses - Pila de Parentesis con Rev...	263
42.4	Longest Substring Without Repeating Characters - Ventana Deslizante con Set	254	45.27	Rotate String - Rotacion de Cadena	263
42.5	Maximum Length Substring With Two Occurrences - Ventana con Frecuencias	254	45.28	Smallest Palindromic Rearrangement I - Palindromo Lexicograficamente Menor	263
42.6	Shortest And Lexicographically Smallest Beautiful String - Ventana con Conteo Exacto	254	45.29	Valid Anagram - Anagrama por Orden	263
43 Hub LeetCode: Sorting			45.30	Valid Palindrome - Palindromo Filtrando Caracteres	263
43.1	Merge Sorted Array - Merge Sort	254	46 Hub LeetCode: Trie		
43.2	Reverse Pairs - Contar en la Fusión del Merge Sort	255	46.1	Counts Prefix And Suffix Pairs I - Trie hacia Adelante y al Reves	264
43.3	Sort An Array - Counting Sort	255	46.2	Implement Trie (prefix Tree) - Trie	264
44 Hub LeetCode: Stack					
44.1	Build An Array With Stack Operations - Simulación de Pila	255			
44.2	Evaluate Reverse Polish Notation - Evaluación en Notación Polaca	256			
44.3	Exclusive Time Of Functions - Pila de Llamadas	256			
44.4	Largest Rectangle In Histogram - Pila Monotona (versión enredada)	256			
44.5	Next Greater Node In Linked List - Siguiente Mayor con Pila	257			
44.6	Remove Duplicate Letters - Pila Monotona Lexicográfica	257			
44.7	Valid Parentheses - Pila de Parentesis	257			
45 Hub LeetCode: String					
45.1	Brace Expansion II - Expansión Recursiva de Llaves	258			
45.2	Concatenate Non Zero Digits And Multiply By Sum I - Manejo de Dígitos	258			
45.3	Counts The Number Of Special Characters I - Sets de Letras	258			
45.4	Counts The Number Of Special Characters II - Orden de Primera y Última Aparición	258			
45.5	Detect Capital - Casos de Mayúsculas	258			
45.6	Evaluate The Bracket Pairs Of A String - Sustitución de Plantilla	258			
45.7	Find The Index Of The First Occurrence In A String - Búsqueda de Subcadena	259			
45.8	Find The Length Of The Longest Common Prefix - Prefijos en un Set	259			
45.9	Length Of Last Word - Split	259			
45.10	License Key Formatting - Agrupar de Atrás hacia Adelante	259			
45.11	Longest Common Prefix - Prefijo Común por Columnas	260			
45.12	Masking Personal Information - Enmascarar por Formato	260			
45.13	Maximum Number Of Ballons - Frecuencias con Letras Repetidas	260			
45.14	Maximum Number Of Non Overlapping Substrings - Intervalos Cerrados por Letra	260			
45.15	Maximum Repeating Substring - Repeticiones Consecutivas	261			
45.16	Maximum Substrings With Distinct Start - Observación que Colapsa el Problema	261			
45.17	Minimum Number Of Pushes To Type Word I - Asignación por Bloques de 8	261			
45.18	Minimum Number Of Pushes To Type Word II - Greedy por Frecuencia	261			
45.19	Number Of Strings That Appear As Substrings In Word - Contar Subcadenas	261			
45.20	Number Of Substrings Containing All Three Characters - Contar Subcadenas con las...	262			
45.21	Process String With Special Operations I - Simulación de Operaciones	262			
45.22	Reformat Date - Formateo de Fecha	262			
45.23	Repeated String Match - Repetir hasta Contener	262			
45.24	Repeated Substring Pattern - Período de una Cadena	262			
45.25	Reverse Degree Of A String - Suma Ponderada	262			

1 - Data Structures

1.1 Brute Force - Count Majority Subarrays

O: (n^2) , todos los subarreglos

Cuenta subarreglos donde "target" es mayoría (aparece en mas de la mitad de las posiciones)

C++/LeetCode/countMajoritySubarrays.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:
7     int countMajoritySubarrays(vector<int>& nums, int target) {
8         int god = 0;
9         for(int i = 0 ; i < nums.size(); i++)
10             {
11                 vector<int> temp;
12                 int tempi = 0;
13                 for(int j = i ; j < nums.size(); j++)
14                     {
15                         temp.push_back(nums[j]);
16                         if(nums[j]==target)
17                             {
18                                 tempi++;
19                             }
20                         if(tempi*2 > temp.size())
21                             {
22                                 god++;
23                             }
24                     }
25             }
26         }
27
28         return god;
29     }
30 };
```

1.2 Compresion de Coordenadas

O: $(n \log n)$ construir, $(\log n)$ consultar

Uso: cp.construir(vals); i = cp.comp(x); x = cp.orig(i) // 0-indexado

Cuando los VALORES llegan a 1e9 pero solo hay $n \leq 2e5$ distintos

C++/Notebook/Compresion de Coordenadas.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // ----- la version corta, la que se escribe en contest -----
5 struct Compresor {
6     vector<long long> v;
7
8     // Se le pasan TODOS los valores que van a aparecer, incluidos los de las
9     // actualizaciones futuras.
10    void construir(vector<long long> vals) {
11        v = vals;
12        sort(v.begin(), v.end());
13        v.erase(unique(v.begin(), v.end()), v.end());
14    }
15
16    int n() const { return v.size(); }
17
18    // valor -> indice comprimido [0, n). El valor TIENE que existir.
19    int comp(long long x) const {
20        return lower_bound(v.begin(), v.end(), x) - v.begin();
21    }
22
23    // indice -> valor original
24    long long orig(int i) const { return v[i]; }
25
26    // Cuantos valores hay < x. Sirve cuando x puede NO estar en la tabla.
27    int menores(long long x) const {
```

1.2 Compresion de Coordenadas (cont.)

```
28     return lower_bound(v.begin(), v.end(), x) - v.begin();
29 }
30
31 // Cuantos son <= x.
32 int menoresIguales(long long x) const {
33     return upper_bound(v.begin(), v.end(), x) - v.begin();
34 }
35 };
36
37 int main() {
38     ios::sync_with_stdio(false);
39     cin.tie(nullptr);
40
41     int n;
42     if (!(cin >> n)) return 0;
43
44     vector<long long> a(n);
45     for (long long &x : a) cin >> x;
46
47     Compresor c;
48     c.construir(a);
49
50     cout << "valores distintos: " << c.n() << '\n';
51
52     for (long long x : a) {
53         cout << x << " -> " << c.comp(x) << '\n';
54     }
55
56     // Uso tipico: el arreglo ya comprimido, listo para cualquier estructura
57     vector<int> comprimido(n);
58     for (int i = 0; i < n; i++) comprimido[i] = c.comp(a[i]);
59
60     cout << "comprimido:";
61     for (int x : comprimido) cout << ' ' << x;
62     cout << '\n';
63     return 0;
64 }
```

1.3 Conteo de Inversiones con Fenwick - Frosh

O: $(n \log n)$, inversiones con Fenwick

Hay n estudianten en fila y solo se pueden intercambiar PARES CONSECUTIVOS

C++/CCPL/frosh.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Fenwick {
5     int n;
6     vector<long long> bit;
7
8     Fenwick(int n) : n(n), bit(n + 1, 0) {}
9
10    void add(int idx, long long val) {
11        for (; idx <= n; idx += idx & -idx)
12            bit[idx] += val;
13    }
14
15    long long sum(int idx) {
16        long long res = 0;
17
18        for (; idx > 0; idx -= idx & -idx)
19            res += bit[idx];
20
21        return res;
22    }
23 };
24
25 int main() {
26     ios::sync_with_stdio(false);
27     cin.tie(nullptr);
28 }
```

1.3 Conteo de Inversiones con Fenwick - Fr sh (cont.)

```

29     int n;
30
31     while (cin >> n) {
32
33         vector<int> a(n);
34
35         for (int i = 0; i < n; i++) {
36             cin >> a[i];
37         }
38
39         vector<int> sorted = a;
40         sort(sorted.begin(), sorted.end());
41
42         Fenwick fw(n);
43
44         long long god = 0;
45
46         for (int i = 0; i < n; i++) {
47
48             int pos = lower_bound(
49                 sorted.begin(),
50                 sorted.end(),
51                 a[i]
52             ) - sorted.begin() + 1;
53
54             god += i - fw.sum(pos);
55
56             fw.add(pos, 1);
57         }
58
59         cout << god << '\n';
60     }
61
62     return 0;
63 }

```

1.4 Conteo de Inversiones y Redondeo Exacto - Sky Line

O: $(n \log n)$, inversiones con Fenwick sobre coordenadas comprimidas

Cuenta las inversiones de la secuencia de alturas (pares $i < j$ con $h[i] > h[j]$) y responde la fracci n que...

C++/ICPC/2024/K - SkyLine.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct Fenwick {
5      int n;
6      vector<long long> bit;
7
8      Fenwick(int n) : n(n), bit(n + 1, 0) {}
9
10     void add(int idx, long long val) {
11         for (; idx <= n; idx += idx & -idx)
12             bit[idx] += val;
13     }
14
15     long long sum(int idx) {
16         long long res = 0;
17
18         for (; idx > 0; idx -= idx & -idx)
19             res += bit[idx];
20
21         return res;
22     }
23 };
24
25 int main() {
26     ios::sync_with_stdio(false);
27     cin.tie(nullptr);
28
29     int N;
30

```

1.4 Conteo de Inversiones y Redondeo Exacto - Sky Line (cont.)

```

31     while (cin >> N && N != 0) {
32
33         vector<int> h(N);
34
35         for (int &x : h)
36             cin >> x;
37
38         if (N == 1) {
39             cout << "0.000\n";
40             continue;
41         }
42
43         // Compresion de coordenadas: no se confia en el rango del enunciado,
44         // porque el propio sample trae una altura de -29.
45         vector<int> v = h;
46         sort(v.begin(), v.end());
47         v.erase(unique(v.begin(), v.end()), v.end());
48
49         Fenwick fw(v.size());
50
51         long long inversions = 0;
52
53         for (int i = 0; i < N; i++) {
54
55             int x = lower_bound(v.begin(), v.end(), h[i]) - v.begin() + 1;
56
57             // Elementos anteriores que son <= h[i]
58             long long leq = fw.sum(x);
59
60             // De los i anteriores, los dem s son > h[i]
61             inversions += i - leq;
62
63             fw.add(x, 1);
64         }
65
66         long long totalPairs = 1LL * N * (N - 1) / 2;
67
68         // Redondeo a 3 decimales con enteros: empates hacia arriba, como pide
69         // el enunciado. Con double + printf un empate exacto (0.0625) puede
70         // redondearse hacia el par y dar 0.062 en vez de 0.063.
71         long long mil = (2000 * inversions + totalPairs) / (2 * totalPairs);
72
73         string frac = to_string(mil % 1000);
74         while (frac.size() < 3)
75             frac = "0" + frac;
76
77         cout << mil / 1000 << "." << frac << '\n';
78     }
79
80     return 0;
81 }

```

1.5 DSU, Fenwick y Sparse Table

O: DSU casi (1) ; Fenwick $(\log n)$; Sparse Table $(n \log n)$ armar, (1) consultar

Uso: TODO  -indexado: DSU $d(n)$; $d.unir(a,b)$; Fenwick $f(n)$; $f.add(i,v)$; $f.sum(l,r)$

Las tres estructuras de todos los d as, genericas y listas para pegar

C++/Notebook/DSU, Fenwick y Sparse Table.cpp

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  typedef long long ll;
6
7  struct DSU
8  {
9      vector<int> p, sz;
10     DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
11     int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
12     bool unir(int a, int b)
13     {

```

1.5 DSU, Fenwick y Sparse Table (cont.)

```

44     a = find(a);
45     b = find(b);
46     if (a == b) return false;
47     if (sz[a] < sz[b]) swap(a, b);
48     p[b] = a;
49     sz[a] += sz[b];
50     return true;
51 }
52 };
53
54 struct Fenwick
55 {
56     int n;
57     vector<ll> t;
58     Fenwick(int n) : n(n), t(n + 1, 0) {}
59     void add(int i, ll v)
60     {
61         for (i++; i <= n; i += i & -i) t[i] += v;
62     }
63     ll sum(int i) // a[0..i]
64     {
65         ll s = 0;
66         for (i++; i > 0; i -= i & -i) s += t[i];
67         return s;
68     }
69     ll sum(int l, int r) { return l > r ? 0 : sum(r) - (l ? sum(l - 1) : 0); }
70     int lowerBound(ll k) // menor i con a[0..i] >= k; n si no hay
71     {
72         int pos = 0;
73         for (int pw = 1 << 25; pw; pw >= 1) // cubre n hasta 6*10^7
74         {
75             if (pos + pw <= n && t[pos + pw] < k)
76             {
77                 pos += pw;
78                 k -= t[pos];
79             }
80         }
81         return pos;
82     }
83 };
84
85 ll contarInversiones(const vector<ll> &a)
86 {
87     vector<ll> v = a;
88     sort(v.begin(), v.end());
89     v.erase(unique(v.begin(), v.end()), v.end());
90     Fenwick f(v.size());
91     ll inv = 0;
92     for (int i = (int)a.size() - 1; i >= 0; i--)
93     {
94         int c = lower_bound(v.begin(), v.end(), a[i]) - v.begin();
95         inv += f.sum(c - 1); // menores a la derecha
96         f.add(c, 1);
97     }
98     return inv;
99 }
100
101 struct SparseMin
102 {
103     vector<vector<ll>> t;
104     SparseMin(const vector<ll> &a)
105     {
106         int n = a.size(), K = 1;
107         while ((1 << K) <= n) K++;
108         t.assign(K, a);
109         for (int k = 1; k < K; k++)
110         {
111             for (int i = 0; i + (1 << k) <= n; i++)
112             {
113                 t[k][i] = min(t[k - 1][i], t[k - 1][i + (1 << (k - 1))]);
114             }
115         }
116     }
117 }

```

1.5 DSU, Fenwick y Sparse Table (cont.)

```

86     }
87     ll query(int l, int r) // minimo de a[l..r]
88     {
89         int k = __lg(r - l + 1);
90         return min(t[k][l], t[k][r - (1 << k) + 1]);
91     }
92 };
93
94 int main()
95 {
96     ios_base::sync_with_stdio(false);
97     cin.tie(nullptr);
98     return 0;
99 }

```

1.6 Fenwick Tree (Consultas offline) - D Query

O((n + q) log n), consultas offline con Fenwick

Dada una secuencia a_{1..a_n} y q consultas (i, j)

C++/Training_Camp_2026/Day 5/H - D - query.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int bit_tree[30005];
5 int n;
6
7 void update(int pos, int val){
8     for(; pos <= n; pos += pos & (-pos))
9         bit_tree[pos] += val;
10 }
11
12 int query(int pos){
13     int res = 0;
14     for(; pos > 0; pos -= pos & (-pos))
15         res += bit_tree[pos];
16     return res;
17 }
18
19 int main(){
20     ios_base::sync_with_stdio(false);
21     cin.tie(nullptr);
22
23     cin >> n;
24     vector<int> a(n + 1);
25     for(int i = 1; i <= n; i++)
26         cin >> a[i];
27
28     int q;
29     cin >> q;
30
31     vector<int> qi(q), qj(q);
32     vector<int> order(q);
33     for(int k = 0; k < q; k++){
34         cin >> qi[k] >> qj[k];
35         order[k] = k;
36     }
37
38     // sort query indices by right endpoint j
39     sort(order.begin(), order.end(), [&](int x, int y){
40         return qj[x] < qj[y];
41     });
42
43     vector<int> answer(q);
44     unordered_map<int, int> lastPos; // value -> last position it was seen
45     lastPos.reserve(1 << 20);
46
47     int curJ = 0; // how far we've processed (1-indexed)
48     for(int idx : order){
49         int j = qj[idx];
50         // advance pointer up to j, updating BIT
51         while(curJ < j){

```

1.6 Fenwick Tree (Consultas offline) - D Query (cont.)

```

52         curJ++;
53         int val = a[curJ];
54         auto it = lastPos.find(val);
55         if(it != lastPos.end()){
56             update(it->second, -1); // remove old occurrence's contribution
57         }
58         update(curJ, 1);
59         lastPos[val] = curJ;
60     }
61     answer[idx] = query(j) - query(qi[idx] - 1);
62 }
63
64 for(int k = 0; k < q; k++)
65     cout << answer[k] << "\n";
66
67 return 0;
68 }

```

1.7 Fenwick Tree (Offline Queries) - K Query

O: $((n + q) \log n)$, consultas offline con Fenwick

Dado un arreglo de N numeros y varias consultas (i, j, k)

C++/Training_Camp_2026/Day 4/I - K Query.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct Element {
5      int value;
6      int pos;
7  };
8
9  struct Query {
10     int l, r, k;
11     int id;
12 };
13
14 bool cmpElement(const Element &a, const Element &b) {
15     return a.value > b.value;
16 }
17
18 bool cmpQuery(const Query &a, const Query &b) {
19     return a.k > b.k;
20 }
21
22 class Fenwick {
23     vector<int> bit;
24     int n;
25
26 public:
27     Fenwick(int n) {
28         this->n = n;
29         bit.assign(n + 1, 0);
30     }
31
32     void update(int idx, int val) {
33         while (idx <= n) {
34             bit[idx] += val;
35             idx += idx & -idx;
36         }
37     }
38
39     int query(int idx) {
40         int sum = 0;
41         while (idx > 0) {
42             sum += bit[idx];
43             idx -= idx & -idx;
44         }
45         return sum;
46     }
47
48     int rangeQuery(int l, int r) {

```

1.7 Fenwick Tree (Offline Queries) - K Query (cont.)

```

48         return query(r) - query(l - 1);
49     }
50 };
51 };
52
53 int main() {
54     ios::sync_with_stdio(false);
55     cin.tie(nullptr);
56
57     int n;
58     cin >> n;
59
60     vector<Element> arr(n);
61
62     for (int i = 0; i < n; i++) {
63         cin >> arr[i].value;
64         arr[i].pos = i + 1;
65     }
66
67     sort(arr.begin(), arr.end(), cmpElement);
68
69     int q;
70     cin >> q;
71
72     vector<Query> queries(q);
73
74     for (int i = 0; i < q; i++) {
75         cin >> queries[i].l >> queries[i].r >> queries[i].k;
76         queries[i].id = i;
77     }
78
79     sort(queries.begin(), queries.end(), cmpQuery);
80
81     Fenwick bit(n);
82     vector<int> ans(q);
83
84     int idx = 0;
85
86     for (auto &qu : queries) {
87         while (idx < n && arr[idx].value > qu.k) {
88             bit.update(arr[idx].pos, 1);
89             idx++;
90         }
91         ans[qu.id] = bit.rangeQuery(qu.l, qu.r);
92     }
93
94     for (int x : ans)
95         cout << x << "\n";
96
97     return 0;
98 }

```

1.8 Fenwick Tree (Prefix Minimum, Dominancia 3D) - The Day Of The Competitors

O: $(n \log n)$, Fenwick de minimos para dominancia 3D

Cada uno de N competidores tiene un ranking en 3 competencias distintas

C++/Training_Camp_2026/Day 4/J - The day of the competitors.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  static char buf_[1 << 25];
5  static int bufLen_, bufPos_;
6  inline int gc() {
7      if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0;
8          if (!bufLen_) return -1; }
9      return buf_[bufPos_++];
10 }
11 inline int readInt() {
12     int c = gc();
13     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
14     bool neg = false; if (c == '-') { neg = true; c = gc(); }
15     int x = 0;

```

1.8 Fenwick Tree (Prefix Minimum, Dominancia 3D) - The Day Of The Competitors (cont.)

```

5   while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
6   return neg ? -x : x;
7 }
8
9 int N;
10 vector<int> bitArr;
11
12 inline void update(int pos, int val) {
13     for (; pos <= N; pos += pos & (-pos)) {
14         if (val < bitArr[pos]) bitArr[pos] = val;
15     }
16 }
17
18 inline int queryPrefixMin(int pos) {
19     int res = INT_MAX;
20     for (; pos > 0; pos -= pos & (-pos)) {
21         if (bitArr[pos] < res) res = bitArr[pos];
22     }
23     return res;
24 }
25
26 int main() {
27     int t = readInt();
28     string out;
29     while (t--) {
30         N = readInt();
31         vector<array<int,3>> people(N);
32         for (int i = 0; i < N; i++) {
33             int a = readInt(), b = readInt(), c = readInt();
34             people[i] = {a, b, c};
35         }
36         sort(people.begin(), people.end(), [](const array<int,3>& x, const array<int,3>& y) {
37             return x[0] < y[0];
38         });
39         bitArr.assign(N + 1, INT_MAX);
40         int count = 0;
41         for (int i = 0; i < N; i++) {
42             int b = people[i][1];
43             int c = people[i][2];
44             int mn = queryPrefixMin(b - 1);
45             if (mn >= c) count++;
46             update(b, c);
47         }
48         out += to_string(count);
49         out += "\n";
50     }
51     fwrite(out.data(), 1, out.size(), stdout);
52     return 0;
53 }

```

1.9 Hash Map y Ordenamiento

O: (n log n) por el sort del volcado

Uso: mp cuenta; luego vector<pair<string,int>> v(mp.begin(), mp.end())

Diferencia clave: unordered_map es tabla hash, O(1) promedio pero SIN orden

C++/Notebook/Hash Map y Ordenamiento.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main()
6 {
7     ios_base::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    // contar frecuencias
11    unordered_map<string, int> mp = {"c", 3}, {"a", 1}, {"b", 2};
12
13    // para ordenar hay que volcar a un vector de pares
14    vector<pair<string,int>> v(mp.begin(), mp.end());

```

1.9 Hash Map y Ordenamiento (cont.)

```

5
6 // por clave ascendente
7 sort(v.begin(), v.end(), [](auto &l, auto &r){ return l.first < r.first; });
8
9 // por valor descendente
10 sort(v.begin(), v.end(), [](auto &l, auto &r){ return l.second > r.second; });
11
12 // por valor descendente con desempate por clave ascendente
13 sort(v.begin(), v.end(), [](auto &l, auto &r){
14     if (l.second == r.second) return l.first < r.first;
15     return l.second > r.second;
16 });
17
18 for (auto &p : v)
19 {
20     cout << p.first << " => " << p.second << "\n";
21 }
22
23 // modificar todos los valores in-place (el & es obligatorio, si no se copia)
24 // con C++17 tambien vale: for (auto &[key, val] : mp) val += 3;
25 for (auto &p : mp)
26 {
27     p.second += 3;
28 }
29
30 // buscar sin crear la clave por accidente (operator[] la crearia)
31 if (mp.count("b"))
32 {
33     cout << "b vale " << mp["b"] << "\n";
34 }
35
36 // invertir el mapa (valor -> clave) para ordenar por valor
37 vector<pair<int,string>> inv;
38 for (auto &p : mp) inv.push_back({p.second, p.first});
39 sort(inv.begin(), inv.end());
40
41 // map ya viene ordenado por clave, no necesita sort
42 map<int, string> m;
43 m[3] = "tres";
44 m[1] = "uno";
45 m[2] = "dos";
46 for (auto &p : m)
47 {
48     cout << p.first << " -> " << p.second << "\n";
49 }
50
51 return 0;
52 }

```

1.10 Hash Set - Dangerous Dive

O: (n), un set

De N buzos voluntarios que fueron a una mision, se conocen los R que regresaron

C++/Training_Camp_2026/Day 2/E - Dangerous Dive.py

```

1 n, m = list(map(int, input().split()))
2 k = set(list(map(int, input().split())))
3 if(n == m):
4     print("***")
5     fin = ""
6     for i in range(1,n+1):
7         if(i not in k):
8             fin+=(str(i)+" ")
9     print(fin)

```

1.11 Heap con Borrado Perezoso - Aquarium Bubble Lights

O: (n log n), heap con borrado perezoso

Hay n enteros no negativos y q operaciones

C++/Codeforces/aquariumBubbleLights.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;

```

1.11 Heap con Borrado Perezoso - Aquarium Bubble Lights (cont.)

```

3
4 using ll = long long;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int n, q;
11    cin >> n >> q;
12
13    vector<ll> expire(n + 1);
14
15    // (expire, index)
16    priority_queue<pair<ll, int>,
17                vector<pair<ll, int>>,
18                greater<pair<ll, int>>> pq;
19
20    for (int i = 1; i <= n; ++i) {
21        ll a;
22        cin >> a;
23
24        expire[i] = a;
25
26        if (a > 0)
27            pq.push({expire[i], i});
28    }
29
30    int positive = 0;
31
32    // Initially, all a[i] > 0 are positive.
33    for (int i = 1; i <= n; ++i)
34        if (expire[i] > 0)
35            positive++;
36
37    for (int t = 1; t <= q; ++t) {
38        int x;
39        ll v;
40        cin >> x >> v;
41
42        // After the mandatory decrement at operation t,
43        // an element is zero iff expire[i] <= t.
44        while (!pq.empty() && pq.top().first <= t) {
45            auto [e, id] = pq.top();
46            pq.pop();
47
48            // Old heap entry.
49            if (expire[id] != e)
50                continue;
51
52            expire[id] = 0;
53            positive--;
54        }
55
56        if (x != 0) {
57            // Current value after the decrement.
58            ll cur = max(0LL, expire[x] - t);
59
60            ll nw = max(cur, v);
61
62            if (cur == 0 && nw > 0)
63                positive++;
64
65            if (nw > 0) {
66                expire[x] = t + nw;
67                pq.push({expire[x], x});
68            } else {
69                expire[x] = 0;
70            }
71        }
72
73        cout << positive << '\n';
74    }
75

```

1.11 Heap con Borrado Perezoso - Aquarium Bubble Lights (cont.)

```

76    // Final values are expire[i] - q.
77    for (int i = 1; i <= n; ++i) {
78        if (i > 1)
79            cout << ' ';
80
81        cout << max(0LL, expire[i] - q);
82    }
83
84    cout << '\n';
85
86    return 0;
87 }

```

1.12 Mo's Algorithm - Wine Production

O: $((n + q) \text{ raíz de } n)$, Mo

Dadas las temperaturas de N días y Q consultas de rango [l,r]

C++/Training_Camp_2026/Day 7/E - Wine Production.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int N, Q;
5 vector<int> A; // valores comprimidos, 1-indexado
6 vector<int> freqArr; // frecuencia actual de cada valor comprimido
7 vector<int> B; // B[k] = cantidad de valores distintos con frecuencia >= k
8 int h;
9
10 inline void addElem(int val) {
11     int f = freqArr[val];
12     B[f + 1] += 1;
13     freqArr[val] = f + 1;
14     // solo puede crecer como maximo 1 (demostrado matematicamente),
15     // pero dejamos el while por seguridad/robustez.
16     while (h + 1 < (int)B.size() && B[h + 1] >= h + 1) h++;
17 }
18
19 inline void removeElem(int val) {
20     int f = freqArr[val];
21     B[f] -= 1;
22     freqArr[val] = f - 1;
23     while (h > 0 && B[h] < h) h--;
24 }
25
26 int main() {
27     scanf("%d %d", &N, &Q);
28     vector<int> rawA(N + 1);
29     vector<int> sortedVals(N);
30     for (int i = 1; i <= N; i++) {
31         scanf("%d", &rawA[i]);
32         sortedVals[i - 1] = rawA[i];
33     }
34     sort(sortedVals.begin(), sortedVals.end());
35     sortedVals.erase(unique(sortedVals.begin(), sortedVals.end()), sortedVals.end());
36     int M = (int)sortedVals.size();
37
38     A.assign(N + 1, 0);
39     for (int i = 1; i <= N; i++) {
40         int idx = (int)(lower_bound(sortedVals.begin(), sortedVals.end(), rawA[i]) -
41                                 sortedVals.begin());
42         A[i] = idx + 1;
43     }
44
45     freqArr.assign(M + 2, 0);
46     B.assign(N + 2, 0);
47     h = 0;
48
49     vector<int> ql(Q), qr(Q), qIdx(Q);
50     for (int i = 0; i < Q; i++) {
51         scanf("%d %d", &ql[i], &qr[i]);
52         qIdx[i] = i;
53     }
54

```

1.12 Mo's Algorithm - Wine Production (cont.)

```

52     }
53
54     int blockSize = max(1, (int)sqrt((double)N));
55     sort(qIdx.begin(), qIdx.end(), [&](int a, int b) {
56         int ba = ql[a] / blockSize, bb = ql[b] / blockSize;
57         if (ba != bb) return ba < bb;
58         if (ba & 1) return qr[a] > qr[b];
59         return qr[a] < qr[b];
60     });
61
62     vector<int> ans(Q);
63     int curL = 1, curR = 0;
64     for (int qi : qIdx) {
65         int l = ql[qi], r = qr[qi];
66         while (curR < r) { curR++; addElem(A[curR]); }
67         while (curL > l) { curL--; addElem(A[curL]); }
68         while (curR > r) { removeElem(A[curR]); curR--; }
69         while (curL < l) { removeElem(A[curL]); curL++; }
70         ans[qi] = h;
71     }
72
73     string out;
74     out.reserve(Q * 4);
75     for (int i = 0; i < Q; i++) {
76         out += to_string(ans[i]);
77         out += '\n';
78     }
79     fputs(out.c_str(), stdout);
80     return 0;
81 }

```

1.13 Ordered Set (PBDS)

O: (log n) insertar, borrar, find_by_order y order_of_key

Uso: s.find_by_order(k) -> k-esimo (0-indexado); s.order_of_key(x) -> menores set indexado de GNU: find_by_order(k) y order_of_key(x), las dos en O(log n)

C++/Notebook/Ordered Set (PBDS).cpp

```

1  #include <bits/stdc++.h>
2  #include <ext/pb_ds/assoc_container.hpp>
3  #include <ext/pb_ds/tree_policy.hpp>
4
5  using namespace std;
6  using namespace __gnu_pbds;
7
8  typedef tree<int, null_type, less<int>, rb_tree_tag,
9             tree_order_statistics_node_update> ordered_set;
10
11 int main()
12 {
13     ios_base::sync_with_stdio(false);
14     cin.tie(nullptr);
15
16     ordered_set s;
17     s.insert(10);
18     s.insert(20);
19     s.insert(30);
20     s.insert(40);
21
22     cout << *s.find_by_order(0) << "\n"; // 10, el primero
23     cout << *s.find_by_order(2) << "\n"; // 30, el tercero
24
25     cout << s.order_of_key(30) << "\n"; // 2, hay dos menores que 30
26     cout << s.order_of_key(35) << "\n"; // 3, no hace falta que 35 exista
27
28     if (s.find_by_order(99) == s.end())
29     {
30         cout << "fuera de rango\n";
31     }
32
33     return 0;
34 }

```

1.14 Persistencia (DFS + Rollback) - The Declaration Of Independence

O: (total de operaciones), DFS sobre el árbol de versiones con rollback

Versiones de un documento que se ramifican: DFS sobre el árbol de versiones con rollback

C++/Training_Camp_2026/Day 5/D - The Declaration of Independence.py

```

1  import sys
2  from collections import deque
3
4
5  sys.setrecursionlimit(3*10**6)
6  n = int(input())
7  queries = []
8
9  dicc = {0: []}
10 deletes = []
11 for i in range(1, n+1):
12     q = input().split(" ")
13     kind = q[0]
14
15     if kind == 'E':
16         father = int(q[1])
17         num = int(q[2])
18         if father not in dicc:
19             dicc[father] = [(i, num)]
20         else:
21             dicc[father].append((i, num))
22     else:
23         father = int(q[1])
24         deletes.append(i)
25         if father not in dicc:
26             dicc[father] = [(i, kind)]
27         else:
28             dicc[father].append((i, kind))
29     arr = deque([])
30     res = [0] * (n + 1)
31     def dfs(currNode):
32         global arr
33         quitado = 0
34         if currNode[1] == "D":
35             if len(arr) > 0:
36                 quitado = arr.popleft()
37         else:
38             if currNode[1] != None:
39                 arr.append(currNode[1])
40             res[currNode[0]] = quitado
41
42         if currNode[0] in dicc:
43             for son in dicc[currNode[0]]:
44                 dfs(son)
45
46         if currNode[1] == "D":
47             arr.appendleft(quitado)
48         else:
49             if len(arr) > 0:
50                 arr.pop()
51
52     dfs((0, None))
53
54     for i in deletes:
55         print(res[i], end=" ")

```

1.15 Prefix Sums - Count Majority Subarrays2

O: (n log n), prefijos mas conteo de inversiones

Misma idea que countMajoritySubarrays.cpp pero en O(n)

C++/LeetCode/countMajoritySubarrays2.cpp

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  class Solution {
6  public:

```

1.15 Prefix Sums - Count Majority Subarrays2 (cont.)

```

1   long long countMajoritySubarrays(vector<int>& nums, int target) {
2       int n = nums.size();
3
4       vector<int> pre(2 * n + 1, 0);
5
6       pre[n] = 1;
7
8       int cnt = n;
9       long long presum = 0;
10      long long ans = 0;
11
12      for (int x : nums) {
13          if (x == target) {
14              presum += pre[cnt];
15
16              ++cnt;
17              ++pre[cnt];
18          } else {
19              --cnt;
20
21              presum -= pre[cnt];
22              ++pre[cnt];
23          }
24
25          ans += presum;
26      }
27
28      return ans;
29  };

```

1.16 Prefix Sums con Primera Aparicion - Accounting Qualifying Con Prefix

O: (n), prefijos con la primera aparicion de cada valor

De una lista de transacciones hay que dar el mayor deposito d, el mayor retiro w (el mas negativo) y r

C++/IICPC/2025/A - Accounting Qualifying con prefix.cpp

```

1   #include <bits/stdc++.h>
2   using namespace std;
3
4   int main() {
5       ios::sync_with_stdio(false);
6       cin.tie(nullptr);
7
8       int n;
9
10      while (cin >> n && n != 0) {
11
12          vector<int> a(n);
13
14          for (int &x : a)
15              cin >> x;
16
17          // Si no hay depositos d es 0, y si no hay retiros w es 0: por eso se
18          // acotan con 0 en vez de tomar el maximo y el minimo a secas.
19          int d = max(0, *max_element(a.begin(), a.end()));
20          int w = min(0, *min_element(a.begin(), a.end()));
21
22          // Convertimos:
23          // deposito -> +1
24          // balance -> 0
25          // retiro -> -1
26          for (int &x : a) {
27              if (x > 0) x = 1;
28              else if (x < 0) x = -1;
29          }
30
31          // first[s] = primera posicion donde aparecio
32          // el prefix sum s
33          unordered_map<int, int> first;
34
35          int sum = 0;

```

1.16 Prefix Sums con Primera Aparicion - Accounting Qualifying Con Prefix (cont.)

```

36      int r = 0;
37
38      // prefix sum 0 aparece antes de empezar
39      first[0] = -1;
40
41      for (int i = 0; i < n; i++) {
42
43          sum += a[i];
44
45          if (first.count(sum)) {
46              // Mismo prefix sum => entre ambas posiciones
47              // la suma es 0
48              r = max(r, i - first[sum]);
49          } else {
50              // Guardamos solamente la primera aparicion,
51              // porque nos da el segmento mas largo
52              first[sum] = i;
53          }
54      }
55
56      cout << d << ' ' << w << ' ' << r << '\n';
57  }
58
59      return 0;
60  }

```

1.17 Segment Tree - Block Placement Queries

O: (q log n), segment tree de maximos

Usa un segment tree de maximos y un set de obstaculos para determinar si un bloque de tamaño sz cabe en...

C++/LeetCode/blockPlacementQueries.cpp

```

1   #include <bits/stdc++.h>
2   using namespace std;
3
4   typedef int T; // tipo de dato del segtree
5   struct segtree {
6       vector<T> st; //, lazy;
7       int n;
8       T neutro = 0; // neutro para MAX (los huecos son >= 0)
9
10      segtree(const vector<T> &v) {
11          n = v.size();
12          st.assign(n*4, 0);
13          //lazy.assign(n*4, neutro);
14          build(1, 0, n-1, v);
15      }
16
17      void build(int p, int L, int R, const vector<T> &v) {
18          if (L == R) st[p] = v[L];
19          else {
20              int m=(L+R)/2, l=p*2, r=l+1;
21              build(l,L,m,v); build(r,m+1,R,v);
22              st[p] = max(st[l], st[r]); // <- max
23          }
24      }
25
26      /*
27      void propagate(int p, int L, int R, T val) {
28          if (val == neutro) return;
29          st[p] = val;
30          lazy[p] = neutro;
31          if (L == R) return;
32          int l = p*2, r = l+1;
33          lazy[l] = lazy[r] = val;
34      }
35      */
36
37      T query(int i, int j) { return query(1, 0, n-1, i, j); }
38      void upd(int i, int j, T val) { upd(1, 0, n-1, i, j, val); }
39
40      T query(int p, int L, int R, int i, int j) {

```

1.17 Segment Tree - Block Placement Queries (cont.)

```

41 //propagate(p, L, R, lazy[p]);
42 if (i > R || j < L) return neutro;
43 if (i <= L && R <= j) return st[p];
44 int m=(L+R)/2, l=p*2, r=l+1;
45 T lf = query(l, L, m, i, j);
46 T rg = query(r, m+1, R, i, j);
47 return max(lf, rg); // <- max
48 }
49
50 void upd(int p, int L, int R, int i, int j, T val) {
51 //propagate(p, L, R, lazy[p]);
52 if (i > R || j < L) return;
53 if (i <= L && R <= j) st[p] = val; // cambiar por propagate(p,L,R,val);
54 else {
55     int m=(L+R)/2, l=p*2, r=l+1;
56     upd(l, L, m, i, j, val);
57     upd(r, m+1, R, i, j, val);
58     st[p] = max(st[l], st[r]); // <- max
59 }
60 }
61 };
62
63 class Solution {
64 public:
65
66
67
68     vector<bool> getResults(vector<vector<int>>& queries) {
69         int MAX = 0;
70         for (auto &q : queries) MAX = max(MAX, q[1]);
71
72         vector<T> base(MAX + 1, 0); //  rbol de huecos, todo en 0
73         segtree seg(base);
74
75         set<int> obs = {0}; // 0 es la pared izquierda
76         vector<bool> res;
77
78         for (auto &q : queries) {
79             if (q[0] == 1) { // colocar obst culo en x
80                 int x = q[1];
81                 auto it = obs.lower_bound(x); // primer elemento > x
82                 int prev = *std::prev(it);
83                 seg.upd(x, x, x - prev); // hueco que termina en x
84                 if (it != obs.end()) {
85                     int next = *it;
86                     seg.upd(next, next, next - x); // el hueco viejo se acorta
87                 }
88                 obs.insert(x);
89             } else { // consulta [2, x, sz]
90                 int x = q[1], sz = q[2];
91                 auto it = obs.upper_bound(x); // primer elemento > x
92                 int prev = *std::prev(it); // mayor obst culo <= x
93                 bool ok = (x - prev >= sz) || (seg.query(0, x) >= sz);
94                 res.push_back(ok);
95             }
96         }
97         return res;
98
99     }
100 };
101 };

```

1.18 Segment Tree - Monarchy In Vertigo

O: (log N) por operaci n, segment tree

Se mantiene un  rbol genealogico donde nacen hijos (agregados al final de la lista de hijos de su padre) y...

C++/Training_Camp_2026/Day 4/H - Monarchy in Vertigo.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static int N;

```

1.18 Segment Tree - Monarchy In Vertigo (cont.)

```

5 static vector<int> seg;
6
7 void update(int node, int lo, int hi, int pos, int val) {
8     if (lo == hi) { seg[node] = val; return; }
9     int mid = (lo + hi) >> 1;
10    if (pos <= mid) update(node<<1, lo, mid, pos, val);
11    else update(node<<1|1, mid+1, hi, pos, val);
12    seg[node] = seg[node<<1] | seg[node<<1|1];
13 }
14
15 int queryFirstAlive(int node, int lo, int hi) {
16     if (seg[node] == 0) return -1;
17     if (lo == hi) return lo;
18     int mid = (lo + hi) >> 1;
19     int r = queryFirstAlive(node<<1, lo, mid);
20     if (r != -1) return r;
21     return queryFirstAlive(node<<1|1, mid+1, hi);
22 }
23
24 int main(){
25     int Q;
26     scanf("%d", &Q);
27     vector<int> type(Q), arg(Q);
28     for (int i = 0; i < Q; i++) scanf("%d %d", &type[i], &arg[i]);
29
30     int births = 0;
31     for (int i = 0; i < Q; i++) if (type[i] == 1) births++;
32     N = births + 1; // total number of people that will ever exist
33
34     vector<vector<int>> children(N + 1);
35     vector<int> childId(Q, -1);
36     int nextId = 2;
37     for (int i = 0; i < Q; i++) {
38         if (type[i] == 1) {
39             int cid = nextId++;
40             children[arg[i]].push_back(cid);
41             childId[i] = cid;
42         }
43     }
44
45     // Pre-order numbering of the FINAL tree (birth order never changes relative order)
46     vector<int> rankOf(N + 1), idAtRank(N + 1);
47     vector<int> st; st.reserve(N + 1);
48     st.push_back(1);
49     int cur = 0;
50     while (!st.empty()) {
51         int u = st.back(); st.pop_back();
52         rankOf[u] = ++cur;
53         idAtRank[cur] = u;
54         for (int i = (int)children[u].size() - 1; i >= 0; i--) st.push_back(children[u][i]);
55     }
56
57     seg.assign(4 * (N + 1), 0);
58     update(1, 1, N, rankOf[1], 1); // Constant starts alive
59
60     string out; out.reserve(Q * 7);
61     for (int i = 0; i < Q; i++) {
62         if (type[i] == 1) {
63             update(1, 1, N, rankOf[childId[i]], 1);
64         } else {
65             update(1, 1, N, rankOf[arg[i]], 0);
66             int pos = queryFirstAlive(1, 1, N);
67             out += to_string(idAtRank[pos]);
68             out += '\n';
69         }
70     }
71     fputs(out.c_str(), stdout);
72     return 0;
73 }

```

1.19 Segment Tree - Running A Penitentiary

O: (log n) por operacion, segment tree

N guardias vigilan cada uno un intervalo de celdas [L,R], y hay que soportar Q operaciones

C++/Training_Camp_2026/Day 7/D - Running a penitentiary.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 int n;
6 vector<ll> treeL, treeR;          // treeL: max de L en el rango, treeR: min de R e
7 n el rango
8 const ll NEG_INF = -2000000000000000000LL;
9 const ll POS_INF = 2000000000000000000LL;
10
11 inline void pull(int i) {
12     treeL[i] = max(treeL[2*i], treeL[2*i+1]);
13     treeR[i] = min(treeR[2*i], treeR[2*i+1]);
14 }
15
16 // actualiza el guarda en posicion 0-indexada 'pos' con nuevo intervalo [lVal, rVal]
17 void updatePos(int pos, ll lVal, ll rVal) {
18     pos += n;
19     treeL[pos] = lVal;
20     treeR[pos] = rVal;
21     for (pos >= 1; pos >= 1; pos >= 1) pull(pos);
22 }
23
24 // consulta semiabierto [l, r) 0-indexada -> devuelve (max L, min R) en ese rango
25 pair<ll, ll> query(int l, int r) {
26     ll resL = NEG_INF, resR = POS_INF;
27     for (l += n, r += n; l < r; l >= 1, r >= 1) {
28         if (l & 1) {
29             resL = max(resL, treeL[l]);
30             resR = min(resR, treeR[l]);
31             ++l;
32         }
33         if (r & 1) {
34             --r;
35             resL = max(resL, treeL[r]);
36             resR = min(resR, treeR[r]);
37         }
38     }
39     return make_pair(resL, resR);
40 }
41
42 int main() {
43     ios_base::sync_with_stdio(false);
44     cin.tie(nullptr);
45
46     int N, Q;
47     cin >> N >> Q;
48     n = N;
49     treeL.assign(2 * n, 0);
50     treeR.assign(2 * n, 0);
51
52     for (int i = 0; i < n; i++) {
53         ll l, r;
54         cin >> l >> r;
55         treeL[n + i] = l;
56         treeR[n + i] = r;
57     }
58     for (int i = n - 1; i >= 1; i--) pull(i);
59
60     string output;
61     output.reserve((size_t)Q * 8);
62
63     for (int qq = 0; qq < Q; qq++) {
64         char op;
65         cin >> op;
66         if (op == 'C') {
67             int i; ll l, r;
```

1.19 Segment Tree - Running A Penitentiary (cont.)

```
67         cin >> i >> l >> r;
68         updatePos(i - 1, l, r);
69     } else { // '?'
70         int a, b;
71         cin >> a >> b;
72         pair<ll, ll> res = query(a - 1, b);
73         ll ans = res.second - res.first + 1;
74         if (ans < 0) ans = 0;
75         output += to_string(ans);
76         output += '\n';
77     }
78 }
79
80 cout << output;
81 return 0;
82 }
```

1.20 Segment Tree con Lazy (Sumar a un Rango)

O: (n) armar, (log n) cada operacion

Uso: SegTree st(a); st.add(l,r,v); st.suma(l,r); st.minimo(l,r); todo 0-indexado, [l,r] Sumar v a todo a[l..r] y preguntar suma o minimo de a[l..r], ambos en O(log n)

C++/Notebook/Segment Tree con Lazy.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 typedef long long ll;
6
7 struct SegTree
8 {
9     int n;
10    vector<ll> sm, mn, lz;
11
12    SegTree(const vector<ll> &a) : n(a.size()), sm(4 * n), mn(4 * n), lz(4 * n, 0)
13    {
14        build(1, 0, n - 1, a);
15    }
16
17    void pull(int p)
18    {
19        sm[p] = sm[2 * p] + sm[2 * p + 1];
20        mn[p] = min(mn[2 * p], mn[2 * p + 1]);
21    }
22
23    void aplicar(int p, int l, int r, ll v)
24    {
25        sm[p] += v * (r - l + 1);
26        mn[p] += v;
27        lz[p] += v;
28    }
29
30    void push(int p, int l, int r)
31    {
32        if (lz[p] == 0) return;
33        int m = (l + r) / 2;
34        aplicar(2 * p, l, m, lz[p]);
35        aplicar(2 * p + 1, m + 1, r, lz[p]);
36        lz[p] = 0;
37    }
38
39    void build(int p, int l, int r, const vector<ll> &a)
40    {
41        if (l == r)
42        {
43            sm[p] = mn[p] = a[l];
44            return;
45        }
46        int m = (l + r) / 2;
47        build(2 * p, l, m, a);
48        build(2 * p + 1, m + 1, r, a);
```

1.20 Segment Tree con Lazy (Sumar a un Rango) (cont.)

```

48     pull(p);
49 }
50
51 void add(int p, int l, int r, int i, int j, ll v)
52 {
53     if (j < l || r < i) return;
54     if (i <= l && r <= j)
55     {
56         aplicar(p, l, r, v);
57         return;
58     }
59     push(p, l, r);
60     int m = (l + r) / 2;
61     add(2 * p, l, m, i, j, v);
62     add(2 * p + 1, m + 1, r, i, j, v);
63     pull(p);
64 }
65
66 // devuelve {suma, minimo} de a[i..j]
67 pair<ll, ll> query(int p, int l, int r, int i, int j)
68 {
69     if (j < l || r < i) return {0, LLONG_MAX};
70     if (i <= l && r <= j) return {sm[p], mn[p]};
71     push(p, l, r);
72     int m = (l + r) / 2;
73     auto a = query(2 * p, l, m, i, j), b = query(2 * p + 1, m + 1, r, i, j);
74     return {a.first + b.first, min(a.second, b.second)};
75 }
76
77 void add(int l, int r, ll v) { add(1, 0, n - 1, l, r, v); }
78 ll suma(int l, int r) { return query(1, 0, n - 1, l, r).first; }
79 ll minimo(int l, int r) { return query(1, 0, n - 1, l, r).second; }
80
81 };
82
83 int main()
84 {
85     ios_base::sync_with_stdio(false);
86     cin.tie(nullptr);
87     return 0;
88 }

```

1.21 Segment Tree sobre Coordenadas Comprimidas - Emergency Rations

O: (q log q), segment tree sobre las coordenadas comprimidas

Consultas con posiciones enormes: comprimir coordenadas y montar el segment tree encima

C++/ICPC/Regionals 2025/E - Emergency Rations.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5
6 #define forn(i,n) for(int i = 0; i < n; i++)
7
8 struct SegmentTree
9 {
10     int n;
11     vector<ll> tree, lazy;
12
13     SegmentTree(vector<ll>& a)
14     {
15         n = a.size();
16         tree.resize(4 * n);
17         lazy.resize(4 * n, 0);
18
19         build(1, 0, n - 1, a);
20     }
21
22     void build(int node, int l, int r, vector<ll>& a)
23     {
24         if (l == r)
25         {

```

1.21 Segment Tree sobre Coordenadas Comprimidas - Emergency Rations (cont.)

```

26         tree[node] = a[l];
27         return;
28     }
29
30     int mid = (l + r) / 2;
31
32     build(node * 2, l, mid, a);
33     build(node * 2 + 1, mid + 1, r, a);
34
35     tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
36 }
37
38 void push(int node)
39 {
40     if (lazy[node] == 0)
41         return;
42
43     ll x = lazy[node];
44
45     tree[node * 2] += x;
46     tree[node * 2 + 1] += x;
47
48     lazy[node * 2] += x;
49     lazy[node * 2 + 1] += x;
50
51     lazy[node] = 0;
52 }
53
54 void update(int node, int l, int r, int ql, int qr, ll val)
55 {
56     if (ql > r || qr < l)
57         return;
58
59     if (ql <= l && r <= qr)
60     {
61         tree[node] += val;
62         lazy[node] += val;
63         return;
64     }
65
66     push(node);
67
68     int mid = (l + r) / 2;
69
70     update(node * 2, l, mid, ql, qr, val);
71     update(node * 2 + 1, mid + 1, r, ql, qr, val);
72
73     tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
74 }
75
76 void update(int l, int r, ll val)
77 {
78     if (l > r)
79         return;
80
81     update(1, 0, n - 1, l, r, val);
82 }
83
84 ll query()
85 {
86     return tree[1];
87 }
88 };
89
90 int main()
91 {
92     ios::sync_with_stdio(false);
93     cin.tie(nullptr);
94
95     int Q;
96     cin >> Q;
97

```

1.21 Segment Tree sobre Coordenadas Comprimidas - Emergency Rations (cont.)

```

98 vector<ll> x(Q);
99 vector<ll> coords;
100
101 forn(i, Q)
102 {
103     cin >> x[i];
104
105     coords.push_back(abs(x[i]));
106 }
107
108 // Coordenadas de los posibles tamanos de las cajas
109 sort(coords.begin(), coords.end());
110 coords.erase(unique(coords.begin(), coords.end()), coords.end());
111
112 /*
113     Incluimos el tamaño 0.
114
115     t = 0 significa:
116     no hacer ninguna operacion de "quitar una racion
117     de cada caja", sino vaciar cada caja individualmente.
118 */
119 vector<ll> values;
120
121 // 0 siempre es una posibilidad
122 values.push_back(0);
123
124 for(ll v : coords)
125     values.push_back(v);
126
127 int M = values.size();
128
129 /*
130     freq[i] = cantidad de cajas cuyo tamaño es values[i]
131 */
132 vector<ll> freq(M, 0);
133
134 /*
135     Al principio no hay cajas.
136
137     Para cada posible t:
138     valor[t] = t + cantidad de cajas con tamaño > t
139
140     Como inicialmente no hay cajas:
141     valor[t] = t
142 */
143 vector<ll> initial(M);
144
145 for(int i = 0; i < M; i++)
146     initial[i] = values[i];
147
148 SegmentTree st(initial);
149
150 /*
151     Posición de cada valor en el vector comprimido.
152 */
153 unordered_map<ll, int> pos;
154
155 for(int i = 0; i < M; i++)
156     pos[values[i]] = i;
157
158 /*
159     Agregar una caja de tamaño x.
160
161     Para todo t < x:
162     cantidad de cajas > t aumenta en 1
163
164     Por lo tanto hacemos:
165     [0, pos[x]-1] += 1
166 */
167 auto addBox = [&](ll x)
168 {
169     int p = pos[x];

```

1.21 Segment Tree sobre Coordenadas Comprimidas - Emergency Rations (cont.)

```

170
171     st.update(0, p - 1, 1);
172
173     freq[p]++;
174 };
175
176 /*
177     Quitar una caja de tamaño x.
178
179     Para todo t < x:
180     cantidad de cajas > t disminuye en 1
181 */
182 auto removeBox = [&](ll x)
183 {
184     int p = pos[x];
185
186     st.update(0, p - 1, -1);
187
188     freq[p]--;
189 };
190
191 forn(i, Q)
192 {
193     if(x[i] > 0)
194     {
195         addBox(x[i]);
196     }
197     else
198     {
199         removeBox(-x[i]);
200     }
201
202     cout << st.query() << ' ';
203 }
204
205 cout << '\n';
206
207 return 0;
208 }

```

1.22 Small-to-Large Merging (DSU on Tree) - Distinct Colors

O: ($N \log^2 N$), small to large sobre el árbol

Dado un árbol con raíz en el nodo 1 donde cada nodo tiene un color

C++/Training_Camp_2026/Day 4/M - Distinct Colors.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     ios_base::sync_with_stdio(false);
6     cin.tie(NULL);
7
8     int n;
9     cin >> n;
10
11     vector<int> color(n + 1);
12     for (int i = 1; i <= n; i++) cin >> color[i];
13
14     vector<vector<int>> adj(n + 1);
15     for (int i = 0; i < n - 1; i++) {
16         int a, b;
17         cin >> a >> b;
18         adj[a].push_back(b);
19         adj[b].push_back(a);
20     }
21
22     vector<int> parent(n + 1, 0);
23     vector<int> order;
24     order.reserve(n);
25
26     // DFS iterativo (evita overflow de pila en arboles muy profundos, tipo cadena)

```

1.22 Small-to-Large Merging (DSU on Tree) - Distinct Colors (cont.)

```

27 vector<int> stck;
28 stck.reserve(n);
29 vector<char> visited(n + 1, 0);
30 stck.push_back(1);
31 visited[1] = 1;
32 while (!stck.empty()) {
33     int u = stck.back();
34     stck.pop_back();
35     order.push_back(u);
36     for (int v : adj[u]) {
37         if (!visited[v]) {
38             visited[v] = 1;
39             parent[v] = u;
40             stck.push_back(v);
41         }
42     }
43 }
44
45 // Invertimos el preorder: en este orden, cada hijo aparece antes que su padre,
46 // asi que al procesar u, todos sus hijos ya estan listos.
47 reverse(order.begin(), order.end());
48
49 vector<set<int>> colorSet(n + 1);
50 vector<int> ans(n + 1, 0);
51
52 for (int u = 1; u <= n; u++) colorSet[u].insert(color[u]);
53
54 // Small-to-large merging: siempre fusionamos el set mas chico dentro del mas grande
55 for (int u : order) {
56     for (int v : adj[u]) {
57         if (v != parent[u]) { // v es hijo de u
58             if (colorSet[u].size() < colorSet[v].size())
59                 colorSet[u].swap(colorSet[v]); // swap de sets es O(1), no copia eleme
60             colorSet[u].insert(colorSet[v].begin(), colorSet[v].end());
61             colorSet[v].clear(); // liberamos memoria que ya no se necesita
62         }
63     }
64     ans[u] = (int)colorSet[u].size();
65 }
66
67 for (int i = 1; i <= n; i++) {
68     cout << ans[i];
69     cout << (i < n ? ' ' : '\n');
70 }
71
72 return 0;
73 }

```

1.23 Sparse Table sobre Bloques Iguales - Frequent

O: (n log n) de tabla y (1) por consulta, sparse table sobre bloques

Un arreglo de n enteros NO DECRECIENTE y q consultas [i,j]

C++/CCPL/frequent.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n, q;
9
10    while (cin >> n && n != 0) {
11        cin >> q;
12
13        vector<int> a(n + 1);
14        for (int i = 1; i <= n; i++) cin >> a[i];
15
16        // Como el arreglo viene ordenado, los iguales quedan en BLOQUES
17        // contiguos: se guarda a que bloque pertenece cada posicion, y donde

```

1.23 Sparse Table sobre Bloques Iguales - Frequent (cont.)

```

8 // empieza y termina cada bloque.
9 vector<int> bloque(n + 1), ini(n + 2), fin(n + 2), tam(n + 2);
10
11 int b = 0;
12
13 for (int i = 1; i <= n; i++) {
14     if (i == 1 || a[i] != a[i - 1]) {
15         b++;
16         ini[b] = i;
17     }
18     bloque[i] = b;
19     fin[b] = i;
20 }
21
22 for (int t = 1; t <= b; t++) tam[t] = fin[t] - ini[t] + 1;
23
24 // Sparse table de MAXIMOS sobre los tamanos de bloque.
25 int K = 1;
26 while ((1 << K) <= b) K++;
27
28 vector<vector<int>> st(K, vector<int>(b + 1, 0));
29
30 for (int i = 1; i <= b; i++) st[0][i] = tam[i];
31
32 for (int k = 1; k < K; k++) {
33     int len = 1 << k;
34     for (int i = 1; i + len - 1 <= b; i++)
35         st[k][i] = max(st[k - 1][i], st[k - 1][i + len / 2]);
36 }
37
38 // Maximo de tam[l..r]; los dos bloques se solapan y no pasa nada,
39 // porque el maximo es idempotente.
40 auto maxRango = [&](int l, int r) {
41     if (l > r) return 0;
42     int k = 31 - __builtin_clz(r - l + 1);
43     return max(st[k][l], st[k][r - (1 << k) + 1]);
44 };
45
46 while (q--) {
47     int i, j;
48     cin >> i >> j;
49
50     int bi = bloque[i], bj = bloque[j];
51
52     // Todo el rango cae dentro de un mismo bloque.
53     if (bi == bj) {
54         cout << j - i + 1 << '\n';
55         continue;
56     }
57
58     int izq = fin[bi] - i + 1; // cola del bloque de i
59     int der = j - ini[bj] + 1; // cabeza del bloque de j
60     int medio = maxRango(bi + 1, bj - 1);
61
62     cout << max(medio, max(izq, der)) << '\n';
63 }
64
65 return 0;
66 }

```

1.24 Sqrt Decomposition - Holes

O: (raiz de N) por operacion, sqrt decomposition

Hay N agujeros en fila, cada uno con una potencia que hace saltar la bola a la posicion i+a_i

C++/Training_Camp_2026/Day 4/K - Holes.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 22];
5 static int bufLen_, bufPos_;
6 inline int gc() {

```

1.24 Sqrt Decomposition - Holes (cont.)

```

    if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0;
    if (!bufLen_) return -1; }
    return buf_[bufPos_++];
}

inline int readInt() {
    int c = gc();
    while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
    bool neg = false; if (c == '-') { neg = true; c = gc(); }
    int x = 0;
    while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
    return neg ? -x : x;
}

int N, M;
vector<int> a_; // power of each hole, 1-indexed
vector<int> jmp_; // first position outside the hole's block reached (or N+1 sentinel =
// exits the row)
vector<int> cnt_; // number of individual jumps taken to reach jmp_[i]
vector<int> last_; // valid only when jmp_[i] == N+1: last hole visited before leaving t
// he row
int blockSize;

void recomputeBlock(int p) {
    int blockIdx = (p - 1) / blockSize;
    int start = blockIdx * blockSize + 1;
    int end = min(N, start + blockSize - 1);
    for (int i = end; i >= start; i--) {
        int nxt = i + a_[i];
        if (nxt > N) {
            jmp_[i] = N + 1;
            cnt_[i] = 1;
            last_[i] = i;
        } else if (nxt > end) {
            // leaves the current block to a later block (nxt > i always since a_[i] >= 1)
            jmp_[i] = nxt;
            cnt_[i] = 1;
            // last_[i] intentionally left as-is; never read while jmp_[i] <= N
        } else {
            jmp_[i] = jmp_[nxt];
            cnt_[i] = cnt_[nxt] + 1;
            last_[i] = last_[nxt];
        }
    }
}

int main() {
    N = readInt();
    M = readInt();
    a_.assign(N + 2, 0);
    jmp_.assign(N + 2, 0);
    cnt_.assign(N + 2, 0);
    last_.assign(N + 2, 0);

    for (int i = 1; i <= N; i++) a_[i] = readInt();

    blockSize = max(1, (int)sqrt((double)N));

    for (int start = 1; start <= N; start += blockSize) {
        recomputeBlock(start);
    }

    string out;
    out.reserve(M * 12);
    char tmp[32];

    for (int q = 0; q < M; q++) {
        int type = readInt();
        if (type == 0) {
            int pos = readInt(), val = readInt();
            a_[pos] = val;
            recomputeBlock(pos);
        } else {

```

1.24 Sqrt Decomposition - Holes (cont.)

```

            int pos = readInt();
            long long jumps = 0;
            int cur = pos;
            int lastHole = -1;
            while (cur <= N) {
                jumps += cnt_[cur];
                if (jmp_[cur] > N) {
                    lastHole = last_[cur];
                }
                cur = jmp_[cur];
            }
            int len = sprintf(tmp, "%d %lld\n", lastHole, jumps);
            out.append(tmp, len);
        }
    }
    fwrite(out.data(), 1, out.size(), stdout);
    return 0;
}

```

1.25 Tree Construction - Binary Tree

O: (n), reconstruccion del arbol
 Dado un arreglo de tripletas [parent, child, isLeft]

C++/LeetCode/binaryTree.cpp

```

#include <bits/stdc++.h>

using namespace std;

class Solution {
public:
    struct TreeNode {
        int val;
        TreeNode *left;
        TreeNode *right;
        TreeNode() : val(0), left(nullptr), right(nullptr) {}
        TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
        TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
    };

    TreeNode* createBinaryTree(vector<vector<int>>& descriptions) {
        unordered_map<int, TreeNode*> nodes;
        unordered_set<int> children;

        for (auto &d : descriptions) {
            int parent = d[0];
            int child = d[1];
            int isLeft = d[2];

            if (!nodes.count(parent))
                nodes[parent] = new TreeNode(parent);

            if (!nodes.count(child))
                nodes[child] = new TreeNode(child);

            if (isLeft)
                nodes[parent]->left = nodes[child];
            else
                nodes[parent]->right = nodes[child];

            children.insert(child);
        }

        for (auto &d : descriptions) {
            int parent = d[0];
            if (!children.count(parent))
                return nodes[parent];
        }

        return nullptr;
    }
}

```

1.25 Tree Construction - Binary Tree (cont.)

};

2 - Dynamic Programming

2.1 Binary Search - Patches

O: (log N), binaria

Una llanta de circunferencia C tiene N agujeros a distancias F_i desde una marca de referencia

C++/Training_Camp_2026/Day 2/I - Patches.py

```

1 from bisect import bisect_right
2
3 memo = {}
4
5 def rec(voy):
6     if voy == n:
7         return 0
8
9     if voy in memo:
10        return memo[voy]
11
12    i1 = bisect_right(l, l[voy] + t1)
13    i2 = bisect_right(l, l[voy] + t2)
14
15    ans = min(
16        t1 + rec(i1),
17        t2 + rec(i2)
18    )
19
20    memo[voy] = ans
21    return ans
22
23
24 n, c, t1, t2 = map(int, input().split())
25 l = list(map(int, input().split()))
26
27 print(rec(0))

```

2.2 Bitmask DP - Music Festival

O: (2^n * n), bitmask DP

Hay N <= 10 escenarios, cada uno con varios shows con horario [inicio

C++/Training_Camp_2026/Day 8/F - Music Festival.py

```

1 from bisect import bisect_left
2
3 n = int(input())
4
5 shows = []
6
7 for stage in range(n):
8     data = list(map(int, input().split()))
9     m = data[0]
10    for i in range(m):
11        s = data[1 + 3 * i]
12        e = data[2 + 3 * i]
13        p = data[3 + 3 * i]
14        shows.append((s, e, stage, p))
15
16 shows.sort()
17
18 m = len(shows)
19
20 starts = [x[0] for x in shows]
21
22 next_show = [0] * m
23 for i in range(m):
24     next_show[i] = bisect_left(starts, shows[i][1])
25
26 FULL = (1 << n) - 1
27 NEG = -10**18
28

```

2.2 Bitmask DP - Music Festival (cont.)

```

29 dp = [[NEG] * (1 << n) for _ in range(m + 1)]
30 dp[m][FULL] = 0
31
32 for i in range(m - 1, -1, -1):
33     for mask in range(1 << n):
34         dp[i][mask] = dp[i + 1][mask]
35
36     _, _, stage, songs = shows[i]
37     bit = 1 << stage
38
39     for mask in range(1 << n):
40         if dp[next_show[i]][mask | bit] != NEG:
41             dp[i][mask] = max(
42                 dp[i][mask],
43                 songs + dp[next_show[i]][mask | bit]
44             )
45
46 ans = dp[0][0]
47
48 print(ans if ans != NEG else -1)

```

2.3 Bitmask SOS DP - El Juego Del Calamar

O: (2^k * k), SOS DP sobre mascarar

Una torre de n pisos con k <= 15 habitaciones por piso (capacidad C[i][j]) y escaleras que solo suben de...

C++/Training_Camp_2026/Day 9/D - El Juego del Calamar.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4 const ll INF = (ll)4e18;
5
6 int main(){
7     int n, k, m;
8     scanf("%d %d %d", &n, &k, &m);
9
10    // C[i][j]: piso i (1..n), habitacion j (0..k-1)
11    vector<vector<ll>> C(n+1, vector<ll>(k));
12    for(int i = 1; i <= n; i++){
13        for(int j = 0; j < k; j++){
14            scanf("%lld", &C[i][j]);
15
16            // adjOut[i][j]: mascara de habitaciones del piso i+1 alcanzables desde la habitacion
17            // j del piso i
18            vector<vector<int>> adjOut(n+1, vector<int>(k, 0));
19            for(int c = 0; c < m; c++){
20                int f, u, v;
21                scanf("%d %d %d", &f, &u, &v);
22                adjOut[f][u-1] |= (1 << (v-1));
23            }
24
25            int SZ = 1 << k;
26            int full = SZ - 1;
27
28            vector<ll> r(SZ), h(SZ), g(SZ);
29            vector<int> Nmask(SZ);
30
31            // r[mask] para el piso 1: suma de C[1][j] para j en mask (sin cuello de botella, fuen
32            // te directa)
33            r[0] = 0;
34            for(int mask = 1; mask < SZ; mask++){
35                int lb = mask & (-mask);
36                int j = __builtin_ctz(lb);
37                r[mask] = r[mask ^ lb] + C[1][j];
38            }
39
40            vector<ll> ans(n+1);
41            ans[1] = r[full];
42
43            for(int i = 1; i <= n-1; i++){
44                // h[B] = r[complemento de B] = flujo maximo usando SOLO las habitaciones de B
45                for(int B = 0; B < SZ; B++){
46                    h[B] = r[full ^ B];
47                }
48            }
49

```

2.3 Bitmask SOS DP - El Juego Del Calamar (cont.)

```

44
45 // Nmask[B] = union de vecinos (piso i+1) de las habitaciones en B
46 Nmask[0] = 0;
47 for(int B = 1; B < SZ; B++){
48     int lb = B & (-B);
49     int j = __builtin_ctz(lb);
50     Nmask[B] = Nmask[B ^ lb] | adjOut[i][j];
51 }
52
53 // g[N] = mejor (min) h[B] entre los B cuya vecindad es exactamente N
54 fill(g.begin(), g.end(), INF);
55 for(int B = 0; B < SZ; B++){
56     int N = Nmask[B];
57     if(h[B] < g[N]) g[N] = h[B];
58 }
59
60 // transformada tipo SOS: de g a r_{i+1}, procesando cada bit (habitacion del piso
61 i+1)
62 for(int mask = 0; mask < SZ; mask++){ r[mask] = g[mask];
63 for(int b = 0; b < k; b++){
64     int bit = 1 << b;
65     ll cap = C[i+1][b];
66     for(int mask = 0; mask < SZ; mask++){
67         if(mask & bit){
68             int m0 = mask ^ bit;
69             ll old0 = r[m0], old1 = r[mask];
70             r[m0] = min(old0, old1);
71             r[mask] = min(old0, old1 + cap);
72         }
73     }
74 }
75 ans[i+1] = r[full];
76 }
77
78 for(int t = 1; t <= n; t++){
79     printf("%lld%c", ans[t], t==n?'\\n':' ');
80 }
81
82 return 0;
83 }

```

2.4 Camino en Grilla con Balance de Parentesis - Has Valid Path

O: $(n*m*(n+m))$ por el estado (celda, balance)

En una grilla de '(' y ')' hay que decir si existe un camino de la esquina superior izquierda a la inferior...

C++/LeetCode/hasValidPath.cpp

```

1 class Solution {
2 public:
3
4     bool valid(string &s) {
5         int balance = 0;
6
7         for (char c : s) {
8             if (c == '(')
9                 balance++;
10            else
11                balance--;
12
13            if (balance < 0)
14                return false;
15        }
16
17        return balance == 0;
18    }
19
20    vector<vector<vector<int>>> memo;
21
22    bool dfs(vector<vector<char>>& grid,
23            int i, int j,
24            int balance,

```

2.4 Camino en Grilla con Balance de Parentesis - Has Valid Path (cont.)

```

25         string &s) {
26
27         int m = grid.size();
28         int n = grid[0].size();
29
30         // Nunca podemos tener mas ')' que '('
31         if (balance < 0)
32             return false;
33
34         // Ya calculamos este estado
35         if (memo[i][j][balance] != -1)
36             return memo[i][j][balance];
37
38         // Llegamos al final
39         if (i == m - 1 && j == n - 1) {
40             return memo[i][j][balance] = (balance == 0);
41         }
42
43         // Abajo
44         if (i + 1 < m) {
45
46             char c = grid[i + 1][j];
47
48             s.push_back(c);
49
50             int newBalance = balance + (c == '(' ? 1 : -1);
51
52             if (dfs(grid, i + 1, j, newBalance, s))
53                 return memo[i][j][balance] = true;
54
55             s.pop_back();
56         }
57
58         // Derecha
59         if (j + 1 < n) {
60
61             char c = grid[i][j + 1];
62
63             s.push_back(c);
64
65             int newBalance = balance + (c == '(' ? 1 : -1);
66
67             if (dfs(grid, i, j + 1, newBalance, s))
68                 return memo[i][j][balance] = true;
69
70             s.pop_back();
71         }
72
73         return memo[i][j][balance] = false;
74     }
75
76     bool hasValidPath(vector<vector<char>>& grid) {
77
78         int m = grid.size();
79         int n = grid[0].size();
80
81         // La longitud maxima del string es m+n-1
82         int maxBalance = m + n;
83
84         memo.assign(
85             m,
86             vector<vector<int>>(
87                 n,
88                 vector<int>(maxBalance + 1, -1)
89             )
90         );
91
92         string s;
93         s.push_back(grid[0][0]);
94
95         int balance = (grid[0][0] == '(' ? 1 : -1);
96

```

2.4 Camino en Grilla con Balance de Parentesis - Has Valid Path (cont.)

```

97         return dfs(grid, 0, 0, balance, s);
98     }
99 };

```

2.5 Convex Hull Trick

O: (n) amortizado, add y query juntos

Uso: cht.add(m,b) con m DECRECIENTE; cht.query(x) con x CRECIENTE

Optimiza DP's de la forma $dp[i] = \min_j (m[j] * x[i] + b[j])$, es decir

C++/Notebook/Convex Hull Trick.cpp

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  struct CHT
6  {
7      struct L { long long m, b; double x; };
8      deque<L> dq;
9
10     double inter(const L &a, const L &c)
11     {
12         return double(c.b - a.b) / double(a.m - c.m);
13     }
14
15     void add(long long m, long long b)
16     {
17         L l = {m, b, 0};
18         while (dq.size() >= 2 && inter(dq[dq.size() - 2], dq.back()) >= inter(dq.back(), l))
19         {
20             dq.pop_back();
21         }
22         if (dq.empty()) l.x = -1e300;
23         else l.x = inter(dq.back(), l);
24         dq.push_back(l);
25     }
26
27     long long query(long long x)
28     {
29         while (dq.size() >= 2 && dq[1].x <= x)
30         {
31             dq.pop_front();
32         }
33         return dq.front().m * x + dq.front().b;
34     }
35 };
36
37 int main()
38 {
39     ios_base::sync_with_stdio(false);
40     cin.tie(nullptr);
41     return 0;
42 }

```

2.6 Digit DP

O: (digitos * estados * 10), aquí 20*200*2*10

Uso: solve_digitdp(B,d) - solve_digitdp(A-1,d) // rango [A,B]

Contar numeros de 0 a X por una propiedad de sus digitos, con el flag tight

C++/Notebook/Digit DP.cpp

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  long long dp[20][200][2];
6  string S;
7  int D;
8
9  long long go(int i, int s, int tight)
10 {
11     if (i == (int)S.size()) return (s % D == 0);

```

2.6 Digit DP (cont.)

```

12
13     long long &r = dp[i][s][tight];
14     if (r != -1) return r;
15     r = 0;
16
17     int lim = 9;
18     if (tight) lim = S[i] - '0';
19
20     for (int d = 0; d <= lim; ++d)
21     {
22         r += go(i + 1, (s + d) % D, tight && d == lim);
23     }
24     return r;
25 }
26
27 long long solve_digitdp(long long X, int d)
28 {
29     S = to_string(X);
30     D = d;
31     memset(dp, -1, sizeof dp);
32     return go(0, 0, 1);
33 }
34
35 int main()
36 {
37     ios_base::sync_with_stdio(false);
38     cin.tie(nullptr);
39
40     long long A, B;
41     int d;
42     while (cin >> A >> B >> d)
43     {
44         cout << solve_digitdp(B, d) - solve_digitdp(A - 1, d) << "\n";
45     }
46
47     return 0;
48 }

```

2.7 Distancia de Damerau-Levenshtein

O: (n*m) tiempo y memoria

Uso: lee dos cadenas por stdin; la DP esta en main, extraer el doble for

Mínimo número de operaciones para convertir a en b, con CUATRO operaciones de costo 1

C++/Notebook/Distancia de Damerau-Levenshtein.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  static int H[1005][1005];
5
6  int main() {
7      ios::sync_with_stdio(false);
8      cin.tie(nullptr);
9
10     string a, b;
11     while (cin >> a >> b) {
12         if (a == "" && b == "") break;
13
14         int n = a.size(), m = b.size();
15         int INF = n + m;
16
17         // H desplazada en 1: H[i+1][j+1] = distancia entre a[0..i] y b[0..j]
18         H[0][0] = INF;
19         for (int i = 0; i <= n; i++) { H[i + 1][0] = INF; H[i + 1][1] = i; }
20         for (int j = 0; j <= m; j++) { H[0][j + 1] = INF; H[1][j + 1] = j; }
21
22         int da[26] = {0}; // ultima fila donde aparecio cada letra en a
23
24         for (int i = 1; i <= n; i++) {
25             int db = 0; // ultima columna en esta fila donde a[i]==b[j]
26             for (int j = 1; j <= m; j++) {
27                 int k = da[b[j] - 'a'];

```

2.7 Distancia de Damerau-Levenshtein (cont.)

```

28         int l = db;
29         int cost = 1;
30         if (a[i - 1] == b[j - 1]) { cost = 0; db = j; }
31
32         H[i + 1][j + 1] = min({
33             H[i][j] + cost,           // sustituir/igual
34             H[i + 1][j] + 1,         // insertar
35             H[i][j + 1] + 1,         // borrar
36             H[k][l] + (i - k - 1) + 1 + (j - l - 1) // intercambiar
37         });
38     }
39     da[a[i - 1] - 'a'] = i;
40 }
41
42 cout << H[n + 1][m + 1] << '\n';
43 }
44 return 0;
45 }

```

2.8 Divide and Conquer Optimization

O: (n log n) por capa, en vez de $O(n^2)$

Uso: compute(1, n, 1, n, C, prev, cur, opt); exige desigualdad de Monge

Optimiza DP por capas del tipo $dp[i][m] = \min \text{ sobre } k \text{ de } (dp[i-1][k] + C(k, m))$

C++/Notebook/Divide and Conquer Optimization.cpp

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  const long long INF = (1LL << 62);
6
7  void compute(int l, int r, int optl, int optr,
8              const function<long long(int, int)>& C,
9              vector<long long>& prev, vector<long long>& cur, vector<int>& opt)
10 {
11     if (l > r) return;
12
13     int m = (l + r) / 2;
14     int bestk = optl;
15     long long best = INF;
16
17     for (int k = optl; k <= min(m - 1, optr); ++k)
18     {
19         long long v = prev[k] + C(k, m);
20         if (v < best)
21         {
22             best = v;
23             bestk = k;
24         }
25     }
26
27     cur[m] = best;
28     opt[m] = bestk;
29
30     compute(l, m - 1, optl, bestk, C, prev, cur, opt);
31     compute(m + 1, r, bestk, optr, C, prev, cur, opt);
32 }
33
34 int main()
35 {
36     ios_base::sync_with_stdio(false);
37     cin.tie(nullptr);
38     return 0;
39 }

```

2.9 DP con Arreglo de Diferencias - Balanced Balloons

O: (K^2) , DP con arreglo de diferencias

Contar secuencias donde el promedio se mantiene entero en cada paso, modulo 998244353

C++/CPC/Regionals 2025/B - Balanced Balloons.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  typedef long long ll;
5
6  const ll MOD = 998244353;
7
8  int main()
9  {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     ll N, K;
14     cin >> N >> K;
15
16     // Despues de K personas ya no aparecen nuevas posibilidades.
17     N = min(N, K);
18
19     /*
20      dp[q] = cantidad de secuencias cuyo promedio actual es q.
21     */
22
23     vector<ll> dp(K + 2, 0);
24
25     // Con una sola persona:
26     // puede aportar cualquier valor entre 1 y K.
27     for (int q = 1; q <= K; q++)
28         dp[q] = 1;
29
30     for (ll i = 1; i < N; i++)
31     {
32         vector<ll> diff(K + 3, 0);
33
34         for (int q = 1; q <= K; q++)
35         {
36             if (dp[q] == 0)
37                 continue;
38
39             /*
40              Queremos:
41
42              1 <= (i+1)*q2 - i*q <= K
43
44              Por lo tanto:
45
46              ceil((i*q + 1)/(i+1)) <= q2
47              q2 <= floor((i*q + K)/(i+1))
48             */
49
50             ll lo = (i * q + 1 + i) / (i + 1);
51             ll hi = (i * q + K) / (i + 1);
52
53             lo = max(lo, 1LL);
54             hi = min(hi, K);
55
56             if (lo <= hi)
57             {
58                 // Sumar dp[q] a todo [lo, hi]
59                 diff[lo] = (diff[lo] + dp[q]) % MOD;
60
61                 diff[hi + 1] =
62                     (diff[hi + 1] - dp[q] + MOD) % MOD;
63             }
64         }
65
66         // Reconstruimos ndp usando prefijos.
67         vector<ll> ndp(K + 2, 0);

```

2.9 DP con Arreglo de Diferencias - Balanced Balloons (cont.)

```

68
69     ll cur = 0;
70
71     for(int q = 1; q <= K; q++)
72     {
73         cur = (cur + diff[q]) % MOD;
74         ndp[q] = cur;
75     }
76
77     dp = ndp;
78 }
79
80 ll ans = 0;
81
82 for(int q = 1; q <= K; q++)
83 {
84     ans = (ans + dp[q]) % MOD;
85 }
86
87 cout << ans << '\n';
88
89 return 0;
90 }

```

2.10 DP sobre Intervalos con Bandera de Inversion - Artic Virus

O: (n^2) por cadena, DP de intervalos con bandera de inversion
Decidir si una cadena es un virus segun la gramatica phi

C++/ICPC/2024/A - Artic Virus.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string s;
5 int n;
6
7 // memo[l][r][rev]: -1 = sin calcular, 0 = no, 1 = si. Un arreglo en vez de
8 // map<tuple>: con n = 1000 el map costaba cientos de ms por caso.
9 signed char memo[1001][1001][2];
10
11 bool solve(int l, int r, bool rev) {
12     if (l > r) return false;
13
14     signed char &res = memo[l][r][rev];
15
16     if (res != -1)
17         return res;
18
19     int len = r - l + 1;
20
21     // Llegamos a una configuracion simple
22     if (len == 1) {
23         char c = rev ? s[r] : s[l];
24         res = (c == 'A' || c == 'T');
25         return res;
26     }
27
28     // Obtener extremos de la cadena actual
29     char first = rev ? s[r] : s[l];
30     char last = rev ? s[l] : s[r];
31
32     bool ans = false;
33
34     // phiC
35     // Si termina en C, quitamos el ultimo caracter
36     if (last == 'C') {
37         if (!rev)
38             ans |= solve(l, r - 1, false);
39         else
40             ans |= solve(l + 1, r, true);
41     }
42
43 }

```

2.10 DP sobre Intervalos con Bandera de Inversion - Artic Virus (cont.)

```

43 // Aphi
44 // Si comienza en A, quitamos el primero
45 if (first == 'A') {
46     if (!rev)
47         ans |= solve(l + 1, r, false);
48     else
49         ans |= solve(l, r - 1, true);
50 }
51
52 // Aphi^1-1
53 // Quitamos A del inicio y luego invertimos
54 if (first == 'A') {
55     if (!rev)
56         ans |= solve(l + 1, r, true);
57     else
58         ans |= solve(l, r - 1, false);
59 }
60
61 // Gphi^1-1C
62 // Debe comenzar en G y terminar en C
63 if (first == 'G' && last == 'C' && len >= 3) {
64     if (!rev)
65         ans |= solve(l + 1, r - 1, true);
66     else
67         ans |= solve(l + 1, r - 1, false);
68 }
69
70 res = ans;
71 return res;
72 }
73
74 int main() {
75     ios::sync_with_stdio(false);
76     cin.tie(nullptr);
77
78     while (cin >> n >> s) {
79         for (int l = 0; l < n; l++)
80             for (int r = 0; r < n; r++)
81                 memo[l][r][0] = memo[l][r][1] = -1;
82
83         // Largo 1: solo A y T son la etapa simple; C y G no son virus.
84         if (n == 1) {
85             if (s[0] == 'A' || s[0] == 'T')
86                 cout << "simple\n";
87             else
88                 cout << "doomed\n";
89         }
90         else {
91             bool ok = solve(0, n - 1, false);
92
93             cout << (ok ? "mutation\n" : "doomed\n");
94         }
95     }
96
97     return 0;
98 }

```

2.11 Interval Scheduling - Yuyuan Market

O: $(n \log n)$ por ordenar por fin; el greedy es (n)
Una fila de 2N jianzhi con simbolos repetidos (cada simbolo aparece dos veces)

C++/Training_Camp_2026/Day 10/A - Yuyuan Market.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 22];
5 static int bufLen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0;
8         if (!bufLen_) return -1; }

```

2.11 Interval Scheduling - Yuyuan Market (cont.)

```

8     return buf_[bufPos_++];
9 }
10 inline int readInt() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18
19 int main() {
20     int N = readInt();
21     int M = 2 * N;
22     vector<int> L(N + 1, 0), R(N + 1, 0);
23     vector<bool> seen(N + 1, false);
24     for (int pos = 1; pos <= M; pos++) {
25         int v = readInt();
26         if (!seen[v]) { L[v] = pos; seen[v] = true; }
27         else R[v] = pos;
28     }
29
30     vector<int> endAtL(M + 1, -1); // endAtL[p] = L of the symbol whose R equals p, or -1
31     for (int v = 1; v <= N; v++) endAtL[R[v]] = L[v];
32
33     vector<int> cnt(M + 1, 0);
34     vector<long long> len(M + 1, 0);
35     for (int p = 1; p <= M; p++) {
36         cnt[p] = cnt[p - 1];
37         len[p] = len[p - 1];
38         if (endAtL[p] != -1) {
39             int Lv = endAtL[p];
40             int candCnt = cnt[Lv - 1] + 1;
41             long long candLen = len[Lv - 1] + (long long)(p - Lv);
42             if (candCnt > cnt[p] || (candCnt == cnt[p] && candLen < len[p])) {
43                 cnt[p] = candCnt;
44                 len[p] = candLen;
45             }
46         }
47     }
48
49     printf("%d %lld\n", cnt[M], len[M]);
50     return 0;
51 }

```

2.12 LCS con Bloques de K Caracteres

O: (n*m) tiempo y memoria

Uso: lee K y las dos cadenas; imprime el largo maximo. K = 0 termina LCS pero tomando solo bloques de K o mas caracteres seguidos en las dos cadenas

C++/Notebook/LCS con Bloques de K.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     ios::sync_with_stdio(false);
7     cin.tie(nullptr);
8
9     int K;
10
11     while(cin >> K && K)
12     {
13         string a, b;
14         cin >> a >> b;
15
16         int n = a.size();
17         int m = b.size();
18
19         vector<vector<int>> dp(n + 1, vector<int>(m + 1));
20         vector<vector<int>> bloque(n + 1, vector<int>(m + 1));

```

2.12 LCS con Bloques de K Caracteres (cont.)

```

21
22     for(int i = 1; i <= n; i++)
23     {
24         for(int j = 1; j <= m; j++)
25         {
26             // sin cerrar bloque en (i, j): lo mejor de los dos vecinos
27             dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
28
29             if(a[i-1] == b[j-1])
30             {
31                 // contador de racha, que pasa a ser valor de DP al llegar a K
32                 bloque[i][j] = bloque[i-1][j-1] + 1;
33
34                 if(bloque[i][j] >= K)
35                 {
36                     bloque[i][j] = max(
37                         bloque[i][j], // estirar el bloque de atras
38                         dp[i-K][j-K] + K // o abrir uno nuevo de K
39                     );
40
41                     dp[i][j] = max(dp[i][j], bloque[i][j]);
42                 }
43             }
44         }
45     }
46
47     cout << dp[n][m] << '\n';
48 }
49
50 return 0;
51 }

```

2.13 LIS en O(n log n) con Reconstruccion

O: (n log n)

Uso: auto idx = lis(a, true); largo = idx.size(); a[idx[k]] es la k-esima de la LIS Subsecuencia creciente mas larga con lower_bound, y los indices de una de ellas

C++/Notebook/LIS.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 typedef long long ll;
6
7 vector<int> lis(const vector<ll> &a, bool estricta = true)
8 {
9     int n = a.size();
10     vector<ll> tail; // tail[k]: menor final de una de largo k+1
11     vector<int> tailIdx, padre(n, -1);
12     for (int i = 0; i < n; i++)
13     {
14         auto it = estricta ? lower_bound(tail.begin(), tail.end(), a[i])
15                             : upper_bound(tail.begin(), tail.end(), a[i]);
16         int k = it - tail.begin();
17         if (k == (int)tail.size())
18         {
19             tail.push_back(a[i]);
20             tailIdx.push_back(i);
21         }
22         else
23         {
24             tail[k] = a[i];
25             tailIdx[k] = i;
26         }
27         padre[i] = k ? tailIdx[k - 1] : -1;
28     }
29     vector<int> res;
30     for (int i = tail.empty() ? -1 : tailIdx.back(); i != -1; i = padre[i])
31     {
32         res.push_back(i);
33     }

```

2.13 LIS en $O(n \log n)$ con Reconstruccion (cont.)

```

34 reverse(res.begin(), res.end());
35 return res; // indices crecientes de una LIS
36 }
37
38 int main()
39 {
40     ios_base::sync_with_stdio(false);
41     cin.tie(nullptr);
42     return 0;
43 }

```

2.14 Prefix Sums - Zig Zag Array

O: (n), sumas de prefijos

Cuenta los arreglos "zigzag" de longitud n con valores en [

C++/LeetCode/ZigZagArray.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:
7     static constexpr int MOD = 1'000'000'007;
8
9     int zigZagArrays(int n, int l, int r) {
10         int m = r - l + 1;
11
12         vector<long long> up(m), down(m);
13         vector<long long> pref(m + 1);
14         vector<long long> suff(m + 1);
15         vector<long long> newUp(m);
16         vector<long long> newDown(m);
17
18         for (int i = 0; i < m; i++) {
19             up[i] = i;
20             down[i] = m - 1 - i;
21         }
22
23         for (int len = 3; len <= n; len++) {
24             pref[0] = 0;
25             for (int i = 0; i < m; i++) {
26                 pref[i + 1] = (pref[i] + down[i]) % MOD;
27             }
28
29             suff[m] = 0;
30             for (int i = m - 1; i >= 0; i--) {
31                 suff[i] = (suff[i + 1] + up[i]) % MOD;
32             }
33
34             for (int i = 0; i < m; i++) {
35                 newUp[i] = pref[i];
36                 newDown[i] = suff[i + 1];
37             }
38
39             up.swap(newUp);
40             down.swap(newDown);
41         }
42
43         long long ans = 0;
44         for (int i = 0; i < m; i++) {
45             ans = (ans + up[i] + down[i]) % MOD;
46         }
47
48         return (int)ans;
49     }
50 };

```

2.15 SOS (Sum over Subsets) + Inclusion-Exclusion - How Many Teams

O: ($2^k * k$), SOS DP mas inclusion-exclusion

N estudiantes tienen cada uno un subconjunto de K habilidades (representado como bitmask)

C++/Training_Camp_2026/Day 6/H - How many teams.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 22];
5 static int bufLen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0;
8         if (!bufLen_) return -1; }
9     return buf_[bufPos_++];
10 }
11 inline int readInt() {
12     int c = gc();
13     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
14     bool neg = false; if (c == '-') { neg = true; c = gc(); }
15     int x = 0;
16     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
17     return neg ? -x : x;
18 }
19 // Reads a binary string of known length K, skipping any non-'0'/'1' characters first,
20 // and returns the mask with position 1 (first character) as bit 0 (matches problem's
21 // convention, verified against both worked examples).
22 inline int readMaskFixedLen(int K) {
23     int c = gc();
24     while (c != '0' && c != '1') c = gc();
25     int mask = 0;
26     for (int pos = 0; pos < K; pos++) {
27         if (c == '1') mask |= (1 << pos);
28         if (pos + 1 < K) c = gc();
29     }
30     return mask;
31 }
32
33 int main() {
34     int N = readInt(), K = readInt();
35     int full = 1 << K;
36     vector<long long> f(full, 0); // will hold cnt[] then get turned into SOS sums
37
38     for (int i = 0; i < N; i++) {
39         int mask = readMaskFixedLen(K);
40         f[mask]++;
41     }
42
43     // SOS: f[mask] = number of students whose mask is a submask of `mask`
44     for (int bit = 0; bit < K; bit++) {
45         int b = 1 << bit;
46         for (int mask = 0; mask < full; mask++) {
47             if (mask & b) f[mask] += f[mask ^ b];
48         }
49     }
50
51     // g[mask] = C(f[mask], 3)
52     vector<long long> h(full, 0);
53     for (int mask = 0; mask < full; mask++) {
54         long long v = f[mask];
55         h[mask] = (v >= 3) ? (v * (v - 1) * (v - 2) / 6) : 0;
56     }
57
58     // Mobius inversion (inclusion-exclusion) to get exact-union counts
59     for (int bit = 0; bit < K; bit++) {
60         int b = 1 << bit;
61         for (int mask = 0; mask < full; mask++) {
62             if (mask & b) h[mask] -= h[mask ^ b];
63         }
64     }
65
66     int M = readInt();
67     string out;

```

2.15 SOS (Sum over Subsets) + Inclusion-Exclusion - How Many Teams (cont.)

```

67 out.reserve((size_t)M * 8);
68 char tmp[32];
69 for (int q = 0; q < M; q++) {
70     int mask = readMaskFixedLen(K);
71     int len = sprintf(tmp, "%lld\n", h[mask]);
72     out.append(tmp, len);
73 }
74 fwrite(out.data(), 1, out.size(), stdout);
75 return 0;
76 }

```

2.16 Steiner Tree - Joining Capitals

O: $(3^k * n + 2^k * m \log n)$, Steiner tree sobre k terminales
Un reino tiene N ciudades, K de ellas capitales

C++/Training_Camp_2026/Day 8/J - Joining Capitals.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int N, K;
6     cin >> N >> K;
7     vector<double> X(N), Y(N);
8     for (int i = 0; i < N; i++) cin >> X[i] >> Y[i];
9
10    vector<vector<double>> dist(N, vector<double>(N, 0.0));
11    for (int i = 0; i < N; i++) {
12        for (int j = 0; j < N; j++) {
13            double dx = X[i] - X[j], dy = Y[i] - Y[j];
14            dist[i][j] = sqrt(dx * dx + dy * dy);
15        }
16    }
17
18    vector<int> nonCap;
19    for (int v = K; v < N; v++) nonCap.push_back(v);
20    int M = (int)nonCap.size();
21
22    const double INF = 1e18;
23    int fullMask = (1 << K) - 1;
24    // dp[mask][idx] : idx indexes into nonCap (0..M-1)
25    vector<vector<double>> dp(1 << K, vector<double>(M, INF));
26
27    // base case: each capital connects directly to each non-capital
28    for (int i = 0; i < K; i++) {
29        for (int idx = 0; idx < M; idx++) {
30            dp[1 << i][idx] = dist[i][nonCap[idx]];
31        }
32    }
33
34    for (int mask = 1; mask <= fullMask; mask++) {
35        // MERGE step: combine two disjoint submasks meeting at the same non-capital
36        for (int sub = (mask - 1) & mask; sub > 0; sub = (sub - 1) & mask) {
37            int other = mask ^ sub;
38            if (sub < other) {
39                for (int idx = 0; idx < M; idx++) {
40                    double cand = dp[sub][idx] + dp[other][idx];
41                    if (cand < dp[mask][idx]) dp[mask][idx] = cand;
42                }
43            }
44        }
45
46        // EXTEND step: simple O(M^2) Dijkstra among non-capitals only
47        vector<bool> done(M, false);
48        for (int iter = 0; iter < M; iter++) {
49            int u = -1;
50            double best = INF;
51            for (int idx = 0; idx < M; idx++) {
52                if (!done[idx] && dp[mask][idx] < best) { best = dp[mask][idx]; u = idx; }
53            }
54            if (u == -1) break;
55            done[u] = true;

```

2.16 Steiner Tree - Joining Capitals (cont.)

```

56     for (int w = 0; w < M; w++) {
57         if (!done[w]) {
58             double cand = dp[mask][u] + dist[nonCap[u]][nonCap[w]];
59             if (cand < dp[mask][w]) dp[mask][w] = cand;
60         }
61     }
62 }
63 }
64
65 double answer = INF;
66 for (int idx = 0; idx < M; idx++) answer = min(answer, dp[fullMask][idx]);
67
68 printf("%.5f\n", answer);
69 return 0;
70 }

```

2.17 Subset Sum (Knapsack) - Karamell

O: $(n * \text{suma})$, subset sum clasico

Alice y Bob reciben N bolsas de caramelos que se reparten en el orden dado

C++/Training_Camp_2026/Day 3/K - Karamell.py

```

1 import sys
2 input = sys.stdin.readline
3
4 n = int(input())
5 ka = sorted(map(int, input().split()))
6 suma = sum(ka)
7
8 if suma % 2 != 0:
9     print(-1)
10 else:
11     mit = suma // 2
12
13     # ---- subset-sum DP: reemplaza el rec exponencial ----
14     # dp[i][s] = con los primeros i elementos se puede formar la suma s
15     dp = [bytearray(mit + 1) for _ in range(n + 1)]
16     dp[0][0] = 1
17     for i in range(1, n + 1):
18         w = ka[i - 1]
19         cur = dp[i]
20         prev = dp[i - 1]
21         for s in range(mit + 1):
22             if prev[s] or (s >= w and prev[s - w]):
23                 cur[s] = 1
24
25     if not dp[n][mit]:
26         print(-1)
27     else:
28         # reconstruyo el subconjunto que suma mit (queda como sub ordenado, igual que f[0])
29
30         sub = []
31         s = mit
32         for i in range(n, 0, -1):
33             w = ka[i - 1]
34             if s >= w and dp[i - 1][s - w]: # este elemento entra al subconjunto
35                 sub.append(w)
36                 s -= w
37         sub.reverse() # ascendente, como lo esperaba tu c digo
38
39         # ---- de aqu  en adelante: TU MISMA reconstrucci n ----
40         alice = []
41         bob = []
42         pos = 0
43         for j in range(len(ka)):
44             if pos < len(sub) and sub[pos] == ka[j]:
45                 alice.append(j)
46                 pos += 1
47             else:
48                 bob.append(j)
49
50         alice1 = 0

```

2.17 Subset Sum (Knapsack) - Karamell (cont.)

```

50 bob1 = 0
51 posA = 0
52 posB = len(bob) - 1
53 imp = []
54 for _ in range(len(ka)):
55     if alice1 < bob1:
56         v = ka[alice[posA]]; posA += 1
57         alice1 += v
58     else:
59         v = ka[bob[posB]]; posB -= 1
60         bob1 += v
61     imp.append(str(v))
62 print(" ".join(imp))

```

2.18 Valor Esperado - Revenge Of The Salary Of At Coder Inc

O: (N), valor esperado hacia atras

Valor esperado del salario con un dado de N caras, modulo 998244353

C++/Training_Camp_2026/Day 10/N - Revenge of The Salary of AiCoder Inc.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5 const ll MOD = 998244353;
6
7 ll binpow(ll a, ll e) {
8     ll r = 1;
9     while (e) {
10         if (e & 1) r = r * a % MOD;
11         a = a * a % MOD;
12         e >>= 1;
13     }
14     return r;
15 }
16
17 ll inv(ll x) {
18     return binpow(x, MOD - 2);
19 }
20
21 int main() {
22     ios::sync_with_stdio(false);
23     cin.tie(nullptr);
24
25     int N;
26     cin >> N;
27
28     vector<ll> A(N + 1);
29     for (int i = 1; i <= N; i++)
30         cin >> A[i];
31
32     ll invN = inv(N);
33
34     vector<ll> E(N + 2, 0);
35
36     ll S = 0;
37
38     for (int i = N; i >= 1; i--) {
39         E[i] = S * invN % MOD;
40         S = (S + A[i] + E[i]) % MOD;
41     }
42
43     ll ans = S * invN % MOD;
44
45     cout << ans << '\n';
46 }

```

3 - Graph

3.1 2-SAT (Kosaraju) - Signal Coverage

O: (n + m), 2-SAT con Kosaraju sobre el grafo de implicaciones

Escoger antenas con restricciones de a dos variables: 2-SAT resuelto con Kosaraju

C++/ICPC/2024/G - Signal Coverage.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Antenna {
5     long long x, y, r;
6     int b, e;
7 };
8
9 int main() {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     int T;
14     cin >> T;
15
16     while (T--) {
17
18         int n, m;
19         cin >> n >> m;
20
21         vector<Antenna> a(n);
22
23         for (int i = 0; i < n; i++) {
24             // el enunciado da el RADIO primero: r x y b e
25             cin >> a[i].r >> a[i].x >> a[i].y
26                 >> a[i].b >> a[i].e;
27         }
28
29         /*
30          2-SAT:
31
32          2*i    -> antena i instalada
33          2*i + 1 -> antena i NO instalada
34
35          negacion:
36          x ^ 1
37          */
38
39         int V = 2 * n;
40
41         vector<vector<int>> g(V), gr(V);
42
43         auto addEdge = [&](int u, int v) {
44             g[u].push_back(v);
45             gr[v].push_back(u);
46         };
47
48         // (u OR v)
49         auto addClause = [&](int u, int v) {
50
51             // !u -> v
52             addEdge(u ^ 1, v);
53
54             // !v -> u
55             addEdge(v ^ 1, u);
56         };
57
58         // -----
59         // Restricciones dadas por las parejas
60         // -----
61
62         for (int i = 0; i < m; i++) {
63
64             int u, v;
65             cin >> u >> v;
66
67             --u;

```

3.1 2-SAT (Kosaraju) - Signal Coverage (cont.)

```

68      --v;
69
70      // Al menos una debe instalarse
71      // xu OR xv
72      addClause(2 * u, 2 * v);
73  }
74
75  // -----
76  // Restricciones entre antenas
77  // -----
78
79  for (int i = 0; i < n; i++) {
80
81      for (int j = i + 1; j < n; j++) {
82
83          // Sus periodos se intersectan?
84          bool sameTime =
85              max(a[i].b, a[j].b) <=
86              min(a[i].e, a[j].e);
87
88          if (!sameTime)
89              continue;
90
91          // Sus circulos se intersectan?
92          long long dx = a[i].x - a[j].x;
93          long long dy = a[i].y - a[j].y;
94
95          long long dist2 = dx * dx + dy * dy;
96
97          long long radius = a[i].r + a[j].r;
98
99          bool samePlace =
100              dist2 <= radius * radius;
101
102          if (samePlace) {
103
104              /*
105               No pueden instalarse las dos:
106
107               !(xi AND xj)
108
109               = !xi OR !xj
110               */
111
112              addClause(
113                  2 * i + 1,
114                  2 * j + 1
115              );
116          }
117      }
118  }
119
120  // -----
121  // Kosaraju - primera pasada
122  // -----
123
124  vector<bool> visited(V, false);
125  vector<int> order;
126
127  function<void(int)> dfs1 = [&](int u) {
128
129      visited[u] = true;
130
131      for (int v : g[u]) {
132          if (!visited[v])
133              dfs1(v);
134      }
135
136      order.push_back(u);
137  };
138
139  for (int i = 0; i < V; i++) {

```

3.1 2-SAT (Kosaraju) - Signal Coverage (cont.)

```

140      if (!visited[i])
141          dfs1(i);
142  }
143
144  // -----
145  // Kosaraju - segunda pasada
146  // -----
147
148  vector<int> comp(V, -1);
149
150  function<void(int, int)> dfs2 =
151      [&](int u, int id) {
152
153      comp[u] = id;
154
155      for (int v : gr[u]) {
156          if (comp[v] == -1)
157              dfs2(v, id);
158      }
159  };
160
161  reverse(order.begin(), order.end());
162
163  int components = 0;
164
165  for (int u : order) {
166
167      if (comp[u] == -1) {
168          dfs2(u, components++);
169      }
170  }
171
172  // -----
173  // Comprobar si existe solucion
174  // -----
175
176  bool possible = true;
177
178  for (int i = 0; i < n; i++) {
179
180      int on = 2 * i;
181      int off = 2 * i + 1;
182
183      if (comp[on] == comp[off]) {
184          possible = false;
185          break;
186      }
187  }
188
189  if (!possible) {
190      cout << "Impossible\n";
191      continue;
192  }
193
194  // -----
195  // Construir respuesta
196  // -----
197
198  string ans(n, '0');
199
200  for (int i = 0; i < n; i++) {
201
202      int on = 2 * i;
203      int off = 2 * i + 1;
204
205      ans[i] = (comp[on] > comp[off]) ? '1' : '0';
206  }
207
208  cout << ans << '\n';
209  }
210
211  return 0;

```

3.1 2-SAT (Kosaraju) - Signal Coverage (cont.)

```
2 2 }
```

3.2 Articulation Points (Tarjan) - Submerging Islands

O: (n + m), puntos de articulaci n con Tarjan

Dado un grafo de N islas conectadas por M puentes, hay que contar cuantas islas, al sumergirse

C++/Training_Camp_2026/Day 4/L - Submerging Islands.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXN = 10005;
5
6 vector<int> adj[MAXN];
7 int disc[MAXN], low[MAXN], timer;
8 bool visited[MAXN], articulation[MAXN];
9
10 void dfs(int u, int parent) {
11     visited[u] = true;
12     disc[u] = low[u] = ++timer;
13     int children = 0;
14
15     for (int v : adj[u]) {
16         if (!visited[v]) {
17             children++;
18             dfs(v, u);
19
20             low[u] = min(low[u], low[v]);
21
22             if (parent != -1 && low[v] >= disc[u])
23                 articulation[u] = true;
24         }
25         else if (v != parent) {
26             low[u] = min(low[u], disc[v]);
27         }
28     }
29
30     if (parent == -1 && children > 1)
31         articulation[u] = true;
32 }
33
34 int main() {
35     ios::sync_with_stdio(false);
36     cin.tie(nullptr);
37
38     int N, M;
39
40     while (cin >> N >> M && (N || M)) {
41
42         for (int i = 1; i <= N; i++) {
43             adj[i].clear();
44             visited[i] = false;
45             articulation[i] = false;
46         }
47
48         timer = 0;
49
50         for (int i = 0; i < M; i++) {
51             int u, v;
52             cin >> u >> v;
53             adj[u].push_back(v);
54             adj[v].push_back(u);
55         }
56
57         // Por si el grafo no fuera conexo.
58         for (int i = 1; i <= N; i++)
59             if (!visited[i])
60                 dfs(i, -1);
61
62         int ans = 0;
63         for (int i = 1; i <= N; i++)
64             if (articulation[i])
```

3.2 Articulation Points (Tarjan) - Submerging Islands (cont.)

```
65         ans++;
66
67         cout << ans << '\n';
68     }
69
70     return 0;
71 }
```

3.3 BFS - Ladder Length

O: (n * largo * 26), BFS sobre el diccionario

Halla la longitud de la transformaci n mas corta de beginWord a endWord cambiando una letra a la vez

C++/LeetCode/ladderLength.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:
7     int ladderLength(string beginWord, string endWord, vector<string>& wordList) {
8         unordered_set<string> words(wordList.begin(), wordList.end());
9
10        if (!words.count(endWord))
11            return 0;
12
13        queue<pair<string, int>> q;
14        q.push({beginWord, 1});
15
16        while (!q.empty()) {
17            auto [word, dist] = q.front();
18            q.pop();
19
20            if (word == endWord)
21                return dist;
22
23            for (int i = 0; i < word.size(); i++) {
24                char original = word[i];
25
26                for (char c = 'a'; c <= 'z'; c++) {
27                    if (c == original) continue;
28
29                    word[i] = c;
30
31                    if (words.count(word)) {
32                        q.push({word, dist + 1});
33                        words.erase(word); // mark as visited
34                    }
35                }
36
37                word[i] = original;
38            }
39        }
40
41        return 0;
42    }
43};
```

3.4 BFS - Boss

O: (n + m), BFS

En un organigrama de N empleados donde X maneja directamente a Y

C++/Training_Camp_2026/Day 2/C - Boss.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int N, M, I;
9 }
```

3.4 BFS - Boss (cont.)

```

0 // El problema puede contener varios casos de prueba
1 while (cin >> N >> M >> I) {
2
3     // age[i] = edad del empleado i
4     vector<int> age(N + 1);
5     for (int i = 1; i <= N; i++)
6         cin >> age[i];
7
8     /*
9      rev representa el grafo invertido.
10
11     Si X maneja directamente a Y:
12         X -> Y
13
14     entonces almacenamos:
15
16         rev[Y].push_back(X)
17
18     porque durante una consulta queremos subir por la cadena
19     de mando (buscar todos los jefes).
20 */
21 vector<vector<int>> rev(N + 1);
22
23 for (int i = 0; i < M; i++) {
24     int X, Y;
25     cin >> X >> Y;
26     rev[Y].push_back(X);
27 }
28
29 /*
30 El grafo nunca cambia.
31
32 Lo único que cambia es qué empleado ocupa cada posición
33 dentro del organigrama.
34
35 empPos[e] = posición donde está actualmente el empleado e.
36
37 posEmp[p] = empleado que ocupa actualmente la posición p.
38 */
39
40 vector<int> empPos(N + 1);
41 vector<int> posEmp(N + 1);
42
43 // Inicialmente cada empleado está en su propia posición.
44 for (int i = 1; i <= N; i++) {
45     empPos[i] = i;
46     posEmp[i] = i;
47 }
48
49 while (I--) {
50
51     char op;
52     cin >> op;
53
54     // Cambio de posiciones
55     if (op == 'T') {
56
57         int A, B;
58         cin >> A >> B;
59
60         // Posiciones actuales de ambos empleados
61         int posA = empPos[A];
62         int posB = empPos[B];
63
64         // Intercambiar las posiciones de los empleados
65         swap(empPos[A], empPos[B]);
66
67         // Actualizar qué empleado queda en cada posición
68         swap(posEmp[posA], posEmp[posB]);
69     }
70
71     // Consulta

```

3.4 BFS - Boss (cont.)

```

82     else {
83
84         int E;
85         cin >> E;
86
87         // Posición donde se encuentra actualmente el empleado E
88         int start = empPos[E];
89
90         vector<int> vis(N + 1, 0);
91         queue<int> q;
92
93         q.push(start);
94         vis[start] = true;
95
96         int ans = INT_MAX;
97
98         /*
99          Hacemos BFS sobre el grafo invertido.
100
101          Desde la posición del empleado vamos visitando
102          todos los jefes (directos e indirectos).
103
104          Cada nodo del BFS representa una POSICIÓN.
105
106          Para conocer la edad del jefe debemos averiguar
107          qué empleado ocupa actualmente esa posición.
108 */
109         while (!q.empty()) {
110
111             int u = q.front();
112             q.pop();
113
114             // Revisar todos los jefes directos
115             for (int bossPos : rev[u]) {
116
117                 if (vis[bossPos])
118                     continue;
119
120                 vis[bossPos] = true;
121                 q.push(bossPos);
122
123                 // Empleado que ocupa esa posición
124                 int bossEmp = posEmp[bossPos];
125
126                 // Actualizar la edad mínima encontrada
127                 ans = min(ans, age[bossEmp]);
128             }
129         }
130
131         // Si nunca encontramos un jefe
132         if (ans == INT_MAX)
133             cout << "\n";
134         else
135             cout << ans << "\n";
136     }
137 }
138
139 return 0;
140 }

```

3.5 BFS - Bacon Number

O: (n + m), BFS por capas

Dadas N películas con sus actores, hay que responder Q consultas que piden conectar dos actores x e y...

C++/Training_Camp_2026/Day 3/B - Bacon Number.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int INF = 1e9;
5

```

3.5 BFS - Bacon Number (cont.)

```

6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int N, M;
11    cin >> N >> M;
12
13    vector<vector<int>> movieActors(N + 1);
14    unordered_map<int, vector<int>> actorMovies;
15
16    for (int i = 1; i <= N; i++) {
17        int k;
18        cin >> k;
19        movieActors[i].resize(k);
20        for (int j = 0; j < k; j++) {
21            cin >> movieActors[i][j];
22            actorMovies[movieActors[i][j]].push_back(i);
23        }
24    }
25
26    // actor común entre películas
27    vector<vector<int>> common(N + 1, vector<int>(N + 1, 0));
28    vector<vector<int>> adj(N + 1);
29
30    for (auto &it : actorMovies) {
31        auto &v = it.second;
32        for (int i = 0; i < (int)v.size(); i++) {
33            for (int j = i + 1; j < (int)v.size(); j++) {
34                int a = v[i], b = v[j];
35                if (!common[a][b]) {
36                    common[a][b] = common[b][a] = it.first;
37                    adj[a].push_back(b);
38                    adj[b].push_back(a);
39                }
40            }
41        }
42    }
43
44    vector<vector<int>> dist(N + 1, vector<int>(N + 1, INF));
45    vector<vector<int>> parent(N + 1, vector<int>(N + 1, -1));
46
47    for (int s = 1; s <= N; s++) {
48        queue<int> q;
49        dist[s][s] = 0;
50        q.push(s);
51
52        while (!q.empty()) {
53            int u = q.front();
54            q.pop();
55
56            for (int v : adj[u]) {
57                if (dist[s][v] == INF) {
58                    dist[s][v] = dist[s][u] + 1;
59                    parent[s][v] = u;
60                    q.push(v);
61                }
62            }
63        }
64    }
65
66    int Q;
67    cin >> Q;
68
69    while (Q--) {
70        int x, y;
71        cin >> x >> y;
72
73        if (!actorMovies.count(x) || !actorMovies.count(y)) {
74            cout << -1 << '\n';
75            continue;
76        }
77    }

```

3.5 BFS - Bacon Number (cont.)

```

78    int bestA = -1, bestB = -1, best = INF;
79
80    for (int a : actorMovies[x])
81        for (int b : actorMovies[y])
82            if (dist[a][b] < best) {
83                best = dist[a][b];
84                bestA = a;
85                bestB = b;
86            }
87
88    if (best == INF) {
89        cout << -1 << '\n';
90        continue;
91    }
92
93    vector<int> movies;
94    int cur = bestB;
95    while (cur != bestA) {
96        movies.push_back(cur);
97        cur = parent[bestA][cur];
98    }
99    movies.push_back(bestA);
100    reverse(movies.begin(), movies.end());
101
102    vector<int> out;
103
104    out.push_back(x);
105
106    if (x != common[movies[0]][movies[0]])
107        ; // nada
108
109    int currentActor = x;
110
111    if (actorMovies[x][0] != movies[0]) {
112        // x aparece en movies[0] por construcción
113    }
114
115    for (int i = 0; i + 1 < (int)movies.size(); i++) {
116        int actor = common[movies[i]][movies[i + 1]];
117        if (currentActor != actor) {
118            out.push_back(movies[i]);
119            out.push_back(actor);
120            currentActor = actor;
121        } else {
122            out.push_back(movies[i]);
123        }
124    }
125
126    out.push_back(movies.back());
127
128    if (currentActor != y)
129        out.push_back(y);
130
131    int actors = 0;
132    for (int i = 0; i < (int)out.size(); i += 2)
133        actors++;
134
135    cout << actors << '\n';
136
137    for (int i = 0; i < (int)out.size(); i++) {
138        if (i) cout << ' ';
139        cout << out[i];
140    }
141    cout << '\n';
142 }
143 }

```

3.6 BFS Backtracking - Find Ladders

O: $(n * largo * 26)$ el BFS, mas el backtracking de los caminos

Encuentra TODAS las secuencias de transformacion mas cortas de beginWord a endWord cambiando una letra a la...

C++/LeetCode/findLadders.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:
7     vector<vector<string>> ans;
8     unordered_map<string, vector<string>> parents;
9
10    void buildPaths(string word,
11                  string beginWord,
12                  vector<string>& path) {
13
14        if (word == beginWord) {
15            vector<string> cur = path;
16            reverse(cur.begin(), cur.end());
17            ans.push_back(cur);
18            return;
19        }
20
21        for (auto& p : parents[word]) {
22            path.push_back(p);
23            buildPaths(p, beginWord, path);
24            path.pop_back();
25        }
26    }
27
28    vector<vector<string>> findLadders(string beginWord,
29                                    string endWord,
30                                    vector<string>& wordList) {
31
32        unordered_set<string> words(wordList.begin(), wordList.end());
33
34        if (!words.count(endWord))
35            return {};
36
37        queue<string> q;
38        q.push(beginWord);
39
40        unordered_set<string> visited;
41        visited.insert(beginWord);
42
43        bool found = false;
44
45        while (!q.empty() && !found) {
46
47            int sz = q.size();
48
49            unordered_set<string> levelVisited;
50
51            while (sz-- > 0) {
52
53                string word = q.front();
54                q.pop();
55
56                string original = word;
57
58                for (int i = 0; i < word.size(); i++) {
59
60                    char old = word[i];
61
62                    for (char c = 'a'; c <= 'z'; c++) {
63
64                        if (c == old) continue;
65
66                        word[i] = c;
```

3.6 BFS Backtracking - Find Ladders (cont.)

```
67
68        if (!words.count(word))
69            continue;
70
71        if (!visited.count(word)) {
72
73            if (!levelVisited.count(word)) {
74                q.push(word);
75                levelVisited.insert(word);
76            }
77
78            parents[word].push_back(original);
79
80            if (word == endWord)
81                found = true;
82        }
83    }
84
85    word[i] = old;
86    }
87 }
88
89 for (auto& w : levelVisited)
90     visited.insert(w);
91 }
92
93 if (!found)
94     return {};
95
96 vector<string> path = {endWord};
97 buildPaths(endWord, beginWord, path);
98
99 return ans;
100 }
101 };
```

3.7 BFS sobre Grafo Producto (dos fichas a la vez) - Infiltration Route

O: $((n*m)^2)$ sobre el grafo producto de las dos fichas

Hay que mover DOS posiciones a la vez por un grafo dirigido y llegar a una configuracion objetivo

C++/ICPC/Regionals 2025/I - Infiltration Route.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXN = 505;
5
6 bool seen[MAXN][MAXN][2];
7 array<int, 3> bck[MAXN][MAXN][2];
8
9 int main() {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     int n, m;
14     cin >> n >> m;
15
16     vector<vector<int>> adj(2 * n);
17
18     for (int i = 0; i < m; i++) {
19         int s, t;
20         cin >> s >> t;
21
22         --s;
23         --t;
24
25         adj[s].push_back(t);
26     }
27
28     bool can = false;
29     vector<int> route;
30 }
```

3.7 BFS sobre Grafo Producto (dos fichas a la vez) - Infiltration Route (cont.)

```

31 queue<array<int, 3>> q;
32 vector<array<int, 3>> toclear;
33
34 /*
35  cross = unica arista que pasa de:
36  [0 ... n-1]
37  a:
38  [n ... 2n-1]
39  */
40
41 for (int cross = n + 1; cross < 2 * n && !can; cross++) {
42     q.push({0, cross - n, 0});
43     seen[0][cross - n][0] = true;
44     bck[0][cross - n][0] = {
45         -1, -1, 0
46     };
47     while (!q.empty()) {
48         auto [p1, p2, t] = q.front();
49         q.pop();
50         toclear.push_back({p1, p2, t});
51         // -----
52         // Mover p1
53         // -----
54         if (p1 < p2 && t == 0) {
55             for (auto v : adj[p1]) {
56                 if (v < n && v != p2) {
57                     if (!seen[v][p2][0]) {
58                         seen[v][p2][0] = true;
59                         bck[v][p2][0] = {
60                             p1, p2, t
61                         };
62                         q.push({v, p2, 0});
63                     }
64                 }
65             }
66             // -----
67             // Mover p2
68             // -----
69             if (t == 1 || (t == 0 && p1 < p2)) {
70                 for (auto v : adj[n + p2]) {
71                     int np2 = v - n;
72                     if (np2 != p1) {
73                         if (!seen[p1][np2][1]) {
74                             seen[p1][np2][1] = true;
75                             bck[p1][np2][1] = {
76                                 p1, p2, t

```

3.7 BFS sobre Grafo Producto (dos fichas a la vez) - Infiltration Route (cont.)

```

77         };
78         }
79     }
80     // -----
81     // Caso p1 > p2
82     // -----
83     else if (t == 0 && p1 > p2) {
84         for (auto v : adj[n + p2]) {
85             int np2 = v - n;
86             if (np2 != p1) {
87                 if (!seen[p1][np2][t]) {
88                     seen[p1][np2][t] = true;
89                     bck[p1][np2][t] = {
90                         p1, p2, t
91                     };
92                     q.push({p1, np2, t});
93                 }
94             }
95         }
96         // -----
97         // Buscamos:
98         //
99         // p1 = bc
100        // p2 = n-1
101        //
102        // y la arista bc -> cross.
103        // -----
104        for (int bc = 0; bc < n - 1 && !can; bc++) {
105            for (int t = 0; t <= 1 && !can; t++) {
106                if (!seen[bc][n - 1][t])
107                    continue;
108                for (auto v : adj[bc]) {
109                    if (v != cross)
110                        continue;
111                    can = true;
112                }
113                // -----
114                // Reconstruir
115                // -----
116                array<int, 3> cur = {
117                    bc,
118                    n - 1,
119                    t
120                };
121                while (cur[0] != -1) {
122                    route.push_back(cur[0]);
123                    route.push_back(n + cur[1]);

```

3.7 BFS sobre Grafo Producto (dos fichas a la vez) - Infiltration Route (cont.)

```

175
176         cur =
177             bck[cur[0]][cur[1]][cur[2]];
178     }
179
180     sort(route.begin(), route.end());
181
182     route.erase(
183         unique(route.begin(), route.end()),
184         route.end()
185     );
186
187     break;
188 }
189 }
190 }
191
192 // -----
193 // Limpiar solamente los estados visitados
194 // -----
195
196 for (auto [u, v, t] : toclear) {
197     seen[u][v][t] = false;
198
199     bck[u][v][t] = {
200         0, 0, 0
201     };
202 }
203
204 toclear.clear();
205 }
206
207 if (can) {
208     cout << route.size() << '\n';
209
210     for (int x : route)
211         cout << x + 1 << ' ';
212
213     cout << '\n';
214 } else {
215     cout << "**\n";
216 }
217
218 return 0;
219 }

```

3.8 Biconnected Components (Block-Cut Tree) + LCA - Hedge Mazes

O: (n + m) el block-cut tree, (log n) por consulta con LCA

Si hay exactamente un camino simple entre dos salas: block-cut tree mas LCA

C++/Training_Camp_2026/Day 4N - Hedge Mazes.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 22];
5 static int bufLen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0;
8     if (!bufLen_) return -1; }
9     return buf_[bufPos_++];
10 }
11 inline int readInt() {
12     int c = gc();
13     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
14     bool neg = false; if (c == '-') { neg = true; c = gc(); }
15     int x = 0;
16     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }

```

3.8 Biconnected Components (Block-Cut Tree) + LCA - Hedge Mazes (cont.)

```

6     return neg ? -x : x;
7 }
8
9 int R, C, Q;
10 vector<pair<int,int>> edges; // edge id -> (u,v)
11 vector<vector<pair<int,int>>> adj; // adj[u] -> list of (v, edgeId)
12 vector<int> disc_, low_;
13 vector<char> visited_, isCut;
14 int timer_;
15
16 vector<int> posOfVertex; // "position" node id in block-cut tree for each original vertex
17 x
18 // for a cut vertex v: position = v itself (1..R)
19 // for a non-cut vertex v: position = its unique block's node
20
21 id (R+1..R+numBlocks)
22 vector<int> blockWeight; // weight[nodeId] : block-size for block nodes, 0 for cut-vertex
23 x nodes (irrelevant)
24 vector<vector<int>> treeAdj; // adjacency list over node ids 1..(R+numBlocks)
25 vector<int> treeCompId; // which tree (of the block-cut forest) each node belongs to
26 int numBlocks;
27
28 vector<int> lastMarked; // lastMarked[v] = block index that last touched vertex v (avoids
29 duplicate adds)
30
31 void buildBiconnected() {
32     disc_.assign(R + 1, -1);
33     low_.assign(R + 1, -1);
34     visited_.assign(R + 1, 0);
35     isCut.assign(R + 1, 0);
36     posOfVertex.assign(R + 1, -1);
37     lastMarked.assign(R + 1, -1);
38     timer_ = 0;
39     numBlocks = 0;
40
41     vector<vector<int>> blockVerts; // blockVerts[b] = list of vertices in block b (0-indexed b)
42     vector<int> edgeStack;
43
44     for (int s = 1; s <= R; s++) {
45         if (visited_[s]) continue;
46         // iterative DFS
47         vector<array<int,4>> stk; // {u, nextIdx, parentEdge, parentVertex}
48         visited_[s] = 1;
49         disc_[s] = low_[s] = timer_++;
50         stk.push_back({s, 0, -1, -1});
51         int rootChildren = 0;
52
53         while (!stk.empty()) {
54             auto &fr = stk.back();
55             int u = fr[0];
56             if (fr[1] < (int)adj[u].size()) {
57                 int v = adj[u][fr[1]].first;
58                 int eid = adj[u][fr[1]].second;
59                 fr[1]++;
60                 if (eid == fr[2]) continue; // skip edge back to parent
61                 if (!visited_[v]) {
62                     visited_[v] = 1;
63                     disc_[v] = low_[v] = timer_++;
64                     edgeStack.push_back(eid);
65                     if (u == s) rootChildren++;
66                     stk.push_back({v, 0, eid, u});
67                 } else if (disc_[v] < disc_[u]) {
68                     edgeStack.push_back(eid);
69                     low_[u] = min(low_[u], disc_[v]);
70                 }
71             } else {
72                 int parentEdge = fr[2];
73                 int parentVertex = fr[3];
74                 int uLow = low_[u];
75                 stk.pop_back();
76                 if (!stk.empty()) {
77                     int p = parentVertex;

```

3.8 Biconnected Components (Block-Cut Tree) + LCA - Hedge Mazes (cont.)

```

83         low[p] = min(low[p], uLow);
84         if (uLow >= disc[p]) {
85             if (p != s) isCut[p] = 1;
86             // pop edges up to and including parentEdge -> forms one block
87             int b = numBlocks++;
88             blockVerts.push_back({});
89             while (!edgeStack.empty()) {
90                 int e = edgeStack.back();
91                 edgeStack.pop_back();
92                 int a1 = edges[e].first, a2 = edges[e].second;
93                 if (lastMarked[a1] != b) { lastMarked[a1] = b; blockVerts[b].p
ush_back(a1); }
94                 if (lastMarked[a2] != b) { lastMarked[a2] = b; blockVerts[b].p
ush_back(a2); }
95                 if (e == parentEdge) break;
96             }
97         }
98     }
99 }
100 }
101 if (rootChildren > 1) isCut[s] = 1;
102 }
103 // assign block weights and non-cut vertex positions
104 blockWeight.assign(1 + R + numBlocks, 0);
105 for (int b = 0; b < numBlocks; b++) {
106     int nodeId = R + 1 + b;
107     blockWeight[nodeId] = (int)blockVerts[b].size();
108     for (int v : blockVerts[b]) {
109         if (!isCut[v]) {
110             posOfVertex[v] = nodeId;
111         }
112     }
113 }
114 }
115 for (int v = 1; v <= R; v++) {
116     if (isCut[v]) posOfVertex[v] = v;
117     else if (posOfVertex[v] == -1) {
118         // isolated vertex (no edges at all) -> give it its own trivial "block" node c
onceptually.
119         // We reuse it as its own position id (acts like a size-1 pseudo node, weight
0, never matches
120         // any other vertex's position unless that vertex IS this same isolated vertex
, which can't
121         // happen for a distinct query since S != T is guaranteed... but S or T might
independently
122         // be isolated vertices in different queries).
123         posOfVertex[v] = v; // treat as its own node; weight will be 0 (irrelevant) an
d it will simply
124         // have no tree edges, forming its own singleton tree.
125     }
126 }
127 // Build tree adjacency
128 treeAdj.assign(1 + R + numBlocks, {});
129 for (int b = 0; b < numBlocks; b++) {
130     int nodeId = R + 1 + b;
131     for (int v : blockVerts[b]) {
132         if (isCut[v]) {
133             treeAdj[v].push_back(nodeId);
134             treeAdj[nodeId].push_back(v);
135         }
136     }
137 }
138 }
139 // LCA with binary lifting, tracking max blockWeight along the path (node weights only mat
ter for block nodes;
140 // cut-vertex nodes / isolated singleton nodes have weight 0 which never triggers the "bad
" condition).
141 int LOG_;
142 vector<vector<int>> up_;
143 vector<vector<int>> upMax_; // upMax_[k][v] = max weight among the 2^k nodes starting at v

```

3.8 Biconnected Components (Block-Cut Tree) + LCA - Hedge Mazes (cont.)

```

144     going upward (v itself
145     // included at k=0 as weight[v], combined appropriately)
146     vector<int> depth_;
147 void bfsBuildLCA(int totalNodes) {
148     LOG_ = 1;
149     while ((1 << LOG_) < totalNodes + 1) LOG_++;
150     up_.assign(LOG_, vector<int>(totalNodes + 1, 0));
151     upMax_.assign(LOG_, vector<int>(totalNodes + 1, 0));
152     depth_.assign(totalNodes + 1, -1);
153     treeCompId.assign(totalNodes + 1, -1);
154     int curComp = -1;
155     for (int start = 1; start <= totalNodes; start++) {
156         if (depth_[start] != -1) continue;
157         curComp++;
158         queue<int> bfsq;
159         depth_[start] = 0;
160         up_[0][start] = 0; // root's parent = 0 (sentinel "no parent")
161         upMax_[0][start] = blockWeight[start];
162         treeCompId[start] = curComp;
163         bfsq.push(start);
164         while (!bfsq.empty()) {
165             int u = bfsq.front(); bfsq.pop();
166             for (int w : treeAdj[u]) {
167                 if (depth_[w] == -1) {
168                     depth_[w] = depth_[u] + 1;
169                     up_[0][w] = u;
170                     upMax_[0][w] = max(blockWeight[w], blockWeight[u]);
171                     treeCompId[w] = curComp;
172                     bfsq.push(w);
173                 }
174             }
175         }
176     }
177 }
178 for (int k = 1; k < LOG_; k++) {
179     for (int v = 1; v <= totalNodes; v++) {
180         int mid = up_[k-1][v];
181         if (mid == 0) { up_[k][v] = 0; upMax_[k][v] = upMax_[k-1][v]; }
182         else {
183             up_[k][v] = up_[k-1][mid];
184             upMax_[k][v] = max(upMax_[k-1][v], upMax_[k-1][mid]);
185         }
186     }
187 }
188 // returns {lca, maxWeightOnPath} or {-1, -1} if different components
189 pair<int,int> queryPath(int u, int v) {
190     if (treeCompId[u] != treeCompId[v]) return {-1, -1};
191     int best = max(blockWeight[u], blockWeight[v]);
192     if (depth_[u] < depth_[v]) swap(u, v);
193     int diff = depth_[u] - depth_[v];
194     for (int k = 0; k < LOG_; k++) {
195         if ((diff >= k) & 1) {
196             best = max(best, upMax_[k][u]);
197             u = up_[k][u];
198         }
199     }
200     if (u == v) return {u, best};
201     for (int k = LOG_ - 1; k >= 0; k--) {
202         if (up_[k][u] != up_[k][v]) {
203             best = max({best, upMax_[k][u], upMax_[k][v]});
204             u = up_[k][u];
205             v = up_[k][v];
206         }
207     }
208     best = max({best, blockWeight[up_[0][u]], upMax_[0][u], upMax_[0][v]});
209     return {up_[0][u], best};
210 }
211 }

```

3.8 Biconnected Components (Block-Cut Tree) + LCA - Hedge Mazes (cont.)

```

271 int main() {
272     while (true) {
273         R = readInt(); C = readInt(); Q = readInt();
274         if (R == 0 && C == 0 && Q == 0) break;
275
276         edges.assign(C, {0,0});
277         adj.assign(R + 1, {});
278         for (int i = 0; i < C; i++) {
279             int a = readInt(), b = readInt();
280             edges[i] = {a, b};
281             adj[a].push_back({b, i});
282             adj[b].push_back({a, i});
283         }
284
285         buildBiconnected();
286         int totalNodes = R + numBlocks;
287         bfsBuildLCA(totalNodes);
288
289         string out;
290         for (int q = 0; q < Q; q++) {
291             int s = readInt(), t = readInt();
292             int ps = posOfVertex[s], pt = posOfVertex[t];
293             auto [lca, mx] = queryPath(ps, pt);
294             if (lca == -1) {
295                 out += "N\n";
296             } else {
297                 out += (mx <= 2) ? "Y\n" : "N\n";
298             }
299         }
300         out += "\n";
301         fwrite(out.data(), 1, out.size(), stdout);
302     }
303     return 0;
304 }

```

3.9 Bipartite Matching - Attacking Rooks

O: (V*E), emparejamiento bipartito con Kuhn

En un tablero NxN con algunas casillas ocupadas por peones ('X')

C++/Training_Camp_2026/Day 7/I - Attacking rooks.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct HopcroftKarp {
5     int n, m;
6     vector<vector<int>> adj;
7     vector<int> matchL, matchR, dist;
8     const int INF = INT_MAX;
9     HopcroftKarp(int n=0, int m=0): n(n), m(m), adj(n), matchL(n,-1), matchR(m,-1), dist(n) {}
10    void addEdge(int u,int v){ adj[u].push_back(v); }
11    bool bfs(){
12        queue<int> q;
13        for(int u=0;u<n;u++){
14            if(matchL[u]==-1){ dist[u]=0; q.push(u); }
15            else dist[u]=INF;
16        }
17        bool found=false;
18        while(!q.empty()){
19            int u=q.front();q.pop();
20            for(int v: adj[u]){
21                int w = matchR[v];
22                if(w!=-1){ found=true; }
23                else if(dist[w]==INF){ dist[w]=dist[u]+1; q.push(w); }
24            }
25        }
26        return found;
27    }
28    bool dfs(int u){
29        for(int v: adj[u]){
30            int w = matchR[v];

```

3.9 Bipartite Matching - Attacking Rooks (cont.)

```

31         if(w==-1 || (dist[w]==dist[u]+1 && dfs(w))){
32             matchL[u]=v; matchR[v]=u;
33             return true;
34         }
35     }
36     dist[u]=INF;
37     return false;
38 }
39 int maxMatching(){
40     int result=0;
41     while(bfs()){
42         for(int u=0;u<n;u++){
43             if(matchL[u]==-1){
44                 if(dfs(u)) result++;
45             }
46         }
47     }
48     return result;
49 }
50 };
51
52 int main(){
53     ios::sync_with_stdio(false);
54     cin.tie(nullptr);
55     int N;
56     while(cin >> N){
57         vector<string> grid(N);
58         for(auto &s: grid) cin >> s;
59
60         vector<vector<int>> rowId(N, vector<int>(N,-1));
61         vector<vector<int>> colId(N, vector<int>(N,-1));
62         int R=0, C=0;
63
64         // etiqueta cada tramo horizontal de casillas vacías
65         for(int i=0;i<N;i++){
66             int j=0;
67             while(j<N){
68                 if(grid[i][j]=='X'){ j++; continue; }
69                 while(j<N && grid[i][j]=='.'){ rowId[i][j]=R; j++; }
70                 R++;
71             }
72         }
73         // etiqueta cada tramo vertical de casillas vacías
74         for(int j=0;j<N;j++){
75             int i=0;
76             while(i<N){
77                 if(grid[i][j]=='X'){ i++; continue; }
78                 while(i<N && grid[i][j]=='.'){ colId[i][j]=C; i++; }
79                 C++;
80             }
81         }
82
83         HopcroftKarp hk(R,C);
84         for(int i=0;i<N;i++)
85             for(int j=0;j<N;j++)
86                 if(grid[i][j]=='.')
87                     hk.addEdge(rowId[i][j], colId[i][j]);
88
89         cout << hk.maxMatching() << "\n";
90     }
91     return 0;
92 }

```

3.10 Bipartite Matching - Knights In The Board

O: (V*E), emparejamiento bipartito sobre el tablero bicoloreado

Dado un tablero NxN con K caballos ya colocados

C++/Training_Camp_2026/Day 7/K - Knights In The Board.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3

```

3.10 Bipartite Matching - Knights In The Board (cont.)

```

4 int N, K;
5 map<pair<int,int>, int> pos2idx;
6 vector<int> R, C, color_;
7 vector<vector<int>> adj;
8 vector<int> matchRight;
9 vector<bool> visited;
10
11 int dr[8] = {1,1,-1,-1,2,2,-2,-2};
12 int dc[8] = {2,-2,2,-2,1,-1,1,-1};
13
14 bool tryKuhn(int u) {
15     for (int v : adj[u]) {
16         if (!visited[v]) {
17             visited[v] = true;
18             if (matchRight[v] == -1 || tryKuhn(matchRight[v])) {
19                 matchRight[v] = u;
20                 return true;
21             }
22         }
23     }
24     return false;
25 }
26
27 int main(){
28     cin >> N >> K;
29     R.resize(K); C.resize(K); color_.resize(K);
30     for (int i = 0; i < K; i++) {
31         cin >> R[i] >> C[i];
32         pos2idx[{R[i], C[i]}] = i;
33         color_[i] = (R[i] + C[i]) % 2;
34     }
35     adj.assign(K, {});
36     for (int i = 0; i < K; i++) {
37         if (color_[i] != 0) continue;
38         for (int d = 0; d < 8; d++) {
39             auto it = pos2idx.find({R[i]+dr[d], C[i]+dc[d]});
40             if (it != pos2idx.end()) adj[i].push_back(it->second);
41         }
42     }
43     matchRight.assign(K, -1);
44     int matching = 0;
45     for (int i = 0; i < K; i++) {
46         if (color_[i] != 0 || adj[i].empty()) continue;
47         visited.assign(K, false);
48         if (tryKuhn(i)) matching++;
49     }
50     cout << matching << "\n";
51 }

```

3.11 Bipartite Matching (Algoritmo de Kuhn) - School Dance

O: (V*E), emparejamiento bipartito con Kuhn

Hay N chicos y M chicas, y una lista de K parejas potenciales (chico, chica) dispuestos a bailar juntos

C++/Training_Camp_2026/Day 5/G - School Dance.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n, m, k;
5 vector<vector<int>> pairs;
6 vector<int> matchGirl;
7 vector<bool> visited;
8
9 bool tryMatch(int boy) {
10     if (visited[boy]) return false;
11     visited[boy] = true;
12
13     for (int girl : pairs[boy]) {
14         if (matchGirl[girl] == 0 || tryMatch(matchGirl[girl])) {
15             matchGirl[girl] = boy;
16             return true;
17         }
18     }
19 }

```

3.11 Bipartite Matching (Algoritmo de Kuhn) - School Dance (cont.)

```

8 }
9     return false;
10 }
11
12 int main() {
13     ios::sync_with_stdio(false);
14     cin.tie(nullptr);
15
16     cin >> n >> m >> k;
17
18     pairs.assign(n + 1, {});
19
20     for (int i = 0; i < k; i++) {
21         int boy, girl;
22         cin >> boy >> girl;
23         pairs[boy].push_back(girl);
24     }
25
26     matchGirl.assign(m + 1, 0);
27
28     int maximumPairs = 0;
29
30     for (int boy = 1; boy <= n; boy++) {
31         visited.assign(n + 1, false);
32         if (tryMatch(boy)) {
33             maximumPairs++;
34         }
35     }
36
37     cout << maximumPairs << endl;
38
39     for (int girl = 1; girl <= m; girl++) {
40         if (matchGirl[girl] != 0) {
41             cout << matchGirl[girl] << " " << girl << endl;
42         }
43     }
44
45     return 0;
46 }

```

3.12 Circuito Euleriano (Algoritmo de Hierholzer) - Mail Delivery

O: (n + m), circuito euleriano con Hierholzer

Dado un plano de n cruces y m calles bidireccionales

C++/Training_Camp_2026/Day 5/J - Mail Delivery.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     ios_base::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n, m;
9     cin >> n >> m;
10
11     vector<vector<pair<int,int>>> adj(n + 1); // (to, edge_id)
12     vector<int> deg(n + 1, 0);
13
14     for(int i = 0; i < m; i++){
15         int a, b;
16         cin >> a >> b;
17         adj[a].push_back({b, i});
18         adj[b].push_back({a, i});
19         deg[a]++;
20         deg[b]++;
21     }
22
23     // 1) every vertex must have even degree
24     for(int v = 1; v <= n; v++){
25         if(deg[v] % 2 != 0){
26             cout << "IMPOSSIBLE\n";
27         }
28     }
29 }

```

3.12 Circuito Euleriano (Algoritmo de Hierholzer) - Mail Delivery (cont.)

```

27     return 0;
28 }
29 }
30
31 // 2) all vertices that have edges must be reachable from vertex 1
32 // (this also catches the case where vertex 1 itself has no edges but m > 0)
33 vector<bool> visited(n + 1, false);
34 queue<int> bfsQ;
35 bfsQ.push(1);
36 visited[1] = true;
37 int reached = 0;
38 while(!bfsQ.empty()){
39     int u = bfsQ.front(); bfsQ.pop();
40     reached++;
41     for(auto &pr : adj[u]){
42         int v = pr.first;
43         if(!visited[v]){
44             visited[v] = true;
45             bfsQ.push(v);
46         }
47     }
48 }
49
50 int totalWithEdges = 0;
51 for(int v = 1; v <= n; v++){
52     if(deg[v] > 0) totalWithEdges++;
53 }
54
55 if(reached < totalWithEdges){
56     cout << "IMPOSSIBLE\n";
57     return 0;
58 }
59
60 // 3) Hierholzer's algorithm (iterative) to build the Eulerian circuit
61 vector<int> ptr(n + 1, 0);
62 vector<bool> usedEdge(m, false);
63 vector<int> circuit;
64 circuit.reserve(m + 1);
65
66 vector<int> st;
67 st.reserve(m + 1);
68 st.push_back(1);
69
70 while(!st.empty()){
71     int v = st.back();
72     if(ptr[v] < (int)adj[v].size()){
73         auto [to, eid] = adj[v][ptr[v]];
74         ptr[v]++;
75         if(usedEdge[eid]) continue;
76         usedEdge[eid] = true;
77         st.push_back(to);
78     } else {
79         circuit.push_back(v);
80         st.pop_back();
81     }
82 }
83
84 reverse(circuit.begin(), circuit.end());
85
86 // circuit must use exactly m edges -> m+1 vertices, and start/end at 1
87 if((int)circuit.size() != m + 1 || circuit.front() != 1 || circuit.back() != 1){
88     cout << "IMPOSSIBLE\n";
89     return 0;
90 }
91
92 for(int i = 0; i < (int)circuit.size(); i++){
93     cout << circuit[i];
94     if(i + 1 < (int)circuit.size()) cout << ' ';
95 }
96 cout << "\n";
97
98 return 0;

```

3.12 Circuito Euleriano (Algoritmo de Hierholzer) - Mail Delivery (cont.)

```

99 }

```

3.13 Clique Maximo (Bron-Kerbosch con Pivote) - Holy Network

O: $(3^{n/3})$ en el peor caso, Bron-Kerbosch con pivote

Dados N numeros, hallar el grupo mas grande en el que CADA PAR comparta un factor mayor que 1

C++/ICPC/2025/H - Holy Network.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef unsigned long long ull;
5
6 int n;
7 ull adj[64];
8 int mejor;
9
10 // Bron-Kerbosch con pivote sobre mascar s de 64 bits.
11 // R = clique que llevamos, P = candidatos, X = ya descartados
12 void bron(ull R, ull P, ull X) {
13     if (P == 0 && X == 0) {
14         mejor = max(mejor, __builtin_popcountll(R));
15         return;
16     }
17
18     // Cota: ni tomando todos los candidatos se mejora lo que ya hay.
19     if (__builtin_popcountll(R) + __builtin_popcountll(P) <= mejor)
20         return;
21
22     // Pivote: el de P|X con mas vecinos en P. Solo hace falta ramificar por
23     // los candidatos que NO son vecinos suyos.
24     ull PX = P | X;
25     int piv = __builtin_ctzll(PX);
26     int mejorGrado = -1;
27
28     for (ull t = PX; t; t &= t - 1) {
29         int u = __builtin_ctzll(t);
30         int g = __builtin_popcountll(adj[u] & P);
31         if (g > mejorGrado) {
32             mejorGrado = g;
33             piv = u;
34         }
35     }
36
37     for (ull t = P & ~adj[piv]; t; t &= t - 1) {
38         int v = __builtin_ctzll(t);
39         ull bit = 1ULL << v;
40
41         bron(R | bit, P & adj[v], X & adj[v]);
42
43         P &= ~bit;
44         X |= bit;
45     }
46 }
47
48 int main() {
49     ios::sync_with_stdio(false);
50     cin.tie(nullptr);
51
52     while (cin >> n && n != 0) {
53
54         vector<long long> a(n);
55
56         for (auto &x : a)
57             cin >> x;
58
59         for (int i = 0; i < n; i++)
60             adj[i] = 0;
61
62         // Dos ingredientes son compatibles si comparten un factor mayor que 1.
63         for (int i = 0; i < n; i++)
64             for (int j = i + 1; j < n; j++)

```

3.13 Clique Maximo (Bron-Kerbosch con Pivote) - Holy Network (cont.)

```

85         if (__gcd(a[i], a[j]) > 1) {
86             adj[i] |= 1ULL << j;
87             adj[j] |= 1ULL << i;
88         }
89
90         // Un grupo de UN solo ingrediente siempre es armonico: no hay pares
91         // que revisar. Por eso la respuesta nunca baja de 1.
92         mejor = 1;
93
94         ull todos = (n == 64) ? ~0ULL : ((1ULL << n) - 1);
95         bron(0, todos, 0);
96
97         cout << mejor << '\n';
98     }
99
100     return 0;
101 }

```

3.14 Clique Maximo con Backtracking Iterativo - Holy Network (clique Iterativo)

O: exponencial en el peor caso, acotada por las podas

El grupo mas grande donde CADA PAR comparte un factor mayor que 1, o sea un clique maximo

C++/ICPC/2025/H - Holy Network (clique iterativo).cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 #include <stack>
5
6 using namespace std;
7
8 struct Estado
9 {
10     vector<long long> candidatos;
11     long long tamGrupo;
12 };
13
14 int main()
15 {
16     ios::sync_with_stdio(false);
17     cin.tie(nullptr);
18
19     long long n;
20
21     while(cin >> n && n != 0)
22     {
23         vector<long long> resonancia(n);
24
25         for(long long i = 0; i < n; i++)
26         {
27             cin >> resonancia[i];
28         }
29
30         vector<vector<bool>> compatibles(n, vector<bool>(n, false));
31
32         for(long long a = 0; a < n; a++)
33         {
34             for(long long b = a + 1; b < n; b++)
35             {
36                 long long x = resonancia[a];
37                 long long y = resonancia[b];
38
39                 while(y != 0)
40                 {
41                     long long resto = x % y;
42                     x = y;
43                     y = resto;
44                 }
45
46                 if(x > 1)
47                 {
48                     compatibles[a][b] = true;

```

3.14 Clique Maximo con Backtracking Iterativo - Holy Network (clique Iterativo) (cont.)

```

48             compatibles[b][a] = true;
49         }
50     }
51 }
52
53 vector<long long> todos(n);
54
55 for(long long i = 0; i < n; i++)
56 {
57     todos[i] = i;
58 }
59
60 stack<Estado> pila;
61 pila.push({todos, 0});
62
63 long long mejorGrupo = 0;
64
65 while(!pila.empty())
66 {
67     Estado actual = pila.top();
68     pila.pop();
69
70     mejorGrupo = max(mejorGrupo, actual.tamGrupo);
71
72     for(long long i = 0; i < (long long)actual.candidatos.size(); i++)
73     {
74         long long quedan = actual.candidatos.size() - i;
75
76         if(actual.tamGrupo + quedan <= mejorGrupo)
77         {
78             break;
79         }
80
81         long long elegido = actual.candidatos[i];
82
83         vector<long long> nuevosCandidatos;
84
85         for(long long j = i + 1; j < (long long)actual.candidatos.size(); j++)
86         {
87             if(compatibles[elegido][actual.candidatos[j]])
88             {
89                 nuevosCandidatos.push_back(actual.candidatos[j]);
90             }
91         }
92
93         pila.push({
94             nuevosCandidatos,
95             actual.tamGrupo + 1
96         });
97     }
98 }
99
100 cout << mejorGrupo << '\n';
101 }
102
103 return 0;
104 }

```

3.15 Componentes Fuente con Kosaraju - Lumina Juan

O: (n^2) por armar el grafo; el Kosaraju es $(n+m)$

El minimo de gemas a encender a mano para que la reaccion en cadena las encienda todas

C++/ICPC/2024/lumina Juan.cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <stack>
5
6 using namespace std;
7
8 struct Circulo

```

3.15 Componentes Fuente con Kosaraju - Lumina Juan (cont.)

```

9 {
10     long long x;
11     long long y;
12     long long r;
13 };
14
15 int main()
16 {
17     ios::sync_with_stdio(false);
18     cin.tie(nullptr);
19
20     long long n;
21
22     while(cin >> n && n != 0)
23     {
24         vector<Circulo> circulos(n);
25
26         for(long long i = 0; i < n; i++)
27         {
28             cin >> circulos[i].x >> circulos[i].y >> circulos[i].r;
29         }
30
31         vector<vector<int>> conexion(n);
32
33         for(long long a = 0; a < n; a++)
34         {
35             for(long long b = 0; b < n; b++)
36             {
37                 if(a == b)
38                 {
39                     continue;
40                 }
41
42                 long long dx = circulos[a].x - circulos[b].x;
43                 long long dy = circulos[a].y - circulos[b].y;
44
45                 long long distancia = dx * dx + dy * dy;
46
47                 if(distancia <= circulos[a].r * circulos[a].r)
48                 {
49                     conexion[a].push_back(b);
50                 }
51             }
52         }
53
54         vector<bool> visitado(n, false);
55         vector<long long> ordenFinal;
56
57         for(long long i = 0; i < n; i++)
58         {
59             if(visitado[i])
60             {
61                 continue;
62             }
63
64             stack<pair<long long, long long>> pila;
65             visitado[i] = true;
66             pila.push({i, 0});
67
68             while(!pila.empty())
69             {
70                 long long actual = pila.top().first;
71                 long long posicion = pila.top().second;
72
73                 if(posicion == (long long)conexion[actual].size())
74                 {
75                     ordenFinal.push_back(actual);
76                     pila.pop();
77                     continue;
78                 }
79
80                 long long siguiente = conexion[actual][posicion];

```

3.15 Componentes Fuente con Kosaraju - Lumina Juan (cont.)

```

81         pila.top().second++;
82
83         if(!visitado[siguiente])
84         {
85             visitado[siguiente] = true;
86             pila.push({siguiente, 0});
87         }
88     }
89
90     vector<bool> encontrado(n, false);
91
92     long long contador = 0;
93
94     for(long long k = n - 1; k >= 0; k--)
95     {
96         long long inicio = ordenFinal[k];
97
98         if(encontrado[inicio])
99         {
100             continue;
101         }
102
103         queue<long long> cola;
104         cola.push(inicio);
105         encontrado[inicio] = true;
106         contador++;
107
108         while(!cola.empty())
109         {
110             long long actual = cola.front();
111             cola.pop();
112
113             for(long long siguiente : conexion[actual])
114             {
115                 if(!encontrado[siguiente])
116                 {
117                     encontrado[siguiente] = true;
118                     cola.push(siguiente);
119                 }
120             }
121         }
122     }
123
124     cout << contador << '\n';
125 }
126
127 return 0;
128 }

```

3.16 Componentes Fuente de la Condensacion (Tarjan) - Lumina

O: (n^2) por armar el grafo de alcance; el Tarjan es (n+m)

Minimo de gemas a encender a mano: componentes fuente de la condensacion de SCC

C++/ICPC/2024/J - Lumina.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Gem {
5     long long x, y, r;
6 };
7
8 int n;
9 vector<Gem> gems;
10
11 vector<int> disc, low, st, component;
12 vector<bool> inStack;
13
14 int timer;
15 int sccCount;
16

```

3.16 Componentes Fuente de la Condensacion (Tarjan) - Lumina (cont.)

```

1 void tarjan(int u) {
2
3     disc[u] = low[u] = timer++;
4
5     st.push_back(u);
6     inStack[u] = true;
7
8     for (int v = 0; v < n; v++) {
9
10        if (u == v)
11            continue;
12
13        long long dx = gems[u].x - gems[v].x;
14        long long dy = gems[u].y - gems[v].y;
15
16        // u puede iluminar v
17        if (dx * dx + dy * dy > gems[u].r * gems[u].r)
18            continue;
19
20        if (disc[v] == -1) {
21
22            tarjan(v);
23
24            low[u] = min(low[u], low[v]);
25
26        }
27        else if (inStack[v]) {
28
29            low[u] = min(low[u], disc[v]);
30
31        }
32    }
33
34    if (low[u] == disc[u]) {
35
36        while (true) {
37
38            int v = st.back();
39            st.pop_back();
40
41            inStack[v] = false;
42
43            component[v] = sccCount;
44
45            if (v == u)
46                break;
47
48        }
49
50        sccCount++;
51    }
52 }
53
54 int main() {
55
56     ios::sync_with_stdio(false);
57     cin.tie(nullptr);
58
59     while (cin >> n && n != 0) {
60
61         gems.resize(n);
62
63         for (auto &gem : gems)
64             cin >> gem.x >> gem.y >> gem.r;
65
66         disc.assign(n, -1);
67         low.assign(n, 0);
68         component.assign(n, -1);
69         inStack.assign(n, false);
70
71         st.clear();
72
73         timer = 0;
74         sccCount = 0;

```

3.16 Componentes Fuente de la Condensacion (Tarjan) - Lumina (cont.)

```

88
89     // Encontrar SCC
90     for (int i = 0; i < n; i++) {
91         if (disc[i] == -1)
92             tarjan(i);
93     }
94
95     // inDegree de cada SCC
96     vector<bool> hasIncoming(sccCount, false);
97
98     // Revisar todas las aristas
99     for (int u = 0; u < n; u++) {
100
101         for (int v = 0; v < n; v++) {
102
103             if (u == v)
104                 continue;
105
106             long long dx = gems[u].x - gems[v].x;
107             long long dy = gems[u].y - gems[v].y;
108
109             // u -> v
110             if (dx * dx + dy * dy <= gems[u].r * gems[u].r) {
111
112                 if (component[u] != component[v]) {
113                     hasIncoming[component[v]] = true;
114                 }
115             }
116         }
117     }
118
119     int answer = 0;
120
121     for (int c = 0; c < sccCount; c++) {
122         if (!hasIncoming[c])
123             answer++;
124     }
125
126     cout << answer << '\n';
127 }
128
129 return 0;
130 }

```

3.17 Componentes Fuertemente Conexas (Kosaraju) - Planets And Kingdoms

O: (n + m), SCC con Kosaraju

Dados n planetas y m teleportadores dirigidos

C++/Training_Camp_2026/Day 5/I - Planets and Kingdoms.py

```

1 import sys
2 sys.setrecursionlimit(300000)
3
4 n, m = map(int, input().split())
5
6 matrix = [[] for _ in range(n)]
7 matrix2 = [[] for _ in range(n)]
8
9 for _ in range(m):
10     p, s = map(int, input().split())
11     p -= 1
12     s -= 1
13
14     matrix[p].append(s)
15     matrix2[s].append(p)
16
17
18 visited = [False] * n
19 cam = []
20
21 def dfs(start):
22     stack = [(start, 0)]

```

3.17 Componentes Fuertemente Conexas (Kosaraju) - Planets And Kingdoms (cont.)

```

23 visited[start] = True
24
25 while stack:
26     nodo, estado = stack[-1]
27
28     if estado < len(matrix[nodo]):
29         vecino = matrix[nodo][estado]
30         stack[-1] = (nodo, estado + 1)
31
32         if not visited[vecino]:
33             visited[vecino] = True
34             stack.append((vecino, 0))
35
36     else:
37         cam.append(nodo)
38         stack.pop()
39
40
41 for i in range(n):
42     if not visited[i]:
43         dfs(i)
44
45
46 visited = [False] * n
47 respuesta = [0] * n
48
49 def dfs_gay(start, reino):
50     stack = [start]
51     visited[start] = True
52
53     while stack:
54         nodo = stack.pop()
55         respuesta[nodo] = reino
56
57         for vecino in matrix2[nodo]:
58             if not visited[vecino]:
59                 visited[vecino] = True
60                 stack.append(vecino)
61
62
63 voy = 0
64
65 for i in reversed(cam):
66     if not visited[i]:
67         voy += 1
68         dfs_gay(i, voy)
69
70
71 print(voy)
72 print(*respuesta)

```

3.18 DFS - Artwork

O: (n²) por comparar todos los pares de sensores

Cruzar una sala esquivando sensores: los que se tocan bloquean el paso, con DFS

C++/Training_Camp_2026/Day 1/A - Artwork.py

```

1 m, n, s = list(map(int, input().split()))
2 sensors = []
3
4 for i in range(s):
5     sensors.append(list(map(int, input().split())))
6
7 ans = "S"
8 visited = set()
9 def rec(x,y,r, start):
10     global ans, m, n
11
12     if ans == "N":
13         return
14     for xi,yi,ri in sensors:
15         if (xi, yi, ri) in visited:
16             continue

```

3.18 DFS - Artwork (cont.)

```

6     dist = ((x-xi)**2+(y-yi)**2)**(1/2)
7     if dist <= (r+ri):
8         if (start == "l" and (xi+ri >= m or yi-ri <= 0)) or (start == "d" and (xi-ri <
9             = 0 or yi+ri >= n)) or (start == "u" and (xi-ri >= m or yi-ri <= 0)) or (start == "r"
10                and (yi+ri >= n or x-ri <= 0)):
11             ans = "N"
12             break
13         visited.add((xi,yi,ri))
14         rec(xi,yi, ri, start)
15
16
17 for i, sensor in enumerate(sensors):
18     r = sensor[2]
19     x = sensor[0]
20     y = sensor[1]
21     if (x + r >= m and x - r <= 0) or (y + r >= n and y - r <= 0) or (x - r <= 0 and y - r
22         <= 0) or (x + r >= m and y + r >= n):
23         ans = "N"
24         break
25     if x-r<=0 or y-r<=0 or y+r>=n or x+r>=m:
26         dir = ""
27         if x-r<=0:
28             dir = "l"
29         elif y-r<=0:
30             dir = "d"
31         elif y+r>=n:
32             dir = "u"
33         elif x+r>=m:
34             dir = "r"
35         if dir != "":
36             visited = set()
37             rec(x,y,r, dir)
38
39 print(ans)

```

3.19 DFS - Denouncing Mafia

O: (n), DFS sobre el grafo funcional

Interrogar a K mafiosos de un arbol para denunciar al maximo, subiendo al jefe

C++/Training_Camp_2026/Day 1/D - Denouncing_Mafia.py

```

1 import copy
2 import sys
3
4 sys.setrecursionlimit(10**6)
5 m, k = list(map(int, input().split()))
6 entry = list(map(int, input().split()))
7
8 currMemb = 1
9 dicc = {1: []}
10 for node in entry:
11     currMemb += 1
12     if node in dicc:
13         dicc[node] = dicc[node] + [currMemb]
14     else:
15         dicc[node] = [currMemb]
16
17 visited = set()
18 ans = []
19
20 def rec(curr, depth, prev):
21     global visited, ans
22     if curr in dicc:
23         children = dicc[curr]
24         first = True
25         leafsDepth = []
26         for son in children:
27             newDep = rec(son, depth + 1, curr)
28             leafsDepth.append(newDep)
29         leafsDepth.sort(reverse=True)
30         max = leafsDepth.pop(0)
31         for i in leafsDepth:
32             ans.append(i - depth)

```

3.19 DFS - Denouncing Mafia (cont.)

```

33     return max
34 else:
35     return depth
36 ans.append(rec(1, 1, None))
37 ans.sort(reverse=True)
38 sum = 0
39
40 curr = 0
41 while(curr < k and curr < len(ans)):
42     sum+=ans[curr]
43     curr+=1
44
45 print(sum)

```

3.20 DFS - A Mazing!

O: (n*m), DFS en la grilla

Un problema interactivo donde hay que explorar un laberinto de celdas cuadradas moviendose...

C++/Training_Camp_2026/Day 11 - CCPL/L - A Mazing!.py

```

1 import sys
2 from collections import deque
3
4 dirs = {
5     "up": (0, 1),
6     "down": (0, -1),
7     "left": (-1, 0),
8     "right": (1, 0),
9 }
10
11 opposite = {
12     "up": "down",
13     "down": "up",
14     "left": "right",
15     "right": "left",
16 }
17
18 curr = (0, 0)
19
20 todo = {
21     curr: deque(["down", "right", "left", "up"])
22 }
23
24 parent = {}
25
26 while True:
27
28     if todo[curr]:
29         move = todo[curr].popleft()
30
31         print(move, flush=True)
32         ans = input()
33
34         if ans == "solved" or ans == "wrong":
35             break
36
37         if ans == "wall":
38             continue
39
40         dx, dy = dirs[move]
41         nxt = (curr[0] + dx, curr[1] + dy)
42
43         if nxt not in todo:
44             todo[nxt] = deque(["down", "right", "left", "up"])
45             # Ya usamos la dirección opuesta para llegar aquí
46             todo[nxt].remove(opposite[move])
47             parent[nxt] = curr
48
49         curr = nxt
50
51     else:
52

```

3.20 DFS - A Mazing! (cont.)

```

53     if curr == (0, 0):
54         print("no way out", flush=True)
55         ans = input()
56         break
57
58     par = parent[curr]
59
60     dx = par[0] - curr[0]
61     dy = par[1] - curr[1]
62
63     if dx == 1:
64         move = "right"
65     elif dx == -1:
66         move = "left"
67     elif dy == 1:
68         move = "up"
69     else:
70         move = "down"
71
72     print(move, flush=True)
73     ans = input()
74
75     if ans == "solved" or ans == "wrong":
76         break
77
78     curr = par

```

3.21 Dijkstra - Fantastic Robot

O: (m log n), Dijkstra

Llegar en una matriz ejecutando ordenes completas, no pasos sueltos: Dijkstra

C++/Training_Camp_2026/Day 9/E - Fantastic Robot.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const ll INF = 1e18;
7
8
9 int n,m;
10
11 vector<vector<int>> mat;
12 vector<string> comandos;
13
14
15 vector<tuple<int,int,int>> generar_vecinos(int x, int y){
16
17     vector<tuple<int,int,int>> vecinos;
18
19
20     for(string &cmd : comandos){
21
22         int xi=x;
23         int yi=y;
24         int pasos=0;
25
26
27         for(char c: cmd){
28
29             if(c=='U'){
30                 pasos++;
31                 xi--;
32
33                 if(xi<0 || mat[xi][yi]==1){
34                     pasos--;
35                     xi++;
36                 }
37
38             }else if(c=='D'){
39

```

3.21 Dijkstra - Fantastic Robot (cont.)

```

40     pasos++;
41     xi++;
42
43     if(xi>=n || mat[xi][yi]==1){
44         pasos--;
45         xi--;
46     }
47
48 }else if(c=='L'){
49
50     pasos++;
51     yi--;
52
53     if(yi<0 || mat[xi][yi]==1){
54         pasos--;
55         yi++;
56     }
57
58 }else{
59
60     pasos++;
61     yi++;
62
63     if(yi>=m || mat[xi][yi]==1){
64         pasos--;
65         yi--;
66     }
67
68 }
69
70 vecinos.push_back({xi,yi,pasos});
71 }
72
73 return vecinos;
74 }
75
76 }
77
78
79
80 int main(){
81
82     ios::sync_with_stdio(false);
83     cin.tie(nullptr);
84
85
86     cin>>n>>m;
87
88
89     mat.assign(n,vector<int>(m));
90
91
92     for(int i=0;i<n;i++){
93
94         string s;
95         cin>>s;
96
97         for(int j=0;j<m;j++){
98             mat[i][j]=s[j]-'0';
99         }
100
101
102
103     int sx,sy;
104     cin>>sx>>sy;
105
106
107
108     int tx,ty;
109     cin>>tx>>ty;
110
111
112     int k;

```

3.21 Dijkstra - Fantastic Robot (cont.)

```

112     cin>>k;
113
114
115     comandos.resize(k);
116
117     for(auto &x:comandos)
118         cin>>x;
119
120
121
122     vector<vector<ll>> dist(n, vector<ll>(m,INF));
123
124
125     priority_queue<
126         tuple<ll,int,int>,
127         vector<tuple<ll,int,int>>,
128         greater<tuple<ll,int,int>>
129     > pq;
130
131
132     dist[sx][sy]=0;
133
134     pq.push({0,sx,sy});
135
136
137     while(!pq.empty()){
138
139
140         auto [d,x,y]=pq.top();
141         pq.pop();
142
143
144         if(d!=dist[x][y])
145             continue;
146
147
148         if(x==tx && y==ty)
149             break;
150
151
152
153         for(auto [nx,ny,costo] : generar_vecinos(x,y)){
154
155
156             if(dist[nx][ny] > d+costo){
157
158                 dist[nx][ny]=d+costo;
159
160                 pq.push({
161                     dist[nx][ny],
162                     nx,
163                     ny
164                 });
165             }
166
167         }
168
169
170
171     if(dist[tx][ty]==INF)
172         cout<<-1<<"\n";
173     else
174         cout<<dist[tx][ty]<<"\n";
175
176 }

```

3.22 Dijkstra con Nodos Virtuales - Chinese Innovation

O: $(m \log n)$, Dijkstra con nodos virtuales por grupo

Un imperio de n ciudades con m caminos bidireccionales con costo, mas k tipos de teletransportadores

C++/Training_Camp_2026/Day 10/C - Chinese Innovation.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using pii = pair<long long, int>;
5
6 const long long INF = 1e18;
7
8 long long dijkstra(unordered_map<int, vector<pair<int, int>>> &graph, int start, int target) {
9     unordered_map<int, long long> dist;
10    unordered_set<int> visited;
11
12    priority_queue<pii, vector<pii>, greater<pii>> pq;
13
14    dist[start] = 0;
15    pq.push({0, start});
16
17    while (!pq.empty()) {
18        auto [d, u] = pq.top();
19        pq.pop();
20
21        if (visited.count(u))
22            continue;
23        visited.insert(u);
24
25        if (u == target)
26            return d;
27
28        for (auto [v, w] : graph[u]) {
29            long long nd = d + w;
30
31            if (!dist.count(v) || nd < dist[v]) {
32                dist[v] = nd;
33                pq.push({nd, v});
34            }
35        }
36    }
37
38    if (dist.count(target))
39        return dist[target];
40    return INF;
41 }
42
43 int main() {
44     ios::sync_with_stdio(false);
45     cin.tie(nullptr);
46
47     int n, m, k;
48     cin >> n >> m >> k;
49
50     unordered_map<int, vector<pair<int, int>>> graph;
51
52     for (int i = 0; i < m; i++) {
53         int a, b, w;
54         cin >> a >> b >> w;
55         graph[a].push_back({b, w});
56         graph[b].push_back({a, w});
57     }
58
59     for (int i = 1; i <= n; i++) {
60         int tel;
61         cin >> tel;
62
63         for (int j = 0; j < tel; j++) {
64             int t, w;
65             cin >> t >> w;
```

3.22 Dijkstra con Nodos Virtuales - Chinese Innovation (cont.)

```
67
68     int virtual_node = n + t;
69
70     graph[i].push_back({virtual_node, w});
71     graph[virtual_node].push_back({i, 0});
72 }
73
74
75 long long costo = dijkstra(graph, 1, n);
76 cout << costo << "\n";
77
78 return 0;
79 }
```

3.23 Dijkstra con Reconstruccion de Camino

O: $((n + m) \log m)$

Uso: $g[u].push_back(\{v, w\})$; $auto d = dijkstra(g, s, padre)$; $camino(padre, t)$

Distancias minimas desde un origen con pesos ≥ 0 , y el camino hasta cualquier nodo

C++/Notebook/Dijkstra.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5
6 const ll INF = 4e18;
7
8 vector<ll> dijkstra(const vector<vector<pair<int, ll>>> &g, int s, vector<int> &padre)
9 {
10     int n = g.size();
11     vector<ll> d(n, INF);
12     padre.assign(n, -1);
13     priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
14     d[s] = 0;
15     pq.push({0, s});
16     while (!pq.empty())
17     {
18         auto [du, u] = pq.top();
19         pq.pop();
20         if (du != d[u]) continue; // entrada vieja del heap
21         for (auto [v, w] : g[u])
22         {
23             if (d[u] + w < d[v])
24             {
25                 d[v] = d[u] + w;
26                 padre[v] = u;
27                 pq.push({d[v], v});
28             }
29         }
30     }
31     return d;
32 }
33
34 // Camino s -> t como lista de nodos (vacio si t es inalcanzable y t != s)
35 vector<int> camino(const vector<int> &padre, int t)
36 {
37     vector<int> c;
38     for (int v = t; v != -1; v = padre[v]) c.push_back(v);
39     reverse(c.begin(), c.end());
40     return c;
41 }
42
43 int main()
44 {
45     ios_base::sync_with_stdio(false);
46     cin.tie(nullptr);
47     return 0;
48 }
```

3.24 DSU (enfoque INCORRECTO para este problema) - Holy Network (DSU, Incorrecta)

O: $(n^2 \log V)$; rápida pero NO resuelve el problema

Une dos numeros cuando comparten un factor, y responde el tamaño de la componente mas grande

C++/ICPC/2025/H - Holy Network (DSU, incorrecta).cpp

```
1 #include <iostream>
2 #include <vector>
3 #include <map>
4
5 using namespace std;
6
7 long long buscar(map<long long, long long>& padre, long long x)
8 {
9     if(padre[x] == x)
10     {
11         return x;
12     }
13
14     return padre[x] = buscar(padre, padre[x]);
15 }
16
17 void unir(map<long long, long long>& padre, map<long long, long long>& tam, long long a, l
18         ong long b)
19 {
20     a = buscar(padre, a);
21     b = buscar(padre, b);
22
23     if(a == b)
24     {
25         return;
26     }
27
28     if(tam[a] < tam[b])
29     {
30         swap(a, b);
31     }
32
33     padre[b] = a;
34     tam[a] += tam[b];
35 }
36
37 int main()
38 {
39     ios::sync_with_stdio(false);
40     cin.tie(nullptr);
41
42     long long n;
43
44     while(cin >> n && n != 0)
45     {
46         vector<long long> numeros(n);
47
48         map<long long, bool> esInput;
49         map<long long, long long> padre;
50         map<long long, long long> tam;
51
52         for(long long i = 0; i < n; i++)
53         {
54             cin >> numeros[i];
55
56             esInput[numeros[i]] = true;
57             padre[numeros[i]] = numeros[i];
58             tam[numeros[i]] = 1;
59         }
60
61         for(long long x : numeros)
62         {
63             for(long long i = 2; i * i <= x; i++)
64             {
65                 if(x % i == 0)
66                 {
67                     long long z = x / i;
```

3.24 DSU (enfoque INCORRECTO para este problema) - Holy Network (DSU, Incorrecta) (cont.)

```
67
68         if(!padre.count(i))
69         {
70             padre[i] = i;
71             tam[i] = 1;
72         }
73
74         if(!padre.count(z))
75         {
76             padre[z] = z;
77             tam[z] = 1;
78         }
79
80         unir(padre, tam, x, i);
81         unir(padre, tam, x, z);
82     }
83 }
84
85 map<long long, long long> familias;
86 long long maxValor = 0;
87
88 for(long long x : numeros)
89 {
90     long long raiz = buscar(padre, x);
91
92     familias[raiz]++;
93
94     maxValor = max(maxValor, familias[raiz]);
95 }
96
97 cout << maxValor << '\n';
98 }
99
100 return 0;
101 }
102 }
```

3.25 Existencia de Camino Euleriano con Aristas Ocultas - Apple Pie

O: $(n + m)$, grados y conectividad del multigrafo

Completar el tramo tapado de una secuencia: existe camino euleriano en un multigrafo?

C++/ICPC/Regionals 2025/A - Apple Pie.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXN = 1005;
5
6 bool used[MAXN][MAXN];
7 int usedDegree[MAXN];
8
9 bool addPart(const vector<int>& a)
10 {
11     for(int i = 1; i < (int)a.size(); i++)
12     {
13         int u = a[i - 1];
14         int v = a[i];
15
16         if(u == v)
17             return false;
18
19         if(used[u][v])
20             return false;
21
22         used[u][v] = used[v][u] = true;
23
24         usedDegree[u]++;
25         usedDegree[v]++;
26     }
27
28     return true;
29 }
```

3.25 Existencia de Camino Euleriano con Aristas Ocultas - Apple Pie (cont.)

```

30
31 bool connected(int n, const vector<int>& degree)
32 {
33     int s = -1;
34
35     for(int i = 1; i <= n; i++)
36     {
37         if(degree[i] > 0)
38         {
39             s = i;
40             break;
41         }
42     }
43
44     if(s == -1)
45         return true;
46
47     vector<bool> vis(n + 1);
48
49     queue<int> q;
50     q.push(s);
51     vis[s] = true;
52
53     while(!q.empty())
54     {
55         int u = q.front();
56         q.pop();
57
58         for(int v = 1; v <= n; v++)
59         {
60             if(u == v)
61                 continue;
62
63             if(!used[u][v] && !vis[v])
64             {
65                 vis[v] = true;
66                 q.push(v);
67             }
68         }
69     }
70
71     for(int i = 1; i <= n; i++)
72     {
73         if(degree[i] > 0 && !vis[i])
74             return false;
75     }
76
77     return true;
78 }
79
80 bool canEuler(int n, int start, int finish, int remainingEdges)
81 {
82     if(remainingEdges == 0)
83         return false;
84
85     vector<int> degree(n + 1);
86
87     for(int i = 1; i <= n; i++)
88         degree[i] = n - 1 - usedDegree[i];
89
90     // Si conocemos un extremo, debe poder salir/entrar
91     // por alguna arista restante.
92     if(start != -1 && degree[start] == 0)
93         return false;
94
95     if(finish != -1 && degree[finish] == 0)
96         return false;
97
98     vector<int> odd;
99
100     for(int i = 1; i <= n; i++)
101     {

```

3.25 Existencia de Camino Euleriano con Aristas Ocultas - Apple Pie (cont.)

```

102         if(degree[i] & 1)
103             odd.push_back(i);
104     }
105
106     // Ambos extremos conocidos.
107     if(start != -1 && finish != -1)
108     {
109         if(start == finish)
110         {
111             // Euler cerrado.
112             if(!odd.empty())
113                 return false;
114         }
115         else
116         {
117             // Euler abierto.
118             if(odd.size() != 2)
119                 return false;
120
121             if(!(odd[0] == start && odd[1] == finish) ||
122                (odd[0] == finish && odd[1] == start)))
123                 return false;
124         }
125     }
126
127     // Solo conocemos el inicio.
128     else if(start != -1)
129     {
130         if(odd.size() == 0)
131         {
132             // Puede terminar donde empezo.
133         }
134         else if(odd.size() == 2)
135         {
136             if(odd[0] != start && odd[1] != start)
137                 return false;
138         }
139         else
140         {
141             return false;
142         }
143     }
144
145     // Solo conocemos el final.
146     else if(finish != -1)
147     {
148         if(odd.size() == 0)
149         {
150             // Puede empezar donde termina.
151         }
152         else if(odd.size() == 2)
153         {
154             if(odd[0] != finish && odd[1] != finish)
155                 return false;
156         }
157         else
158         {
159             return false;
160         }
161     }
162
163     // Ningun extremo conocido.
164     else
165     {
166         if(odd.size() != 0 && odd.size() != 2)
167             return false;
168     }
169
170     return connected(n, degree);
171 }
172
173

```

3.25 Existencia de Camino Euleriano con Aristas Ocultas - Apple Pie (cont.)

```

174 // Caso en el que la torta cubrió 0 elementos.
175 // Entonces simplemente tenemos L + R.
176 bool zeroCovered(int n,
177                  const vector<int>& L,
178                  const vector<int>& R)
179 {
180     int E = n * (n - 1) / 2;
181
182     if((int)L.size() + (int)R.size() != E + 1)
183         return false;
184
185     vector<int> a;
186
187     for(int x : L)
188         a.push_back(x);
189
190     for(int x : R)
191         a.push_back(x);
192
193     bool seen[MAXN][MAXN] = {};
194
195     for(int i = 1; i < (int)a.size(); i++)
196     {
197         int u = a[i - 1];
198         int v = a[i];
199
200         if(u == v)
201             return false;
202
203         if(seen[u][v])
204             return false;
205
206         seen[u][v] = seen[v][u] = true;
207     }
208
209     return true;
210 }
211
212 int main()
213 {
214     ios::sync_with_stdio(false);
215     cin.tie(nullptr);
216
217     int n;
218     cin >> n;
219
220     int p;
221     cin >> p;
222
223     vector<int> L(p);
224
225     for(int &x : L)
226         cin >> x;
227
228     int q;
229     cin >> q;
230
231     vector<int> R(q);
232
233     for(int &x : R)
234         cin >> x;
235
236
237     // -----
238     // Caso: no hubo elementos cubiertos
239     // -----
240
241     if(zeroCovered(n, L, R))
242     {
243         cout << "Y\n";
244         return 0;
245     }

```

3.25 Existencia de Camino Euleriano con Aristas Ocultas - Apple Pie (cont.)

```

246     }
247
248     // -----
249     // Marcar las aristas conocidas
250     // -----
251
252     if(!addPart(L))
253     {
254         cout << "N\n";
255         return 0;
256     }
257
258     if(!addPart(R))
259     {
260         cout << "N\n";
261         return 0;
262     }
263
264
265     int totalEdges = n * (n - 1) / 2;
266
267     int usedEdges = 0;
268
269     for(int i = 1; i <= n; i++)
270         usedEdges += usedDegree[i];
271
272     usedEdges /= 2;
273
274     int remainingEdges = totalEdges - usedEdges;
275
276
277     // -----
278     // Determinar extremos de la parte cubierta
279     // -----
280
281     int start = -1;
282     int finish = -1;
283
284     if(!L.empty())
285         start = L.back();
286
287     if(!R.empty())
288         finish = R.front();
289
290
291     // -----
292     // Las aristas restantes pueden formar
293     // el camino que falta?
294     // -----
295
296     if(canEuler(n, start, finish, remainingEdges))
297         cout << "Y\n";
298     else
299         cout << "N\n";
300
301     return 0;
302 }

```

3.26 Flujo Maximo (Dinic), Corte Minimo y Matching Bipartito (Kuhn) - Flujo Maximo Y Matching
O: Dinic $O(V^2 E)$ en general, $O(E \text{ raíz } V)$ en bipartito; Kuhn $O(V E)$
Uso: D.add(a,b,cap); D.maxflow(s,t); D.ladoS(v). Kuhn: K.g[izq].push_back(der); K.run()
 Flujo maximo = corte minimo, y el emparejamiento maximo entre dos grupos

C++/Notebook/Flujo Maximo y Matching.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 typedef long long ll;
6

```

3.26 Flujo Maximo (Dinic), Corte M nimo y Matching Bipartito (Kuhn) - Flujo Maximo Y Matching (cont.)

```

1 struct Dinic
2 {
3     struct Arista
4     {
5         int to;
6         ll cap;
7     };
8     vector<Arista> e;
9     vector<vector<int>> g;
10    vector<int> nivel, it;
11
12    Dinic(int n) : g(n), nivel(n), it(n) {}
13
14    void add(int a, int b, ll cap, ll capReversa = 0)
15    {
16        g[a].push_back(e.size());
17        e.push_back({b, cap});
18        g[b].push_back(e.size());
19        e.push_back({a, capReversa});
20    }
21
22    bool bfs(int s, int t)
23    {
24        fill(nivel.begin(), nivel.end(), -1);
25        queue<int> q;
26        q.push(s);
27        nivel[s] = 0;
28        while (!q.empty())
29        {
30            int v = q.front();
31            q.pop();
32            for (int id : g[v])
33            {
34                if (e[id].cap > 0 && nivel[e[id].to] == -1)
35                {
36                    nivel[e[id].to] = nivel[v] + 1;
37                    q.push(e[id].to);
38                }
39            }
40        }
41        return nivel[t] != -1;
42    }
43
44    ll dfs(int v, int t, ll f)
45    {
46        if (v == t) return f;
47        for (int &i = it[v]; i < (int)g[v].size(); i++)
48        {
49            int id = g[v][i];
50            int u = e[id].to;
51            if (e[id].cap > 0 && nivel[u] == nivel[v] + 1)
52            {
53                ll d = dfs(u, t, min(f, e[id].cap));
54                if (d > 0)
55                {
56                    e[id].cap -= d;
57                    e[id ^ 1].cap += d;
58                    return d;
59                }
60            }
61        }
62        return 0;
63    }
64
65    ll maxflow(int s, int t)
66    {
67        ll flujo = 0;
68        while (bfs(s, t))
69        {
70            fill(it.begin(), it.end(), 0);
71            while (ll d = dfs(s, t, LLONG_MAX)) flujo += d;
72        }
73    }

```

3.26 Flujo Maximo (Dinic), Corte M nimo y Matching Bipartito (Kuhn) - Flujo Maximo Y Matching (cont.)

```

74    return flujo;
75 }
76
77 bool ladoS(int v) { return nivel[v] != -1; } // valido despues de maxflow
78
79 struct Kuhn
80 {
81     int nL, nR, marca = 0;
82     vector<vector<int>> g; // g[izquierda] = derechas vecinas
83     vector<int> mR, vis;
84
85     Kuhn(int nL, int nR) : nL(nL), nR(nR), g(nL) {}
86
87     bool intentar(int v)
88     {
89         for (int u : g[v])
90         {
91             if (vis[u] == marca) continue;
92             vis[u] = marca;
93             if (mR[u] == -1 || intentar(mR[u]))
94             {
95                 mR[u] = v;
96                 return true;
97             }
98         }
99         return false;
100    }
101
102    int run()
103    {
104        mR.assign(nR, -1);
105        vis.assign(nR, 0);
106        int res = 0;
107        for (int v = 0; v < nL; v++)
108        {
109            marca++;
110            if (intentar(v)) res++;
111        }
112        return res;
113    }
114 };
115
116 int main()
117 {
118     ios_base::sync_with_stdio(false);
119     cin.tie(nullptr);
120     return 0;
121 }

```

3.27 Heavy-Light Decomposition - Los Ratones III

O: (log² N) por consulta, heavy-light decomposition

Un arbol de n casas conectadas por calles con valor

C++/Training_Camp_2026/Day 9/K - Los Ratones III.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const ll MOD = 1000000007LL;
7 const int MAXN = 100005;
8
9 struct Edge{
10     int to;
11     ll w;
12 };
13
14 int N,Q;
15
16 vector<Edge> g[MAXN];

```

3.27 Heavy-Light Decomposition - Los Ratones III (cont.)

```

71
72 int parent[MAXN], depth[MAXN], heavy[MAXN], sz[MAXN];
73 int head[MAXN], pos[MAXN];
74 ll val[MAXN];
75 int curPos;
76
77 struct Node{
78     ll sum, lazy;
79 };
80
81 Node seg[4*MAXN];
82
83 int dfs(int v, int p){
84     parent[v]=p;
85     sz[v]=1;
86     heavy[v]=-1;
87     int mx=0;
88     for(auto e:g[v]){
89         if(e.to==p) continue;
90         depth[e.to]=depth[v]+1;
91         val[e.to]=e.w;
92         dfs(e.to, v);
93         sz[v]+=sz[e.to];
94         if(sz[e.to]>mx){
95             mx=sz[e.to];
96             heavy[v]=e.to;
97         }
98     }
99     return sz[v];
100 }
101
102 ll base[MAXN];
103
104 void decompose(int v, int h){
105     head[v]=h;
106     pos[v]=++curPos;
107     base[curPos]=val[v];
108
109     if(heavy[v]!=-1)
110         decompose(heavy[v], h);
111
112     for(auto e:g[v]){
113         if(e.to==parent[v] || e.to==heavy[v]) continue;
114         decompose(e.to, e.to);
115     }
116 }
117
118 void build(int node, int l, int r){
119     seg[node].lazy=1;
120     if(l==r){
121         seg[node].sum=base[l]%MOD;
122         return;
123     }
124     int mid=(l+r)>>1;
125     build(node<<1, l, mid);
126     build(node<<1|1, mid+1, r);
127     seg[node].sum=(seg[node<<1].sum+seg[node<<1|1].sum)%MOD;
128 }
129
130 void push(int node, int l, int r){
131     if(seg[node].lazy==1) return;
132     ll x=seg[node].lazy;
133     seg[node].sum=seg[node].sum*x%MOD;
134     if(l==r){
135         seg[node<<1].lazy=seg[node<<1].lazy*x%MOD;
136         seg[node<<1|1].lazy=seg[node<<1|1].lazy*x%MOD;
137     }
138     seg[node].lazy=1;
139 }
140
141 void update(int node, int l, int r, int ql, int qr, ll x){
142     push(node, l, r);

```

3.27 Heavy-Light Decomposition - Los Ratones III (cont.)

```

89     if(r<ql||qr<l) return;
90     if(ql<=l&&r<=qr){
91         seg[node].lazy=x%MOD;
92         push(node, l, r);
93         return;
94     }
95     int mid=(l+r)>>1;
96     update(node<<1, l, mid, ql, qr, x);
97     update(node<<1|1, mid+1, r, ql, qr, x);
98     seg[node].sum=(seg[node<<1].sum+seg[node<<1|1].sum)%MOD;
99 }
100
101 ll query(int node, int l, int r, int ql, int qr){
102     push(node, l, r);
103     if(r<ql||qr<l) return 0;
104     if(ql<=l&&r<=qr) return seg[node].sum;
105     int mid=(l+r)>>1;
106     return (query(node<<1, l, mid, ql, qr)+
107            query(node<<1|1, mid+1, r, ql, qr))%MOD;
108 }
109
110 void path_update(int a, int b, ll x){
111     while(head[a]!=head[b]){
112         if(depth[head[a]]<depth[head[b]]) swap(a, b);
113         update(1, 1, N, pos[head[a]], pos[a], x);
114         a=parent[head[a]];
115     }
116     if(depth[a]>depth[b]) swap(a, b);
117     if(pos[a]+1<=pos[b])
118         update(1, 1, N, pos[a]+1, pos[b], x);
119 }
120
121 ll path_query(int a, int b){
122     ll ans=0;
123     while(head[a]!=head[b]){
124         if(depth[head[a]]<depth[head[b]]) swap(a, b);
125         ans=(ans+query(1, 1, N, pos[head[a]], pos[a]))%MOD;
126         a=parent[head[a]];
127     }
128     if(depth[a]>depth[b]) swap(a, b);
129     if(pos[a]+1<=pos[b])
130         ans=(ans+query(1, 1, N, pos[a]+1, pos[b]))%MOD;
131     return ans;
132 }
133
134 int main(){
135     ios::sync_with_stdio(false);
136     cin.tie(nullptr);
137
138     cin>>N>>Q;
139
140     for(int i=1;i<N;i++){
141         int u, v;
142         ll c;
143         cin>>u>>v>>c;
144         g[u].push_back({v, c});
145         g[v].push_back({u, c});
146     }
147
148     depth[1]=0;
149     val[1]=0;
150
151     dfs(1, 0);
152     decompose(1, 1);
153
154     build(1, 1, N);
155
156     while(Q--){
157         int u, v;
158         ll x;
159         cin>>u>>v>>x;
160

```

3.27 Heavy-Light Decomposition - Los Ratones III (cont.)

```

51     path_update(u,v,x);
52     cout<<path_query(u,v)%MOD<<"\n";
53 }
54
55     return 0;
56 }

```

3.28 Kruskal Reconstruction Tree - Trucks

O: ($m \log m$), Kruskal reconstruction tree

Nlogonia es un archipi lago de N islas unidas por M puentes, cada uno con un l mite m ximo de peso W

C++/Training_Camp_2026/Day 2/J - Trucks.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  static char buf_[1 << 22];
5  static int bufLen_, bufPos_;
6  inline int gc() {
7      if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0;
8          if (!bufLen_) return -1; }
9      return buf_[bufPos_++];
10 }
11 inline int readInt() {
12     int c = gc();
13     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
14     bool neg = false; if (c == '-') { neg = true; c = gc(); }
15     int x = 0;
16     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
17     return neg ? -x : x;
18 }
19
20 int N, M, S;
21 vector<int> dsuParent;
22 int find(int x) {
23     while (dsuParent[x] != x) { dsuParent[x] = dsuParent[dsuParent[x]]; x = dsuParent[x]; }
24     return x;
25 }
26
27 int main() {
28     N = readInt(); M = readInt(); S = readInt();
29     vector<array<int,3>> edgeList(M); // {W, A, B} for easy sort by weight
30     for (int i = 0; i < M; i++) {
31         int a = readInt(), b = readInt(), w = readInt();
32         edgeList[i] = {w, a, b};
33     }
34     sort(edgeList.begin(), edgeList.end(), [](const array<int,3>&x, const array<int,3>&y){
35         return x[0] > y[0]; // decreasing weight
36     });
37
38     int totalNodes = 2 * N; // leaves 1..N, internal nodes N+1..2N-1 (at most N-1 of them)
39     dsuParent.assign(totalNodes + 1, 0);
40     iota(dsuParent.begin(), dsuParent.end(), 0);
41
42     vector<int> nodeRepr(totalNodes + 1, 0);
43     for (int i = 1; i <= N; i++) nodeRepr[i] = i;
44
45     vector<int> leftChild(totalNodes + 1, 0), rightChild(totalNodes + 1, 0), weightOf(totalNodes + 1, 0);
46
47     int nextInternal = N + 1;
48     int merges = 0;
49     for (auto &e : edgeList) {
50         if (merges == N - 1) break;
51         int w = e[0], a = e[1], b = e[2];
52         int ra = find(a), rb = find(b);
53         if (ra == rb) continue;
54         int newNode = nextInternal++;
55         weightOf[newNode] = w;
56         leftChild[newNode] = nodeRepr[ra];

```

3.28 Kruskal Reconstruction Tree - Trucks (cont.)

```

57         rightChild[newNode] = nodeRepr[rb];
58         dsuParent[ra] = rb; // union
59         nodeRepr[rb] = newNode; // rb is now the representative root (arbitrary choice), pointing to new node
60         merges++;
61     }
62     int root = nodeRepr[find(1)];
63
64     int totalUsed = nextInternal - 1; // highest node id actually used
65     int LOG = 1;
66     while ((1 << LOG) < totalUsed + 1) LOG++;
67     vector<vector<int>> up(LOG, vector<int>(totalUsed + 1, 0));
68     vector<int> depth(totalUsed + 1, 0);
69
70     // iterative pre-order traversal to set depth & up[0]
71     {
72         vector<int> stk;
73         stk.push_back(root);
74         depth[root] = 0;
75         up[0][root] = 0;
76         while (!stk.empty()) {
77             int u = stk.back(); stk.pop_back();
78             if (u <= N) continue; // leaf, no children
79             int lc = leftChild[u], rc = rightChild[u];
80             depth[lc] = depth[u] + 1; up[0][lc] = u; stk.push_back(lc);
81             depth[rc] = depth[u] + 1; up[0][rc] = u; stk.push_back(rc);
82         }
83     }
84     for (int k = 1; k < LOG; k++) {
85         for (int v = 1; v <= totalUsed; v++) {
86             up[k][v] = up[k-1][up[k-1][v]];
87         }
88     }
89
90     auto lca = [&](int u, int v) -> int {
91         if (depth[u] < depth[v]) swap(u, v);
92         int diff = depth[u] - depth[v];
93         for (int k = 0; k < LOG; k++) if ((diff >> k) & 1) u = up[k][u];
94         if (u == v) return u;
95         for (int k = LOG - 1; k >= 0; k--) {
96             if (up[k][u] != up[k][v]) { u = up[k][u]; v = up[k][v]; }
97         }
98         return up[0][u];
99     };
100
101     string out;
102     out.reserve(S * 8);
103     for (int i = 0; i < S; i++) {
104         int L = readInt(), H = readInt();
105         int anc = lca(L, H);
106         out += to_string(weightOf[anc]);
107         out += "\n";
108     }
109     fwrite(out.data(), 1, out.size(), stdout);
110     return 0;
111 }

```

3.29 LCA (Binary Lifting) - Subway Lines

O: ($n \log n$) de tabla y ($\log n$) por consulta, LCA con binary lifting

En un sistema de subte con forma de  rbol

C++/Training_Camp_2026/Day 8/L - Subway Lines.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  static char buf_[1 << 22];
5  static int bufLen_, bufPos_;
6  inline int gc() {
7      if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0;
8          if (!bufLen_) return -1; }
9      return buf_[bufPos_++];

```

3.29 LCA (Binary Lifting) - Subway Lines (cont.)

```

9 }
10 inline int readInt() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18
19 int N, Q;
20 vector<vector<int>> adj;
21 vector<int> depth_;
22 int LOG_;
23 vector<vector<int>> up_;
24
25 void bfsSetup(int root) {
26     depth_.assign(N + 1, -1);
27     up_.assign(LOG_, vector<int>(N + 1, root));
28     depth_[root] = 0;
29     up_[0][root] = root;
30     queue<int> q;
31     q.push(root);
32     while (!q.empty()) {
33         int u = q.front(); q.pop();
34         for (int v : adj[u]) {
35             if (depth_[v] == -1) {
36                 depth_[v] = depth_[u] + 1;
37                 up_[0][v] = u;
38                 q.push(v);
39             }
40         }
41     }
42     for (int k = 1; k < LOG_; k++) {
43         for (int v = 1; v <= N; v++) {
44             up_[k][v] = up_[k - 1][up_[k - 1][v]];
45         }
46     }
47 }
48
49 int lca(int u, int v) {
50     if (depth_[u] < depth_[v]) swap(u, v);
51     int diff = depth_[u] - depth_[v];
52     for (int k = 0; k < LOG_; k++) if ((diff >> k) & 1) u = up_[k][u];
53     if (u == v) return u;
54     for (int k = LOG_ - 1; k >= 0; k--) {
55         if (up_[k][u] != up_[k][v]) { u = up_[k][u]; v = up_[k][v]; }
56     }
57     return up_[0][u];
58 }
59
60 inline int dist(int u, int v) {
61     return depth_[u] + depth_[v] - 2 * depth_[lca(u, v)];
62 }
63
64 inline bool isAncestor(int u, int v) {
65     return lca(u, v) == u;
66 }
67
68 int solveQuery(int a, int b, int c, int d) {
69     int l1 = lca(a, b), l2 = lca(c, d);
70     if (depth_[l1] > depth_[l2]) {
71         swap(a, c); swap(b, d); swap(l1, l2);
72     }
73     // is l2 on path(a,b)?
74     if (dist(a, l2) + dist(l2, b) != dist(a, b)) return 0;
75
76     if (l1 == l2) {
77         int lac = lca(a, c), lad = lca(a, d);
78         int deepA = (depth_[lac] >= depth_[lad]) ? lac : lad;
79         int lbc = lca(b, c), lbd = lca(b, d);
80         int deepB = (depth_[lbc] >= depth_[lbd]) ? lbc : lbd;

```

3.29 LCA (Binary Lifting) - Subway Lines (cont.)

```

81     return dist(deepA, deepB) + 1;
82 } else {
83     if (isAncestor(l2, a)) {
84         int lac = lca(a, c), lad = lca(a, d);
85         int deepA = (depth_[lac] >= depth_[lad]) ? lac : lad;
86         return dist(l2, deepA) + 1;
87     } else {
88         int lbc = lca(b, c), lbd = lca(b, d);
89         int deepB = (depth_[lbc] >= depth_[lbd]) ? lbc : lbd;
90         return dist(l2, deepB) + 1;
91     }
92 }
93 }
94
95 int main() {
96     N = readInt(); Q = readInt();
97     adj.assign(N + 1, {});
98     for (int i = 0; i < N - 1; i++) {
99         int u = readInt(), v = readInt();
100         adj[u].push_back(v);
101         adj[v].push_back(u);
102     }
103
104     LOG_ = 1;
105     while ((1 << LOG_) < N + 1) LOG_++;
106     bfsSetup(1);
107
108     string out;
109     out.reserve(Q * 7);
110     char tmp[16];
111     for (int q = 0; q < Q; q++) {
112         int a = readInt(), b = readInt(), c = readInt(), d = readInt();
113         int res = solveQuery(a, b, c, d);
114         int len = sprintf(tmp, "%d\n", res);
115         out.append(tmp, len);
116     }
117     fwrite(out.data(), 1, out.size(), stdout);
118     return 0;
119 }

```

3.30 LCA con Binary Lifting

O: (n log n) armar, (log n) cada consulta

Uso: LCA L(g, raiz); L.lca(a,b); L.dist(a,b); L.subir(v,k); g lista de adyacencia
 Ancestro comun mas bajo, distancia entre dos nodos y k-esimo ancestro en un arbol

C++/Notebook/LCA.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 struct LCA
6 {
7     int n, LOG;
8     vector<vector<int>> up;
9     vector<int> dep;
10
11     LCA(const vector<vector<int>> &g, int raiz = 0) : n(g.size()), LOG(1), dep(n, -1)
12     {
13         while ((1 << LOG) < n) LOG++;
14         up.assign(LOG, vector<int>(n, raiz));
15         queue<int> q;
16         q.push(raiz);
17         dep[raiz] = 0;
18         while (!q.empty())
19         {
20             int v = q.front();
21             q.pop();
22             for (int u : g[v])
23             {
24                 if (dep[u] != -1) continue;
25                 dep[u] = dep[v] + 1;

```

3.30 LCA con Binary Lifting (cont.)

```

26         up[0][u] = v;
27         q.push(u);
28     }
29 }
30 for (int j = 1; j < LOG; j++)
31 {
32     for (int v = 0; v < n; v++) up[j][v] = up[j - 1][up[j - 1][v]];
33 }
34 }
35
36 int subir(int v, int k) // ancestro k niveles arriba (raiz si se pasa)
37 {
38     k = min(k, dep[v]); // sin esto, un k >= 2^LOG perdia sus bits altos
39     for (int j = 0; j < LOG; j++)
40     {
41         if (k >> j & 1) v = up[j][v];
42     }
43     return v;
44 }
45
46 int lca(int a, int b)
47 {
48     if (dep[a] < dep[b]) swap(a, b);
49     a = subir(a, dep[a] - dep[b]);
50     if (a == b) return a;
51     for (int j = LOG - 1; j >= 0; j--)
52     {
53         if (up[j][a] != up[j][b])
54         {
55             a = up[j][a];
56             b = up[j][b];
57         }
58     }
59     return up[0][a];
60 }
61
62 int dist(int a, int b) { return dep[a] + dep[b] - 2 * dep[lca(a, b)]; }
63 };
64
65 int main()
66 {
67     ios_base::sync_with_stdio(false);
68     cin.tie(nullptr);
69     return 0;
70 }

```

3.31 Max Flow - Download Speed

O: ($V^2 * E$) con Dinic; en grafos de unidades baja mucho
Dada una red de n computadoras y m conexiones dirigidas con capacidad c

C++/Training_Camp_2026/Day 7/H - Download Speed.cpp

```

1 #include <bits/stdc++.h>
2
3 typedef long long ll;
4
5 using namespace std;
6
7 struct edge { ll v, cap, inv, flow; };
8
9 struct network
10 {
11     ll n, s, t;
12     vector<ll> lvl;
13     vector<vector<edge>> g;
14     network(ll n) : n(n), lvl(n), g(n) {}
15     void add_edge(ll u, ll v, ll c)
16     {
17         g[u].push_back({v, c, (ll)g[v].size(), 0});
18         g[v].push_back({u, 0, (ll)g[u].size()-1, c});
19     }
20 }

```

3.31 Max Flow - Download Speed (cont.)

```

21 bool bfs()
22 {
23     fill(lvl.begin(), lvl.end(), -1);
24     queue<ll> q;
25     lvl[s] = 0;
26     for(q.push(s); q.size(); q.pop())
27     {
28         ll u = q.front();
29         for(auto &e : g[u]) {
30             if(e.cap > 0 && lvl[e.v] == -1)
31             {
32                 lvl[e.v] = lvl[u]+1;
33                 q.push(e.v);
34             }
35         }
36     }
37     return lvl[t] != -1;
38 }
39
40 ll dfs(ll u, ll nf)
41 {
42     if(u == t) return nf;
43     ll res = 0;
44     for (auto &e : g[u])
45     {
46         if(e.cap > 0 && lvl[e.v] == lvl[u]+1)
47         {
48             ll tf = dfs(e.v, min(nf, e.cap));
49             res += tf; nf -= tf; e.cap -= tf;
50             g[e.v][e.inv].cap += tf;
51             g[e.v][e.inv].flow -= tf;
52             e.flow += tf;
53             if(nf == 0) return res;
54         }
55     }
56     if(!res) lvl[u] = -1;
57     return res;
58 }
59
60 ll max_flow(ll so, ll si, ll res = 0) {
61     s = so; t = si;
62     while(bfs()) res += dfs(s, 1000000000000);
63     return res;
64 }
65
66 };
67
68 int main()
69 {
70
71     ll a,b; cin >> a >> b;
72     network pcs(a+1);
73     for(int i = 0 ; i < b;i++)
74     {
75         ll x,y,z;cin >> x >> y >> z;
76         pcs.add_edge(x,y,z);
77     }
78     cout << pcs.max_flow(1,a)<< endl;
79
80     return 0;
81 }

```

3.32 Max Flow - March Of The Penguins

O: ($V^2 * E$) con Dinic, probando cada tempano como destino
Hay N tempanos de hielo con coordenadas (x,y)

C++/Training_Camp_2026/Day 7/I - March of the Penguins.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Dinic {

```

3.32 Max Flow - March Of The Penguins (cont.)

```

5 struct Edge { int to; int cap; int flow; };
6 int n;
7 vector<Edge> edges;
8 vector<vector<int>> g;
9 vector<int> level, it;
10 Dinic(int n=0): n(n), g(n) {}
11 int addEdge(int from,int to,int cap){
12     g[from].push_back((int)edges.size());
13     edges.push_back({to,cap,0});
14     g[to].push_back((int)edges.size());
15     edges.push_back({from,0,0});
16     return (int)edges.size()-2;
17 }
18 void resetFlow(){ for(auto &e: edges) e.flow = 0; }
19 bool bfs(int s,int t){
20     level.assign(n,-1);
21     queue<int> q; level[s]=0;q.push(s);
22     while(!q.empty()){
23         int u=q.front();q.pop();
24         for(int id: g[u]){
25             auto &e=edges[id];
26             if(e.cap - e.flow>0 && level[e.to]<0){
27                 level[e.to]=level[u]+1;
28                 q.push(e.to);
29             }
30         }
31     }
32     return level[t]>=0;
33 }
34 int dfs(int u,int t,int f){
35     if(u==t) return f;
36     for(int &i=it[u]; i<(int)g[u].size(); i++){
37         int id=g[u][i];
38         auto &e=edges[id];
39         if(e.cap-e.flow>0 && level[e.to]==level[u]+1){
40             int d=dfs(e.to,t,min(f,e.cap-e.flow));
41             if(d>0){ e.flow+=d; edges[id^1].flow-=d; return d; }
42         }
43     }
44     return 0;
45 }
46 int maxflow(int s,int t){
47     int flow=0;
48     while(bfs(s,t)){
49         it.assign(n,0);
50         int f;
51         while((f=dfs(s,t,INT_MAX))>0) flow+=f;
52     }
53     return flow;
54 }
55 };
56
57 int main(){
58     ios::sync_with_stdio(false);
59     cin.tie(nullptr);
60
61     int T;
62     if(!(cin >> T)) return 0;
63     while(T--){
64         int N; double D;
65         cin >> N >> D;
66         vector<double> x(N), y(N);
67         vector<int> n(N), m(N);
68         for(int i=0;i<N;i++) cin >> x[i] >> y[i] >> n[i] >> m[i];
69
70         int totalPenguins = 0;
71         for(int i=0;i<N;i++) totalPenguins += n[i];
72
73         vector<vector<bool>> canJump(N, vector<bool>(N,false));
74         const double eps = 1e-6;
75         for(int i=0;i<N;i++)
76             for(int j=i+1;j<N;j++){

```

3.32 Max Flow - March Of The Penguins (cont.)

```

77         double dx = x[i]-x[j], dy = y[i]-y[j];
78         double dist = sqrt(dx*dx+dy*dy);
79         if(dist <= D + eps) canJump[i][j] = canJump[j][i] = true;
80     }
81
82     // in_node(i)=i, out_node(i)=N+i, S=2N
83     int S = 2*N;
84     int numNodes = 2*N+1;
85     Dinic dinic(numNodes);
86
87     for(int i=0;i<N;i++) dinic.addEdge(i, N+i, m[i]); // presupuesto de saltos
88     for(int i=0;i<N;i++) dinic.addEdge(S, i, n[i]); // pingüinos que parten de i
89
90     const int INF = 1000000;
91     for(int i=0;i<N;i++){
92         for(int j=i+1;j<N;j++){
93             if(canJump[i][j]){
94                 dinic.addEdge(N+i, j, INF);
95                 dinic.addEdge(N+j, i, INF);
96             }
97         }
98     }
99     vector<int> answer;
100     for(int t=0;t<N;t++){
101         dinic.resetFlow();
102         int f = dinic.maxflow(S, t); // sink = in_node(t)
103         if(f == totalPenguins) answer.push_back(t);
104     }
105     if(answer.empty()) cout << -1 << "\n";
106     else {
107         for(size_t i=0;i<answer.size();i++){
108             if(i) cout << ' ';
109             cout << answer[i];
110         }
111         cout << "\n";
112     }
113 }
114 return 0;
115 }

```

3.33 Max Flow (Binary Search + Dinic) - Gasoline

O: ($V^2 * E * \log(\text{rango})$): binaria sobre la respuesta y un Dinic por intento

Menor tiempo para abastecer todas las gasolineras: binaria sobre la respuesta y Dinic

C++/Training_Camp_2026/Day 8/G - Gasoline.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct edge {
5     int v, cap, inv, flow;
6 };
7
8 struct network {
9     int n, s, t;
10    vector<int> lvl;
11    vector<vector<edge>> g;
12    network(int n) : n(n), lvl(n), g(n) {}
13    void add_edge(int u, int v, int c) {
14        g[u].push_back({v, c, (int)g[v].size(), 0});
15        g[v].push_back({u, 0, (int)g[u].size()-1, c});
16    }
17    bool bfs() {
18        fill(lvl.begin(), lvl.end(), -1);
19        queue<int> q;
20        lvl[s] = 0;
21        for(q.push(s); q.size(); q.pop()) {
22            int u = q.front();
23            for (auto &e : g[u]) {
24                if(e.cap > 0 && lvl[e.v] == -1) {
25                    lvl[e.v] = lvl[u]+1;
26                    q.push(e.v);

```

3.33 Max Flow (Binary Search + Dinic) - Gasoline (cont.)

```

27     }
28 }
29 }
30 return lvl[t] != -1;
31 }
32 int dfs(int u, int nf) {
33     if(u == t) return nf;
34     int res = 0;
35     for (auto &e : g[u]) {
36         if(e.cap > 0 && lvl[e.v] == lvl[u]+1) {
37             int tf = dfs(e.v, min(nf, e.cap));
38             res += tf;
39             nf -= tf;
40             e.cap -= tf;
41             g[e.v][e.inv].cap += tf;
42             g[e.v][e.inv].flow -= tf;
43             e.flow += tf;
44             if(nf == 0) return res;
45         }
46     }
47     if(!res) lvl[u] = -1;
48     return res;
49 }
50 int max_flow(int so, int si, int res = 0) {
51     s = so; t = si;
52     while(bfs()) res += dfs(s, INT_MAX);
53     return res;
54 }
55 };
56
57 int P, R, C;
58 vector<int> D, E; // D[1..P], E[1..R]
59 vector<array<int,3>> edges; // I (station), J (refinery), T
60 long long sumD;
61
62 // nodos: 0 = source
63 // 1..R = refineries
64 // R+1..R+P = gasolineras
65 // R+P+1 = sumidero
66 bool feasible(int t) {
67     int n = R + P + 2;
68     int S = 0, T = R + P + 1;
69     network net(n);
70
71     // source -> refinera, capacidad = stock
72     for (int j = 1; j <= R; j++) {
73         net.add_edge(S, j, E[j]);
74     }
75     // gasolinera -> sumidero, capacidad = demanda
76     for (int i = 1; i <= P; i++) {
77         net.add_edge(R + i, T, D[i]);
78     }
79     // refinera -> gasolinera, solo si T_edge <= t
80     int INF = 1000000000;
81     for (auto &e : edges) {
82         int i = e[0], j = e[1], time = e[2];
83         if (time <= t) {
84             net.add_edge(j, R + i, INF);
85         }
86     }
87
88     long long f = net.max_flow(S, T);
89     return f >= sumD;
90 }
91
92 int main(){
93     ios::sync_with_stdio(false);
94     cin.tie(nullptr);
95
96     cin >> P >> R >> C;
97     D.assign(P+1, 0);
98     E.assign(R+1, 0);

```

3.33 Max Flow (Binary Search + Dinic) - Gasoline (cont.)

```

99     for (int i = 1; i <= P; i++) cin >> D[i];
100    for (int j = 1; j <= R; j++) cin >> E[j];
101
102    edges.resize(C);
103    vector<int> times(C);
104    for (int k = 0; k < C; k++) {
105        int i, j, t;
106        cin >> i >> j >> t;
107        edges[k] = {i, j, t};
108        times[k] = t;
109    }
110
111    sumD = 0;
112    for (int i = 1; i <= P; i++) sumD += D[i];
113
114    // valores distintos de tiempo, ordenados
115    sort(times.begin(), times.end());
116    times.erase(unique(times.begin(), times.end()), times.end());
117
118    // primero chequeamos si es factible usando TODOS los arcos (t = maximo)
119    if (times.empty() || !feasible(times.back())) {
120        cout << -1 << "\n";
121        return 0;
122    }
123
124    // binary search: encontrar el menor t factible
125    int lo = 0, hi = (int)times.size() - 1;
126    while (lo < hi) {
127        int mid = (lo + hi) / 2;
128        if (feasible(times[mid])) {
129            hi = mid;
130        } else {
131            lo = mid + 1;
132        }
133    }
134
135    cout << times[lo] << "\n";
136    return 0;
137 }

```

3.34 MST con Prim (Grafo Denso) - Brute

O: (n^2), Prim para grafo denso

Una caja fuerte con N llaves de 4 dígitos que hay que desbloquear todas

C++/CCPL/brute.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 // Giros para pasar de la combinacion a a la b, por el lado corto de cada rueda.
6 int distancia(const string& a, const string& b)
7 {
8     int dis = 0;
9
10    for (int i = 0; i < 4; i++)
11    {
12        int x = a[i] - '0';
13        int y = b[i] - '0';
14
15        dis += min(abs(x - y), 10 - abs(x - y));
16    }
17
18    return dis;
19 }
20
21 int main()
22 {
23     ios::sync_with_stdio(false);
24     cin.tie(nullptr);
25
26     int T;

```

3.34 MST con Prim (Grafo Denso) - Brute (cont.)

```

27     cin >> T;
28
29     while (T--)
30     {
31         int N;
32         cin >> N;
33
34         vector<string> locks(N);
35
36         for (int i = 0; i < N; i++)
37             cin >> locks[i];
38
39         // Costo de llegar inicialmente desde 0000
40         // a cada una de las llaves.
41         int respuestaInicial = INT_MAX;
42
43         for (int i = 0; i < N; i++)
44         {
45             respuestaInicial = min(
46                 respuestaInicial,
47                 distancia("0000", locks[i])
48             );
49         }
50
51         // Prim solamente entre las llaves.
52         // 0000 NO forma parte del grafo.
53         const int INF = INT_MAX;
54
55         vector<int> mejor(N, INF);
56         vector<bool> visitado(N, false);
57
58         // Podemos comenzar el MST desde cualquier llave,
59         // porque el costo inicial desde 0000 ya lo escogimos
60         // como el minimo posible.
61         mejor[0] = 0;
62
63         int total = respuestaInicial;
64
65         for (int k = 0; k < N; k++)
66         {
67             int u = -1;
68
69             for (int i = 0; i < N; i++)
70             {
71                 if (!visitado[i] &&
72                     (u == -1 || mejor[i] < mejor[u]))
73                 {
74                     u = i;
75                 }
76             }
77
78             visitado[u] = true;
79             total += mejor[u];
80
81             for (int v = 0; v < N; v++)
82             {
83                 if (!visitado[v])
84                 {
85                     int peso = distancia(locks[u], locks[v]);
86
87                     mejor[v] = min(mejor[v], peso);
88                 }
89             }
90         }
91
92         cout << total << '\n';
93     }
94
95     return 0;
96 }

```

3.35 Puentes y Puntos de Articulacion

O: (n + m)

Uso: agregarArista(a,b) con ids 0..m-1; calcular(); puentes = ids; esArt[v]

Que aristas o que nodos, si se quitan, desconectan el grafo (no dirigido)

C++/Notebook/Puentes y Puntos de Articulacion.cpp

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int n, m = 0, timer_;
6  vector<vector<pair<int, int>>> g;           // (vecino, id de la arista)
7  vector<int> tin, low, puentes;
8  vector<bool> esArt;
9
10 void agregarArista(int a, int b)
11 {
12     g[a].push_back({b, m});
13     g[b].push_back({a, m});
14     m++;
15 }
16
17 void dfs(int v, int aristaPadre)
18 {
19     tin[v] = low[v] = timer_++;
20     int hijos = 0;
21     for (auto [u, id] : g[v])
22     {
23         if (id == aristaPadre) continue;
24         if (tin[u] != -1)
25         {
26             low[v] = min(low[v], tin[u]);
27         }
28         else
29         {
30             dfs(u, id);
31             low[v] = min(low[v], low[u]);
32             if (low[u] > tin[v]) puentes.push_back(id);
33             if (low[u] >= tin[v] && aristaPadre != -1) esArt[v] = true;
34             hijos++;
35         }
36     }
37     if (aristaPadre == -1 && hijos > 1) esArt[v] = true;
38 }
39
40 void calcular()
41 {
42     tin.assign(n, -1);
43     low.assign(n, 0);
44     esArt.assign(n, false);
45     puentes.clear();
46     timer_ = 0;
47     for (int v = 0; v < n; v++)
48     {
49         if (tin[v] == -1) dfs(v, -1);
50     }
51 }
52
53 int main()
54 {
55     ios_base::sync_with_stdio(false);
56     cin.tie(nullptr);
57     // cin >> n >> m; g.assign(n, {}); for (...) agregarArista(a, b); calcular();
58     return 0;
59 }

```

3.36 Quitar el DAG de Rutas Mínimas - Almost Shortest Path

O: (m log n), dos Dijkstra mas un barrido del DAG

Uso: leer el grafo dirigido, s y d; imprime la ruta mas corta que NO usa ninguna arista...
Dado un grafo dirigido con pesos, hallar la ruta mas corta de s a d que no use NINGUNA arista que aparezca...

C++/Notebook/Almost Shortest Path.cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <utility>
5 #include <climits>
6 #include <algorithm>
7
8 using namespace std;
9
10 int main()
11 {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14
15     long long n, m;
16     long long s, d;
17
18     while (cin >> n >> m && n != 0 && m != 0)
19     {
20         cin >> s >> d;
21
22         vector<vector<pair<long long, long long>>> conexiones(n);
23         vector<vector<bool>> paso(n, vector<bool>(n, false));
24
25         for (long long i = 0; i < m; i++)
26         {
27             long long a, b, c;
28             cin >> a >> b >> c;
29
30             conexiones[a].push_back({b, c});
31             paso[a][b] = true;
32         }
33
34         vector<vector<long long>> caminos(n);
35         vector<long long> distancia(n, LLONG_MAX);
36
37         priority_queue<
38             pair<long long, long long>,
39             vector<pair<long long, long long>>,
40             greater<pair<long long, long long>>
41         > cola;
42
43         caminos[s] = {-1};
44         distancia[s] = 0;
45         cola.push({0, s});
46
47         while (!cola.empty())
48         {
49             long long peso = cola.top().first;
50             long long actual = cola.top().second;
51             cola.pop();
52
53             if (peso != distancia[actual])
54             {
55                 continue;
56             }
57
58             for (pair<long long, long long> siguiente : conexiones[actual])
59             {
60                 long long hacia = siguiente.first;
61                 long long cuanto = siguiente.second;
62
63                 long long nuevaDistancia = peso + cuanto;
64
65                 if (distancia[hacia] > nuevaDistancia)
66                 {

```

3.36 Quitar el DAG de Rutas Mínimas - Almost Shortest Path (cont.)

```

67         distancia[hacia] = nuevaDistancia;
68
69         caminos[hacia] = {actual}; // mejora: se REINICIA la lista
70
71         cola.push({nuevaDistancia, hacia});
72     }
73     else if (distancia[hacia] == nuevaDistancia)
74     {
75         caminos[hacia].push_back(actual); // empate: se AGREGA
76     }
77 }
78
79 queue<long long> colita;
80 vector<bool> visto(n, false);
81 colita.push(d);
82 visto[d] = true;
83
84 while (!colita.empty())
85 {
86     long long ahora = colita.front();
87     colita.pop();
88
89     for (long long poraca : caminos[ahora])
90     {
91         if (poraca == -1) continue;
92         paso[poraca][ahora] = false;
93         if (visto[poraca]) continue; // sin esta linea el BFS es exponencial
94         visto[poraca] = true;
95         colita.push(poraca);
96     }
97 }
98
99 distancia.assign(n, LLONG_MAX);
100
101 distancia[s] = 0;
102 cola.push({0, s});
103
104 while (!cola.empty())
105 {
106     long long peso = cola.top().first;
107     long long actual = cola.top().second;
108     cola.pop();
109
110     if (peso != distancia[actual])
111     {
112         continue;
113     }
114
115     for (pair<long long, long long> siguiente : conexiones[actual])
116     {
117         long long hacia = siguiente.first;
118         long long cuanto = siguiente.second;
119
120         if (!paso[actual][hacia]) continue;
121
122         long long nuevaDistancia = peso + cuanto;
123
124         if (distancia[hacia] > nuevaDistancia)
125         {
126             distancia[hacia] = nuevaDistancia;
127
128             cola.push({nuevaDistancia, hacia});
129         }
130     }
131 }
132
133 cout << (distancia[d] == LLONG_MAX ? -1 : distancia[d]) << '\n';
134 }
135
136 return 0;

```

3.36 Quitar el DAG de Rutas Mínimas - Almost Shortest Path (cont.)
 88 }

3.37 SCC (Tarjan) y 2-SAT - SCC Y 2 SAT

O: (n + m)

Uso: SCC s(n); s.add(a,b); s.run(); s.comp[v]. TwoSat t(vars); t.alMenosUno(a,va,b,vb)
 Componentes fuertemente conexas, y con eso: existe asignacion verdadero/falso?

C++/Notebook/SCC y 2-SAT.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 struct SCC
6 {
7     int n, timer_ = 0, nc = 0;
8     vector<vector<int>> g;
9     vector<int> comp, tin, low, pila;
10    vector<bool> enPila;
11
12    SCC(int n) : n(n), g(n) {}
13    void add(int a, int b) { g[a].push_back(b); }
14
15    void dfs(int v)
16    {
17        tin[v] = low[v] = timer_++;
18        pila.push_back(v);
19        enPila[v] = true;
20        for (int u : g[v])
21        {
22            if (tin[u] == -1)
23            {
24                dfs(u);
25                low[v] = min(low[v], low[u]);
26            }
27            else if (enPila[u])
28            {
29                low[v] = min(low[v], tin[u]);
30            }
31        }
32        if (low[v] == tin[v]) // v es la cabeza de su componente
33        {
34            while (true)
35            {
36                int u = pila.back();
37                pila.pop_back();
38                enPila[u] = false;
39                comp[u] = nc;
40                if (u == v) break;
41            }
42            nc++;
43        }
44    }
45
46    int run() // devuelve la cantidad de componentes
47    {
48        comp.assign(n, -1);
49        tin.assign(n, -1);
50        low.assign(n, 0);
51        enPila.assign(n, false);
52        for (int v = 0; v < n; v++)
53        {
54            if (tin[v] == -1) dfs(v);
55        }
56        return nc;
57    }
58 };
59
60 struct TwoSat
61 {
62     int n;
63     SCC s;
```

3.37 SCC (Tarjan) y 2-SAT - SCC Y 2 SAT (cont.)

```
64     vector<bool> valor;
65     TwoSat(int n) : n(n), s(2 * n) {}
66     int nodo(int x, bool v) { return 2 * x + (v ? 0 : 1); }
67
68     // (x == vx) o (y == vy)
69     void alMenosUno(int x, bool vx, int y, bool vy)
70     {
71         s.add(nodo(x, !vx), nodo(y, vy));
72         s.add(nodo(y, !vy), nodo(x, vx));
73     }
74
75     bool resolver()
76     {
77         s.run();
78         valor.assign(n, false);
79         for (int x = 0; x < n; x++)
80         {
81             if (s.comp[2 * x] == s.comp[2 * x + 1]) return false;
82             valor[x] = s.comp[2 * x] < s.comp[2 * x + 1];
83         }
84         return true;
85     }
86 };
87
88 int main()
89 {
90     ios_base::sync_with_stdio(false);
91     cin.tie(nullptr);
92     return 0;
93 }
```

3.38 State-Space BFS - Folding Machine

O: (estados * transiciones), BFS sobre el espacio de estados

Doblar una cinta sumando las celdas superpuestas: BFS sobre el espacio de estados

C++/Training_Camp_2026/Day 2/D - Folding Machine.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 // A fold at either end may reverse the tape, so both directions
7 // represent the same state.
8 vector<ll> normalized(vector<ll> tape) {
9     vector<ll> reversed = tape;
10    reverse(reversed.begin(), reversed.end());
11    return min(tape, reversed);
12 }
13
14 int main() {
15     ios::sync_with_stdio(false);
16     cin.tie(nullptr);
17
18     int n;
19     cin >> n;
20     vector<ll> input(n);
21     for (ll &x : input) cin >> x;
22
23     int m;
24     cin >> m;
25     vector<ll> output(m);
26     for (ll &x : output) cin >> x;
27
28     // The sum of all cells never changes after a fold.
29     if (accumulate(input.begin(), input.end(), 0LL) !=
30         accumulate(output.begin(), output.end(), 0LL)) {
31         cout << "N\n";
32         return 0;
33     }
34
35     vector<set<vector<ll>>> states(n + 1);
```

3.38 State-Space BFS - Folding Machine (cont.)

```

36 states[n].insert(normalized(input));
37
38 // Every real fold lowers the tape length. Thus a state only needs
39 // to be generated once, in the set indexed by its length.
40 for (int len = n; len > m; --len) {
41     for (const vector<ll> &tape : states[len]) {
42         for (int h = 1; h <= len / 2; ++h) {
43             // Fold the h cells at the left end over the rest.
44             vector<ll> next;
45             next.reserve(len - h);
46             for (int j = 0; j < h; ++j)
47                 next.push_back(tape[h + j] + tape[h - 1 - j]);
48             for (int j = 2 * h; j < len; ++j)
49                 next.push_back(tape[j]);
50             if (len - h >= m)
51                 states[len - h].insert(normalized(next));
52
53             // Fold the h cells at the right end over the rest.
54             next.clear();
55             const int leftSize = len - h;
56             for (int j = 0; j < leftSize - h; ++j)
57                 next.push_back(tape[j]);
58             for (int j = 0; j < h; ++j)
59                 next.push_back(tape[leftSize - h + j] + tape[len - 1 - j]);
60             if (len - h >= m)
61                 states[len - h].insert(normalized(next));
62         }
63     }
64 }
65
66 cout << (states[m].count(normalized(output)) ? "S\n" : "N\n");
67 }

```

3.39 Strongly Connected Components (Tarjan) - Proving Equivalences

O: (n + m), SCC con Tarjan y contar fuentes y sumideros

Dadas n proposiciones y m implicaciones ya probadas entre ellas

C++/Training_Camp_2026/Day 4/O - Proving Equivalences.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 20];
5 static int bufLen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0;
8         if (!bufLen_) return -1; }
9     return buf_[bufPos_++];
10 }
11
12 inline int readInt() {
13     int c = gc();
14     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
15     bool neg = false; if (c == '-') { neg = true; c = gc(); }
16     int x = 0;
17     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
18     return neg ? -x : x;
19 }
20
21 int n, m;
22 vector<vector<int>> adj;
23 vector<int> disc_, low_, comp_;
24 vector<char> onStack_;
25 vector<int> tarjanStack;
26 int timer_, sccCount;
27
28 // Iterative Tarjan's SCC algorithm.
29 void tarjanSCC() {
30     disc_.assign(n + 1, -1);
31     low_.assign(n + 1, -1);
32     comp_.assign(n + 1, -1);
33     onStack_.assign(n + 1, 0);
34     tarjanStack.clear();

```

3.39 Strongly Connected Components (Tarjan) - Proving Equivalences (cont.)

```

33 timer_ = 0;
34 sccCount = 0;
35
36 for (int s = 1; s <= n; s++) {
37     if (disc_[s] != -1) continue;
38     vector<array<int, 2>> stk; // {vertex, nextIdx}
39     stk.push_back({s, 0});
40     disc_[s] = low_[s] = timer_++;
41     tarjanStack.push_back(s);
42     onStack_[s] = 1;
43
44     while (!stk.empty()) {
45         auto &fr = stk.back();
46         int u = fr[0];
47         if (fr[1] < (int)adj[u].size()) {
48             int v = adj[u][fr[1]];
49             fr[1]++;
50             if (disc_[v] == -1) {
51                 disc_[v] = low_[v] = timer_++;
52                 tarjanStack.push_back(v);
53                 onStack_[v] = 1;
54                 stk.push_back({v, 0});
55             } else if (onStack_[v]) {
56                 low_[u] = min(low_[u], disc_[v]);
57             } else {
58                 stk.pop_back();
59                 if (!stk.empty()) {
60                     int p = stk.back()[0];
61                     low_[p] = min(low_[p], low_[u]);
62                 }
63             }
64             if (low_[u] == disc_[u]) {
65                 // pop the SCC
66                 while (true) {
67                     int w = tarjanStack.back();
68                     tarjanStack.pop_back();
69                     onStack_[w] = 0;
70                     comp_[w] = sccCount;
71                     if (w == u) break;
72                 }
73                 sccCount++;
74             }
75         }
76     }
77 }
78
79 int main() {
80     int T = readInt();
81     string out;
82     while (T--) {
83         n = readInt(); m = readInt();
84         adj.assign(n + 1, {});
85         vector<pair<int, int>> rawEdges(m);
86         for (int i = 0; i < m; i++) {
87             int a = readInt(), b = readInt();
88             rawEdges[i] = {a, b};
89             adj[a].push_back(b);
90         }
91
92         tarjanSCC();
93
94         vector<char> hasIn(sccCount, 0), hasOut(sccCount, 0);
95         for (auto &e : rawEdges) {
96             int ca = comp_[e.first], cb = comp_[e.second];
97             if (ca != cb) {
98                 hasOut[ca] = 1;
99                 hasIn[cb] = 1;
100             }
101         }
102     }
103
104     int ans;

```

3.39 Strongly Connected Components (Tarjan) - Proving Equivalences (cont.)

```

05     if (sccCount <= 1) {
06         ans = 0;
07     } else {
08         int sources = 0, sinks = 0;
09         for (int i = 0; i < sccCount; i++) {
10             if (!hasIn[i]) sources++;
11             if (!hasOut[i]) sinks++;
12         }
13         ans = max(sources, sinks);
14     }
15     out += to_string(ans);
16     out += "\n";
17 }
18 fwrite(out.data(), 1, out.size(), stdout);
19 return 0;
20 }

```

3.40 Tree Degree Analysis - Colmillos

O: (n), grados del árbol

Calcula cuántos pares de nodos pueden removerse de un árbol para desconectarlo

C++/Codeforces/colmillos.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9     cin >> n;
10
11     vector<int> deg(n + 1, 0);
12
13     for (int i = 0; i < n - 1; i++) {
14         int a, b;
15         cin >> a >> b;
16
17         deg[a]++;
18         deg[b]++;
19     }
20
21     int mx1 = 0, mx2 = 0;
22
23     for (int i = 1; i <= n; i++) {
24         if (deg[i] >= mx1) {
25             mx2 = mx1;
26             mx1 = deg[i];
27         } else if (deg[i] > mx2) {
28             mx2 = deg[i];
29         }
30     }
31
32     cout << n - mx1 - mx2 << '\n';
33
34     return 0;
35 }

```

3.41 Tree Reconstruction (BFS por capas) - Expansion Of The Road Network

O: (n + m), BFS por capas

Reconstruir el árbol original de un grafo al que le agregaron atajos: BFS por capas

C++/Training_Camp_2026/Day 6/E - Expansion of the road network.py

```

1 import sys
2 from collections import deque
3
4 def solve(data):
5     n = int(data[0]); m = int(data[1])
6     B = n + 1
7     adj = [[] for _ in range(B)]
8     eset = set()

```

3.41 Tree Reconstruction (BFS por capas) - Expansion Of The Road Network (cont.)

```

9     ptr = 2
10    for _ in range(m):
11        a = int(data[ptr]); b = int(data[ptr + 1]); ptr += 2
12        adj[a].append(b); adj[b].append(a)
13        eset.add(a * B + b if a < b else b * B + a)
14
15    # Grafo completo <=> el árbol original era una estrella
16    if m == n * (n - 1) // 2:
17        return '\n'.join('1 %d' % i for i in range(2, n + 1))
18
19    def bfs(src):
20        dist = [-1] * B
21        dist[src] = 0
22        q = deque([src])
23        while q:
24            u = q.popleft()
25            du = dist[u] + 1
26            for v in adj[u]:
27                if dist[v] < 0:
28                    dist[v] = du
29                    q.append(v)
30        return dist
31
32    # 1) Hoja garantizada del árbol original:
33    # el de grado mínimo dentro de la última capa BFS
34    d1 = bfs(1)
35    maxd = max(d1[1:])
36    leaf = -1; best = float('inf')
37    for v in range(1, n + 1):
38        if d1[v] == maxd and len(adj[v]) < best:
39            best = len(adj[v]); leaf = v
40
41    # 2) Capas BFS desde la hoja: la capa k contiene las profundidades
42    # 2k-2 y 2k-1 del árbol enraizado en el soporte de la hoja
43    layer = bfs(leaf)
44    inK = bytearray(B)
45    for v in adj[leaf]:
46        inK[v] = 1
47
48    # 3) Candidatos a soporte: todo vértice externo con exactamente 2 vecinos
49    # dentro de N(hoja) es "nieto" del soporte y esos 2 vecinos son
50    # {soporte, su padre}. Intersección de esos pares => <= 2 candidatos.
51    cand = None
52    for z in range(1, n + 1):
53        if layer[z] >= 2:
54            c = 0; p1 = 0; p2 = 0
55            for y in adj[z]:
56                if inK[y]:
57                    c += 1
58                    if c == 1: p1 = y
59                    elif c == 2: p2 = y
60                    else: break
61            if c == 2:
62                if cand is None:
63                    cand = {p1, p2}
64            else:
65                cand.intersection_update((p1, p2))
66            if not cand:
67                return '*'
68
69    if not cand:
70        return '*'
71
72    maxl = max(layer[1:])
73    layers = [[] for _ in range(maxl + 1)]
74    for v in range(1, n + 1):
75        layers[layer[v]].append(v)
76
77    def build(w):
78        parent = [0] * B
79        role = bytearray(B) # 1 = prof. par ("arriba"), 2 = prof. impar ("abajo")
80        role[w] = 1
81        parent[leaf] = w

```

3.41 Tree Reconstruction (BFS por capas) - Expansion Of The Road Network (cont.)

```

81     for v in layers[1]:
82         if v != w:
83             role[v] = 2
84             parent[v] = w
85     for k in range(2, maxl + 1):
86         lows = []
87         km1 = k - 1
88         for x in layers[k]:
89             c = 0; a = 0; b = 0
90             for y in adj[x]:
91                 if layer[y] == km1:
92                     c += 1
93                     if c == 1: a = y
94                     elif c == 2: b = y
95                     else: return None
96             if c == 2:
97                 ra = role[a]
98                 if ra == role[b]: return None
99                 if ra == 1: a, b = b, a
100                 if parent[a] != b: return None # el "arriba" debe ser el abuelo
101                 role[x] = 1
102                 parent[x] = a
103             elif c == 1:
104                 if role[a] != 2: return None # unico vecino previo = abuelo
105                 role[x] = 2
106                 lows.append((x, a))
107             else:
108                 return None
109     for x, g in lows:
110         p = 0
111         for y in adj[x]:
112             if layer[y] == k and role[y] == 1 and parent[y] == g:
113                 p = y; break
114         if not p: return None
115         parent[x] = p
116     return parent
117
118 def verify(parent, root):
119     degT = [0] * B
120     for v in range(1, n + 1):
121         if v != root:
122             degT[v] += 1
123             degT[parent[v]] += 1
124     extra = m - (n - 1)
125     s = 0
126     for v in range(1, n + 1):
127         d = degT[v]
128         s += d * (d - 1) // 2
129         if s > extra: return False
130     if s != extra: return False
131     ch = [[] for _ in range(B)]
132     for v in range(1, n + 1):
133         if v != root:
134             p = parent[v]
135             ch[p].append(v)
136             key = v * B + p if v < p else p * B + v
137             if key not in eset: return False
138     for v in range(1, n + 1):
139         nb = ch[v]
140         if v != root:
141             nb = nb + [parent[v]]
142         ln = len(nb)
143         for i in range(ln - 1):
144             x = nb[i]
145             for j in range(i + 1, ln):
146                 y = nb[j]
147                 key = x * B + y if x < y else y * B + x
148                 if key not in eset: return False
149     return True
150
151 for w in cand:
152     parent = build(w)

```

3.41 Tree Reconstruction (BFS por capas) - Expansion Of The Road Network (cont.)

```

53     if parent is not None and verify(parent, w):
54         out = []
55         ap = out.append
56         for v in range(1, n + 1):
57             if v != w:
58                 ap('%d %d' % (v, parent[v]))
59         return '\n'.join(out)
60     return '*'
61
62 def main():
63     data = sys.stdin.buffer.read().split()
64     sys.stdout.write(solve(data) + '\n')
65
66 if __name__ == '__main__':
67     main()

```

4 - Geometry

4.1 2D Difference Array (Coordinate Compression) - Office Space

O: (n^2) sobre las coordenadas comprimidas, arreglo de diferencias 2D

En una oficina rectangular de w x h pies

C++/Training_Camp_2026/Day 11 - CCPL/K - Office Space.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 static char buf_[1 << 22];
6 static int bufLen_, bufPos_;
7 inline int gc() {
8     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0;
9         if (!bufLen_) return -1; }
10    return buf_[bufPos_++];
11 }
12
13 inline bool readInt(int &out) {
14     int c = gc();
15     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
16     if (c == -1) return false;
17     bool neg = false; if (c == '-') { neg = true; c = gc(); }
18     int x = 0;
19     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
20     out = neg ? -x : x;
21     return true;
22 }
23
24 inline bool isSpaceChar(int c) { return c == ' ' || c == '\n' || c == '\r' || c == '\t'; }
25 inline bool readWord(string &s) {
26     s.clear();
27     int c = gc();
28     while (c != -1 && isSpaceChar(c)) c = gc();
29     if (c == -1) return false;
30     while (c != -1 && !isSpaceChar(c)) { s.push_back((char)c); c = gc(); }
31     return true;
32 }
33
34 int main() {
35     int W, H;
36     string out;
37     while (readInt(W)) {
38         readInt(H);
39         int n;
40         readInt(n);
41
42         vector<string> names(n);
43         vector<int> X1(n), Y1(n), X2(n), Y2(n);
44         vector<int> xs, ys;
45         xs.reserve(2 * n + 2);
46         ys.reserve(2 * n + 2);
47         xs.push_back(0); xs.push_back(W);
48         ys.push_back(0); ys.push_back(H);
49
50         for (int i = 0; i < n; i++) {

```

4.1 2D Difference Array (Coordinate Compression) - Office Space (cont.)

```

48     readWord(names[i]);
49     readInt(X1[i]); readInt(Y1[i]); readInt(X2[i]); readInt(Y2[i]);
50     xs.push_back(X1[i]); xs.push_back(X2[i]);
51     ys.push_back(Y1[i]); ys.push_back(Y2[i]);
52 }
53
54 sort(xs.begin(), xs.end());
55 xs.erase(unique(xs.begin(), xs.end()), xs.end());
56 sort(ys.begin(), ys.end());
57 ys.erase(unique(ys.begin(), ys.end()), ys.end());
58
59 int M = (int)xs.size() - 1; // number of cell columns
60 int K = (int)ys.size() - 1; // number of cell rows
61 int W2 = K + 1;           // row width for flat indexing over the (M+1) x (K+1)
62 corner grid
63
64     auto xIdx = [&](int v) { return (int)(lower_bound(xs.begin(), xs.end(), v) - xs.be
65 gin()); };
66     auto yIdx = [&](int v) { return (int)(lower_bound(ys.begin(), ys.end(), v) - ys.be
67 gin()); };
68     auto at = [&](int i, int j) { return i * W2 + j; };
69
70 // Flat 2D difference array (row-major) over the (M+1) x (K+1) grid of "corner"
71 // points, computed in place into the count grid (avoids a second full grid, and
72 // flat 1D storage avoids the per-row bookkeeping overhead of nested vectors).
73 vector<int> grid((size_t)(M + 1) * (K + 1), 0);
74 // Parallel difference array summing (employee_index + 1) instead of 1. When a
75 // cell's final count is exactly 1, this sum unambiguously equals that single
76 // covering employee's (index + 1), letting us identify the owner without a
77 // separate large prefix-sum array.
78 vector<ll> sumIdx((size_t)(M + 1) * (K + 1), 0);
79
80 for (int i = 0; i < n; i++) {
81     int a = xIdx(X1[i]), b = xIdx(X2[i]);
82     int c = yIdx(Y1[i]), d = yIdx(Y2[i]);
83     grid[at(a, c)] += 1;
84     grid[at(b, c)] -= 1;
85     grid[at(a, d)] -= 1;
86     grid[at(b, d)] += 1;
87
88     ll idPlus1 = (ll)i + 1;
89     sumIdx[at(a, c)] += idPlus1;
90     sumIdx[at(b, c)] -= idPlus1;
91     sumIdx[at(a, d)] -= idPlus1;
92     sumIdx[at(b, d)] += idPlus1;
93 }
94
95 // 2D prefix sum in place -> grid[.] becomes the count of rectangles covering each
96 // cell, and sumIdx[.] becomes the sum of (index+1) of covering rectangles.
97 for (int i = 0; i <= M; i++) {
98     int rowBase = i * W2;
99     int prevRowBase = rowBase - W2;
100     for (int j = 0; j <= K; j++) {
101         int idx = rowBase + j;
102         if (i > 0) { grid[idx] += grid[prevRowBase + j]; sumIdx[idx] += sumIdx[pre
103 vRowBase + j]; }
104         if (j > 0) { grid[idx] += grid[idx - 1]; sumIdx[idx] += sumIdx[idx - 1]; }
105         if (i > 0 && j > 0) { grid[idx] -= grid[prevRowBase + j - 1]; sumIdx[idx]
106 -= sumIdx[prevRowBase + j - 1]; }
107     }
108 }
109
110 ll total = (ll)W * H;
111 ll unalloc = 0, contested = 0;
112 vector<ll> guaranteed(n, 0);
113
114 for (int i = 0; i < M; i++) {
115     ll colWidth = (ll)(xs[i + 1] - xs[i]);
116     int rowBase = i * W2;
117     for (int j = 0; j < K; j++) {
118         ll cellArea = colWidth * (ll)(ys[j + 1] - ys[j]);
119         int c = grid[rowBase + j];

```

4.1 2D Difference Array (Coordinate Compression) - Office Space (cont.)

```

115         if (c == 0) {
116             unalloc += cellArea;
117         } else if (c >= 2) {
118             contested += cellArea;
119         } else { // c == 1
120             int owner = (int)(sumIdx[rowBase + j] - 1);
121             guaranteed[owner] += cellArea;
122         }
123     }
124 }
125
126 out += "Total " + to_string(total) + "\n";
127 out += "Unallocated " + to_string(unalloc) + "\n";
128 out += "Contested " + to_string(contested) + "\n";
129
130 for (int i = 0; i < n; i++) {
131     out += names[i] + " " + to_string(guaranteed[i]) + "\n";
132 }
133 out += "\n";
134 }
135
136 fwrite(out.data(), 1, out.size(), stdout);
137 return 0;
138 }

```

4.2 Area Circulo-Poligono

O: (n) por círculo, con n = lados del poligono

Uso: circleRectangleArea(cx,cy,r,w,h); triangleCircleArea(p,q,r)

Area de la interseccion entre un círculo y un poligono

C++/Notebook/Area Circulo-Poligono.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const double EPS = 1e-12;
5
6 struct Pt { double x, y; };
7
8 double cross(Pt a, Pt b) { return a.x * b.y - a.y * b.x; }
9 double dot(Pt a, Pt b) { return a.x * b.x + a.y * b.y; }
10 Pt sub(Pt a, Pt b) { return {a.x - b.x, a.y - b.y}; }
11 Pt add(Pt a, Pt b) { return {a.x + b.x, a.y + b.y}; }
12 Pt mul(Pt a, double t) { return {a.x * t, a.y * t}; }
13 double norm2(Pt a) { return dot(a, a); }
14
15 // Area con signo de la interseccion entre el círculo centrado en (0,0) de radio r
16 // y el triangulo (0,0)-p-q.
17 double triangleCircleArea(Pt p, Pt q, double r) {
18     Pt d = sub(q, p);
19
20     // |p + t*d|^2 = r^2 -> A t^2 + B t + C = 0
21     double A = norm2(d);
22     double B = 2.0 * dot(p, d);
23     double C = norm2(p) - r * r;
24
25     vector<double> cuts = {0.0, 1.0};
26
27     if (A > EPS) {
28         double disc = B * B - 4.0 * A * C;
29         if (disc > EPS) {
30             double s = sqrt(disc);
31             double t1 = (-B - s) / (2.0 * A);
32             double t2 = (-B + s) / (2.0 * A);
33             if (t1 > EPS && t1 < 1.0 - EPS) cuts.push_back(t1);
34             if (t2 > EPS && t2 < 1.0 - EPS) cuts.push_back(t2);
35         }
36     }
37
38     sort(cuts.begin(), cuts.end());
39
40     double ans = 0.0;

```

4.2 Area Circulo-Poligono (cont.)

```

41     for (size_t i = 0; i + 1 < cuts.size(); i++) {
42         double l = cuts[i], rr = cuts[i + 1];
43         double mid = (l + rr) / 2.0;
44
45         Pt a = add(p, mul(d, l));
46         Pt b = add(p, mul(d, rr));
47         Pt m = add(p, mul(d, mid));
48
49         if (norm2(m) <= r * r) {
50             ans += cross(a, b) / 2.0;           // tramo dentro: triangulo
51         } else {
52             ans += r * r * atan2(cross(a, b), dot(a, b)) / 2.0; // tramo fuera: sector
53         }
54     }
55     return ans;
56 }
57
58 double circleRectangleArea(double cx, double cy, double r, double w, double h) {
59     // Trasladar para que el circulo quede en el origen.
60     Pt rect[4] = {{-cx, -cy}, {w - cx, -cy}, {w - cx, h - cy}, {-cx, h - cy}};
61
62     double area = 0.0;
63     for (int i = 0; i < 4; i++) area += triangleCircleArea(rect[i], rect[(i + 1) % 4], r);
64     return fabs(area);
65 }
66
67 int main() {
68     ios::sync_with_stdio(false);
69     cin.tie(nullptr);
70
71     int n;
72     double r, w, h;
73     cin >> n >> r >> w >> h;
74
75     double ans = 0.0;
76     for (int i = 0; i < n; i++) {
77         double x, y, v;
78         cin >> x >> y >> v;
79         ans += v * circleRectangleArea(x, y, r, w, h) / (w * h);
80     }
81
82     cout << fixed << setprecision(15) << ans << "\n";
83     return 0;
84 }

```

4.3 Bounding Box de una Elipse Rotada - Eclipse

O: (1), formulas de la elipse rotada

NO ES MIO: código de Stiven Correa, del repo del equipo (carpeta RPC/)

RPC/2026-07/E - Eclipse.py

```

1  import math
2
3  x1, y1, x2, y2, a = map(int, input().split())
4
5  # Distancia entre los focos
6  d = ((x2 - x1)**2 + (y2 - y1)**2)**(1/2)
7
8  # Semieje mayor
9  A = a / 2
10
11 # Distancia del centro a cada foco
12 c = d / 2
13
14 # Semieje menor
15 B = math.sqrt(A * A - c * c)
16
17 # Centro
18 cx = (x1 + x2) / 2
19 cy = (y1 + y2) / 2
20
21 # Direccion del eje mayor

```

4.3 Bounding Box de una Elipse Rotada - Eclipse (cont.)

```

22 ux = (x2 - x1) / d
23 uy = (y2 - y1) / d
24
25 # Direccion perpendicular
26 vx = -uy
27 vy = ux
28
29 # Extension en X
30 rx = math.sqrt((A * ux) ** 2 + (B * vx) ** 2)
31
32 # Extension en Y
33 ry = math.sqrt((A * uy) ** 2 + (B * vy) ** 2)
34
35 print(cx - rx, cy - ry, cx + rx, cy + ry)

```

4.4 Contar Cruces de Cuerdas con Fenwick - Lonely Creatures

O: (n log n), cruces de cuerdas contadas como inversiones con Fenwick

Cada criatura define una recta que corta una parabola en dos puntos, o sea una CUERDA de la region

C++/ICPC/Regionals 2025/L - Lonely Creatures.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5  using ld = long double;
6
7  const ld EPS = 1e-12L;
8
9  struct Chord {
10     ld L, R;
11 };
12
13 struct Fenwick {
14     int n;
15     vector<int> bit;
16
17     Fenwick(int n) : n(n), bit(n + 1, 0) {}
18
19     void add(int pos, int val) {
20         for (; pos <= n; pos += pos & -pos)
21             bit[pos] += val;
22     }
23
24     int sum(int pos) {
25         int ans = 0;
26
27         for (; pos > 0; pos -= pos & -pos)
28             ans += bit[pos];
29
30         return ans;
31     }
32 };
33
34 // -----
35 // Primer indice cuyo valor es > x
36 // -----
37 int upperBound(const vector<ld>& a, ld x) {
38     int l = 0;
39     int r = a.size();
40
41     while (l < r) {
42         int m = (l + r) / 2;
43
44         if (a[m] <= x + EPS)
45             l = m + 1;
46         else
47             r = m;
48     }
49
50     return l;
51 }

```

4.4 Contar Cruces de Cuerdas con Fenwick - Lonely Creatures (cont.)

```

52
53 // -----
54 // Primer indice cuyo valor es >= x
55 // -----
56 int lowerBound(const vector<ld>& a, ld x) {
57     int l = 0;
58     int r = a.size();
59
60     while (l < r) {
61         int m = (l + r) / 2;
62
63         if (a[m] < x - EPS)
64             l = m + 1;
65         else
66             r = m;
67     }
68
69     return l;
70 }
71
72 int main() {
73     ios::sync_with_stdio(false);
74     cin.tie(nullptr);
75
76     int N;
77     ll A, B;
78
79     cin >> N >> A >> B;
80
81     vector<Chord> chords;
82
83     for (int i = 0; i < N; i++) {
84
85         ll M, C;
86         cin >> M >> C;
87
88         /*
89           $Mx + C = A \cdot x^2 + B$ 
90           $A \cdot x^2 - M \cdot x + (B - C) = 0$ 
91           $D = M^2 - 4A(B - C)$ 
92          */
93
94         ll D = M * M - 4LL * A * (B - C);
95
96         // No entra al interior.
97         if (D <= 0)
98             continue;
99
100         ld sq = sqrtl((ld)D);
101
102         ld L = ((ld)M - sq) / (2.0L * A);
103         ld R = ((ld)M + sq) / (2.0L * A);
104
105         chords.push_back({L, R});
106     }
107
108     if (chords.size() < 2) {
109         cout << 0 << '\n';
110         return 0;
111     }
112
113     // -----
114     // Ordenar por L.
115     //
116     // Si L es igual, NO pueden cruzarse dentro:
117     // se encuentran sobre la frontera.
118     // -----
119
120     sort(chords.begin(), chords.end(), [](const Chord& a,
121                                           const Chord& b) {

```

4.4 Contar Cruces de Cuerdas con Fenwick - Lonely Creatures (cont.)

```

122         if (fabsl(a.L - b.L) > EPS)
123             return a.L < b.L;
124     });
125
126     // -----
127     // Compresion de R
128     // -----
129
130     vector<ld> vals;
131
132     for (auto &c : chords)
133         vals.push_back(c.R);
134
135     sort(vals.begin(), vals.end());
136
137     vals.erase(
138         unique(vals.begin(), vals.end(),
139               [](ld a, ld b) {
140                 return fabsl(a - b) <= EPS;
141             }),
142         vals.end()
143     );
144
145     Fenwick fw(vals.size());
146
147     ll ans = 0;
148
149     int i = 0;
150     int processed = 0;
151
152     while (i < (int)chords.size()) {
153
154         int j = i + 1;
155
156         while (j < (int)chords.size() &&
157               fabsl(chords[j].L - chords[i].L) <= EPS) {
158             j++;
159         }
160
161         /*
162          Todas las cuerdas anteriores tienen:
163
164          L_anterior < L_actual
165
166          porque estamos procesando un grupo completo
167          de L iguales.
168
169          Queremos:
170
171          L_actual < R_anterior < R_actual
172
173          Entonces:
174
175          count(R <= L_actual)
176          -
177          count(R >= R_actual)
178
179          equivalente a:
180
181          # R en (L_actual, R_actual)
182          */
183
184         for (int k = i; k < j; k++) {
185
186             ld L = chords[k].L;
187             ld R = chords[k].R;
188
189             // cantidad de R <= L
190             int p1 = upperBound(vals, L);

```

4.4 Contar Cruces de Cuerdas con Fenwick - Lonely Creatures (cont.)

```

86         int left = fw.sum(p1);
87
88         // cantidad de R < R
89         int p2 = lowerBound(vals, R);
90         int beforeR = fw.sum(p2);
91
92         /*
93          Queremos estrictamente:
94
95          L < R_previous < R
96          */
97         ans += beforeR - left;
98     }
99
100    // Ahora agregamos las cuerdas de este grupo.
101    for (int k = i; k < j; k++) {
102
103        int pos = lowerBound(vals, chords[k].R) + 1;
104
105        fw.add(pos, 1);
106        processed++;
107    }
108
109    i = j;
110 }
111
112 cout << ans << '\n';
113
114 return 0;
115 }

```

4.5 Convex Hull y Barrido Angular - Guard Deployment

O: (H^2) por el barrido angular; el convex hull es ($H \log H$)
 Perímetro del convex hull de los edificios y altura mínima para que las torres se vean

C++/IIPC/2025/G - Guard Deployment.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ld = long double;
5
6 const ld PI = acosl(-1.0L);
7 const ld EPS = 1e-12L;
8
9 struct Point {
10     ld x, y;
11
12     bool operator<(const Point& p) const {
13         if (fabsl(x - p.x) > EPS)
14             return x < p.x;
15         return y < p.y;
16     }
17
18     bool operator==(const Point& p) const {
19         return fabsl(x - p.x) < EPS &&
20             fabsl(y - p.y) < EPS;
21     }
22 };
23
24 struct Building {
25     Point p[4];
26     int h;
27 };
28
29 ld cross(Point a, Point b, Point c) {
30     return (b.x - a.x) * (c.y - a.y)
31         - (b.y - a.y) * (c.x - a.x);
32 }
33
34

```

4.5 Convex Hull y Barrido Angular - Guard Deployment (cont.)

```

35 // -----
36 // CONVEX HULL
37 // -----
38
39 vector<Point> convexHull(vector<Point> p) {
40
41     sort(p.begin(), p.end());
42
43     p.erase(unique(p.begin(), p.end(), p.end()));
44
45     int n = p.size();
46
47     if (n <= 1)
48         return p;
49
50     vector<Point> h(2 * n);
51     int k = 0;
52
53     // Lower
54     for (int i = 0; i < n; i++) {
55
56         while (k >= 2 &&
57             cross(h[k - 2], h[k - 1], p[i]) <= EPS)
58             k--;
59
60         h[k++] = p[i];
61     }
62
63     // Upper
64     for (int i = n - 2, t = k + 1; i >= 0; i--) {
65
66         while (k >= t &&
67             cross(h[k - 2], h[k - 1], p[i]) <= EPS)
68             k--;
69
70         h[k++] = p[i];
71     }
72
73     h.resize(k - 1);
74
75     return h;
76 }
77
78 // -----
79 // DISTANCIA
80 // -----
81
82 ld dist(Point a, Point b) {
83     ld dx = a.x - b.x;
84     ld dy = a.y - b.y;
85
86     return sqrtl(dx * dx + dy * dy);
87 }
88
89
90 // -----
91 // PUNTO SOBRE RECTANGULO
92 // Como los edificios son convexos, basta revisar sus 4 lados.
93 // -----
94
95 bool pointOnSegment(Point a, Point b, Point p) {
96
97     if (fabsl(cross(a, b, p)) > EPS)
98         return false;
99
100     return p.x >= min(a.x, b.x) - EPS &&
101         p.x <= max(a.x, b.x) + EPS &&
102         p.y >= min(a.y, b.y) - EPS &&
103         p.y <= max(a.y, b.y) + EPS;
104 }
105
106

```

4.5 Convex Hull y Barrido Angular - Guard Deployment (cont.)

```

07 bool pointInBuilding(const Building& b, Point p) {
08     for (int i = 0; i < 4; i++) {
09         Point a = b.p[i];
10         Point c = b.p[(i + 1) % 4];
11         if (pointOnSegment(a, c, p))
12             return true;
13     }
14     bool pos = false;
15     bool neg = false;
16     for (int i = 0; i < 4; i++) {
17         ld cr = cross(
18             b.p[i],
19             b.p[(i + 1) % 4],
20             p
21         );
22         if (cr > EPS) pos = true;
23         if (cr < -EPS) neg = true;
24     }
25     return !(pos && neg);
26 }
27 // -----
28 // ANGULO
29 // -----
30 ld angle(Point a, Point b) {
31     ld x = b.x - a.x;
32     ld y = b.y - a.y;
33     ld ang = atan2l(y, x);
34     if (ang < 0)
35         ang += 2 * PI;
36     return ang;
37 }
38 // -----
39 // EL EDIFICIO ES ATRAVESADO POR ALGUN SEGMENTO
40 // ENTRE DOS VERTICES DEL HULL?
41 // Para un vertice v del hull:
42 // todas las direcciones que desde v alcanzan el rectangulo
43 // forman un intervalo angular.
44 // Los cuatro corners del rectangulo determinan ese intervalo.
45 // Si existe otro vertice del hull dentro de ese intervalo,
46 // el segmento entre ambos atraviesa/toca el rectangulo.
47 // -----
48 bool buildingBlocks(
49     const vector<Point>& hull,
50     const Building& b,
51     const vector<vector<ld>>& hullAngles
52 ) {
53     int H = hull.size();
54     for (int v = 0; v < H; v++) {

```

4.5 Convex Hull y Barrido Angular - Guard Deployment (cont.)

```

78     // Si el vertice del hull pertenece al edificio,
79     // claramente este edificio participa.
80     if (pointInBuilding(b, hull[v]))
81         return true;
82     // Angulos hacia las 4 esquinas del edificio
83     vector<ld> a(4);
84     for (int k = 0; k < 4; k++)
85         a[k] = angle(hull[v], b.p[k]);
86     sort(a.begin(), a.end());
87     // Buscamos el mayor hueco circular.
88     // El intervalo complementario es el menor intervalo
89     // que contiene las 4 esquinas.
90     int bestGap = 0;
91     ld maxGap = -1;
92     for (int k = 0; k < 4; k++) {
93         ld next = (k == 3 ? a[0] + 2 * PI : a[k + 1]);
94         ld gap = next - a[k];
95         if (gap > maxGap) {
96             maxGap = gap;
97             bestGap = k;
98         }
99     }
100     ld L = a[(bestGap + 1) % 4];
101     ld R = a[bestGap];
102     if (R < L)
103         R += 2 * PI;
104     const vector<ld>& angles = hullAngles[v];
105     // Tenemos los angulos de los demas vertices.
106     // Probamos tambien una copia desplazada 2PI para
107     // resolver el caso en que el intervalo cruza 0.
108     auto check = [&](ld x) {
109         return x + EPS >= L && x <= R + EPS;
110     };
111     auto it = lower_bound(
112         angles.begin(),
113         angles.end(),
114         L - EPS
115     );
116     if (it != angles.end() && check(*it))
117         return true;
118     // Si L > 2PI, usamos los angulos originales + 2PI.
119     if (L > 2 * PI - EPS) {
120         ld L2 = L - 2 * PI;
121         ld R2 = R - 2 * PI;
122         auto it2 = lower_bound(
123             angles.begin(),
124             angles.end(),
125             L2 - EPS
126         );
127         if (it2 != angles.end() &&
128             *it2 <= R2 + EPS)
129             return true;
130     }
131 }

```

4.5 Convex Hull y Barrido Angular - Guard Deployment (cont.)

```

251
252     return false;
253 }
254
255 // -----
256 // MAIN
257 // -----
258
259 int main() {
260
261     ios::sync_with_stdio(false);
262     cin.tie(nullptr);
263
264     int T;
265     cin >> T;
266
267     while (T--) {
268
269         int N;
270         cin >> N;
271
272         vector<Building> buildings(N);
273
274         vector<Point> allPoints;
275
276         for (int i = 0; i < N; i++) {
277             for (int j = 0; j < 4; j++) {
278                 cin >> buildings[i].p[j].x
279                     >> buildings[i].p[j].y;
280
281                 allPoints.push_back(buildings[i].p[j]);
282             }
283
284             cin >> buildings[i].h;
285         }
286
287         // -----
288         // 1. Convex Hull
289         // -----
290
291         vector<Point> hull = convexHull(allPoints);
292
293         int H = hull.size();
294
295         // -----
296         // 2. Perimetro
297         // -----
298
299         ld perimeter = 0;
300
301         for (int i = 0; i < H; i++)
302             perimeter += dist(
303                 hull[i],
304                 hull[(i + 1) % H]
305             );
306
307         // -----
308         // 3. Angulos de los vertices del hull
309         // -----
310         // Para cada vertice guardamos los angulos hacia
311         // todos los demas vertices.
312         // -----
313
314         vector<vector<ld>> hullAngles(H);
315
316         for (int i = 0; i < H; i++) {
317             for (int j = 0; j < H; j++) {

```

4.5 Convex Hull y Barrido Angular - Guard Deployment (cont.)

```

323             if (i == j)
324                 continue;
325
326             hullAngles[i].push_back(
327                 angle(hull[i], hull[j])
328             );
329         }
330
331         sort(
332             hullAngles[i].begin(),
333             hullAngles[i].end()
334         );
335     }
336
337     // -----
338     // 4. Altura minima
339     // -----
340
341     int minHeight = 3;
342
343     for (int i = 0; i < N; i++) {
344         if (buildingBlocks(
345             hull,
346             buildings[i],
347             hullAngles
348         )) {
349             minHeight = max(
350                 minHeight,
351                 buildings[i].h + 1
352             );
353         }
354     }
355
356     cout << fixed << setprecision(6)
357         << (double)perimeter << ' '
358         << minHeight << '\n';
359 }
360
361 return 0;
362 }

```

4.6 Coordenadas Polares

O: (1) cada conversion, ($n \log n$) ordenar por angulo

Uso: aPolar(x,y) -> {r,th}; normalizar(th) deja th en $[0, 2\pi]$

Pasar de (x,y) a (r, theta) y al contrario

C++/Notebook/Coordenadas Polares.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const double PI = acos(-1.0);
5
6 // ----- conversion -----
7 struct Polar { double r, th; };
8
9 Polar aPolar(double x, double y) {
10     return {hypot(x, y), atan2(y, x)}; // th queda en (-pi, pi]
11 }
12
13 pair<double, double> aCartesiano(double r, double th) {
14     return {r * cos(th), r * sin(th)};
15 }
16
17 // Lleva cualquier angulo al rango [0, 2pi).
18 double normalizar(double th) {
19     th = fmod(th, 2 * PI);
20     if (th < 0) th += 2 * PI;
21     return th;
22 }

```

4.6 Coordenadas Polares (cont.)

```

23
24 double grados(double rad) { return rad * 180.0 / PI; }
25 double radianes(double deg) { return deg * PI / 180.0; }
26
27 // ----- barrido angular exacto, sin atan2 -----
28 struct P {
29     long long x, y;
30 };
31
32 long long cross(const P &a, const P &b) { return a.x * b.y - a.y * b.x; }
33
34 // semiplano: 0 = angulo en [0, pi), 1 = en [pi, 2pi). El (0,0) cae en 0.
35 int mitad(const P &p) {
36     if (p.y > 0) return 0;
37     if (p.y < 0) return 1;
38     if (p.x >= 0) return 0;          // sobre el eje x: el +x va en la 1a
39     return 1;                      // mitad y el -x en la 2a
40 }
41
42 // Ordena por angulo en [0, 2pi) arrancando en el +x. Los colineales quedan
43 // juntos y entre ellos ordenados por distancia al origen.
44 bool menorAngulo(const P &a, const P &b) {
45     int ha = mitad(a), hb = mitad(b);
46     if (ha != hb) return ha < hb;
47
48     long long c = cross(a, b);
49     if (c != 0) return c > 0;        // a va antes si b queda a su izq
50
51     return a.x * a.x + a.y * a.y < b.x * b.x + b.y * b.y;
52 }
53
54 int main() {
55     ios::sync_with_stdio(false);
56     cin.tie(nullptr);
57     cout << fixed << setprecision(6);
58
59     double x, y;
60     while (cin >> x >> y) {
61         Polar p = aPolar(x, y);
62         pair<double, double> q = aCartesiano(p.r, p.th);
63
64         cout << "r = " << p.r
65              << " theta = " << p.th
66              << " rad = " << grados(p.th) << " grados"
67              << " [0,2pi) = " << normalizar(p.th)
68              << " vuelta = (" << q.first << ", " << q.second << ")\\n";
69     }
70     return 0;
71 }

```

4.7 Geometria 1D - Kilometric Intersection

O: (1), interseccion de dos intervalos

Largo de la interseccion de dos intervalos de la recta, 0 si solo se tocan en un punto

C++/Training_Camp_2026/Day 9/J - Kilometric Intersection.py

```

1 import sys
2
3 n = int(input())
4
5 for _ in range(n):
6     a, b, c, d = map(int, input().split())
7
8     l = max(a, c)
9     r = min(b, d)
10
11     print(max(0, r - l))

```

4.8 Intersecciones, Distancias y Par Mas Cercano - Intersecciones Y Distancias

O: (1) cada primitiva, (n) inPolygon, (n log n) closestPair

Uso: inPolygon(P,q): 1 dentro, 0 borde, -1 fuera; circleInter da 0, 1 o 2 puntos

El PUNTO de corte de rectas, segmentos y circulos; distancias; punto en poligono

C++/Notebook/Intersecciones y Distancias.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 const double EPS = 1e-9;
6
7 int sgn(double x) { return (x > EPS) - (x < -EPS); }
8
9 struct Pt
10 {
11     double x, y;
12     Pt() {}
13     Pt(double x, double y) : x(x), y(y) {}
14     Pt operator+(const Pt &o) const { return Pt(x + o.x, y + o.y); }
15     Pt operator-(const Pt &o) const { return Pt(x - o.x, y - o.y); }
16     Pt operator*(double k) const { return Pt(x * k, y * k); }
17 };
18
19 double dot(Pt a, Pt b) { return a.x * b.x + a.y * b.y; }
20 double cross(Pt a, Pt b) { return a.x * b.y - a.y * b.x; }
21 double cross(Pt a, Pt b, Pt c) { return cross(b - a, c - a); }
22 double len(Pt a) { return sqrt(dot(a, a)); }
23
24 bool onSeg(Pt a, Pt b, Pt p)
25 {
26     return sgn(cross(a, b, p)) == 0 && sgn(dot(p - a, p - b)) <= 0;
27 }
28
29 // Proyeccion de p sobre la recta ab
30 Pt proyeccion(Pt p, Pt a, Pt b)
31 {
32     Pt d = b - a;
33     return a + d * (dot(p - a, d) / dot(d, d));
34 }
35
36 double distLine(Pt p, Pt a, Pt b) { return fabs(cross(a, b, p)) / len(b - a); }
37
38 double distSeg(Pt p, Pt a, Pt b)
39 {
40     if (sgn(dot(p - a, b - a)) <= 0) return len(p - a);
41     if (sgn(dot(p - b, a - b)) <= 0) return len(p - b);
42     return distLine(p, a, b);
43 }
44
45 // Corte de las RECTAS ab y cd. Precondicion: no paralelas
46 Pt lineInter(Pt a, Pt b, Pt c, Pt d)
47 {
48     Pt r = b - a, s = d - c;
49     return a + r * (cross(c - a, s) / cross(r, s));
50 }
51
52 // Corte de los SEGMENTOS ab y cd en un unico punto
53 bool segInterPoint(Pt a, Pt b, Pt c, Pt d, Pt &out)
54 {
55     if (sgn(cross(b - a, d - c)) == 0) return false; // paralelos o colineales
56     out = lineInter(a, b, c, d);
57     return onSeg(a, b, out) && onSeg(c, d, out);
58 }
59
60 // 1 dentro, 0 en el borde, -1 fuera. Poligono simple, concavo o convexo
61 int inPolygon(const vector<Pt> &P, Pt q)
62 {
63     int n = P.size();
64     bool dentro = false;
65     for (int i = 0; i < n; i++)
66     {

```

4.8 Intersecciones, Distancias y Par Mas Cercano - Intersecciones Y Distancias (cont.)

```

67 Pt a = P[i], b = P[(i + 1) % n];
68 if (onSeg(a, b, q)) return 0;
69 if ((a.y > q.y) != (b.y > q.y)) // la arista cruza la horizontal de q
70 {
71     if ((cross(a, b, q) > 0) == (b.y > a.y)) dentro = !dentro;
72 }
73 }
74 return dentro ? 1 : -1;
75 }
76
77 vector<Pt> circleInter(Pt c1, double r1, Pt c2, double r2)
78 {
79     Pt d = c2 - c1;
80     double D = len(d);
81     if (sgn(D) == 0) return {}; // concentricos
82     if (sgn(D - (r1 + r2)) > 0 || sgn(D - fabs(r1 - r2)) < 0) return {};
83     double a = (r1 * r1 - r2 * r2 + D * D) / (2 * D);
84     Pt m = c1 + d * (a / D);
85     // Tangencia por DISTANCIAS, no con sgn(h): en tangencia r1^2-a^2 da ~1e-14, la
86     // raiz lo sube a ~1e-7 > EPS y salian 2 puntos casi iguales (13 fallos al probar)
87     if (sgn(D - (r1 + r2)) == 0 || sgn(D - fabs(r1 - r2)) == 0) return {m};
88     double h = sqrt(max(0.0, r1 * r1 - a * a));
89     Pt per = Pt(-d.y, d.x) * (h / D);
90     return {m + per, m - per};
91 }
92
93 // Recta ab contra el circulo (c, r)
94 vector<Pt> lineCircle(Pt a, Pt b, Pt c, double r)
95 {
96     Pt p = proyeccion(c, a, b);
97     double d = len(p - c);
98     if (sgn(d - r) > 0) return {};
99     if (sgn(d - r) == 0) return {p}; // tangente (ver circleInter)
100     double h = sqrt(max(0.0, r * r - d * d));
101     Pt dir = (b - a) * (1.0 / len(b - a));
102     return {p - dir * h, p + dir * h};
103 }
104
105 double closestPair(vector<Pt> p)
106 {
107     sort(p.begin(), p.end(), [](const Pt &a, const Pt &b) { return a.x < b.x; });
108     multiset<pair<double, double>> s; // (y, x) de los de la franja
109     double best = 1e18;
110     int j = 0;
111     for (int i = 0; i < (int)p.size(); i++)
112     {
113         while (p[i].x - p[j].x > best)
114         {
115             s.erase(s.find({p[j].y, p[j].x}));
116             j++;
117         }
118         auto it = s.lower_bound({p[i].y - best, -1e18});
119         for (; it != s.end() && it->first <= p[i].y + best; ++it)
120         {
121             best = min(best, len(p[i] - Pt(it->second, it->first)));
122         }
123         s.insert({p[i].y, p[i].x});
124     }
125     return best;
126 }
127
128 int main()
129 {
130     ios_base::sync_with_stdio(false);
131     cin.tie(nullptr);
132     return 0;
133 }

```

4.9 K Vecinos Mutuos y Clasificacion de Componentes - Celestial Veins

O: (n^2) por comparar todos los pares de nodos

Cada estrella mira a sus K vecinas mas cercanas

C++/ICPC/2025/C - Celestial Veins.cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <algorithm>
5
6 using namespace std;
7
8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     long long n, k;
14
15     while (cin >> n >> k && !(n == 0 && k == 0))
16     {
17         vector<long long> coordX(n), coordY(n);
18
19         for (long long i = 0; i < n; i++)
20         {
21             cin >> coordX[i] >> coordY[i];
22         }
23
24         vector<vector<long long>> cercanas(n);
25
26         long long cuantasVecinas = min(k, n - 1);
27
28         for (long long estrella = 0; estrella < n; estrella++)
29         {
30             vector<pair<long long, long long>> candidatas;
31
32             for (long long otra = 0; otra < n; otra++)
33             {
34                 if (otra == estrella)
35                 {
36                     continue;
37                 }
38
39                 long long dx = coordX[estrella] - coordX[otra];
40                 long long dy = coordY[estrella] - coordY[otra];
41
42                 long long distancia2 = dx * dx + dy * dy;
43
44                 candidatas.push_back({distancia2, otra});
45             }
46
47             partial_sort(candidatas.begin(),
48                         candidatas.begin() + cuantasVecinas,
49                         candidatas.end());
50
51             for (long long p = 0; p < cuantasVecinas; p++)
52             {
53                 cercanas[estrella].push_back(candidatas[p].second);
54             }
55         }
56
57         vector<vector<long long>> vinculos(n);
58
59         for (long long a = 0; a < n; a++)
60         {
61             for (long long b : cercanas[a])
62             {
63                 if (b < a)
64                 {
65                     continue;
66                 }
67             }

```

4.9 K Vecinos Mutuos y Clasificación de Componentes - Celestial Veins (cont.)

```

88     bool bTieneA = false;
89
90     for(long long c : cercanas[b])
91     {
92         if(c == a)
93         {
94             bTieneA = true;
95         }
96     }
97
98     if(bTieneA)
99     {
100         vinculos[a].push_back(b);
101         vinculos[b].push_back(a);
102     }
103 }
104
105 long long travelersPath = 0;
106 long long cosmicOuroboros = 0;
107 long long stellarCluster = 0;
108 long long dragonsClaw = 0;
109 long long amorphousNebula = 0;
110
111 vector<bool> visitada(n, false);
112
113 for(long long inicio = 0; inicio < n; inicio++)
114 {
115     if(visitada[inicio])
116     {
117         continue;
118     }
119
120     vector<long long> grupo;
121     queue<long long> cola;
122
123     cola.push(inicio);
124     visitada[inicio] = true;
125
126     while(!cola.empty())
127     {
128         long long actual = cola.front();
129         cola.pop();
130
131         grupo.push_back(actual);
132
133         for(long long vecina : vinculos[actual])
134         {
135             if(!visitada[vecina])
136             {
137                 visitada[vecina] = true;
138                 cola.push(vecina);
139             }
140         }
141     }
142
143     long long estrellas = grupo.size();
144     long long sumaGrados = 0;
145     long long conGrado1 = 0;
146     long long conGrado2 = 0;
147     long long gradoMaximo = 0;
148
149     for(long long estrella : grupo)
150     {
151         long long grado = vinculos[estrella].size();
152
153         sumaGrados += grado;
154         gradoMaximo = max(gradoMaximo, grado);
155
156         if(grado == 1) conGrado1++;
157         if(grado == 2) conGrado2++;
158     }
159 }

```

4.9 K Vecinos Mutuos y Clasificación de Componentes - Celestial Veins (cont.)

```

160
161     long long enlaces = sumaGrados / 2;
162
163     if(estrellas == 1)
164     {
165         amorphousNebula++;
166     }
167     else if(enlaces == estrellas * (estrellas - 1) / 2)
168     {
169         stellarCluster++;
170     }
171     else if(enlaces == estrellas && conGrado2 == estrellas)
172     {
173         cosmicOuroboros++;
174     }
175     else if(enlaces == estrellas - 1 &&
176             gradoMaximo == estrellas - 1 &&
177             conGrado1 == estrellas - 1)
178     {
179         dragonsClaw++;
180     }
181     else if(enlaces == estrellas - 1 && gradoMaximo <= 2)
182     {
183         travelersPath++;
184     }
185     else
186     {
187         amorphousNebula++;
188     }
189 }
190
191 cout << travelersPath << ' '
192 << cosmicOuroboros << ' '
193 << stellarCluster << ' '
194 << dragonsClaw << ' '
195 << amorphousNebula << '\n';
196 }
197
198 return 0;
199 }

```

4.10 K-Means (Clustering)

O: (iteraciones * n * k * d)

Uso: kmeansOnce(p,k,rng,maxIter) -> centroides y etiquetas; correr varias veces

Agrupar n puntos (de cualquier dimension d) en k clusters minimizando el SSE

C++/Notebook/K-Means.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 typedef vector<double> Vec;
6
7 double dist2(const Vec &a, const Vec &b)
8 {
9     double s = 0;
10    for (size_t t = 0; t < a.size(); t++) s += (a[t] - b[t]) * (a[t] - b[t]);
11    return s;
12 }
13
14 struct KMeans
15 {
16     vector<Vec> c; // centroides
17     vector<int> label; // cluster de cada punto
18     double sse; // suma de distancias^2 de cada punto a su centroide
19 };
20
21 // Centroide mas cercano a x (en empate, el de menor indice)
22 int nearest(const Vec &x, const vector<Vec> &c)
23 {
24     int best = 0;

```

4.10 K-Means (Clustering) (cont.)

```

25 double bd = dist2(x, c[0]);
26 for (int j = 1; j < (int)c.size(); j++)
27 {
28     double d = dist2(x, c[j]);
29     if (d < bd)
30     {
31         bd = d;
32         best = j;
33     }
34 }
35 return best;
36 }
37
38 KMeans kmeansOnce(const vector<Vec> &p, int k, mt19937 &rng, int maxIter)
39 {
40     int n = p.size(), d = p[0].size();
41
42     // Semilla k-means++
43     vector<Vec> c(1, p[uniform_int_distribution<int>(0, n - 1)(rng)]);
44     vector<double> md(n, 1e300); // distancia^2 al centroide elegido mas cercano
45     while ((int)c.size() < k)
46     {
47         double total = 0;
48         for (int i = 0; i < n; i++)
49         {
50             md[i] = min(md[i], dist2(p[i], c.back()));
51             total += md[i];
52         }
53         int pick = n - 1;
54         if (total > 0)
55         {
56             double r = uniform_real_distribution<double>(0, total)(rng);
57             for (int i = 0; i < n; i++)
58             {
59                 r -= md[i];
60                 if (r <= 0)
61                 {
62                     pick = i;
63                     break;
64                 }
65             }
66         }
67         else pick = uniform_int_distribution<int>(0, n - 1)(rng); // todos repetidos
68         c.push_back(p[pick]);
69     }
70
71     // Lloyd
72     vector<int> label(n, -1);
73     for (int it = 0; it < maxIter; it++)
74     {
75         bool changed = false;
76         for (int i = 0; i < n; i++)
77         {
78             int j = nearest(p[i], c);
79             if (j != label[i])
80             {
81                 label[i] = j;
82                 changed = true;
83             }
84         }
85         if (!changed) break;
86
87         vector<Vec> sum(k, Vec(d, 0.0));
88         vector<int> cnt(k, 0);
89         for (int i = 0; i < n; i++)
90         {
91             cnt[label[i]]++;
92             for (int t = 0; t < d; t++) sum[label[i]][t] += p[i][t];
93         }
94
95         // Para re-sembrar un cluster vacio: el punto mas lejano a su centroide actual
96         vector<double> err(n);

```

4.10 K-Means (Clustering) (cont.)

```

97     for (int i = 0; i < n; i++) err[i] = dist2(p[i], c[label[i]]);
98
99     for (int j = 0; j < k; j++)
100     {
101         if (cnt[j] > 0)
102         {
103             for (int t = 0; t < d; t++) c[j][t] = sum[j][t] / cnt[j];
104         }
105         else
106         {
107             int far = max_element(err.begin(), err.end()) - err.begin();
108             c[j] = p[far];
109             err[far] = -1; // que otro cluster vacio no tome el mismo punto
110         }
111     }
112 }
113
114 KMeans res;
115 res.c = c;
116 res.label.assign(n, 0);
117 res.sse = 0;
118 for (int i = 0; i < n; i++)
119 {
120     res.label[i] = nearest(p[i], c);
121     res.sse += dist2(p[i], c[res.label[i]]);
122 }
123 return res;
124 }
125
126 // Mejor de `restarts` corridas. Semilla fija: resultado reproducible
127 KMeans kmeans(const vector<Vec> &p, int k, int restarts = 10, int maxIter = 300,
128               unsigned seed = 7)
129 {
130     k = max(1, min(k, (int)p.size()));
131     mt19937 rng(seed);
132     KMeans best;
133     best.sse = numeric_limits<double>::infinity();
134     for (int r = 0; r < restarts; r++)
135     {
136         KMeans cur = kmeansOnce(p, k, rng, maxIter);
137         if (cur.sse < best.sse) best = cur;
138     }
139     return best;
140 }
141
142 int main()
143 {
144     ios_base::sync_with_stdio(false);
145     cin.tie(nullptr);
146
147     int n, k, d;
148     cin >> n >> k >> d;
149     vector<Vec> p(n, Vec(d));
150     for (auto &x : p)
151         for (auto &v : x) cin >> v;
152
153     KMeans r = kmeans(p, k);
154
155     cout << fixed << setprecision(6) << r.sse << "\n";
156     for (auto &cc : r.c)
157     {
158         for (int t = 0; t < d; t++) cout << cc[t] << (t + 1 < d ? ' ' : '\n');
159     }
160     for (int i = 0; i < n; i++) cout << r.label[i] << (i + 1 < n ? ' ' : '\n');
161     return 0;
162 }

```

4.11 Primitivas y Convex Hull

O: (n log n) el hull, (1) las primitivas

Uso: cross(a,b,c)>0 izquierda; convexHull(p) antihorario; polygonArea(P)

Struct Pt con operadores, producto punto y producto cruz

C++/Notebook/Primitivas y Convex Hull.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 const double EPS = 1e-9;
6
7 int sgn(double x)
8 {
9     return (x > EPS) - (x < -EPS);
10 }
11
12 struct Pt
13 {
14     double x, y;
15     Pt() {}
16     Pt(double x, double y) : x(x), y(y) {}
17     Pt operator+(const Pt &o) const { return Pt(x + o.x, y + o.y); }
18     Pt operator-(const Pt &o) const { return Pt(x - o.x, y - o.y); }
19     Pt operator*(double k) const { return Pt(x * k, y * k); }
20 };
21
22 double dot(Pt a, Pt b) { return a.x * b.x + a.y * b.y; }
23 double cross(Pt a, Pt b) { return a.x * b.y - a.y * b.x; }
24 double cross(Pt a, Pt b, Pt c) { return cross(b - a, c - a); }
25
26 // p esta sobre el segmento a-b (asumiendo colinealidad)
27 bool onSeg(Pt a, Pt b, Pt p)
28 {
29     return sgn(cross(a, b, p)) == 0 && sgn(dot(p - a, p - b)) <= 0;
30 }
31
32 bool segIntersect(Pt a, Pt b, Pt c, Pt d)
33 {
34     int o1 = sgn(cross(a, b, c)), o2 = sgn(cross(a, b, d));
35     int o3 = sgn(cross(c, d, a)), o4 = sgn(cross(c, d, b));
36     if (o1 * o2 < 0 && o3 * o4 < 0) return true;
37     return onSeg(a, b, c) || onSeg(a, b, d) || onSeg(c, d, a) || onSeg(c, d, b);
38 }
39
40 // Monotone chain: devuelve el casco convexo en sentido antihorario
41 vector<Pt> convexHull(vector<Pt> p)
42 {
43     sort(p.begin(), p.end(), [](const Pt &a, const Pt &b){
44         if (a.x == b.x) return a.y < b.y;
45         return a.x < b.x;
46     });
47
48     vector<Pt> lo, up;
49     for (auto &pt : p)
50     {
51         while (lo.size() >= 2 && sgn(cross(lo[lo.size() - 2], lo.back(), pt)) <= 0)
52             lo.pop_back();
53         lo.push_back(pt);
54     }
55     for (int i = (int)p.size() - 1; i >= 0; --i)
56     {
57         Pt pt = p[i];
58         while (up.size() >= 2 && sgn(cross(up[up.size() - 2], up.back(), pt)) <= 0)
59             up.pop_back();
60         up.push_back(pt);
61     }
62 }
```

4.11 Primitivas y Convex Hull (cont.)

```
63     lo.pop_back();
64     up.pop_back();
65     lo.insert(lo.end(), up.begin(), up.end());
66     return lo;
67 }
68
69 // Formula del cord n (shoelace) para poligono simple
70 double polygonArea(const vector<Pt> &P)
71 {
72     long double s = 0;
73     int n = P.size();
74     for (int i = 0; i < n; i++)
75     {
76         s += (long double)P[i].x * P[(i + 1) % n].y - (long double)P[i].y * P[(i + 1) % n].x;
77     }
78     return fabs((double)s) * 0.5;
79 }
80
81 int main()
82 {
83     ios_base::sync_with_stdio(false);
84     cin.tie(nullptr);
85     return 0;
86 }
```

4.12 Probabilidad de C rculo Contenido - Omens

O: (n) por consulta, cuatro desigualdades sobre el borde convexo

Caen dos piedras uniformemente al azar dentro de un rect ngulo L x W y se dibuja el c rculo que las tiene...

C++/ICPC/2024/I - Omens.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7     cout << fixed << setprecision(4);
8
9     double L, W;
10
11     // La entrada termina con la linea "0 0", que no se procesa.
12     while (cin >> L >> W && (L != 0 || W != 0)) {
13         double t = min(L, W) / max(L, W);
14
15         cout << acos(-1.0) / 6.0 * t * (2 - t) << '\n';
16     }
17
18     return 0;
19 }
```

4.13 Punto en Poligono Convexo (O(log n))

O: (n log n) el hull, (log n) cada consulta

Uso: h = convexHull(p); inside(h,q) con h antihorario y sin colineales

Casco convexo y desp s muchas consultas de punto adentro, en log n por consulta

C++/Notebook/Punto en Poligono Convexo.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 struct P { ll x, y; };
6
7 ll cross(const P&O, const P&A, const P&B){
8     return (A.x-O.x)*(B.y-O.y) - (A.y-O.y)*(B.x-O.x);
9 }
10
11 vector<P> convexHull(vector<P> p){
12     int n = p.size(), k = 0;
13     sort(p.begin(), p.end(), [](const P&a, const P&b){
14         return a.x != b.x ? a.x < b.x : a.y < b.y;
15     });
```

4.13 Punto en Poligono Convexo (O(log n)) (cont.)

```

5     });
6     p.erase(unique(p.begin(), p.end(), [](const P&a, const P&b){
7         return a.x==b.x && a.y==b.y;
8     }), p.end());
9     n = p.size();
10    if (n < 3) return p;
11
12    vector<P> h(2*n);
13    for (int i = 0; i < n; i++){           // cadena inferior
14        while (k >= 2 && cross(h[k-2], h[k-1], p[i]) <= 0) k--;
15        h[k++] = p[i];
16    }
17    for (int i = n-2, t = k+1; i >= 0; i--){ // cadena superior
18        while (k >= t && cross(h[k-2], h[k-1], p[i]) <= 0) k--;
19        h[k++] = p[i];
20    }
21    h.resize(k-1);                        // quita el repetido final
22    return h;                             // CCW, sin colineales
23 }
24
25 bool inside(const vector<P>& h, const P& q){
26     int n = h.size();
27     if (n == 1) return h[0].x==q.x && h[0].y==q.y;
28     if (n == 2){                          // segmento degenerado
29         if (cross(h[0], h[1], q) != 0) return false;
30         return min(h[0].x, h[1].x) <= q.x && q.x <= max(h[0].x, h[1].x)
31             && min(h[0].y, h[1].y) <= q.y && q.y <= max(h[0].y, h[1].y);
32     }
33     if (cross(h[0], h[1], q) < 0) return false;
34     if (cross(h[0], h[n-1], q) > 0) return false;
35
36     int lo = 1, hi = n-1;                 // busca el sector
37     while (hi - lo > 1){
38         int mid = (lo + hi) / 2;
39         if (cross(h[0], h[mid], q) >= 0) lo = mid;
40         else hi = mid;
41     }
42     return cross(h[lo], h[lo+1], q) >= 0;
43 }
44
45 int main(){
46     ios::sync_with_stdio(false);
47     cin.tie(nullptr);
48     int L;
49     while (cin >> L){
50         vector<P> big(L);
51         for (auto &b : big) cin >> b.x >> b.y;
52         vector<P> h = convexHull(big);
53
54         int S; cin >> S;
55         int cnt = 0;
56         for (int i = 0; i < S; i++){
57             P q; cin >> q.x >> q.y;
58             if (inside(h, q)) cnt++;
59         }
60         cout << cnt << "\n";
61     }
62     return 0;
63 }

```

4.14 Rotating Calipers (Par Mas Lejano)

O: (n log n) el hull, (n) el diametro

Uso: h = convexHull(p); diameter(h) -> {dist^2, {P,Q}} // dist^2, no dist

Par de puntos mas lejano (diametro del conjunto) en O(n log n)

C++/Notebook/Rotating Calipers.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 typedef long long ll;

```

4.14 Rotating Calipers (Par Mas Lejano) (cont.)

```

6
7 struct P
8 {
9     ll x, y;
10    bool operator<(const P &o) const { return x != o.x ? x < o.x : y < o.y; }
11    bool operator==(const P &o) const { return x == o.x && y == o.y; }
12 };
13
14 P operator-(P a, P b) { return {a.x - b.x, a.y - b.y}; }
15 ll cross(P a, P b) { return a.x * b.y - a.y * b.x; }
16 ll cross(P o, P a, P b) { return cross(a - o, b - o); }
17 ll dist2(P a, P b)
18 {
19     P d = a - b;
20     return d.x * d.x + d.y * d.y;
21 }
22
23 // Monotone chain: casco en sentido antihorario, sin repetidos ni colineales
24 vector<P> convexHull(vector<P> p)
25 {
26     sort(p.begin(), p.end());
27     p.erase(unique(p.begin(), p.end(), p.end()));
28     int n = p.size();
29     if (n <= 2) return p;
30
31     vector<P> h(2 * n);
32     int k = 0;
33     for (int i = 0; i < n; i++)
34     {
35         while (k >= 2 && cross(h[k - 2], h[k - 1], p[i]) <= 0) k--;
36         h[k++] = p[i];
37     }
38     for (int i = n - 2, lo = k + 1; i >= 0; i--)
39     {
40         while (k >= lo && cross(h[k - 2], h[k - 1], p[i]) <= 0) k--;
41         h[k++] = p[i];
42     }
43     h.resize(k - 1);
44     return h;
45 }
46
47 // {distancia^2 maxima, par de puntos}. h = casco antihorario (el de convexHull)
48 pair<ll, pair<P, P>> diameter(const vector<P> &h)
49 {
50     int n = h.size();
51     if (n == 1) return {0, {h[0], h[0]}};
52     if (n == 2) return {dist2(h[0], h[1]), {h[0], h[1]}};
53
54     ll best = -1;
55     P a = h[0], b = h[0];
56     for (int i = 0, j = 1; i < n; i++)
57     {
58         int ni = (i + 1) % n;
59         // avanzar j mientras el area del triangulo (i, ni, j) siga creciendo
60         while (cross(h[i], h[ni], h[(j + 1) % n]) > cross(h[i], h[ni], h[j])) j = (j + 1) % n;
61
62         if (dist2(h[i], h[j]) > best)
63         {
64             best = dist2(h[i], h[j]);
65             a = h[i];
66             b = h[j];
67         }
68         if (dist2(h[ni], h[j]) > best)
69         {
70             best = dist2(h[ni], h[j]);
71             a = h[ni];
72             b = h[j];
73         }
74     }
75     return {best, {a, b}};
76 }

```

4.14 Rotating Calipers (Par Mas Lejano) (cont.)

```

77
78 int main()
79 {
80     ios_base::sync_with_stdio(false);
81     cin.tie(nullptr);
82
83     int n;
84     cin >> n;
85     vector<P> p(n);
86     for (auto &q : p) cin >> q.x >> q.y;
87
88     auto r = diameter(convexHull(p));
89
90     cout << r.first << "\n"; // distancia al cuadrado (exacta)
91     cout << fixed << setprecision(6) << sqrt((double)r.first) << "\n";
92     cout << r.second.first.x << " " << r.second.first.y << " "
93         << r.second.second.x << " " << r.second.second.y << "\n";
94     return 0;
95 }

```

4.15 Shoelace (Area de Poligono)

O: (n)

Uso: area2(p) = 2*area con signo; puntosInteriores(p) por Pick

Area de CUALQUIER poligono simple en O(n), convexo o concavo, dando solo sus vertices en orden

C++/Notebook/Shoelace.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 struct P { ll x, y; };
6
7 // Doble del area CON SIGNO. Exacto con coordenadas enteras.
8 // > 0 antihorario, < 0 horario, = 0 degenerado (todos colineales).
9 ll area2(const vector<P>& p) {
10     ll s = 0;
11     int n = p.size();
12     for (int i = 0; i < n; i++) {
13         int j = (i + 1) % n;
14         s += p[i].x * p[j].y - p[j].x * p[i].y;
15     }
16     return s;
17 }
18
19 // Area de verdad. Solo aqui aparece el punto flotante.
20 double area(const vector<P>& p) {
21     return llabs(area2(p)) / 2.0;
22 }
23
24 // Puntos de coordenadas enteras SOBRE el borde.
25 // Cada arista aporta gcd(|dx|, |dy|) puntos sin contar dos veces los vertices.
26 ll puntosBorde(const vector<P>& p) {
27     ll b = 0;
28     int n = p.size();
29     for (int i = 0; i < n; i++) {
30         int j = (i + 1) % n;
31         b += __gcd(llabs(p[i].x - p[j].x), llabs(p[i].y - p[j].y));
32     }
33     return b;
34 }
35
36 // Puntos enteros ESTRICTAMENTE interiores, despejando Pick: A = I + B/2 - 1.
37 // Con 2A entero la cuenta sale exacta y sin dividir.
38 ll puntosInteriores(const vector<P>& p) {
39     return (llabs(area2(p)) - puntosBorde(p) + 2) / 2;
40 }
41
42 // Centroide (centro de masa) del poligono. No es el promedio de los vertices.
43 pair<double, double> centroide(const vector<P>& p) {
44     int n = p.size();
45     double a = 0, cx = 0, cy = 0;

```

4.15 Shoelace (Area de Poligono) (cont.)

```

46     for (int i = 0; i < n; i++) {
47         int j = (i + 1) % n;
48         double cr = (double)p[i].x * p[j].y - (double)p[j].x * p[i].y;
49         a += cr;
50         cx += (p[i].x + p[j].x) * cr;
51         cy += (p[i].y + p[j].y) * cr;
52     }
53     a /= 2;
54     return {cx / (6 * a), cy / (6 * a)};
55 }
56
57 int main() {
58     ios::sync_with_stdio(false);
59     cin.tie(nullptr);
60
61     int n;
62     if (!(cin >> n)) return 0;
63
64     vector<P> p(n);
65     for (auto &v : p) cin >> v.x >> v.y;
66
67     ll a2 = area2(p);
68
69     // Imprimir sin perder el medio punto: parte entera y el .5 aparte.
70     cout << llabs(a2) / 2;
71     if (llabs(a2) % 2) cout << ".5";
72     cout << "\n";
73
74     return 0;
75 }
76
77 // OJO con el overflow: si las coordenadas llegan a 1e9, cada producto
78 // llega a 1e18 y la suma de varios se pasa de long long. En ese caso
79 // acumula en __int128, o traslada todos los puntos restando el primer
80 // vertice antes de sumar (las coordenadas se vuelven chicas y el area
81 // no cambia).

```

4.16 Triangle Classification - AROD

O: (1) por caso, clasificar el triangulo

Clasifica triángulos en una grilla de p×q puntos como agudos, rectos, obtusos o degenerados

C++/RPCI/AROD.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using int64 = long long;
5
6 static inline int64 comb3(int64 n) {
7     if (n < 3) return 0;
8     return n * (n - 1) * (n - 2) / 6;
9 }
10
11 int main() {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14
15     int p, q;
16     cin >> p >> q;
17
18     if (p > q) swap(p, q); // p = lado pequeño, q = lado grande
19
20     const int W = p + 1; // cantidad de columnas de puntos
21     const int H = q + 1; // cantidad de filas de puntos
22
23     // gtProd[s][k] = cantidad de alpha en [0..s] con alpha*(s-alpha) > k
24     // eqProd[s][k] = cantidad de alpha en [0..s] con alpha*(s-alpha) = k
25     int maxS = p;
26     int maxK = maxS * maxS / 4;
27
28     vector<vector<int>> gtProd(maxS + 1, vector<int>(maxK + 1, 0));
29     vector<vector<int>> eqProd(maxS + 1, vector<int>(maxK + 1, 0));

```

4.16 Triangle Classification - AROD (cont.)

```

30
31   for (int s = 0; s <= maxS; s++) {
32       int localMax = s * s / 4;
33       vector<int> freq(localMax + 1, 0);
34
35       for (int alpha = 0; alpha <= s; alpha++) {
36           int val = alpha * (s - alpha);
37           freq[val]++;
38       }
39
40       int suf = 0;
41       for (int k = localMax; k >= 0; k--) {
42           gtProd[s][k] = suf;
43           suf += freq[k];
44           eqProd[s][k] = freq[k];
45       }
46   }
47
48   auto getGt = [&](int s, int64 k) -> int64 {
49       int64 localMax = 1LL * s * s / 4;
50       if (k > localMax) return 0;
51       return gtProd[s][(int)k];
52   };
53
54   auto getEq = [&](int s, int64 k) -> int64 {
55       int64 localMax = 1LL * s * s / 4;
56       if (k > localMax) return 0;
57       return eqProd[s][(int)k];
58   };
59
60   int64 acute = 0, right = 0, obtuse = 0, degenerate = 0;
61
62   // -----
63   // 1) Todos en la misma fila -> degenerados
64   // -----
65   degenerate += 1LL * H * comb3(W);
66
67   // -----
68   // 2) Triángulos usando exactamente 2 filas
69   // -----
70   // Si la distancia vertical es h:
71   // - caso "punto solo entre la base"
72   // - caso "punto solo afuera de la base"
73   // -----
74   for (int h = 1; h <= q; h++) {
75       int64 verticalWays = 2LL * (H - h); // escoger las dos filas y cuál tiene 2 puntos
76
77       for (int s = 1; s <= p; s++) {
78           int64 waysPlaceX = W - s;
79           int64 weight = verticalWays * waysPlaceX;
80
81           // Punto solo entre los dos de la base
82           int64 rightBetween = 2 + getEq(s, 1LL * h * h); // extremos + posibles interno
83
84           s
85           int64 obtuseBetween = getGt(s, 1LL * h * h);
86           int64 acuteBetween = (s + 1) - rightBetween - obtuseBetween;
87
88           // Punto solo fuera del intervalo horizontal de la base
89           // todos esos son obtusos, salvo el caso alineado con un extremo,
90           // que ya quedó contado en "between" con alpha=0 o beta=0
91           int64 obtuseOutside = 2LL * (s - 1);
92
93           right += weight * rightBetween;
94           obtuse += weight * (obtuseBetween + obtuseOutside);
95           acute += weight * acuteBetween;
96       }
97   }
98
99   // -----
100  // 3) Triángulos usando exactamente 3 filas distintas
101  // -----
102  // Distancias verticales:

```

4.16 Triangle Classification - AROD (cont.)

```

01  // fila1 -> fila2 = a
02  // fila2 -> fila3 = b
03  //
04  // Se cuenta por span horizontal total s.
05  // -----
06  for (int a = 1; a < q; a++) {
07      for (int b = 1; a + b <= q; b++) {
08          int64 verticalWays = H - (a + b);
09          int64 ab = 1LL * a * b;
10          int64 A = 1LL * a * (a + b);
11          int64 B = 1LL * b * (a + b);
12
13          for (int s = 0; s <= p; s++) {
14              int64 waysPlaceX = W - s;
15              int64 weight = verticalWays * waysPlaceX;
16
17              // Caso monótono:
18              // x1<=x2<=x3 o x1>=x2>=x3
19              // Todo no degenerado aquí es obtuso.
20              int64 orient = (s == 0 ? 1 : 2);
21
22              int64 degHit = 0;
23              if ((1LL * a * s) % (a + b) == 0) {
24                  degHit = 1;
25              }
26
27              degenerate += weight * orient * degHit;
28              obtuse += weight * orient * ((s + 1) - degHit);
29
30              if (s == 0) continue;
31
32              // -----
33              // Caso donde x1 queda estrictamente "en medio"
34              // u < 0 y u+v > 0
35              // alpha en [1 .. s-1]
36              // -----
37              {
38                  int64 total = s - 1;
39                  if (total > 0) {
40                      int64 low = min<int64>(total, (ab - 1) / s);
41                      int64 rightLin = ((ab % s == 0) && (1 <= ab / s) && (ab / s <= s -
42                          1)) ? 1 : 0;
43
44                      int64 obt1 = low + getGt(s, A);
45                      int64 rig1 = rightLin + getEq(s, A);
46                      int64 acu1 = total - obt1 - rig1;
47
48                      obtuse += weight * 2 * obt1;
49                      right += weight * 2 * rig1;
50                      acute += weight * 2 * acu1;
51                  }
52              }
53
54              // -----
55              // Caso donde x3 queda en medio o empata con x1
56              // v < 0 y u+v >= 0
57              // alpha en [0 .. s-1]
58              // -----
59              {
60                  int64 total = s;
61                  int64 low = min<int64>(total, (ab - 1) / s);
62                  int64 rightLin = ((ab % s == 0) && (1 <= ab / s) && (ab / s <= s)) ? 1
63                      : 0;
64
65                  int64 obt3 = low + getGt(s, B);
66                  int64 rig3 = rightLin + getEq(s, B);
67                  int64 acu3 = total - obt3 - rig3;
68
69                  obtuse += weight * 2 * obt3;
70                  right += weight * 2 * rig3;
71                  acute += weight * 2 * acu3;
72              }

```

4.16 Triangle Classification - AROD (cont.)

```
11     }
12 }
13 }
14
15 cout << acute << '\n';
16 cout << right << '\n';
17 cout << obtuse << '\n';
18 cout << degenerate << '\n';
19
20 return 0;
21 }
```

5 - Math

5.1 2-Coloreo por Suma Potencia de 2 - Drug Test

O: (N), un for aprovechando que el complemento siempre es menor

Partir 1..N en dos tipos para que x y su complemento a potencia de 2 queden opuestos

C++/ICPC/2024/I/D - Drug Test.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int N, Q;
9
10    while (cin >> N >> Q && (N || Q)) {
11
12        vector<char> tipo(N + 1);
13
14        // 0 siempre es placebo
15        tipo[0] = 'P';
16
17        for (int x = 1; x <= N; x++) {
18
19            int p = 1;
20
21            while (p < x)
22                p *= 2;
23
24            if (p == x) {
25                // x + 0 = potencia de 2
26                tipo[x] = 'A';
27            }
28            else {
29                int complement = p - x;
30
31                // x y complement tienen tipos opuestos
32                tipo[x] = (tipo[complement] == 'A' ? 'P' : 'A');
33            }
34        }
35
36        while (Q--) {
37            int x;
38            cin >> x;
39            cout << tipo[x];
40        }
41
42        cout << '\n';
43    }
44
45    return 0;
46 }
```

5.2 Ad-hoc - Attention To Meeting

O: (1), se despeja de una desigualdad

Duracion maxima de N discursos iguales con pausas de 1 minuto dentro de un tiempo tope

C++/Training_Camp_2026/Day 3/A - Attention to Meeting.py

```
1 n = int(input())
2 t = int(input())
3
4 print((t - (n-1))/n)
```

5.3 Algebra - The Wisdom Of Master Wei

O: (1), se despeja con algebra

El maestro Wei empezo a programar en el ano W y el discipulo Kai en el ano K ($W < K$)

C++/Training_Camp_2026/Day 10/E - The Wisdom of Master Wei.py

```
1 n,m = list(map(int,input().split()))
2 print(m+(m-n))
```

5.4 Bit Manipulation - Meme Wars

O: ($n * \text{bits}$)

La letra en la posicion n de una palabra recursiva gigante: ruler sequence

C++/Training_Camp_2026/Day 7/G - Meme Wars.py

```
1 dic = {
2     1: "a",
3     2: "b",
4     4: "c",
5     8: "d",
6     16: "e",
7     32: "f",
8     64: "g",
9     128: "h",
10    256: "i",
11    512: "j",
12    1024: "k",
13    2048: "l",
14    4096: "m",
15    8192: "n",
16    16384: "o",
17    32768: "p",
18    65536: "q",
19    131072: "r",
20    262144: "s",
21    524288: "t",
22    1048576: "u",
23    2097152: "v",
24    4194304: "w",
25    8388608: "x",
26    16777216: "y",
27    33554432: "z"
28 }
29 n = int(input())
30 cual = 1
31 for clave,valor in dic.items():
32     if(n%clave == 0):
33         cual = clave
34         continue
35     else:
36         print(dic[cual])
37         break
```

5.5 Brute Force - Antiarithmetic

O: (n^2), probando todos los pares

Dada una permutacion de 0..n-1, determina si es antiarritmetica

C++/Training_Camp_2026/Day 11 - CCPL/H - Antiarithmetic.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 22];
5 static int bufLen_, bufPos_;
```

5.5 Brute Force - Antiarithmetic (cont.)

```

5 inline int gc() {
6     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0;
7         if (!bufLen_) return -1; }
8     return buf_[bufPos_++];
9 }
10 inline bool readInt(int &out) {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     if (c == -1) return false;
14     bool neg = false; if (c == '-') { neg = true; c = gc(); }
15     int x = 0;
16     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
17     out = neg ? -x : x;
18     return true;
19 }
20 // Reads up to (and past) the colon before the first number is read normally, since ':' is
21 // simply skipped as a non-digit character by readInt's own skip loop.
22
23 bool isAntiarithmetic(int n, const vector<int> &p) {
24     vector<int> pos(n);
25     for (int idx = 0; idx < n; idx++) pos[p[idx]] = idx;
26     for (int i = 0; i < n; i++) {
27         for (int k = i + 1; k < n; k++) {
28             int s = p[i] + p[k];
29             if (s % 2 == 0) {
30                 int v = s / 2;
31                 int j = pos[v];
32                 if (i < j && j < k) return false;
33             }
34         }
35     }
36     return true;
37 }
38
39 int main() {
40     int n;
41     string out;
42     while (readInt(n)) {
43         if (n == 0) break;
44         vector<int> p(n);
45         for (int i = 0; i < n; i++) readInt(p[i]);
46         out += isAntiarithmetic(n, p) ? "yes\n" : "no\n";
47     }
48     fwrite(out.data(), 1, out.size(), stdout);
49     return 0;
50 }

```

5.6 Construction - Ezraft And Array

O: (n), construcción directa

Para cada n, imprime un arreglo de n enteros positivos (o -1 si n=2, caso imposible)

C++/Codeforces/ezraft And Array.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios_base::sync_with_stdio(false);
6     cin.tie(NULL);
7
8     int t;
9     cin >> t;
10    while (t--) {
11        int n;
12        cin >> n;
13
14        if (n == 2) {
15            cout << -1 << "\n";
16            continue;
17        }
18
19        vector<long long> a;

```

5.6 Construction - Ezraft And Array (cont.)

```

20    if (n == 1) {
21        a = {1};
22    } else {
23        a = {1, 2, 3};
24        long long S = 6;
25        for (int i = 4; i <= n; i++) {
26            a.push_back(S);
27            S *= 2;
28        }
29    }
30
31    for (size_t i = 0; i < a.size(); i++) {
32        cout << a[i];
33        if (i + 1 != a.size()) cout << ' ';
34    }
35    cout << "\n";
36 }
37 return 0;
38 }

```

5.7 Divisores de la Suma Total - Puntos Equiespaciados En Un Circulo

O: $(\sqrt{S} + d(S) \cdot n^2)$ en el peor caso, pero los dos filtros lo dejan casi lineal

Uso: lee n y los n huecos del círculo; imprime cuantos puntos quitar, o -1

Quitar los mínimos puntos de un círculo para que el resto quede equiespaciado

C++/Notebook/Puntos Equiespaciados en un Círculo.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     int n;
7     while (cin >> n)
8     {
9         if (n == 0) return 0;
10
11         vector<int> dist(n);
12         int maxi = -1;
13         bool iguales = true;
14         int ant = 0;
15         int sum = 0;
16
17         for (int i = 0; i < n; i++)
18         {
19             cin >> dist[i];
20
21             if (i == 0)
22                 ant = dist[i];
23
24             if (dist[i] > maxi)
25             {
26                 maxi = dist[i];
27             }
28
29             if (ant != dist[i])
30             {
31                 iguales = false;
32             }
33
34             sum += dist[i];
35         }
36
37         if (iguales)
38         {
39             cout << 0 << endl;
40         }
41         else
42         {
43             bool entro = false;
44
45             vector<int> divisores;

```

5.7 Divisores de la Suma Total - Puntos Equiespaciados En Un Circulo (cont.)

```

46
47     for(int i = 1 ; i * i <= sum ; i++)
48     {
49         if(sum % i == 0)
50         {
51             divisores.push_back(i);
52
53             if(i != sum / i)
54             {
55                 divisores.push_back(sum / i);
56             }
57         }
58     }
59
60     sort(divisores.begin(), divisores.end());
61
62     for(int i : divisores)
63     {
64         // i es el largo del arco: no puede ser menor que el hueco mas
65         // grande, ni tan grande que queden menos de 3 puntos.
66         if(i < maxi || i > sum / 3)
67             continue;
68
69         for(int inicio = 0 ; inicio < n ; inicio++)
70         {
71             int anti = 0;
72             int quitados = 0;
73             int cont = 0;
74             bool ok = true;
75
76             for(int j = 0 ; j < n ; j++)
77             {
78                 int pos = (inicio + j) % n;
79
80                 anti += dist[pos];
81                 cont++;
82
83                 if(anti == i)
84                 {
85                     anti = 0;
86                     quitados += cont - 1; // el arco conserva 1 punto
87                     cont = 0;
88                 }
89                 else if(anti > i)
90                 {
91                     ok = false; // este arranque no es corte
92                     break;
93                 }
94             }
95
96             if(ok && anti == 0)
97             {
98                 entro = true;
99
100                 if(n - quitados >= 3)
101                 {
102                     cout << quitados << endl;
103                 }
104                 else
105                 {
106                     cout << -1 << endl;
107                 }
108
109                 break;
110             }
111         }
112
113         if(entro)
114         {
115             break;
116         }
117     }

```

5.7 Divisores de la Suma Total - Puntos Equiespaciados En Un Circulo (cont.)

```

18
19     if(!entro)
20     {
21         cout << -1 << endl;
22     }
23 }
24 }
25
26 return 0;
27 }

```

5.8 Eliminacion Gaussiana (mod p)

O: ($n^2 * m$) en modulo p primo

Uso: gauss(a,m,sol,incompatible,p) -> grados de libertad; determinante(a,p)

Resolver $Ax = b$ modulo p primo: Gauss con inverso modular en vez de dividir

C++/Notebook/Eliminacion Gaussiana.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 ll mpow(ll a, ll e, ll p) {
6     ll r = 1; a %= p;
7     while (e) { if (e & 1) r = r * a % p; a = a * a % p; e >>= 1; }
8     return r;
9 }
10
11 ll inverso(ll a, ll p) { return mpow((a % p + p) % p, p - 2, p); }
12
13 // Matriz AUMENTADA de n filas por (m+1) columnas: las m incognitas y el termino
14 // independiente al final. Devuelve el rango; deja una solucion en sol.
15 int gauss(vector<vector<ll>> a, int m, vector<ll>& sol, bool& incompatible, ll p) {
16     int n = a.size(), fila = 0;
17     vector<int> donde(m, -1); // donde[col] = fila que tiene el pivote de esa col
18     umna
19
20     for (int col = 0; col < m && fila < n; col++) {
21         int piv = -1;
22         for (int i = fila; i < n; i++)
23             if (a[i][col] % p) { piv = i; break; }
24         if (piv < 0) continue; // columna sin pivote: variable libre
25
26         swap(a[piv], a[fila]);
27
28         ll iv = inverso(a[fila][col], p);
29         for (int j = col; j <= m; j++)
30             a[fila][j] = a[fila][j] * iv % p;
31
32         for (int i = 0; i < n; i++)
33             if (i != fila && a[i][col] % p) {
34                 ll f = a[i][col];
35                 for (int j = col; j <= m; j++)
36                     a[i][j] = ((a[i][j] - f * a[fila][j]) % p + p) % p;
37             }
38         donde[col] = fila++;
39     }
40
41     sol.assign(m, 0); // las variables libres quedan en 0
42     for (int j = 0; j < m; j++)
43         if (donde[j] >= 0) sol[j] = a[donde[j]][m] % p;
44
45     // Una fila 0 = c con c != 0 significa que no hay solucion.
46     incompatible = false;
47     for (int i = fila; i < n; i++) {
48         bool cero = true;
49         for (int j = 0; j < m; j++)
50             if (a[i][j] % p) { cero = false; break; }
51         if (cero && a[i][m] % p) { incompatible = true; break; }
52     }
53     return fila;

```

5.8 Eliminación Gaussiana (mod p) (cont.)

```

54 }
55
56 // Determinante mod p de una matriz cuadrada. 0 significa que es singular.
57 ll determinante(vector<vector<ll>> a, ll p) {
58     int n = a.size();
59     ll det = 1;
60     for (int c = 0; c < n; c++) {
61         int piv = -1;
62         for (int i = c; i < n; i++)
63             if (a[i][c] % p) { piv = i; break; }
64         if (piv < 0) return 0;
65         if (piv != c) { swap(a[piv], a[c]); det = (p - det) % p; } // cada swap cambia el signo
66
67         det = det * a[c][c] % p;
68         ll iv = inverso(a[c][c], p);
69         for (int i = c + 1; i < n; i++) {
70             ll f = a[i][c] * iv % p;
71             for (int j = c; j < n; j++)
72                 a[i][j] = ((a[i][j] - f * a[c][j]) % p + p) % p;
73         }
74     }
75     return det;
76 }
77
78 int main() {
79     ios::sync_with_stdio(false);
80     cin.tie(nullptr);
81
82     const ll P = 1000000007;
83     int n, m;
84     if (!(cin >> n >> m)) return 0;
85
86     vector<vector<ll>> a(n, vector<ll>(m + 1));
87     for (int i = 0; i < n; i++)
88         for (int j = 0; j <= m; j++) { cin >> a[i][j]; a[i][j] = ((a[i][j] % P) + P) % P; }
89
90     vector<ll> sol;
91     bool incompatible;
92     int rango = gauss(a, m, sol, incompatible, P);
93
94     if (incompatible) {
95         cout << "sin solucion\n";
96     } else {
97         cout << "rango " << rango << ", soluciones = P^" << (m - rango) << "\n";
98         for (int j = 0; j < m; j++) cout << sol[j] << " \n"[j == m - 1];
99     }
100     return 0;
101 }

```

5.9 Exponenciación de Matrices (Recurrencias Lineales con n hasta 10^{18})O: ($k^3 \log n$)

Uso: `recurrencia({c1..ck}, {f0..f(k-1)}, n)` para $f(n) = c_1 f(n-1) + \dots + c_k f(n-k)$
 $f(n)$ de una recurrencia lineal de orden k en $O(k^3 \log n)$, con n de hasta 10^{18}

C++/Notebook/Exponenciación de Matrices.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 typedef long long ll;
6 typedef vector<vector<ll>> Mat;
7
8 const ll MOD = 1e9 + 7;
9
10 Mat mul(const Mat &A, const Mat &B)
11 {
12     int n = A.size(), m = B[0].size(), k = B.size();
13     Mat C(n, vector<ll>(m, 0));
14     for (int i = 0; i < n; i++)

```

5.9 Exponenciación de Matrices (Recurrencias Lineales con n hasta 10^{18}) (cont.)

```

5 {
6     for (int t = 0; t < k; t++)
7     {
8         if (A[i][t] == 0) continue;
9         for (int j = 0; j < m; j++)
10         {
11             C[i][j] = (C[i][j] + A[i][t] * B[t][j]) % MOD;
12         }
13     }
14 }
15 return C;
16 }
17
18 Mat matpow(Mat A, ll e)
19 {
20     int n = A.size();
21     Mat R(n, vector<ll>(n, 0));
22     for (int i = 0; i < n; i++) R[i][i] = 1;
23     while (e)
24     {
25         if (e & 1) R = mul(R, A);
26         A = mul(A, A);
27         e >>= 1;
28     }
29     return R;
30 }
31
32 // f(n) = c[0]*f(n-1) + c[1]*f(n-2) + ... + c[k-1]*f(n-k), con f(0..k-1) = f0
33 ll recurrencia(const vector<ll> &c, const vector<ll> &f0, ll n)
34 {
35     int k = c.size();
36     if (n < k) return ((f0[n] % MOD) + MOD) % MOD;
37     Mat T(k, vector<ll>(k, 0));
38     for (int j = 0; j < k; j++) T[0][j] = ((c[j] % MOD) + MOD) % MOD;
39     for (int i = 1; i < k; i++) T[i][i - 1] = 1;
40     Mat P = matpow(T, n - k + 1);
41     // estado inicial (f(k-1), f(k-2), ..., f(0)) como columna
42     ll res = 0;
43     for (int j = 0; j < k; j++)
44     {
45         ll v = ((f0[k - 1 - j] % MOD) + MOD) % MOD;
46         res = (res + P[0][j] * v) % MOD;
47     }
48     return res;
49 }
50
51 int main()
52 {
53     ios_base::sync_with_stdio(false);
54     cin.tie(nullptr);
55     return 0;
56 }

```

5.10 Fibonacci (Fast Doubling) - Fibonacci Sum

O: ($\log n$), fast doublingPara T casos con N y M (hasta $1e9$), calcula la suma $F(N) + F(N+1) + \dots + F(M)$ modulo $1e9+7$

C++/Training_Camp_2026/Day 10/M - Fibonacci Sum.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5 const ll MOD = 1000000007;
6
7 ll fibmod(ll n, ll mod) {
8     ll a = 0, b = 1, c;
9     for (int i = 63 - __builtin_clzll(n); i >= 0; i--) {
10         c = a;
11         a = (c * (2 * b - c + mod)) % mod;
12         b = (c * c + b * b) % mod;
13     }

```

5.10 Fibonacci (Fast Doubling) - Fibonacci Sum (cont.)

```

4 if ((n >> i) & 1) {
5     c = (a + b) % mod;
6     a = b;
7     b = c;
8 }
9 }
10 return a;
11 }
12
13 int main() {
14     ios::sync_with_stdio(false);
15     cin.tie(nullptr);
16
17     int q;
18     cin >> q;
19
20     while (q--) {
21         ll a, b;
22         cin >> a >> b;
23
24         ll ans = (fibmod(b + 2, MOD) - fibmod(a + 1, MOD) + MOD) % MOD;
25         cout << ans << '\n';
26     }
27 }

```

5.11 Fibonacci (Zeckendorf-like Greedy) - Getting In Shape

O: (log n), greedy tipo Zeckendorf sobre Fibonacci

Dado N, hay que construir una secuencia de ejercicios tipo A/B (terminada en B) tal que la cantidad de...

C++/Training_Camp_2026/Day 4/D - Getting In Shape.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // fib[i] = Fib(i+1), i.e. fib[0]=Fib(1)=1, fib[1]=Fib(2)=1, fib[2]=Fib(3)=2, ...
5 vector<long long> fib;
6
7 void buildFib() {
8     fib.push_back(1);
9     fib.push_back(1);
10    while (true) {
11        long long a = fib[fib.size() - 1];
12        long long b = fib[fib.size() - 2];
13        if (a > (long long)4e18 - b) break; // stop well before overflow
14        fib.push_back(a + b);
15        if (fib.back() > (long long)4e15) break; // more than enough headroom past 1e15
16    }
17 }
18
19 bool solve(long long N, string &result);
20
21 unordered_map<long long, pair<bool, string>> memo;
22
23 // Find lex-smallest V (ending in B) with h_V(1) + h_V(2) = S.
24 bool solveSum(long long S, string &result) {
25     // Try the largest k such that Fib(k+2) divides S exactly; Fib(k+2) = fib[k+1].
26     for (int k = (int)fib.size() - 2; k >= 1; k--) {
27         long long fibK2 = fib[k + 1]; // Fib(k+2)
28         if (fibK2 > S) continue;
29         if (S % fibK2 == 0) {
30             long long m = S / fibK2;
31             if (m == 1) {
32                 result = string(k - 1, 'A') + "B";
33                 return true;
34             } else {
35                 string sub;
36                 if (solve(m, sub)) {
37                     result = string(k - 1, 'A') + "B" + sub;
38                     return true;
39                 }
40             }
41         }
42     }
43 }

```

5.11 Fibonacci (Zeckendorf-like Greedy) - Getting In Shape (cont.)

```

42 }
43 return false;
44 }
45
46 // Find lex-smallest string (ending in B) with h(1) = N.
47 // Memoized: the same sub-value N can be reached via many different divisor
48 // orderings when the original input has several Fibonacci-related factors,
49 // so without caching this can blow up (verified: >5,000,000 redundant calls
50 // on some inputs without memoization, vs ~1 with it).
51 bool solve(long long N, string &result) {
52     if (N == 1) {
53         result = "B";
54         return true;
55     }
56     auto it = memo.find(N);
57     if (it != memo.end()) {
58         if (it->second.first) result = it->second.second;
59         return it->second.first;
60     }
61     string sub;
62     bool ok = solveSum(N, sub);
63     string ans = ok ? ("A" + sub) : "";
64     memo[N] = {ok, ans};
65     if (ok) result = ans;
66     return ok;
67 }
68
69 int main() {
70     ios_base::sync_with_stdio(false);
71     cin.tie(nullptr);
72     buildFib();
73
74     long long N;
75     cin >> N;
76
77     string result;
78     if (solve(N, result)) {
79         cout << result << "\n";
80     } else {
81         cout << "IMPOSSIBLE\n";
82     }
83     return 0;
84 }

```

5.12 Fibonacci Fast Doubling - Pou

O: (log n), Fibonacci con fast doubling

Calcula el n-ésimo número de Fibonacci modulo 2026 usando exponenciación rápida de matriz

C++/Codeforces/pou.cpp

```

1 #include <bits/stdc++.h>
2 typedef long long ll;
3 using namespace std;
4
5 int fibmod(ll n, int m)
6 {
7     int a = 0, b = 1, c;
8     for (int i = 63 - __builtin_clzll(n); i >= 0; i--)
9     {
10         c = a;
11         a = (1ll*c*(2ll*b-c+m)) % m;
12         b = (1ll*c*c + 1ll*b*b) % m;
13         if ((n>>i) & 1)
14             c = (a+b) % m; a = b; b = c;
15     }
16     return a;
17 }
18
19 int main() {
20
21 }
22

```

5.12 Fibonacci Fast Doubling - Pou (cont.)

```

23     int n;
24     cin >> n;
25
26     cout << fibmod(n+1,2026) << endl;
27
28     return 0;
29 }

```

5.13 Greedy - A Healthy Menu

O: (n log n) por el sort

Un colegio tiene M clases y N frutas, y para cada fruta i y clase j se conoce $G[i][j]$

C++/Training_Camp_2026/Day 6/A - A healthy menu.cpp

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      int n; cin >> n;
8      int m; cin >> m;
9
10     vector<vector<int>> clases;
11     long long maxi = 0;
12
13     for(int i = 0 ; i < n; i++)
14     {
15         vector<int> pp;
16         for(int j = 0 ; j < m; j++)
17         {
18             int a; cin>>a;
19             pp.push_back(a);
20         }
21         clases.push_back(pp);
22     }
23     for(int i = 0 ; i < m; i++)
24     {
25         long long maxxx = 0;
26         for(int j = 0 ; j < n; j++)
27         {
28             if(clases[j][i] >= maxxx)
29             {
30                 maxxx = clases[j][i];
31             }
32         }
33         maxi += maxxx;
34     }
35
36     cout << maxi<< endl;
37
38     return 0;
39 }

```

5.14 Implementation - Hour For A Run

O: (n)

Vinicius corre V vueltas a una pista con N carteles igualmente espaciados y quiere saber

C++/Training_Camp_2026/Day 1/H - Hour_for_a_run.py

```

1  import math
2
3  n,c = list(map(int, input().split()))
4
5  mult = c*n
6  res = ""
7  for i in range(1, 10):
8      res += str(math.ceil((mult*i)/10))+ " "
9  print(res)

```

5.15 Logaritmos - Gatuno's Descent Into Psychopathy

O: (1), se despeja con logaritmos

Minimo n con $H1 * ((B-1)/B)^n$ por debajo de un umbral: se despeja con logaritmos

C++/Training_Camp_2026/Day 9/F - Gatuno's Descent into Psychopathy.py

```

1  import math
2  n = int(input())
3
4  for _ in range(n):
5      h, o, r = list(map(int, input().split()))
6
7      r = (r-1)/r
8
9      print(math.ceil(math.log(o/h, r)))

```

5.16 Matrix Exponentiation - Buses

O: ($k^3 \log m$), exponenciacion de matrices

Dada una fila de longitud N metros formada por buses (10 m) y minibuses (5 m)

C++/Training_Camp_2026/Day 2/H - Buses.py

```

1  def mat_mult(A, B, MOD):
2      return [
3          [(A[0][0]*B[0][0] + A[0][1]*B[1][0]) % MOD,
4           (A[0][0]*B[0][1] + A[0][1]*B[1][1]) % MOD],
5          [(A[1][0]*B[0][0] + A[1][1]*B[1][0]) % MOD,
6           (A[1][0]*B[0][1] + A[1][1]*B[1][1]) % MOD],
7      ]
8
9  def mat_pow(M, p, MOD):
10     R = [[1, 0], [0, 1]] # identidad
11     while p > 0:
12         if p & 1:
13             R = mat_mult(R, M, MOD)
14             M = mat_mult(M, M, MOD)
15             p >>= 1
16     return R
17
18 n, k, l = map(int, input().split())
19 MOD = 1000000
20 m = n // 5
21
22 M = [[k % MOD, l % MOD], [1, 0]]
23 P = mat_pow(M, m, MOD)
24 # f(m) = P[1][0]*f(1) + P[1][1]*f(0) = P[1][0]*k + P[1][1]
25 res = (P[1][0] * (k % MOD) + P[1][1]) % MOD
26 print(f"{res:06d}")

```

5.17 NTT (Multiplicar Polinomios mod 998244353)

O: (n log n)

Uso: c = multiplicar(a, b); $c[k]$ = suma de $a[i]*b[k-i]$ mod 998244353

Producto de dos polinomios, o convolucion de dos arreglos, en $O(n \log n)$

C++/Notebook/NTT.cpp

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  typedef long long ll;
6
7  const ll MOD = 998244353, RAIZ = 3;
8
9  ll power(ll a, ll e)
10 {
11     ll r = 1;
12     a %= MOD;
13     while (e)
14     {
15         if (e & 1) r = r * a % MOD;
16         a = a * a % MOD;
17         e >>= 1;
18     }
19 }

```

5.17 NTT (Multiplicar Polinomios mod 998244353) (cont.)

```

8   return r;
20 }
21
22 void ntt(vector<ll> &a, bool inversa)
23 {
24     int n = a.size();
25     for (int i = 1, j = 0; i < n; i++)          // permutacion de bits invertidos
26     {
27         int bit = n >> 1;
28         for (; j & bit; bit >>= 1) j ^= bit;
29         j ^= bit;
30         if (i < j) swap(a[i], a[j]);
31     }
32     for (int len = 2; len <= n; len <= 1)
33     {
34         ll w = power(RAIZ, (MOD - 1) / len);
35         if (inversa) w = power(w, MOD - 2);
36         for (int i = 0; i < n; i += len)
37         {
38             ll wn = 1;
39             for (int j = 0; j < len / 2; j++)
40             {
41                 ll u = a[i + j], v = a[i + j + len / 2] * wn % MOD;
42                 a[i + j] = u + v < MOD ? u + v : u + v - MOD;
43                 a[i + j + len / 2] = u - v >= 0 ? u - v : u - v + MOD;
44                 wn = wn * w % MOD;
45             }
46         }
47     }
48     if (inversa)
49     {
50         ll inv = power(n, MOD - 2);
51         for (ll &x : a) x = x * inv % MOD;
52     }
53 }
54
55 vector<ll> multiplicar(vector<ll> a, vector<ll> b)
56 {
57     if (a.empty() || b.empty()) return {};
58     int tam = a.size() + b.size() - 1, n = 1;
59     while (n < tam) n <= 1;
60     for (ll &x : a) x = ((x % MOD) + MOD) % MOD;
61     for (ll &x : b) x = ((x % MOD) + MOD) % MOD;
62     a.resize(n);
63     b.resize(n);
64     ntt(a, false);
65     ntt(b, false);
66     for (int i = 0; i < n; i++) a[i] = a[i] * b[i] % MOD;
67     ntt(a, true);
68     a.resize(tam);
69     return a;
70 }
71
72 int main()
73 {
74     ios_base::sync_with_stdio(false);
75     cin.tie(nullptr);
76     return 0;
77 }

```

5.18 Numeros Romanos

O: (1), tabla fija de 13 valores

Uso: toRoman(n) con $1 \leq n \leq 3999$

Convierte un entero a numeral romano con un greedy sobre una tabla de valores ordenada de mayor a menor

C++/Notebook/Numeros Romanos.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 string toRoman(int num)

```

5.18 Numeros Romanos (cont.)

```

6 {
7     vector<pair<int, string>> roman = {
8         {1000, "M"}, {900, "CM"}, {500, "D"}, {400, "CD"},
9         {100, "C"}, {90, "XC"}, {50, "L"}, {40, "XL"},
10        {10, "X"}, {9, "IX"}, {5, "V"}, {4, "IV"},
11        {1, "I"}
12    };
13    string res;
14    for (const auto &p : roman)
15    {
16        while (num >= p.first)
17        {
18            res += p.second;
19            num -= p.first;
20        }
21    }
22    return res;
23 }
24
25 int main()
26 {
27     ios_base::sync_with_stdio(false);
28     cin.tie(nullptr);
29
30     int n;
31     while (cin >> n)
32     {
33         cout << toRoman(n) << "\n";
34     }
35
36     return 0;
37 }

```

5.19 Optimization - Satoru Gojo

O: $(n \log n)$ por el sort

Dado S y M, halla el costo minimo de escoger k enteros positivos cuyo producto sea al menos S

C++/Codeforces/SatoruGojo.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 long long S;
5
6 // Multiply acc by base, but cap at S+1 (i.e. "definitely enough") to avoid overflow;
7 // once acc exceeds S we no longer care about the exact value, only that it's >= S.
8 inline long long capMul(long long acc, long long base) {
9     if (acc > S) return acc; // already known to be enough
10    // check for overflow before multiplying: if acc * base would exceed a safe bound, cap
11    if (base != 0 && acc > (S + 5) / base + 1) {
12        return S + 5; // definitely exceeds S, safe sentinel
13    }
14    long long r = acc * base;
15    if (r > S) r = S + 5; // cap
16    return r;
17 }
18
19 // largest r (>=1) such that r^k <= S
20 long long integerKthRoot(int k) {
21     long long lo = 1, hi = S;
22     while (lo < hi) {
23         long long mid = lo + (hi - lo + 1) / 2;
24         long long val = 1;
25         bool exceeds = false;
26         for (int i = 0; i < k; i++) {
27             val = capMul(val, mid);
28             if (val > S) { exceeds = true; break; }
29         }
30         if (!exceeds) lo = mid;
31         else hi = mid - 1;
32     }
33     return lo;
34 }

```

5.19 Optimization - Satoru Gojo (cont.)

```

34 }
35
36 // minimal sum of k positive integers with product >= S
37 long long minSumFork(int k) {
38     long long r = integerKthRoot(k);
39     // try j = 0..k bumps from r to r+1, find smallest j with r^(k-j)*(r+1)^j >= S
40     for (int j = 0; j <= k; j++) {
41         long long val = 1;
42         for (int t = 0; t < k - j; t++) {
43             val = capMul(val, r);
44         }
45         for (int t = 0; t < j; t++) {
46             val = capMul(val, r + 1);
47         }
48         if (val >= S) {
49             return (long long)k * r + j;
50         }
51     }
52     // fallback (shouldn't happen): all r+1
53     return (long long)k * (r + 1);
54 }
55
56 int main() {
57     long long M;
58     cin >> S >> M;
59
60     long long best = LLONG_MAX;
61     // k up to ~62 is more than enough: 2^62 far exceeds 1e12, and the natural optimum
62     // (ignoring M) is around ln(S) <= ~28 for S<=1e12; checking generously up to 62
63     // safely covers the region where increasing k could still help when M is small.
64     int maxK = 62;
65     for (int k = 1; k <= maxK; k++) {
66         long long sum = minSumFork(k);
67         long long total = sum + (long long)(k - 1) * M;
68         if (total < best) best = total;
69     }
70
71     cout << best << "\n";
72     return 0;
73 }

```

5.20 Paridad - Call

O: (n)

NO ES MIO: código de Stiven Correa, del repo del equipo (carpeta RPC/)

RPC/2026-07/C - Call.py

```

1 n = int(input())
2
3 res = 0
4 for _ in range(n):
5     r = int(input())
6     res += r%2
7 print(res)

```

5.21 Polynomial Composition - Coach

O: (n), composición de polinomios

Calcula la composición de polinomios $P(Q(x))$ usando multiplicación polinómica modular

C++/Codeforces/coach.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const long long MOD = 998244353;
5
6 // Multiplica dos polinomios A y B
7 // A[i] representa el coeficiente de x^i
8 // B[j] representa el coeficiente de x^j
9 vector<long long> multiply(const vector<long long>& A, const vector<long long>& B) {
10     vector<long long> result(A.size() + B.size() - 1, 0);
11
12     for (int i = 0; i < (int)A.size(); i++) {

```

5.21 Polynomial Composition - Coach (cont.)

```

8         for (int j = 0; j < (int)B.size(); j++) {
9             // A[i] * x^i multiplicado por B[j] * x^j
10            // queda en el grado i + j
11            result[i + j] += A[i] * B[j];
12            result[i + j] %= MOD;
13        }
14    }
15
16    return result;
17 }
18
19 int main() {
20     ios::sync_with_stdio(false);
21     cin.tie(nullptr);
22
23     int n, m;
24     cin >> n >> m;
25
26     // P tiene grado n, por eso tiene n + 1 coeficientes
27     vector<long long> P(n + 1);
28
29     // Q tiene grado m, por eso tiene m + 1 coeficientes
30     vector<long long> Q(m + 1);
31
32     for (int i = 0; i <= n; i++) {
33         cin >> P[i];
34     }
35
36     for (int i = 0; i <= m; i++) {
37         cin >> Q[i];
38     }
39
40     // El grado máximo de P(Q(x)) es n * m
41     // Por eso la respuesta tiene n*m + 1 coeficientes
42     vector<long long> ans(n * m + 1, 0);
43
44     // power representa Q^i
45     // Al inicio i = 0, entonces Q^0 = 1
46     vector<long long> power = {1};
47
48     // Calculamos:
49     // P(Q(x)) = p0*Q^0 + p1*Q^1 + p2*Q^2 + ... + pn*Q^n
50     for (int i = 0; i <= n; i++) {
51         // Sumamos P[i] * Q^i a la respuesta
52         for (int j = 0; j < (int)power.size(); j++) {
53             ans[j] += P[i] * power[j];
54             ans[j] %= MOD;
55         }
56
57         // Preparamos la siguiente potencia:
58         // si ahora power = Q^i,
59         // después de multiplicar por Q queda power = Q^(i+1)
60         if (i < n) {
61             power = multiply(power, Q);
62         }
63     }
64
65     // Imprimos los coeficientes r0, r1, ..., r(n*m)
66     for (int i = 0; i <= n * m; i++) {
67         cout << ans[i] << " ";
68     }
69
70     cout << "\n";
71
72     return 0;
73 }

```

5.22 Prefix Sums - Triangles

O: (n), sumas de prefijos

Dados N puntos sobre un círculo, descritos por las longitudes de arco entre puntos consecutivos

C++/Training_Camp_2026/Day 2/F - Triangles.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     long long n; cin >> n;
7     long long tot = 0;
8     long long god = 0;
9     vector<long long> nums1;
10    for(long long i = 0 ; i < n ; i++)
11    {
12        long long a; cin >>a;
13        tot+= a;
14        nums1.push_back(tot);
15    }
16    vector<long long> nums(10000000,0);
17    for(long long i = 0 ; i < nums1.size() ; i++)
18    {
19        nums[nums1[i]]=1;
20    }
21
22    if(tot%3!=0)
23    {
24        cout<< 0 <<endl;
25    }
26    else
27    {
28        long long tri = tot/3;
29        for(long long i = 0 ; i < (10000000) ; i++)
30        {
31            if(nums[i]==1 && nums[i]==nums[i+tri]&&nums[i]==nums[i+tri+tri]&&i<10000000&i
32            +tri<10000000&i+tri+tri<10000000)
33            {
34
35                god++;
36                nums[i]=0;
37                nums[i+tri] = 0;
38                nums[i+tri+tri] = 0;
39            }
40        }
41
42        cout << god << endl;
43    }
44    return 0;
45 }
```

5.23 Recurrencia (Fibonacci) - Fractions Are Better When Continued

O: (N) con enteros grandes, o (log N) con fast doubling

Numerador de la N-esima fraccion continua, que resulta ser Fibonacci

C++/Training_Camp_2026/Day 3/F - Fractions are better when continued.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main()
6 {
7     long long n ; cin >> n;
8
9     if(n==1)
10    {
11        cout<<1<<endl;
12    }
13    else if(n == 2)
14    {
15        cout<<2<<endl;
```

5.23 Recurrencia (Fibonacci) - Fractions Are Better When Continued (cont.)

```
6    }
7    else
8    {
9        long long num = 1;
10       long long dem = 2;
11       long long temp = 0;
12       long long temp2 = 0;
13       while(n>1)
14       {
15           temp2 = num + dem;
16           num = temp2;
17
18           temp = num;
19           num = dem;
20           dem = temp;
21
22           n--;
23       }
24
25       cout<<num<<endl;
26   }
27
28   return 0;
29 }
```

5.24 Recurrencias y búsqueda de intervalo factible - Investigating Quadradômeda

O: (n), recurrencia con el intervalo factible de radios

Radios enteros tangentes entre estrellas alineadas: recurrencia e intervalo factible

C++/Training_Camp_2026/Day 6/I - Investigating Quadradômeda.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     ios_base::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9     cin >> n;
10
11    vector<long long> x(n), y(n);
12    for(int i = 0; i < n; i++) cin >> x[i] >> y[i];
13
14    // R_i = (-1)^(i-1) * R_1 + C_i, with C_1 = 0 and C_{i+1} = d_i - C_i
15    // Need R_i >= 1 for all i:
16    // i odd -> R_1 >= 1 - C_i (lower bound)
17    // i even -> R_1 <= C_i - 1 (upper bound)
18    // Also R_i < d_i for i < n is automatically implied by R_{i+1} >= 1.
19
20    long long C = 0; // C_i, starts at C_1 = 0
21    long long L = LLONG_MIN;
22    long long U = LLONG_MAX;
23
24    for(int i = 1; i <= n; i++){
25        if(i % 2 == 1){
26            // odd index: lower bound constraint
27            L = max(L, 1 - C);
28        } else {
29            // even index: upper bound constraint
30            U = min(U, C - 1);
31        }
32        if(i < n){
33            long long d;
34            if(x[i - 1] == x[i]){
35                d = llabs(y[i] - y[i - 1]);
36            } else {
37                d = llabs(x[i] - x[i - 1]);
38            }
39            C = d - C;
40        }
41    }
```

5.24 Recurrencias y búsqueda de intervalo factible - Investigating Quadradômeda (cont.)

```

42
43     if(L > U){
44         cout << -1 << "\n";
45     } else {
46         cout << U << "\n";
47     }
48
49     return 0;
50 }

```

5.25 Recursion (Simulacion tipo Josephus) - Metal Detector

O: (n), recursion tipo Josephus

Fila donde uno de cada dos pasa al final: recursion tipo Josephus

C++/Training_Camp_2026/Day 5/C - Metal detector.py

```

1  import math
2  t = int(input())
3  for i in range(t):
4      n, p = list(map(int, input().split()))
5      if(p%2 != 0):
6          print(math.ceil(p/2))
7          continue
8      llevo = 0
9      cual = 1
10     q = n
11     while True:
12         if(cual == 1):
13             if(p%2!=0):
14                 print(llevo + math.ceil(p/2))
15                 break
16             a = math.ceil(n/2)
17             llevo += a
18             n = math.floor(n/2)
19             p = math.floor(p/2)
20         else:
21             if(p%2!=1):
22                 print(llevo + math.ceil(p/2))
23                 break
24             a = math.ceil(n/2)
25             llevo += a
26             n = math.floor(n/2)
27             p = math.floor(p/2)
28
29         if(q%2==0):
30             cual = 1
31         else:
32             cual = 2
33
34         q = n

```

5.26 Simulacion - Cahokia Ruins

O: (H), una pasada por las filas

En cuanto tiempo chocan dos paredes que se acercan a 1 m/s en una sala H x W

C++/Training_Camp_2026/Day 5/A - Cahokia ruins.py

```

1  h, w = list(map(int, input().split()))
2
3  left = list(map(int, input().split()))
4  right = list(map(int, input().split()))
5
6  mins = float("inf")
7  for i in range(len(right)):
8      x = w - right[i]
9      mins = min(mins, x-left[i])
10
11  if mins // 2 == mins / 2:
12      print(int(mins/2))
13  else:
14      print(mins/2)

```

6 - Number Theory

6.1 Aritmetica Modular (Euclides Extendido, Phi, CRT General)

O: (log n) extgcd, inverso y cada paso de CRT; (raiz n) phi; (n log log n) la criba de...

Uso: inverso(a,m) da -1 si no existe; crt({r1,r2},{m1,m2}) -> {x, lcm} o {-1,-1}

Inverso con modulo NO primo, phi de Euler, y CRT aunque los modulos no sean coprimos

C++/Notebook/Aritmetica Modular.cpp

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  typedef long long ll;
6
7  // Devuelve g = gcd(a,b) y deja x, y con a*x + b*y = g
8  ll extgcd(ll a, ll b, ll &x, ll &y)
9  {
10     if (b == 0)
11     {
12         x = 1;
13         y = 0;
14         return a;
15     }
16     ll x1, y1;
17     ll g = extgcd(b, a % b, x1, y1);
18     x = y1;
19     y = x1 - (a / b) * y1;
20     return g;
21 }
22
23 // Inverso de a modulo m (m cualquiera, no hace falta que sea primo). -1 si no existe
24 ll inverso(ll a, ll m)
25 {
26     ll x, y;
27     ll g = extgcd((a % m) + m, m, x, y);
28     if (g != 1) return -1;
29     return ((x % m) + m) % m;
30 }
31
32 // phi de Euler de un solo n, por factorizacion hasta la raiz
33 ll phi(ll n)
34 {
35     ll r = n;
36     for (ll p = 2; p * p <= n; p++)
37     {
38         if (n % p == 0)
39         {
40             while (n % p == 0) n /= p;
41             r -= r / p;
42         }
43     }
44     if (n > 1) r -= r / n;
45     return r;
46 }
47
48 // phi de 0..n de una vez, tipo criba
49 vector<int> phiCriba(int n)
50 {
51     vector<int> ph(n + 1);
52     iota(ph.begin(), ph.end(), 0);
53     for (int i = 2; i <= n; i++)
54     {
55         if (ph[i] == i) // i es primo
56         {
57             for (int j = i; j <= n; j += i) ph[j] -= ph[j] / i;
58         }
59     }
60     return ph;
61 }
62
63 // x = r[i] (mod m[i]) para todo i. Devuelve {x, lcm} con 0 <= x < lcm, o {-1,-1}

```

6.1 Aritm tica Modular (Euclides Extendido, Phi, CRT General) (cont.)

```

84 pair<ll, ll> crt(const vector<ll> &r, const vector<ll> &m)
85 {
86     ll x = 0, M = 1; // x = 0 (mod 1) al empezar
87     for (int i = 0; i < (int)r.size(); i++)
88     {
89         ll ri = ((r[i] % m[i]) + m[i]) % m[i];
90         ll p, q;
91         ll g = extgcd(M, m[i], p, q);
92         if ((ri - x) % g != 0) return {-1, -1};
93         // M*p = g (mod m[i]) -> k = (ri - x)/g * p (mod m[i]/g)
94         ll mg = m[i] / g;
95         __int128 k = (__int128)((ri - x) / g) % mg * (p % mg) % mg;
96         if (k < 0) k += mg;
97         __int128 nuevoM = (__int128)M * mg;
98         x = (ll)((x + (__int128)M * k) % nuevoM);
99         M = (ll)nuevoM;
100     }
101     return {x, M};
102 }
103
104 int main()
105 {
106     ios_base::sync_with_stdio(false);
107     cin.tie(nullptr);
108     return 0;
109 }

```

6.2 BFS sobre Residuos - Only1s0s

O: (N), BFS sobre los N residuos

El menor m ltiplo de N escrito solo con unos y ceros: BFS sobre los residuos modulo N

C++/ICPC/2024/H - Only1s0s.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string solve(int N) {
5
6     vector<int> parent(N, -1);
7     vector<char> digit(N);
8     vector<bool> visited(N, false);
9
10    queue<int> q;
11
12    // Empezamos con el numero 1
13    int r = 1 % N;
14
15    visited[r] = true;
16    digit[r] = '1';
17
18    q.push(r);
19
20    while (!q.empty()) {
21
22        int cur = q.front();
23        q.pop();
24
25        // Encontramos un m ltiplo de N
26        if (cur == 0)
27            break;
28
29        // Agregar 0
30        int r0 = (cur * 10LL) % N;
31
32        if (!visited[r0]) {
33
34            visited[r0] = true;
35            parent[r0] = cur;
36            digit[r0] = '0';
37
38            q.push(r0);
39        }
40    }

```

6.2 BFS sobre Residuos - Only1s0s (cont.)

```

41 // Agregar 1
42 int r1 = (cur * 10LL + 1) % N;
43
44 if (!visited[r1]) {
45
46     visited[r1] = true;
47     parent[r1] = cur;
48     digit[r1] = '1';
49
50     q.push(r1);
51 }
52 }
53
54 // Reconstruir M
55 string M;
56
57 int cur = 0;
58
59 while (cur != -1) {
60     M.push_back(digit[cur]);
61     cur = parent[cur];
62 }
63
64 reverse(M.begin(), M.end());
65
66 return M;
67 }
68
69 // Divide un numero gigante representado como string
70 // entre N.
71 string divide(string M, int N) {
72
73     string ans;
74     long long rem = 0;
75
76     for (char c : M) {
77
78         rem = rem * 10 + (c - '0');
79
80         ans += char('0' + rem / N);
81
82         rem %= N;
83     }
84
85     // Quitar ceros iniciales
86     int pos = 0;
87
88     while (pos + 1 < (int)ans.size() &&
89            ans[pos] == '0') {
90         pos++;
91     }
92
93     return ans.substr(pos);
94 }
95
96
97 int main() {
98
99     ios::sync_with_stdio(false);
100    cin.tie(nullptr);
101
102    int N;
103
104    while (cin >> N && N != 0) {
105
106        string M = solve(N);
107
108        string D = divide(M, N);
109
110        cout << D << '\n';
111    }

```

6.2 BFS sobre Residuos - Only1s0s (cont.)

```

12     }
13
14     return 0;
15 }

```

6.3 Big Exponentiation - Uj (cpp)

O: (log n) multiplicaciones de enteros grandes

Calcula p^q con precision arbitraria, para exponentes muy grandes

C++/CCPLUj.cpp

```

1  #include <bits/stdc++.h>
2  #include <boost/multiprecision/cpp_int.hpp>
3
4  using namespace std;
5  using namespace boost::multiprecision;
6
7  int main() {
8      while (true) {
9          long long p, q;
10         cin >> p >> q;
11
12         if (p == 0 && q == 0) return 0;
13
14         cpp_int b = 1;
15
16         for (long long i = 0; i < q; i++) {
17             b *= p;
18         }
19
20         cout << b << "\n";
21     }
22 }

```

6.4 Big Exponentiation - Uj (java)

O: (log n) multiplicaciones de BigInteger

Calcula N^D usando BigInteger para manejar resultados con exponentes muy grandes

JAVA/CCPLUj.java

```

1  import java.util.*;
2  import java.math.*;
3
4  class Main {
5      public static void main (String[] args) throws Exception {
6          Scanner sc = new Scanner(System.in);
7
8          while (sc.hasNext()) {
9              int N = sc.nextInt();
10             int D = sc.nextInt();
11
12             if (N == 0 && D == 0) break;
13
14             System.out.println(BigInteger.valueOf(N).pow(D));
15         }
16     }
17 }

```

6.5 Criba de Eratostenes (Solo Impares)

O: (n log log n), memoria n/2 bits

Uso: vector<int> p = criba(n); // devuelve los primos hasta n

La criba que uno debería escribir por defecto

C++/Notebook/Criba de Eratostenes (Solo Impares).cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // Todos los primos <= n, en orden.
5  vector<int> criba(int n) {
6      if (n < 2) return {};
7
8      int m = (n - 1) / 2;          // el indice k es el impar 2k+1

```

6.5 Criba de Eratostenes (Solo Impares) (cont.)

```

9      vector<bool> compuesto(m + 1, false);
10
11      vector<int> primos;
12      primos.push_back(2);          // el unico par, va aparte
13
14      for (int k = 1; k <= m; k++) {
15          if (compuesto[k]) continue;
16
17          long long p = 2LL * k + 1;    // long long: p*p se pasa de int
18          primos.push_back((int)p);
19
20          for (long long j = (p * p - 1) / 2; j <= m; j += p)
21              compuesto[j] = true;
22      }
23
24      return primos;
25 }
26
27 // Version que deja la tabla para consultar despues si x es primo en O(1).
28 // esPrimo(x) = x == 2, o x impar con !compuesto[(x-1)/2].
29
30 int main() {
31     ios::sync_with_stdio(false);
32     cin.tie(nullptr);
33
34     int n;
35     if (!(cin >> n)) return 0;
36
37     vector<int> primos = criba(n);
38
39     cout << primos.size() << " primos hasta " << n << '\n';
40     for (int p : primos) cout << p << ' ';
41     cout << '\n';
42     return 0;
43 }

```

6.6 Criba de Factor Primo Mínimo + Inclusion-Exclusion - Ingredients That May Harm You

O: (n log n) la criba, (2^primos de X) por consulta

Platillos que comparten un primo con X: criba de menor factor e inclusion-exclusion

C++/Training_Camp_2026/Day 3/I - Ingredients that may Harm You.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXV = 1000000;
5  const int MOD = 1000000007;
6
7  int spf[MAXV + 1];
8  int cnt[MAXV + 1];
9  int pw[100005];
10
11 int main() {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14
15     // SPF
16     for (int i = 0; i <= MAXV; i++) spf[i] = i;
17     for (int i = 2; i * i <= MAXV; i++) {
18         if (spf[i] == i) {
19             for (int j = i * i; j <= MAXV; j += i)
20                 if (spf[j] == j) spf[j] = i;
21         }
22     }
23
24     int N;
25     cin >> N;
26
27     pw[0] = 1;
28     for (int i = 1; i <= N; i++)
29         pw[i] = (2LL * pw[i - 1]) % MOD;
30 }

```

6.6 Criba de Factor Primo M nimo + Inclusion-Exclusion - Ingredients That May Harm You (cont.)

```

31 while (N--) {
32     int x;
33     cin >> x;
34
35     vector<int> p;
36     while (x > 1) {
37         int pr = spf[x];
38         p.push_back(pr);
39         while (x % pr == 0) x /= pr;
40     }
41
42     int m = p.size();
43
44     for (int mask = 0; mask < (1 << m); mask++) {
45         int d = 1;
46         for (int i = 0; i < m; i++)
47             if (mask & (1 << i))
48                 d *= p[i];
49         cnt[d]++;
50     }
51 }
52
53 int Q;
54 cin >> Q;
55
56 while (Q--) {
57     int x;
58     cin >> x;
59
60     vector<int> p;
61     while (x > 1) {
62         int pr = spf[x];
63         p.push_back(pr);
64         while (x % pr == 0) x /= pr;
65     }
66
67     int m = p.size();
68
69     int coprime = 0;
70
71     for (int mask = 0; mask < (1 << m); mask++) {
72         int d = 1;
73         int bits = 0;
74
75         for (int i = 0; i < m; i++) {
76             if (mask & (1 << i)) {
77                 d *= p[i];
78                 bits++;
79             }
80         }
81
82         if (bits & 1)
83             coprime -= cnt[d];
84         else
85             coprime += cnt[d];
86     }
87
88     cout << pw[coprime] << '\n';
89 }
90
91 return 0;
92 }

```

6.7 Diferencia de Cuadrados - Square Diff

O: (1), es la condici n $n \bmod 4 \neq 2$ Si n es diferencia de dos cuadrados; toda la soluci n es la condici n $n \bmod 4 \neq 2$

C++/ICPC/2024/C - SquareDiff.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3

```

6.7 Diferencia de Cuadrados - Square Diff (cont.)

```

4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9
10    while (cin >> n && n != 0) {
11        cout << (n % 4 == 2 ? 'N' : 'Y') << '\n';
12    }
13
14    return 0;
15 }

```

6.8 Divisibilidad - Archmage's Crystals

O: (raiz de n) por divisibilidadUn archimago tiene n cristales con potencias a_i

C++/Training_Camp_2026/Day 9/A - Archmage's Crystals.py

```

1 n = int(input())
2 l = list(map(int, input().split()))
3
4 suma = sum(l)
5 c = 0
6 for i in range(n, 0, -1):
7     if (suma%i == 0):
8         break
9     c+=1
10    print(c)

```

6.9 Divisors - Aesthetics In Poetry

O: (raiz de n) por buscar divisoresDado un poema de N versos con longitudes $l_1..l_N$

C++/Training_Camp_2026/Day 7/A - Aesthetics in poetry.cpp

```

1 #include <bits/stdc++.h>
2 typedef long long ll;
3 using namespace std;
4
5 vector<ll> divisores(ll n)
6 {
7     vector<ll> div;
8     for (ll i = 1; i*i <= n; i++)
9     {
10         if (n % i == 0)
11         {
12             div.push_back(i);
13             if (i != n/i) div.push_back(n/i);
14         }
15     }
16     return div;
17 }
18
19 int main()
20 {
21     ll n; cin >> n;
22     vector<ll> xd(n);
23     for (int i = 0; i < n; i++)
24     {
25         cin >> xd[i];
26     }
27
28     vector<ll> juan = divisores(n);
29     sort(juan.begin(), juan.end());
30
31     for (ll k : juan)
32     {
33         if (k >= 2)
34         {
35
36             vector<ll> cont(k, 0);
37             for (ll v : xd)

```

6.9 Divisors - Aesthetics In Poetry (cont.)

```

38     {
39         cont[v % k]++;
40     }
41
42     bool ok = true;
43     for(ll c : cont)
44     {
45         if(c != n/k) { ok = false; break; }
46     }
47
48     if(ok)
49     {
50         cout << k << endl;
51         return 0;
52     }
53 }
54 }
55
56 cout << -1 << endl;
57 return 0;
58 }

```

6.10 Exponenciacion Modular (Torre de Exponentes) - Exponentiation II

O: (log n) por nivel de la torre, con el teorema de Euler

Calcula $a^{(b^c)}$ modulo $1e9+7$ para n consultas, con a, b, c hasta $1e9$

C++/Training_Camp_2026/Day 10/K - Exponentiation II.cpp

```

1  #include <iostream>
2  using namespace std;
3  typedef long long ll;
4  ll MOD = 1000000007;
5
6  ll expmod(ll b, ll e, ll m)
7  {
8      ll ans = 1;
9      while (e) {
10         if (e&1) ans = (1ll*ans*b) % m;
11         b = (1ll*b*b) % m;
12         e /= 2;
13     }
14     return ans;
15 }
16
17
18 int main()
19 {
20     ll n; cin >> n;
21
22     while(n--)
23     {
24         ll a , b , c; cin >> a >> b >> c;
25
26         cout<< expmod(a,expmod(b,c,MOD-1),MOD)<< endl;
27     }
28
29     return 0;
30 }

```

6.11 Factorizacion de un Numero Gigante Redondeado - Fuzzy Factorization

O: (raiz de Y), probando divisores

Factorizar un numero que llega como cadena y que esta redondeado

C++/ICPC/Regionals 2025/F - Fuzzy Factorization.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  typedef long long ll;
4  #define SIZE(c) int((c).size())
5  #define forn(i,n) for(int i = 0 ; i < n ; i++ )
6
7  const int DIGITS = 10;
8

```

6.11 Factorizacion de un Numero Gigante Redondeado - Fuzzy Factorization (cont.)

```

9  int main() {
10
11     ios::sync_with_stdio(false);
12     cin.tie(nullptr);
13
14     string s;
15
16     cin >> s;
17
18     map<ll,int> f;
19     forn(t,SIZE(s)-DIGITS)
20     {
21         f[2]++;
22         f[5]++;
23     }
24     ll Y = stoll(s.substr(0,min(10, SIZE(s))));
25
26     for(ll p = 2; p*p <= Y;p++)
27     {
28         while(Y % p == 0)
29         {
30             Y /= p;
31             f[p]++;
32         }
33     }
34 }
35
36 if(Y>1)
37     f[Y]++;
38
39
40 cout << SIZE(f) << "\n";
41 for(auto it : f)
42     cout << it.first << " " << it.second << "\n";
43
44 return 0;
45 }

```

6.12 Factorizacion en Digos

O: (log n * 8) probando divisores de 9 a 2

Uso: recursive(num) -> string con los digitos, o vacio si no se puede

Halla el menor numero cuyos digitos multiplicados dan N

C++/Notebook/Factorizacion en Digos.cpp

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  string recursive(long long num, string sol = "")
6  {
7      for (int i = 9; i > 1; --i)
8      {
9          if (num % i == 0)
10         {
11             return recursive(num / i, sol + to_string(i));
12         }
13     }
14     return sol;
15 }
16
17 int main()
18 {
19     ios::sync_with_stdio(false);
20     cin.tie(nullptr);
21
22     int t;
23     cin >> t;
24     while (t--)
25     {
26         long long n;
27         cin >> n;

```

6.12 Factorizacion en Digos (cont.)

```

28
29     if (n < 2)
30     {
31         cout << n << "\n";
32         continue;
33     }
34
35     string factors = recursive(n);
36
37     long long mult = 1;
38     for (char ch : factors)
39     {
40         mult *= (ch - '0');
41     }
42
43     if (mult == n)
44     {
45         sort(factors.begin(), factors.end());
46         cout << factors << "\n";
47     }
48     else
49     {
50         cout << -1 << "\n";
51     }
52 }
53
54 return 0;
55 }

```

6.13 GCD and LCM - Journeygrapes

O: $(\log(\min(a,b)))$ por el gcd

Calcula a dividido entre mcd(a,b), que es el multiplicador para obtener mcm(a,b) desde b

C++/Codeforces/journeygrapes.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     long long a, b;
6     cin >> a >> b;
7     //c++17
8     cout << a / gcd(a, b) << endl;
9
10    return 0;
11 }

```

6.14 Miller-Rabin y Pollard Rho (Factorizar hasta 10^{18})

O: $(\log^3 n)$ isPrime, $\sim(n^{1/4})$ esperado por factor en Pollard

Uso: isPrime(n); auto f = factorizar(n); -> primos ordenados, con repeticion

Primalidad exacta y factorizacion de cualquier numero de 64 bits en milisegundos

C++/Notebook/Miller-Rabin y Pollard Rho.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 typedef unsigned long long ull;
6
7 ull mulmod(ull a, ull b, ull m)
8 {
9     return ((__uint128_t)a * b % m;
10 }
11
12 ull powmod(ull a, ull e, ull m)
13 {
14     ull r = 1;
15     a %= m;
16     while (e)
17     {
18         if (e & 1) r = mulmod(r, a, m);
19         a = mulmod(a, a, m);
20     }
21 }

```

6.14 Miller-Rabin y Pollard Rho (Factorizar hasta 10^{18}) (cont.)

```

20     e >>= 1;
21 }
22 return r;
23 }
24
25 bool isPrime(ull n)
26 {
27     if (n < 2) return false;
28     for (ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
29     {
30         if (n % p == 0) return n == p;
31     }
32     ull d = n - 1;
33     int s = 0;
34     while ((d & 1) == 0)
35     {
36         d >>= 1;
37         s++;
38     }
39     for (ull a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
40     {
41         ull x = powmod(a, d, n);
42         if (x == 1 || x == n - 1) continue;
43         bool compuesto = true;
44         for (int r = 1; r < s && compuesto; r++)
45         {
46             x = mulmod(x, x, n);
47             if (x == n - 1) compuesto = false;
48         }
49         if (compuesto) return false;
50     }
51     return true;
52 }
53
54 // Un divisor no trivial de n (n compuesto, impar, sin factores < 100)
55 ull pollard(ull n)
56 {
57     static mt19937_64 rng(1234567);
58     while (true)
59     {
60         ull c = rng() % (n - 1) + 1, x = rng() % n, y = x, g = 1;
61         // v*v + c en 128 bits: con n > 2^63, (mulmod(v,v,n) + c) DESBORDA 64 bits, la
62         // secuencia deja de ser congruente mod p y rho tarda segundos (medido: 8 s)
63         auto f = [&](ull v) { return (ull)((__uint128_t)v * v + c) % n; };
64         while (g == 1)
65         {
66             ull prod = 1, xs = x, ys = y; // se guardan por si hay que rehacer
67             for (int i = 0; i < 64; i++) // un gcd cada 64 pasos, no cada paso
68             {
69                 x = f(x);
70                 y = f(f(y));
71                 prod = mulmod(prod, x > y ? x - y : y - x, n);
72             }
73             g = __gcd(prod, n);
74             if (g == n) // el lote se paso: rehacerlo de a uno
75             {
76                 x = xs;
77                 y = ys;
78                 g = 1;
79                 for (int i = 0; i < 64 && g == 1; i++)
80                 {
81                     x = f(x);
82                     y = f(f(y));
83                     g = __gcd(x > y ? x - y : y - x, n);
84                 }
85             }
86         }
87         if (g != n) return g; // g == n: mala suerte, otra c
88     }
89 }
90
91 void factorRec(ull n, vector<ull> &out)

```

6.14 Miller-Rabin y Pollard Rho (Factorizar hasta 10^{18}) (cont.)

```

92 {
93     if (n == 1) return;
94     if (isPrime(n))
95     {
96         out.push_back(n);
97         return;
98     }
99     ull d = pollard(n);
100     factorRec(d, out);
101     factorRec(n / d, out);
102 }
103
104 vector<ull> factorizar(ull n)
105 {
106     vector<ull> out;
107     for (ull p = 2; p < 100 && p * p <= n; p++)
108     {
109         while (n % p == 0)
110         {
111             out.push_back(p);
112             n /= p;
113         }
114     }
115     if (n > 1) factorRec(n, out);
116     sort(out.begin(), out.end());
117     return out;
118 }
119
120 int main()
121 {
122     ios_base::sync_with_stdio(false);
123     cin.tie(nullptr);
124     return 0;
125 }

```

6.15 Modular Arithmetic - Broul

O: ($\log n$), exponenciación modularCalcula el número mínimo de saltos de tamaño 3^b necesarios para cubrir la distancia vertical $\text{abs}(a-c)$

C++/CCPL/broul.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main() {
6     while(true){
7         int a,b,c;
8         cin >> a >> b >> c;
9         if(a==0 && b ==0 && c ==0)
10         {
11             return 0;
12         }
13
14         int god = abs(a-c);
15         if(god%b!=0)
16         {
17             cout << "No accounting tablet" << "\n";
18         }
19         else
20         {
21             if((god/b) % 3 ==0)
22             {
23                 cout << (god/b)/3<< "\n";
24             }
25             else
26             {
27
28                 cout << ((god/b)/3)+1<< "\n";
29             }
30         }
31     }

```

6.15 Modular Arithmetic - Broul (cont.)

```

32
33 }
34
35
36
37
38     return 0;
39 }

```

6.16 Modular Arithmetic - Creating Multiples

O: ($L * B$), probando bajar cada dígito

Bajar un solo dígito para que el número sea múltiplo de B+1, usando la suma alternada

C++/Training_Camp_2026/Day 4/B - Creating Multiples.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int B, L;
9     cin >> B >> L;
10
11     vector<int> D(L);
12     for (int &x : D) cin >> x;
13
14     int mod = B + 1;
15
16     // Calculate N modulo (B + 1).
17     // B = -1 (mod B+1), so powers alternate.
18     int rem = 0;
19     for (int i = 0; i < L; i++) {
20         if ((L - 1 - i) % 2 == 0)
21             rem += D[i];
22         else
23             rem -= D[i];
24         rem %= mod;
25     }
26     if (rem < 0) rem += mod;
27
28     if (rem == 0) {
29         cout << "0\n";
30         return 0;
31     }
32
33     // Try positions from left to right. The first possible position
34     // gives the smallest resulting number.
35     for (int i = 0; i < L; i++) {
36         int sign = ((L - 1 - i) % 2 == 0) ? 1 : -1;
37
38         // Need a decrease delta such that:
39         // rem - delta * sign == 0 (mod B+1)
40         int delta;
41         if (sign == 1)
42             delta = rem;
43         else
44             delta = (mod - rem) % mod;
45
46         // Digits are smaller than B+1, so this is the only possible delta.
47         if (delta <= D[i] && delta > 0) {
48             cout << i + 1 << ' ' << D[i] - delta << '\n';
49             return 0;
50         }
51     }
52 {
53     cout << "-1 -1\n";
54     return 0;
55 }

```

6.17 Modulo de Entero Gigante

O: (largo de la cadena)

Uso: modString(P,N) con P el numero como string y N el modulo

Cuando el numero de entrada tiene miles de digitos no cabe en long long

C++/Notebook/Modulo de Entero Gigante.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 long long modString(const string& P, long long N)
6 {
7     long long rem = 0;
8     for (char c : P)
9     {
10         int d = c - '0';
11         rem = (rem * 10 + d) % N;
12     }
13     return rem;
14 }
15
16 int main()
17 {
18     ios_base::sync_with_stdio(false);
19     cin.tie(nullptr);
20
21     string P;
22     long long N;
23     while (cin >> P >> N)
24     {
25         cout << modString(P, N) << "\n";
26     }
27
28     return 0;
29 }
```

6.18 Modulo de un Binario Gigante - Binary Dozens

O: (largo del binario), un barrido acumulando el residuo

Dado un numero en binario de hasta 500 bits

C++/ICPC/2025/B - Binary Dozens.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     string b;
6
7     while (cin >> b && b != "") {
8
9         int rem = 0;
10
11         for (char c : b) {
12             rem = (rem * 2 + (c - '0')) % 12;
13         }
14
15         cout << rem << '\n';
16     }
17
18     return 0;
19 }
```

6.19 Modulo de un Binario Gigante (Python) - Binary Dozens

O: (largo del binario); en Python los enteros son ilimitados

El resto entre 12 de un numero binario de hasta 500 bits

C++/ICPC/2025/B - Binary Dozens.py

```
1 while True:
2     b = input()
3
4     if b == "":
5         break
```

6.19 Modulo de un Binario Gigante (Python) - Binary Dozens (cont.)

```
6
7     n = int(b, 2)
8     print(n % 12)
```

6.20 Number Theory (Criba y Factorizacion) - Counting Divisors

O: (n log log n) la criba, (log n) factorizar cada consulta

Dados n enteros, imprime la cantidad de divisores de cada uno

C++/Training_Camp_2026/Day 10/I - Counting Divisors.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const int MX = 1000000;
7
8 bool marked[MX + 1];
9 vector<int> primes;
10
11 void sieve() {
12     marked[0] = marked[1] = true;
13
14     for (int i = 2; i <= MX; i++) {
15         if (marked[i]) continue;
16
17         primes.push_back(i);
18
19         if (1LL * i * i > MX) continue;
20         for (ll j = 1LL * i * i; j <= MX; j += i)
21             marked[j] = true;
22     }
23 }
24
25 ll divisors(ll n) {
26     ll ans = 1;
27
28     for (ll p : primes) {
29         if (p * p > n) break;
30
31         if (n % p == 0) {
32             int cnt = 0;
33             while (n % p == 0) {
34                 n /= p;
35                 cnt++;
36             }
37             ans *= (cnt + 1);
38         }
39     }
40
41     if (n > 1) ans *= 2;
42
43     return ans;
44 }
45
46 int main() {
47     ios::sync_with_stdio(false);
48     cin.tie(nullptr);
49
50     sieve();
51
52     int t;
53     cin >> t;
54
55     while (t--) {
56         ll k;
57         cin >> k;
58         cout << divisors(k) << '\n';
59     }
60
61     return 0;
62 }
```

6.21 Number Theory (GCD y Divisores) - The Seals Of Shanghai

O: (raiz de n) por buscar divisores

Eligiendo un modulo fijo $M > 1$, cada sello con numero A_i se desbloquea en la ronda de su residuo $A_i \bmod M$

C++/Training_Camp_2026/Day 10/6 - The Seals of Shanghai.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4 typedef unsigned long long ull;
5
6 static char buf_[1 << 22];
7 static int buflen_, bufPos_;
8 inline int gc() {
9     if (bufPos_ == buflen_) { buflen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0;
10     if (!buflen_) return -1; }
11     return buf_[bufPos_++];
12 }
13 inline ll readLL() {
14     int c = gc();
15     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
16     bool neg = false; if (c == '-') { neg = true; c = gc(); }
17     ll x = 0;
18     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
19     return neg ? -x : x;
20 }
21
22 const ll MAXM = 1000000000LL;
23
24 ll gcdll(ll a, ll b) { while (b) { a %= b; swap(a, b); } return a; }
25
26 // Counts divisors of d that are > 1.
27 ll countDivisorsGt1(ll d) {
28     if (d <= 1) return 0;
29     ll cnt = 0;
30     for (ll i = 1; i * i <= d; i++) {
31         if (d % i == 0) {
32             if (i > 1) cnt++;
33             ll other = d / i;
34             if (other != i && other > 1) cnt++;
35         }
36     }
37     return cnt;
38 }
39
40 void collectDivisorsGt1(ll d, vector<ll> &out) {
41     if (d <= 1) return;
42     for (ll i = 1; i * i <= d; i++) {
43         if (d % i == 0) {
44             if (i > 1) out.push_back(i);
45             ll other = d / i;
46             if (other != i && other > 1) out.push_back(other);
47         }
48     }
49 }
50
51 int main() {
52     int N = (int)readLL();
53     vector<ll> A(N);
54     for (auto &x : A) x = readLL();
55
56     if (N == 1) {
57         printf("1 %lld\n", MAXM - 1);
58         return 0;
59     }
60
61     ll D = 0;
62     for (int i = 1; i < N; i++) D = gcdll(D, llabs(A[i] - A[0]));
63
64     if (D == 0) {
65         printf("1 %lld\n", MAXM - 1);
66         return 0;
67     }
68 }

```

6.21 Number Theory (GCD y Divisores) - The Seals Of Shanghai (cont.)

```

69 if (D >= 2) {
70     printf("1 %lld\n", countDivisorsGt1(D));
71     return 0;
72 }
73
74 // D == 1: minimum is provably exactly 2 (M=2 always splits into exactly the two parit
75 y
76 // classes, and D=1 rules out cost=1 for any M>1). We still need to COUNT every M that
77 // achieves cost=2. Key fact used for counting: for ANY 3 elements, since there are on
78 ly
79 // 2 residue classes under an optimal M, by pigeonhole at least 2 of the 3 must share
80 a
81 // class, so M must divide at least one of their 3 pairwise differences. This holds fo
82 r
83 // every valid M and every triple, so scanning divisors of one arbitrary triple's 3
84 // differences is guaranteed to contain the full set of valid M's.
85 vector<ll> distinctVals;
86 {
87     vector<ll> tmp = A;
88     sort(tmp.begin(), tmp.end());
89     tmp.erase(unique(tmp.begin(), tmp.end()), tmp.end());
90     distinctVals = tmp;
91 }
92
93 if ((int)distinctVals.size() < 3) {
94     // Only 2 distinct values overall, differing by exactly 1 (since D=1): every M>1
95     // fails to merge them, so cost=2 for every single M from 2..MAXM.
96     printf("2 %lld\n", MAXM - 1);
97     return 0;
98 }
99
100 ll a = distinctVals[0], b = distinctVals[1], c = distinctVals[2];
101 ll d1 = llabs(a - b), d2 = llabs(b - c), d3 = llabs(a - c);
102
103 vector<ll> candidates;
104 collectDivisorsGt1(d1, candidates);
105 collectDivisorsGt1(d2, candidates);
106 collectDivisorsGt1(d3, candidates);
107 sort(candidates.begin(), candidates.end());
108 candidates.erase(unique(candidates.begin(), candidates.end()), candidates.end());
109
110 // Lightweight open-addressing hash set, sized by N (not by M), reused across all
111 // candidates via a version stamp. This keeps memory at O(N) regardless of how large
112 // M gets (unlike indexing directly by residue, which would need an array of size M,
113 // and M can be up to just under 1e9 here).
114 int tableSize = 1;
115 while (tableSize < 4 * N + 16) tableSize <= 1;
116 int mask = tableSize - 1;
117 vector<ll> tableKey(tableSize);
118 vector<int> tableStamp(tableSize, -1);
119
120 int bestCost = INT_MAX;
121 ll bestCount = 0;
122
123 for (int idx = 0; idx < (int)candidates.size(); idx++) {
124     ll M = candidates[idx];
125     int distinctCount = 0;
126     for (ll x : A) {
127         ll r = x % M;
128         ull h = (ull)r * 0x9E3779B97F4A7C15ULL;
129         int pos = (int)(h >> 40) & mask;
130         while (tableStamp[pos] == idx && tableKey[pos] != r) {
131             pos = (pos + 1) & mask;
132         }
133         if (tableStamp[pos] != idx) {
134             tableStamp[pos] = idx;
135             tableKey[pos] = r;
136             distinctCount++;
137         }
138     }
139     if (distinctCount < bestCost) {
140         bestCost = distinctCount;
141     }
142 }

```

6.21 Number Theory (GCD y Divisores) - The Seals Of Shanghai (cont.)

```

85         bestCount = 1;
86     } else if (distinctCount == bestCost) {
87         bestCount++;
88     }
89 }
90
91 printf("%d %lld\n", bestCost, bestCount);
92 return 0;
93 }
```

6.22 Number Theory (LCM via Factorizacion de Ciclos) - Permutation Rounds

O: (n log n), ciclos mas LCM por factorizacion

Rondas hasta que una permutacion vuelve al orden: LCM de los largos de sus ciclos

C++/Training_Camp_2026/Day 10/L - Permutation Rounds.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  typedef long long ll;
4
5  static char buf_[1 << 22];
6  static int bufLen_, bufPos_;
7  inline int gc() {
8      if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0;
9          if (!bufLen_) return -1; }
10     return buf_[bufPos_++];
11 }
12 inline int readInt() {
13     int c = gc();
14     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
15     bool neg = false; if (c == '-') { neg = true; c = gc(); }
16     int x = 0;
17     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
18     return neg ? -x : x;
19 }
20
21 const ll MOD = 1000000007LL;
22 ll power_mod(ll base, ll exp, ll mod) {
23     base %= mod;
24     ll result = 1;
25     while (exp > 0) {
26         if (exp & 1) result = (result * base) % mod;
27         base = (base * base) % mod;
28         exp >>= 1;
29     }
30     return result;
31 }
32
33 int main() {
34     int n = readInt();
35     vector<int> p(n + 1);
36     for (int i = 1; i <= n; i++) p[i] = readInt();
37
38     vector<char> visited(n + 1, 0);
39     unordered_map<int, int> maxPower; // prime -> max exponent seen across all cycle lengths
40
41     for (int i = 1; i <= n; i++) {
42         if (visited[i]) continue;
43         int length = 0;
44         int j = i;
45         while (!visited[j]) {
46             visited[j] = 1;
47             j = p[j];
48             length++;
49         }
50         // factorize `length`, updating maxPower
51         int x = length;
52         for (int d = 2; (ll)d * d <= x; d++) {
53             if (x % d == 0) {
54                 int exp = 0;

```

6.22 Number Theory (LCM via Factorizacion de Ciclos) - Permutation Rounds (cont.)

```

55         while (x % d == 0) { x /= d; exp++; }
56         auto it = maxPower.find(d);
57         if (it == maxPower.end() || it->second < exp) maxPower[d] = exp;
58     }
59 }
60 if (x > 1) {
61     auto it = maxPower.find(x);
62     if (it == maxPower.end() || it->second < 1) maxPower[x] = 1;
63 }
64 }
65
66 ll result = 1;
67 for (auto &pr : maxPower) {
68     result = (result * power_mod(pr.first, pr.second, MOD)) % MOD;
69 }
70
71 printf("%lld\n", result);
72 return 0;
73 }
```

6.23 Reconstruccion con Divisores - Remainders

O: (raiz de n) buscando divisores

Reconstruye el multiconjunto oculto K a partir de la sucesion S(n) = suma de (n mod k) para todo k en K

C++/RPC/Remainders.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int N;
6      if (!(cin >> N)) return 0;
7
8      vector<long long> S(N + 1, 0); // S[0] = 0
9      for (int i = 1; i <= N; i++) cin >> S[i];
10
11     long long m = S[1]; // |K| = S(1)
12     vector<long long> cnt(N + 1, 0);
13
14     for (int n = 2; n <= N; n++) {
15         long long D = m - (S[n] - S[n - 1]); // suma de los k de K que dividen a n
16         for (int k = 2; k < n; k++)
17             if (n % k == 0) D -= (long long)k * cnt[k];
18         if (D > 0) cnt[n] = D / n; // lo que queda es cnt[n]*n
19     }
20
21     vector<int> K;
22     for (int k = 2; k <= N; k++)
23         for (int i = 0; i < cnt[k]; i++) K.push_back(k);
24
25     cout << K.size();
26     for (int x : K) cout << ' ' << x;
27     cout << '\n';
28     return 0;
29 }
```

6.24 Sieve of Eratosthenes - Prime Sieve

O: (n log log n) la criba, (1) por consulta

Dado un limite n (hasta 10^8) y q consultas x

C++/Training_Camp_2026/Day 11 - CCPLUE - Prime Sieve.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6
7
8  const int MX = 100000000;
9
10
11

```

6.24 Sieve of Eratosthenes - Prime Sieve (cont.)

```

2 vector<bool> marked(MX + 1, false);
3 int cont = 0;
4
5
6 void sieve(int uwu) {
7     marked[0] = marked[1] = true;
8
9     for (int i = 2; i <= MX; i++) {
10         if (marked[i]) continue;
11
12         if (i <= uwu)
13         {
14             cont++;
15         }
16         if (1LL * i * i > MX) continue;
17         for (ll j = 1LL * i * i; j <= MX; j += i)
18             marked[j] = true;
19     }
20 }
21
22 int main() {
23     ios::sync_with_stdio(false);
24     cin.tie(nullptr);
25
26     int t;
27     cin >> t;
28
29     sieve(t);
30
31     int n; cin >> n;
32     cout << cont << endl;
33     while (n--) {
34         int aa; cin >> aa;
35
36         if (marked[aa] == false)
37         {
38             cout << 1 << endl;
39         }
40         else
41         {
42             cout << 0 << endl;
43         }
44     }
45
46     return 0;
47 }

```

6.25 Ternas Pitagoricas - Pythagorean

O: (n) sobre las ternas generadas

Dado n, cuenta en cuantas ternas pitagoricas participa, separando cuatro casos

C++/RPC/Pythagorean.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 int main() {
6     ll n;
7     if (!(cin >> n)) return 0;
8
9     ll hypPrim = 0, hypNon = 0, legPrim = 0, legNon = 0;
10    ll n2 = n * n;
11
12    // --- n como hipotenusa ---
13    for (ll a = 1; a < n; a++) {
14        ll t = n2 - a * a;
15        ll b = (ll)sqrtl((long double)t);
16        while (b * b < t) b++;
17        while (b * b > t) b--;
18        if (b * b == t && b > a) { // b > a evita contar dos veces
19            if (__gcd(a, b) == 1) hypPrim++;

```

6.25 Ternas Pitagoricas - Pythagorean (cont.)

```

20        else hypNon++;
21    }
22 }
23
24 // --- n como cateto: (c-b)(c+b) = n^2 ---
25 for (ll d = 1; d < n; d++) { // d = c-b, d < e obliga d < n
26     if (n2 % d) continue;
27     ll e = n2 / d; // e = c+b
28     if (((e - d) & 1LL)) continue; // misma paridad
29     ll b = (e - d) / 2;
30     ll c = (e + d) / 2;
31     if (b <= 0) continue;
32     if (__gcd(n, b) == 1) legPrim++;
33     else legNon++;
34 }
35
36 cout << hypPrim << ' ' << hypNon << ' '
37      << legPrim << ' ' << legNon << '\n';
38 return 0;
39 }

```

7 - Combinatorics

7.1 Combinatoria (Factoriales Modulares) - Binomial Coefficients

O: (MAXN) de precalculo y (1) por binomial

Calcula n coeficientes binomiales C(a,b) modulo 1e9+7

C++/Training_Camp_2026/Day 10/J - Binomial Coefficients.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const int MOD = 1000000007;
7 const int MAXN = 1000000;
8
9 vector<ll> fact(MAXN + 1), invFact(MAXN + 1);
10
11 ll modPow(ll a, ll e) {
12     ll res = 1;
13     while (e > 0) {
14         if (e & 1) res = res * a % MOD;
15         a = a * a % MOD;
16         e >>= 1;
17     }
18     return res;
19 }
20
21 int main() {
22     ios::sync_with_stdio(false);
23     cin.tie(nullptr);
24
25     fact[0] = 1;
26     for (int i = 1; i <= MAXN; i++)
27         fact[i] = fact[i - 1] * i % MOD;
28
29     invFact[MAXN] = modPow(fact[MAXN], MOD - 2);
30     for (int i = MAXN; i >= 1; i--)
31         invFact[i - 1] = invFact[i] * i % MOD;
32
33     int n;
34     cin >> n;
35
36     while (n--) {
37         int a, b;
38         cin >> a >> b;
39
40         ll ans = fact[a];
41         ans = ans * invFact[b] % MOD;
42         ans = ans * invFact[a - b] % MOD;

```

7.1 Combinatoria (Factoriales Modulares) - Binomial Coefficients (cont.)

```

44
45     cout << ans << '\n';
46 }
47
48     return 0;
49 }

```

7.2 Combinatoria Avanzada (Lucas, Catalan, Stirling)

O: (1) por C tras precomputar(); Lucas $O(\log_p n)$

Uso: precomputar() una vez; luego $C(n, k)$, multinomial(v), catalan(n)
Lucas, Catalan, Stirling y multinomial, mas alla del binomial basico

C++/Notebook/Combinatoria Avanzada.cpp

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  using ll = long long;
6
7  ll modpow(ll a, ll e, ll m)
8  {
9      ll r = 1 % m;
10     a %= m;
11     while (e)
12     {
13         if (e & 1) r = r * a % m;
14         a = a * a % m;
15         e >>= 1;
16     }
17     return r;
18 }
19
20 // C(n,k) mod p directo en O(k), para n y k chicos (uso interno de Lucas)
21 ll Csmall(ll n, ll k, ll p)
22 {
23     if (k < 0 || k > n) return 0;
24     ll num = 1, den = 1;
25     for (ll i = 1; i <= k; i++)
26     {
27         num = num * ((n - k + i) % p) % p;
28         den = den * i % p;
29     }
30     return num * modpow(den, p - 2, p) % p;
31 }
32
33 // Teorema de Lucas: C(n,k) mod p con n, k enormes y p primo pequeno
34 ll C_lucas(ll n, ll k, ll p)
35 {
36     ll res = 1;
37     while (n || k)
38     {
39         ll ni = n % p, ki = k % p;
40         res = res * Csmall(ni, ki, p) % p;
41         n /= p;
42         k /= p;
43         if (!res) return 0;
44     }
45     return res;
46 }
47
48 const int MOD = 1e9 + 7;
49 const int MAXN = 1000000;
50 vector<ll> fact(MAXN + 1), invFact(MAXN + 1);
51
52 void precomputar()
53 {
54     fact[0] = 1;
55     for (int i = 1; i <= MAXN; i++) fact[i] = fact[i - 1] * i % MOD;
56     invFact[MAXN] = modpow(fact[MAXN], MOD - 2, MOD);
57     for (int i = MAXN; i >= 1; i--) invFact[i - 1] = invFact[i] * i % MOD;
58 }

```

7.2 Combinatoria Avanzada (Lucas, Catalan, Stirling) (cont.)

```

59
60 ll C(int n, int k)
61 {
62     if (k < 0 || k > n) return 0;
63     return fact[n] * invFact[k] % MOD * invFact[n - k] % MOD;
64 }
65
66 // n! / (a1! * a2! * ... * ak!) : repartir n objetos en grupos de tamanos dados
67 ll multinomial(const vector<int>& a)
68 {
69     ll n = 0;
70     for (int x : a) n += x;
71     ll r = fact[n];
72     for (int x : a) r = r * invFact[x] % MOD;
73     return r;
74 }
75
76 // Catalan(n) = C(2n,n) / (n+1) : parentesis balanceados, arboles binarios, caminos de Dyc
77 ll catalan(int n)
78 {
79     return C(2 * n, n) * modpow(n + 1, MOD - 2, MOD) % MOD;
80 }
81
82 // Stirling de segunda especie: partir n elementos en k grupos no vacios
83 vector<vector<ll>> stirling2(int n, int k)
84 {
85     vector<vector<ll>> S(n + 1, vector<ll>(k + 1, 0));
86     S[0][0] = 1;
87     for (int i = 1; i <= n; i++)
88     {
89         for (int j = 1; j <= k; j++)
90         {
91             S[i][j] = (S[i - 1][j - 1] + (ll)j * S[i - 1][j]) % MOD;
92         }
93     }
94     return S;
95 }
96
97 int main()
98 {
99     ios_base::sync_with_stdio(false);
100    cin.tie(nullptr);
101    return 0;
102 }

```

7.3 Construction - Farmpiggy And Subset Sum

O: (n), intercambiar consecutivos de a dos

Construir una permutacion sin puntos fijos intercambiando cada par de consecutivos

C++/Codeforces/farmpiggy And Subset Sum.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios_base::sync_with_stdio(false);
6      cin.tie(NULL);
7
8      int t;
9      cin >> t;
10     while (t--) {
11         int n;
12         cin >> n;
13
14         vector<int> p(n + 1);
15         for (int i = 1; i <= n; i += 2) {
16             p[i] = i + 1;
17             p[i + 1] = i;
18         }
19
20         for (int i = 1; i <= n; i++) {

```

7.3 Construction - Farmpiggy And Subset Sum (cont.)

```

21         cout << p[i];
22         if (i != n) cout << ' ';
23     }
24     cout << "\n";
25 }
26
27     return 0;
28 }

```

7.4 Conteo de Distintos con Inventario de Frecuencias - Total Numbers

O: (1000), tres ciclos sobre los dígitos con inventario de frecuencias

Dado un arreglo de dígitos, contar cuantos números DISTINTOS de tres cifras se pueden armar usando tres de...

C++/LeetCode/totalNumbers.cpp

```

1  class Solution {
2  public:
3      int totalNumbers(vector<int>& digits) {
4          int cnt[10] = {};
5
6          for (int x : digits) {
7              cnt[x]++;
8          }
9
10         int ans = 0;
11
12         for (int last : {0, 2, 4, 6, 8}) {
13             if (cnt[last] == 0) continue;
14
15             cnt[last]--;
16
17             for (int first = 1; first <= 9; first++) {
18                 if (cnt[first] == 0) continue;
19
20                 cnt[first]--;
21
22                 for (int second = 0; second <= 9; second++) {
23                     if (cnt[second] > 0) ans++;
24                 }
25
26                 cnt[first]++;
27             }
28
29             cnt[last]++;
30         }
31
32         return ans;
33     }
34 };

```

7.5 Conteo de Inversiones (cadena repetida) - Inversions

O: (n log n) con Fenwick sobre la cadena repetida

Cuenta las inversiones de S repetida N veces (N hasta 1e12) modulo 1e9+7

C++/Codeforces/Inversions.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  long long MOD = 1000000007;
5
6  long long modpow(long long a, long long e) {
7      long long ans = 1;
8
9      while (e) {
10         if (e & 1)
11             ans = ans * a % MOD;
12
13         a = a * a % MOD;
14         e >>= 1;
15     }
16 }

```

7.5 Conteo de Inversiones (cadena repetida) - Inversions (cont.)

```

17     return ans;
18 }
19
20 long long division_mod(long long a, long long b) {
21     return a % MOD * modpow(b, MOD - 2) % MOD;
22 }
23
24
25
26 int main()
27 {
28     string s; cin >> s;
29     long long n; cin >> n;
30
31     vector<vector<long long>> freqAtras;
32     vector<long long> freq(26,0);
33     vector<long long> vectorsito(26,0);
34
35     for(long long i = s.size()-1; i >=0; i--)
36     {
37         char letra = s[i];
38         freq[letra-'0'-49]++;
39         for(long long j = 0; j < 26; j++)
40         {
41             if(letra-'0'-49 < j)
42             {
43                 vectorsito[j]++;
44             }
45         }
46         freqAtras.push_back(vectorsito);
47     }
48
49     vector<long long> freqGod(26,0);
50     long long sum = 0;
51     for(long long i =1; i <26; i++)
52     {
53         sum += freq[i-1];
54         freqGod[i] = sum;
55     }
56
57
58     long long GOD = 0;
59     for(long long i = 0; i < s.size(); i++)
60     {
61         if(freqGod[s[i]-'0'-49] == freqAtras[s.size()-i-1][s[i]-'0'-49])
62         {
63             GOD = (GOD + division_mod((n % MOD) * ((n + 1) % MOD) % MOD, 2) * freqGod[s[
64 i]-'0'-49] % MOD) % MOD;
65         }
66         else
67         {
68             GOD = (GOD + division_mod(((n-1) % MOD) * ((n) % MOD) % MOD, 2) * freqGod[s[
69 i]-'0'-49] % MOD) % MOD;
70             GOD = (GOD + (freqAtras[s.size()-i-1][s[i]-'0'-49] * n) % MOD) % MOD;
71         }
72     }
73
74     cout << GOD << endl;
75
76     return 0;
77 }

```

7.6 Euler's Formula - Pizza Cutter

O: (1), fórmula de Euler para caras

El abuelo Giuseppe corta una pizza rectangular con H cortes que entran por el lado izquierdo y salen por el...

C++/Training_Camp_2026/Day 8/C - Pizza Cutter.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;

```

7.6 Euler's Formula - Pizza Cutter (cont.)

```

3
4 static char buf_[1 << 22];
5 static int buflen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == buflen_) { buflen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0;
8         if (!buflen_) return -1; }
9     return buf_[bufPos_++];
10 }
11 inline long long readLL() {
12     int c = gc();
13     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
14     bool neg = false; if (c == '-') { neg = true; c = gc(); }
15     long long x = 0;
16     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
17     return neg ? -x : x;
18 }
19 // Counts inversions in 'a' (pairs i<j with a[i] > a[j]) via merge sort. O(n log n).
20 long long countInversions(vector<long long>& a) {
21     int n = (int)a.size();
22     if (n <= 1) return 0;
23     vector<long long> buf(n);
24     long long inv = 0;
25     for (int width = 1; width < n; width *= 2) {
26         for (int i = 0; i < n; i += 2 * width) {
27             int mid = min(i + width, n);
28             int end = min(i + 2 * width, n);
29             int p = i, q = mid, k = i;
30             while (p < mid && q < end) {
31                 if (a[p] <= a[q]) buf[k++] = a[p++];
32                 else { buf[k++] = a[q++]; inv += (mid - p); }
33             }
34             while (p < mid) buf[k++] = a[p++];
35             while (q < end) buf[k++] = a[q++];
36             for (int t = i; t < end; t++) a[t] = buf[t];
37         }
38     }
39     return inv;
40 }
41
42 int main() {
43     // X, Y (pizza dimensions) are read for input-format purposes only; the piece
44     // count depends solely on H, V, and the relative order of cut endpoints.
45     (void)readLL(); (void)readLL();
46     long long H = readLL(), V = readLL();
47
48     vector<pair<long long, long long>> hcuts(H), vcuts(V);
49     for (auto& p : hcuts) { p.first = readLL(); p.second = readLL(); }
50     for (auto& p : vcuts) { p.first = readLL(); p.second = readLL(); }
51
52     sort(hcuts.begin(), hcuts.end());
53     vector<long long> y2seq;
54     y2seq.reserve(H);
55     for (auto& p : hcuts) y2seq.push_back(p.second);
56     long long hh = countInversions(y2seq);
57
58     sort(vcuts.begin(), vcuts.end());
59     vector<long long> x2seq;
60     x2seq.reserve(V);
61     for (auto& p : vcuts) x2seq.push_back(p.second);
62     long long vv = countInversions(x2seq);
63
64     long long answer = 1 + H + V + hh + vv + H * V;
65     printf("%lld\n", answer);
66     return 0;
67 }

```

7.7 Formula Cerrada con Inverso Modular - Number Of Sets

O: (k) por el binomial con inverso modular

Contar k segmentos sin solaparse sobre n puntos: colapsa en el binomial $C(n+k-1, 2k)$

C++/LeetCode/NumberOfSets.cpp

```

1 class Solution {
2 public:
3     long long MOD = 1000000007;
4
5     long long modpow(long long a, long long e) {
6         long long ans = 1;
7
8         while (e) {
9             if (e & 1)
10                 ans = ans * a % MOD;
11
12             a = a * a % MOD;
13             e >>= 1;
14         }
15
16         return ans;
17     }
18
19     long long division_mod(long long a, long long b) {
20         return a % MOD * modpow(b, MOD - 2) % MOD;
21     }
22
23     int numberOfSets(int n, int k) {
24         long long limite = n + k - 1;
25
26         vector<long long> factorial(limite + 1);
27         vector<long long> inverso(limite + 1);
28
29         factorial[0] = 1;
30
31         for (long long i = 1; i <= limite; i++) {
32             factorial[i] = factorial[i - 1] * i % MOD;
33         }
34
35         inverso[limite] = modpow(factorial[limite], MOD - 2);
36
37         for (long long i = limite; i >= 1; i--) {
38             inverso[i - 1] = inverso[i] * i % MOD;
39         }
40
41         long long GOD = factorial[limite];
42
43         GOD = GOD * inverso[2 * k] % MOD;
44         GOD = GOD * inverso[limite - 2 * k] % MOD;
45
46         return GOD;
47     }
48
49     // Lo mismo sin la variable limite, escribiendo C(n+k-1, 2k) completo:
50     //
51     //     long long GOD = factorial[n + k - 1];
52     //
53     //     GOD = GOD * inverso[2 * k] % MOD;
54     //     GOD = GOD * inverso[n - k - 1] % MOD;
55     //
56     //     return GOD;
57     //
58     // Es identico porque limite - 2k = (n + k - 1) - 2k = n - k - 1.
59     // Ayuda a ver que esto es literalmente C(arriba, abajo) con la forma
60     // factorial[arriba] * inverso[abajo] * inverso[arriba - abajo].
61 };

```

7.8 Paridad de Binomiales (Kummer) - Less Coint Tosses

O: ($\log n$), paridad del binomial por Kummer

Hay que repartir las 2^N cadenas binarias de longitud N entre Carla y Daniel de forma que la probabilidad...

C++/Training_Camp_2026/Day 1/L - Less_coint_tosses.py

```
1 import math
2 n = int(input())
3 m = bin(n)[2:]
4 j = m.count("1")
5 print(2**j)
```

7.9 Permutation Cycles - Lines Of Containers

O: ($L \cdot C$), ciclos de la permutacion de filas y columnas

Si la grua puede reordenar los contenedores: ciclos de la permutacion por filas

C++/Training_Camp_2026/Day 2/G - Lines of Containers.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 /*
5  Calcula el mínimo número de swaps necesarios para convertir
6  una permutación en la identidad.
7
8  Si una permutación tiene un ciclo de tamaño k,
9  se necesitan k-1 intercambios para corregirlo.
10 */
11 int minSwaps(vector<int> &perm) {
12     int n = perm.size();
13     vector<bool> vis(n, false);
14     int ans = 0;
15     for (int i = 0; i < n; i++) {
16         // Si ya fue visitado o ya está en su lugar
17         if (vis[i] || perm[i] == i)
18             continue;
19         int cur = i;
20         int cycleSize = 0;
21         // Recorremos el ciclo completo
22         while (!vis[cur]) {
23             vis[cur] = true;
24             cur = perm[cur];
25             cycleSize++;
26         }
27         // Un ciclo de tamaño k necesita k-1 swaps
28         ans += cycleSize - 1;
29     }
30     return ans;
31 }
32
33 int main() {
34     ios::sync_with_stdio(false);
35     cin.tie(nullptr);
36     int L, C;
37     cin >> L >> C;
38     // Matriz con la configuración actual
39     vector<vector<int>> a(L, vector<int>(C));
40     for (int i = 0; i < L; i++)
41         for (int j = 0; j < C; j++)
42             cin >> a[i][j];
```

7.9 Permutation Cycles - Lines Of Containers (cont.)

```
51 /*
52     rowPerm[i] = fila ORIGINAL a la que realmente pertenece
53     la fila actual i.
54
55     Ejemplo:
56     rowPerm = [3,1,2,0]
57
58     significa:
59     la fila actual 0 debería ser la fila 3,
60     la fila actual 1 debería ser la fila 1,
61     etc.
62 */
63 vector<int> rowPerm(L, -1);
64
65 // Revisamos fila por fila
66 for (int i = 0; i < L; i++) {
67     /*
68         La fila original de un contenedor se obtiene como
69         (id-1)/C
70         porque cada fila contiene exactamente C elementos.
71 */
72     int expectedRow = (a[i][0] - 1) / C;
73     // Todos los elementos de la fila deben pertenecer
74     // a la misma fila original.
75     for (int j = 1; j < C; j++) {
76         if ((a[i][j] - 1) / C != expectedRow) {
77             cout << "\n";
78             return 0;
79         }
80     }
81     rowPerm[i] = expectedRow;
82 }
83
84 /*
85     Además debe ser una permutación válida.
86     No pueden existir dos filas actuales que correspondan
87     a la misma fila original.
88 */
89 vector<bool> used(L, false);
90 for (int x : rowPerm) {
91     if (used[x]) {
92         cout << "\n";
93         return 0;
94     }
95     used[x] = true;
96 }
97
98 /*
99     colPerm[j] = columna ORIGINAL a la que corresponde
100     la columna actual j.
101 */
102 vector<int> colPerm(C, -1);
103
104 // Revisamos columna por columna
105 for (int j = 0; j < C; j++) {
106     /*
107         La columna correcta de un contenedor es
108         (id-1)%C
109     */
110     int expectedCol = (a[0][j] - 1) % C;
```

7.9 Permutation Cycles - Lines Of Containers (cont.)

```

89
90 // Todos los elementos de la columna deben pertenecer
91 // a la misma columna original.
92 for (int i = 1; i < L; i++) {
93
94     if ((a[i][j] - 1) % C != expectedCol) {
95         cout << "\n";
96         return 0;
97     }
98 }
99
100 colPerm[j] = expectedCol;
101 }
102
103 // Verificamos nuevamente que sea una permutación válida
104 used.assign(C, false);
105
106 for (int x : colPerm) {
107
108     if (used[x]) {
109         cout << "\n";
110         return 0;
111     }
112
113     used[x] = true;
114 }
115
116 /*
117 Si llegamos aquí, existe una secuencia de intercambios.
118 El número mínimo es:
119
120     swaps para ordenar las filas
121     +
122     swaps para ordenar las columnas
123 */
124 cout << minSwaps(rowPerm) + minSwaps(colPerm) << "\n";
125
126 return 0;
127 }

```

7.10 Probabilidad (Paradoja del cumpleaños) - Mount Rushmore And Birthdays

O: (n), probabilidad por complemento
 Dado N, el numero de días que tiene un año

C++/Training_Camp_2026/Day 5/F - Mount Rushmore and Birthdays.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4
5 int main() {
6     ios::sync_with_stdio(false);
7     cin.tie(nullptr);
8     double n; cin >> n;
9     double prob = 1;
10    double persons = 2;
11    prob *= (n-1)/n;
12
13    if(n==1)
14    {
15        cout << 2 << endl;
16        return 0;
17    }
18
19    while(true)
20    {
21        prob *= (n-persons)/n;
22
23
24        persons++;
25        if(1-prob>0.5)

```

7.10 Probabilidad (Paradoja del cumpleaños) - Mount Rushmore And Birthdays (cont.)

```

26    {
27        cout << persons << endl;
28        return 0;
29    }
30
31    }
32
33
34
35
36
37    return 0;
38 }

```

7.11 Probabilidad (Valor esperado con inverso modular) - Frangolino Ali Na Mesa

O: (Q log MOD) por el inverso modular de cada paso

Valor esperado cuando cada comando es una de dos cosas al 50%, con inverso modular

C++/Training_Camp_2026/Day 6/F - Frangolino ali na mesa.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static const int MOD = 1000000007;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int N, Q;
11    cin >> N >> Q;
12
13    vector<int> X(Q + 1);
14
15    long long inv2 = (MOD + 1) / 2;
16
17    vector<long long> invPow(Q + 1), pow2(Q + 1);
18    invPow[0] = 1;
19    pow2[0] = 1;
20    for (int i = 1; i <= Q; i++) {
21        invPow[i] = invPow[i - 1] * inv2 % MOD;
22        pow2[i] = pow2[i - 1] * 2 % MOD;
23    }
24
25    vector<long long> coeff(Q + 1);
26    for (int i = 1; i <= Q; i++) {
27        cin >> X[i];
28        coeff[i] = 1LL * X[i] * invPow[i] % MOD;
29    }
30
31    vector<long long> suffix(Q + 2, 0);
32    for (int i = Q; i >= 1; i--) {
33        suffix[i] = (suffix[i + 1] + coeff[i]) % MOD;
34    }
35
36    vector<long long> ans(N + 1, 0);
37
38    // initial value at table 1
39    ans[1] = suffix[1];
40
41    // increments
42    for (int i = 1; i <= Q; i++) {
43        long long add = pow2[i - 1] * suffix[i + 1] % MOD;
44        ans[X[i]] += add;
45        if (ans[X[i]] >= MOD) ans[X[i]] -= MOD;
46    }
47
48    for (int i = 1; i <= N; i++)
49        cout << ans[i] % MOD << "\n";
50
51    return 0;
52 }

```

7.12 Ranking - Palindromic Lexi Small

O: (n + 26), frecuencias y las dos mitades

Construye el k-esimo palindromo mas pequeno lexicograficamente que se puede formar reordenando el string S...

C++/LeetCode/palindromicLexiSmall.cpp

```
1 class Solution {
2 public:
3     const long long LIM = 1000000;
4
5     // C(n, r) saturado a LIM+1 si el valor real lo supera.
6     // O(r), con divisi n exacta garantizada en cada paso (identidad de binomiales),
7     // en vez de armar vectores de tama o r y simplificar con gcd (eso era O(r^2) y el cue
8     // llo de botella real).
9     long long comb(long long n, long long r) {
10         if (r < 0 || r > n) return 0;
11         r = min(r, n - r);
12         long long result = 1;
13         for (long long i = 1; i <= r; i++) {
14             result = result * (n - i + 1) / i;
15             if (result > LIM) return LIM + 1;
16         }
17         return result;
18     }
19
20     // Permutaciones distintas del multiset cnt[0..25] (n = total restante), saturado a LI
21     // M+1.
22     long long perms(long long n, const array<int,26>& cnt) {
23         long long ans = 1;
24         long long restantes = n;
25         for (int c = 0; c < 26; c++) {
26             if (cnt[c] == 0) continue;
27             long long ways = comb(restantes, cnt[c]);
28             if (ways > LIM) return LIM + 1;
29             if (ans > LIM / ways) return LIM + 1;
30             ans *= ways;
31             restantes -= cnt[c];
32             if (ans > LIM) return LIM + 1;
33         }
34         return ans;
35     }
36
37     string smallestPalindrome(string s, int k) {
38         int n = s.size();
39         bool mid = (n % 2 != 0);
40         char midc = mid ? s[n / 2] : '\0';
41         string half = s.substr(0, n / 2);
42
43         array<int,26> cnt{};
44         long long tot = 0;
45         for (char c : half) { cnt[c - 'a']++; tot++; }
46
47         if ((long long)k > perms(tot, cnt)) return "";
48
49         string god;
50         god.reserve(half.size());
51         long long cont = 0;
52         long long curTot = tot;
53
54         for (size_t i = 0; i < half.size(); i++) {
55             for (int c = 0; c < 26; c++) {
56                 if (cnt[c] == 0) continue;
57                 cnt[c]--;
58                 curTot--;
59                 long long permi = perms(curTot, cnt);
60                 if (cont + permi < k) {
61                     cont += permi;
62                     cnt[c]++;
63                     curTot++;
64                 } else {
65                     god += char('a' + c);
66                     break;
67                 }
68             }
69         }
70     }
71 }
```

7.12 Ranking - Palindromic Lexi Small (cont.)

```
65     }
66 }
67 }
68
69 string sol = god;
70 if (mid) sol += midc;
71 reverse(god.begin(), god.end());
72 sol += god;
73 return sol;
74 }
75 };
```

7.13 Ranking de Combinaciones - Discrete Catalog

O: (n*k), ranking de combinaciones con binomiales

Dado un subconjunto de tamano k escrito en orden creciente

C++/ICPC/2025/D - Discrete Catalog.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 ll nCr(int n, int k) {
7     if (k < 0 || k > n)
8         return 0;
9
10    k = min(k, n - k);
11
12    ll r = 1;
13
14    for (int i = 1; i <= k; i++) {
15        r = r * (n - i + 1) / i;
16    }
17
18    return r;
19 }
20
21 int main() {
22     ios::sync_with_stdio(false);
23     cin.tie(nullptr);
24
25     int n, k;
26
27     while (cin >> n >> k) {
28         vector<int> a(k);
29
30         for (int &x : a)
31             cin >> x;
32
33         ll god = 0;
34
35         int anterior = 0;
36
37         for (int i = 0; i < k; i++)
38         {
39             for (int j = anterior + 1; j < a[i]; j++)
40             {
41                 god += nCr(n - j, k - i - 1);
42             }
43
44             anterior = a[i];
45         }
46
47         cout << god << endl;
48     }
49
50     return 0;
51 }
```

7.14 Suma de Subconjuntos (2^n) - Group Work

O: (2^n), todos los subconjuntos

Dada una clase de N estudiantes, cuenta cuantos grupos distintos de al menos 2 estudiantes se pueden formar...

C++/Training_Camp_2026/Day 7/C - Group work.py

```
1 import math
2 n = int(input())
3
4 suma = 0
5 for m in range(2,n+1):
6     suma+=math.comb(n,m)
7 print(suma)
```

7.15 Tecnica de Contribucion - Bytelandia's Stones

O: (n), tecnica de contribucion

Suma de todos los subarreglos: tecnica de contribucion, cada a_i aparece un numero fijo

C++/Training_Camp_2026/Day 9/B - Bytelandia's stones.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9     cin >> n;
10
11     long long ans = 0;
12
13     for (int i = 1; i <= n; i++)
14     {
15         long long x;
16         cin >> x;
17         ans += x * 1LL * i * (n - i + 1);
18     }
19
20     cout << ans << endl;
21     return 0;
22 }
```

7.16 Triangulos en Poligono Convexo

O: (1), es $C(n,3)$

Uso: $c3(n) = n*(n-1)*(n-2)/6$ // todo trio de un convexo es triangulo

Dado n, cuantos triangulos distintos salen escogiendo 3 vertices de un poligono regular convexo de n lados

C++/Notebook/Triangulos en Poligono Convexo.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 // C(n,3) sin formar el producto completo.
6 ll c3(ll n) {
7     if (n < 3) return 0;
8
9     ll a = n, b = n - 1, c = n - 2;
10
11     // Entre tres consecutivos hay exactamente un multiplo de 3.
12     if (a % 3 == 0) {
13         a /= 3;
14     } else if (b % 3 == 0) {
15         b /= 3;
16     } else {
17         c /= 3;
18     }
19
20     // Y al menos uno par (puede que el de arriba ya haya cambiado la paridad).
21     if (a % 2 == 0) {
22         a /= 2;
23     } else if (b % 2 == 0) {
```

7.16 Triangulos en Poligono Convexo (cont.)

```
24     b /= 2;
25     } else {
26         c /= 2;
27     }
28
29     return a * b * c;
30 }
31
32 int main() {
33     ios::sync_with_stdio(false);
34     cin.tie(nullptr);
35
36     ll n;
37     while (cin >> n) {
38         cout << c3(n) << '\n';
39     }
40     return 0;
41 }
```

8 - Game Theory

8.1 Misere Nim y Grundy Acotado

O: (n) misere_nim, (1) Grundy_bounded

Uso: misere_nim(pilas) -> gana el primero?; Grundy_bounded(n,k) = $n \% (k+1)$

Las dos variantes del Nim que NO salen con el XOR normal

C++/Notebook/Misere Nim y Grundy Acotado.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 // devuelve true si gana el primer jugador (pierde quien mueve al final)
6 bool misere_nim(vector<int> a)
7 {
8     bool all_ones = all_of(a.begin(), a.end(), [](int x){ return x == 1; });
9     int xorsum = 0;
10    for (int v : a) xorsum ^= v;
11
12    if (all_ones)
13    {
14        return (a.size() % 2 == 0);
15    }
16    return (xorsum != 0);
17 }
18
19 // se pueden quitar entre 1 y k piedras: Grundy(n) =  $n \% (k+1)$ 
20 int Grundy_bounded(int n, int k)
21 {
22     return n % (k + 1);
23 }
24
25 int main()
26 {
27     ios_base::sync_with_stdio(false);
28     cin.tie(nullptr);
29
30     int n, k;
31     cin >> n >> k;
32
33     vector<int> pilas(n);
34     for (int i = 0; i < n; i++) cin >> pilas[i];
35
36     cout << (misere_nim(pilas) ? "First" : "Second") << "\n";
37
38     // en la version acotada, la suma de subjuegos sigue siendo el XOR de los Grundy
39     int total = 0;
40     for (int x : pilas) total ^= Grundy_bounded(x, k);
41     cout << (total ? "First" : "Second") << "\n";
42
43     return 0;
44 }
```

8.1 Misere Nim y Grundy Acotado (cont.)

```

44 }

```

8.2 Nim con Segment Tree - Juan Vs Frank

O: (log n) por operaci n: Nim con segment tree de XOR
 Frank y Juan juegan duelos de Nim sobre un rango de columnas [l,r]

C++/Training_Camp_2026/Day 9/I - Juan vs Frank.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int N, Q;
11    cin >> N >> Q;
12
13    vector<ll> a(N);
14
15    int SZ = N;
16    vector<ll> seg(2 * SZ);
17
18    for (int i = 0; i < N; i++) {
19        cin >> a[i];
20        seg[SZ + i] = a[i];
21    }
22
23    for (int i = SZ - 1; i > 0; i--)
24        seg[i] = seg[i << 1] ^ seg[i << 1 | 1];
25
26    auto update = [&](int pos, ll val) {
27        pos--;
28        a[pos] += val;
29        int p = pos + SZ;
30        seg[p] = a[pos];
31        while (p >= 1)
32            seg[p] = seg[p << 1] ^ seg[p << 1 | 1];
33    };
34
35    auto query = [&](int l, int r) {
36        ll ans = 0;
37        l = l - 1 + SZ;
38        r = r + SZ;
39        while (l < r) {
40            if (l & 1) ans ^= seg[l++];
41            if (r & 1) ans ^= seg[--r];
42            l >>= 1;
43            r >>= 1;
44        }
45        return ans;
46    };
47
48    while (Q--) {
49        char op;
50        cin >> op;
51
52        if (op == 'P') {
53            int l, r;
54            cin >> l >> r;
55            if (query(l, r) == 0)
56                cout << "JUAN\n";
57            else
58                cout << "FRANK\n";
59        } else {
60            int k;
61            ll x;
62            cin >> k >> x;
63            update(k, x);
64        }
65    }

```

8.2 Nim con Segment Tree - Juan Vs Frank (cont.)

```

65 }
66
67 return 0;
68 }

```

8.3 Posiciones Perdedoras (potencias de 2) - Digitsduel

O: (log n), posiciones perdedoras en potencias de 2
 Determina el ganador de un juego analizando si N+2 es potencia de 2

C++/Codeforces/digitsduel.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool isPowerOfTwo(long long x) {
5     return x > 0 && (x & (x - 1)) == 0;
6 }
7
8 int main() {
9     ios::sync_with_stdio(false);
10    cin.tie(nullptr);
11
12    int T;
13    cin >> T;
14
15    while (T--) {
16        long long N;
17        cin >> N;
18
19        if (isPowerOfTwo(N + 2)) {
20            cout << "the greatest\n";
21        } else {
22            cout << "mastermei\n";
23        }
24    }
25
26    return 0;
27 }

```

8.4 Sprague-Grundy - Marbles

O: (n), XOR de los valores de Grundy
 Juego de canicas hacia el origen: Sprague-Grundy sobre las dos coordenadas

C++/Training_Camp_2026/Day 8/B - Marbles.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 16];
5 static int bufLen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0;
8         if (!bufLen_) return -1; }
9     return buf_[bufPos_++];
10 }
11 inline int readInt() {
12     int c = gc();
13     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
14     bool neg = false; if (c == '-') { neg = true; c = gc(); }
15     int x = 0;
16     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
17     return neg ? -x : x;
18 }
19
20 const int MAXV = 101; // coordinates go from 0 to 100
21 const int INSTA = 1000000; // sentinel for "instant win" positions: (i,0), (0,i)
22 , (i,i)
23 int grundy_[MAXV][MAXV];
24
25 void precomputeGrundy() {
26     // process states in order of increasing max(l,c): every move strictly decreases
27     // max(l,c) or keeps l the same while decreasing c (or vice versa), so processing
28     // by increasing (l+c) is a safe topological order (all moves reduce l+c).
29 }

```

8.4 Sprague-Grundy - Marbles (cont.)

```

vector<int> seen(3 * MAXV + 10, -1); // seen[v] == stamp means v currently marked reac
27   habile
28   int stamp = 0;
29
30   vector<pair<int,int>> order;
31   for (int l = 0; l < MAXV; l++)
32     for (int c = 0; c < MAXV; c++)
33       order.push_back({l, c});
34   sort(order.begin(), order.end(), [](const pair<int,int>&a, const pair<int,int>&b){
35     return (a.first + a.second) < (b.first + b.second);
36   });
37
38   for (auto &pr : order) {
39     int l = pr.first, c = pr.second;
40     if (l == 0 || c == 0 || l == c) {
41       grundy_[l][c] = INSTA;
42       continue;
43     }
44     stamp++;
45     for (int u = 1; u <= max(l, c); u++) {
46       int g;
47       if (l - u >= 0) {
48         g = grundy_[l - u][c];
49         if (g < (int)seen.size()) seen[g] = stamp;
50       }
51       if (c - u >= 0) {
52         g = grundy_[l][c - u];
53         if (g < (int)seen.size()) seen[g] = stamp;
54       }
55       if (l - u >= 0 && c - u >= 0) {
56         g = grundy_[l - u][c - u];
57         if (g < (int)seen.size()) seen[g] = stamp;
58       }
59     }
60     int mex = 0;
61     while (mex < (int)seen.size() && seen[mex] == stamp) mex++;
62     grundy_[l][c] = mex;
63   }
64 }
65
66 int main() {
67   precomputeGrundy();
68
69   int N = readInt();
70   vector<pair<int,int>> marbles(N);
71   for (auto &m : marbles) { m.first = readInt(); m.second = readInt(); }
72
73   bool instaWin = false;
74   for (auto &m : marbles) {
75     if (m.first == m.second) { instaWin = true; break; }
76   }
77
78   if (instaWin) {
79     printf("Y\n");
80     return 0;
81   }
82
83   int x = 0;
84   for (auto &m : marbles) {
85     x ^= grundy_[m.first][m.second];
86   }
87   printf(x != 0 ? "Y\n" : "N\n");
88   return 0;
89 }

```

9 - String

9.1 Brute Force Matching - Enigma

O: (n*m), comparacion directa

Dado un mensaje cifrado y un crib (palabra que se asume esta en el mensaje original)

C++/Training_Camp_2026/Day 8/E - Enigma.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int main()
4  {
5     string s1,s2;
6     cin >> s1 >> s2;
7     int god=0;
8
9     for(int i = 0 ; i <= s1.size()-s2.size();i++)
10    {
11        string tempi = s1.substr(i,s2.size());
12        bool ok = false;
13        for(int j = 0 ; j < s2.size() && ok == false;j++)
14        {
15            if(tempi[j]==s2[j])
16            {
17                ok = true;
18            }
19        }
20        if(ok == false)
21        {
22            god++;
23        }
24    }
25
26    cout << god << endl;
27
28    return 0;
29 }

```

9.2 Chequeo de Palindromo

O: (1) por consulta tras O(n^2) de tabla; manacher es O(n)

Uso: tablaPalindromos(s)[l][r]; manacher(s) -> {d1,d2} impar/par

Las tres formas de preguntar "esto es palindromo", segun cuantas veces lo vayas a preguntar

C++/Notebook/Chequeo de Palindromo.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // 1) Consulta suelta: O(n) tiempo, O(1) memoria.
5  // Se pasa por referencia para no copiar el string en cada llamada.
6  bool esPalindromo(string &s, int l, int r) {
7     while (l < r) {
8         if (s[l] != s[r]) return false;
9         l++;
10        r--;
11    }
12    return true;
13 }
14
15 // El string completo, con la STL:
16 // equal(s.begin(), s.begin() + s.size() / 2, s.rbegin())
17
18 // 2) Muchas consultas de rango: tabla O(n^2), cada consulta O(1).
19 // pal[i][j] depende del interior [i+1][j-1], asi que i va de DERECHA a
20 // IZQUIERDA para que ese interior ya este resuelto.
21 vector<vector<bool>> tablaPalindromos(const string &s) {
22     int n = s.size();
23     vector<vector<bool>> pal(n, vector<bool>(n, false));
24     for (int i = n - 1; i >= 0; i--)
25         for (int j = i; j < n; j++)
26             if (s[i] == s[j] && (j - i < 2 || pal[i + 1][j - 1]))
27                 pal[i][j] = true;
28     return pal;
29 }
30

```

9.2 Chequeo de Palindromo (cont.)

```

31 // 3) Palindromo mas largo con n grande: Manacher, O(n).
32 // d1[i] = cuantos palindromos IMPARES centrados en i
33 // d2[i] = cuantos palindromos PARES centrados entre i-1 e i
34 // El impar mas largo centrado en i mide 2*d1[i]-1, el par mide 2*d2[i].
35 pair<vector<int>, vector<int>> manacher(const string &s) {
36     int n = s.size();
37     vector<int> d1(n), d2(n);
38     for (int i = 0, l = 0, r = -1; i < n; i++) {
39         int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
40         while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;
41         d1[i] = k--;
42         if (i + k > r) { l = i - k; r = i + k; }
43     }
44     for (int i = 0, l = 0, r = -1; i < n; i++) {
45         int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
46         while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) k++;
47         d2[i] = k--;
48         if (i + k > r) { l = i - k - 1; r = i + k; }
49     }
50     return {d1, d2};
51 }
52
53 int main() {
54     ios::sync_with_stdio(false);
55     cin.tie(nullptr);
56
57     string s;
58     while (cin >> s) {
59         int n = s.size();
60         cout << (esPalindromo(s, 0, n - 1) ? "SI" : "NO") << "\n";
61     }
62     return 0;
63 }
64
65 // OJO con los casos borde del chequeo de dos punteros: con l == r (un solo
66 // caracter) el while no entra y devuelve true, que es lo correcto; con un
67 // rango vacio (l > r) tambien devuelve true. Si en tu problema el palindromo
68 // vacio NO cuenta, filtra el largo ANTES de llamar a la funcion.

```

9.3 Hashing de Cadenas (mod 2^61-1)

O: (n) construir, (1) cada get, (log n) cada lcp

Uso: Hash H(s); H.get(l,r) es [l,r)  -indexado; iguales si los get coinciden

Comparar dos subcadenas en O(1) tras O(n) de preproceso, y LCP por binaria

C++/Notebook/Hashing de Cadenas.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 typedef unsigned long long ull;
6
7 const ull MOD = (1ULL << 61) - 1;
8
9 ull mulmod(ull a, ull b)
10 {
11     __uint128_t c = (__uint128_t)a * b;
12     ull r = (ull)(c & MOD) + (ull)(c >> 61);
13     return r >= MOD ? r - MOD : r;
14 }
15
16 mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
17 const ull BASE = rng() % (MOD - 1000) + 500; // la MISMA para todas las cadenas
18
19 struct Hash
20 {
21     vector<ull> h, p;
22
23     Hash(const string &s) : h(s.size() + 1, 0), p(s.size() + 1, 1)
24     {
25         for (int i = 0; i < (int)s.size(); i++)
26             {

```

9.3 Hashing de Cadenas (mod 2^61-1) (cont.)

```

27         h[i + 1] = (mulmod(h[i], BASE) + (unsigned char)s[i]) % MOD;
28         p[i + 1] = mulmod(p[i], BASE);
29     }
30 }
31
32 // hash de s[l, r),  -indexado, r exclusivo
33 ull get(int l, int r)
34 {
35     return (h[r] + MOD - mulmod(h[l], p[r - l])) % MOD;
36 }
37
38 // Largo del prefijo comun mas largo de A[i..] y B[j..]
39 int lcp(Hash &A, int i, Hash &B, int j)
40 {
41     int lo = 0, hi = min((int)A.h.size() - 1 - i, (int)B.h.size() - 1 - j);
42     while (lo < hi)
43     {
44         int m = (lo + hi + 1) / 2;
45         if (A.get(i, i + m) == B.get(j, j + m)) lo = m;
46         else hi = m - 1;
47     }
48     return lo;
49 }
50
51 // Compara s[i..i+len) contra t[j..j+len) en orden alfabetico: -1, 0 o 1
52 int comparar(Hash &A, const string &s, int i, Hash &B, const string &t, int j, int len)
53 {
54     int k = min(lcp(A, i, B, j), len);
55     if (k == len) return 0;
56     return s[i + k] < t[j + k] ? -1 : 1;
57 }
58
59 int main()
60 {
61     ios_base::sync_with_stdio(false);
62     cin.tie(nullptr);
63     return 0;
64 }

```

9.4 KMP para Contar Ocurrencias Solapadas - Judgmental Crowd

O: (|s| + |p|), KMP contando ocurrencias solapadas

La reaccion del publico se calcula contando cuantas veces aparecen tres palabras en la cadena y...

C++/ICPC/Regionals 2025/J - Judgmental Crowd.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int KMP(const string& s, const string& p) {
5
6     int n = s.size();
7     int m = p.size();
8
9     // Prefix function del patron
10    vector<int> pi(m);
11
12    for (int i = 1; i < m; i++) {
13
14        int j = pi[i - 1];
15
16        while (j > 0 && p[i] != p[j])
17            j = pi[j - 1];
18
19        if (p[i] == p[j])
20            j++;
21
22        pi[i] = j;
23    }
24
25    // Buscar p dentro de s
26    int ans = 0;

```

9.4 KMP para Contar Ocurrencias Solapadas - Judgmental Crowd (cont.)

```

27     int j = 0;
28
29     for (int i = 0; i < n; i++) {
30
31         while (j > 0 && s[i] != p[j])
32             j = pi[j - 1];
33
34         if (s[i] == p[j])
35             j++;
36
37         if (j == m) {
38             ans++;
39
40             // Importante para permitir solapamientos
41             j = pi[j - 1];
42         }
43     }
44
45     return ans;
46 }
47
48 int main() {
49     ios::sync_with_stdio(false);
50     cin.tie(nullptr);
51
52     string s;
53
54     cin >> s;
55
56     cout << KMP(s, "ha") - KMP(s, "booooo") + 3*KMP(s, "bravo") << '\n';
57
58     return 0;
59 }

```

9.5 KMP sobre Cadena Recursiva (Duplicacion) - Song Of The Sirens

O: $(|t| + |s| * \log k)$, KMP sobre la cadena que se duplica

Hay $n+1$ canciones definidas por $s_{i+1} = s_i + t[i] + s_i$

C++/Codeforces/songOfTheSirens.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  const ll MOD = 1000000007;
7  const ll INV2 = 500000004;
8
9  vector<int> build_pi(const string& p) {
10     int m = p.size();
11     vector<int> pi(m);
12
13     for (int i = 1; i < m; i++) {
14         int j = pi[i - 1];
15
16         while (j > 0 && p[i] != p[j])
17             j = pi[j - 1];
18
19         if (p[i] == p[j])
20             j++;
21
22         pi[i] = j;
23     }
24
25     return pi;
26 }
27
28 ll count_occ(const string& s, const string& p, const vector<int>& pi) {
29     int j = 0;
30     ll ans = 0;

```

9.5 KMP sobre Cadena Recursiva (Duplicacion) - Song Of The Sirens (cont.)

```

31
32     for (char c : s) {
33         while (j > 0 && c != p[j])
34             j = pi[j - 1];
35
36         if (c == p[j])
37             j++;
38
39         if (j == (int)p.size()) {
40             ans++;
41             j = pi[j - 1];
42         }
43     }
44
45     return ans;
46 }
47
48 int main() {
49     ios::sync_with_stdio(false);
50     cin.tie(nullptr);
51
52     int N, Q;
53     cin >> N >> Q;
54
55     string s0, t;
56     cin >> s0 >> t;
57
58     // pow2[i] = 2^i
59     // invpow2[i] = 2^(-i)
60     vector<ll> pow2(N + 1), invpow2(N + 1);
61
62     pow2[0] = 1;
63     invpow2[0] = 1;
64
65     for (int i = 1; i <= N; i++) {
66         pow2[i] = pow2[i - 1] * 2 % MOD;
67         invpow2[i] = invpow2[i - 1] * INV2 % MOD;
68     }
69
70     /*
71         pref[c][i] =
72         suma de 2^(-j) para j < i donde t[j] == c
73     */
74
75     vector<vector<ll>> pref(26, vector<ll>(N + 1));
76
77     for (int c = 0; c < 26; c++) {
78         for (int i = 0; i < N; i++) {
79             pref[c][i + 1] = pref[c][i];
80
81             if (t[i] - 'a' == c)
82                 pref[c][i + 1] =
83                     (pref[c][i + 1] + invpow2[i]) % MOD;
84         }
85     }
86
87     while (Q--) {
88         int n;
89         string w;
90
91         cin >> n >> w;
92
93         int m = w.size();
94
95         vector<int> pi = build_pi(w);
96
97         /*
98             Construimos s_k hasta que:
99
100             |s_k| >= |w|
101
102             o hasta llegar a s_n si n es menor.

```

9.5 KMP sobre Cadena Recursiva (Duplicacion) - Song Of The Sirens (cont.)

```

03  */
04
05  string cur = s0;
06
07  int k = 0;
08
09  while (k < n && (int)cur.size() < m) {
10      cur = cur + t[k] + cur;
11      k++;
12  }
13
14  /*
15   Si todavía |cur| < |w|, entonces estamos en s_n
16   y simplemente hacemos KMP directamente.
17  */
18
19  if ((int)cur.size() < m) {
20      cout << 0 << '\n';
21      continue;
22  }
23
24  /*
25   cur = s_k
26  */
27
28  ll base = count_occ(cur, w, pi);
29
30  /*
31   Si k == n, ya tenemos exactamente s_n.
32  */
33
34  if (k == n) {
35      cout << base % MOD << '\n';
36      continue;
37  }
38
39  /*
40   Para calcular g(c):
41
42   suffix(|w|-1) + c + prefix(|w|-1)
43
44   Todas las apariciones de w en este string
45   contienen obligatoriamente el caracter central c.
46  */
47
48  string prefix = cur.substr(0, m - 1);
49  string suffix = cur.substr(cur.size() - (m - 1), m - 1);
50
51  vector<ll> g(26);
52
53  for (int c = 0; c < 26; c++) {
54      string x;
55      x.reserve(2 * m - 1);
56
57      x += suffix;
58      x += char('a' + c);
59      x += prefix;
60
61      g[c] = count_occ(x, w, pi);
62  }
63
64  /*
65   f(n,w) =
66   f(k,w) * 2^(n-k)
67   +
68   sum_{i=k+1}^n g(i,w) * 2^(n-i)
69
70   Como g(i,w) depende unicamente de t[i-1]:
71
72   g(i,w) = g[t[i-1]]
73  */
74

```

9.5 KMP sobre Cadena Recursiva (Duplicacion) - Song Of The Sirens (cont.)

```

75  ll ans = base * pow2[n - k] % MOD;
76
77  /*
78   Queremos:
79
80   sum_{j=k}^{n-1}
81   g[t[j]] * 2^(n-j-1)
82
83   = 2^(n-1) *
84   sum_{j=k}^{n-1}
85   g[t[j]] * 2^(-j)
86  */
87
88  ll sum = 0;
89
90  for (int c = 0; c < 26; c++) {
91      ll x = (pref[c][n] - pref[c][k] + MOD) % MOD;
92      sum = (sum + g[c] * x) % MOD;
93  }
94
95  ans = (ans + sum * pow2[n - 1]) % MOD;
96
97  cout << ans << '\n';
98  }
99
100 return 0;
101 }

```

9.6 KMP y Funcion Z

O: (n + m)

Uso: buscar(texto, patron) -> inicios, con solapamiento; prefijo(s); funcionZ(s)

Todas las apariciones de un patron en O(n + m), bordes, periodo mínimo y funcion Z

C++/Notebook/KMP y Funcion Z.cpp

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  vector<int> prefijo(const string &s)
6  {
7      int n = s.size();
8      vector<int> pi(n, 0);
9      for (int i = 1; i < n; i++)
10     {
11         int k = pi[i - 1];
12         while (k > 0 && s[i] != s[k]) k = pi[k - 1];
13         if (s[i] == s[k]) k++;
14         pi[i] = k;
15     }
16     return pi;
17 }
18
19 vector<int> buscar(const string &t, const string &p)
20 {
21     vector<int> res;
22     if (p.empty()) return res;
23     vector<int> pi = prefijo(p);
24     int k = 0;
25     for (int i = 0; i < (int)t.size(); i++)
26     {
27         while (k > 0 && t[i] != p[k]) k = pi[k - 1];
28         if (t[i] == p[k]) k++;
29         if (k == (int)p.size())
30         {
31             res.push_back(i - k + 1);
32             k = pi[k - 1];
33         }
34     }
35     return res;
36 }
37

```

9.6 KMP y Funcion Z (cont.)

```

38 vector<int> funcionZ(const string &s)
39 {
40     int n = s.size();
41     vector<int> z(n, 0);
42     for (int i = 1, l = 0, r = 0; i < n; i++)
43     {
44         if (i < r) z[i] = min(r - i, z[i - l]);
45         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
46         if (i + z[i] > r)
47         {
48             l = i;
49             r = i + z[i];
50         }
51     }
52     return z;
53 }
54
55 int main()
56 {
57     ios_base::sync_with_stdio(false);
58     cin.tie(nullptr);
59     return 0;
60 }

```

9.7 Maximo Solape Sufijo-Prefijo (KMP) - Sign

O: (n + m), el solape sale de la funcion de prefijos de KMP

Un letrero muestra k caracteres; en cada paso todo se corre una posición a la izquierda y entra una letra...

C++/CCPL/sign.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Mayor L tal que los ultimos L caracteres de a son los primeros L de b.
5 int solape(const string &a, const string &b) {
6     string s = b + "#" + a;
7     int m = s.size();
8
9     vector<int> f(m, 0);
10
11     for (int i = 1; i < m; i++) {
12         int j = f[i - 1];
13
14         while (j > 0 && s[i] != s[j])
15             j = f[j - 1];
16
17         if (s[i] == s[j])
18             j++;
19
20         f[i] = j;
21     }
22
23     return f[m - 1];
24 }
25
26 int main() {
27     ios::sync_with_stdio(false);
28     cin.tie(nullptr);
29
30     int casos;
31     cin >> casos;
32
33     while (casos--) {
34         int k, w;
35         cin >> k >> w;
36
37         vector<string> pal(w);
38         for (int i = 0; i < w; i++) cin >> pal[i];
39
40         // El letrero arranca vacio, asi que la primera palabra cuesta k.
41         long long total = k;
42     }

```

9.7 Maximo Solape Sufijo-Prefijo (KMP) - Sign (cont.)

```

43     for (int i = 1; i < w; i++) {
44         total += k - solape(pal[i - 1], pal[i]);
45     }
46
47     cout << total << '\n';
48 }
49     return 0;
50 }

```

9.8 Monotonic Stack - Remove Duplicates1

O: (n), pila monotona

Devuelve el menor string lexicografico que contiene cada letra de s exactamente una vez

C++/LeetCode/removeDuplicates1.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:
7     string removeDuplicateLetters(string s) {
8         // cnt for storing frequency of characters
9         // vis for marking visited characters
10        vector<int> cnt(26, 0), vis(26, 0);
11
12        string res = "";
13        int n = s.size();
14
15        for (int i = 0; i < n; ++i)
16            cnt[s[i] - 'a']++;
17
18        for (int i = 0; i < n; ++i)
19        {
20            // decrease cnt of current character
21            cnt[s[i] - 'a']--;
22
23            // If character is not already
24            // in answer
25            if (!vis[s[i] - 'a'])
26            {
27                // Last character > s[i]
28                // and its count > 0
29                while (res.size() > 0 && res.back() > s[i] && cnt[res.back() - 'a'] > 0)
30                {
31                    // marking letter visited
32                    vis[res.back() - 'a'] = 0;
33                    res.pop_back();
34                }
35
36                // Add s[i] in res and
37                // mark it visited
38                res += s[i];
39                vis[s[i] - 'a'] = 1;
40            }
41        }
42        // return resultant string
43        return res;
44    }
45 };

```

9.9 Palindromo Ignorando Simbolos - Just Palindromes

O: (n), dos punteros saltando los simbolos

Decir si una frase es palindroma ignorando espacios, puntuacion y mayusculas

C++/CPC/2025/J - Just Palindromes.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);

```

9.9 Palindromo Ignorando Simbolos - Just Palindromes (cont.)

```

1
2 string s;
3
4 while (getline(cin, s) && s != "") {
5
6     string t;
7
8     // Dejamos unicamente letras y pasamos a minusculas
9     for (char c : s) {
10         if (isalpha(c))
11             t += tolower(c);
12     }
13
14     bool ok = true;
15
16     int l = 0;
17     int r = t.size() - 1;
18
19     while (l < r) {
20         if (t[l] != t[r]) {
21             ok = false;
22             break;
23         }
24         l++;
25         r--;
26     }
27
28     cout << (ok ? 'Y' : 'N') << '\n';
29 }
30
31 return 0;
32 }

```

9.10 Palindromos No Solapados (Greedy) - Max Palindromes

O: $(n \cdot k)$, probando solo los largos k y $k+1$

Dado s y un k , hay que escoger la mayor cantidad posible de subcadenas palindromas que no se solapen y que...

C++/LeetCode/maxPalindromes.cpp

```

1 class Solution {
2 public:
3     bool esPalindromo(string &s, int l, int r) {
4         while (l < r) {
5             if (s[l] != s[r]) return false;
6             l++;
7             r--;
8         }
9         return true;
10    }
11
12    int maxPalindromes(string s, int k) {
13        int n = s.size();
14        int ans = 0;
15        int inicio = 0;
16
17        for (int r = k - 1; r < n; r++) {
18            for (int l = inicio; l <= r - k + 1; l++) {
19                if (esPalindromo(s, l, r)) {
20                    ans++;
21                    inicio = r + 1;
22                    break;
23                }
24            }
25        }
26
27        return ans;
28    }
29 };

```

9.11 Parentesis Balanceados (Todas las Variantes)

O: (n) todas las variantes

Uso: `valido(s)`; `validoTipos(s)`; `parejas(s)[i]` = pareja de i , o -1

El BALANCE, $+1$ por cada '(' y -1 por cada ')'

C++/Notebook/Parentesis Balanceados.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 // 1) Un solo tipo.
6 bool valido(const string &s) {
7     int bal = 0;
8     for (char c : s) {
9         if (c == '(') bal++;
10        else bal--;
11        if (bal < 0) return false;
12    }
13    return bal == 0;
14 }
15
16 // 2) Varios tipos: la pila guarda los que abren.
17 bool validoTipos(const string &s) {
18     string pila;
19     for (char c : s) {
20         if (c == '(' || c == '[' || c == '{') {
21             pila.push_back(c);
22             continue;
23         }
24         char abre = '(';
25         if (c == ']') abre = '[';
26         if (c == '}') abre = '{';
27         if (pila.empty() || pila.back() != abre) return false;
28         pila.pop_back();
29     }
30     return pila.empty();
31 }
32
33 // 3) Pareja de cada parentesis (-1 si queda suelto) y anidamiento maximo.
34 vector<int> parejas(const string &s) {
35     vector<int> match(s.size(), -1), pila;
36     for (int i = 0; i < (int)s.size(); i++) {
37         if (s[i] == '(') {
38             pila.push_back(i);
39         } else if (!pila.empty()) {
40             match[i] = pila.back();
41             match[pila.back()] = i;
42             pila.pop_back();
43         }
44     }
45     return match;
46 }
47
48 int profundidad(const string &s) { // para cadenas validas
49     int bal = 0, mx = 0;
50     for (char c : s) {
51         if (c == '(') bal++;
52         else bal--;
53         mx = max(mx, bal);
54     }
55     return mx;
56 }
57
58 // 4) Minimo de inserciones para balancear.
59 int minInserciones(const string &s) {
60     int abiertos = 0, faltan = 0;
61     for (char c : s) {
62         if (c == '(') abiertos++;
63         else if (abiertos > 0) abiertos--;
64         else faltan++; // ')' sin pareja: pide un '('
65     }
66     return abiertos + faltan; // cada '(' abierto pide un ')'

```

9.11 Paréntesis Balanceados (Todas las Variantes) (cont.)

```

87 }
88
89 // 5) Mínimo de volteos '(' <-> ')' para balancear; -1 si es imposible.
90 int minVolteos(const string &s) {
91     if (s.size() % 2) return -1;
92     int abiertos = 0, cerrados = 0;
93     for (char c : s) {
94         if (c == '(') abiertos++;
95         else if (abiertos > 0) abiertos--;
96         else cerrados++;
97     }
98     return (cerrados + 1) / 2 + (abiertos + 1) / 2;
99 }
100
101 // 6) Borrar el mínimo para que quede válida (devuelve una de las óptimas).
102 string borrarMinimo(const string &s) {
103     vector<int> pila;
104     vector<bool> borrar(s.size(), false);
105     for (int i = 0; i < (int)s.size(); i++) {
106         if (s[i] == '(') {
107             pila.push_back(i);
108         } else if (s[i] == ')') {
109             if (!pila.empty()) pila.pop_back();
110             else borrar[i] = true;
111         }
112     }
113     for (int i : pila) borrar[i] = true;
114     string res;
115     for (int i = 0; i < (int)s.size(); i++)
116         if (!borrar[i]) res += s[i];
117     return res;
118 }
119
120 // 7) Largo de la subcadena válida más larga.
121 int masLarga(const string &s) {
122     vector<int> pila = {-1}; // índice antes del tramo válido
123     int mejor = 0;
124     for (int i = 0; i < (int)s.size(); i++) {
125         if (s[i] == '(') {
126             pila.push_back(i);
127             continue;
128         }
129         pila.pop_back();
130         if (pila.empty()) pila.push_back(i); // ')' suelto: nuevo tope
131         else mejor = max(mejor, i - pila.back());
132     }
133     return mejor;
134 }
135
136 // 8) Cantidad de subcadenas válidas (distintas por posición).
137 ll contarSubcadenas(const string &s) {
138     int n = s.size();
139     vector<ll> dp(n + 1, 0); // dp[i+1] = válidas que terminan en i
140     vector<int> pila;
141     ll total = 0;
142     for (int i = 0; i < n; i++) {
143         if (s[i] == '(') {
144             pila.push_back(i);
145             continue;
146         }
147         if (pila.empty()) continue;
148         int j = pila.back();
149         pila.pop_back();
150         dp[i + 1] = dp[j] + 1;
151         total += dp[i + 1];
152     }
153     return total;
154 }
155
156 // 9) Con comodines '*' que valen '(' o ')' o nada.
157 bool validoComodines(const string &s) {
158     int lo = 0, hi = 0; // balance mínimo y máximo posibles

```

9.11 Paréntesis Balanceados (Todas las Variantes) (cont.)

```

159     for (char c : s) {
160         if (c == '(') {
161             lo++;
162             hi++;
163         } else if (c == ')') {
164             lo--;
165             hi--;
166         } else {
167             lo--;
168             hi++;
169         }
170         if (hi < 0) return false;
171         lo = max(lo, 0);
172     }
173     return lo == 0;
174 }
175
176 // 10) Todas las válidas de n pares, en orden lexicográfico.
177 void generar(int n, int abiertos, int cerrados, string &cur, vector<string> &res) {
178     if ((int)cur.size() == 2 * n) {
179         res.push_back(cur);
180         return;
181     }
182     if (abiertos < n) {
183         cur.push_back('(');
184         generar(n, abiertos + 1, cerrados, cur, res);
185         cur.pop_back();
186     }
187     if (cerrados < abiertos) {
188         cur.push_back(')');
189         generar(n, abiertos, cerrados + 1, cur, res);
190         cur.pop_back();
191     }
192 }
193
194 // 11) La k-ésima válida de n pares (k desde 1), en orden lexicográfico.
195 string kEsima(int n, ll k) {
196     vector<vector<ll>> formas(2 * n + 1, vector<ll>(2 * n + 2, 0));
197     formas[0][0] = 1;
198     for (int i = 1; i <= 2 * n; i++)
199         for (int b = 0; b <= 2 * n; b++) {
200             formas[i][b] = formas[i - 1][b + 1]; // poner '('
201             if (b > 0) formas[i][b] += formas[i - 1][b - 1]; // poner ')'
202         }
203     string res;
204     int b = 0;
205     for (int i = 2 * n; i > 0; i--) {
206         ll conAbre = formas[i - 1][b + 1];
207         if (k <= conAbre) {
208             res += '(';
209             b++;
210         } else {
211             k -= conAbre;
212             res += ')';
213             b--;
214         }
215     }
216     return res;
217 }
218
219 // 12) Subsecuencia balanceada más larga en s[l..r], con muchas consultas.
220 struct Nodo {
221     int par, abre, cierra;
222 };
223
224 Nodo unir(Nodo a, Nodo b) {
225     int nuevos = min(a.abre, b.cierra);
226     return {a.par + b.par + nuevos, a.abre + b.abre - nuevos,
227             a.cierra + b.cierra - nuevos};
228 }
229
230 struct SegTree {

```

9.11 Parentesis Balanceados (Todas las Variantes) (cont.)

```

211 int n;
212 vector<Nodo> t;
213
214 SegTree(const string &s) : n(s.size()), t(4 * s.size()) {
215     build(1, 0, n - 1, s);
216 }
217
218 void build(int p, int l, int r, const string &s) {
219     if (l == r) {
220         if (s[l] == '(') t[p] = {0, 1, 0};
221         else t[p] = {0, 0, 1};
222         return;
223     }
224     int m = (l + r) / 2;
225     build(2 * p, l, m, s);
226     build(2 * p + 1, m + 1, r, s);
227     t[p] = unir(t[2 * p], t[2 * p + 1]);
228 }
229
230 Nodo query(int p, int l, int r, int ql, int qr) {
231     if (ql <= l && r <= qr) return t[p];
232     int m = (l + r) / 2;
233     if (qr <= m) return query(2 * p, l, m, ql, qr);
234     if (ql > m) return query(2 * p + 1, m + 1, r, ql, qr);
235     return unir(query(2 * p, l, m, ql, qr), query(2 * p + 1, m + 1, r, ql, qr));
236 }
237
238 int maxBalanceada(int l, int r) { // indices desde 0
239     return 2 * query(1, 0, n - 1, l, r).par;
240 }
241 };
242
243 // 13) Varios tipos: subsecuencia balanceada mas larga (DP de intervalos).
244 bool casan(char a, char b) {
245     return (a == '(' && b == ')') || (a == '[' && b == ']') || (a == '{' && b == '}');
246 }
247
248 int maxSubsecuenciaTipos(const string &s) {
249     int n = s.size();
250     if (n == 0) return 0;
251     vector<vector<int>> dp(n, vector<int>(n, 0));
252     for (int len = 2; len <= n; len++)
253         for (int i = 0; i + len - 1 < n; i++) {
254             int j = i + len - 1, mejor = 0;
255             if (casan(s[i], s[j])) mejor = dp[i + 1][j - 1] + 2;
256             for (int k = i; k < j; k++) mejor = max(mejor, dp[i][k] + dp[k + 1][j]);
257             dp[i][j] = mejor;
258         }
259     return dp[0][n - 1];
260 }
261
262 int main() {
263     ios::sync_with_stdio(false);
264     cin.tie(nullptr);
265
266     string s;
267     while (cin >> s) {
268         cout << s << ": valida=" << valido(s)
269             << " inserciones=" << minInserciones(s)
270             << " masLarga=" << masLarga(s)
271             << " subcadenas=" << contarSubcadenas(s)
272             << " queda=" << borrarMinimo(s) << '\n';
273     }
274     return 0;
275 }

```

9.12 Partition and Permutation - Policianacional

O: (n log n) por el sort

Divide una cadena en 3 partes, las reorganiza y verifica si forma otra cadena objetivo

C++/Codeforces/policianacional.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9     cin >> n;
10
11     string s1, s2;
12     cin >> s1 >> s2;
13
14     // Probar todos los cortes posibles:
15     // A = [0,i)
16     // B = [i,j)
17     // C = [j,n)
18
19     for (int i = 1; i < n-1; i++)
20     {
21         for (int j = i+1; j < n; j++)
22         {
23             string A = s1.substr(0, i);
24             string B = s1.substr(i, j-i);
25             string C = s1.substr(j);
26
27             vector<string> p = {A,B,C};
28
29             sort(p.begin(), p.end());
30
31             do
32             {
33                 string formed = p[0] + p[1] + p[2];
34
35                 if (formed == s2)
36                 {
37                     cout << "YES\n";
38
39                     // Descomposición original del pasaporte
40                     cout << A << " " << B << " " << C << "\n";
41
42                     // Descomposición de la visa
43                     cout << p[0] << " " << p[1] << " " << p[2] << "\n";
44
45                     return 0;
46                 }
47             } while (next_permutation(p.begin(), p.end()));
48         }
49     }
50
51     cout << "NO\n";
52
53     return 0;
54 }

```

9.13 Remove Duplicate Letters - Remove Duplicates

O: (n), pila monotona con conteo de lo que falta

Busca construir el menor string lexicografico que contiene cada letra una sola vez

C++/LeetCode/removeDuplicates.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:

```

9.13 Remove Duplicate Letters - Remove Duplicates (cont.)

```

7   string removeDuplicateLetters(string s) {
8       vector<int> nums(s.size());
9       bool letras[26] = {false};
10      int cont[26] = {0};
11      for(char c : s)
12      {
13          nums.push_back(c - '0');
14          cont[c-'a']++;
15          letras[c - 'a'] = true;
16      }
17
18      int letmax = 0;
19      for (int i = 0; i < 26; i++)
20      {
21          letmax += letras[i];
22      }
23
24      char ini;
25      for (int i = 0; i < 26; i++)
26      {
27          if(cont[i] > 0)
28          {
29              ini = i + 'a';
30              break;
31          }
32      }
33
34
35      string god="";
36      int idx = 0;
37      bool contGod[26] = {false};
38      for(int i = 0 ; i < s.size() || god.size() < letmax; i++)
39      {
40          if(s[i] == ini && contGod[s[i]-'a'] == false)
41          {
42              idx = i + 1;
43              for (int i = ini - 'a' + 1; i < 26; i++)
44              {
45                  if(cont[i] > 0 && contGod[i] == false)
46                  {
47                      ini = i + 'a';
48                      break;
49                  }
50              }
51              god+=s[i];
52              contGod[s[i]-'a'] = true;
53          }
54          else if(cont[s[i]-'a']==1 && contGod[s[i]-'a'] == false)
55          {
56              char mini = s[i];
57              int idxMini = i;
58              while(idx < i)
59              {
60                  for(int j = idx; j < i; j++)
61                  {
62                      if(s[j] < mini && contGod[s[j]-'a'] == false)
63                      {
64                          mini = s[j];
65                          idxMini = j;
66                      }
67                  }
68                  if(mini != s[i])
69                  {
70                      god+= s[idxMini];
71                      contGod[s[idxMini]-'a'] = true;
72
73                      idx = idxMini + 1;
74                      mini = s[i];
75                      idxMini = i;
76                  }
77              }
78              else

```

9.13 Remove Duplicate Letters - Remove Duplicates (cont.)

```

79      {
80          break;
81      }
82
83      }
84
85      for(int j = idx; j <= i; j++)
86      {
87          if(s[j] == s[i])
88          {
89              god+=s[j];
90              idx = j + 1;
91              contGod[s[j]-'a']= true;
92              break;
93          }
94      }
95      }
96      cont[s[i]-'a']--;
97  }
98
99      return god;
100 }
101 };

```

9.14 Rotacion Minima (Dos Punteros) - Fingerprints

O(n), rotacion minima con dos punteros

Las huellas se guardan de forma circular, asi que ABCD, BCDA, CDAB y DABC son la misma

C++/ICPC/2025/F - Fingerprints.cpp

```

1   #include <bits/stdc++.h>
2   using namespace std;
3
4   string minimalRotation(string s) {
5       int n = s.size();
6
7       s += s;
8
9       int i = 0, j = 1;
10
11       while (i < n && j < n) {
12           int k = 0;
13
14           while (k < n && s[i + k] == s[j + k])
15               k++;
16
17           if (k == n)
18               break;
19
20           if (s[i + k] > s[j + k])
21               i = i + k + 1;
22           else
23               j = j + k + 1;
24
25           if (i == j)
26               j++;
27       }
28
29       int pos = min(i, j);
30
31       return s.substr(pos, n);
32  }
33
34  int main() {
35      ios::sync_with_stdio(false);
36      cin.tie(nullptr);
37
38      int n;
39
40      while (cin >> n && n != 0) {
41
42          // El set solo sirve para detectar repetidas. El ORDEN de salida es el

```

9.14 Rotacion Minima (Dos Punteros) - Fingerprints (cont.)

```

43 // de aparicion en la entrada, asi que se guarda aparte en un vector:
44 // imprimir el set directamente las sacaria en orden alfabetico.
45 set<string> vistas;
46 vector<string> ans;
47
48 for (int i = 0; i < n; i++) {
49     int m;
50     string s;
51
52     cin >> m >> s;
53
54     string canon = minimalRotation(s);
55
56     if (vistas.insert(canon).second)
57         ans.push_back(canon);
58 }
59
60 for (const string &s : ans)
61     cout << s << '\n';
62 }
63
64 return 0;
65 }

```

9.15 Simulation - Process Str

O: (n^2) , el duplicar y el invertir copian todo

Procesa una cadena aplicando operaciones sobre un string acumulador

C++/LeetCode/processStr.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:
7     string processStr(string s) {
8         string god = "";
9         for(char a : s)
10             {
11                 if(a == '*')
12                 {
13                     if(god.size()>0)
14                     {
15                         god.erase(god.size()-1);
16                     }
17                 }
18                 else if(a == '#')
19                 {
20                     god += god;
21                 }
22                 else if(a == '%')
23                 {
24                     reverse(god.begin(),god.end());
25                 }
26                 else
27                 {
28                     god += a;
29                 }
30             }
31         return god;
32     }
33 };

```

9.16 Suffix Array + LCP - Couple Of Bip Bop

O: $(N \log N)$ por el suffix array; con LCP el barrido es (N)

Bob y Charlie empiezan a bailar cada uno en un verso aleatorio de una cancion de N versos y siguen la...

C++/Training_Camp_2026/Day 3/C - Couple of BipBop.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3

```

9.16 Suffix Array + LCP - Couple Of Bip Bop (cont.)

```

4 using ll = long long;
5
6 ll gcdll(ll a, ll b) {
7     while (b) {
8         a %= b;
9         swap(a, b);
10    }
11    return a;
12 }
13
14 vector<int> suffix_array(vector<int> s) {
15     int n = s.size();
16     vector<int> sa(n), rnk(n), tmp(n);
17
18     for (int i = 0; i < n; i++) {
19         sa[i] = i;
20         rnk[i] = s[i];
21     }
22
23     for (int k = 1;; k <= 1) {
24         auto cmp = [&](int a, int b) {
25             if (rnk[a] != rnk[b])
26                 return rnk[a] < rnk[b];
27
28             int ra = a + k < n ? rnk[a + k] : -1;
29             int rb = b + k < n ? rnk[b + k] : -1;
30             return ra < rb;
31         };
32
33         sort(sa.begin(), sa.end(), cmp);
34
35         tmp[sa[0]] = 0;
36
37         for (int i = 1; i < n; i++)
38             tmp[sa[i]] = tmp[sa[i-1]] + cmp(sa[i-1], sa[i]);
39
40         rnk = tmp;
41
42         if (rnk[sa[n-1]] == n - 1)
43             break;
44     }
45
46     return sa;
47 }
48
49 vector<int> kasai(vector<int> &s, vector<int> &sa) {
50     int n = s.size();
51
52     vector<int> rank(n);
53     for (int i = 0; i < n; i++)
54         rank[sa[i]] = i;
55
56     vector<int> lcp(n - 1);
57
58     int h = 0;
59
60     for (int i = 0; i < n; i++) {
61         int r = rank[i];
62
63         if (r == n - 1)
64             continue;
65
66         int j = sa[r + 1];
67
68         while (i + h < n && j + h < n && s[i+h] == s[j+h])
69             h++;
70
71         lcp[r] = h;
72
73         if (h)
74             h--;
75     }

```

9.16 Suffix Array + LCP - Couple Of Bip Bop (cont.)

```

76
77     return lcp;
78 }
79
80 int main() {
81     ios::sync_with_stdio(false);
82     cin.tie(nullptr);
83
84     int N;
85     cin >> N;
86
87     vector<int> a(N);
88
89     for (int i = 0; i < N; i++)
90         cin >> a[i];
91
92     auto sa = suffix_array(a);
93     auto lcp = kasai(a, sa);
94
95     // suma de LCP entre pares distintos
96     ll sumPairs = 0;
97
98     vector<pair<ll, ll>> st;
99     ll accumulated = 0;
100
101     for (int i = 0; i < (int)lcp.size(); i++) {
102         ll cnt = 1;
103         ll val = lcp[i];
104
105         while (!st.empty() && st.back().first >= val) {
106             accumulated -= st.back().first * st.back().second;
107             cnt += st.back().second;
108             st.pop_back();
109         }
110
111         st.push_back({val, cnt});
112
113         accumulated += val * cnt;
114
115         sumPairs += accumulated;
116     }
117
118     ll numerator = 1LL * N * (N + 1) / 2 + 2 * sumPairs;
119     ll denominator = 1LL * N * N;
120
121     ll g = gcdll(numerator, denominator);
122
123     cout << numerator / g << '/' << denominator / g << '\n';
124
125     return 0;
126 }

```

9.17 Suffix Automaton - Beautiful Words

O: (n) de construcción, (largo) por palabra consultada
 Dado un string A de N caracteres y un conjunto S de M strings

C++/Training_Camp_2026/Day 4/A - Beautiful Words.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Suffix automaton over an alphabet of 26 lowercase letters + 1 separator symbol.
5 struct SAM {
6     static const int ALPHA = 27;
7     vector<array<int, ALPHA>> trans;
8     vector<int> len, link_;
9     int last;
10

```

9.17 Suffix Automaton - Beautiful Words (cont.)

```

11     void init(int maxStates) {
12         trans.clear(); len.clear(); link_.clear();
13         trans.reserve(maxStates);
14         len.reserve(maxStates);
15         link_.reserve(maxStates);
16         newNode();
17         len[0] = 0; link_[0] = -1;
18         last = 0;
19     }
20     int newNode() {
21         array<int, ALPHA> a; a.fill(-1);
22         trans.push_back(a);
23         len.push_back(0);
24         link_.push_back(-1);
25         return (int)trans.size() - 1;
26     }
27     void extend(int c) {
28         int cur = newNode();
29         len[cur] = len[last] + 1;
30         int p = last;
31         while (p != -1 && trans[p][c] == -1) {
32             trans[p][c] = cur;
33             p = link_[p];
34         }
35         if (p == -1) {
36             link_[cur] = 0;
37         } else {
38             int q = trans[p][c];
39             if (len[p] + 1 == len[q]) {
40                 link_[cur] = q;
41             } else {
42                 int clone = newNode();
43                 len[clone] = len[p] + 1;
44                 link_[clone] = link_[q];
45                 trans[clone] = trans[q];
46                 while (p != -1 && trans[p][c] == q) {
47                     trans[p][c] = clone;
48                     p = link_[p];
49                 }
50                 link_[q] = clone;
51                 link_[cur] = clone;
52             }
53         }
54         last = cur;
55     }
56 };
57
58 inline int charIdx(char c) {
59     if (c >= 'a' && c <= 'z') return c - 'a';
60     return 26; // separator
61 }
62
63 int main() {
64     ios_base::sync_with_stdio(false);
65     cin.tie(nullptr);
66
67     int N, M;
68     cin >> N >> M;
69     string A;
70     cin >> A;
71     vector<string> S(M);
72     long long totalLen = 0;
73     for (auto& s : S) { cin >> s; totalLen += (long long)s.size(); }
74
75     int maxStates = 2 * (int)(totalLen + M + 5) + 10;
76     SAM sam;
77     sam.init(maxStates);
78
79     for (int i = 0; i < M; i++) {
80         string rev = S[i];
81         reverse(rev.begin(), rev.end());
82         for (char c : rev) sam.extend(charIdx(c));

```

9.17 Suffix Automaton - Beautiful Words (cont.)

```

83     if (i + 1 < M) sam.extend(26); // separator between strings
84 }
85
86 int n2 = 2 * N;
87 string doubledA = A + A;
88 string revDoubledA(doubledA.rbegin(), doubledA.rend());
89
90 vector<int> G(n2);
91 {
92     int v = 0, l = 0;
93     for (int i = 0; i < n2; i++) {
94         int c = charIdx(revDoubledA[(size_t)i]);
95         while (v != 0 && sam.trans[v][c] == -1) {
96             v = sam.link[v];
97             l = sam.len[v];
98         }
99         if (sam.trans[v][c] != -1) {
100             v = sam.trans[v][c];
101             l++;
102         } else {
103             l = 0;
104         }
105         G[i] = l;
106     }
107 }
108
109 vector<int> C(n2);
110 for (int k = 0; k < n2; k++) C[k] = G[n2 - 1 - k];
111
112 auto feasible = [&](int ans) -> bool {
113     int W = N - ans;
114     if (W <= 0) return true;
115     int runStart = -1;
116     for (int k = 0; k <= n2; k++) {
117         bool bad = (k == n2) || (C[k] > ans);
118         if (!bad) {
119             if (runStart == -1) runStart = k;
120         } else {
121             if (runStart != -1) {
122                 int runEnd = k - 1;
123                 int runLen = runEnd - runStart + 1;
124                 if (runLen >= W && runStart <= N - 1) return true;
125             }
126             runStart = -1;
127         }
128     }
129     return false;
130 };
131
132 int lo = 0, hi = N;
133 while (lo < hi) {
134     int mid = (lo + hi) / 2;
135     if (feasible(mid)) hi = mid; else lo = mid + 1;
136 }
137 cout << lo << "\n";
138 return 0;
139 }

```

9.18 Trie + DP de Segmentacion (Maximo) - Spacebar Tokenizer

O: (n * largo maximo) tras armar el trie del diccionario
Partir cada cadena en tokens para que la suma de puntajes sea MAXIMA

C++/ICPC/2024/E - Spacebar Tokenizer.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Node {
5     int next[26];
6     int score;
7
8     Node() {

```

9.18 Trie + DP de Segmentacion (Maximo) - Spacebar Tokenizer (cont.)

```

9         fill(next, next + 26, -1);
10        score = -1;
11    }
12 };
13
14 int main() {
15     ios::sync_with_stdio(false);
16     cin.tie(nullptr);
17
18     int m, s;
19
20     while (cin >> m >> s && (m || s)) {
21
22         vector<Node> trie(1);
23
24         // Insertar palabras
25         for (int i = 0; i < m; i++) {
26
27             string word;
28             int value;
29
30             cin >> word >> value;
31
32             int node = 0;
33
34             for (char c : word) {
35
36                 int x = c - 'a';
37
38                 if (trie[node].next[x] == -1) {
39                     trie[node].next[x] = trie.size();
40                     trie.emplace_back();
41                 }
42
43                 node = trie[node].next[x];
44             }
45
46             trie[node].score = value;
47         }
48
49         while (s--) {
50
51             string a;
52             cin >> a;
53
54             int n = a.size();
55
56             vector<long long> dp(n + 1, 0);
57
58             for (int i = 0; i < n; i++) {
59
60                 // a[i] como parte de un token NO reconocido: suma 0
61                 dp[i + 1] = max(dp[i + 1], dp[i]);
62
63                 int node = 0;
64
65                 for (int j = i; j < n; j++) {
66
67                     int x = a[j] - 'a';
68
69                     if (trie[node].next[x] == -1)
70                         break;
71
72                     node = trie[node].next[x];
73
74                     if (trie[node].score != -1) {
75
76                         dp[j + 1] = max(
77                             dp[j + 1],
78                             dp[i] + trie[node].score
79                         );
80                     }

```

9.18 Trie + DP de Segmentación (Maximo) - Spacebar Tokenizer (cont.)

```
81         }
82     }
83
84     cout << dp[n] << '\n';
85 }
86 }
87
88 return 0;
89 }
```

9.19 Z-Function - A String Problem

O: (n + m), función Z

Cuenta cuantos t != "a" permiten partir s en trozos que sean t o "a", usando al menos un t

C++/Codeforces/aStringProblem.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> z_function(string s) {
5     int n = s.size();
6     vector<int> z(n);
7
8     for (int i = 1, l = 0, r = 0; i < n; i++) {
9         if (i <= r)
10             z[i] = min(r - i + 1, z[i - l]);
11
12         while (i + z[i] < n && s[z[i]] == s[i + z[i]])
13             z[i]++;
14
15         if (i + z[i] - 1 > r) {
16             l = i;
17             r = i + z[i] - 1;
18         }
19     }
20
21     return z;
22 }
23
24 int main() {
25     ios::sync_with_stdio(false);
26     cin.tie(nullptr);
27
28     int T;
29     cin >> T;
30
31     while (T--) {
32         string s;
33         cin >> s;
34
35         int n = s.size();
36         int p = 0;
37
38         while (p < n && s[p] == 'a')
39             p++;
40
41         if (p == n) {
42             cout << n - 1 << '\n';
43             continue;
44         }
45
46         string x = s.substr(p);
47         int m = x.size();
48         vector<int> z = z_function(x);
49
50         vector<int> nxt(m + 1, m);
51
52         for (int i = m - 1; i >= 0; i--) {
53             if (x[i] != 'a')
54                 nxt[i] = i;
55             else
56                 ;
57         }
58     }
59 }
```

9.19 Z-Function - A String Problem (cont.)

```
51     nxt[i] = nxt[i + 1];
52 }
53
54 long long ans = 0;
55
56 for (int len = 1; len <= m; len++) {
57     int pos = 0;
58     int mn = p;
59     bool ok = true;
60
61     while (true) {
62         if (pos + len > m) {
63             ok = false;
64             break;
65         }
66
67         if (pos != 0 && z[pos] < len) {
68             ok = false;
69             break;
70         }
71
72         pos += len;
73
74         int next = nxt[pos];
75
76         if (next == m)
77             break;
78
79         mn = min(mn, next - pos);
80         pos = next;
81     }
82
83     if (ok)
84         ans += mn + 1;
85 }
86
87 cout << ans << '\n';
88 }
89
90 return 0;
91 }
```

10 - Greedy

10.1 Binary Search - Maratona Brasileira De Popcorn

O: (n log(suma)), binaria sobre la respuesta con un greedy (n)

Repartir bolsas en C tramos contiguos minimizando el peor: binaria sobre la respuesta

C++/Training_Camp_2026/Day 1/M - Maratona Brasileira de Popcorn.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n, c;
9     long long t;
10    cin >> n >> c >> t;
11
12    vector<long long> p(n);
13    long long lo = 0, hi = 0;
14    for (int i = 0; i < n; i++) {
15        cin >> p[i];
16        lo = max(lo, p[i]); // la bolsa más grande fija una cota inferior
17        hi += p[i]; // un solo competidor comiendo todo fija la cota superior
18    }
19
20    // Dada una cota S, calcula cuántos grupos contiguos (competidores)
21    // se necesitan para que ningún grupo tenga suma mayor que S.
22    auto gruposNecesarios = [&](long long S) -> long long {
```

10.1 Binary Search - Maratona Brasileira De Popcorn (cont.)

```

23     long long grupos = 1;
24     long long acumulado = 0;
25     for (int i = 0; i < n; i++) {
26         if (acumulado + p[i] > S) {
27             grupos++;
28             acumulado = p[i];
29         } else {
30             acumulado += p[i];
31         }
32     }
33     return grupos;
34 };
35
36 // Búsqueda binaria de la mínima "suma máxima" S alcanzable
37 // usando a lo sumo C grupos (competidores).
38 while (lo < hi) {
39     long long mid = lo + (hi - lo) / 2;
40     if (gruposNecesarios(mid) <= c) {
41         hi = mid;
42     } else {
43         lo = mid + 1;
44     }
45 }
46
47 long long S = lo; // suma máxima óptima que come un competidor
48 long long respuesta = (S + t - 1) / t; // techo(S / T)
49
50 cout << respuesta << "\n";
51 return 0;
52 }

```

10.2 Bit Manipulation - Lecographically Maximum

O: (n * bits), se reparten los bits por posición

Secuencia lexicográficamente máxima intercambiando bits sueltos entre elementos

C++/Training_Camp_2026/Day 3/L - Lecographically Maximum.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int N;
9     cin >> N;
10
11     vector<int> cnt(31, 0);
12
13     for (int i = 0; i < N; i++) {
14         int x;
15         cin >> x;
16         for (int b = 0; b <= 30; b++)
17             if (x & (1 << b))
18                 cnt[b]++;
19     }
20
21     vector<int> ans(N, 0);
22
23     for (int b = 0; b <= 30; b++)
24         for (int i = 0; i < cnt[b]; i++)
25             ans[i] |= (1 << b);
26
27     for (int i = 0; i < N; i++) {
28         if (i) cout << ' ';
29         cout << ans[i];
30     }
31     cout << '\n';
32 }

```

10.3 Budget Optimization - Posters

O: (1), fórmula sobre el presupuesto

Maximo de posters con presupuesto c, comprando al menos uno de cada tipo

C++/RPC/Posters.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using int64 = long long;
5
6 int main() {
7     long long a,b,c;
8
9     cin >> a >> b >> c;
10
11     cout << (min((c-a) / b, (c-b) / a) < 0 ? 0 : min((c-a) / b, (c-b) / a)) << endl;
12
13     return 0;
14 }

```

10.4 Buy Low Sell High (Comprar y Vender Acciones)

O: (n) las tres primeras, (n*K) la de K transacciones

Uso: maxProfit(p) -> {ganancia,{compra,venta}}; maxProfitK(p,K)

Precio de una acción por día: comprar un día y vender un día POSTERIOR (una transacción) maximizando la...

C++/Notebook/Buy Low Sell High.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5
6 const ll NEG = LLONG_MIN / 4;
7
8 // Una transaccion: {ganancia, {dia de compra, dia de venta}} (dias -1 si no conviene operar)
9 pair<ll, pair<int, int>> maxProfit(const vector<ll> &p)
10 {
11     ll best = 0;
12     int low = 0, buy = -1, sell = -1;
13     for (int i = 1; i < (int)p.size(); i++)
14     {
15         if (p[i] < p[low]) low = i;
16         else if (p[i] - p[low] > best)
17         {
18             best = p[i] - p[low];
19             buy = low;
20             sell = i;
21         }
22     }
23     return {best, {buy, sell}};
24 }
25
26 // Transacciones ilimitadas
27 ll maxProfitUnlimited(const vector<ll> &p)
28 {
29     ll s = 0;
30     for (int i = 1; i < (int)p.size(); i++) s += max(0LL, p[i] - p[i - 1]);
31     return s;
32 }
33
34 // Ilimitadas, pagando `fee` por cada transaccion completa
35 ll maxProfitFee(const vector<ll> &p, ll fee)
36 {
37     ll cash = 0, hold = NEG;
38     for (ll x : p)
39     {
40         hold = max(hold, cash - x);
41         cash = max(cash, hold + x - fee);
42     }
43 }

```

10.4 Buy Low Sell High (Comprar y Vender Acciones) (cont.)

```

43     }
44     return cash;
45 }
46
47 // A lo sumo K transacciones
48 ll maxProfitK(const vector<ll> &p, int K)
49 {
50     if (K <= 0) return 0;
51     if (2 * K >= (int)p.size()) return maxProfitUnlimited(p);
52     vector<ll> hold(K + 1, NEG), cash(K + 1, 0);
53     for (ll x : p)
54     {
55         for (int t = 1; t <= K; t++)
56         {
57             hold[t] = max(hold[t], cash[t - 1] - x);
58             cash[t] = max(cash[t], hold[t] + x);
59         }
60     }
61     return cash[K];
62 }
63
64 int main()
65 {
66     ios_base::sync_with_stdio(false);
67     cin.tie(nullptr);
68
69     int n;
70     cin >> n;
71     vector<ll> p(n);
72     for (auto &x : p) cin >> x;
73
74     auto r = maxProfit(p);
75     cout << r.first << "\n";
76     cout << r.second.first << " " << r.second.second << "\n"; // -1 -1 si no se opera
77     return 0;
78 }

```

10.5 Constructive Algorithms - Nomad

O: (n), construccion directa

Kai debe dormir exactamente D noches alternando entre N lugares

C++/Training_Camp_2026/Day 10/H - Nomad.py

```

1 n,m = list(map(int,input().split()))
2 l = list(map(int,input().split()))
3 d1 = -1
4 d2 = -1
5 for i in range(len(l)):
6     if(l[i]!= 0 and d1 == -1):
7         d1 = i
8     elif(l[i]!= 0 and d1 != -1):
9         d2 = i
10        break
11 if(d2 == -1):
12     if(d1!= -1 and l[d1]>=m):
13         for i in range(m):
14             print(d1+1, end=" ")
15     else:
16         print(-1)
17 else:
18     for i in range(m):
19         if(i%2 == 0):
20             print(d1+1, end=" ")
21         else:
22             print(d2+1, end=" ")

```

10.6 DP - Exhibition Of Clownfish

O: (n log n) por ordenar; la decision despues es lineal

Minimo de movimientos para que ninguna pecera quede con machos y sin hembra

C++/Training_Camp_2026/Day 1/E - Exhibition of Clownfish.cpp

#include <bits/stdc++.h>

10.6 DP - Exhibition Of Clownfish (cont.)

```

2 using namespace std;
3
4 int main(){
5     ios_base::sync_with_stdio(false);
6     cin.tie(NULL);
7
8     int n;
9     if(!(cin >> n)) return 0;
10
11     vector<long long> M(n), F(n);
12     long long S = 0; // total males
13     for(int i = 0; i < n; i++){
14         cin >> M[i] >> F[i];
15         S += M[i];
16     }
17
18     if(S == 0){ cout << 0 << "\n"; return 0; } // ya todas hembras
19
20     long long E = 0, femonly = 0, negmixed = 0;
21     long long PosV = 0, PosM = 0; // suma de v y de M sobre "mixtos positivo s"
22     long long bestSingle = LLONG_MIN; // mejor v de una sola cámara
23     vector<pair<long long, long long>> pos; // (peso = M, valor = v) de mixtos con v> 0
24
25     for(int i = 0; i < n; i++){
26         long long v = 1 + M[i] - F[i];
27         bestSingle = max(bestSingle, v);
28         if(M[i] == 0 && F[i] == 0) E++; // vacío
29         else if(M[i] == 0) femonly++; // solo hembras
30         else { // mixto {M>0 => F>0 por restricci
31             if(v > 0){ PosV += v; PosM += M[i]; pos.push_back({M[i], v}); }
32             else negmixed++;
33         }
34     }
35
36     long long FM0 = S - PosM; // machos libres provenientes de mixtos con v<=0
37
38     // Familia principal: max sobre w in [0,E] de bestPv(S-w) + w
39     long long best_family = PosV + min(E, FM0); // w <= FM0 : incluir todos los positivo s
40
41     if(E > FM0){
42         long long T = E - FM0; // T <= E <= n <= 3000
43         const long long INF = LLONG_MAX / 4;
44         vector<long long> dp(T + 1, INF);
45         dp[0] = 0;
46         for(auto &pr : pos){ // cover DP: min sum(v) para quitar >= j
47             de peso
48                 long long w = pr.first, v = pr.second;
49                 for(long long j = T; j >= 0; j--){
50                     if(dp[j] < INF){
51                         long long nj = (j + w >= T) ? T : (j + w);
52                         if(dp[j] + v < dp[nj]) dp[nj] = dp[j] + v;
53                     }
54                 }
55             for(long long j = 1; j <= T; j++)
56                 if(dp[j] < INF)
57                     best_family = max(best_family, PosV + FM0 + (j - dp[j]));
58         }
59
60         bool family_valid = true;
61         if(PosV == 0 && E == 0) family_valid = false; // daría 0 cámaras
62
63         // Caso frontera: TODOS los tanques son mixtos positivos -> hay que dejar 1 como drena je
64         if(E == 0 && femonly == 0 && negmixed == 0){
65             long long minv = LLONG_MAX;
66             for(auto &pr : pos) minv = min(minv, pr.second);
67             best_family = PosV - minv;

```

10.6 DP - Exhibition Of Clownfish (cont.)

```

88     family_valid = true;
89 }
90
91 long long maxB = bestSingle;           // cámara única (siempre válida,
92 n>=2)
93 if(family_valid) maxB = max(maxB, best_family);
94
95 cout << (2 * S - maxB) << "\n";
96 return 0;
97 }

```

10.7 Emparejar Extremos - Axles

O: (n log n) por ordenar los diámetros de cada eje

NO ES MIO: código de Stiven Correa, del repo del equipo (carpeta RPC/)

RPC/2026-07/A - Axles.py

```

1 from collections import defaultdict, deque
2 import math
3
4 dicc= defaultdict(list)
5
6 n = int(input())
7
8 for _ in range(n):
9     s, c = map(int, input().split())
10    dicc[s].append(c)
11
12
13 diccS = dict()
14 for key, val in dicc.items():
15     diccS[key] = deque(sorted(dicc[key]))
16
17 res = 0
18 for key, val in diccS.items():
19     while(len(val) > 1):
20         a, b = val.popleft(), val.pop()
21         res += math.log(max(a,b)/min(a,b))
22 print(res)

```

10.8 Huffman

O: (n log n) con priority_queue

Uso: lee frecuencias por stdin; el costo es la suma de las fusiones

Costo mínimo para combinar N elementos de a dos

C++/Notebook/Huffman.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main()
6 {
7     ios_base::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int n;
11    cin >> n;
12
13    priority_queue<long long, vector<long long>, greater<long long>> pq;
14    for (int i = 0; i < n; i++)
15    {
16        long long x;
17        cin >> x;
18        pq.push(x);
19    }
20
21    long long cost = 0;
22    while (pq.size() > 1)
23    {
24        long long x = pq.top(); pq.pop();
25        long long y = pq.top(); pq.pop();
26        cost += x + y;

```

10.8 Huffman (cont.)

```

27        pq.push(x + y);
28    }
29
30    cout << cost << "\n";
31    return 0;
32 }

```

10.9 Huffman sobre Frecuencias Variables en el Tiempo - Efficient Encoding

O: (n log n) por el heap de Huffman

Cada símbolo tiene una frecuencia que cambia con el tiempo, $f_i(t)$, lineal a trozos con un quiebre en m_i

C++/ICPC/2025/E - Efficient Encoding.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Symbol {
7     ll a, m, b, c;
8 };
9
10 // Costo optimo de un codigo prefijo (Huffman). Las frecuencias son REALES,
11 // porque en un t intermedio  $f_i(t)$  no es entera.
12 double huffman(vector<double> freq) {
13
14     priority_queue<double, vector<double>, greater<double>> pq;
15
16     for (double x : freq)
17         pq.push(x);
18
19     // Con un solo simbolo igual hace falta 1 bit por ocurrencia.
20     if (pq.size() == 1)
21         return pq.top();
22
23     double ans = 0;
24
25     while (pq.size() > 1) {
26
27         double x = pq.top();
28         pq.pop();
29
30         double y = pq.top();
31         pq.pop();
32
33         ans += x + y;
34         pq.push(x + y);
35     }
36
37     return ans;
38 }
39
40 //  $f_i(t)$ , lineal a trozos con el quiebre en  $m_i$ .
41 double frecuencia(const Symbol &s, double t, double T) {
42     if (t <= s.m)
43         return s.a + (double)(s.b - s.a) / s.m * t;
44     return s.b + (double)(s.c - s.b) / (T - s.m) * (t - s.m);
45 }
46
47 int main() {
48     ios::sync_with_stdio(false);
49     cin.tie(nullptr);
50
51     int K;
52     cin >> K;
53
54     while (K--) {
55
56         int n;
57         ll T;
58
59         cin >> n >> T;

```

10.9 Huffman sobre Frecuencias Variables en el Tiempo - Efficient Encoding (cont.)

```

80
81     vector<Symbol> s(n);
82
83     for (int i = 0; i < n; i++)
84         cin >> s[i].a >> s[i].m >> s[i].b >> s[i].c;
85
86     // Con las longitudes FIJAS, el costo es lineal a trozos con quiebres en
87     // los m_i. El optimo es el minimo de esas lineales, o sea CONCAVO entre
88     // quiebres, y una concava alcanza su minimo en un extremo del intervalo.
89     // Asi que basta mirar 0, T y cada m_i.
90     vector<ll> cand;
91     cand.push_back(0);
92     cand.push_back(T);
93     for (int i = 0; i < n; i++)
94         cand.push_back(s[i].m);
95
96     sort(cand.begin(), cand.end());
97     cand.erase(unique(cand.begin(), cand.end(), cand.end()), cand.end());
98
99     ll mejorT = 0;
100    double mejorC = 1e18;
101
102    for (ll t : cand) {
103
104        vector<double> f(n);
105
106        for (int i = 0; i < n; i++)
107            f[i] = frecuencia(s[i], t, T);
108
109        double c = huffman(f);
110
111        // Se queda el t mas chico: por eso la comparacion es estricta y los
112        // candidatos vienen ordenados.
113        if (c < mejorC - 1e-9) {
114            mejorC = c;
115            mejorT = t;
116        }
117    }
118
119    cout << mejorT << ' ' << fixed << setprecision(3) << mejorC << '\n';
120 }
121
122 return 0;
123 }

```

10.10 Interval Scheduling - Interval Scheduling Greedy

O: (n log n), ordenar por FIN y tomar el que no choque

Uso: ordenar por v[i].second y avanzar mientras inicio >= ultimo_fin

Maximo de intervalos sin solaparse: ordenar por hora de FIN, no de inicio

C++/Notebook/Interval Scheduling Greedy.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main()
6 {
7     ios_base::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int n;
11    cin >> n;
12
13    vector<pair<int,int>> v(n);
14    for (int i = 0; i < n; i++)
15    {
16        cin >> v[i].first >> v[i].second;
17    }
18
19    // ordenar por hora de fin
20    sort(v.begin(), v.end(), [](const pair<int,int> &a, const pair<int,int> &b){

```

10.10 Interval Scheduling - Interval Scheduling Greedy (cont.)

```

21     return a.second < b.second;
22 });
23
24 int cnt = 0;
25 int last = INT_MIN;
26
27 for (auto &it : v)
28 {
29     if (it.first >= last)
30     {
31         cnt++;
32         last = it.second;
33     }
34 }
35
36 cout << cnt << "\n";
37 return 0;
38 }

```

10.11 Maximo Conjunto a Distancia d en Arbol - Oficinas A Distancia D En Arbol

O: (n), un BFS para ordenar y un barrido de hojas a raiz

Uso: lee el arbol por stdin; el greedy va de las hojas hacia la raiz

Hay n ciudades en forma de arbol, cada una con una oficina

C++/Notebook/Oficinas a Distancia d en Arbol.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n, d;
9     cin >> n >> d;
10    vector<vector<int>> adj(n + 1);
11    for (int i = 0; i < n - 1; i++) {
12        int a, b;
13        cin >> a >> b;
14        adj[a].push_back(b);
15        adj[b].push_back(a);
16    }
17
18    // Orden BFS desde la raiz 1
19    vector<int> order, par(n + 1, 0);
20    order.reserve(n);
21    vector<char> vis(n + 1, 0);
22    order.push_back(1);
23    vis[1] = 1;
24    for (int i = 0; i < (int)order.size(); i++) {
25        int v = order[i];
26        for (int u : adj[v]) {
27            if (!vis[u]) {
28                vis[u] = 1;
29                par[u] = v;
30                order.push_back(u);
31            }
32        }
33    }
34
35    vector<int> dist(n + 1, 0), rep(n + 1);
36    vector<char> removed(n + 1, 0);
37    // todas las ciudades empiezan con oficina
38    for (int v = 1; v <= n; v++) rep[v] = v;
39
40    // Procesar de hojas a raiz, uniendo cada nodo con su padre
41    for (int i = n - 1; i >= 1; i--) {
42        int c = order[i], p = par[c];
43        int dc = dist[c] + 1;
44        if (dist[p] + dc < d) {
45            // conflicto: cerrar la oficina mas cercana a p
46            if (dist[p] < dc) {

```

10.11 Maximo Conjunto a Distancia d en Arbol - Oficinas A Distancia D En Arbol (cont.)

```

47         removed[rep[p]] = 1;
48         dist[p] = dc;
49         rep[p] = rep[c];
50     } else {
51         removed[rep[c]] = 1;
52     }
53     } else if (dc < dist[p]) {
54         dist[p] = dc;
55         rep[p] = rep[c];
56     }
57 }
58
59 vector<int> ans;
60 for (int v = 1; v <= n; v++)
61     if (!removed[v]) ans.push_back(v);
62
63 string out = to_string(ans.size()) + "\n";
64 for (int x : ans) out += to_string(x) + ' ';
65 out += '\n';
66 cout << out;
67 return 0;
68 }

```

10.12 Prefijos sobre Secuencias Balanceadas - Incompatible Pairs

O: (n), prefijos sobre la secuencia balanceada

Una secuencia de N bailarines Yang ('(') y Yin (')') donde cada Yang debe emparejarse con un Yin posterior

C++/Training_Camp_2026/Day 10/D - Incompatible Pairs.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 static char buf[1 << 22];
6 static int bufLen_, bufPos_;
7 inline int gc() {
8     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0;
9     if (!bufLen_) return -1; }
10    return buf_[bufPos_++];
11 }
12 inline int readInt() {
13     int c = gc();
14     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
15     bool neg = false; if (c == '-') { neg = true; c = gc(); }
16     int x = 0;
17     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
18     return neg ? -x : x;
19 }
20 inline void readString(string &s, int expectedLen) {
21     s.clear();
22     s.reserve(expectedLen);
23     int c = gc();
24     while (c != '(' && c != ')') c = gc();
25     while (c == '(' || c == ')') {
26         s.push_back((char)c);
27         c = gc();
28     }
29 }
30 int main() {
31     int N = readInt();
32     string s;
33     readString(s, N);
34     // in case the read stopped early due to buffering edge cases, N is authoritative
35     // for sizing arrays; s.size() should equal N given well-formed input.
36
37     vector<int> prefix(N + 1, 0);
38     for (int i = 1; i <= N; i++) {
39         prefix[i] = prefix[i - 1] + (s[i - 1] == '(' ? 1 : -1);
40     }
41
42     // f[i] = smallest k >= i with prefix[k] == 0, for i = 1..N (computed right to left).

```

10.12 Prefijos sobre Secuencias Balanceadas - Incompatible Pairs (cont.)

```

43     vector<int> f(N + 2, 0);
44     f[N] = (prefix[N] == 0) ? N : N + 1;
45     for (int i = N - 1; i >= 1; i--) {
46         f[i] = (prefix[i] == 0) ? i : f[i + 1];
47     }
48
49     // suffixClose[k] = number of ')' at 1-indexed positions > k, for k = 0..N.
50     vector<ll> suffixClose(N + 2, 0);
51     for (int k = N - 1; k >= 0; k--) {
52         suffixClose[k] = suffixClose[k + 1] + (s[k] == ')') ? 1 : 0;
53     }
54
55     ll total = 0;
56     for (int i = 1; i <= N; i++) {
57         if (s[i - 1] == '(') {
58             total += suffixClose[f[i]];
59         }
60     }
61
62     printf("%lld\n", total);
63     return 0;
64 }

```

10.13 Prefix Sums - Maximizing Advertising

O: (n), sumas de prefijos

Dados N votantes con coordenadas (x,y) y preferencia 'b' o 'w'

C++/Training_Camp_2026/Day 7/B - Maximizing Advertising.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 static char buf[1 << 25];
6 static int bufLen = 0, bufPos = 0;
7
8 inline int gc() {
9     if (bufPos == bufLen) {
10         bufLen = (int)fread(buf, 1, sizeof(buf), stdin);
11         bufPos = 0;
12         if (bufLen <= 0) return -1;
13     }
14     return buf[bufPos++];
15 }
16
17 inline int readInt() {
18     int c = gc();
19     while (c == ' ' || c == '\n' || c == '\r' || c == '\t') c = gc();
20     bool neg = false;
21     if (c == '-') { neg = true; c = gc(); }
22     int x = 0;
23     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
24     return neg ? -x : x;
25 }
26
27 inline char readChar() {
28     int c = gc();
29     while (c == ' ' || c == '\n' || c == '\r' || c == '\t') c = gc();
30     return (char)c;
31 }
32
33 int N;
34 vector<int> Xs, Ys;
35 vector<char> Cs;
36
37 // dado un vector de coordenadas (alineado con Cs), calcula el mejor
38 // resultado separando en dos grupos (izq/der) por esa coordenada
39 ll bestSplit(vector<int>& coord) {
40     int n = N;
41     vector<pair<int, char>> arr(n);
42     for (int i = 0; i < n; i++) arr[i] = {coord[i], Cs[i]};
43     sort(arr.begin(), arr.end(), [](const pair<int, char>& a, const pair<int, char>& b) {

```

10.13 Prefix Sums - Maximizing Advertising (cont.)

```

44     return a.first < b.first;
45 });
46
47 ll totalB = 0, totalW = 0;
48 for (auto& pr : arr) { if (pr.second == 'b') totalB++; else totalW++; }
49
50 ll best = max(totalB, totalW); // caso: todo a un lado, el otro rectangulo vacio
51
52 ll prefB = 0, prefW = 0;
53 int i = 0;
54 while (i < n) {
55     int j = i;
56     int cur = arr[i].first;
57     ll gB = 0, gW = 0;
58     while (j < n && arr[j].first == cur) {
59         if (arr[j].second == 'b') gB++; else gW++;
60         j++;
61     }
62     prefB += gB; prefW += gW;
63     // corte justo despues de este grupo de coordenada 'cur'
64     ll optA = prefB + (totalW - prefW); // b a la izquierda, w a la derecha
65     ll optB = prefW + (totalB - prefB); // w a la izquierda, b a la derecha
66     best = max(best, max(optA, optB));
67     i = j;
68 }
69 return best;
70 }
71
72 int main() {
73     N = readInt();
74     Xs.resize(N); Ys.resize(N); Cs.resize(N);
75     for (int i = 0; i < N; i++) {
76         Xs[i] = readInt();
77         Ys[i] = readInt();
78         Cs[i] = readChar();
79     }
80
81     ll ans = max(bestSplit(Xs), bestSplit(Ys));
82     printf("%lld\n", ans);
83     return 0;
84 }

```

10.14 Puente y Linterna - The Bridge At Night

O: (n log n) por el sort; el greedy despues es (n)

Cruzar un puente de a dos con una linterna en tiempo minimo: greedy sobre los ordenados

C++/ICPC/2024/B - The Bridge At Night.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9
10    while (cin >> n && n != 0) {
11
12        vector<int> t(n);
13
14        for (int &x : t)
15            cin >> x;
16
17        sort(t.begin(), t.end());
18
19        int ans = 0;
20
21        while (n > 3) {
22
23            int a = t[0];
24            int b = t[1];

```

10.14 Puente y Linterna - The Bridge At Night (cont.)

```

25         int y = t[n - 2];
26         int z = t[n - 1];
27
28         int option1 = a + 2 * b + z;
29         int option2 = 2 * a + y + z;
30
31         ans += min(option1, option2);
32
33         n -= 2;
34     }
35
36     // Quedan 1, 2 o 3 personas
37     if (n == 3) {
38         ans += t[0] + t[1] + t[2];
39     }
40     else if (n == 2) {
41         ans += t[1];
42     }
43     else if (n == 1) {
44         ans += t[0];
45     }
46
47     cout << ans << '\n';
48 }
49
50 return 0;
51 }

```

10.15 Puente y Linterna (version con indices) - Bridge Juan

O: (n log n) por el sort, igual que la version B

La otra solucion del problema B de ICPC 2024 ("The Bridge at Night")

C++/ICPC/2024/bridge Juan.cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 using namespace std;
6
7 int main()
8 {
9     long long n;
10
11    while (cin >> n && n != 0)
12    {
13        vector<long long> tiempo(n);
14
15        for (long long i = 0; i < n; i++)
16        {
17            cin >> tiempo[i];
18        }
19
20        sort(tiempo.begin(), tiempo.end());
21
22        long long total = 0;
23        long long quedan = n;
24
25        while (quedan > 3)
26        {
27            long long escolta = 2 * tiempo[0] + tiempo[quedan - 2] + tiempo[quedan - 1];
28            long long parejaLenta = tiempo[0] + 2 * tiempo[1] + tiempo[quedan - 1];
29
30            total += min(escolta, parejaLenta);
31            quedan -= 2;
32        }
33
34        if (quedan == 3)
35        {
36            total += tiempo[0] + tiempo[1] + tiempo[2];
37        }
38        else if (quedan == 2)

```

10.15 Puente y Linterna (version con indices) - Bridge Juan (cont.)

```
38 {
39     total += tiempo[1];
40 }
41 else
42 {
43     total += tiempo[0];
44 }
45 }
46
47 cout << total << '\n';
48 }
49
50 return 0;
51 }
```

10.16 Segment Counting - Cerros

O: (n), un barrido contando segmentos

Cuenta la cantidad de segmentos contiguos con elementos $\geq k$ en un arreglo

C++/Codeforces/cerros.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     long long n,k;
9     cin >> n >> k;
10    long long god=0;
11    bool abierto = false;
12    for(int i = 0; i < n;i++)
13    {
14        int a; cin >> a;
15
16        if(a>=k&&!abierto)
17        {
18            god++;
19            abierto = true;
20        }
21        else if(a<k)
22        {
23            abierto = false;
24        }
25    }
26
27    cout << god << "\n";
28
29    return 0;
30 }
```

10.17 Simulation - Kathmandu

O: (n), simulacion

Durante un vuelo de duracion D hay M comidas servidas en instantes y_i (hay que estar despierto para comer)

C++/Training_Camp_2026/Day 4/F - Kathmandu.py

```
1 t, v, n = list(map(int,input().split()))
2 c = [0]
3 for i in range(n):
4     c.append(int(input()))
5 c.append(v)
6
7 f = False
8 for i in range(len(c)-1):
9     if(t <= abs(c[i]-c[i+1])):
10        f = True
11        break
12
13 if(f):
14     print("Y")
15 else:
16     print("N")
```

10.18 Sliding Window

O: (n) amortizado, cada elemento entra y sale una vez

Uso: exige condicion monotona; con negativos NO sirve, usa prefijos + map

Ventana de tamano variable para el subarreglo mas largo con condicion monotona

C++/Notebook/Sliding Window.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main()
6 {
7     ios_base::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int n;
11    long long K;
12    cin >> n >> K;
13
14    vector<long long> a(n);
15    for (int i = 0; i < n; i++) cin >> a[i];
16
17    int l = 0;
18    long long cur = 0, ans = 0;
19
20    for (int r = 0; r < n; r++)
21    {
22        cur += a[r];
23        while (cur > K)
24        {
25            cur -= a[l];
26            l++;
27        }
28        ans = max(ans, (long long)(r - l + 1));
29    }
30
31    cout << ans << "\n";
32    return 0;
33 }
```

10.19 Sorted Pairing - Balatro

O: (n log n) por el sort

Calcula la suma de diferencias absolutas entre pares ordenados de dos arreglos

C++/Codeforces/balatro.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main() {
6
7     long long n;
8     cin >> n;
9     vector<long long > yo(n);
10    vector<long long > atk(n);
11
12    for(int i = 0; i<n ; i++)
13    {
14        long long a;
15        cin >> a;
16        yo[i] = a;
17    }
18
19    for(int i = 0; i<n ; i++)
20    {
21        long long a;
22        cin >> a;
23        atk[i] = a;
24    }
25
26    long long ans = 0;
```

10.19 Sorted Pairing - Balatro (cont.)

```

27
28     sort(yo.begin(), yo.end());
29     sort(atk.begin(), atk.end());
30
31     for(int i = 0; i<n ; i++)
32     {
33         ans+= abs(atk[i] - yo[i]);
34     }
35
36     cout << ans << endl;
37
38     return 0;
39 }

```

10.20 Sorting - Finding Forks

O: (n log n) por el sort

En un cajon con n tipos de tenedores, cada tipo con a_i unidades

C++/Training_Camp_2026/Day 11 - CCPLA - Finding Forks.py

```

1 n = int(input())
2 l = list(map(int, input().split()))
3
4 l.sort()
5
6 print(l[0]+l[1])

```

10.21 Sorting (Prefix Sums) - Coven Complications

O: (n log n) por el sort, mas prefijos lineales

Un grafo de n comunidades brujas unidas por m portales

C++/Training_Camp_2026/Day 11 - CCPLF - Coven Complications.py

```

1 n, c = list(map(int, input().split()))
2
3 hurones = list(map(int, input().split()))
4
5 citiesW = [0]*(n+1)
6 for i in range(c):
7     a, b = list(map(int, input().split()))
8     a, b = (hurones[a-1], a), (hurones[b-1], b)
9     if a[0] == 0 or b[0] == 0:
10         obj = min(a,b)
11         citiesW[obj[1]] += max(a[0],b[0])
12
13 pre = []
14 citiesW.sort()
15 prev = 0
16 for i in citiesW:
17     prev += i
18     if i != 0:
19         pre.append(prev)
20
21 sol = 0
22 for i in range(len(pre)-1):
23     sol += pre[i]
24
25 print(sol)

```

10.22 Two Pointers (Container With Most Water)

O: (n), dos punteros desde los extremos

Uso: maxArea(h) -> {area, {i, j}}

Paredes verticales de altura h[i] en la posicion i

C++/Notebook/Container With Most Water.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 typedef long long ll;
6
7 // {area maxima, {i, j}} (i = j = -1 si hay menos de dos paredes)
8 pair<ll, pair<int, int>> maxArea(const vector<ll> &h)

```

10.22 Two Pointers (Container With Most Water) (cont.)

```

9 {
10     int l = 0, r = (int)h.size() - 1, bi = -1, bj = -1;
11     ll best = 0;
12     while (l < r)
13     {
14         ll a = (ll)(r - l) * min(h[l], h[r]);
15         if (a > best || bi == -1)
16         {
17             best = a;
18             bi = l;
19             bj = r;
20         }
21         if (h[l] < h[r]) l++;
22         else r--;
23     }
24     return {best, {bi, bj}};
25 }
26
27 int main()
28 {
29     ios_base::sync_with_stdio(false);
30     cin.tie(nullptr);
31
32     int n;
33     cin >> n;
34     vector<ll> h(n);
35     for (auto &x : h) cin >> x;
36
37     auto r = maxArea(h);
38     cout << r.first << "\n";
39     cout << r.second.first << " " << r.second.second << "\n";
40     return 0;
41 }

```

11 - Binary Search

11.1 Búsqueda sobre la Respuesta con Fracciones Exactas - Clean Streets

O: (N log(rango)), binaria sobre la respuesta con fracciones exactas

Elegir trabajadores con tarifas y ventanas de disponibilidad para limpiar S calles al menor costo

C++/ICPC/Regionals 2025/C - Clean Streets.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Cleaner {
7     ll H, L, U;
8 };
9
10 struct SegTree {
11     int n;
12     vector<ll> cnt;
13     vector<ll> cost;
14
15     SegTree(int n) : n(n) {
16         cnt.assign(4 * n + 5, 0);
17         cost.assign(4 * n + 5, 0);
18     }
19
20     void update(int node, int l, int r, int pos, ll dc, ll dcost) {
21         if (l == r) {
22             cnt[node] += dc;
23             cost[node] += dcost;
24             return;
25         }
26
27         int mid = (l + r) / 2;
28
29         if (pos <= mid)
30             update(node * 2, l, mid, pos, dc, dcost);

```

11.1 Búsqueda sobre la Respuesta con Fracciones Exactas - Clean Streets (cont.)

```

31     else
32         update(node * 2 + 1, mid + 1, r, pos, dc, dcost);
33
34     cnt[node] = cnt[node * 2] + cnt[node * 2 + 1];
35     cost[node] = cost[node * 2] + cost[node * 2 + 1];
36 }
37
38 void update(int pos, ll dc) {
39     update(1, 1, n, pos, dc, dc * pos);
40 }
41
42 // Minimo costo para tomar exactamente need calles
43 // usando los H mas pequenos.
44 ll query(int node, int l, int r, ll need) {
45     if (l == r) {
46         return need * l;
47     }
48
49     int mid = (l + r) / 2;
50
51     if (cnt[node * 2] >= need) {
52         return query(node * 2, l, mid, need);
53     }
54
55     ll leftCost = cost[node * 2];
56     ll leftCnt = cnt[node * 2];
57
58     return leftCost +
59         query(node * 2 + 1, mid + 1, r, need - leftCnt);
60 }
61
62 ll query(ll need) {
63     if (cnt[1] < need)
64         return -1;
65
66     return query(1, 1, n, need);
67 }
68 };
69
70 // a/b < c/d
71 bool fracLess(ll a, ll b, ll c, ll d) {
72     return a * d < c * b;
73 }
74
75 // a/b <= c/d
76 bool fracLE(ll a, ll b, ll c, ll d) {
77     return a * d <= c * b;
78 }
79
80 int main() {
81     ios::sync_with_stdio(false);
82     cin.tie(nullptr);
83
84     int N;
85     ll S, K;
86
87     cin >> N >> S >> K;
88
89     vector<Cleaner> a(N);
90
91     for (auto &[H, L, U] : a) {
92         cin >> H >> L >> U;
93     }
94
95     // Orden por L/H
96     vector<int> byL(N);
97     iota(byL.begin(), byL.end(), 0);
98
99     sort(byL.begin(), byL.end(), [&](int i, int j) {
100         return fracLess(
101             a[i].L, a[i].H,
102             a[j].L, a[j].H

```

11.1 Búsqueda sobre la Respuesta con Fracciones Exactas - Clean Streets (cont.)

```

103         ));
104     });
105
106     // Orden por U/H
107     vector<int> byU(N);
108     iota(byU.begin(), byU.end(), 0);
109
110     sort(byU.begin(), byU.end(), [&](int i, int j) {
111         return fracLess(
112             a[i].U, a[i].H,
113             a[j].U, a[j].H
114         ));
115     });
116
117     // Candidatos r = L_i / H_i
118     vector<int> candidates = byL;
119
120     SegTree st(100000);
121
122     int pL = 0;
123     int pU = 0;
124
125     bool found = false;
126
127     ll bestNum = 0;
128     ll bestDen = 1;
129
130     for (int id : candidates) {
131
132         ll num = a[id].L;
133         ll den = a[id].H;
134
135         // Agregar todos los trabajadores cuyo
136         // intervalo ya comenzo.
137         while (pL < N) {
138             int j = byL[pL];
139
140             if (!fracLE(
141                 a[j].L, a[j].H,
142                 num, den
143             ))
144                 break;
145
146             ll capacity = K / a[j].H;
147
148             st.update(a[j].H, capacity);
149
150             pL++;
151         }
152
153         // Eliminar trabajadores cuyo intervalo
154         // ya termino antes de r.
155         while (pU < N) {
156             int j = byU[pU];
157
158             if (!fracLess(
159                 a[j].U, a[j].H,
160                 num, den
161             ))
162                 break;
163
164             ll capacity = K / a[j].H;
165
166             st.update(a[j].H, -capacity);
167
168             pU++;
169         }
170
171         // Podemos limpiar S calles?
172         ll totalHours = st.query(S);
173
174         if (totalHours == -1)

```

11.1 Búsqueda sobre la Respuesta con Fracciones Exactas - Clean Streets (cont.)

```

175         continue;
176
177         // costo = r * totalHours
178         //
179         // r = num / den
180         //
181         // costo = num * totalHours / den
182
183         ll curNum = num * totalHours;
184         ll curDen = den;
185
186         ll g = gcd(curNum, curDen);
187
188         curNum /= g;
189         curDen /= g;
190
191         if (!found ||
192             (__int128)curNum * bestDen <
193             (__int128)bestNum * curDen) {
194
195             found = true;
196             bestNum = curNum;
197             bestDen = curDen;
198         }
199     }
200
201     if (!found) {
202         cout << "\n";
203         return 0;
204     }
205
206     cout << bestNum << ' ' << bestDen << '\n';
207 }

```

11.2 Búsqueda Ternaria (Real y Entera)

O: (log rango) evaluaciones de f

Uso: ternariaReal(lo, hi, f) y ternariaEntera(lo, hi, f) MAXIMIZAN; para minimizar, -f
 Maximo o minimo de una funcion que sube y despues baja (unimodal), sin derivar

C++/Notebook/Búsqueda Ternaria.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 typedef long long ll;
6
7 // Argumento x en [lo, hi] que maximiza f (f unimodal en reales)
8 double ternariaReal(double lo, double hi, function<double(double)> f)
9 {
10     for (int it = 0; it < 200; it++)
11     {
12         double m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
13         if (f(m1) < f(m2)) lo = m1;
14         else hi = m2;
15     }
16     return (lo + hi) / 2;
17 }
18
19 // Entero x en [lo, hi] que maximiza f (estrictamente sube, despues estrictamente baja)
20 ll ternariaEntera(ll lo, ll hi, function<ll(ll)> f)
21 {
22     while (lo < hi)
23     {
24         ll m = lo + (hi - lo) / 2;
25         if (f(m) < f(m + 1)) lo = m + 1; // todavia sube: el pico esta a la derecha
26         else hi = m;
27     }
28     return lo;
29 }
30
31 int main()

```

11.2 Búsqueda Ternaria (Real y Entera) (cont.)

```

32 {
33     ios_base::sync_with_stdio(false);
34     cin.tie(nullptr);
35     return 0;
36 }

```

11.3 Greedy (búsqueda binaria sobre la respuesta) - Minas Gerais' Walls

O: (n log(rango)), binaria sobre la respuesta

Una muralla de N segmentos consecutivos con alturas x_i se refuerza una unica vez eligiendo un segmento

C++/Training_Camp_2026/Day 6/M - Minas Gerais' walls.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 int N;
6 ll K;
7 vector<ll> x; // 0-indexed internamente, x[i] = altura del segmento (i+1)
8
9 // Dado un candidato H, ¿existe algun p tal que reforzando en p,
10 // TODOS los segmentos queden con altura >= H?
11 bool feasible(ll H) {
12     ll L = 0; // mayor indice (1-indexado) con x[i] < H, o 0 si no hay ninguno
13     ll U = LLONG_MAX; // cota superior para p, viene de la condicion de "alcanzar" cada deficit
14
15     for (int i = 1; i <= N; i++) {
16         if (x[i-1] < H) {
17             L = i; // como i crece, la ultima asignacion es el maximo indice con deficit
18             // Necesitamos: K - (p - i) >= H - x[i-1] => p <= i + K - H + x[i-1]
19             ll candidate = (ll)i + K - H + x[i-1];
20             U = min(U, candidate);
21         }
22     }
23
24     ll lo = max((ll)1, L);
25     ll hi = min((ll)N, U);
26     return lo <= hi;
27 }
28
29 int main(){
30     ios_base::sync_with_stdio(false);
31     cin.tie(NULL);
32
33     cin >> N >> K;
34     x.resize(N);
35     for (int i = 0; i < N; i++) cin >> x[i];
36
37     ll lo = *min_element(x.begin(), x.end()); // siempre alcanzable (bonus >= 0 siempre)
38     ll hi = *max_element(x.begin(), x.end()) + K; // cota superior segura
39
40     ll ans = lo;
41     while (lo <= hi) {
42         ll mid = lo + (hi - lo) / 2;
43         if (feasible(mid)) {
44             ans = mid;
45             lo = mid + 1;
46         } else {
47             hi = mid - 1;
48         }
49     }
50
51     cout << ans << "\n";
52     return 0;
53 }

```

11.4 Two Pointers - Black Hills Golden Jewels

O: (n log n) por el sort; los dos punteros son (n)

Dado el precio a_i de cada una de N categorías de joyas, un cliente debe elegir dos categorías distintas

C++/Training_Camp_2026/Day 5/B - Black Hills golden jewels.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int N;
11    ll K;
12    cin >> N >> K;
13
14    vector<ll> a(N);
15
16    for (int i = 0; i < N; i++)
17        cin >> a[i];
18
19    sort(a.begin(), a.end());
20
21    // La respuesta está entre la suma mínima y máxima posibles.
22    ll lo = a[0] + a[1];
23    ll hi = a[N - 1] + a[N - 2];
24
25    while (lo < hi) {
26
27        ll mid = (lo + hi) / 2;
28
29        /*
30         Contaremos cuántos pares tienen suma <= mid.
31
32         Usamos dos punteros.
33
34         Para cada i, buscamos el mayor j tal que
35
36             a[i] + a[j] <= mid
37
38         Entonces todos los índices entre
39         (i+1 ... j) también sirven.
40         */
41        ll cnt = 0;
42
43        int j = N - 1;
44
45        for (int i = 0; i < N; i++) {
46
47            while (j > i && a[i] + a[j] > mid)
48                j--;
49
50            if (j <= i)
51                break;
52
53            cnt += (j - i);
54        }
55
56        if (cnt >= K)
57            hi = mid;
58        else
59            lo = mid + 1;
60    }
61
62    cout << lo << '\n';
63
64    return 0;
65 }
```

12 - Constructive Algorithms

12.1 Hashing - Handling The Blocks

O: (n), hashing de bloques

N bloques numerados 1..N tienen cada uno un color

C++/Training_Camp_2026/Day 4/E - Handling the Blocks.py

```
1 n, k = list(map(int, input().split()))
2 l = []
3 for i in range(k):
4     l.append(set())
5 val = []
6 for i in range(n):
7     c, v = list(map(int, input().split()))
8     l[v-1].add(c)
9     val.append([c, v])
10
11
12 f = True
13 for i in range(n):
14     #h = val[i][0]
15     if (i+1 in l[val[i][1]-1]):
16         continue
17     else:
18         f = False
19         break
20
21 if(f):
22     print("Y")
23 else:
24     print("N")
```

12.2 Secuencias de Langford - Doubled Sequence II

O: (n), construcción de Langford

Para un n dado, construir una secuencia de longitud 2n donde cada valor de 1 a n aparece exactamente dos...

C++/Training_Camp_2026/Day 9/C - Doubled Sequence II.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 22];
5 static int bufLen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0;
8         if (!bufLen_) return -1; }
9     return buf_[bufPos_++];
10 }
11 inline int readInt() {
12     int c = gc();
13     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
14     bool neg = false; if (c == '-') { neg = true; c = gc(); }
15     int x = 0;
16     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
17     return neg ? -x : x;
18 }
19
20 string out;
21
22 inline void appendInt(int v) {
23     char tmp[12];
24     int len = 0;
25     if (v == 0) { tmp[len++] = '0'; }
26     else {
27         int x = v;
28         while (x > 0) { tmp[len++] = char('0' + x % 10); x /= 10; }
29     }
30     for (int i = len - 1; i >= 0; i--) out += tmp[i];
31     out += ' ';
32 }
33 // Appends consecutive even/odd terms from a to b inclusive (a,b same parity),
```

12.2 Secuencias de Langford - Doubled Sequence II (cont.)

```

34 // increasing if a<=b, decreasing if a>b.
35 inline void appendRange(int a, int b) {
36     if (a <= b) {
37         for (int v = a; v <= b; v += 2) appendInt(v);
38     } else {
39         for (int v = a; v >= b; v -= 2) appendInt(v);
40     }
41 }
42
43 // Davies (1959) construction for n = 4m, m >= 2.
44 void build4m(int m) {
45     appendRange(4*m-4, 2*m);
46     appendInt(4*m-2);
47     appendRange(2*m-3, 1);
48     appendInt(4*m-1);
49     appendRange(1, 2*m-3);
50     appendRange(2*m, 4*m-4);
51     appendInt(4*m);
52     appendRange(4*m-3, 2*m+1);
53     appendInt(4*m-2);
54     appendRange(2*m-2, 2);
55     appendInt(2*m-1);
56     appendInt(4*m-1);
57     appendRange(2, 2*m-2);
58     appendRange(2*m+1, 4*m-3);
59     appendInt(2*m-1);
60     appendInt(4*m);
61 }
62
63 // Davies (1959) construction for n = 4m-1, m >= 2.
64 void build4mMinus1(int m) {
65     appendRange(4*m-4, 2*m);
66     appendInt(4*m-2);
67     appendRange(2*m-3, 1);
68     appendInt(4*m-1);
69     appendRange(1, 2*m-3);
70     appendRange(2*m, 4*m-4);
71     appendInt(2*m-1);
72     appendRange(4*m-3, 2*m+1);
73     appendInt(4*m-2);
74     appendRange(2*m-2, 2);
75     appendInt(2*m-1);
76     appendInt(4*m-1);
77     appendRange(2, 2*m-2);
78     appendRange(2*m+1, 4*m-3);
79 }
80
81 int main() {
82     int t = readInt();
83     out.reserve(1 << 22);
84     while (t--) {
85         int n = readInt();
86         int r = n % 4;
87         if (r != 0 && r != 3) {
88             out += "-1\n";
89             continue;
90         }
91         if (n == 3) {
92             out += "2 3 1 2 1 3\n";
93         } else if (n == 4) {
94             out += "2 3 4 2 1 3 1 4\n";
95         } else if (r == 0) {
96             build4m(n / 4);
97             out.back() = '\n';
98         } else { // r == 3, n = 4m-1 with m = (n+1)/4
99             build4mMinus1((n + 1) / 4);
100             out.back() = '\n';
101         }
102     }
103     fwrite(out.data(), 1, out.size(), stdout);
104     return 0;
105 }

```

13 - Implementation

13.1 Ad Hoc - Toilet Seat

O: (n) por caso

Dada una secuencia de preferencias 'U' (arriba) o 'D' (abajo) de personas que usan el baño una tras otra

C++/Training_Camp_2026/Day 11 - CCPL/C - Toilet Seat.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    string s;
11    cin >> s;
12    ll ans1=0, ans2=0, ans3=0;
13
14    //1
15    if(s[0]== 'U' && s[1] == 'U') ans1+=0;
16    else if(s[0]== 'U' && s[1] == 'D') ans1+=2;
17    else if(s[0]== 'D' && s[1] == 'D') ans1+=1;
18    else ans1+=1;
19    for(int i = 2; i < s.size(); i++)
20    {
21        if(s[i] == 'D') ans1+=2;
22    }
23
24    //2
25    if(s[0]== 'U' && s[1] == 'U') ans2+=1;
26    else if(s[0]== 'U' && s[1] == 'D') ans2+=1;
27    else if(s[0]== 'D' && s[1] == 'D') ans2+=0;
28    else ans2+=2;
29    for(int i = 2; i < s.size(); i++)
30    {
31        if(s[i] == 'U') ans2+=2;
32    }
33
34    //3
35    char act = s[0];
36    for(int i = 1; i < s.size(); i++)
37    {
38        if(s[i] != act)
39        {
40            ans3++;
41            act = s[i];
42        }
43    }
44
45    cout << ans1 << endl;
46    cout << ans2 << endl;
47    cout << ans3 << endl;
48
49    return 0;
50 }

```

13.2 Ad Hoc - Rock Paper Scissors Tournament

O: (n log n) por ordenar el ranking final

En un torneo de piedra-papel-tijera con n jugadores

C++/Training_Camp_2026/Day 11 - CCPL/D - Rock-Paper-Scissors Tournament.py

```

1 n = list(map(int, input().split()))
2 if len(n) != 1:
3     n, k = n
4     while(n != 0):
5         if n == 1:
6             print("-")
7             n = list(map(int, input().split()))

```

13.2 Ad Hoc - Rock Paper Scissors Tournament (cont.)

```

8         if len(n) == 1:
9             break
10        else:
11            n, k = n
12        w = [0]*(n+1)
13        l = [0]*(n+1)
14
15        for i in range((k*n*(n-1))//2):
16            num1, res1, num2, res2 = input().split()
17            num1, num2 = int(num1), int(num2)
18            if res1 == res2:
19                continue
20            if res1 == "rock" and res2 == "paper":
21                w[num2] += 1
22                l[num1] += 1
23            elif res1 == "rock" and res2 == "scissors":
24                w[num1] += 1
25                l[num2] += 1
26            elif res1 == "paper" and res2 == "rock":
27                w[num1] += 1
28                l[num2] += 1
29            elif res1 == "paper" and res2 == "scissors":
30                w[num2] += 1
31                l[num1] += 1
32            elif res1 == "scissors" and res2 == "rock":
33                w[num2] += 1
34                l[num1] += 1
35            elif res1 == "scissors" and res2 == "paper":
36                w[num1] += 1
37                l[num2] += 1
38
39        for i in range(1, n+1):
40            wins = w[i]
41            loses = l[i]
42
43            if wins+loses == 0:
44                print("-")
45            else:
46                print(f"{{wins/(wins+loses)}}.3f}")
47            print("")
48            n = list(map(int, input().split()))
49            if len(n) == 1:
50                break
51            else:
52                n, k = n

```

13.3 Ad Hoc - Zero Or One

O: (n)

Alice, Beto y Clara juegan "cero o uno" mostrando cada uno la mano con un valor 0 o 1

C++/Training_Camp_2026/Day 2/A - Zero or One.py

```

1  a, b, z = list(map(int, input().split()))
2
3  if 0<(a+b+z)< 3:
4      if (a+b+z == 2 and a == 0) or (a+b+z == 1 and a == 1):
5          print("A")
6      if (a+b+z == 2 and b == 0) or (a+b+z == 1 and b == 1):
7          print("B")
8      if (a+b+z == 2 and z == 0) or (a+b+z == 1 and z == 1):
9          print("C")
10 else:
11     print('')

```

13.4 Ad Hoc - Collatz Polynomial

O: (pasos de Collatz)

Aline crea una variante de la Conjetura de Collatz para polinomios con coeficientes 0 o 1

C++/Training_Camp_2026/Day 6/C - Collatz polynomial.py

```

1  grado = int(input())
2  val = list(reversed(list(map(int, input().split()))))
3

```

13.4 Ad Hoc - Collatz Polynomial (cont.)

```

4  m = [1, 1]
5  c = 0
6  while val.count(1) > 1 or val[0] != 1:
7      #print(val)
8      if(val[0] == 1):
9          n = [0] * len(val)
10         for i in range(len(val)):
11             if(val[i]==0):
12                 continue
13             for j in range(len(m)):
14                 try:
15                     n[i+j]+=1
16                 except:
17                     n.append(1)
18             n[0]+=1
19             for i in range(len(val)):
20                 if(n[i]>=2):
21                     n[i]=0
22             val = n
23         else:
24             val.pop(0)
25             #if(c==5):
26                 # break
27             c+= 1
28     print(c)

```

13.5 Ad Hoc - Unraveling Monty Hall

O: (1)

Se simulan N juegos del problema de Monty Hall donde el jugador siempre elige la puerta 1 y luego siempre...

C++/Training_Camp_2026/Day 8/D - Unraveling Monty Hall.cpp

```

1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      int n; cin >> n;
6      int sum = 0;
7      while(n--)
8      {
9          int a; cin >>a;
10         if(a==3 or a==2)
11             {
12                 sum++;
13             }
14     }
15     cout << sum << endl;
16     //a
17     return 0;
18 }

```

13.6 Arreglos - Buffoon

O: (n), es el maximo del arreglo

En la eleccion del bufon de la corte, N candidatos reciben votos y gana el mas votado

C++/Training_Camp_2026/Day 1/B - Buffoon.py

```

1  n = int(input())
2  l = []
3  for i in range(n):
4      l.append(int(input()))
5
6  ma = max(l)
7  if(ma == l[0]):
8      print("S")
9  else:
10     print("N")

```

13.7 Backtracking (Enumeracion de bits desconocidos) - Harmonics With Interference

O: ($2^{\text{desconocidos}}$), backtracking sobre los bits perdidos

Rellenar los bits perdidos de M y N para que M sea divisible por N: backtracking

C++/Training_Camp_2026/Day 3/H - Harmonics with Interference.py

```
1 import copy
2 import sys
3
4 sys.setrecursionlimit(3*10**6)
5 m = input().strip()
6 n = input().strip()
7
8 ns = []
9 def generateBins(s, i, objective):
10     if i < 0:
11         objective.append(int("".join(s), 2))
12         return
13     elif s[i] == "*":
14         way = copy.copy(s)
15         way[i] = "1"
16         generateBins(way, i-1, objective)
17         way = copy.copy(s)
18         way[i] = "0"
19         generateBins(way, i-1, objective)
20     else:
21         generateBins(s, i-1, objective)
22
23 generateBins(list(m), len(m)-1, ns)
24
25
26 L = len(m)
27 estrellas = [idx for idx, c in enumerate(m) if c == "*"]
28 base = int("".join("0" if c == "*" else c for c in m), 2) # M con todos los * = 0
29 peso = [1 << (L - 1 - idx) for idx in estrellas] # lo que suma poner un 1 en es
30     e *
31
32 def generateAndCheck(t, val):
33     if t == len(estrellas):
34         for k in ns:
35             if k != 0 and val % k == 0:
36                 print(bin(val)[2:].zfill(L)) # una sola vez, ya con la longitud correcta
37                 sys.exit(0)
38         return
39     generateAndCheck(t + 1, val + peso[t]) # ese * = 1
40     generateAndCheck(t + 1, val) # ese * = 0
41
42 generateAndCheck(0, base)
43 print(-1)
```

13.8 Brute Force - Cake Hater

O: (n^2), fuerza bruta

Una receta de pastel con N pasos, cada uno con su propia lista de ingredientes

C++/Training_Camp_2026/Day 10/G - Cake Hater.py

```
1 n = int(input())
2
3 a, b = list(map(int, input().split()))
4
5 dongs = set(map(int, input().split()))
6
7 sol = False
8 for i in range(n):
9     ingreds = list(map(int, input().split()))
10
11     counter = 0
12     if not sol:
13         for j in range(1, ingreds[0]+1):
14             if ingreds[j] in dongs:
15                 counter += 1
16             if counter > ingreds[0]/3:
17                 print(i+1)
18                 sol = True
```

13.8 Brute Force - Cake Hater (cont.)

```
8 break
20 if not sol:
21     print(-1)
```

13.9 Brute Force - LLMs

O: (n^2), fuerza bruta

Bruno arma un predictor de palabras "SBC" a partir de un diccionario de palabras con vectores 2D

C++/Training_Camp_2026/Day 6/L - LLMs.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     ios_base::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int N;
9     cin >> N;
10
11     vector<string> dictWord(N);
12     vector<pair<long long, long long>> dictVec(N);
13     unordered_map<string, int> firstIndex; // first occurrence of a word in the dictionary
14
15     for(int i = 0; i < N; i++){
16         string w; long long x, y;
17         cin >> w >> x >> y;
18         dictWord[i] = w;
19         dictVec[i] = {x, y};
20         if(firstIndex.find(w) == firstIndex.end())
21             firstIndex[w] = i;
22     }
23
24     int M;
25     cin >> M;
26     vector<string> kb(M);
27     for(int i = 0; i < M; i++){
28         cin >> kb[i];
29     }
30
31     int Q, K;
32     cin >> Q >> K;
33
34     for(int q = 0; q < Q; q++){
35         int F;
36         cin >> F;
37         vector<string> query(F);
38         for(int i = 0; i < F; i++){
39             cin >> query[i];
40         }
41
42         int curK = K;
43         vector<string> candidates;
44
45         while(curK >= 1){
46             // last curK words of the ORIGINAL query
47             vector<string> ctx(query.end() - curK, query.end());
48             candidates.clear();
49
50             // search knowledge base for consecutive occurrences of ctx
51             for(int j = 0; j + curK - 1 < M; j++){
52                 bool match = true;
53                 for(int t = 0; t < curK; t++){
54                     if(kb[j + t] != ctx[t]){ match = false; break; }
55                 }
56                 if(match){
57                     int nextPos = j + curK;
58                     if(nextPos < M){
59                         candidates.push_back(kb[nextPos]);
60                     }
61                 }
62             }
63
64             if(!candidates.empty()) break;
```

13.9 Brute Force - LLMs (cont.)

```

63     curK--;
64 }
65
66 // print the query words
67 for(int i = 0; i < F; i++){
68     cout << query[i];
69     cout << " ";
70 }
71
72 if(candidates.empty()){
73     cout << "\n";
74     continue;
75 }
76
77 // Precompute candidate vectors (0,0) if word not in dictionary
78 vector<pair<long long,long long>> candVecs;
79 candVecs.reserve(candidates.size());
80 for(auto &c : candidates){
81     auto it = firstIndex.find(c);
82     if(it != firstIndex.end()){
83         candVecs.push_back(dictVec[it->second]);
84     }
85     else
86         candVecs.push_back({0,0});
87 }
88
89 // sum of candidate vectors, so S(d) = d . sum(candidates)
90 long long sumX = 0, sumY = 0;
91 for(auto &cv : candVecs){
92     sumX += cv.first;
93     sumY += cv.second;
94 }
95
96 long long bestS = LLONG_MIN;
97 int bestIdx = -1;
98
99 for(int i = 0; i < N; i++){
100     long long s = dictVec[i].first * sumX + dictVec[i].second * sumY;
101     if(s > bestS){
102         bestS = s;
103         bestIdx = i;
104     }
105 }
106
107 cout << dictWord[bestIdx] << "\n";
108 }
109
110 return 0;
111 }

```

13.10 Circular Queue - Eeny Meeny

O: (n*k) simulando la ronda; con la formula de Josephus seria (n)
 Repartir niños en dos equipos contando una rima en circulo, tipo Josephus

C++/Training_Camp_2026/Day 11 - CCPLUB - Eeny Meeny.py

```

1  n = input().split()
2
3  n = len(n)
4
5  k = int(input())
6
7  people = []
8
9  for i in range(k):
10     nombre = input()
11     people.append(nombre)
12
13
14 curr = 0
15 fuera = set()
16 group = 0
17 groups = [[], []]
18 curr = 0

```

13.10 Circular Queue - Eeny Meeny (cont.)

```

8  currN = 0
9  while(len(fuera) != len(people)):
10     if curr == k:
11         curr = 0
12     while curr in fuera:
13         curr+=1
14         if curr == k:
15             curr = 0
16     currN += 1
17     if currN == n:
18         groups[group].append(people[curr])
19         fuera.add(curr)
20         currN = 0
21         if group == 1:
22             group = 0
23         else:
24             group = 1
25         curr += 1
26
27 for i in groups:
28     print(len(i))
29     for j in i:
30         print(j)

```

13.11 Clasificación de Cadenas por Prefijo - Joker Or Judger

O: (n) por cadena

Cada reporte que llega es una cadena de una lista fija y vale cierto puntaje

C++/Codeforces/JokerOrJudger.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int score(string s) {
5      if (s == "UnreasonableProblemArrangement")
6          return 10;
7
8      if (s.size() == 13 && s.substr(0, 12) == "WrongProblem" &&
9          s.back() >= 'A' && s.back() <= 'L')
10         return 100;
11
12      if (s.size() == 12 && s.substr(0, 11) == "SameProblem" &&
13          s.back() >= 'A' && s.back() <= 'L')
14         return 30;
15
16      if (s.size() == 28 && s.substr(0, 27) == "UnreasonableLimitForProblem" &&
17          s.back() >= 'A' && s.back() <= 'L')
18         return 5;
19
20      if (s.size() == 20 && s.substr(0, 19) == "WeakTestsForProblem" &&
21          s.back() >= 'A' && s.back() <= 'L')
22         return 3;
23
24      if (s.size() == 11 && s.substr(0, 10) == "BadProblem" &&
25          s.back() >= 'A' && s.back() <= 'L')
26         return 1;
27
28      return 0;
29 }
30
31 int main() {
32     ios::sync_with_stdio(false);
33     cin.tie(nullptr);
34
35     int T;
36     cin >> T;
37
38     while (T--) {
39         int n, P;
40         cin >> n >> P;
41
42         int total = 0;

```

13.11 Clasificación de Cadenas por Prefijo - Joker Or Judger (cont.)

```

43
44     while (n--) {
45         string s;
46         cin >> s;
47         total += score(s);
48     }
49
50     cout << (total > P ? "Joker" : "Judger") << '\n';
51 }
52 }

```

13.12 Conjuntos - João João

O: (n), operaciones de conjuntos

Ya se crearon 10 tareas para un examen, cada una con un nivel de dificultad entre 1 y 4

C++/Training_Camp_2026/Day 6/J - João João.py

```

1 n = set(map(int,input().split()))
2 print(4-len(n))

```

13.13 Constructive - A Story About Tea

O: (n), construcción directa

N botes deben viajar de Portugal a Inglaterra usando China como puerto auxiliar

C++/Training_Camp_2026/Day 7/I - A story about tea.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string out;
5
6 inline void addMove(char from, char to) {
7     out += from;
8     out += ' ';
9     out += to;
10    out += '\n';
11 }
12
13 // mueve n botes de X a Y usando Z como auxiliar, minimal (2^n - 1 movimientos)
14 void h(int n, char X, char Y, char Z) {
15     if (n == 0) return;
16     h(n - 1, X, Z, Y);
17     addMove(X, Y);
18     h(n - 1, Z, Y, X);
19 }
20
21 // mueve n botes de X a Y usando Z como auxiliar, con exactamente 1 movimiento extra (2^n
22 // movimientos)
23 void g(int n, char X, char Y, char Z) {
24     if (n == 1) {
25         addMove(X, Z);
26         addMove(Z, Y);
27         return;
28     }
29     h(n - 1, X, Z, Y);
30     addMove(X, Y);
31     g(n - 1, Z, Y, X);
32 }
33
34 int main() {
35     long long N, K;
36     cin >> N >> K;
37
38     long long minK = (1LL << N) - 1;
39
40     if (K < minK) {
41         cout << "N\n";
42         return 0;
43     }
44
45     long long delta = K - minK;
46     bool useG = (delta % 2 == 1);
47     long long padCount = useG ? (delta - 1) / 2 : delta / 2;

```

13.13 Constructive - A Story About Tea (cont.)

```

43
44     out.reserve((size_t)K * 4 + 16);
45
46     for (long long i = 0; i < padCount; i++) {
47         addMove('A', 'B');
48         addMove('B', 'A');
49     }
50
51     if (useG) {
52         g((int)N, 'A', 'C', 'B');
53     } else {
54         h((int)N, 'A', 'C', 'B');
55     }
56
57     cout << "Y\n";
58     cout << out;
59     return 0;
60 }

```

13.14 Conteo de Records - Harder Horizons

O: (n), contar records en una pasada

Cuenta cuantos elementos son mayores que TODOS los anteriores, o sea los "records" o maximos de prefijo

C++/ICPC/Regionals 2025/H - Harder Horizons.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 int main() {
6     ios::sync_with_stdio(false);
7     cin.tie(nullptr);
8     ll n;
9     cin >> n;
10    ll maxi = -1;
11    ll res = 0;
12    while(n--)
13    {
14        ll x; cin >> x;
15        if(x>maxi)
16        {
17            res++;
18            maxi = x;
19        }
20    }
21
22    cout << res << endl;
23
24    return 0;
25 }

```

13.15 Cycle Detection - Switches

O: (n), detección de ciclos en el grafo funcional

Hay N interruptores que alternan (encienden/apagan) subconjuntos de M focos

C++/Training_Camp_2026/Day 8/I - Switches.py

```

1 n,m = list(map(int,input().split()))
2 bomb = ["0"]*m
3 k = list(map(int,input().split()))
4 for i in range(1,len(k)):
5     bomb[k[i]-1]="1"
6
7 q = bomb.count("0")
8 c = 0
9 if(q == m):
10    print(0)
11 else:
12    entrada = []
13    pos = set()
14    c = 0
15    for j in range(n):

```

13.15 Cycle Detection - Switches (cont.)

```

6      #print(bomb)
7      w = input().split()
8      s = list(map(int,w))
9      entrada.append(s)
10     for i in range(1,len(s)):
11         if(bomb[s[i]-1] == "0"):
12             bomb[s[i]-1] = "1"
13         else:
14             bomb[s[i]-1] = "0"
15     q = bomb.count("0")
16     c += 1
17     if(q == m):
18         break
19     pos.add(str("".join(bomb))+str(j))
20
21 if(q == m):
22     print(c)
23 else:
24     i = 0
25     while True:
26         #print(bomb)
27         s = entrada[i]
28         for j in range(1,len(s)):
29             if(bomb[s[j]-1] == "0"):
30                 bomb[s[j]-1] = "1"
31             else:
32                 bomb[s[j]-1] = "0"
33         q = bomb.count("0")
34         c += 1
35         if(q == m):
36             print(c)
37             break
38         inte = str("".join(bomb))+str(i)
39         if(inte in pos):
40             print(-1)
41             break
42         pos.add(str("".join(bomb))+str(i))
43
44 i+=1
45 if(i >= len(entrada)):
46     i = 0

```

13.16 Date Parsing - Crochet

O: (1) por caso, arreglo de 14 días para el wrap

Calcula la diferencia de tiempo (hacia adelante) entre dos momentos dados como día de la semana + hora

C++/RPC/Crochet.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using int64 = long long;
5
6  int main() {
7      string d1, d2;
8      string h1, h2;
9      cin >> d1 >> h1 >> d2 >> h2;
10
11     string dias[] = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun",
12                     "Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};
13
14     // Parsear horas HH:MM
15     int hh1, mm1, hh2, mm2;
16     sscanf(h1.c_str(), "%d:%d", &hh1, &mm1);
17     sscanf(h2.c_str(), "%d:%d", &hh2, &mm2);
18
19     int t1 = hh1 * 60 + mm1;
20     int t2 = hh2 * 60 + mm2;
21
22     // Encontrar el índice de d1 en la primera semana
23     int idx1 = 0;
24     for (int i = 0; i < 7; i++) {

```

13.16 Date Parsing - Crochet (cont.)

```

25         if (dias[i] == d1) { idx1 = i; break; }
26     }
27
28     // Buscar d2 desde idx1 en adelante (en el arreglo de 14 días)
29     // Si es el mismo día y la hora de fin es menor o igual, avanzar a la siguiente semana
30     int idx2 = -1;
31     for (int i = idx1; i < 14; i++) {
32         if (dias[i] == d2) {
33             if (i == idx1 && t2 <= t1) continue;
34             idx2 = i;
35             break;
36         }
37     }
38
39     int64 diff = (int64)(idx2 - idx1) * 24 * 60 + (t2 - t1);
40
41     int64 days = diff / (24 * 60);
42     int64 hours = (diff % (24 * 60)) / 60;
43     int64 mins = diff % 60;
44
45     vector<string> parts;
46     if (days) parts.push_back(to_string(days) + (days == 1 ? " day" : " days"));
47     if (hours) parts.push_back(to_string(hours) + (hours == 1 ? " hour" : " hours"));
48     if (mins) parts.push_back(to_string(mins) + (mins == 1 ? " minute" : " minutes"));
49
50     if (parts.size() == 3)
51         cout << parts[0] << ", " << parts[1] << ", " << parts[2] << "\n";
52     else if (parts.size() == 2)
53         cout << parts[0] << " and " << parts[1] << "\n";
54     else
55         cout << parts[0] << "\n";
56
57     return 0;
58 }

```

13.17 Día Siguiente y Años Bisiestos

O: (1) siguienteDia, (n) el barrido de registros

Uso: siguienteDia(D1,M1,Y1, D2,M2,Y2) -> la segunda fecha es el día de después?

Aritmética de calendario sin usar <ctime>

C++/Notebook/Día Siguiente y Años Bisiestos.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  bool siguienteDia(long long D1, long long M1, long long Y1,
5                  long long D2, long long M2, long long Y2)
6  {
7      int dias[] = {0,31,28,31,30,31,30,31,31,30,31,30,31};
8
9      if(Y1 % 400 == 0 || (Y1 % 4 == 0 && Y1 % 100 != 0))
10         dias[2] = 29;
11
12     D1++;
13
14     if(D1 > dias[M1])
15     {
16         D1 = 1;
17         M1++;
18
19         if(M1 == 13)
20         {
21             M1 = 1;
22             Y1++;
23         }
24     }
25
26     return D1 == D2 && M1 == M2 && Y1 == Y2;
27 }
28
29 int main()
30 {

```

13.17 Día Siguiente y Años Bisiestos (cont.)

```

31 ios::sync_with_stdio(false);
32 cin.tie(nullptr);
33
34 int n;
35
36 while(cin >> n && n)
37 {
38     long long ans = 0, cnt = 0;
39
40     long long D0, M0, Y0, C0;
41
42     for(int i = 0; i < n; i++)
43     {
44         long long D, M, Y, C;
45         cin >> D >> M >> Y >> C;
46
47         if(i > 0 && siguienteDia(D0, M0, Y0, D, M, Y))
48         {
49             cnt++;
50             ans += C - C0;
51         }
52
53         D0 = D;
54         M0 = M;
55         Y0 = Y;
56         C0 = C;
57     }
58
59     cout << cnt << ' ' << ans << '\n';
60 }
61 }

```

13.18 Fuerza Bruta $O(n^2)$ sobre Subperiodos - Accounting Qualifying

O: (n^2); medido, 5 casos de 10000 en 94 ms

Contar subperiodos con tantos depositos como retiros, por fuerza bruta $O(n^2)$

C++/ICPC/2025/A - Accounting Qualifying.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9
10    while (cin >> n && n != 0) {
11
12        vector<int> a(n);
13
14        for (int &x : a)
15            cin >> x;
16
17        // Si no hay depositos d es 0, y si no hay retiros w es 0: por eso se
18        // acotan con 0 en vez de tomar el maximo y el minimo a secas.
19        int d = max(0, *max_element(a.begin(), a.end()));
20        int w = min(0, *min_element(a.begin(), a.end()));
21
22        // +1 = deposito
23        // 0 = balance
24        // -1 = retiro
25        for (int &x : a) {
26            if (x > 0) x = 1;
27            else if (x < 0) x = -1;
28        }
29
30        int r = 0;
31
32        // Todos los subperiodos, sin podas: si la suma de signos es 0, hay
33        // tantos depositos como retiros.
34        for (int i = 0; i < n; i++) {

```

13.18 Fuerza Bruta $O(n^2)$ sobre Subperiodos - Accounting Qualifying (cont.)

```

35
36        int sum = 0;
37
38        for (int j = i; j < n; j++) {
39
40            sum += a[j];
41
42            if (sum == 0)
43                r = max(r, j - i + 1);
44        }
45
46        cout << d << ' ' << w << ' ' << r << '\n';
47    }
48
49    return 0;
50 }

```

13.19 Generacion de Numeros con Digos Distintos - Division Digos Distintos

O: (cantidad de numeros con digitos distintos), generados una vez

Para cada N del caso de prueba, busca todos los pares (s1, s2) con $s1 / s2 = N$ donde s1 y s2 tienen

C++/CCPL/divisionDigitosDistintos.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const ll MAX_NUM = 9876543210LL;
7
8 vector<ll> nums;
9
10 // usado[d] = true si el digito d ya esta puesto en el numero que armamos.
11 bool usado[10];
12
13 // Arma numeros digito por digito, probando todas las opciones.
14 // numero = lo que llevamos armado hasta ahora
15 // cuantos = cuantos digitos tiene ese numero
16 // Cada llamada decide CUAL es el siguiente digito que se pega al final.
17 void armar(ll numero, int cuantos) {
18     // Si ya pusimos por lo menos un digito, lo que llevamos YA es un numero
19     // valido (todos sus digitos son distintos), asi que se guarda.
20     if (cuantos > 0) {
21         nums.push_back(numero);
22     }
23
24     // Con 10 digitos distintos ya se usaron todos, no hay mas que pegar.
25     if (cuantos == 10) {
26         return;
27     }
28
29     // Se prueba cada digito que todavia no hayamos usado.
30     for (int d = 0; d <= 9; d++) {
31         if (usado[d]) {
32             continue;
33         }
34
35         // El 0 no puede ir de primero (021 no es un numero de 3 digitos).
36         if (cuantos == 0 && d == 0) {
37             continue;
38         }
39
40         usado[d] = true; // se marca: ya lo usamos
41
42         armar(numero * 10 + d, cuantos + 1); // se pega al final y se sigue
43
44         usado[d] = false; // se libera para la otra rama
45     }
46 }
47
48 // Revisa si x tiene todos sus digitos distintos, sacandolos uno por uno.

```

13.19 Generación de Números con Dígitos Distintos - División Dígitos Distintos (cont.)

```

48 bool distinctDigits(ll x) {
49     int mask = 0;
50
51     while (x > 0) {
52         int d = x % 10;
53         x /= 10;
54
55         if (mask & (1 << d))
56             return false;
57
58         mask |= (1 << d);
59     }
60
61     return true;
62 }
63
64 int main() {
65     ios::sync_with_stdio(false);
66     cin.tie(nullptr);
67
68     nums.reserve(9000000); // el total es 8877690, sin realocar
69
70     armar(0, 0);
71
72     sort(nums.begin(), nums.end());
73
74     int T;
75     cin >> T;
76
77     while (T--) {
78         ll N;
79         cin >> N;
80
81         for (ll s2 : nums) {
82             // Si s2 * N se pasa del maximo, los siguientes tambien: corta.
83             if (s2 > MAX_NUM / N)
84                 break;
85
86             ll s1 = s2 * N;
87
88             if (!distinctDigits(s1))
89                 continue;
90
91             cout << s1 << " / " << s2 << " = " << N << '\n';
92         }
93
94         if (T)
95             cout << '\n';
96     }
97
98     return 0;
99 }

```

13.20 Greedy - Escalator

O: (N), greedy en una pasada

Una escalera doble tarda 10 segundos en cruzar a cualquier persona

C++/Training_Camp_2026/Day 4/C - Escalator.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     int n;
6     cin >> n;
7
8     long long busy_until = 0;
9     int dir = -1;
10    bool waiting = false;
11
12    for (int i = 0; i < n; i++){
13        long long t; int d;

```

13.20 Greedy - Escalator (cont.)

```

14        cin >> t >> d;
15
16        while (waiting && t >= busy_until){
17            dir = 1 - dir;
18            busy_until = busy_until + 10;
19            waiting = false;
20        }
21
22        if (busy_until <= t){
23            dir = d;
24            busy_until = t + 10;
25        } else {
26            if (d == dir){
27                busy_until = max(busy_until, t + 10);
28            } else {
29                waiting = true;
30            }
31        }
32    }
33
34    if (waiting){
35        busy_until = busy_until + 10;
36    }
37
38    cout << busy_until << "\n";
39    return 0;
40 }

```

13.21 Lectura de Entrada

O: (1) por token, es E/S

Uso: los seis patrones de lectura: EOF, centinela, T casos, bloques, linea, n
Hasta EOF, con centinela, con T casos, bloques separados por lineas en blanco, grids de caracteres

C++/Notebook/Lectura de Entrada.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 // 1) Hasta EOF: no hay T y el archivo termina sin aviso
6 void leerHastaEOF()
7 {
8     int a, b;
9     while (cin >> a >> b)
10     {
11         // procesar (a, b)
12     }
13 }
14
15 // 2) Hasta centinela (clasico "0 0" para terminar)
16 void leerHastaCentinela()
17 {
18     int a, b;
19     while (cin >> a >> b)
20     {
21         if (a == 0 && b == 0) break;
22         // procesar (a, b)
23     }
24 }
25
26 // 3) T al inicio y luego T casos
27 void leerTCasos()
28 {
29     int T;
30     if (cin >> T)
31     {
32         while (T--)
33         {
34             int n, m;
35             cin >> n >> m;
36             // procesar
37         }

```

13.21 Lectura de Entrada (cont.)

```

38     }
39 }
40
41 // 4) Cabecera por caso: cada caso empieza con "n m" y trae m lineas
42 void leerCabeceraPorCaso()
43 {
44     int n, m;
45     while (cin >> n >> m)
46     {
47         vector<pair<int,int>> edges;
48         edges.reserve(m);
49         for (int i = 0; i < m; i++)
50         {
51             int u, v;
52             cin >> u >> v;
53             edges.push_back({u, v});
54         }
55         // procesar caso
56     }
57 }
58
59 // 5) Bloques separados por lineas en blanco, sin saber cuantos hay
60 void leerBloques()
61 {
62     string line;
63     while (true)
64     {
65         // saltar lineas vacias entre bloques
66         while (getline(cin, line) && line.empty()) {}
67         if (!cin) break;
68
69         vector<string> bloque;
70         do
71         {
72             if (!line.empty()) bloque.push_back(line);
73         } while (getline(cin, line) && !line.empty());
74
75         // parsear cada linea del bloque con istringstream
76         for (auto &L : bloque)
77         {
78             istringstream iss(L);
79             int x;
80             while (iss >> x)
81             {
82                 // procesar x
83             }
84         }
85     }
86 }
87
88 // 6) Mezclar cin >> con getline: hay que consumir el '\n' colgante
89 void leerLineaConEspacios()
90 {
91     int n;
92     cin >> n;
93     string line;
94     getline(cin, line); // consume el salto de linea pendiente
95     getline(cin, line); // esta si es la linea util (puede traer espacios)
96 }
97
98 // 7) Leer N numeros aunque vengan repartidos en varias lineas
99 void leerNNumeros()
100 {
101     int n;
102     cin >> n;
103     vector<long long> a;
104     a.reserve(n);
105     while ((int)a.size() < n)
106     {
107         long long x;
108         if (!(cin >> x)) break;
109         a.push_back(x);

```

13.21 Lectura de Entrada (cont.)

```

110     }
111 }
112
113 // 8) Grid de caracteres R x C (getline es mas robusto que >> por fila)
114 void leerGrid()
115 {
116     int R, C;
117     cin >> R >> C;
118     string dummy;
119     getline(cin, dummy); // consume el '\n'
120     vector<string> g(R);
121     for (int i = 0; i < R; i++)
122     {
123         getline(cin, g[i]);
124     }
125 }
126
127 // 9) Leer linea por linea y tokenizar, saltando vacias y comentarios
128 void leerYTokenizar()
129 {
130     string line;
131     while (getline(cin, line))
132     {
133         if (line.empty() || line[0] == '#') continue;
134         istringstream iss(line);
135         int x;
136         while (iss >> x)
137         {
138             // procesar x
139         }
140     }
141 }
142
143 // 10) Tokens con separadores mixtos (comas, tabs, espacios)
144 void leerSeparadoresMixtos()
145 {
146     string line;
147     getline(cin, line);
148     for (char &c : line)
149     {
150         if (c == ',' || c == '\t') c = ' ';
151     }
152     istringstream iss(line);
153     vector<string> tok;
154     for (string t; iss >> t; ) tok.push_back(t);
155 }
156
157 // 11) Enteros gigantes: leerlos como string para no desbordar
158 void leerBigInt()
159 {
160     string P;
161     while (cin >> P)
162     {
163         // procesar P digito por digito (ver modString)
164     }
165 }
166
167 // 12) Consumir TODO el archivo como un solo string (parser propio)
168 void leerTodo()
169 {
170     string all((istreambuf_iterator<char>(cin)), istreambuf_iterator<char>());
171 }
172
173 // 13) Normalizar fin de linea de Windows (CRLF) y espacios finales
174 void rtrim(string &s)
175 {
176     while (!s.empty() && (s.back() == '\r' || s.back() == ' ' || s.back() == '\t'))
177     {
178         s.pop_back();
179     }
180 }
181

```

13.21 Lectura de Entrada (cont.)

```
82 int main()
83 {
84     ios_base::sync_with_stdio(false);
85     cin.tie(nullptr);
86     return 0;
87 }
```

13.22 Lights Out (Primera Fila Forzada) - Turnswitch

O: ($2^n * n^2$): se prueban las 2^n primeras filas y cada una propaga Lights Out en grilla $n \times n$: fijada la primera fila, todo el resto queda forzado

C++/ICPC/2024/F - Turnswitch.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9
10    while (cin >> n && n != 0) {
11
12        vector<string> a(n);
13
14        for (auto &row : a)
15            cin >> row;
16
17        int ans = 1e9;
18
19        // Probamos los dos objetivos:
20        // 1. Todas verticales '|'
21        // 2. Todas horizontales '-'
22        for (int target = 0; target < 2; target++) {
23
24            char wanted = (target == 0 ? '|' : '-');
25
26            // Probamos todas las formas de girar la primera fila
27            for (int mask = 0; mask < (1 << n); mask++) {
28
29                vector<string> b = a;
30                int cnt = 0;
31
32                // Funcion para girar una casilla
33                auto turn = [&](int r, int c) {
34                    cnt++;
35
36                    // Propia
37                    b[r][c] = (b[r][c] == '|') ? '-' : '|';
38
39                    // Arriba
40                    if (r > 0)
41                        b[r - 1][c] =
42                            (b[r - 1][c] == '|') ? '-' : '|';
43
44                    // Abajo
45                    if (r + 1 < n)
46                        b[r + 1][c] =
47                            (b[r + 1][c] == '|') ? '-' : '|';
48
49                    // Izquierda
50                    if (c > 0)
51                        b[r][c - 1] =
52                            (b[r][c - 1] == '|') ? '-' : '|';
53
54                    // Derecha
55                    if (c + 1 < n)
56                        b[r][c + 1] =
57                            (b[r][c + 1] == '|') ? '-' : '|';
58                };
59            }
60        }
61    }
```

13.22 Lights Out (Primera Fila Forzada) - Turnswitch (cont.)

```
62 // Decidimos que switches girar en la primera fila
63 for (int c = 0; c < n; c++) {
64     if (mask & (1 << c))
65         turn(0, c);
66 }
67
68 // Desde la segunda fila, los movimientos son obligatorios
69 for (int r = 1; r < n; r++) {
70     for (int c = 0; c < n; c++) {
71
72         // Si la casilla de arriba esta mal,
73         // debemos girar esta.
74         if (b[r - 1][c] != wanted)
75             turn(r, c);
76     }
77 }
78
79 // Comprobamos si la ultima fila quedo correcta
80 bool ok = true;
81
82 for (int c = 0; c < n; c++) {
83     if (b[n - 1][c] != wanted) {
84         ok = false;
85         break;
86     }
87 }
88
89 if (ok)
90     ans = min(ans, cnt);
91 }
92
93 cout << ans << '\n';
94 }
95
96 return 0;
97 }
```

13.23 Máquina de Estados por Objeto - Fraud

O: (n), una maquina de estados por objeto

NO ES MIO: codigo de Stiven Correa, del repo del equipo (carpeta RPC/)

RPC/2026-07/F - Fraud.py

```
1 from collections import defaultdict
2
3 n = int(input())
4
5 state = defaultdict(int)
6 ans = 'yes'
7
8 for _ in range(n):
9     inst, item = input().split()
10
11     if inst == 'pickup':
12         if state[item] != 0:
13             ans = 'no'
14             break
15         state[item] = 1
16
17     else:
18         if state[item] != 1:
19             ans = 'no'
20             break
21         state[item] = 2
22
23 if any(x != 2 for x in state.values()):
24     ans = 'no'
25
26 print(ans)
```


13.25 Python para Competencias (Enteros Gigantes y Atajos) (cont.)

```

35 # ----- Division: al reves que C++ -----
36 -7 // 2, -7 % 2          # -> (-4, 1)      C++ da -3 y -1
37 int(-7 / 2)              # -> -3          truncar hacia cero como C++
38
39 # ----- Fracciones exactas y decimales con precision -----
40 Fraction(1, 3) + Fraction(1, 6)          # -> Fraction(1, 2)
41 Fraction("0.75")                         # -> Fraction(3, 4)
42 getcontext().prec = 30                   # 30 cifras significativas
43 Decimal(2).sqrt()                        # -> Decimal('1.41421356237309504880168872421')
44 round(2.5), round(3.5)                   # -> (2, 4)      redondeo al PAR, no hacia arriba
45 Decimal("2.5").quantize(Decimal("1"), rounding=ROUND_HALF_UP) # -> Decimal('3')
46 f"{2 / 3:.4f}"                          # -> '0.6667'
47
48 # ----- Estructuras -----
49 h = [5, 1, 4]; heapq.heapify(h)           # heap de MINIMO; para maximo, meter -x
50 heapq.heappop(h)                         # -> 1
51 bisect_left([1, 3, 3, 5], 3), bisect_right([1, 3, 3, 5], 3) # -> (1, 3)
52 Counter("banana").most_common(1)         # -> [('a', 3)]
53 list(accumulate([1, 2, 3, 4]))            # -> [1, 3, 6, 10] prefijos
54 len(list(combinations(range(5), 2)))      # -> 10
55 sorted(["bb", "a", "ccc"], key=len)       # -> ['a', 'bb', 'ccc']
56 sorted([3, 1, 2], key=cmp_to_key(lambda x, y: y - x)) # -> [3, 2, 1]
57
58
59 # ----- Memoizacion y recursion profunda -----
60 @lru_cache(maxsize=None)                  # comodo, pero solo si la recursion es POCO profunda
61 def fib(n):
62     return n if n < 2 else fib(n - 1) + fib(n - 2)
63
64
65 fib(200)                                 # -> 280571172992510140037611932413038677189525
66
67 memo = {}                                # para recursion PROFUNDA: dict a mano
68
69
70 def dp(n):
71     if n in memo:
72         return memo[n]
73     r = 0 if n == 0 else 1 + dp(n - 1)
74     memo[n] = r
75     return r
76
77
78 # Niveles de recursion que aguanta, medido en Python 3.11 con setrecursionlimit alto:
79 #
80 # @lru_cache      sin hilo      hilo 128 MB      hilo 512 MB
81 # dict a mano    < 50.000      200.000      500.000
82 # 10^6           10^6          10^6          10^6
83 # lru_cache pasa por C y gasta la pila de C; desde 3.11 una llamada Python a Python no.
84 # En Python 3.10 o menos TODA recursion gasta pila de C: ahi el hilo es obligatorio.
85 def resolver():
86     pass                                  # aqui va la solucion
87
88
89 if __name__ == "__main__":
90     sys.setrecursionlimit(1 << 25)
91     import threading
92     threading.stack_size(1 << 27)        # 128 MB de pila
93     t = threading.Thread(target=resolver)
94     t.start()
95     t.join()

```

13.26 Simulacion con Parsing deCodigo - Impossible Primebox

O: (lineas de codigo * pasos simulados)

Hay L "primelocks", cada uno con la forma prlck i

C++/ICPC/2025/I - Impossible Primebox.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5

```

13.26 Simulacion con Parsing deCodigo - Impossible Primebox (cont.)

```

6 struct Lock {
7     ll A, B, P;
8     int siTrue, siFalse;    // -1 significa "end"
9 };
10
11 int main() {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14
15     int L;
16
17     while (cin >> L && L != 0) {
18
19         vector<Lock> lk(L);
20
21         for (int i = 0; i < L; i++) {
22
23             string s;
24
25             // "prlck i:" (dos palabras)
26             cin >> s >> s;
27
28             // "x = x" y despues los opcionales
29             cin >> s >> s >> s;
30
31             lk[i].A = 1;
32             lk[i].B = 0;
33
34             // Se leen operadores hasta toparse con el "if" de la linea 3.
35             while (cin >> s && s != "if") {
36                 ll v;
37                 cin >> v;
38
39                 if (s == "=") lk[i].A = v;
40                 else lk[i].B = v;
41             }
42
43             // "P div x jumpto <destino> else jumpto <destino>"
44             cin >> lk[i].P;
45             cin >> s >> s >> s;    // div x jumpto
46
47             cin >> s;
48             if (s == "end") {
49                 lk[i].siTrue = -1;
50             } else {
51                 cin >> lk[i].siTrue;    // venia "prlck j"
52             }
53
54             cin >> s >> s;    // else jumpto
55
56             cin >> s;
57             if (s == "end") {
58                 lk[i].siFalse = -1;
59             } else {
60                 cin >> lk[i].siFalse;
61             }
62         }
63
64         vector<int> K(L);
65         for (int i = 0; i < L; i++) cin >> K[i];
66
67         int pasosMax = 0;
68         for (int i = 0; i < L; i++) pasosMax += K[i];
69
70         // Con los primos DISTINTOS (a lo sumo 10 y cada uno <= 1000) se arma
71         // M = su producto. Si M cabe con margen para multiplicar por A, basta
72         // llevar UN residuo x mod M: como cada P divide a M, preguntar por
73         // r % P sigue valiendo, y el paso cuesta una multiplicacion en vez de
74         // una por candado. Si M no cabe, se cae al vector de residuos.
75         vector<ll> primos;
76         for (int i = 0; i < L; i++) primos.push_back(lk[i].P);
77         sort(primos.begin(), primos.end());

```

13.26 Simulacion con Parsing deCodigo - Impossible Primebox (cont.)

```

78 primos.erase(unique(primos.begin(), primos.end()), primos.end());
79
80 const ll LIMITE = (ll)9e18 / 1001; // deja espacio para r*A + B
81 ll M = 1;
82 bool unico = true;
83
84 for (ll p : primos) {
85     if (M > LIMITE / p) { unico = false; break; }
86     M *= p;
87 }
88
89 ll clave = -1;
90
91 vector<int> veces(L, r(L);
92
93 for (ll x0 = 0; x0 < 50000 && clave < 0; x0++) {
94
95     // x se dispara (se multiplica por A hasta 1000 en cada paso) y no
96     // cabe en ningun entero. Pero de x solo se pregunta si es divisible
97     // por P, asi que basta llevar x modulo esos primos:
98     // (x*A + B) mod P se calcula con (x mod P), sin conocer x.
99     for (int i = 0; i < L; i++) veces[i] = 0;
100
101     ll ru = x0 % M;
102
103     if (!unico)
104         for (int i = 0; i < L; i++) r[i] = (int)(x0 % lk[i].P);
105
106     int cur = 0;
107     int pasos = 0;
108     bool sirve = true;
109
110     while (cur != -1) {
111
112         if (++pasos > pasosMax + 1) { // ciclo o demasiadas vueltas
113             sirve = false;
114             break;
115         }
116
117         veces[cur]++;
118
119         if (veces[cur] > K[cur]) { // ya se paso de su cuota
120             sirve = false;
121             break;
122         }
123
124         bool divide;
125
126         if (unico) {
127             ru = (ru * lk[cur].A + lk[cur].B) % M;
128             divide = (ru % lk[cur].P == 0);
129         } else {
130             for (int i = 0; i < L; i++)
131                 r[i] = (int)((r[i] * lk[cur].A + lk[cur].B) % lk[i].P);
132             divide = (r[cur] == 0);
133         }
134
135         if (divide) cur = lk[cur].siTrue;
136         else cur = lk[cur].siFalse;
137     }
138
139     if (sirve && veces == K) clave = x0;
140 }
141
142 cout << clave << '\n';
143 }
144
145 return 0;
146 }

```

13.27 Simulation - Enigma Of The Jewelry Case

O: (N^2) por rotacion, con K mod 4 rotaciones efectivas

Deshacer K rotaciones de 90 grados de una matriz NxN creciente por filas y columnas

C++/Training_Camp_2026/Day 3/E - Enigma of the Jewelry Case.py

```

1 n = int(input())
2 matrix = []
3 for i in range(n):
4     matrix.append(list(map(int,input().split())))
5 pos = [matrix[0][0], matrix[0][n-1], matrix[n-1][n-1], matrix[n-1][0]]
6 me = min(pos)
7 print(pos.index(me))

```

13.28 Stress Test (Encontrar el Caso que da WA)

O: no aplica: es un procedimiento, 5 minutos de armar

Uso: gen.cpp (este archivo), brute.cpp, sol.cpp; correr el script de abajo en bash
Generador aleatorio + fuerza bruta + script que compara hasta hallar la diferencia

C++/Notebook/Stress Test.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 mt19937 rng;
6
7 int rnd(int a, int b) { return uniform_int_distribution<int>(a, b)(rng); }
8
9 string rndStr(int n, char desde, char hasta)
10 {
11     string s(n, desde);
12     for (char &c : s) c = (char)rnd(desde, hasta);
13     return s;
14 }
15
16 int main(int, char *argv[])
17 {
18     rng.seed(atoi(argv[1])); // misma semilla -> mismo caso
19
20     int n = rnd(1, 8); // CHICO: el caso que falla se lee a ojo
21     printf("%d\n", n);
22     for (int i = 0; i < n; i++) printf("%d ", rnd(1, 10));
23     printf("\n");
24
25     // Arbol al azar con n nodos (1-indexado): el padre de i es alguno anterior
26     // vector<int> perm(n); iota(perm.begin(), perm.end(), 1);
27     // shuffle(perm.begin(), perm.end(), rng); // para que 1 no sea siempre la raiz
28     // for (int i = 1; i < n; i++) printf("%d %d\n", perm[rnd(0, i - 1)], perm[i]);
29
30     // Permutacion al azar
31     // vector<int> p(n); iota(p.begin(), p.end(), 1); shuffle(p.begin(), p.end(), rng);
32
33     // Cadena al azar con alfabeto CHICO para que haya repeticiones
34     // printf("%s\n", rndStr(n, 'a', 'b').c_str());
35     return 0;
36 }
37
38 /* ===== stress.sh ===== correr: bash stress.sh
39
40 g++ -O2 -o sol sol.cpp && g++ -O2 -o brute brute.cpp && g++ -O2 -o gen gen.cpp || exit
41 for i in $(seq 1 2000); do
42     ./gen $i > in.txt
43     timeout 2 ./sol < in.txt > out1.txt
44     est=$?
45     if [ $est -eq 124 ]; then echo "TLE con semilla $i"; cat in.txt; exit; fi
46     if [ $est -ne 0 ]; then echo "RE con semilla $i"; cat in.txt; exit; fi
47     ./brute < in.txt > out2.txt
48     if ! diff -bq out1.txt out2.txt > /dev/null; then
49         echo "DIFERENCIA con semilla $i"; cat in.txt
50         echo "--- sol:"; cat out1.txt
51         echo "--- brute:"; cat out2.txt
52         exit

```

13.28 Stress Test (Encontrar el Caso que da WA) (cont.)

```

53     f1
54     done
55     echo "2000 casos iguales"
56
57     diff -b ignora espacios de mas al final de linea. Si la salida tiene decimales, comparar
58     con tolerancia en un checker propio en vez de diff. En Windows: correrlo en Git Bash.
59     */

```

13.29 Template Base

O: (1), es el esqueleto

Uso: punto de partida: fast IO, lectura de matriz, gcd/lcm, setprecision

Plantilla de arranque: fast IO, alias, constantes, macros y los utiles que se olvidan

C++/Notebook/Template Base.cpp

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  using ll = long long;
6
7  const int MOD = 1e9 + 7;
8  const ll LLINF = 0x3f3f3f3f3f3f3fLL;
9  const int INF = 0x3f3f3f3f;
10 const double EPS = 1e-9;
11 const double PI = acos(-1.0);
12
13 #define all(x) (x).begin(), (x).end()
14 #define forn(i, n) for(int i = 0; i < int(n); i++)
15 #define forr(i, a, b) for(int i = a; i <= b; i++)
16 #define ford(i, n) for(int i = int(n) - 1; i >= 0; i--)
17 #define endl '\n'
18
19 int main()
20 {
21     ios_base::sync_with_stdio(false);
22     cin.tie(nullptr);
23     cout.tie(nullptr);
24
25     int filas = 3, columnas = 4;
26     vector<vector<int>> matriz(filas, vector<int>(columnas, 0));
27
28     vector<int> v = {5, 1, 9, 3, 7};
29
30     // indice del maximo y del minimo (no el valor)
31     int idx_max = max_element(all(v)) - v.begin();
32     int idx_min = min_element(all(v)) - v.begin();
33     cout << idx_max << " " << idx_min << endl;
34
35     // borrar desde el indice 1 hasta el 3 (sin incluir el 3)
36     v.erase(v.begin() + 1, v.begin() + 3);
37
38     // imprimir con decimales fijos
39     double value = 1.0 / 3.0;
40     cout << fixed << setprecision(6) << value << endl;
41
42     // gcd ya viene en la STL (__gcd, o std::gcd/std::lcm si el juez tiene C++17).
43     // Para el lcm, dividir ANTES de multiplicar evita desbordar.
44     int a = 4, b = 6;
45     cout << __gcd(12, 18) << " " << (a / __gcd(a, b)) * b << endl;
46
47     return 0;
48 }

```

13.30 Umbral de Frecuencia - Lottery

O: (n log n) por ordenar la respuesta; el conteo es (n)

NO ES MIO: código de Stiven Correa, del repo del equipo (carpeta RPC/)

RPC/2026-07/L - Lottery.py

```

1  from collections import defaultdict
2  n = int(input())
3

```

13.30 Umbral de Frecuencia - Lottery (cont.)

```

4  dicc = defaultdict(int)
5  res = set()
6  total = 5*10*n
7  for _ in range(10*n):
8      nums = list(map(int, input().split()))
9      for i in nums:
10         dicc[i] += 1
11         if dicc[i] > 2*n:
12             res.add(i)
13 res = sorted(list(res))
14 if len(res) == 0:
15     print(-1)
16 else:
17     print(*res)

```

14 - Formulario

14.1 Algebra y Ecuaciones

O: (1) cada formula; es hoja de consulta, no algoritmo

Resolver e igualar ecuaciones: cuadratica con discriminante

C++/Formulario/Algebra y Ecuaciones.cpp

Ecuacion cuadratica $ax^2 + bx + c = 0$

$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$
 discriminante $D = b^2 - 4ac$
 $D > 0$: dos raices reales distintas
 $D = 0$: una raiz doble, $x = -b/(2a)$
 $D < 0$: ninguna raiz real

Vieta (verificado) : $x_1 + x_2 = -b/a$ $x_1 * x_2 = c/a$
Sirve para responder sobre las raices sin calcularlas.

Version estable, cuando b^2 es mucho mayor que $4ac$:

$q = -(b + \text{sign}(b) * \sqrt{D}) / 2$
 $x_1 = q / a$ $x_2 = c / q$

Si $a = 0$ no es cuadratica: es $b x + c = 0$, y si $b = 0$ tambien hay que tratarlo aparte. Ese caso borde tumba muchas soluciones.

Sistema de dos ecuaciones (Cramer, verificado)

$a_1 x + b_1 y = c_1$
 $a_2 x + b_2 y = c_2$
 $\det = a_1 b_2 - a_2 b_1$
 $\det \neq 0$: solucion unica
 $\det = 0$: o no hay solucion, o hay infinitas (misma recta)
 $x = (c_1 b_2 - c_2 b_1) / \det$ $y = (a_1 c_2 - a_2 c_1) / \det$
 Igualar dos funciones $y = f(x)$, $y = g(x)$ es resolver $f(x) - g(x) = 0$.

Productos notables

$(a + b)^2 = a^2 + 2ab + b^2$
 $(a - b)^2 = a^2 - 2ab + b^2$
 $(a + b)(a - b) = a^2 - b^2$
 $(a + b)^3 = a^3 + 3a^2 b + 3a b^2 + b^3$
 $a^3 + b^3 = (a + b)(a^2 - ab + b^2)$
 $a^3 - b^3 = (a - b)(a^2 + ab + b^2)$
 $a^n - b^n$ siempre es divisible por $(a - b)$
 binomio de Newton : $(a+b)^n = \sum_k C(n,k) a^{n-k} b^k$

Exponentes y logaritmos

$a^m * a^n = a^{(m+n)}$ $a^m / a^n = a^{(m-n)}$
 $(a^m)^n = a^{(m*n)}$ $(ab)^n = a^n b^n$
 $a^0 = 1$ $a^{(-n)} = 1 / a^n$
 $a^{(1/n)}$ = raiz n-esima de a
 $\log(x*y) = \log x + \log y$ $\log(x/y) = \log x - \log y$
 $\log(x^n) = n * \log x$ $\log_b(x) = \log(x) / \log(b)$
 En C++ $\log()$ es natural; existen $\log2()$ y $\log10()$ aparte.
 OJO: $\log2(n)$ puede dar 2.9999 por flotante. Si necesitas el exponente exacto, usa $31 - \text{__builtin_clz}(n)$ o un bucle.

Proporciones y porcentajes

regla de tres: si a corresponde a b, entonces c da $b*c/a$

14.1 Algebra y Ecuaciones (cont.)

aumentar p% : $x * (1 + p/100)$ disminuir : $x * (1 - p/100)$
 variacion porcentual = $(\text{nuevo} - \text{viejo}) / \text{viejo} * 100$
Subir p% y luego bajar p% NO devuelve al valor original.
 promedio simple = suma / n
 promedio ponderado = $\text{sum}(w_i * x_i) / \text{sum}(w_i)$
 media geometrica = $(\text{producto de los } x_i)^{1/n}$
se usa para tasas de crecimiento encadenadas
 media armonica = $n / \text{sum}(1/x_i)$
se usa para promediar velocidades sobre la misma distancia
 Siempre: armonica <= geometrica <= aritmetica

Valor absoluto e inecuaciones

$|x| < a$ equivale a $-a < x < a$
 $|x| > a$ equivale a $x < -a$ o $x > a$
 $|a + b| \leq |a| + |b|$ desigualdad triangular
Al multiplicar o dividir una inecuacion por un NEGATIVO, el sentido se voltea. Es el error clasico al despejar.

En C++

Cuadratica numericamente estable. Devuelve cuantas raices reales hay.

```
int cuadratica(double a, double b, double c, double& x1, double& x2) {
    double D = b*b - 4*a*c;
    if (D < 0) return 0;
    double q = -(b + copysign(sqrt(D), b)) / 2;
    x1 = q / a;
    x2 = (fabs(q) > 1e-300) ? c / q : x1;
    if (x1 > x2) swap(x1, x2);
    return D > 0 ? 2 : 1;
}
El copysign evita restar dos numeros casi iguales. Con a=1, b=1e8,
c=1 la formula directa da 0 en una raiz; esta da -1e-8.
Sistema 2x2 por Cramer. false = no hay solucion unica.
bool cramer(double a1, double b1, double c1,
             double a2, double b2, double c2, double& x, double& y) {
    double d = a1*b2 - a2*b1;
    if (fabs(d) < 1e-12) return false;
    x = (c1*b2 - c2*b1) / d;
    y = (a1*c2 - a2*c1) / d;
    return true;
}
```

14.2 Areas y Volúmenes

O: (1) cada formula; es hoja de consulta, no algoritmo

Areas de figuras planas y volúmenes de cuerpos, incluidas las que uno cree recordar y termina equivocando

C++/Formulario/Areas y Volúmenes.cpp

Figuras planas (A = area, P = perimetro)

cuadrado A = L^2 P = 4L diagonal = $L*\sqrt{2}$
 rectangulo A = $b*h$ P = 2(b+h) diagonal = $\sqrt{b^2+h^2}$
 rombo A = $d1*d2 / 2$ P = 4L
las diagonales se cortan perpendicularmente en el centro
 el lado sale de ellas: $L = \sqrt{(d1/2)^2 + (d2/2)^2}$
 tambien $A = L^2 * \sin(\text{angulo})$
 romboide A = $b*h$ P = 2(a+b)
 trapecio A = $(B + b) * h / 2$
 triangulo A = $b*h/2$ (Heron esta en la ficha Geometria)
 circulo A = πr^2 P = 2 πr
 corona circular A = $\pi (R^2 - r^2)$
 sector de angulo t en radianes
 A = $r^2 * t / 2$ arco = $r * t$
 elipse A = $\pi a b$
El perimetro no tiene formula cerrada elemental.
 Ramanujan, APROXIMADO (error ~1e-5 con a=5, b=2, medido):
 $P \sim \pi (3(a+b) - \sqrt{(3a+b)(a+3b)})$
 poligono regular de n lados de largo L (verificado)
 A = $n * L^2 / (4 * \tan(\pi/n))$
 apotema = $L / (2 * \tan(\pi/n))$
 angulo interior = $(n-2)*180/n$ suma total = $(n-2)*180$
 angulo central = $360/n$

Cuerpos (V = volumen, S = area total)

cubo V = a^3 S = $6a^2$
 diagonal = $a*\sqrt{3}$
 ortoedro V = $a*b*c$ S = 2(ab + ac + bc)
 diagonal = $\sqrt{a^2 + b^2 + c^2}$
 prisma V = (area base) * h
 cilindro V = $\pi r^2 h$ lateral = 2 $\pi r h$
 S = 2 $\pi r (r + h)$
 cono V = $\pi r^2 h / 3$ lateral = $\pi r g$
 $g = \sqrt{r^2 + h^2}$ S = $\pi r (r + g)$
 esfera V = $4/3 \pi r^3$ S = 4 πr^2
casquete esferico de altura h (verificado)
 V = $\pi h^2 (3r - h) / 3$
 piramide V = (area base) * h / 3
tetraedro regular de arista a (verificado)
 V = $a^3 / (6*\sqrt{2})$ S = $\sqrt{3} * a^2$
 toro, R al centro del tubo y r el radio del tubo (verificado)
 V = 2 $\pi^2 R r^2$ S = 4 $\pi^2 R r$

Reglas que se olvidan

Al escalar una figura por k: longitudes x k, areas x k^2 , volúmenes x k^3 . Sirve para problemas de semejanza.
*Pappus: al girar una figura plana alrededor de un eje externo, V = area de la figura * recorrido de su centroide.*
El cono y la piramide son un tercio del prisma de igual base y altura; la esfera es dos tercios del cilindro que la encierra.

14.3 Checklist de Contest

O: no aplica: es la lista para el momento en que algo sale mal

Que revisar cuando da WA, TLE o RE, antes de enviar, y como elegir que resolver

C++/Formulario/Checklist de Contest.cpp

Primeros 20 minutos

plantilla	uno la escribe y la compila; los otros leen problemas
cada problema leído	una linea en papel: que pide, y facil / medio / dificil
a los 20-30 minutos	mirar el marcador: lo que muchos ya resolvieron es facil
orden	de mas resueltos a menos, nunca en orden de letra

Durante el contest

teclado	nunca quieto: uno escribe, los otros piensan EN PAPEL
30-40 min sin avanzar	explicarlo en voz alta a un companero, o cambiar
falla y no se ve	imprimirlo, revisarlo en papel y soltar el PC
ultima hora	terminar el mas cercano al AC; no empezar uno dificil

Antes de enviar (2 minutos)

ejemplos	TODOS, comparados caracter por caracter
formato	YES/Yes, espacios, salto de linea final, decimales
varios casos	reiniciar globales, vectores y contadores en CADA caso
bordes	el caso mas chico, y uno del tamano maximo (tiempo)
overflow	long long si una suma o un producto pasa de 2^{10^9}
al enviar	sin cout de depuracion; problema y lenguaje correctos

Dio WA (en este orden)

1. enunciado	releer: formato, "al menos" o "exactamente", 0/1-indexado
2. overflow	int*int, suma de n valores de 10^9 , acumulador en int
3. casos borde	n = 1, todos iguales, ceros, negativos, valores maximos
4. varios casos	algo que no se reinicio entre un caso y el siguiente
5. tamanos	arreglo justo (off-by-one), variable sin inicializar
6. supuestos	grafo desconectado, aristas repetidas, lazos, desorden
7. decimales	comparar con EPS; imprimir con setprecision suficiente
8. nada de eso	STRESS TEST contra fuerza bruta (en Implementation)

Dio TLE

1. complejidad	contra el n maximo (hoja de GATILLOS)
2. entrada rapida	sync_with_stdio(false), cin.tie(nullptr), "\n" no endl
3. copias	vector por VALOR en una recursion: pasarlo con &
4. estructuras	map donde alcanza un vector, o sort + binaria
5. por caso	memset de un arreglo enorme en cada caso de prueba

14.3 Checklist de Contest (cont.)

6. bucle infinito un while que no termina (el stress test lo encuentra)
7. Python pasarlo a C++

Dio RE

1. indices fuera del arreglo: tamaño, off-by-one, negativo
2. division por cero, o modulo por cero
3. vacio top(), front(), back() o *begin() de algo vacio
4. pila arreglo grande dentro de una funcion: hacerlo global
5. recursion DFS de 10^5 - 10^6 niveles: hacerla iterativa

Atascados

- a mano 4 o 5 casos chicos resueltos en papel; buscar el patron
- fuerza bruta imprimir la respuesta de $n = 1..15$: la formula se ve
- la cota de n $n \leq 20$ bitmask, $n \leq 5000$ $O(n^2)$: hoja de GATILLOS
- al revés binaria sobre la respuesta, consultas al revés, complemento

14.4 Combinatoria

O: (k) nCr y nPr por el ciclo; (1) si se precaculan factoriales
 Conteos clasicos con sus primeros valores para identificar la sucesion en la salida de ejemplo

C++/Formulario/Combinatoria.cpp

Los cuatro conteos base

- $P(n, k) = n! / (n - k)!$ ordenado, sin repetir
- $C(n, k) = n! / (k!(n - k)!)$ sin orden, sin repetir
- n^k ordenado, con repeticion
- $C(n + k - 1, k)$ sin orden, con repeticion
- $n! / (a! b! c! \dots)$ permutaciones de un multiconjunto

Identidades de binomiales (sirven para colapsar sumatorias)

- $C(n, k) = C(n - 1, k - 1) + C(n - 1, k)$ Pascal
- $C(n, k) = C(n, n - k)$ simetria
- $\sum_k C(n, k) = 2^n$ fila completa
- $\sum_k (-1)^k C(n, k) = 0$ para $n > 1$
- $\sum_{i=k..n} C(i, k) = C(n + 1, k + 1)$ hockey stick
- $\sum_i C(m, i) C(n, k - i) = C(m + n, k)$ Vandermonde
- $k * C(n, k) = n * C(n - 1, k - 1)$ absorcion

Triangulo de Pascal

n=0	1																		
n=1	1	1																	
n=2	1	2	1																
n=3	1	3	3	1															
n=4	1	4	6	4	1														
n=5	1	5	10	10	5	1													
n=6	1	6	15	20	15	6	1												
n=7	1	7	21	35	35	21	7	1											
n=8	1	8	28	56	70	56	28	8	1										
n=9	1	9	36	84	126	126	84	36	9	1									
n=10	1	10	45	120	210	252	210	120	45	10	1								

Stars and bars: repartir n objetos iguales en k grupos

- permitiendo grupos vacios : $C(n + k - 1, k - 1)$
- todos con al menos uno : $C(n - 1, k - 1)$

Catalan $Cat(n) = C(2n, n) / (n + 1) = \sum_i Cat(i) * Cat(n - 1 - i)$

Parentesis balanceados, arboles binarios de n nodos,
 triangulaciones de un poligono de n+2 lados, caminos de Dyck.

n	0	1	2	3	4	5	6	7	8	9
Cat(n)	1	1	2	5	14	42	132	429	1430	4862
Cat(12)	208012									
Cat(13)	742900									
Cat(14)	2674440									

Desarreglos: permutaciones sin ningun punto fijo

- $!n = (n - 1) * (! (n - 1) + ! (n - 2)) \sim n! / e$
- | | | | | | | | |
|----|---|---|---|---|---|----|-----|
| n | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
| !n | 1 | 0 | 1 | 2 | 9 | 44 | 265 |

Bell: formas de particionar un conjunto de n elementos

n	0	1	2	3	4	5	6	7	8
B(n)	1	1	2	5	15	52	203	877	4140

Inclusion-exclusion

- $|A \cup B| = |A| + |B| - |A \cap B|$
- $|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$
- General: sumar los de tamaño impar, restar los de tamaño par.

En C++

- `ll nCr(ll n, ll k) { // 0(k), sin precomputar nada`
`if (k < 0 || k > n) return 0;`
`k = min(k, n - k);`
`ll r = 1;`
`for (ll i = 1; i <= k; i++) r = r * (n - i + 1) / i;`
`return r; // la division siempre es exacta`
`}`
- `ll nPr(ll n, ll k) { // 0(k), con orden`
`if (k < 0 || k > n) return 0;`
`ll r = 1;`
`for (ll i = 0; i < k; i++) // SIN el min(k, n-k): esa`
`r *= (n - i); // simetria es solo de C`
`return r; // crece rapido: P(60,10) cabe,`
`// P(60,11) ya se desborda`
`ll catalan(int n) { // 0(n), sin binomiales`
`ll c = 1;`
`for (int i = 0; i < n; i++) c = c * 2 * (2*i + 1) / (i + 2);`
`return c;`
`}`
- `ll desarreglos(int n) { // 0(n), sin memoria extra`
`ll a = 1, b = 0;`
`for (int i = 2; i <= n; i++) {`
`ll c = (i - 1) * (a + b); a = b; b = c;`
`}`
`return n == 0 ? 1 : b;`
`}`
- `ll bell(int n) { // triangulo de Bell, 0(n^2)`
`vector<ll> f = {1};`
`for (int i = 0; i < n; i++) {`
`vector<ll> g = {f.back()};`
`for (ll x : f) g.push_back(g.back() + x);`
`f = g;`
`}`
`return f[0];`
`}`

Si n es grande y hay muchas consultas, precomputa factoriales e inversos
 (esta en la ficha Binomial Coefficients) y cada nCr sale en $O(1)$.

14.5 Cotas y Limites

O: no aplica: es la tabla que se mira ANTES de escribir codigo
 La tabla que se mira ANTES de escribir codigo

C++/Formulario/Cotas y Limites.cpp

n maximo que aguanta cada complejidad (~10^8 operaciones, 1 seg)

- $O(n!)$ $n \leq 11$
- $O(2^n * n)$ $n \leq 20$
- $O(2^n)$ $n \leq 25$
- $O(n^3)$ $n \leq 450$
- $O(n^2 \log n)$ $n \leq 5000$
- $O(n^2)$ $n \leq 10^4$
- $O(n \sqrt{n})$ $n \leq 10^5$
- $O(n \log^2 n)$ $n \leq 10^5$
- $O(n \log n)$ $n \leq 10^6$
- $O(n)$ $n \leq 10^8$
- Si $n \leq 10^{18}$ la respuesta es formula cerrada, log o binaria.

Rangos de los enteros

- int hasta 2147483647 ($\sim 2.1 * 10^9$)
- unsigned int hasta 4294967295

14.5 Cotas y Limites (cont.)

```
long long          hasta 9223372036854775807
unsigned long hasta 18446744073709551615
__int128          hasta ~1.7*10^38 (no se imprime con cout)
~9.2*10^18 es el techo de long long: dos numeros de 10^9
multiplicados caben, tres no.
```

Precision de punto flotante

```
float   ~7 digitos significativos
double ~15-16 digitos (usar siempre este)
long double ~18-19 en GCC
Comparar con EPS: fabs(a-b) < 1e-9, nunca a == b
Si el problema pide enteros exactos, evitar sqrt y pow.
```

Modulos comunes

```
1000000007 (10^9+7) primo, el mas usado
998244353  primo, el de NTT (= 119*2^23 + 1)
1000000009 primo
Con 10^9+7 dos residuos multiplicados llegan a ~10^18:
cabe en long long, pero sumar tres productos sin reducir NO.
```

Tamanos de memoria (limite tipico 256 MB)

```
int[10^7]      40 MB      long long[10^7]  80 MB
int[10^8]      400 MB     (no cabe)
bool[10^8]     100 MB     bitset<10^8>     12.5 MB
vector<vector<int>> de 10^4 x 10^4 = 400 MB, no cabe
```

14.6 Datos que no se Deducen

O: no aplica: son datos duros que hay que saberse

Hechos sueltos que si no te los sabes no los inventas en media hora de competencia

C++/Formulario/Datos que no se Deducen.cpp

XOR de 1 hasta n (verificado)

```
n % 4 == 0 -> n
n % 4 == 1 -> 1
n % 4 == 2 -> n + 1
n % 4 == 3 -> 0
n      : 1 2 3 4 5 6 7 8 9 10 11 12
1^..^n : 1 3 0 4 1 7 0 8 1 11 0 12
Xor de un rango [l,r] = f(r) ^ f(l-1)
```

Reglas de divisibilidad

```
2 : ultimo digito par      5 : termina en 0 o 5
3 : suma de digitos div 3  9 : suma de digitos div 9
4 : los ultimos 2 digitos  8 : los ultimos 3 digitos
6 : divisible por 2 y por 3
11 : suma alternada de digitos divisible por 11
7 : no hay regla comoda; saca el modulo digito a digito (Horner)
```

Ultimo digito de a^b: el ciclo dura a lo sumo 4 (verificado)

```
2: 2 4 8 6      3: 3 9 7 1      7: 7 9 3 1      8: 8 4 2 6
4: 4 6          9: 9 1          5: 5           6: 6
0: 0           1: 1
Entonces a^b mod 10 depende de b mod 4, tratando b = 0 aparte.
```

Cantidad de digitos

```
digitos de n = floor(log10(n)) + 1 para n >= 1
Mas seguro sin flotantes: to_string(n).size()
digitos de n! = suma de log10(i) para i=1..n, redondeado arriba
```

Josephus: n en circulo, se elimina cada k-esima persona

```
k = 2 : escribe n = 2^m + L, la respuesta es 2L + 1
equivala a rotar a la izquierda el bit mas alto de n
k general (0-indexado):
J = 0; for (i = 2; i <= n; i++) J = (J + k) % i;
la respuesta 1-indexada es J + 1
```

Principio del palomar

n+1 objetos en n cajas -> alguna caja tiene 2 o mas
Entre n+1 numeros cualesquiera de 1..2n hay dos consecutivos
Con n+1 prefijos y n residuos, dos comparten residuo
-> siempre existe un subarreglo con suma divisible por n

Grafos: hechos que se olvidan

arbol de n nodos: exactamente n-1 aristas y ningun ciclo
grafo completo de n nodos: n(n-1)/2 aristas
suma de todos los grados = 2 * cantidad de aristas
la cantidad de nodos de grado impar siempre es PAR
camino euleriano existe si hay 0 o 2 nodos de grado impar
ciclo euleriano existe si todos los grados son pares
arboles etiquetados con n nodos: n^(n-2) (Cayley)
un grafo bipartito no tiene ciclos de longitud impar

Varios que aparecen disfrazados

ceros al final de n! = n/5 + n/25 + n/125 + ...
n es cuadrado perfecto <=> tiene una cantidad IMPAR de divisores
(de ahi el problema clasico de los casilleros que se abren)
suma de los primeros n impares = n^2
la mediana minimiza la suma de distancias absolutas
el promedio minimiza la suma de distancias al cuadrado
para repartir en dos grupos lo mas parejo posible: subset sum
(a+b) % m = ((a%m) + (b%m)) % m , igual con el producto,
pero NO con la division: ahi va el inverso modular

En C++

```
ll xorHasta(ll n) { // 1^2^...^n en O(1)
    switch (n % 4) {
        case 0: return n;
        case 1: return 1;
        case 2: return n + 1;
    }
    return 0;
}
xor del rango [l,r] = xorHasta(r) ^ xorHasta(l-1)
int josephus(int n, int k) { // O(n), 1-indexado
    int j = 0;
    for (int i = 2; i <= n; i++) j = (j + k) % i;
    return j + 1;
}
int josephus2(int n) { // caso k=2 en O(1)
    return 2 * (n - (1 << (31 - __builtin_clz(n)))) + 1;
}
int numDigitos(ll n) { // sin log10, sin error de flotante
    int d = 0;
    while (n) d++, n /= 10;
    return d ? d : 1;
}
```

14.7 Geometria

O: (1) las primitivas; (n) area y perimetro de poligono

Formulas de geometria que se usan sin pensar pero se olvidan bajo presion

C++/Formulario/Geometria.cpp

Triangulo

```
area = base * altura / 2
area = |cross(B-A, C-A)| / 2 con coordenadas
Heron: s=(a+b+c)/2, area = sqrt(s(s-a)(s-b)(s-c))
ley de cosenos : c^2 = a^2 + b^2 - 2ab*cos(C)
ley de senos : a/sin(A) = b/sin(B) = c/sin(C) = 2R
desigualdad triangular: a+b > c para que exista
```

Poligono simple (vertices en orden)

```
Shoelace: area = |sum (x_i * y_{i+1} - x_{i+1} * y_i)| / 2
El signo de la suma dice la orientacion: + antihorario.
Regular de n lados y lado L: area = n*L^2 / (4*tan(pi/n))
angulo interior = (n-2)*180/n grados
```

Pick: para poligono con vertices en enteros

area = I + B/2 - 1
I = puntos enteros interiores, B = puntos enteros en el borde
puntos enteros en el segmento (x1,y1)-(x2,y2):
 $\gcd(|x2-x1|, |y2-y1|) + 1$ (contando los dos extremos)

Circulo de radio r

perimetro = $2\pi r$ area = πr^2
sector de angulo t (rad) : area = $r^2 * t / 2$
segmento circular : area = $r^2 (t - \sin t) / 2$
cuerda de angulo t : largo = $2r \sin(t/2)$
PI = $\text{acos}(-1.0)$ 1 rad = $180/\pi$ grados

Producto cruz y punto (la base de todo)

cross(A,B) = ax*by - ay*bx
cross(A,B,C) = cross(B-A, C-A)
 > 0 giro antihorario, < 0 horario, = 0 colineales
dot(A,B) = ax*bx + ay*by
 > 0 angulo agudo, < 0 obtuso, = 0 perpendicular
Con enteros no uses sqrt: compara distancias al cuadrado.

Distancias

punto a recta por A,B : $|\text{cross}(B-A, P-A)| / |B-A|$
Manhattan $|dx|+|dy|$, Chebyshev $\max(|dx|,|dy|)$
Rotar 45 grados convierte Manhattan en Chebyshev:
(x,y) -> (x+y, x-y)

Euler para grafos planos y poliedros

V - E + F = 2
Con n rectas en posicion general el plano queda en
1 + n + C(n,2) regiones: 2, 4, 7, 11, 16, 22, 29, 37

14.8 Grafos, Esperanza y Calendario

O: no aplica: son resultados para usar directo

Teoremas de grafos que se usan sin demostrar, valor esperado, y día de la semana

C++/Formulario/Grafos, Esperanza y Calendario.cpp

Grafos: grados y estructura

suma de grados = 2 * aristas; la cantidad de nodos de grado impar es par
arbol n nodos, n-1 aristas, conexo y sin ciclos (2 de 3 bastan)
bosque componentes = n - aristas
bipartito si y solo si no tiene ciclos de largo impar (2-colorear BFS)
planar aristas $\leq 3n - 6$ (n ≥ 3); si tiene mas, no es planar
Euler (formula) V - E + F = 2 en un planar conexo (F cuenta la cara externa)

Grafos: caminos que pasan por todo

camino euleriano conexo y 0 o 2 nodos de grado impar (empieza en un impar)
circuito euleriano conexo y todos los grados pares
dirigido entrada = salida en todos (circuito); o uno con salida-entrada
= 1 (inicio) y otro con entrada-salida = 1 (fin) (camino)
Dirac (hamiltoniano) n ≥ 3 y todo grado $\geq n/2$ -> hay ciclo hamiltoniano

Grafos: matching y coberturas (ver Flujo Maximo y Matching)

Konig (bipartito) matching maximo = cubrimiento minimo de vertices
independiente maximo = V - matching maximo (bipartito)
cubrir con aristas minimo de aristas que tocan todo = V - matching (sin aislados)
Hall hay matching que cubre toda la izquierda si y solo si cada grupo S de la izquierda tiene $\geq |S|$ vecinos en total
caminos en un DAG minimo de caminos disjuntos que cubren todo = n - matching
Dilworth minimo de cadenas que cubren un orden = la anticadena maxima

Grafos: contar arboles

Cayley arboles con n nodos etiquetados = $n^{(n-2)}$ (K4 -> 16)
Kirchhoff arboles generadores de G = determinante de L sin una fila y su columna; L = grados en la diagonal menos la adyacencia (eliminacion gaussiana; mod p si el numero es enorme)
Erdos-Gallai $d_1 \geq \dots \geq d_n$ es secuencia de grados de un grafo simple

14.8 Grafos, Esperanza y Calendario (cont.)

si (suma par) y para todo k = 1..n:
 $d_1 + \dots + d_k \leq k(k-1) + \min(d(k+1), k) + \dots + \min(d_n, k)$

Valor esperado y probabilidad

linealidad E[X + Y] = E[X] + E[Y] SIEMPRE, aunque no sean independientes
truco del indicador E[cuantos cumplen] = suma de P(el i-esimo cumple)
producto E[X * Y] = E[X] * E[Y] solo si son independientes
geometrica intentos hasta el primer exito con prob p: E = 1/p
coleccionista E[tiradas hasta ver los n valores] = n * (1 + 1/2 + ... + 1/n)
(dado de 6 caras: 14.7 tiradas)
union P(A o B) = P(A) + P(B) - P(A y B)
Bayes P(A | B) = P(B | A) * P(A) / P(B)
DP de esperanza E[estado] = costo + suma de p(siguiente) * E[siguiente];
si un estado vuelve a si mismo: E = c + p*E -> despejar E
modulo p respuesta P/Q se imprime como P * inverso(Q) mod p

Dia de la semana (Sakamoto, calendario gregoriano; 0 = domingo ... 6 = sabado)

```
int diaSemana(int y, int m, int d) {
    static int t[] = {0, 3, 2, 5, 0, 3, 5, 1, 4, 6, 2, 4};
    if (m < 3) y--;
    return (y + y / 4 - y / 100 + y / 400 + t[m - 1] + d) % 7;
}
diaSemana(2026, 10, 3) = 6 -> sabado    diaSemana(2000, 1, 1) = 6 -> sabado
```

14.9 Probabilidad Geometrica y Areas Raras

O: (1) cada formula; es hoja de consulta

Para los problemas que en realidad son matematica pura y lo dificil no es programarlos sino plantear el...

C++/Formulario/Probabilidad Geometrica.cpp

La idea base

P(evento) = medida favorable / medida total
En 1D la medida es largo, en 2D area, en 3D volumen.
Si el punto se elige UNIFORME, no hay nada mas que calcular areas. El problema "de programacion" es en realidad geometria.

Valor esperado

E[X] = suma de (valor * probabilidad de ese valor)
LINEALIDAD: E[A + B] = E[A] + E[B] SIEMPRE, aunque A y B esten correlacionados. Es lo que permite sumar la contribucion de cada objeto por separado en vez de analizar el conjunto.
Ejemplo tipico: cada circulo i tiene valor v_i y cubre un area a_i del jardin de W x H, entonces
respuesta = suma de v_i * (a_i / (W*H))
Si los circulos se traslapan NO importa: la linealidad sigue valiendo, cada uno aporta su parte por separado.

Interseccion de DOS circulos (area de la lente) (verificado)

d = distancia entre centros
si d $\geq r_1 + r_2$ -> 0, no se tocan
si d $\leq |r_1 - r_2|$ -> $\pi * \min(r_1, r_2)^2$, uno dentro del otro
si no:
 a1 = $2 * \text{acos}((d^2 + r_1^2 - r_2^2) / (2 * d * r_1))$
 a2 = $2 * \text{acos}((d^2 + r_2^2 - r_1^2) / (2 * d * r_2))$
 A = $0.5 * r_1^2 * a1 - \sin(a1) + 0.5 * r_2^2 * a2 - \sin(a2)$
O sea: la lente son dos segmentos circulares pegados.
union de los dos = $\pi r_1^2 + \pi r_2^2 - A$

Segmento y sector (los ladrillos de todo lo anterior)

sector de angulo t : A = $r^2 * t / 2$
segmento (lo que queda entre la cuerda y el arco):
A = $r^2 * (t - \sin t) / 2$
cuerda del angulo t : largo = $2 r \sin(t/2)$

Circulo contra rectangulo o poligono

No hay formula cerrada corta. Se recorre el borde del poligono sumando areas con signo, tratando cada arista aparte segun si el tramo cae dentro o fuera del circulo.
Esta implementado en la seccion Geometry, ficha

14.9 Probabilidad Geometrica y Areas Raras (cont.)

"Area Circulo-Poligono". Es el metodo del problema del jardin.

Que una FIGURA QUEPA: traducir "cabe" a desigualdades

Cuando piden la probabilidad de que algo quepa dentro de otra cosa, casi nunca se integra a lo bruto: conviene escribir el "cabe" como desigualdades sobre las coordenadas, que suelen simplificarse muchísimo.

LA HERRAMIENTA CLAVE (arco capaz): el circulo que tiene al segmento AB como DIAMETRO cabe dentro de un convexo K si y solo si todo punto de la frontera de K ve a AB con angulo de a lo sumo 90 grados. Es porque ese circulo es justo donde el angulo vale 90, y por fuera de el es menor.

"Angulo <= 90 desde Q" se escribe sin trigonometria:

$$(A - Q) \cdot (B - Q) \geq 0$$

y si Q recorre un lado recto, se minimiza ese producto punto.

Ejemplo, para el lado x = 0 con Q = (0,q):

$$x1 \cdot x2 + (y1 - q)(y2 - q) \geq 0 \text{ para todo } q$$

$$\text{el minimo cae en } q = (y1 + y2)/2 \text{ y vale } x1 \cdot x2 - (y1 - y2)^2/4$$

$$\text{o sea la condicion es } 4 \cdot x1 \cdot x2 \geq (y1 - y2)^2$$

En un rectangulo L x W salen cuatro asi, sin una sola raiz:

$$4 \cdot x1 \cdot x2 \geq (y1 - y2)^2 \quad 4 \cdot (W - x1) \cdot (W - x2) \geq (y1 - y2)^2$$

$$4 \cdot y1 \cdot y2 \geq (x1 - x2)^2 \quad 4 \cdot (L - y1) \cdot (L - y2) \geq (x1 - x2)^2$$

(verificado contra la condicion directa de centro y radio en 3000000 de pruebas, sin una sola diferencia)

Y como las desigualdades son homogeneas de grado 2, escalar el rectangulo no cambia la probabilidad: solo importa la PROPORCION entre los lados. Para dos piedras uniformes en L x W, con

$$t = \min(L, W) / \max(L, W), \text{ la probabilidad de que el circulo quepa es}$$

$$P = (\pi/6) \cdot t \cdot (2 - t)$$

(formula verificada numericamente en 8 proporciones, no demostrada; el problema es "I - Omens" de ICPC 2024, esta en este cuaderno)

Lo transferible es el metodo, no la formula: primero traducir "cabe" a desigualdades, despues mirar si colapsan.

Uniones de varias figuras

$$|A \cup B| = |A| + |B| - |A \cap B|$$

Con tres o mas, inclusion-exclusion (2^n terminos): solo sirve si n es chico. Si no, integracion por barrido o Monte Carlo.

Para "cuanta area cubre al menos un circulo" con muchos circulos, lo estandar es barrido angular, no inclusion-exclusion.

Clasicos que se resuelven dibujando el espacio de casos

Cita: dos personas llegan uniformemente en [0,T] y se esperan t minutos. El espacio es el cuadrado T x T, y se encuentran en la franja |x - y| <= t. (verificado)

$$P = 1 - ((T - t)/T)^2$$

Palito partido en tres pedazos al azar: forman triangulo con probabilidad 1/4. (verificado)

Aguja de Buffon: aguja de largo L sobre lineas separadas d, con L <= d, cruza una linea con $P = 2L / (\pi \cdot d)$ (verificado)

Como atacar uno de estos en competencia

1. Identifica que se elige al azar y con que distribucion.
2. Dibuja el espacio completo de posibilidades y su medida.
3. Sombrea la region favorable y calcula SU medida.
4. Si hay varios objetos con valor, usa linealidad y suma valor por probabilidad, uno por uno.
5. Si el area no sale cerrada, piensa en barrido o en integrar. Antes de entregar, comprueba con Monte Carlo: es cinco lineas y te dice si la formula que planteaste esta bien.

En C++

Area de la interseccion de dos circulos (la lente).

```
double arealente(double r1, double r2, double d) {
    if (d >= r1 + r2) return 0; // separados
    if (d <= fabs(r1 - r2)) // uno dentro del otro
        return PI * min(r1, r2) * min(r1, r2);
    double a1 = 2 * acos((d*d + r1*r1 - r2*r2) / (2*d*r1));
    double a2 = 2 * acos((d*d + r2*r2 - r1*r1) / (2*d*r2));
```

14.9 Probabilidad Geometrica y Areas Raras (cont.)

```
return 0.5*r1*r1*(a1 - sin(a1)) + 0.5*r2*r2*(a2 - sin(a2));
}
```

union de los dos = PI*r1*r1 + PI*r2*r2 - arealente(r1, r2, d)

Los dos primeros if no son adorno: sin ellos, acos recibe un valor

fuera de [-1,1] y devuelve NaN.

14.10 Rectas y Conicas

O: (1) cada formula, incluido el corte por Cramer

Formulario: la recta en sus cuatro formas, paralelas, perpendiculares, corte y conicas

C++/Formulario/Rectas y Conicas.cpp

Pendiente por dos puntos

m = (y2 - y1) / (x2 - x1) indefinida si x1 == x2 (vertical)
 m > 0 sube, m < 0 baja, m = 0 horizontal
 angulo con el eje x : theta = atan2(y2-y1, x2-x1)

Formas de la recta

pendiente-intercepto : y = m x + b con b = y1 - m*x1
 punto-pendiente : y - y1 = m (x - x1)
 general : A x + B y + C = 0
 desde dos puntos: A = y2-y1, B = -(x2-x1),
 C = -(A*x1 + B*y1)
 pendiente = -A/B ; es vertical cuando B = 0
 simetrica : x/p + y/q = 1 (corta los ejes en p y q)

Dos rectas

paralelas : m1 == m2 o A1*B2 - A2*B1 == 0
 perpendiculares : m1 * m2 == -1 o A1*A2 + B1*B2 == 0
 angulo entre ellas : tan(t) = |(m2 - m1) / (1 + m1*m2)|

Con datos enteros compara pendientes cruzando fracciones y

evita dividir: dy1*dx2 == dy2*dx1

Interseccion: igualar las dos ecuaciones (Cramer, verificado)

A1 x + B1 y = C1
 A2 x + B2 y = C2
 det = A1*B2 - A2*B1
 si det == 0 -> paralelas (o la misma recta, sin punto unico)
 x = (C1*B2 - C2*B1) / det
 y = (A1*C2 - A2*C1) / det
 Para cortar y = f(x) con y = g(x), resuelve f(x) - g(x) = 0.

Distancias y punto medio

|P1P2| = sqrt((x2-x1)^2 + (y2-y1)^2)
 punto medio = ((x1+x2)/2, (y1+y2)/2)
 punto a recta Ax+By+C=0 : |A*px + B*py + C| / sqrt(A^2 + B^2)
 entre dos paralelas : |C1 - C2| / sqrt(A^2 + B^2)
 El SIGNO de A*px+B*py+C dice de que lado del corte esta el punto.

Circunferencia

(x - h)^2 + (y - k)^2 = r^2 centro (h,k), radio r
 general : x^2 + y^2 + Dx + Ey + F = 0
 centro (-D/2, -E/2)
 r = sqrt(D^2/4 + E^2/4 - F)

Dos circunferencias, con d la distancia entre centros:

se cortan en 2 puntos si |r1-r2| < d < r1+r2
 tangentes exteriores si d == r1+r2
 tangentes interiores si d == |r1-r2|
 una dentro de la otra sin tocarse si d < |r1-r2|

Parabola

y = a x^2 + b x + c
 vertice : x = -b/(2a) , y = c - b^2/(4a)
 abre hacia arriba si a > 0, hacia abajo si a < 0
 canonica : y - k = a (x - h)^2 con vertice (h,k)

Elipse e hiperbola centradas en (h,k)

elipse : (x-h)^2/a^2 + (y-k)^2/b^2 = 1

14.10 Rectas y Conicas (cont.)

a y b son los semiejes; si a = b es una circunferencia
 hipérbola : $(x-h)^2/a^2 - (y-k)^2/b^2 = 1$
 asíntotas : $y - k = \pm(b/a)(x - h)$

En C++

```
double distPuntoRecta(double A, double B, double C,
                     double px, double py) {
    return fabs(A*px + B*py + C) / hypot(A, B);
}
bool corte(double A1, double B1, double C1,
           double A2, double B2, double C2, double& x, double& y) {
    double d = A1*B2 - A2*B1;
    if (fabs(d) < 1e-12) return false;           // paralelas
    x = (C1*B2 - C2*B1) / d;
    y = (A1*C2 - A2*C1) / d;
    return true;
}
```

Con coordenadas enteras, para saber si tres puntos son colineales NO
 calcules pendientes: usa el producto cruz, que no divide ni pierde
 precision. $(x_2-x_1)(y_3-y_1) - (y_2-y_1)(x_3-x_1) == 0$

14.11 Secuencias Notables

O: (1) por la formula cerrada de cada sucesion

Las sucesiones que hay que reconocer de vista en un caso de ejemplo

C++/Formulario/Secuencias Notables.cpp

Fibonacci F(0)=0, F(1)=1, F(n)=F(n-1)+F(n-2)

```
n      0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
F(n)   0  1  1  2  3  5  8 13 21 34 55 89 144 233 377 610
F(20)=6765 F(30)=832040 F(40)=102334155
F(92) = 7540113804746346429 es el ULTIMO que cabe en long long;
F(93) = 12200160415121876738 ya se pasa (el techo es 9.22*10^18).
Identidades:
F(m+n) = F(m)F(n+1) + F(m-1)F(n)      fast doubling
sum_{i<=n} F(i) = F(n+2) - 1
F(n-1)F(n+1) - F(n)^2 = (-1)^n         Cassini
gcd(F(m),F(n)) = F(gcd(m,n))
```

Lucas L(0)=2, L(1)=1, misma recurrencia

```
n      0  1  2  3  4  5  6  7  8  9 10 11 12 13
L(n)   2  1  3  4  7 11 18 29 47 76 123 199 322 521
L(n) = F(n-1) + F(n+1)
```

Numeros figurados

```
n      1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
triangular 1  3  6 10 15 21 28 36 45 55 66 78 91 105 120
cuadrado   1  4  9 16 25 36 49 64 81 100 121 144 169 196 225
pentagonal 1  5 12 22 35 51 70 92 117 145 176 210 247 287 330
hexagonal  1  6 15 28 45 66 91 120 153 190 231 276 325 378 435
triangular T(n)=n(n+1)/2, T(n)+T(n-1) = n^2
```

Factoriales (20! es el ultimo que cabe en long long)

```
n      0  1  2  3  4  5  6  7  8  9
n!     1  1  2  6  24 120 720 5040 40320 362880
10! = 3628800      13! = 6227020800 (ya se pasa de int)
11! = 39916800     15! = 1307674368000
12! = 479001600    20! = 2432902008176640000
21! desborda long long. Si aparece n! con n > 20, es modular.
```

Potencias de 2

```
k      0  1  2  3  4  5  6  7  8  9 10
2^k    1  2  4  8 16 32 64 128 256 512 1024
2^20 = 1048576 (~10^6)
2^30 = 1073741824 (~10^9, limite de int)
2^31 = 2147483648 2^32 = 4294967296
2^62 = 4611686018427387904 (long long aguanta hasta 2^63-1)
```

Numeros perfectos (iguales a la suma de sus divisores propios)

6, 28, 496, 8128, 33550336
 Todos los pares son $2^p(2^{p-1}-1)$ con 2^p-1 primo (Mersenne).

En C++

```
Fibonacci por fast doubling: O(log n), aguanta n = 10^18.
pair<ll,ll> fib(ll n) {
    // devuelve {F(n), F(n+1)} mod M
    if (!n) return {0, 1};
    pair<ll,ll> p = fib(n >> 1);
    ll a = p.first, b = p.second;
    ll c = a * ((2*b % M - a + M) % M) % M;
    ll d = (a*a + b*b) % M;
    return {n & 1 ? make_pair(d, (c+d) % M) : make_pair(c, d);
}
Los figurados son O(1) directo:
ll triangular(ll n) { return n*(n+1)/2; }
ll pentagonal(ll n) { return n*(3*n-1)/2; }
ll hexagonal(ll n) { return n*(2*n-1); }
```

Para factoriales, tabla hasta 20 y listo: mas alla no cabe sin modulo.

14.12 STL Indispensable

O: depende del contenedor: (log n) en map y set, (1) en los hash

Las funciones de la libreria estandar que la gente reimplementa a mano por no saber que existen

C++/Formulario/STL Indispensable.cpp

Ordenar y buscar (el vector debe estar ordenado para los bound)

```
sort(all(v))                O(n log n)
stable_sort(all(v))          conserva el orden de los iguales
sort(all(v), greater<int>()) descendente
lower_bound(all(v), x)       primer elemento >= x
upper_bound(all(v), x)       primer elemento > x
cuantos x hay = upper_bound - lower_bound
posicion = lower_bound(all(v), x) - v.begin()
binary_search(all(v), x) devuelve solo true/false
nth_element(v.begin(), v.begin()+k, v.end())
deja el k-esimo en su sitio en O(n) promedio, sin ordenar todo
partial_sort(v.begin(), v.begin()+k, v.end()) solo los k primeros
```

Permutaciones

```
next_permutation(all(v)) siguiente en orden lexicografico
devuelve false cuando llega a la ultima y reinicia
ORDENA PRIMERO si quieres recorrer las n! (verificado)
prev_permutation(all(v))
```

Lo que no hay que reimplementar

```
__gcd(a,b)                  en C++17 tambien gcd(a,b) y lcm(a,b)
iota(all(v), 0)             llena con 0,1,2,3,...
accumulate(all(v), 0LL) suma (usa 0LL para no desbordar)
max_element / min_element devuelven ITERADOR: resta v.begin()
count(all(v), x)             find(all(v), x) reverse(all(v))
rotate(v.begin(), v.begin()+k, v.end()) rota a la izquierda
fill(all(v), x)              v.assign(n, x) swap(a, b)
minmax_element(all(v)) devuelve los dos de una
is_sorted(all(v))            next(it) prev(it)
```

Conversiones

```
to_string(x) stoi(s) stoll(s) stod(s)
s.substr(i, LARGO) <- cantidad, NO posicion final
s.find(t) devuelve string::npos si no aparece
getline(cin, s) lee la linea entera con espacios
stringstream ss(linea); while (ss >> token) ... partir por espacio
```

Estructuras y cuando usarlas

```
vector<int> v(n, 0)          vector<vector<int>>> g(n)
set / multiset ordenado, O(log n), tiene .lower_bound propio
unordered_set O(1) promedio, sin orden
map / unordered_map igual pero clave -> valor
deque          push_front y push_back en O(1)
```

14.12 STL Indispensable (cont.)

priority_queue max-heap; greater<> para convertirlo en min-heap
 bitset<N> operaciones de 64 bits por vez, 8x menos memoria
 array<int,N> tamaño fijo, sin costo de heap

Detalles que sirven

v.reserve(n) evita realocar si sabes el tamaño de antemano
 emplace_back construye en el sitio, evita una copia
 structured bindings: for (auto &[k, val] : mapa) (C++17)
 lambda: [](int a, int b){ return a > b; }
 capturar por referencia para comparadores con datos externos:
 sort(all(idx), [&](int i, int j){ return v[i] < v[j]; });

14.13 Sumas y Series

O: (1) cada suma cerrada, contra (n) de sumar a mano

Formulario: sumas cerradas con sus primeros valores, para reconocer la sucesión

C++/Formulario/Sumas y Series.cpp

Gauss: suma de los primeros n enteros

1 + 2 + ... + n = n(n+1)/2
 n 1 2 3 4 5 6 7 8 9 10 11 12
 S(n) 1 3 6 10 15 21 28 36 45 55 66 78

Suma de cuadrados

1^2 + 2^2 + ... + n^2 = n(n+1)(2n+1)/6
 n 1 2 3 4 5 6 7 8 9 10 11 12
 S(n) 1 5 14 30 55 91 140 204 285 385 506 650

Suma de cubos = el cuadrado de la suma de Gauss

1^3 + 2^3 + ... + n^3 = [n(n+1)/2]^2
 n 1 2 3 4 5 6 7 8 9 10 11 12
 S(n) 1 9 36 100 225 441 784 1296 2025 3025 4356 6084

Impares dan cuadrados perfectos, pares dan n(n+1)

1+3+5+...+(2n-1) = n^2 2+4+...+2n = n(n+1)
 n 1 2 3 4 5 6 7 8 9 10 11 12
 impares 1 4 9 16 25 36 49 64 81 100 121 144
 pares 2 6 12 20 30 42 56 72 90 110 132 156

Progresión aritmética (a, a+d, a+2d, ...)

término k : a + (k-1)d
 suma de n : n(2a + (n-1)d)/2 = n(primeros + último)/2

Progresión geométrica (a, ar, ar^2, ...)

término k : a*r^(k-1)
 suma de n : a(r^n - 1)/(r - 1) si r != 1
 suma inf : a/(1 - r) si |r| < 1
 En modular /(r-1) es inverso modular; si r = 1 mod p no existe.

Potencias de 2: la suma es la siguiente menos 1

1 + 2 + 4 + ... + 2^n = 2^(n+1) - 1
 n 0 1 2 3 4 5 6 7 8 9 10 11 12
 2^n 1 2 4 8 16 32 64 128 256 512 1024 2048 4096
 suma 1 3 7 15 31 63 127 255 511 1023 2047 4095 8191

Serie armónica H(n) = 1 + 1/2 + ... + 1/n ~ ln(n) + 0.5772

H(10) ~ 2.93 H(100) ~ 5.19 H(10^6) ~ 14.39
 Aparece al contar divisores: sum_{d=1..n} n/d ~ n*ln(n).

Conteos sobre un arreglo de n elementos

pares i<j : n(n-1)/2
 pares ordenados i!=j : n(n-1)
 subarreglos contiguos : n(n+1)/2
 subconjuntos : 2^n no vacíos: 2^n - 1

En C++ (todo O(1); dividir ANTES de multiplicar evita el overflow)

```
ll gauss(ll n) { // 1+2+...+n
    return n % 2 ? (n+1)/2*n : n/2*(n+1);
}
ll sumaCuadrados(ll n) { // 1^2+...+n^2
    ll a = n, b = n+1, c = 2*n+1;
    if (a % 2 == 0) a /= 2; else b /= 2;
    if (a % 3 == 0) a /= 3;
    else if (b % 3 == 0) b /= 3; else c /= 3;
    return a * b * c;
}
ll sumaCubos(ll n) { ll g = gauss(n); return g * g; }
Con n = 10^9, gauss(n) = 500000000500000000 y cabe en long long;
pero n*(n+1) sin dividir primero YA se desborda.
Suma geometrica modular, O(log n):
ll sumaGeoMod(ll a, ll r, ll n) { // a(r^n-1)/(r-1) mod M
    if (r % M == 1) return a % M * (n % M) % M;
    ll num = (modpow(r, n, M) - 1 + M) % M;
    return a % M * num % M * modpow((r-1) % M, M-2, M) % M;
}
```

14.14 Tabla ASCII

O: no aplica: es la tabla de códigos

Tabla ASCII de los caracteres imprimibles (32 a 126) en cuatro columnas

C++/Formulario/Tabla ASCII.cpp

dec chr	dec chr	dec chr	dec chr
32 SP	56 8	80 P	104 h
33 !	57 9	81 Q	105 i
34 "	58 :	82 R	106 j
35 #	59 ;	83 S	107 k
36 \$	60 <	84 T	108 l
37 %	61 =	85 U	109 m
38 &	62 >	86 V	110 n
39 '	63 ?	87 W	111 o
40 (	64 @	88 X	112 p
41)	65 A	89 Y	113 q
42 *	66 B	90 Z	114 r
43 +	67 C	91 [	115 s
44 ,	68 D	92 \	116 t
45 -	69 E	93]	117 u
46 .	70 F	94 ^	118 v
47 /	71 G	95 _	119 w
48 0	72 H	96 `	120 x
49 1	73 I	97 a	121 y
50 2	74 J	98 b	122 z
51 3	75 K	99 c	123 {
52 4	76 L	100 d	124
53 5	77 M	101 e	125 }
54 6	78 N	102 f	126 ~
55 7	79 O	103 g	

Lo que de verdad se usa

'0' = 48 '9' = 57 digito: c - '0'
 'A' = 65 'Z' = 90
 'a' = 97 'z' = 122 indice: c - 'a' (0..25)
 'a' - 'A' = 32 -> minuscula: c | 32, mayuscula: c & ~32
 ' ' = 32 '\n' = 10 '\0' = 0 '\t' = 9
 Cambiar de caso con xor: c ^ 32

Cheques de <cctype>

isalpha isdigit isalnum isupper islower isspace ispunct
 toupper tolower
 Rapido sin librería: (c>='0' && c<='9')

Control mas comunes (no imprimibles)

0 NUL 9 TAB 10 LF (\n) 13 CR (\r) 27 ESC
 Archivos de Windows traen CR+LF: si lees con getline puede
 quedar un '\r' al final de la línea. Quitarlo antes de comparar.

14.15 Teoría de Números

O: (n log log n) la criba, (log n) el gcd y la exponenciación

Divisores, phi, gcd, aritmética modular y CRT, con las tablas hasta 20 para revisar a mano un caso chico

C++/Formulario/Teoría de Números.cpp

Con la factorización $n = p_1^{a_1} * p_2^{a_2} * \dots$

cantidad de divisores $d(n) = (a_1+1)(a_2+1)\dots$
 suma de divisores $\sigma(n) = \prod (p^{a_i+1}-1)/(p-1)$
 función de Euler $\phi(n) = n * \prod (1 - 1/p)$
 Ejemplo $n=12=2^2*3$: $d=3*2=6$, $\sigma=7*4=28$, $\phi=12*(1/2)(2/3)=4$

Tablas hasta 20

n	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
d(n)	1	2	2	3	2	4	2	4	3	4	2	6	2	4	4	5	2	6	2	6
sig(n)	1	3	4	7	6	12	8	15	13	18	12	28	14	24	24	31	18	39	20	42
phi(n)	1	1	2	2	4	2	6	4	6	4	10	4	12	6	8	8	16	6	18	8

gcd y lcm

$\gcd(a,b) * \text{lcm}(a,b) = a*b$ (dividir antes de multiplicar)
 $\gcd(a,b) = \gcd(b, a \bmod b)$ Euclides
 $\gcd(F(m), F(n)) = F(\gcd(m,n))$ Fibonacci
 a y b coprimos $\Leftrightarrow \gcd(a,b) = 1$

Modular

Fermat : $a^{p-1} \equiv 1 \pmod p$ si p primo y p no divide a
 inverso : $a^{p-2} \pmod p$ si p primo
 Euler : $a^{\phi(m)} \equiv 1 \pmod m$ si $\gcd(a,m) = 1$
 Wilson : $(p-1)! \equiv -1 \pmod p$ si y solo si p es primo
 $(a/b) \pmod p = a * b^{-1} \pmod p$

OJO: restar en modular siempre $((a-b) \% m + m) \% m$

Teorema chino del resto (CRT)

$x \equiv r_1 \pmod{m_1}$, $x \equiv r_2 \pmod{m_2}$, con m_1, m_2 coprimos
 -> solución única módulo m_1*m_2

Legendre: exponente de p en n!

$e = n/p + n/p^2 + n/p^3 + \dots$ (divisiones enteras)
Sirve para contar ceros finales de n! (exponente de 5).

Primos menores que 100 (hay 25)

2 3 5 7 11 13 17 19 23 29 31 37
 41 43 47 53 59 61 67 71 73 79 83 89
 97

Criba: O(n log log n). Cantidad de primos ~ n/ln(n).

hasta 10^3 hay 168	hasta 10^6 hay 78498
hasta 10^4 hay 1229	hasta 10^7 hay 664579
hasta 10^5 hay 9592	hasta 10^9 hay $\sim 5.08*10^7$

En C++ (factorizar en O(sqrt n) da los tres de una)

```
ll numDivisores(ll n) {
    ll r = 1;
    for (ll p = 2; p*p <= n; p++) if (n % p == 0) {
        ll e = 0; while (n % p == 0) n /= p, e++;
        r *= e + 1;
    }
    if (n > 1) r *= 2; // quedo un primo grande
    return r;
}
ll sumaDivisores(ll n) {
    ll r = 1;
    for (ll p = 2; p*p <= n; p++) if (n % p == 0) {
        ll t = 1, q = 1;
        while (n % p == 0) n /= p, q *= p, t += q;
        r *= t;
    }
    if (n > 1) r *= 1 + n;
```

14.15 Teoría de Números (cont.)

```
    return r;
}
ll phi(ll n) {
    ll r = n;
    for (ll p = 2; p*p <= n; p++) if (n % p == 0) {
        while (n % p == 0) n /= p;
        r -= r / p;
    }
    if (n > 1) r -= r / n;
    return r;
}
ll modpow(ll b, ll e, ll m) { // O(log e)
    ll r = 1; b %= m;
    while (e) { if (e & 1) r = r*b % m; b = b*b % m; e >>= 1; }
    return r;
}
// inverso modular con m primo: modpow(a, m-2, m)
ll legendre(ll n, ll p) { // exponente de p en n!
    ll e = 0;
    for (ll q = p; q <= n; q *= p) e += n / q;
    return e;
}
Criba lineal, O(n): cada compuesto se tacha UNA sola vez.
vector<int> criba(int n) {
    vector<int> pr; vector<char> c(n+1, 1); c[0] = c[1] = 0;
    for (int i = 2; i <= n; i++) {
        if (c[i]) pr.push_back(i);
        for (int p : pr) {
            if ((ll)i*p > n) break;
            c[i*p] = 0;
            if (i % p == 0) break;
        }
    }
    return pr;
}
```

14.16 Trampas de C++

O: no aplica: es la lista de lo que muerde en C++

Los errores que compilan sin una sola advertencia y dan respuesta equivocada

C++/Formulario/Trampas de C++.cpp

Division y modulo con negativos (verificado)

$-7 / 2 = -3$ la división trunca HACIA CERO, no hacia abajo
 $-7 \% 2 = -1$ el resto se lleva el signo del dividendo
 $7 / -2 = -3$ $7 \% -2 = 1$
 $\text{floor}(-7.0/2) = -4$ -> distinto de $-7/2$
 modulo siempre positivo : $((a \% m) + m) \% m$
 división hacia abajo : $(a - ((a \% m) + m) \% m) / m$
Esto rompe hashing, índices circulares y aritmética modular.

Overflow silencioso (verificado)

```
int a = 100000, b = 100000;
long long x = a * b; // -> 1410065408 MAL
long long x = (long long)a * b; // -> 100000000000 bien
La conversión ocurre DESPUES de multiplicar: castea un operando.
1 << 40 -> 0 1LL << 40 -> 1099511627776
accumulate(v.begin(), v.end(), 0) suma en int, desborda
accumulate(v.begin(), v.end(), 0LL) suma en long long
Regla: si el resultado pasa de  $2*10^9$ , todo el camino en ll.
```

pow y sqrt son de punto flotante (verificado que falla)

```
(long long)pow(10,15) = 999999999999999 le falta 1
(long long)pow(5,15) = 30517578124 le falta 1
Para enteros usa exponenciación binaria propia.
Para raíz entera, redondea y ajusta:
long long r = sqrtl(n);
while (r*r > n) r--;
while ((r+1)*(r+1) <= n) r++;
```

Comparadores de sort

Tiene que ser Estricto: return a < b; nunca a <= b
 Con <= el sort puede salirse del arreglo y tumbar el programa.
 Descendente: return a > b;
 Por varios campos:
 if (a.x != b.x) return a.x < b.x;
 return a.y < b.y;

Contenedores

m[k] CREA la clave con valor por defecto si no existe.
 Para solo consultar: *m.count(k)* o *m.find(k)*.
unique() exige el vector ORDENADO y no borra por si solo:
 sort(all(v)); v.erase(unique(all(v)), v.end());
 En set y map usa el miembro *s.lower_bound(x)*: es $O(\log n)$.
std::lower_bound(s.begin(), s.end(), x) recorre en $O(n)$.
priority_queue es MAXIMO por defecto. Para minimo:
priority_queue<int, vector<int>, greater<int>> pq;
 Borrar de un vector mientras lo recorres invalida iteradores.

Entrada y salida

cin >> x salta espacios y DEJA el '\n' pendiente. Si despues
 usas *getline*, lees una linea vacia: consume el salto primero.
endl hace flush cada vez; dentro de un bucle usa '\n'.
 Al inicio del main:
ios::sync_with_stdio(false); cin.tie(nullptr);
printf de long long es %lld, no %d.
 Si mezclas *printf* y *cout* con *sync* desactivado, el orden se rompe.

Punto flotante

Nunca compares con ==. Usa *fabs(a-b) < 1e-9*.
 double guarda ~15-16 digitos; por encima de 2^{53} pierde enteros.
 Imprimir con decimales: *cout << fixed << setprecision(k)*
 Si el problema es de enteros, resuelvelo con enteros.

14.17 Trigonometria

O: (1) cada formula; es hoja de consulta
 Conversion grados-radianes, la tabla de valores exactos

C++/Formulario/Trigonometria.cpp

Conversion

radianes = grados * pi / 180
 grados = radianes * 180 / pi
 PI = acos(-1.0) (mas exacto que escribir 3.14159...)
 En C++ sin/cos/tan reciben RADIANTES, no grados.

Valores exactos (verificados)

	0	30	45	60	90
grados	0	30	45	60	90
radianes	0	pi/6	pi/4	pi/3	pi/2
sin	0	1/2	r2/2	r3/2	1
cos	1	r3/2	r2/2	1/2	0
tan	0	r3/3	1	r3	inf

r2 = sqrt(2) ~ 1.4142136 r3 = sqrt(3) ~ 1.7320508

Triangulo rectangulo

sin = cateto opuesto / hipotenusa
 cos = cateto adyacente / hipotenusa
 tan = opuesto / adyacente = sin / cos
 Pitagoras : $a^2 + b^2 = c^2$
 Ternas enteras comunes: 3-4-5, 5-12-13, 8-15-17, 7-24-25

Identidades (verificadas)

$\sin^2 t + \cos^2 t = 1$
 $\sin(2t) = 2 \sin t \cos t$
 $\cos(2t) = \cos^2 t - \sin^2 t = 1 - 2 \sin^2 t = 2 \cos^2 t - 1$
 $\sin(a+b) = \sin a \cos b + \cos a \sin b$
 $\cos(a+b) = \cos a \cos b - \sin a \sin b$
 $\tan(a+b) = (\tan a + \tan b) / (1 - \tan a \tan b)$
 $\sin(-t) = -\sin t$ $\cos(-t) = \cos t$

14.17 Trigonometria (cont.)

$\sin(\pi - t) = \sin t$ $\cos(\pi - t) = -\cos t$
 periodo de sin y cos : 2π periodo de tan : π

Triangulo cualquiera (verificados con el triangulo 3-4-5)

ley de senos : $a/\sin A = b/\sin B = c/\sin C = 2R$
 ley de cosenos : $c^2 = a^2 + b^2 - 2ab \cos C$
 despejando : $\cos C = (a^2 + b^2 - c^2) / (2ab)$
 si $\cos C > 0$ el angulo es agudo, si < 0 es obtuso
 $area = (1/2) * a * b * \sin C$
 radio circunscrito $R = a*b*c / (4 * area)$
 radio inscrito $r = area / s$ con s el semiperimetro
 Los angulos suman 180 grados: sabiendo dos, el tercero sale solo.

atan2 contra atan

atan2(y, x) devuelve el angulo correcto en los cuatro cuadrantes,
 en el rango $(-\pi, \pi]$. Ejemplo: *atan2(-1, -1)* = -135 grados.
atan(y/x) pierde el cuadrante y se rompe cuando $x = 0$.
 Angulo CON SIGNO entre dos vectores:
 $\text{atan2}(\text{cross}(a,b), \text{dot}(a,b))$
 Para ordenar puntos alrededor de un centro se puede usar *atan2*,
 pero con enteros es mas exacto comparar por cuadrante y luego
 por producto cruz, sin tocar flotantes.

14.18 Trucos de Bits

O: (1) cada truco, que es justamente la gracia

Identidades de bits con la situacion concreta en la que sirve cada una, que es la parte que no se deduce

C++/Formulario/Trucos de Bits.cpp

Operar el bit k (k empieza en 0)

leer : $(n \gg k) \& 1$
 prender : $n |= 1LL \ll k$
 apagar : $n \&= ~(1LL \ll k)$
 voltear : $n ^= 1LL \ll k$
 mascara de k bits en 1 : $(1LL \ll k) - 1$
 OJO: $1 \ll 40$ da 0 porque el 1 es int. Usa $1LL \ll 40$.

Identidades y para que sirve cada una

$n \& (n-1)$ apaga el bit encendido mas bajo
 -> $n \& (n-1) == 0$ dice si n es potencia de 2
 -> contar bits: while (n) { n &= n-1; c++; }
 $n \& -n$ deja SOLO el bit mas bajo (lowbit)
 -> es el avance del Fenwick tree: $i += i \& -i$
 $n | (n+1)$ prende el cero mas bajo
 $n \& (n+1)$ apaga la cola de unos del final
 codigo Gray: dos consecutivos difieren en 1 bit
 -> recorrer subconjuntos cambiando un elemento
 $a \wedge a = 0$ y $a \wedge 0 = a$
 -> el que aparece un numero impar de veces sale solo
 -> xor de rango $[l,r] = \text{pre}[r] \wedge \text{pre}[l-1]$
 $a \wedge b \gg 0$ a y b tienen el mismo signo

Codigo Gray, primeros valores

i	0	1	2	3	4	5	6	7
$i \wedge (i \gg 1)$	0	1	3	2	6	7	5	4
binario	000	001	011	010	110	111	101	100

Builtins de GCC (verificados con x = 40 = 101000b)

`__builtin_popcount(40)` = 2 bits en 1
`__builtin_ctz(40)` = 3 ceros al final = indice del lowbit
`__builtin_clz(40)` = 26 ceros al inicio (sobre 32 bits)
`__builtin_parity(7)` = 1 1 si la cantidad de bits es impar
 Para long long agrega ll: `__builtin_popcountll`, `__builtin_ctzll`
 PELIGRO: `ctz` y `clz` con $n = 0$ son comportamiento indefinido.
 bit mas alto de $n = 31 - \text{clz}(n) = \text{floor}(\log_2(n))$
 siguiente potencia de 2 $\gg n : 1 \ll (32 - \text{clz}(n-1))$

Subconjuntos

recorrer TODAS las submascaras de m (sin el vacio):

14.18 Trucos de Bits (cont.)

```

for (int s = m; s; s = (s-1) & m)
    hacerlo para todas las mascaras cuesta 3^n, no 4^n
complemento dentro de n bits : m ^ ((1<<n) - 1)
recorrer los bits: for (b=0;b<n;b++) if (m >> b & 1) ...
agregar elemento b: m | (1<<b)    quitarlo: m & ~(1<<b)

```

Desplazamientos como aritmetica

```

n << 1 = n*2      n << k = n * 2^k
n >> 1 = n/2 solo si n >= 0
OJO: >> redondea hacia abajo y la division hacia cero:
-7 >> 1 = -4      pero -7 / 2 = -3

```

Cuando conviene pensar en bitmask

```

n <= 20 -> DP sobre 2^n estados es viable (~10^6)
n <= 25 -> 2^n cabe, pero ya va justo
Estado tipico: dp[mascara] = mejor valor usando ese subconjunto.
Si el enunciado dice n <= 20, casi siempre es bitmask DP.
bitset<N> para cribas o DP booleana: 64 veces menos memoria y
opera and/or/shift de a 64 bits por vez.

```

15 - Divide and Conquer

15.1 Conteo de Inversiones con Merge Sort - Frosh Merge Sort

O: (n log n), inversiones contando en la fusion
 La otra forma de resolver "Frosh Week" (problema F, CCPL)

C++/CCPL/froshMergeSort.cpp

```

1 #include <bits/stdc++.h>
2 #include <vector>
3
4 using namespace std;
5
6 long long respuesta = 0;
7
8 void merge(vector<int>& arr, int left, int mid, int right) {
9     int n1 = mid - left + 1;
10    int n2 = right - mid;
11
12
13    vector<int> L(n1);
14    vector<int> R(n2);
15
16    for (int i = 0; i < n1; i++)
17        L[i] = arr[left + i];
18    for (int j = 0; j < n2; j++)
19        R[j] = arr[mid + 1 + j];
20
21    int i = 0, j = 0, k = left;
22
23    while (i < n1 && j < n2) {
24        if (L[i] <= R[j]) {
25            arr[k] = L[i];
26            i++;
27        } else {
28            arr[k] = R[j];
29            respuesta += n1 - i;
30            j++;
31        }
32        k++;
33    }
34
35    while (i < n1) {
36        arr[k] = L[i];
37        i++;
38        k++;
39    }
40
41    while (j < n2) {
42        arr[k] = R[j];
43

```

15.1 Conteo de Inversiones con Merge Sort - Frosh Merge Sort (cont.)

```

44        j++;
45        k++;
46    }
47
48    void mergeSort(vector<int>& arr, int left, int right) {
49        if (left >= right)
50            return;
51
52        int mid = left + (right - left) / 2;
53
54        mergeSort(arr, left, mid);
55        mergeSort(arr, mid + 1, right);
56
57        merge(arr, left, mid, right);
58    }
59
60    int main() {
61
62        int n = 0;
63
64        while (cin >> n){
65
66            respuesta = 0;
67            vector<int> arr;
68
69
70            for(int i = 0; i < n; i++){
71                int num = 0;
72                cin >> num;
73                arr.push_back(num);
74            }
75
76            mergeSort(arr, 0, arr.size() - 1);
77            cout << respuesta << endl;
78        }
79
80        return 0;
81    }
82

```

16 - CSES: Advanced Techniques

16.1 Meet in the Middle

O: (2^(n/2) * n), contra (2^n) de la fuerza bruta

Uso: generateSums(a, l, r, sums) por mitad; ordenar una y buscar en ella
 Cuantos subconjuntos suman exactamente x, con n hasta 40

C++/CSES/Advanced Techniques/Meet in the Middle.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 void generateSums(vector<ll>& a, int l, int r, vector<ll>& sums)
7 {
8     int len = r - l;
9
10    for(int mask = 0; mask < (1 << len); mask++)
11    {
12        ll sum = 0;
13
14        for(int i = 0; i < len; i++)
15        {
16            if(mask & (1 << i))
17                sum += a[l + i];
18        }
19
20        sums.push_back(sum);
21    }
22

```

16.1 Meet in the Middle (cont.)

```

23
24 int main()
25 {
26     ios::sync_with_stdio(false);
27     cin.tie(nullptr);
28
29     int n;
30     ll x;
31
32     cin >> n >> x;
33
34     vector<ll> a(n);
35
36     for(auto &v : a)
37         cin >> v;
38
39     int mid = n / 2;
40
41     vector<ll> left;
42     vector<ll> right;
43
44     generateSums(a, 0, mid, left);
45     generateSums(a, mid, n, right);
46
47     sort(right.begin(), right.end());
48
49     ll ans = 0;
50
51     for(ll sum : left)
52     {
53         ll need = x - sum;
54
55         ans += upper_bound(right.begin(), right.end(), need)
56               - lower_bound(right.begin(), right.end(), need);
57     }
58
59     cout << ans << '\n';
60 }

```

17 - CSES: Dynamic Programming

17.1 Counting Numbers - Digit DP con Estado 'ya Empezo'

O: (digitos * 11 * 2 * 2 * 10), o sea constante para 10^{18}

Uso: count(b) - count(a-1); el estado es (pos, dígito previo, tight, started)
 Cuantos numeros entre a y b no tienen dos dígitos iguales seguidos

C++/CSES/Dynamic Programming/Counting Numbers.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 ll dp[20][11][2][2];
7 bool vis[20][11][2][2];
8
9 vector<int> dig;
10
11 ll solve(int pos, int prev, bool tight, bool started)
12 {
13     if(pos == dig.size())
14         return 1;
15
16     if(vis[pos][prev][tight][started])
17         return dp[pos][prev][tight][started];
18
19     vis[pos][prev][tight][started] = true;
20
21     ll ans = 0;
22
23     int limit = tight ? dig[pos] : 9;

```

17.1 Counting Numbers - Digit DP con Estado 'ya Empezo' (cont.)

```

24
25     for(int d = 0; d <= limit; d++)
26     {
27         bool ntight = tight && (d == dig[pos]);
28
29         if(!started && d == 0)
30         {
31             ans += solve(pos + 1, 10, ntight, false);
32         }
33         else
34         {
35             if(started && d == prev)
36                 continue;
37
38             ans += solve(pos + 1, d, ntight, true);
39         }
40     }
41
42     return dp[pos][prev][tight][started] = ans;
43 }
44
45 ll count(ll x)
46 {
47     if(x < 0)
48         return 0;
49
50     string s = to_string(x);
51
52     dig.clear();
53
54     for(char c : s)
55         dig.push_back(c - '0');
56
57     memset(vis, false, sizeof(vis));
58
59     return solve(0, 10, true, false);
60 }
61
62 int main()
63 {
64     ios::sync_with_stdio(false);
65     cin.tie(nullptr);
66
67     ll a, b;
68     cin >> a >> b;
69
70     cout << count(b) - count(a - 1) << '\n';
71 }

```

17.2 Counting Tilings - Broken Profile DP (Dominos)

O: ($m * 2^n * n$), con n el lado corto

Uso: generate(0, mask, 0, dp[mask], dp, ndp) por cada mascara de la columna
 De cuantas formas se llena una cuadrícula n x m con fichas de domino

C++/CSES/Dynamic Programming/Counting Tilings.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const ll MOD = 1e9 + 7;
7
8 int n, m;
9
10 void generate(int row, int mask, int nextMask, ll ways,
11              vector<ll>& dp, vector<ll>& ndp)
12 {
13     if(row == n)
14     {
15         ndp[nextMask] += ways;
16         ndp[nextMask] %= MOD;

```

17.2 Counting Tilings - Broken Profile DP (Dominos) (cont.)

```

17  return;
18 }
19
20 // Esta celda ya esta ocupada
21 if(mask & (1 << row))
22 {
23     generate(row + 1, mask, nextMask, ways, dp, ndp);
24     return;
25 }
26
27 // Ficha horizontal
28 if(row + 1 < n && !(mask & (1 << (row + 1))))
29 {
30     generate(row + 2, mask, nextMask, ways, dp, ndp);
31 }
32
33 // Ficha vertical hacia la siguiente columna
34 generate(row + 1,
35         mask,
36         nextMask | (1 << row),
37         ways,
38         dp,
39         ndp);
40 }
41
42 int main()
43 {
44     ios::sync_with_stdio(false);
45     cin.tie(nullptr);
46
47     cin >> n >> m;
48
49     int states = 1 << n;
50
51     vector<ll> dp(states);
52     dp[0] = 1;
53
54     for(int col = 0; col < m; col++)
55     {
56         vector<ll> ndp(states);
57
58         for(int mask = 0; mask < states; mask++)
59         {
60             if(dp[mask] == 0)
61                 continue;
62
63             generate(0, mask, 0, dp[mask], dp, ndp);
64         }
65
66         dp = ndp;
67     }
68
69     cout << dp[0] << '\n';
70 }

```

17.3 Elevator Rides - Bitmask DP con Estado de Dos Partes

O: $(2^n * n)$

Uso: $dp[mask] = \{viajes, peso\}$ del ultimo; se minimiza el par con $\min()$

Minimo de viajes de ascensor para subir a n personas con limite de peso x

C++/CSES/Dynamic Programming/Elevator Rides.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 int main()
7 {
8     ios::sync_with_stdio(false);
9     cin.tie(nullptr);
10

```

17.3 Elevator Rides - Bitmask DP con Estado de Dos Partes (cont.)

```

11 int n;
12 ll x;
13
14 cin >> n >> x;
15
16 vector<ll> w(n);
17
18 for(auto &v : w)
19     cin >> v;
20
21 int N = 1 << n;
22
23 // {numero de viajes, peso del ultimo viaje}
24 vector<pair<int, ll>> dp(N, {n + 1, 0});
25
26 dp[0] = {1, 0};
27
28 for(int mask = 0; mask < N; mask++)
29 {
30     for(int i = 0; i < n; i++)
31     {
32         if(mask & (1 << i))
33             continue;
34
35         auto [rides, weight] = dp[mask];
36
37         pair<int, ll> nxt;
38
39         if(weight + w[i] <= x)
40         {
41             // La persona cabe en el ultimo viaje
42             nxt = {rides, weight + w[i]};
43         }
44         else
45         {
46             // Abrimos un nuevo viaje
47             nxt = {rides + 1, w[i]};
48         }
49
50         int newMask = mask | (1 << i);
51
52         dp[newMask] = min(dp[newMask], nxt);
53     }
54 }
55
56 cout << dp[N - 1].first << '\n';
57 }

```

17.4 Increasing Subsequence II - Contar Subsecuencias Crecientes con Fenwick

O: $(n \log n)$, Fenwick sobre coordenadas comprimidas

Uso: $dp = 1 + fw.query(pos-1)$; $fw.add(pos, dp)$ // 1-INDEXADO

Cuántas subsecuencias estrictamente crecientes tiene el arreglo, modulo 10^9+7

C++/CSES/Dynamic Programming/Increasing Subsequence II.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const ll MOD = 1e9 + 7;
7
8 struct Fenwick
9 {
10     int n;
11     vector<ll> bit;
12
13     Fenwick(int n)
14     {
15         this->n = n;
16         bit.assign(n + 1, 0);
17     }

```

17.4 Increasing Subsequence II - Contar Subsecuencias Crecientes con Fenwick (cont.)

```

8
8 void add(int pos, ll val)
20 {
21     for(; pos <= n; pos += pos & -pos)
22     {
23         bit[pos] += val;
24         bit[pos] %= MOD;
25     }
26 }
27
28 ll query(int pos)
29 {
30     ll ans = 0;
31
32     for(; pos > 0; pos -= pos & -pos)
33     {
34         ans += bit[pos];
35         ans %= MOD;
36     }
37
38     return ans;
39 }
40 };
41
42 int main()
43 {
44     ios::sync_with_stdio(false);
45     cin.tie(nullptr);
46
47     int n;
48     cin >> n;
49
50     vector<ll> a(n);
51
52     for(auto &x : a)
53         cin >> x;
54
55     // Coordinate compression
56     vector<ll> vals = a;
57
58     sort(vals.begin(), vals.end());
59
60     vals.erase(unique(vals.begin(), vals.end()), vals.end());
61
62     Fenwick fw(vals.size());
63
64     ll ans = 0;
65
66     for(int i = 0; i < n; i++)
67     {
68         int pos = lower_bound(vals.begin(), vals.end(), a[i])
69             - vals.begin() + 1;
70
71         // Todos los valores estrictamente menores
72         ll dp = 1 + fw.query(pos - 1);
73
74         dp %= MOD;
75
76         fw.add(pos, dp);
77
78         ans += dp;
79         ans %= MOD;
80     }
81
82     cout << ans << '\n';
83 }

```

17.5 Projects - Weighted Interval Scheduling

O: $(n \log n)$, sort mas una binaria por proyecto

Uso: ordenar por FIN; $dp[i] = \max(dp[i-1], dp[j] + p)$ con $j = \text{lower_bound}$ del inicio
Escoger proyectos que no se solapen maximizando la ganancia total

C++/CSES/Dynamic Programming/Projects.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Project
7 {
8     ll a, b, p;
9 };
10
11 int main()
12 {
13     ios::sync_with_stdio(false);
14     cin.tie(nullptr);
15
16     int n;
17     cin >> n;
18
19     vector<Project> projects(n);
20
21     for(auto &[a, b, p] : projects)
22         cin >> a >> b >> p;
23
24     sort(projects.begin(), projects.end(),
25         [](Project A, Project B)
26         {
27             return A.b < B.b;
28         });
29
30     vector<ll> ends(n);
31
32     for(int i = 0; i < n; i++)
33         ends[i] = projects[i].b;
34
35     vector<ll> dp(n + 1, 0);
36
37     for(int i = 1; i <= n; i++)
38     {
39         auto [a, b, p] = projects[i - 1];
40
41         // Primer proyecto cuyo final >= a
42         int j = lower_bound(ends.begin(), ends.end(), a)
43             - ends.begin();
44
45         dp[i] = max(
46             dp[i - 1],
47             dp[j] + p
48         );
49     }
50
51     cout << dp[n] << '\n';
52 }

```

18 - CSES: Graph Algorithms

18.1 Building Roads - DSU (Union-Find)

O: $(n + m)$ con DSU casi constante por operacion

DSU con compresion de camino y union por tamano, que juntas dan casi $O(1)$ amortizado por operacion

C++/CSES/Graph Algorithms/Building Roads.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<long long> padre;
5 vector<long long> tamano;

```

18.1 Building Roads - DSU (Union-Find) (cont.)

```

6
7 long long encontrar(long long x)
8 {
9     if(padre[x] == x)
10    {
11        return x;
12    }
13
14    return padre[x] = encontrar(padre[x]);
15 }
16
17 void unir(long long a, long long b)
18 {
19     a = encontrar(a);
20     b = encontrar(b);
21
22     if(a == b)
23     {
24         return;
25     }
26
27     if(tamano[a] < tamano[b])
28     {
29         swap(a, b);
30     }
31
32     padre[b] = a;
33     tamano[a] += tamano[b];
34 }
35
36 int main()
37 {
38     ios::sync_with_stdio(false);
39     cin.tie(nullptr);
40
41     long long n, m;
42     cin >> n >> m;
43
44     padre.resize(n + 1);
45     tamano.resize(n + 1, 1);
46
47     for(long long i = 1; i <= n; i++)
48     {
49         padre[i] = i;
50     }
51
52     long long a, b;
53
54     for(long long i = 0; i < m; i++)
55     {
56         cin >> a >> b;
57         unir(a, b);
58     }
59
60     vector<long long> familias;
61
62     for(long long i = 1; i <= n; i++)
63     {
64         if(encontrar(i) == i)
65         {
66             familias.push_back(i);
67         }
68     }
69
70     cout << (familias.size()-1) << '\n';
71
72     for(long long i = 0; i + 1 < familias.size(); i++)
73     {
74         cout << familias[i] << ' ' << familias[i + 1] << '\n';
75     }
76
77     return 0;

```

18.1 Building Roads - DSU (Union-Find) (cont.)

```

78 }

```

18.2 Building Teams - BFS de Bicoloreo (Bipartito)

O: (n + m), BFS pintando dos colores

Bicoloreo por BFS: se pinta el primer nodo de 1 y cada vecino del color opuesto

C++/CSES/Graph Algorithms/Building Teams.cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4
5 using namespace std;
6
7 int main()
8 {
9     long long n, m;
10
11     cin >> n >> m;
12
13     vector<vector<long long>> vecinos(n + 1);
14
15     long long a, b;
16
17     while(m--)
18     {
19         cin >> a >> b;
20
21         vecinos[a].push_back(b);
22         vecinos[b].push_back(a);
23     }
24
25     vector<long long> valores(n + 1, -1);
26
27     for(long long i = 1; i <= n; i++)
28     {
29         if(valores[i] == -1)
30         {
31             queue<long long> cola;
32
33             valores[i] = 1;
34             cola.push(i);
35
36             while(!cola.empty())
37             {
38                 long long actual = cola.front();
39                 cola.pop();
40
41                 for(long long conexion : vecinos[actual])
42                 {
43                     if(valores[conexion] == -1)
44                     {
45                         if(valores[actual] == 1)
46                         {
47                             valores[conexion] = 2;
48                         }
49                         else
50                         {
51                             valores[conexion] = 1;
52                         }
53
54                         cola.push(conexion);
55                     }
56                     else if(valores[conexion] == valores[actual])
57                     {
58                         cout << "IMPOSSIBLE\n";
59                         return 0;
60                     }
61                 }
62             }
63         }
64     }

```

18.2 Building Teams - BFS de Bicoloreo (Bipartito) (cont.)

```

85
86     for(long long i = 1; i <= n; i++)
87     {
88         cout << valores[i];
89
90         if(i < n)
91         {
92             cout << ' ';
93         }
94     }
95
96     cout << '\n';
97
98     return 0;
99 }

```

18.3 Counting Rooms - Flood Fill en Grilla (BFS)

O: $(n \times m)$, cada celda se visita una vez

Contar componentes conexas en una grilla

C++/CSES/Graph Algorithms/Counting Rooms.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     ios::sync_with_stdio(false);
7     cin.tie(nullptr);
8
9     long long n, m;
10    cin >> n >> m;
11
12    vector<string> matriz(n);
13
14    for(long long i = 0; i < n; i++)
15    {
16        cin >> matriz[i];
17    }
18
19    queue<pair<long long, long long>> cuartos;
20
21    long long numCuartos = 0;
22
23    for(long long i = 0; i < n; i++)
24    {
25        for(long long j = 0; j < m; j++)
26        {
27            if(matriz[i][j] == '.')
28            {
29                matriz[i][j] = 'X';
30                cuartos.push({i, j});
31
32                while(!cuartos.empty())
33                {
34                    long long x = cuartos.front().first;
35                    long long y = cuartos.front().second;
36                    cuartos.pop();
37
38                    if(x - 1 >= 0 && matriz[x - 1][y] == '.')
39                    {
40                        matriz[x - 1][y] = 'X';
41                        cuartos.push({x - 1, y});
42                    }
43
44                    if(x + 1 < n && matriz[x + 1][y] == '.')
45                    {
46                        matriz[x + 1][y] = 'X';
47                        cuartos.push({x + 1, y});
48                    }
49
50                    if(y - 1 >= 0 && matriz[x][y - 1] == '.')

```

18.3 Counting Rooms - Flood Fill en Grilla (BFS) (cont.)

```

51                {
52                    matriz[x][y - 1] = 'X';
53                    cuartos.push({x, y - 1});
54                }
55
56                if(y + 1 < m && matriz[x][y + 1] == '.')
57                {
58                    matriz[x][y + 1] = 'X';
59                    cuartos.push({x, y + 1});
60                }
61            }
62
63            numCuartos++;
64        }
65    }
66
67    cout << numCuartos;
68
69    return 0;
70 }

```

18.4 Course Schedule - Orden Topologico (Kahn)

O: $(n + m)$, Kahn con grados de entrada

Orden topologico con el algoritmo de Kahn

C++/CSES/Graph Algorithms/Course Schedule.cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4
5 using namespace std;
6
7 int main()
8 {
9     ios::sync_with_stdio(false);
10    cin.tie(nullptr);
11
12    long long n, m;
13    cin >> n >> m;
14
15    vector<vector<long long>> conexiones(n);
16    vector<long long> grado(n, 0);
17
18    for(long long i = 0; i < m; i++)
19    {
20        long long a, b;
21        cin >> a >> b;
22
23        a--;
24        b--;
25
26        conexiones[a].push_back(b);
27        grado[b]++;
28    }
29
30    queue<long long> cola;
31
32    for(long long i = 0; i < n; i++)
33    {
34        if(grado[i] == 0)
35        {
36            cola.push(i);
37        }
38    }
39
40    vector<long long> orden;
41
42    while(!cola.empty())
43    {
44        long long actual = cola.front();

```

18.4 Course Schedule - Orden Topologico (Kahn) (cont.)

```

45     cola.pop();
46
47     orden.push_back(actual);
48
49     for(long long siguiente : conexiones[actual])
50     {
51         grado[siguiente]--;
52
53         if(grado[siguiente] == 0)
54         {
55             cola.push(siguiente);
56         }
57     }
58 }
59
60 if(orden.size() != n)
61 {
62     cout << "IMPOSSIBLE\n";
63     return 0;
64 }
65
66 for(long long nodo : orden)
67 {
68     cout << nodo + 1 << ' ';
69 }
70
71 cout << '\n';
72
73 return 0;
74 }

```

18.5 Cycle Finding - Bellman-Ford (Ciclo Negativo)

O: (n^3m), Bellman-Ford con la n -esima ronda como detector

Bellman-Ford usado no para hallar distancias sino para DETECTAR un ciclo negativo y devolverlo

C++/CSES/Graph Algorithms/Cycle Finding.cpp

```

1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  int main()
7  {
8      ios::sync_with_stdio(false);
9      cin.tie(nullptr);
10
11     long long n, m;
12     cin >> n >> m;
13
14     vector<vector<pair<long long, long long>>> conexion(n);
15
16     for(long long i = 0; i < m; i++)
17     {
18         long long a, b, c;
19         cin >> a >> b >> c;
20
21         conexion[--a].push_back({--b, c});
22     }
23
24     vector<long long> distancia(n, 0);
25     vector<long long> padre(n, -1);
26
27     long long ciclo = -1;
28
29     for(long long i = 0; i < n; i++)
30     {
31         ciclo = -1;
32
33         for(long long a = 0; a < n; a++)
34         {
35             for(auto [b, c] : conexion[a])

```

18.5 Cycle Finding - Bellman-Ford (Ciclo Negativo) (cont.)

```

36         {
37             if(distancia[a] + c < distancia[b])
38             {
39                 distancia[b] = distancia[a] + c;
40                 padre[b] = a;
41                 ciclo = b;
42             }
43         }
44     }
45
46     if(ciclo == -1)
47         break;
48 }
49
50 if(ciclo == -1)
51 {
52     cout << "NO\n";
53     return 0;
54 }
55
56 for(long long i = 0; i < n; i++)
57     ciclo = padre[ciclo];
58
59 vector<long long> respuesta;
60
61 long long actual = ciclo;
62
63 do
64 {
65     respuesta.push_back(actual);
66     actual = padre[actual];
67 }
68 while(actual != ciclo);
69
70 respuesta.push_back(ciclo);
71
72 cout << "YES\n";
73
74 for(long long i = respuesta.size() - 1; i >= 0; i--)
75     cout << respuesta[i] + 1 << ' ';
76
77 cout << '\n';
78
79 return 0;
80 }

```

18.6 Flight Discount - Dijkstra sobre Grafo de Capas

O: ($m \log n$), Dijkstra sobre el grafo de dos capas

Dijkstra donde el estado no es solo la ciudad sino el par (ciudad, ya use el descuento)

C++/CSES/Graph Algorithms/Flight Discount.cpp

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  #include <array>
5  #include <limits>
6
7  using namespace std;
8
9  int main()
10 {
11     ios::sync_with_stdio(false);
12     cin.tie(nullptr);
13
14     long long n, m;
15
16     cin >> n >> m;
17
18     vector<vector<pair<long long, long long>>> vuelos(n);
19
20     long long a,b,c;

```

18.6 Flight Discount - Dijkstra sobre Grafo de Capas (cont.)

```

21
22 while(m--)
23 {
24     cin >> a >> b >> c;
25
26     vuelos[--a].push_back({--b,c});
27 }
28
29 //Estados
30 long long USADO = 1, NO_USADO = 0;
31
32 vector<vector<long long>> costoMinimo(n, vector<long long>(2, LLONG_MAX));
33
34 priority_queue<array<long long, 3>,
35               vector<array<long long, 3>>,
36               greater<array<long long, 3>>> cola;
37
38 costoMinimo[0][NO_USADO] = 0;
39
40 cola.push({0,0,NO_USADO});
41
42 while(!cola.empty())
43 {
44     long long costo, ciudad, estado;
45
46     costo = cola.top()[0];
47     ciudad = cola.top()[1];
48     estado = cola.top()[2];
49
50     cola.pop();
51
52     if(costoMinimo[ciudad][estado] < costo) continue;
53
54     for(pair<long long, long long> vuelo : vuelos[ciudad])
55     {
56         long long adonde = vuelo.first;
57         long long cuanto = vuelo.second;
58
59         if(costoMinimo[adonde][estado] > costo + cuanto)
60         {
61             costoMinimo[adonde][estado] = costo + cuanto;
62
63             cola.push({costo+cuanto,adonde,estado});
64         }
65
66         if(estado == NO_USADO)
67         {
68             if(costoMinimo[adonde][USADO] > costo + cuanto/2)
69             {
70                 costoMinimo[adonde][USADO] = costo + cuanto/2;
71
72                 cola.push({costo+cuanto/2,adonde,USADO});
73             }
74         }
75     }
76 }
77
78 cout << min(costoMinimo[n-1][NO_USADO], costoMinimo[n-1][USADO]) << endl;
79
80 return 0;
81 }

```

18.7 Flight Routes - K Caminos Mas Cortos

O: $(k \cdot m \log(k \cdot m))$, guardando las k mejores por nodo

Los k caminos mas cortos al nodo n, con la variante de Dijkstra que en vez de UNA distancia por nodo guarda...

C++/CSES/Graph Algorithms/Flight Routes.cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>

```

18.7 Flight Routes - K Caminos Mas Cortos (cont.)

```

4 #include <functional>
5
6 using namespace std;
7
8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     long long n, m, k;
14     cin >> n >> m >> k;
15
16     vector<vector<pair<long long, long long>>> conexion(n);
17
18     for(long long i = 0; i < m; i++)
19     {
20         long long a, b, c;
21         cin >> a >> b >> c;
22
23         conexion[--a].push_back({--b, c});
24     }
25
26     vector<priority_queue<long long>> distancia(n);
27     vector<long long> usados(n, 0);
28
29     priority_queue<
30         pair<long long, long long>,
31         vector<pair<long long, long long>>,
32         greater<pair<long long, long long>>
33     > cola;
34
35     distancia[0].push(0);
36     cola.push({0, 0});
37
38     while(!cola.empty())
39     {
40         long long costo = cola.top().first;
41         long long actual = cola.top().second;
42         cola.pop();
43
44         usados[actual]++;
45
46         if(usados[actual] > k)
47             continue;
48
49         for(auto vecino : conexion[actual])
50         {
51             long long siguiente = vecino.first;
52             long long peso = vecino.second;
53
54             long long nuevoCosto = costo + peso;
55
56             if(distancia[siguiente].size() < k)
57             {
58                 distancia[siguiente].push(nuevoCosto);
59                 cola.push({nuevoCosto, siguiente});
60             }
61             else if(nuevoCosto < distancia[siguiente].top())
62             {
63                 distancia[siguiente].pop();
64                 distancia[siguiente].push(nuevoCosto);
65                 cola.push({nuevoCosto, siguiente});
66             }
67         }
68     }
69
70     vector<long long> respuesta;
71
72     while(!distancia[n - 1].empty())
73     {
74         respuesta.push_back(distancia[n - 1].top());
75         distancia[n - 1].pop();
76     }

```

18.7 Flight Routes - K Caminos Mas Cortos (cont.)

```

76     }
77
78     for(long long i = respuesta.size() - 1; i >= 0; i--)
79         cout << respuesta[i] << ' ';
80
81     cout << '\n';
82
83     return 0;
84 }

```

18.8 Game Routes - DP sobre DAG en Orden Topologico

O: $(n + m)$, DP en orden topologico

Contar caminos de 1 a n en un DAG: caminos[v] = suma de caminos[u] sobre las aristas $u \rightarrow v$

C++/CSES/Graph Algorithms/Game Routes.cpp

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4
5  using namespace std;
6
7  int main()
8  {
9      ios::sync_with_stdio(false);
10     cin.tie(nullptr);
11
12     const long long MOD = 1000000007;
13
14     long long n, m;
15     cin >> n >> m;
16
17     vector<vector<long long>>> conexiones(n);
18     vector<long long> grado(n, 0);
19
20     for(long long i = 0; i < m; i++)
21     {
22         long long a, b;
23         cin >> a >> b;
24
25         a--;
26         b--;
27
28         conexiones[a].push_back(b);
29         grado[b]++;
30     }
31
32     vector<long long> caminos(n, 0);
33     queue<long long> cola;
34
35     caminos[0] = 1;
36
37     for(long long i = 0; i < n; i++)
38     {
39         if(grado[i] == 0)
40         {
41             cola.push(i);
42         }
43     }
44
45     while(!cola.empty())
46     {
47         long long actual = cola.front();
48         cola.pop();
49
50         for(long long siguiente : conexiones[actual])
51         {
52             caminos[siguiente] += caminos[actual];
53             caminos[siguiente] %= MOD;
54
55             grado[siguiente]--;
56

```

18.8 Game Routes - DP sobre DAG en Orden Topologico (cont.)

```

57         if(grado[siguiente] == 0)
58         {
59             cola.push(siguiente);
60         }
61     }
62 }
63
64 cout << caminos[n - 1] << '\n';
65
66 return 0;
67 }

```

18.9 High Score - Bellman-Ford (Camino Maximo)

O: $(n*m)$, Bellman-Ford maximizando y marcando ciclos positivos

Camino de peso MAXIMO con aristas negativas, que es Bellman-Ford con la desigualdad volteada

C++/CSES/Graph Algorithms/High Score.cpp

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  #include <climits>
5
6  using namespace std;
7
8  int main()
9  {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     // INPUT
14     long long n, m;
15     cin >> n >> m;
16
17     vector<vector<pair<long long, long long>>>> conexion(n);
18     vector<vector<pair<long long, long long>>>> conexionInversa(n);
19
20     for(long long i = 0; i < m; i++)
21     {
22         long long a, b, c;
23         cin >> a >> b >> c;
24
25         a--;
26         b--;
27
28         conexion[a].push_back({b, c});
29         conexionInversa[b].push_back({a, c});
30     }
31
32     vector<bool> de0(n, false);
33     vector<bool> antesN(n, false);
34
35     // BFS desde 0
36     queue<long long> cola;
37     cola.push(0);
38     de0[0] = true;
39
40     while(!cola.empty())
41     {
42         long long actual = cola.front();
43         cola.pop();
44
45         for(auto vecino : conexion[actual])
46         {
47             long long siguiente = vecino.first;
48
49             if(!de0[siguiente])
50             {
51                 de0[siguiente] = true;
52                 cola.push(siguiente);
53             }
54         }

```

18.9 High Score - Bellman-Ford (Camino Maximo) (cont.)

```

55     }
56 }
57 // BFS desde n-1 usando el grafo inverso
58 cola.push(n - 1);
59 antesN[n - 1] = true;
60
61 while(!cola.empty())
62 {
63     long long actual = cola.front();
64     cola.pop();
65
66     for(auto vecino : conexionInversa[actual])
67     {
68         long long siguiente = vecino.first;
69
70         if(!antesN[siguiente])
71         {
72             antesN[siguiente] = true;
73             cola.push(siguiente);
74         }
75     }
76 }
77
78 // OUTPUT
79 vector<long long> distancia(n, LLONG_MIN);
80 distancia[0] = 0;
81
82 bool ciclo = false;
83
84 for(long long i = 0; i < n; i++)
85 {
86     ciclo = false;
87
88     for(long long a = 0; a < n; a++)
89     {
90         if(!de0[a] || !antesN[a] || distancia[a] == LLONG_MIN)
91             continue;
92
93         for(auto vecino : conexion[a])
94         {
95             long long b = vecino.first;
96             long long c = vecino.second;
97
98             if(!de0[b] || !antesN[b])
99                 continue;
100
101             if(distancia[b] < distancia[a] + c)
102             {
103                 distancia[b] = distancia[a] + c;
104                 ciclo = true;
105             }
106         }
107     }
108 }
109
110 if(ciclo)
111     cout << -1 << '\n';
112 else
113     cout << distancia[n - 1] << '\n';
114
115 return 0;
116 }

```

18.10 Investigation - Dijkstra con Cuatro Cantidades a la Vez

O: (m log n), Dijkstra arrastrando cuatro cantidades a la vez

Un solo Dijkstra que ademas del costo minimo lleva otras tres cosas

C++/CSES/Graph Algorithms/investigation.cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>

```

18.10 Investigation - Dijkstra con Cuatro Cantidades a la Vez (cont.)

```

4 #include <utility>
5 #include <limits>
6 #include <algorithm>
7
8 using namespace std;
9
10 int main()
11 {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14
15     const long long MOD = 1000000007;
16
17     long long n, m;
18     cin >> n >> m;
19
20     vector<vector<pair<long long, long long>>> conexiones(n);
21
22     for(long long i = 0; i < m; i++)
23     {
24         long long a, b, c;
25         cin >> a >> b >> c;
26
27         a--;
28         b--;
29
30         conexiones[a].push_back({b, c});
31     }
32
33     vector<long long> caminos(n, 0);
34     vector<long long> distancia(n, LLONG_MAX);
35     vector<long long> minvuelos(n, LLONG_MAX);
36     vector<long long> maxvuelos(n, 0);
37
38     priority_queue<
39         pair<long long, long long>,
40         vector<pair<long long, long long>>,
41         greater<pair<long long, long long>>
42     > cola;
43
44     caminos[0] = 1;
45     distancia[0] = 0;
46     minvuelos[0] = 0;
47
48     cola.push({0, 0});
49
50     while(!cola.empty())
51     {
52         long long peso = cola.top().first;
53         long long actual = cola.top().second;
54         cola.pop();
55
56         if(peso != distancia[actual])
57         {
58             continue;
59         }
60
61         for(pair<long long, long long> siguiente : conexiones[actual])
62         {
63             long long hacia = siguiente.first;
64             long long cuanto = siguiente.second;
65
66             long long nuevaDistancia = peso + cuanto;
67             long long nuevosVuelos = minvuelos[actual] + 1;
68
69             if(distancia[hacia] > nuevaDistancia)
70             {
71                 distancia[hacia] = nuevaDistancia;
72                 caminos[hacia] = caminos[actual];
73
74                 minvuelos[hacia] = nuevosVuelos;
75                 maxvuelos[hacia] = maxvuelos[actual] + 1;

```

18.10 Investigation - Dijkstra con Cuatro Cantidades a la Vez (cont.)

```

76
77     cola.push({nuevaDistancia, hacia});
78 }
79 else if(distancia[hacia] == nuevaDistancia)
80 {
81     caminos[hacia] += caminos[actual];
82     caminos[hacia] %= MOD;
83
84     minvuelos[hacia] = min(minvuelos[hacia], nuevosVuelos);
85     maxvuelos[hacia] = max(maxvuelos[hacia], maxvuelos[actual] + 1);
86 }
87 }
88 }
89
90 cout << distancia[n - 1] << " "
91      << caminos[n - 1] << " "
92      << minvuelos[n - 1] << " "
93      << maxvuelos[n - 1] << "\n";
94
95 return 0;
96 }

```

18.11 Labyrinth - BFS con Reconstruccion de Camino

O: (n*m), BFS guardando de donde se llego a cada celda

BFS en grilla guardando de donde vino cada celda en la matriz anterior[]

C++/CSES/Graph Algorithms/Labyrinth.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     ios::sync_with_stdio(false);
7     cin.tie(nullptr);
8
9     long long n, m;
10    cin >> n >> m;
11
12    vector<string> matriz(n);
13
14    long long Ax = -1;
15    long long Ay = -1;
16
17    for(long long i = 0; i < n; i++)
18    {
19        cin >> matriz[i];
20
21        if(Ax == -1)
22        {
23            for(long long j = 0; j < m; j++)
24            {
25                if(matriz[i][j] == 'A')
26                {
27                    Ax = i;
28                    Ay = j;
29                    break;
30                }
31            }
32        }
33    }
34
35    vector<vector<pair<long long, long long>>> anterior(
36        n,
37        vector<pair<long long, long long>>(m, {-1, -1})
38    );
39
40    queue<pair<long long, long long>> bfs;
41
42    bfs.push({Ax, Ay});
43
44    bool encontrado = false;

```

18.11 Labyrinth - BFS con Reconstruccion de Camino (cont.)

```

45
46    long long Bx = -1;
47    long long By = -1;
48
49    while(!bfs.empty() && !encontrado)
50    {
51        long long x = bfs.front().first;
52        long long y = bfs.front().second;
53
54        bfs.pop();
55
56        if(x > 0 && (matriz[x - 1][y] == '.' || matriz[x - 1][y] == 'B'))
57        {
58            anterior[x - 1][y] = {x, y};
59
60            if(matriz[x - 1][y] == 'B')
61            {
62                Bx = x - 1;
63                By = y;
64                encontrado = true;
65                break;
66            }
67
68            matriz[x - 1][y] = 'X';
69            bfs.push({x - 1, y});
70        }
71
72        if(x + 1 < n && (matriz[x + 1][y] == '.' || matriz[x + 1][y] == 'B'))
73        {
74            anterior[x + 1][y] = {x, y};
75
76            if(matriz[x + 1][y] == 'B')
77            {
78                Bx = x + 1;
79                By = y;
80                encontrado = true;
81                break;
82            }
83
84            matriz[x + 1][y] = 'X';
85            bfs.push({x + 1, y});
86        }
87
88        if(y > 0 && (matriz[x][y - 1] == '.' || matriz[x][y - 1] == 'B'))
89        {
90            anterior[x][y - 1] = {x, y};
91
92            if(matriz[x][y - 1] == 'B')
93            {
94                Bx = x;
95                By = y - 1;
96                encontrado = true;
97                break;
98            }
99
100            matriz[x][y - 1] = 'X';
101            bfs.push({x, y - 1});
102        }
103
104        if(y + 1 < m && (matriz[x][y + 1] == '.' || matriz[x][y + 1] == 'B'))
105        {
106            anterior[x][y + 1] = {x, y};
107
108            if(matriz[x][y + 1] == 'B')
109            {
110                Bx = x;
111                By = y + 1;
112                encontrado = true;
113                break;
114            }
115
116            matriz[x][y + 1] = 'X';

```

18.11 Labyrinth - BFS con Reconstruccion de Camino (cont.)

```

17     bfs.push({x, y + 1});
18 }
19 }
20
21 if(!encontrado)
22 {
23     cout << "NO\n";
24     return 0;
25 }
26
27 string recorrido = "";
28
29 long long x = Bx;
30 long long y = By;
31
32 while(x != Ax || y != Ay)
33 {
34     long long anteriorX = anterior[x][y].first;
35     long long anteriorY = anterior[x][y].second;
36
37     if(x - anteriorX > 0)
38     {
39         recorrido += "D";
40     }
41     else if(x - anteriorX < 0)
42     {
43         recorrido += "U";
44     }
45     else if(y - anteriorY > 0)
46     {
47         recorrido += "R";
48     }
49     else
50     {
51         recorrido += "L";
52     }
53
54     x = anteriorX;
55     y = anteriorY;
56 }
57
58 reverse(recorrido.begin(), recorrido.end());
59
60 cout << "YES\n";
61 cout << recorrido.size() << '\n';
62 cout << recorrido << '\n';
63
64 return 0;
65 }

```

18.12 Longest Flight Route - Camino Mas Largo en DAG (DFS con Memo)

O: (n + m), DFS con memo sobre el DAG

Camino con mas nodos de 1 a n en un DAG, con DFS memoizado: $mejor[u] = 1 + \max \text{ de } mejor[hijo]$

C++/CSES/Graph Algorithms/Longest Flight Route.cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 using namespace std;
6
7 long long n, m;
8
9 vector<vector<long long>> conexiones;
10 vector<long long> mejor;
11 vector<long long> siguiente;
12
13 long long dfs(long long actual)
14 {
15     if(actual == n - 1)
16         return 1;

```

18.12 Longest Flight Route - Camino Mas Largo en DAG (DFS con Memo) (cont.)

```

17
18     if(mejor[actual] != -1)
19         return mejor[actual];
20
21     mejor[actual] = 0;
22
23     for(long long hijo : conexiones[actual])
24     {
25         long long camino = dfs(hijo);
26
27         if(camino > 0 && camino + 1 > mejor[actual])
28         {
29             mejor[actual] = camino + 1;
30             siguiente[actual] = hijo;
31         }
32     }
33
34     return mejor[actual];
35 }
36
37 int main()
38 {
39     ios::sync_with_stdio(false);
40     cin.tie(nullptr);
41
42     cin >> n >> m;
43
44     conexiones.resize(n);
45
46     for(long long i = 0; i < m; i++)
47     {
48         long long a, b;
49         cin >> a >> b;
50
51         conexiones[a - 1].push_back(b - 1);
52     }
53
54     mejor.assign(n, -1);
55     siguiente.assign(n, -1);
56
57     long long largo = dfs(0);
58
59     if(largo == 0)
60     {
61         cout << "IMPOSSIBLE\n";
62         return 0;
63     }
64
65     cout << largo << '\n';
66
67     long long actual = 0;
68
69     while(actual != -1)
70     {
71         cout << actual + 1 << ' ';
72
73         if(actual == n - 1)
74             break;
75
76         actual = siguiente[actual];
77     }
78
79     cout << '\n';
80
81     return 0;
82 }

```

18.13 Message Route - BFS de Camino Minimo con Reconstruccion**O:** (n + m), BFS con arreglo de padres

Mismo patron que Labyrinth pero en grafo general

C++/CSES/Graph Algorithms/Message Route.cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4
5 using namespace std;
6
7 int main()
8 {
9     ios::sync_with_stdio(false);
10    cin.tie(nullptr);
11
12    long long n, m;
13
14    cin >> n >> m;
15
16    vector<vector<long long>> vecinos(n + 1);
17
18    long long a, b;
19
20    while(m--)
21    {
22        cin >> a >> b;
23
24        vecinos[a].push_back(b);
25        vecinos[b].push_back(a);
26    }
27
28    vector<long long> siguiente(n + 1, -1);
29    vector<long long> cuantos(n + 1, 0);
30
31    queue<long long> cola;
32
33    cola.push(n);
34    cuantos[n] = 1;
35
36    while(!cola.empty())
37    {
38        long long actual = cola.front();
39        cola.pop();
40
41        if(actual == 1)
42        {
43            break;
44        }
45
46        for(long long vecino : vecinos[actual])
47        {
48            if(cuantos[vecino] == 0)
49            {
50                siguiente[vecino] = actual;
51                cuantos[vecino] = cuantos[actual] + 1;
52                cola.push(vecino);
53            }
54        }
55    }
56
57    if(cuantos[1] == 0)
58    {
59        cout << "IMPOSSIBLE\n";
60        return 0;
61    }
62
63    cout << cuantos[1] << '\n';
64
65    long long actual = 1;
66
67    while(actual != n)

```

18.13 Message Route - BFS de Camino Minimo con Reconstruccion (cont.)

```

68    {
69        cout << actual << ' ';
70        actual = siguiente[actual];
71    }
72
73    cout << n << '\n';
74
75    return 0;
76 }

```

18.14 Monsters - BFS Multifuente + BFS de Escape**O:** (n*m), un BFS multifuente de monstruos y otro de escape

Dos BFS encadenados, y el primero es el truco que vale aprender

C++/CSES/Graph Algorithms/Monsters.cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <algorithm>
5
6 using namespace std;
7
8 int main()
9 {
10    ios::sync_with_stdio(false);
11    cin.tie(nullptr);
12
13    long long n, m;
14
15    cin >> n >> m;
16
17    vector<string> matriz(n);
18
19    vector<pair<long long, long long>> monstruos;
20
21    long long Ax = -1;
22    long long Ay = -1;
23
24    for(long long i = 0; i < n; i++)
25    {
26        cin >> matriz[i];
27
28        for(long long j = 0; j < m; j++)
29        {
30            if(matriz[i][j] == 'M')
31            {
32                monstruos.push_back({i, j});
33            }
34            else if(matriz[i][j] == 'A')
35            {
36                Ax = i;
37                Ay = j;
38            }
39        }
40    }
41
42    if(Ax == 0 || Ax == n - 1 || Ay == 0 || Ay == m - 1)
43    {
44        cout << "YES\n";
45        cout << "0\n";
46        cout << '\n';
47        return 0;
48    }
49
50    long long dx[4] = {-1, 1, 0, 0};
51    long long dy[4] = {0, 0, -1, 1};
52
53    vector<vector<long long>> distanciaM(n, vector<long long>(m, -1));
54
55    queue<pair<long long, long long>> cola;
56

```

18.14 Monsters - BFS Multifuente + BFS de Escape (cont.)

```

57 for(pair<long long, long long> monstruo : monstruos)
58 {
59     distanciaM[monstruo.first][monstruo.second] = 0;
60     cola.push(monstruo);
61 }
62
63 while(!cola.empty())
64 {
65     long long x = cola.front().first;
66     long long y = cola.front().second;
67     cola.pop();
68
69     for(long long k = 0; k < 4; k++)
70     {
71         long long nx = x + dx[k];
72         long long ny = y + dy[k];
73
74         if(nx < 0 || nx >= n || ny < 0 || ny >= m)
75         {
76             continue;
77         }
78
79         if(matriz[nx][ny] == '#')
80         {
81             continue;
82         }
83
84         if(distanciaM[nx][ny] == -1)
85         {
86             distanciaM[nx][ny] = distanciaM[x][y] + 1;
87             cola.push({nx, ny});
88         }
89     }
90 }
91
92 vector<vector<long long>> distanciaA(n, vector<long long>(m, -1));
93 vector<vector<char>> direccion(n, vector<char>(m));
94
95 distanciaA[Ax][Ay] = 0;
96 cola.push({Ax, Ay});
97
98 long long salidaX = -1;
99 long long salidaY = -1;
100
101 while(!cola.empty())
102 {
103     long long x = cola.front().first;
104     long long y = cola.front().second;
105     cola.pop();
106
107     for(long long k = 0; k < 4; k++)
108     {
109         long long nx = x + dx[k];
110         long long ny = y + dy[k];
111
112         if(nx < 0 || nx >= n || ny < 0 || ny >= m)
113         {
114             continue;
115         }
116
117         if(matriz[nx][ny] == '#')
118         {
119             continue;
120         }
121
122         if(distanciaA[nx][ny] != -1)
123         {
124             continue;
125         }
126
127         long long nuevaDistancia = distanciaA[x][y] + 1;
128

```

18.14 Monsters - BFS Multifuente + BFS de Escape (cont.)

```

129     if(distanciaM[nx][ny] != -1 &&
130        nuevaDistancia >= distanciaM[nx][ny])
131     {
132         continue;
133     }
134
135     distanciaA[nx][ny] = nuevaDistancia;
136
137     if(k == 0)
138     {
139         direccion[nx][ny] = 'U';
140     }
141     else if(k == 1)
142     {
143         direccion[nx][ny] = 'D';
144     }
145     else if(k == 2)
146     {
147         direccion[nx][ny] = 'L';
148     }
149     else
150     {
151         direccion[nx][ny] = 'R';
152     }
153
154     if(nx == 0 || nx == n - 1 || ny == 0 || ny == m - 1)
155     {
156         salidaX = nx;
157         salidaY = ny;
158         break;
159     }
160
161     cola.push({nx, ny});
162 }
163
164 if(salidaX != -1)
165 {
166     break;
167 }
168
169 if(salidaX == -1)
170 {
171     cout << "NO\n";
172     return 0;
173 }
174
175 string recorridoFinal;
176
177 long long x = salidaX;
178 long long y = salidaY;
179
180 while(x != Ax || y != Ay)
181 {
182     char movimiento = direccion[x][y];
183
184     recorridoFinal += movimiento;
185
186     if(movimiento == 'R')
187     {
188         y--;
189     }
190     else if(movimiento == 'L')
191     {
192         y++;
193     }
194     else if(movimiento == 'D')
195     {
196         x--;
197     }
198     else
199     {
200

```

18.14 Monsters - BFS Multifuente + BFS de Escape (cont.)

```

201         x++;
202     }
203 }
204
205 reverse(recorridoFinal.begin(), recorridoFinal.end());
206
207 cout << "YES\n";
208 cout << recorridoFinal.size() << '\n';
209 cout << recorridoFinal << '\n';
210
211 return 0;
212 }

```

18.15 Planetqueriesi - Binary Lifting sobre Grafo Funcional

O: $(n \log k)$ de tabla y $(\log k)$ por consulta

Cada planeta tiene EXACTAMENTE un teletransporte de salida, así que la estructura es un grafo funcional

C++/CSES/Graph Algorithms/planetqueriesi.cpp

```

1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 int main()
7 {
8     ios::sync_with_stdio(false);
9     cin.tie(nullptr);
10
11     long long n, q;
12     cin >> n >> q;
13
14     const long long LOG = 31;
15
16     vector<vector<long long>> siguiente(LOG, vector<long long>(n));
17
18     for(long long i = 0; i < n; i++)
19     {
20         cin >> siguiente[0][i];
21         siguiente[0][i]--;
22     }
23
24     for(long long j = 1; j < LOG; j++)
25     {
26         for(long long i = 0; i < n; i++)
27         {
28             siguiente[j][i] = siguiente[j - 1][siguiente[j - 1][i]];
29         }
30     }
31
32     while(q--)
33     {
34         long long x, k;
35         cin >> x >> k;
36
37         x--;
38
39         for(long long j = 0; j < LOG; j++)
40         {
41             if(k & (1LL << j))
42             {
43                 x = siguiente[j][x];
44             }
45         }
46
47         cout << x + 1 << '\n';
48     }
49
50     return 0;
51 }

```

18.16 Round Trip - Detección de Ciclo con BFS

O: $(n + m)$, BFS excluyendo al padre inmediato

Hallar un ciclo en grafo no dirigido y reconstruirlo subiendo por los padres

C++/CSES/Graph Algorithms/Round Trip.cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <algorithm>
5
6 using namespace std;
7
8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     long long n, m;
14
15     cin >> n >> m;
16
17     vector<vector<long long>> matriz(n + 1);
18
19     long long a, b;
20
21     while(m--)
22     {
23         cin >> a >> b;
24
25         matriz[a].push_back(b);
26         matriz[b].push_back(a);
27     }
28
29     vector<long long> anterior(n + 1, -1);
30     vector<bool> recorrido(n + 1, false);
31
32     for(long long i = 1; i <= n; i++)
33     {
34         if(recorrido[i]) continue;
35
36         queue<long long> cola;
37
38         recorrido[i] = true;
39         cola.push(i);
40
41         while(!cola.empty())
42         {
43             long long x = cola.front();
44             cola.pop();
45
46             for(long long z : matriz[x])
47             {
48                 if(!recorrido[z])
49                 {
50                     recorrido[z] = true;
51                     anterior[z] = x;
52                     cola.push(z);
53                 }
54                 else if(anterior[x] != z)
55                 {
56                     vector<long long> recorridoX;
57                     vector<long long> recorridoZ;
58
59                     long long actual = x;
60
61                     while(actual != -1)
62                     {
63                         recorridoX.push_back(actual);
64
65                         if(actual == i)
66                         {
67                             break;

```

18.16 Round Trip - Detección de Ciclo con BFS (cont.)

```

88     }
89
90     actual = anterior[actual];
91 }
92
93 actual = z;
94
95 while(actual != -1)
96 {
97     recorridoZ.push_back(actual);
98
99     if(actual == i)
100     {
101         break;
102     }
103
104     actual = anterior[actual];
105 }
106
107 reverse(recorridoX.begin(), recorridoX.end());
108 reverse(recorridoZ.begin(), recorridoZ.end());
109
110 long long posicion = 0;
111
112 while(posicion < recorridoX.size() &&
113        posicion < recorridoZ.size() &&
114        recorridoX[posicion] == recorridoZ[posicion])
115 {
116     posicion++;
117 }
118
119 posicion--;
120
121 cout << recorridoX.size() + recorridoZ.size()
122      << " - 2 * posicion << '\n';
123
124 for(long long j = posicion; j < recorridoX.size(); j++)
125 {
126     cout << recorridoX[j] << ' ';
127 }
128
129 for(long long j = recorridoZ.size() - 1; j >= posicion; j--)
130 {
131     cout << recorridoZ[j] << ' ';
132 }
133
134 cout << '\n';
135
136 return 0;
137 }
138 }
139 }
140 }
141
142 cout << "IMPOSSIBLE\n";
143
144 return 0;
145 }

```

18.17 Round Trip (Stack) - Detección de Ciclo con Pila (DFS Iterativo)

O: (n + m), DFS iterativo con pila explícita

El mismo problema que Round Trip pero cambiando la cola por una PILA, o sea DFS en vez de BFS

C++/CSES/Graph Algorithms/Round Trip (Stack).cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <stack>
4 #include <algorithm>
5
6 using namespace std;
7

```

18.17 Round Trip (Stack) - Detección de Ciclo con Pila (DFS Iterativo) (cont.)

```

8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     long long n, m;
14
15     cin >> n >> m;
16
17     vector<vector<long long>> matriz(n + 1);
18
19     long long a, b;
20
21     while(m--)
22     {
23         cin >> a >> b;
24
25         matriz[a].push_back(b);
26         matriz[b].push_back(a);
27     }
28
29     vector<long long> anterior(n + 1, -1);
30     vector<bool> recorrido(n + 1, false);
31
32     for(long long i = 1; i <= n; i++)
33     {
34         if(recorrido[i]) continue;
35
36         stack<long long> pila;
37
38         recorrido[i] = true;
39         pila.push(i);
40
41         while(!pila.empty())
42         {
43             long long x = pila.top();
44             pila.pop();
45
46             for(long long z : matriz[x])
47             {
48                 if(!recorrido[z])
49                 {
50                     recorrido[z] = true;
51                     anterior[z] = x;
52                     pila.push(z);
53                 }
54                 else if(anterior[x] != z)
55                 {
56                     vector<long long> recorridoX;
57                     vector<long long> recorridoZ;
58
59                     long long actual = x;
60
61                     while(actual != -1)
62                     {
63                         recorridoX.push_back(actual);
64
65                         if(actual == i)
66                         {
67                             break;
68                         }
69
70                         actual = anterior[actual];
71                     }
72
73                     actual = z;
74
75                     while(actual != -1)
76                     {
77                         recorridoZ.push_back(actual);
78
79                         if(actual == i)

```

18.17 Round Trip (Stack) - Deteccion de Ciclo con Pila (DFS Iterativo) (cont.)

```

80         {
81             break;
82         }
83
84         actual = anterior[actual];
85     }
86
87     reverse(recorridoX.begin(), recorridoX.end());
88     reverse(recorridoZ.begin(), recorridoZ.end());
89
90     long long posicion = 0;
91
92     while(posicion < recorridoX.size() &&
93           posicion < recorridoZ.size() &&
94           recorridoX[posicion] == recorridoZ[posicion])
95     {
96         posicion++;
97     }
98
99     posicion--;
100
101     cout << recorridoX.size() + recorridoZ.size()
102          << " - 2 * posicion << '\n';
103
104     for(long long j = posicion; j < recorridoX.size(); j++)
105     {
106         cout << recorridoX[j] << ' ';
107     }
108
109     for(long long j = recorridoZ.size() - 1; j >= posicion; j--)
110     {
111         cout << recorridoZ[j] << ' ';
112     }
113
114     cout << '\n';
115
116     return 0;
117 }
118 }
119 }
120 }
121
122 cout << "IMPOSSIBLE\n";
123
124 return 0;
125 }

```

18.18 Round Trip II - Ciclo en Grafo Dirigido (DFS de Tres Colores)

O: (n + m), DFS de tres colores

Encontrar un ciclo en un grafo DIRIGIDO con DFS de tres colores

C++/CSES/Graph Algorithms/Round Trip II.cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <stack>
4 #include <algorithm>
5
6 using namespace std;
7
8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     long long n, m;
14
15     cin >> n >> m;
16
17     vector<vector<long long>> matriz(n + 1);
18
19     long long a, b;

```

18.18 Round Trip II - Ciclo en Grafo Dirigido (DFS de Tres Colores) (cont.)

```

20
21     while (m--)
22     {
23         cin >> a >> b;
24         matriz[a].push_back(b);
25     }
26
27     vector<long long> anterior(n + 1, -1);
28     vector<long long> estado(n + 1, 0);
29
30     for (long long i = 1; i <= n; i++)
31     {
32         if (estado[i] != 0) continue;
33
34         stack<pair<long long, long long>> pila;
35
36         estado[i] = 1;
37         pila.push({i, 0});
38
39         while (!pila.empty())
40         {
41             long long x = pila.top().first;
42             long long posicion = pila.top().second;
43
44             if (posicion == matriz[x].size())
45             {
46                 estado[x] = 2;
47                 pila.pop();
48                 continue;
49             }
50
51             long long z = matriz[x][posicion];
52
53             pila.top().second++;
54
55             if (estado[z] == 0)
56             {
57                 anterior[z] = x;
58                 estado[z] = 1;
59                 pila.push({z, 0});
60             }
61             else if (estado[z] == 1)
62             {
63                 vector<long long> ciclo;
64
65                 ciclo.push_back(z);
66
67                 long long actual = x;
68
69                 while (actual != z)
70                 {
71                     ciclo.push_back(actual);
72                     actual = anterior[actual];
73                 }
74
75                 ciclo.push_back(z);
76                 reverse(ciclo.begin(), ciclo.end());
77
78                 cout << ciclo.size() << '\n';
79
80                 for (long long ciudad : ciclo)
81                     cout << ciudad << ' ';
82
83                 cout << '\n';
84
85                 return 0;
86             }
87         }
88     }
89
90     cout << "IMPOSSIBLE\n";
91
92

```

18.18 Round Trip II - Ciclo en Grafo Dirigido (DFS de Tres Colores) (cont.)

```
92     return 0;
93 }
```

18.19 Shortest Routes I - Dijkstra con Priority Queue

O: ($m \log n$), Dijkstra con heap

Dijkstra clasico en O($m \log n$) con priority_queue de minimos

C++/CSES/Graph Algorithms/Shortest Routes I.cpp

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <climits>
5
6 using namespace std;
7
8 int main()
9 {
10     // INPUT
11     long long n, m;
12     cin >> n >> m;
13
14     vector<vector<pair<long long, long long>>> matriz(n + 1);
15
16     for(long long i = 0; i < m; i++)
17     {
18         long long a, b, c;
19         cin >> a >> b >> c;
20
21         matriz[a].push_back({b, c});
22     }
23
24     // OUTPUT
25     vector<long long> distancia(n + 1, LLONG_MAX);
26     priority_queue<pair<long long, long long>,
27                 vector<pair<long long, long long>>,
28                 greater<pair<long long, long long>>> cola;
29
30     distancia[1] = 0;
31     cola.push({0, 1});
32
33     while(!cola.empty())
34     {
35         long long dist = cola.top().first;
36         long long actual = cola.top().second;
37         cola.pop();
38
39         if(dist > distancia[actual])
40             continue;
41
42         for(auto vecino : matriz[actual])
43         {
44             long long siguiente = vecino.first;
45             long long peso = vecino.second;
46
47             if(distancia[actual] + peso < distancia[siguiente])
48             {
49                 distancia[siguiente] = distancia[actual] + peso;
50                 cola.push({distancia[siguiente], siguiente});
51             }
52         }
53     }
54
55     for(long long i = 1; i < n+1; i++)
56     {
57         cout << distancia[i];
58         if(i < n) cout << " ";
59     }
60
61     cout << endl;
62
63     return 0;
64 }
```

18.19 Shortest Routes I - Dijkstra con Priority Queue (cont.)

```
54 }
```

18.20 Shortest Routes II - Dijkstra desde Cada Nodo

O: ($n^2 m \log n$), un Dijkstra por nodo

Distancias entre TODOS los pares resueltas con n Dijkstras, uno por nodo

C++/CSES/Graph Algorithms/Shortest Routes II.cpp

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <climits>
5
6 using namespace std;
7
8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12     // INPUT
13     long long n, m, q;
14     cin >> n >> m >> q;
15
16     vector<vector<pair<long long, long long>>> conexion(n);
17
18     vector<vector<long long>> conexiondirecta(
19         n, vector<long long>(n, LLONG_MAX));
20
21     for(long long i = 0; i < m; i++)
22     {
23         long long a, b, c;
24         cin >> a >> b >> c;
25
26         a--;
27         b--;
28
29         if(conexiondirecta[a][b] > c)
30         {
31             conexiondirecta[a][b] = c;
32             conexiondirecta[b][a] = c;
33         }
34     }
35
36     for(long long i = 0; i < n; i++)
37     {
38         for(long long j = i + 1; j < n; j++)
39         {
40             if(conexiondirecta[i][j] != LLONG_MAX)
41             {
42                 conexion[i].push_back({j, conexiondirecta[i][j]});
43                 conexion[j].push_back({i, conexiondirecta[i][j]});
44             }
45         }
46     }
47
48     // OUTPUT
49     vector<vector<long long>> camino(n, vector<long long>(n, -1));
50     vector<bool> dijkstrados(n, false);
51
52     for(long long x = 0; x < n; x++)
53     {
54         if(conexion[x].size() == 1)
55         {
56             long long z = conexion[x][0].first;
57             long long distanciaXZ = conexion[x][0].second;
58
59             if(!dijkstrados[z])
60             {
61                 vector<long long> distancia(n, LLONG_MAX);
62
63                 priority_queue<
64                     pair<long long, long long>,
65                     vector<pair<long long, long long>>,
66                     greater<pair<long long, long long>>> cola;
67
68                 distancia[z] = 0;
69                 cola.push({0, z});
70
71                 while(!cola.empty())
72                 {
73                     long long dist = cola.top().first;
74                     long long actual = cola.top().second;
75                     cola.pop();
76
77                     if(dist > distancia[actual])
78                         continue;
79
80                     for(auto vecino : conexion[actual])
81                     {
82                         long long siguiente = vecino.first;
83                         long long peso = vecino.second;
84
85                         if(distancia[actual] + peso < distancia[siguiente])
86                         {
87                             distancia[siguiente] = distancia[actual] + peso;
88                             cola.push({distancia[siguiente], siguiente});
89                         }
90                     }
91                 }
92
93                 camino[x][z] = distancia[z];
94                 dijkstrados[z] = true;
95             }
96         }
97     }
98
99     for(long long i = 0; i < n; i++)
100     {
101         for(long long j = i + 1; j < n; j++)
102         {
103             if(camino[i][j] != -1)
104                 cout << camino[i][j] << " ";
105             else
106                 cout << "-1 ";
107         }
108         cout << endl;
109     }
110
111     return 0;
112 }
```

18.20 Shortest Routes II - Dijkstra desde Cada Nodo (cont.)

```

85         vector<pair<long long, long long>>,
86         greater<pair<long long, long long>>
87         > cola;
88
89         distancia[z] = 0;
90         cola.push({0, z});
91
92         while(!cola.empty())
93         {
94             long long distanciaActual = cola.top().first;
95             long long actual = cola.top().second;
96             cola.pop();
97
98             if(distanciaActual > distancia[actual])
99                 continue;
100
101             for(auto vecino : conexion[actual])
102             {
103                 long long siguiente = vecino.first;
104                 long long peso = vecino.second;
105
106                 if(distancia[actual] + peso < distancia[siguiente])
107                 {
108                     distancia[siguiente] = distancia[actual] + peso;
109                     cola.push({distancia[siguiente], siguiente});
110                 }
111             }
112         }
113
114         for(long long i = 0; i < n; i++)
115         {
116             if(distancia[i] != LLONG_MAX)
117                 camino[z][i] = distancia[i];
118         }
119
120         dijkstrados[z] = true;
121     }
122 }
123
124 for(long long x = 0; x < n; x++)
125 {
126     if(!dijkstrados[x])
127     {
128         vector<long long> distancia(n, LLONG_MAX);
129
130         priority_queue<
131             pair<long long, long long>,
132             vector<pair<long long, long long>>,
133             greater<pair<long long, long long>>
134             > cola;
135
136         distancia[x] = 0;
137         cola.push({0, x});
138
139         while(!cola.empty())
140         {
141             long long distanciaActual = cola.top().first;
142             long long actual = cola.top().second;
143             cola.pop();
144
145             if(distanciaActual > distancia[actual])
146                 continue;

```

18.20 Shortest Routes II - Dijkstra desde Cada Nodo (cont.)

```

87         for(auto vecino : conexion[actual])
88         {
89             long long siguiente = vecino.first;
90             long long peso = vecino.second;
91
92             if(distancia[actual] + peso < distancia[siguiente])
93             {
94                 distancia[siguiente] = distancia[actual] + peso;
95                 cola.push({distancia[siguiente], siguiente});
96             }
97         }
98     }
99
100     for(long long i = 0; i < n; i++)
101     {
102         if(distancia[i] != LLONG_MAX)
103             camino[x][i] = distancia[i];
104     }
105
106     dijkstrados[x] = true;
107 }
108 }
109
110 // INPUT
111 for(long long i = 0; i < q; i++)
112 {
113     long long a, b;
114     cin >> a >> b;
115
116     cout << camino[a - 1][b - 1] << '\n';
117 }
118
119 return 0;
120 }

```

18.21 Shortest Routes II (Floyd Warshall) - Floyd-Warshall

O: (n^3) tiempo y (n^2) memoria

Floyd-Warshall, la solución canónica de todos-contras-todos

C++/CSES/Graph Algorithms/Shortest Routes II (Floyd-Warshall).cpp

```

1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 int main()
7 {
8     ios::sync_with_stdio(false);
9     cin.tie(nullptr);
10
11     long long n, m, q;
12     cin >> n >> m >> q;
13
14     const long long INF = 1e18;
15
16     vector<vector<long long>> dist(n, vector<long long>(n, INF));
17
18     for(long long i = 0; i < n; i++)
19     {
20         dist[i][i] = 0;
21     }
22
23     for(long long i = 0; i < m; i++)
24     {
25         long long a, b, c;
26         cin >> a >> b >> c;
27
28         a--;
29         b--;
30

```

18.21 Shortest Routes II (Floyd Warshall) - Floyd-Warshall (cont.)

```

31 // puede haber varias carreteras entre a y b: me quedo con la menor
32 if(c < dist[a][b])
33 {
34     dist[a][b] = c;
35     dist[b][a] = c;
36 }
37 }
38
39 // Floyd-Warshall
40 for(long long k = 0; k < n; k++) // nodo intermedio
41 {
42     for(long long i = 0; i < n; i++)
43     {
44         if(dist[i][k] == INF) // no llego a k desde i
45         {
46             continue;
47         }
48
49         for(long long j = 0; j < n; j++)
50         {
51             if(dist[i][k] + dist[k][j] < dist[i][j])
52             {
53                 dist[i][j] = dist[i][k] + dist[k][j];
54             }
55         }
56     }
57 }
58
59 for(long long i = 0; i < q; i++)
60 {
61     long long a, b;
62     cin >> a >> b;
63
64     a--;
65     b--;
66
67     if(dist[a][b] == INF)
68     {
69         cout << -1 << '\n';
70     }
71     else
72     {
73         cout << dist[a][b] << '\n';
74     }
75 }
76
77 return 0;
78 }

```

19 - CSES: Introductory Problems

19.1 Apple Divisions - Fuerza Bruta 2^n O: (2^n), todos los subconjuntosN <= 20, o sea $2^{20} = \sim 10^6$ subconjuntos: cabe de sobra, as  que se prueban todos

C++/CSES/Introductory Problems/Apple Divisions.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 long long ans = LLONG_MAX;
5 long long total = 0;
6 vector<long long> a;
7
8 void solve(int pos, long long sum) {
9     if (pos == a.size()) {
10         ans = min(ans, abs(total - 2 * sum));
11         return;
12     }
13
14     solve(pos + 1, sum + a[pos]);
15     solve(pos + 1, sum);

```

19.1 Apple Divisions - Fuerza Bruta 2^n (cont.)

```

6 }
7
8 int main() {
9     int n;
10    cin >> n;
11
12    a.resize(n);
13
14    for (auto &x : a) {
15        cin >> x;
16        total += x;
17    }
18
19    solve(0, 0);
20
21    cout << ans << '\n';
22 }

```

19.2 Bit Strings - Exponenciacion Modular Rapida

O: ($\log n$), exponenciacion rapida $2^n \bmod (10^9+7)$ con exponenciacion binaria

C++/CSES/Introductory Problems/Bit Strings.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const long long MOD = 1e9 + 7;
5
6 long long modpow(long long a, long long b) {
7     long long res = 1;
8
9     while (b) {
10         if (b & 1)
11             res = res * a % MOD;
12
13         a = a * a % MOD;
14         b >>= 1;
15     }
16
17     return res;
18 }
19
20 int main() {
21     long long n;
22     cin >> n;
23
24     cout << modpow(2, n) << '\n';
25 }

```

19.3 Chessboard And Queens - Backtracking con Poda por Diagonales

O: ($8!$) acotado por las podas de diagonales

Las 8 reinas con backtracking fila por fila, que es lo que baja el espacio de busqueda

C++/CSES/Introductory Problems/Chessboard and Queens.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 char board[8][8];
5 bool col[8], diag1[15], diag2[15];
6 int ans = 0;
7
8 void solve(int row) {
9     if (row == 8) {
10         ans++;
11         return;
12     }
13
14     for (int c = 0; c < 8; c++) {
15         if (board[row][c] == '*')
16             continue;
17

```

19.3 Chessboard And Queens - Backtracking con Poda por Diagonales (cont.)

```

8         if (col[c] || diag1[row - c + 7] || diag2[row + c])
9             continue;
20
21         col[c] = true;
22         diag1[row - c + 7] = true;
23         diag2[row + c] = true;
24
25         solve(row + 1);
26
27         col[c] = false;
28         diag1[row - c + 7] = false;
29         diag2[row + c] = false;
30     }
31 }
32
33 int main() {
34     for (int i = 0; i < 8; i++)
35         for (int j = 0; j < 8; j++)
36             cin >> board[i][j];
37
38     solve(0);
39
40     cout << ans << '\n';
41 }

```

19.4 Coin Piles - Invariantes Aritmeticos

O: (1) por caso, son dos invariantes

Sin simular nada: dos condiciones cerradas

C++/CSES/Introductory Problems/Coin Piles.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int t;
9     cin >> t;
10
11     while (t--) {
12         long long a, b;
13         cin >> a >> b;
14
15         if ((a + b) % 3 == 0 && max(a, b) <= 2 * min(a, b))
16             cout << "YES\n";
17         else
18             cout << "NO\n";
19     }
20 }

```

19.5 Creating Strings - Backtracking sobre Frecuencias

O: (cantidad de permutaciones * n)

Generar todas las permutaciones DISTINTAS

C++/CSES/Introductory Problems/Creating Strings.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<string> ans;
5 string s;
6 int freq[26];
7
8 void solve(string cur) {
9     if (cur.size() == s.size()) {
10         ans.push_back(cur);
11         return;
12     }
13
14     for (int i = 0; i < 26; i++) {
15         if (freq[i] == 0) continue;

```

19.5 Creating Strings - Backtracking sobre Frecuencias (cont.)

```

6         freq[i]--;
7         solve(cur + char('a' + i));
8         freq[i]++;
9     }
10 }
11
12 int main() {
13     cin >> s;
14
15     for (char c : s)
16         freq[c - 'a']++;
17
18     solve("");
19
20     cout << ans.size() << '\n';
21
22     for (auto &x : ans)
23         cout << x << '\n';
24 }

```

19.6 Digit Queries - Búsqueda por Bloques de Longitud

O: (log k), saltando por bloques de longitud

La cadena infinita 123456789101112... se recorre por BLOQUES segun la cantidad de digitos

C++/CSES/Introductory Problems/Digit Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int q;
6     cin >> q;
7
8     while (q--) {
9         long long k;
10        cin >> k;
11
12        long long digits = 1;
13        long long count = 9;
14        long long start = 1;
15
16        while (k > digits * count) {
17            k -= digits * count;
18            digits++;
19            count *= 10;
20            start *= 10;
21        }
22
23        long long num = start + (k - 1) / digits;
24        long long pos = (k - 1) % digits;
25
26        string s = to_string(num);
27
28        cout << s[pos] << '\n';
29    }
30 }

```

19.7 Gray Code - Identidad i XOR (i >> 1)

O: (2^n), que es el tamaño de la salida

El código Gray de i es exactamente i ^ (i >> 1)

C++/CSES/Introductory Problems/Gray Code.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7
8     for (int i = 0; i < (1 << n); i++) {
9         int x = i ^ (i >> 1);

```


19.10 Increasing Array - Greedy de Maximo Prefijo (cont.)

```
25
26     return 0;
27 }
```

19.11 Knight Moves Grid - BFS en Grilla

O: (n^2), BFS en la grilla

BFS de caballo sobre la grilla: la distancia en SALTOS es el numero de aristas

C++/CSES/Introductory Problems/Knight Moves Grid.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7
8     vector<vector<int>> dist(n, vector<int>(n, -1));
9
10    int dx[] = {2, 2, -2, -2, 1, 1, -1, -1};
11    int dy[] = {1, -1, 1, -1, 2, -2, 2, -2};
12
13    queue<pair<int, int>> q;
14
15    q.push({0, 0});
16    dist[0][0] = 0;
17
18    while (!q.empty()) {
19        auto [x, y] = q.front();
20        q.pop();
21
22        for (int k = 0; k < 8; k++) {
23            int nx = x + dx[k];
24            int ny = y + dy[k];
25
26            if (nx < 0 || nx >= n || ny < 0 || ny >= n)
27                continue;
28
29            if (dist[nx][ny] != -1)
30                continue;
31
32            dist[nx][ny] = dist[x][y] + 1;
33            q.push({nx, ny});
34        }
35    }
36
37    for (int i = 0; i < n; i++) {
38        for (int j = 0; j < n; j++) {
39            cout << dist[i][j] << ' ';
40        }
41        cout << '\n';
42    }
43 }
```

19.12 Mex Grid Construction - Identidad MEX = XOR

O: (n^2), que es el tamaño de la salida

Se pide la grilla donde cada celda es el MEX de lo que ya hay a su izquierda en la fila y arriba en la...

C++/CSES/Introductory Problems/Mex Grid Construction.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7
8     for (int i = 0; i < n; i++) {
9         for (int j = 0; j < n; j++) {
10             cout << (i ^ j) << ' ';
11         }
12         cout << '\n';
13     }
14 }
```

19.13 Missing Number - Suma de Gauss

O: (n), suma de Gauss menos la suma real

La suma de 1..n se sabe de formula, $n*(n+1)/2$

C++/CSES/Introductory Problems/Missing Number.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     long long n; cin >> n;
7     long long god = 1;
8     long long pre = 0;
9     for(int i = 0 ; i < n - 1 ; i++)
10    {
11        god += i + 2;
12        long long a;
13        cin >> a;
14        pre += a;
15    }
16    cout << god-pre << endl;
17
18    return 0;
19 }
```

19.14 Number Spiral - Formula por Capas

O: (1) por consulta, formula por capas

La espiral se lee por CAPAS: la capa k son las celdas con $\max(\text{fila}, \text{columna}) = k$

C++/CSES/Introductory Problems/Number Spiral.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int t;
9     cin >> t;
10
11    while (t--) {
12        long long y, x;
13        cin >> y >> x;
14
15        if (y > x) {
16            if (y % 2 == 0)
17                cout << y * y - x + 1 << '\n';
18            else
19                cout << (y - 1) * (y - 1) + x << '\n';
20        } else {
21            if (x % 2 == 0)
22                cout << (x - 1) * (x - 1) + y << '\n';
23            else
24                cout << x * x - y + 1 << '\n';
25        }
26    }
27 }
```

19.15 Palindrome Reorder - Conteo de Frecuencias

O: ($n + 26$), frecuencias y armar las dos mitades

Un palindromo se puede armar si y solo si a lo sumo UNA letra tiene frecuencia impar

C++/CSES/Introductory Problems/Palindrome Reorder.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     string s;
6     cin >> s;
7
8     vector<int> freq(26);
```

19.15 Palindrome Reorder - Conteo de Frecuencias (cont.)

```
9
10 for (char c : s)
11     freq[c - 'A']++;
12
13 int odd = 0;
14 for (int x : freq)
15     odd += x % 2;
16
17 if (odd > 1) {
18     cout << "NO SOLUTION\n";
19     return 0;
20 }
21
22 string half, mid;
23
24 for (int i = 0; i < 26; i++) {
25     half += string(freq[i] / 2, 'A' + i);
26
27     if (freq[i] % 2)
28         mid += char('A' + i);
29 }
30
31 string rev = half;
32 reverse(rev.begin(), rev.end());
33
34 cout << half << mid << rev << '\n';
35 }
```

19.16 Permutations - Construcc n Directa

O: (n), construcc n directa

En vez de buscar, se CONSTRUYE: primero todos los pares y despu s todos los impares

C++/CSES/Introductory Problems/Permutations.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9     cin >> n;
10
11     if (n == 2 || n == 3) {
12         cout << "NO SOLUTION\n";
13         return 0;
14     }
15
16     for (int i = 2; i <= n; i += 2)
17         cout << i << ' ';
18
19     for (int i = 1; i <= n; i += 2)
20         cout << i << ' ';
21
22     cout << '\n';
23 }
```

19.17 Rab Game I - Construcc n por Rotaci n de Bloque

O: (n), rotar un bloque

Construcc n con dos condiciones de imposibilidad

C++/CSES/Introductory Problems/Rab Game I.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int t;
6     cin >> t;
7
8     while (t--) {
9         int n, a, b;
```

19.17 Rab Game I - Construcc n por Rotaci n de Bloque (cont.)

```
10     cin >> n >> a >> b;
11
12     if (a + b > n || (a == 0) != (b == 0)) {
13         cout << "NO\n";
14         continue;
15     }
16
17     cout << "YES\n";
18
19     for (int i = 1; i <= n; i++)
20         cout << i << ' ';
21     cout << '\n';
22
23     int m = a + b;
24
25     for (int i = a + 1; i <= m; i++)
26         cout << i << ' ';
27
28     for (int i = 1; i <= a; i++)
29         cout << i << ' ';
30
31     for (int i = m + 1; i <= n; i++)
32         cout << i << ' ';
33
34     cout << '\n';
35 }
36 }
```

19.18 Repetitions - Barrido Lineal de Rachas

O: (n), un barrido de rachas

Una pasada llevando el largo de la racha actual y el m ximo visto

C++/CSES/Introductory Problems/Repetitions.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     string s; cin >> s;
7     long long maxi = 0;
8     char ant = '-';
9     long long temp = 0;
10    for(int i = 0 ; i < (int) s.size() ; i++)
11    {
12        if(s[i]== ant)
13        {
14            temp++;
15        }
16        else
17        {
18            if(temp>maxi)
19            {
20                maxi = temp;
21            }
22            temp =1;
23            ant = s[i];
24        }
25    }
26    if(temp>maxi)
27    {
28        maxi = temp;
29    }
30    cout << maxi << endl;
31
32    return 0;
33 }
```

19.19 String Reorder - Greedy Lexicografico con Cota de Factibilidad**O:** ($n * 26$), greedy lexicografico con cota de factibilidad

Se arma la respuesta caracter por caracter tomando siempre la letra mas chica posible

C++/CSES/Introductory Problems/String Reorder.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     string s;
6     cin >> s;
7
8     int freq[26] = {};
9
10    for (char c : s)
11        freq[c - 'A']++;
12
13    string ans;
14    char last = '#';
15
16    for (int pos = 0; pos < s.size(); pos++) {
17        bool found = false;
18
19        for (int c = 0; c < 26; c++) {
20            if (freq[c] == 0 || 'A' + c == last)
21                continue;
22
23            freq[c]--;
24
25            int remaining = s.size() - pos - 1;
26            int mx = 0;
27
28            for (int i = 0; i < 26; i++)
29                mx = max(mx, freq[i]);
30
31            if (mx <= (remaining + 1) / 2) {
32                ans += char('A' + c);
33                last = 'A' + c;
34                found = true;
35                break;
36            }
37
38            freq[c]++;
39        }
40
41        if (!found) {
42            cout << -1 << '\n';
43            return 0;
44        }
45
46        cout << ans << '\n';
47    }
48 }

```

19.20 Tower Of Hanoi - Recursion Clasica**O:** (2^n), que es la cantidad de movimientos

La recursion de siempre: mover n-1 al auxiliar, mover el disco grande al destino

C++/CSES/Introductory Problems/Tower of Hanoi.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void hanoi(int n, int origen, int destino, int auxiliar) {
5     if (n == 0) return;
6
7     hanoi(n - 1, origen, auxiliar, destino);
8     cout << origen << ' ' << destino << '\n';
9     hanoi(n - 1, auxiliar, destino, origen);
10 }
11
12 int main() {
13     int n;

```

19.20 Tower Of Hanoi - Recursion Clasica (cont.)

```

14     cin >> n;
15
16     cout << (1 << n) - 1 << '\n';
17
18     hanoi(n, 1, 3, 2);
19 }

```

19.21 Trailing Zeros - Formula de Legendre**O:** ($\log n$), formula de Legendre dividiendo entre potencias de 5
Ceros al final de n!: los deciden los factores 5, por la formula de Legendre

C++/CSES/Introductory Problems/Trailing Zeros.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     long long n;
6     cin >> n;
7
8     long long ans = 0;
9
10    while (n) {
11        n /= 5;
12        ans += n;
13    }
14
15    cout << ans << '\n';
16 }

```

19.22 Two Knights - Conteo por Complemento**O:** (n), una formula por tablero

Contar por el complemento: total de parejas menos las que se atacan

C++/CSES/Introductory Problems/Two Knights.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     long long n;
6     cin >> n;
7
8     for (long long k = 1; k <= n; k++) {
9         long long total = k * k * (k * k - 1) / 2;
10        long long attack = 4 * (k - 1) * (k - 2);
11
12        cout << total - attack << '\n';
13    }
14 }

```

19.23 Two Sets - Construccin por Bloques de 4**O:** (n), construccion por bloques de 4La suma total es $n*(n+1)/2$, y para partirla en dos mitades iguales tiene que ser PAR

C++/CSES/Introductory Problems/Two Sets.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7
8     if (n % 4 == 1 || n % 4 == 2) {
9         cout << "NO\n";
10        return 0;
11    }
12
13    vector<int> a, b;
14
15    if (n % 4 == 3) {
16        a.push_back(1);

```

19.23 Two Sets - Construcción por Bloques de 4 (cont.)

```

17     a.push_back(2);
18     b.push_back(3);
19
20     for (int i = 4; i <= n; i += 4) {
21         a.push_back(i);
22         a.push_back(i + 3);
23         b.push_back(i + 1);
24         b.push_back(i + 2);
25     }
26 } else {
27     for (int i = 1; i <= n; i += 4) {
28         a.push_back(i);
29         a.push_back(i + 3);
30         b.push_back(i + 1);
31         b.push_back(i + 2);
32     }
33 }
34
35 cout << "YES\n";
36
37 cout << a.size() << '\n';
38 for (int x : a) cout << x << ' ';
39 cout << '\n';
40
41 cout << b.size() << '\n';
42 for (int x : b) cout << x << ' ';
43 cout << '\n';
44 }

```

19.24 Weird Algorithm - Simulación Directa

O: (largo de la secuencia de Collatz), que no se sabe acotar
 Simular el proceso de Collatz tal cual lo dice el enunciado, sin mas

C++/CSes/Introductory Problems/Weird Algorithm.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      long long n; cin >> n;
7      cout << n << " ";
8      while(n!=1)
9      {
10         if(n%2==0)
11         {
12             n=n/2;
13         }
14         else
15         {
16             n=(n*3)+1;
17         }
18         cout << n << " ";
19     }
20
21     return 0;
22 }

```

20 - CSes: Mathematics

20.1 Counting Coprimes Pairs - Mobius (Inversión para Contar Coprimos)

O: (maxValue log maxValue) la suma armonica, (maxValue) la criba lineal

Uso: mu = mobius(maxValue); ans = suma de mu[d] * C(multiplos de d, 2)

Cuántos pares del arreglo son coprimos entre si

C++/CSes/Mathematics/Counting Coprimes Pairs.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5

```

20.1 Counting Coprimes Pairs - Mobius (Inversión para Contar Coprimos) (cont.)

```

6  const int MAXN = 1e6;
7
8  // -----
9  // Funcion de Mobius usando criba lineal
10 // -----
11
12 vector<int> mobius(int n)
13 {
14     vector<int> mu(n + 1);
15     vector<int> primes;
16     vector<bool> composite(n + 1, false);
17
18     mu[1] = 1;
19
20     for(int i = 2; i <= n; i++)
21     {
22         if(!composite[i])
23         {
24             primes.push_back(i);
25             mu[i] = -1;
26         }
27
28         for(int p : primes)
29         {
30             if(i * p > n)
31                 break;
32
33             composite[i * p] = true;
34
35             if(i % p == 0)
36             {
37                 mu[i * p] = 0;
38                 break;
39             }
40
41             mu[i * p] = -mu[i];
42         }
43     }
44
45     return mu;
46 }
47
48 // -----
49 // Cuenta cuantos elementos del arreglo son divisibles
50 // por d
51 // -----
52
53 int countMultiples(int d, int maxValue, vector<int>& freq)
54 {
55     int cnt = 0;
56
57     for(int x = d; x <= maxValue; x += d)
58         cnt += freq[x];
59
60     return cnt;
61 }
62
63 // -----
64 // Cuenta los pares coprimos
65 // -----
66
67 ll countCoprimePairs(vector<int>& freq, vector<int>& mu, int maxValue)
68 {
69     ll ans = 0;
70
71     for(int d = 1; d <= maxValue; d++)
72     {
73         if(mu[d] == 0)
74             continue;
75
76         int cnt = countMultiples(d, maxValue, freq);
77

```

20.1 Counting Coprimes Pairs - Mobius (Inversion para Contar Coprimos) (cont.)

```

78     ll pairs = 1LL * cnt * (cnt - 1) / 2;
79
80     ans += 1LL * mu[d] * pairs;
81 }
82
83 return ans;
84 }
85
86 // -----
87 // MAIN
88 // -----
89
90 int main()
91 {
92     ios::sync_with_stdio(false);
93     cin.tie(nullptr);
94
95     int n;
96     cin >> n;
97
98     vector<int> freq(MAXN + 1);
99
100    int maxValue = 0;
101
102    for(int i = 0; i < n; i++)
103    {
104        int x;
105        cin >> x;
106
107        freq[x]++;
108        maxValue = max(maxValue, x);
109    }
110
111    vector<int> mu = mobius(maxValue);
112
113    cout << countCoprimePairs(freq, mu, maxValue) << '\n';
114
115    return 0;
116 }

```

21 - CSSES: Range Queries

21.1 Distinct Value Queries II - Segment Tree sobre Ocurrencia Anterior (cont.)

O: (log n) por operacion

Uso: SegmentTree sobre el anterior-igual; Operation guarda el historial

Saber si en [x,y] hay valores repetidos, y la idea es preciosa

C++/CSSES/Range Queries/Distinct Value Queries II.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int INF = 1e9;
5
6  struct SegmentTree {
7      int n;
8      vector<int> tree;
9
10     SegmentTree(int n) : n(n), tree(4 * n, INF) {}
11
12     void update(int node, int l, int r, int pos, int val) {
13         if (l == r) {
14             tree[node] = val;
15             return;
16         }
17
18         int mid = (l + r) / 2;
19
20         if (pos <= mid)
21             update(node * 2, l, mid, pos, val);

```

21.1 Distinct Value Queries II - Segment Tree sobre Ocurrencia Anterior (cont.)

```

22     else
23         update(node * 2 + 1, mid + 1, r, pos, val);
24
25     tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
26 }
27
28 void update(int pos, int val) {
29     update(1, 1, n, pos, val);
30 }
31
32 int query(int node, int l, int r, int ql, int qr) {
33     if (qr < l || r < ql)
34         return INF;
35
36     if (ql <= l && r <= qr)
37         return tree[node];
38
39     int mid = (l + r) / 2;
40
41     return min(
42         query(node * 2, l, mid, ql, qr),
43         query(node * 2 + 1, mid + 1, r, ql, qr)
44     );
45 }
46
47 int query(int l, int r) {
48     return query(1, 1, n, l, r);
49 }
50 };
51
52 struct Operation {
53     int type, x, y;
54 };
55
56 int main() {
57     ios::sync_with_stdio(false);
58     cin.tie(nullptr);
59
60     int n, q;
61     cin >> n >> q;
62
63     vector<int> a(n + 1);
64     vector<int> values;
65
66     for (int i = 1; i <= n; i++) {
67         cin >> a[i];
68         values.push_back(a[i]);
69     }
70
71     vector<Operation> ops(q);
72
73     for (auto &[type, x, y] : ops) {
74         cin >> type >> x >> y;
75
76         if (type == 1)
77             values.push_back(y);
78     }
79
80     sort(values.begin(), values.end());
81     values.erase(unique(values.begin(), values.end()), values.end());
82
83     auto get = [&](int x) {
84         return lower_bound(values.begin(), values.end(), x)
85             - values.begin();
86     };
87
88     for (int i = 1; i <= n; i++)
89         a[i] = get(a[i]);
90
91     int m = values.size();
92
93     vector<set<int>> pos(m);

```

21.1 Distinct Value Queries II - Segment Tree sobre Ocurrencia Anterior (cont.)

```

94
95     for (int i = 1; i <= n; i++)
96         pos[a[i]].insert(i);
97
98     vector<int> nxt(n + 1, INF);
99
100    for (int x = 0; x < m; x++) {
101        auto &s = pos[x];
102
103        for (auto it = s.begin(); it != s.end(); ++it) {
104            auto nx = next(it);
105
106            if (nx != s.end())
107                nxt[*it] = *nx;
108        }
109    }
110
111    SegmentTree st(n);
112
113    for (int i = 1; i <= n; i++)
114        st.update(i, nxt[i]);
115
116    for (auto [type, x, y] : ops) {
117
118        if (type == 2) {
119
120            if (st.query(x, y) <= y)
121                cout << "NO\n";
122            else
123                cout << "YES\n";
124
125        } else {
126
127            int k = x;
128            int newValue = get(y);
129            int oldValue = a[k];
130
131            if (oldValue == newValue)
132                continue;
133
134            // Quitar k del valor anterior
135            {
136                auto &s = pos[oldValue];
137                auto it = s.find(k);
138
139                int prevPos = -1;
140                int nextPos = INF;
141
142                if (it != s.begin()) {
143                    auto p = prev(it);
144                    prevPos = *p;
145                }
146
147                auto nx = next(it);
148
149                if (nx != s.end())
150                    nextPos = *nx;
151
152                if (prevPos != -1)
153                    st.update(prevPos, nextPos);
154
155                st.update(k, INF);
156
157                s.erase(it);
158            }
159
160            // Agregar k al nuevo valor
161            {
162                auto &s = pos[newValue];
163
164                auto it = s.lower_bound(k);
165

```

21.1 Distinct Value Queries II - Segment Tree sobre Ocurrencia Anterior (cont.)

```

85         int nextPos = INF;
86
87         if (it != s.end())
88             nextPos = *it;
89
90         if (it != s.begin()) {
91             auto p = prev(it);
92             st.update(*p, k);
93         }
94
95         st.update(k, nextPos);
96
97         s.insert(k);
98     }
99
100     a[k] = newValue;
101 }
102 }
103 }
104 }

```

21.2 Distinct Values Queries - Mo's Algorithm

O: $((n+q) \sqrt{n})$ con Mo

Uso: ordenar Query por (bloque de l, r); add(i) y remove(i) mueven la ventana CUANDO USAR MO: esta es la pregunta importante, porque Mo no es la primera opcion casi nunca

C++/CSES/Range Queries/Distinct Values Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int B; // tamano de bloque
5 vector<int> a, cnt;
6 int distinct = 0;
7
8 struct Query { int l, r, idx; };
9
10 void add(int i) { if (cnt[a[i]]++ == 0) distinct++; }
11 void remove(int i) { if (--cnt[a[i]] == 0) distinct--; }
12
13 vector<int> mo(vector<Query>& qs) {
14     sort(qs.begin(), qs.end(), [](const Query& x, const Query& y) {
15         if (x.l / B != y.l / B) return x.l / B < y.l / B; // por bloque de l
16         return x.r < y.r; // luego por r
17     });
18
19     vector<int> ans(qs.size());
20     int L = 0, R = -1; // ventana actual vacia
21     for (auto& q : qs) {
22         while (R < q.r) add(++R);
23         while (L > q.l) add(--L);
24         while (R > q.r) remove(R--);
25         while (L < q.l) remove(L++);
26         ans[q.idx] = distinct;
27     }
28     return ans;
29 }
30
31 int main() {
32     ios::sync_with_stdio(false); cin.tie(nullptr);
33     int n, q; cin >> n >> q;
34     a.resize(n);
35     for (int& x : a) cin >> x;
36
37     // compresion de coordenadas
38     vector<int> v(a);
39     sort(v.begin(), v.end());
40     v.erase(unique(v.begin(), v.end()), v.end());
41     for (int& x : a) x = lower_bound(v.begin(), v.end(), x) - v.begin();
42     cnt.assign(v.size(), 0);
43
44     B = max(1, (int)sqrt(n));
45     vector<Query> qs(q);

```

21.2 Distinct Values Queries - Mo's Algorithm (cont.)

```

46     for (int i = 0; i < q; i++) {
47         cin >> qs[i].l >> qs[i].r;
48         qs[i].l--; qs[i].r--;
49         qs[i].idx = i;
50     }
51
52     for (int x : mo(qs)) cout << x << '\n';
53 }

```

21.3 Dynamic Range Minimum Queries - Segment Tree (Minimo)

O: (log n) update y query, (n) build

Uso: st.build(1,1,n,a); st.update(1,1,n,pos,v); st.query(1,1,n,l,r)

Segment tree de minimos con actualizacion puntual, en O(log n) cada operacion

C++/CSES/Range Queries/Dynamic Range Minimum Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct SegmentTree {
7     int n;
8     vector<ll> tree;
9
10    SegmentTree(int n) : n(n), tree(4 * n) {}
11
12    void build(int node, int l, int r, vector<ll>& arr) {
13        if (l == r) {
14            tree[node] = arr[l];
15            return;
16        }
17
18        int mid = (l + r) / 2;
19
20        build(node * 2, l, mid, arr);
21        build(node * 2 + 1, mid + 1, r, arr);
22
23        tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
24    }
25
26    void update(int node, int l, int r, int pos, ll val) {
27        if (l == r) {
28            tree[node] = val;
29            return;
30        }
31
32        int mid = (l + r) / 2;
33
34        if (pos <= mid)
35            update(node * 2, l, mid, pos, val);
36        else
37            update(node * 2 + 1, mid + 1, r, pos, val);
38
39        tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
40    }
41
42    ll query(int node, int l, int r, int ql, int qr) {
43        if (ql <= l && r <= qr)
44            return tree[node];
45
46        int mid = (l + r) / 2;
47        ll ans = LLONG_MAX;
48
49        if (ql <= mid)
50            ans = min(ans, query(node * 2, l, mid, ql, qr));
51
52        if (qr > mid)
53            ans = min(ans, query(node * 2 + 1, mid + 1, r, ql, qr));
54
55        return ans;
56    }

```

21.3 Dynamic Range Minimum Queries - Segment Tree (Minimo) (cont.)

```

51 };
52
53 int main() {
54     ios::sync_with_stdio(false);
55     cin.tie(nullptr);
56
57     int n, q;
58     cin >> n >> q;
59
60     vector<ll> arr(n + 1);
61
62     for (int i = 1; i <= n; i++)
63         cin >> arr[i];
64
65     SegmentTree st(n);
66
67     st.build(1, 1, n, arr);
68
69     while (q--) {
70         int a, b;
71         ll c;
72         cin >> a >> b >> c;
73
74         if (a == 1) {
75             st.update(1, 1, n, b, c);
76         } else {
77             cout << st.query(1, 1, n, b, c) << '\n';
78         }
79     }
80 }

```

21.4 Dynamic Range Sum Queries - Fenwick Tree (BIT)

O: (log n) add y query, (n log n) construir

Uso: Fenwick fw(n); fw.add(i,x); fw.query(l,r) // 1-INDEXADO

Fenwick (o BIT): sumas de prefijo con actualizacion puntual, ambas en O(log n), en ~10 lineas

C++/CSES/Range Queries/Dynamic Range Sum Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Fenwick {
7     int n;
8     vector<ll> bit;
9
10    Fenwick(int n) : n(n), bit(n + 1, 0) {}
11
12    void add(int i, ll x) {
13        for (; i <= n; i += i & -i)
14            bit[i] += x;
15    }
16
17    ll sum(int i) {
18        ll res = 0;
19        for (; i > 0; i -= i & -i)
20            res += bit[i];
21        return res;
22    }
23
24    ll query(int l, int r) {
25        return sum(r) - sum(l - 1);
26    }
27 };
28
29 int main() {
30     ios::sync_with_stdio(false);
31     cin.tie(nullptr);
32
33     int n, q;
34     cin >> n >> q;

```

21.4 Dynamic Range Sum Queries - Fenwick Tree (BIT) (cont.)

```

35
36 vector<ll> arr(n + 1);
37 Fenwick fw(n);
38
39 for (int i = 1; i <= n; i++) {
40     cin >> arr[i];
41     fw.add(i, arr[i]);
42 }
43
44 while (q--) {
45     int a, b;
46     ll c;
47     cin >> a >> b >> c;
48
49     if (a == 1) {
50         ll diff = c - arr[b];
51         arr[b] = c;
52         fw.add(b, diff);
53     } else {
54         cout << fw.query(b, c) << '\n';
55     }
56 }
57 }

```

21.5 Forest Queries - Prefix Sums 2D

O: (n²) de construcci n y (1) por consulta

Prefix sums en dos dimensiones, donde lo  nico nuevo es la inclusi n-exclusi n

C++/CSES/Range Queries/Forest Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n, q;
9     cin >> n >> q;
10
11     vector<vector<int>> pref(n + 1, vector<int>(n + 1));
12
13     for (int i = 1; i <= n; i++) {
14         string s;
15         cin >> s;
16
17         for (int j = 1; j <= n; j++) {
18             int tree = (s[j - 1] == '*');
19
20             pref[i][j] = tree
21                 + pref[i - 1][j]
22                 + pref[i][j - 1]
23                 - pref[i - 1][j - 1];
24         }
25     }
26
27     while (q--) {
28         int y1, x1, y2, x2;
29         cin >> y1 >> x1 >> y2 >> x2;
30
31         cout << pref[y2][x2]
32             - pref[y1 - 1][x2]
33             - pref[y2][x1 - 1]
34             + pref[y1 - 1][x1 - 1]
35             << '\n';
36     }
37 }

```

21.6 Forest Queries II - Fenwick 2D

O: (log² n) por operaci n, Fenwick 2D

Fenwick en dos dimensiones: un for anidado dentro de otro, cada uno con el mismo i & -i de siempre

C++/CSES/Range Queries/Forest Queries II.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n, q;
5 int bit[1005][1005];
6 bool a[1005][1005];
7
8 void add(int x, int y, int v) {
9     for (int i = x; i <= n; i += i & -i)
10         for (int j = y; j <= n; j += j & -j)
11             bit[i][j] += v;
12 }
13
14 int sum(int x, int y) {
15     int ans = 0;
16
17     for (int i = x; i > 0; i -= i & -i)
18         for (int j = y; j > 0; j -= j & -j)
19             ans += bit[i][j];
20
21     return ans;
22 }
23
24 int query(int x1, int y1, int x2, int y2) {
25     return sum(x2, y2)
26         - sum(x1 - 1, y2)
27         - sum(x2, y1 - 1)
28         + sum(x1 - 1, y1 - 1);
29 }
30
31 int main() {
32     ios::sync_with_stdio(false);
33     cin.tie(nullptr);
34
35     cin >> n >> q;
36
37     for (int i = 1; i <= n; i++) {
38         string s;
39         cin >> s;
40
41         for (int j = 1; j <= n; j++) {
42             if (s[j - 1] == '*') {
43                 a[i][j] = true;
44                 add(i, j, 1);
45             }
46         }
47     }
48
49     while (q--) {
50         int type;
51         cin >> type;
52
53         if (type == 1) {
54             int y, x;
55             cin >> y >> x;
56
57             if (a[y][x]) {
58                 a[y][x] = false;
59                 add(y, x, -1);
60             } else {
61                 a[y][x] = true;
62                 add(y, x, 1);
63             }
64         } else {
65             int y1, x1, y2, x2;
66             cin >> y1 >> x1 >> y2 >> x2;
67
68

```

21.6 Forest Queries II - Fenwick 2D (cont.)

```

88         cout << query(y1, x1, y2, x2) << '\n';
89     }
90 }
91 }

```

21.7 Hotel Queries - Segment Tree con Descenso

O: (log n) por consulta, con descenso por el  rbol

Uso: ST st; st.find(x) -> primer hotel con cupo >= x, 0 si no hay

Segment tree de MAXIMOS, pero lo interesante no es la consulta sino el DESCENSO

C++/CSES/Range Queries/Hotel Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct ST {
5     int n;
6     vector<int> t;
7
8     ST(vector<int>& a) {
9         n = a.size() - 1;
10        t.resize(4 * n);
11        build(1, 1, n, a);
12    }
13
14    void build(int p, int l, int r, vector<int>& a) {
15        if (l == r) {
16            t[p] = a[l];
17            return;
18        }
19
20        int m = (l + r) / 2;
21
22        build(p * 2, l, m, a);
23        build(p * 2 + 1, m + 1, r, a);
24
25        t[p] = max(t[p * 2], t[p * 2 + 1]);
26    }
27
28    int find(int p, int l, int r, int x) {
29        if (l == r) {
30            t[p] -= x;
31            return l;
32        }
33
34        int m = (l + r) / 2;
35        int pos;
36
37        if (t[p * 2] >= x)
38            pos = find(p * 2, l, m, x);
39        else
40            pos = find(p * 2 + 1, m + 1, r, x);
41
42        t[p] = max(t[p * 2], t[p * 2 + 1]);
43
44        return pos;
45    }
46
47    int find(int x) {
48        if (t[1] < x)
49            return 0;
50
51        return find(1, 1, n, x);
52    }
53 };
54
55 int main() {
56     ios::sync_with_stdio(false);
57     cin.tie(nullptr);
58
59     int n, m;
60     cin >> n >> m;

```

21.7 Hotel Queries - Segment Tree con Descenso (cont.)

```

61     vector<int> a(n + 1);
62
63     for (int i = 1; i <= n; i++)
64         cin >> a[i];
65
66     ST st(a);
67
68     while (m--) {
69         int x;
70         cin >> x;
71         cout << st.find(x) << ' ';
72     }
73 }

```

21.8 Increasing Array Queries - Monotonic Stack + Binary Lifting con Costos

O: (n log n), pila monotona mas binary lifting con costos

Para cada consulta [l, r], el m nimo de sumas para que el tramo quede no decreciente

C++/CSES/Range Queries/Increasing Array Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const int N = 200005;
7 const int LOG = 20;
8
9 ll a[N], pref[N], cost[LOG][N];
10 int up[LOG][N];
11
12 int main() {
13     ios::sync_with_stdio(false);
14     cin.tie(nullptr);
15
16     int n, q;
17     cin >> n >> q;
18
19     for (int i = 1; i <= n; i++) {
20         cin >> a[i];
21         pref[i] = pref[i - 1] + a[i];
22     }
23
24     a[n + 1] = 1e18;
25
26     for (int k = 0; k < LOG; k++)
27         up[k][n + 1] = n + 1;
28
29     stack<int> st;
30     st.push(n + 1);
31
32     for (int i = n; i >= 1; i--) {
33         while (a[st.top()] <= a[i])
34             st.pop();
35
36         up[0][i] = st.top();
37
38         int j = up[0][i];
39
40         cost[0][i] =
41             a[i] * (j - i - 1)
42             - (pref[j - 1] - pref[i]);
43
44         st.push(i);
45     }
46
47     for (int k = 1; k < LOG; k++) {
48         for (int i = 1; i <= n; i++) {
49             int mid = up[k - 1][i];
50
51             up[k][i] = up[k - 1][mid];

```

21.8 Increasing Array Queries - Monotonic Stack + Binary Lifting con Costos (cont.)

```

52
53     cost[k][i] =
54         cost[k - 1][i] +
55         cost[k - 1][mid];
56     }
57 }
58
59 while (q--) {
60     int l, r;
61     cin >> l >> r;
62
63     int x = l;
64     ll ans = 0;
65
66     for (int k = LOG - 1; k >= 0; k--) {
67         if (up[k][x] <= r) {
68             ans += cost[k][x];
69             x = up[k][x];
70         }
71     }
72
73     if (x < r) {
74         ans += a[x] * (r - x)
75             - (pref[r] - pref[x]);
76     }
77
78     cout << ans << '\n';
79 }
80 }

```

21.9 List Removals - Fenwick con Descenso Binario (k-esimo)

O: $(\log^2 n)$ por consulta, o $(\log n)$ con descenso binario

Uso: fw con 1 en cada vivo; busca el k-esimo vivo y pon 0 al sacarlo
Fenwick de unos y ceros (1 = la posici n todav a est ) donde lo interesante es kth()

C++/CSES/Range Queries/List Removals.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Fenwick {
5     int n;
6     vector<int> bit;
7
8     Fenwick(int n) : n(n), bit(n + 1, 0) {}
9
10    void add(int i, int x) {
11        for (; i <= n; i += i & -i)
12            bit[i] += x;
13    }
14
15    int sum(int i) {
16        int res = 0;
17        for (; i > 0; i -= i & -i)
18            res += bit[i];
19        return res;
20    }
21
22    int kth(int k) {
23        int pos = 0;
24
25        for (int p = 1 << 18; p; p >>= 1) {
26            int next = pos + p;
27
28            if (next <= n && bit[next] < k) {
29                pos = next;
30                k -= bit[next];
31            }
32        }
33
34        return pos + 1;
35    }

```

21.9 List Removals - Fenwick con Descenso Binario (k-esimo) (cont.)

```

36 };
37
38 int main() {
39     ios::sync_with_stdio(false);
40     cin.tie(nullptr);
41
42     int n;
43     cin >> n;
44
45     vector<int> a(n + 1);
46
47     for (int i = 1; i <= n; i++) {
48         cin >> a[i];
49     }
50
51     Fenwick fw(n);
52
53     for (int i = 1; i <= n; i++)
54         fw.add(i, 1);
55
56     for (int i = 0; i < n; i++) {
57         int k;
58         cin >> k;
59
60         int pos = fw.kth(k);
61
62         cout << a[pos] << ' ';
63
64         fw.add(pos, -1);
65     }
66
67     cout << '\n';
68 }

```

21.10 Missing Coin Sum Queries - Cubetas por Potencia de 2 + Segment Tree

O: $(n \log n + q \cdot 30 \log n)$, cubetas por potencia de 2

La versi n por rangos del cl sico "menor suma que no se puede formar"

C++/CSES/Range Queries/Missing Coin Sum Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const int N = 200005;
7 const int K = 30;
8 const int INF = 2e9;
9
10 int st[2 * N][K];
11 ll pref[N][K];
12
13 int main() {
14     ios::sync_with_stdio(false);
15     cin.tie(nullptr);
16
17     int n, q;
18     cin >> n >> q;
19
20     for (int i = 0; i < 2 * N; i++)
21         for (int j = 0; j < K; j++)
22             st[i][j] = INF;
23
24     for (int i = 1; i <= n; i++) {
25         int x;
26         cin >> x;
27
28         int k = 31 - __builtin_clz(x);
29
30         st[N + i][k] = x;
31         pref[i][k] = x;
32     }

```

21.10 Missing Coin Sum Queries - Cubetas por Potencia de 2 + Segment Tree (cont.)

```

33
34     for (int i = 1; i <= n; i++)
35         for (int k = 0; k < K; k++)
36             pref[i][k] += pref[i - 1][k];
37
38     for (int i = N - 1; i >= 1; i--)
39         for (int k = 0; k < K; k++)
40             st[i][k] = min(st[i * 2][k], st[i * 2 + 1][k]);
41
42     while (q--) {
43         int l, r;
44         cin >> l >> r;
45
46         int L = l + N;
47         int R = r + N;
48
49         int mn[K];
50
51         for (int k = 0; k < K; k++)
52             mn[k] = INF;
53
54         while (L <= R) {
55             if (L & 1) {
56                 for (int k = 0; k < K; k++)
57                     mn[k] = min(mn[k], st[L][k]);
58                 L++;
59             }
60
61             if (!(R & 1)) {
62                 for (int k = 0; k < K; k++)
63                     mn[k] = min(mn[k], st[R][k]);
64                 R--;
65             }
66
67             L /= 2;
68             R /= 2;
69         }
70
71         ll sum = 0;
72
73         for (int k = 0; k < K; k++) {
74             if (sum + 1 < mn[k] &&
75                 sum + 1 < (1LL << (k + 1))) {
76                 cout << sum + 1 << '\n';
77                 goto done;
78             }
79
80             sum += pref[r][k] - pref[l - 1][k];
81         }
82
83         cout << sum + 1 << '\n';
84
85         done;;
86     }
87 }

```

21.11 Movie Festival Queries - Greedy + Binary Lifting sobre el Tiempo

O: $(n \log n + q \log T)$, binary lifting sobre el tiempo

Máximo de películas que se pueden ver enteras dentro de $[a, b]$

C++/CSES/Range Queries/Movie Festival Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXT = 1000000;
5 const int LOG = 20;
6
7 int up[LOG][MAXT + 2];
8 int nxt[MAXT + 2];
9 int en[MAXT + 2];
10

```

21.11 Movie Festival Queries - Greedy + Binary Lifting sobre el Tiempo (cont.)

```

11 int main() {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14
15     int n, q;
16     cin >> n >> q;
17
18     for (int i = 0; i <= MAXT + 1; i++)
19         en[i] = MAXT + 1;
20
21     for (int i = 0; i < n; i++) {
22         int a, b;
23         cin >> a >> b;
24         en[a] = min(en[a], b);
25     }
26
27     nxt[MAXT + 1] = MAXT + 1;
28
29     for (int i = MAXT; i >= 1; i--)
30         nxt[i] = min(en[i], nxt[i + 1]);
31
32     for (int i = 1; i <= MAXT; i++)
33         up[0][i] = nxt[i];
34
35     up[0][MAXT + 1] = MAXT + 1;
36
37     for (int k = 1; k < LOG; k++) {
38         up[k][MAXT + 1] = MAXT + 1;
39
40         for (int i = 1; i <= MAXT; i++) {
41             int mid = up[k - 1][i];
42             up[k][i] = up[k - 1][mid];
43         }
44     }
45
46     while (q--) {
47         int a, b;
48         cin >> a >> b;
49
50         int pos = a;
51         int ans = 0;
52
53         for (int k = LOG - 1; k >= 0; k--) {
54             if (up[k][pos] <= b) {
55                 ans += (1 << k);
56                 pos = up[k][pos];
57             }
58         }
59
60         cout << ans << '\n';
61     }
62 }

```

21.12 Pizzeria Queries - Truco de $p[i]$ mas y menos i

O: $(\log n)$ por operacion, dos segment trees de mínimo

Uso: un árbol para $a[i] + i$ y otro para $a[i] - i$; la respuesta es el menor

Hay que minimizar $p[j] + |i - j|$ sobre todo j , con actualizaciones

C++/CSES/Range Queries/Pizzeria Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct SegmentTree {
7     int n;
8     vector<ll> tree;
9
10     SegmentTree(vector<ll>& a) {
11         n = a.size() - 1;
12         tree.resize(4 * n);

```

21.12 Pizzeria Queries - Truco de $p[i]$ mas y menos i (cont.)

```

18     build(1, 1, n, a);
19 }
20
21 void build(int node, int l, int r, vector<ll>& a) {
22     if (l == r) {
23         tree[node] = a[l];
24         return;
25     }
26     int mid = (l + r) / 2;
27     build(node * 2, l, mid, a);
28     build(node * 2 + 1, mid + 1, r, a);
29     tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
30 }
31
32 void update(int node, int l, int r, int pos, ll value) {
33     if (l == r) {
34         tree[node] = value;
35         return;
36     }
37     int mid = (l + r) / 2;
38     if (pos <= mid)
39         update(node * 2, l, mid, pos, value);
40     else
41         update(node * 2 + 1, mid + 1, r, pos, value);
42     tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
43 }
44
45 ll query(int node, int l, int r, int ql, int qr) {
46     if (ql <= l && r <= qr)
47         return tree[node];
48     int mid = (l + r) / 2;
49     ll ans = LLONG_MAX;
50     if (ql <= mid)
51         ans = min(ans, query(node * 2, l, mid, ql, qr));
52     if (qr > mid)
53         ans = min(ans, query(node * 2 + 1, mid + 1, r, ql, qr));
54     return ans;
55 }
56
57 void update(int pos, ll value) {
58     update(1, 1, n, pos, value);
59 }
60
61 ll query(int l, int r) {
62     if (l > r)
63         return LLONG_MAX;
64     return query(1, 1, n, l, r);
65 }
66
67 int main() {
68     ios::sync_with_stdio(false);
69     cin.tie(nullptr);
70
71     int n, q;
72     cin >> n >> q;
73
74     vector<ll> p(n + 1);
75     vector<ll> a(n + 1), b(n + 1);

```

21.12 Pizzeria Queries - Truco de $p[i]$ mas y menos i (cont.)

```

86     for (int i = 1; i <= n; i++) {
87         cin >> p[i];
88         a[i] = p[i] - i;
89         b[i] = p[i] + i;
90     }
91     SegmentTree leftTree(a);
92     SegmentTree rightTree(b);
93     while (q--) {
94         int type, k;
95         ll x;
96         cin >> type >> k;
97         if (type == 1) {
98             cin >> x;
99             p[k] = x;
100             leftTree.update(k, x - k);
101             rightTree.update(k, x + k);
102         }
103         else {
104             ll left = leftTree.query(1, k) + k;
105             ll right = rightTree.query(k, n) - k;
106             cout << min(left, right) << '\n';
107         }
108     }
109 }

```

21.13 Polynomial Queries - Lazy Propagation de Progresiones Aritmeticas

O: (log n) por operacion, lazy de progresiones aritmeticas

Lazy propagation donde la actualizacion no suma una constante sino 1, 2, 3, ... a lo largo del rango

C++/CSES/Range Queries/Polynomial Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const int N = 200005;
7
8 ll st[4 * N];
9 ll lazyA[4 * N];
10 ll lazyD[4 * N];
11
12 void build(int p, int l, int r, vector<ll>& a) {
13     if (l == r) {
14         st[p] = a[l];
15         return;
16     }
17     int m = (l + r) / 2;
18     build(p * 2, l, m, a);
19     build(p * 2 + 1, m + 1, r, a);
20     st[p] = st[p * 2] + st[p * 2 + 1];
21 }
22
23 void apply(int p, int l, int r, ll A, ll D) {
24     ll len = r - l + 1;
25     st[p] += len * (2 * A + (len - 1) * D) / 2;
26     lazyA[p] += A;
27     lazyD[p] += D;
28 }

```

21.13 Polynomial Queries - Lazy Propagation de Progresiones Aritmeticas (cont.)

```

34
35 void push(int p, int l, int r) {
36     if (lazyA[p] == 0 && lazyD[p] == 0)
37         return;
38
39     int m = (l + r) / 2;
40     int lenLeft = m - l + 1;
41
42     apply(p * 2, l, m, lazyA[p], lazyD[p]);
43
44     apply(
45         p * 2 + 1,
46         m + 1,
47         r,
48         lazyA[p] + lenLeft * lazyD[p],
49         lazyD[p]
50     );
51
52     lazyA[p] = 0;
53     lazyD[p] = 0;
54 }
55
56 void update(int p, int l, int r, int ql, int qr) {
57     if (qr < l || r < ql)
58         return;
59
60     if (ql <= l && r <= qr) {
61         ll A = l - ql + 1;
62         apply(p, l, r, A, 1);
63         return;
64     }
65
66     push(p, l, r);
67
68     int m = (l + r) / 2;
69
70     update(p * 2, l, m, ql, qr);
71     update(p * 2 + 1, m + 1, r, ql, qr);
72
73     st[p] = st[p * 2] + st[p * 2 + 1];
74 }
75
76 ll query(int p, int l, int r, int ql, int qr) {
77     if (qr < l || r < ql)
78         return 0;
79
80     if (ql <= l && r <= qr)
81         return st[p];
82
83     push(p, l, r);
84
85     int m = (l + r) / 2;
86
87     return query(p * 2, l, m, ql, qr)
88         + query(p * 2 + 1, m + 1, r, ql, qr);
89 }
90
91 int main() {
92     ios::sync_with_stdio(false);
93     cin.tie(nullptr);
94
95     int n, q;
96     cin >> n >> q;
97
98     vector<ll> a(n + 1);
99
100     for (int i = 1; i <= n; i++)
101         cin >> a[i];
102
103     build(1, 1, n, a);
104
105     while (q--) {

```

21.13 Polynomial Queries - Lazy Propagation de Progresiones Aritmeticas (cont.)

```

106     int type, l, r;
107     cin >> type >> l >> r;
108
109     if (type == 1)
110         update(1, 1, n, l, r);
111     else
112         cout << query(1, 1, n, l, r) << '\n';
113 }
114 }

```

21.14 Prefix Sum Queries - Segment Tree con Nodo Compuesto

O: (log n) por operaci n; el Node guarda suma y mejor prefijo

Uso: SegmentTree st(a); st.update(1,1,n,pos,v); st.query(1,1,n,l,r)

Segment tree donde cada nodo guarda DOS cosas: la suma del rango y el mejor prefijo del rango

C++/CSES/Range Queries/Prefix Sum Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Node {
7     ll sum, pref;
8 };
9
10 Node merge(Node a, Node b) {
11     return {
12         a.sum + b.sum,
13         max(a.pref, a.sum + b.pref)
14     };
15 }
16
17 struct SegmentTree {
18     int n;
19     vector<Node> tree;
20
21     SegmentTree(vector<ll>& a) {
22         n = a.size() - 1;
23         tree.resize(4 * n);
24         build(1, 1, n, a);
25     }
26
27     void build(int node, int l, int r, vector<ll>& a) {
28         if (l == r) {
29             tree[node] = {a[l], max(0LL, a[l])};
30             return;
31         }
32
33         int mid = (l + r) / 2;
34
35         build(node * 2, l, mid, a);
36         build(node * 2 + 1, mid + 1, r, a);
37
38         tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
39     }
40
41     void update(int node, int l, int r, int pos, ll value) {
42         if (l == r) {
43             tree[node] = {value, max(0LL, value)};
44             return;
45         }
46
47         int mid = (l + r) / 2;
48
49         if (pos <= mid)
50             update(node * 2, l, mid, pos, value);
51         else
52             update(node * 2 + 1, mid + 1, r, pos, value);
53
54         tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
55     }

```

21.14 Prefix Sum Queries - Segment Tree con Nodo Compuesto (cont.)

```

56
57 Node query(int node, int l, int r, int ql, int qr) {
58     if (ql <= l && r <= qr)
59         return tree[node];
60
61     int mid = (l + r) / 2;
62
63     if (qr <= mid)
64         return query(node * 2, l, mid, ql, qr);
65
66     if (ql > mid)
67         return query(node * 2 + 1, mid + 1, r, ql, qr);
68
69     return merge(
70         query(node * 2, l, mid, ql, qr),
71         query(node * 2 + 1, mid + 1, r, ql, qr)
72     );
73 }
74
75 void update(int pos, ll value) {
76     update(1, 1, n, pos, value);
77 }
78
79 Node query(int l, int r) {
80     return query(1, 1, n, l, r);
81 }
82 };
83
84 int main() {
85     ios::sync_with_stdio(false);
86     cin.tie(nullptr);
87
88     int n, q;
89     cin >> n >> q;
90
91     vector<ll> a(n + 1);
92
93     for (int i = 1; i <= n; i++)
94         cin >> a[i];
95
96     SegmentTree st(a);
97
98     while (q--) {
99         int type, a, b;
100         cin >> type >> a >> b;
101
102         if (type == 1) {
103             st.update(a, b);
104         } else {
105             cout << st.query(a, b).pref << '\n';
106         }
107     }
108 }

```

21.15 Range Interval Queries - Barrido Offline con Fenwick

O: $((n+q) \log n)$ con barrido de eventos

Uso: ordenar Event por posición y responder offline con el Fenwick
 Contar cuantos elementos de $[l, r]$ tienen valor en $[a, b]$, sin actualizaciones

C++/CSSES/Range Queries/Range Interval Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Fenwick {
5     int n;
6     vector<int> bit;
7
8     Fenwick(int n) : n(n), bit(n + 1, 0) {}
9
10    void add(int i, int x) {
11        for (; i <= n; i += i & -i)

```

21.15 Range Interval Queries - Barrido Offline con Fenwick (cont.)

```

12         bit[i] += x;
13     }
14
15     int sum(int i) {
16         int res = 0;
17
18         for (; i > 0; i -= i & -i)
19             res += bit[i];
20
21         return res;
22     }
23 };
24
25 struct Event {
26     int value, pos, id, sign;
27
28     bool operator<(const Event& other) const {
29         return value < other.value;
30     }
31 };
32
33 int main() {
34     ios::sync_with_stdio(false);
35     cin.tie(nullptr);
36
37     int n, q;
38     cin >> n >> q;
39
40     vector<pair<int, int>> a(n);
41
42     for (int i = 1; i <= n; i++) {
43         int x;
44         cin >> x;
45         a[i - 1] = {x, i};
46     }
47
48     sort(a.begin(), a.end());
49
50     vector<Event> events;
51
52     for (int i = 0; i < q; i++) {
53         int l, r, c, d;
54         cin >> l >> r >> c >> d;
55
56         events.push_back({d, r, i, 1});
57         events.push_back({d, l - 1, i, -1});
58         events.push_back({c - 1, r, i, -1});
59         events.push_back({c - 1, l - 1, i, 1});
60     }
61
62     sort(events.begin(), events.end());
63
64     Fenwick fw(n);
65     vector<int> ans(q);
66
67     int p = 0;
68
69     for (auto e : events) {
70
71         while (p < n && a[p].first <= e.value) {
72             fw.add(a[p].second, 1);
73             p++;
74         }
75
76         ans[e.id] += e.sign * fw.sum(e.pos);
77     }
78
79     for (int x : ans)
80         cout << x << '\n';
81 }

```

21.16 Range Queries And Copies - Segment Tree Persistente

O: (log n) por operacion, (n log n) de memoria

Uso: raiz[k]=update(raiz[j],1,n,pos,x); query(raiz[k],1,n,l,r) // persistente

Segment tree PERSISTENTE: cada actualizacion no modifica nodos

C++/CSES/Range Queries/Range Queries and Copies.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Node {
7     int l, r;
8     ll sum;
9 };
10
11 const int MAXN = 200005;
12 const int MAXNODE = MAXN * 25;
13
14 Node st[MAXNODE];
15 int root[MAXN];
16 int nodes = 0;
17
18 int build(int l, int r, vector<ll>& a) {
19     int p = ++nodes;
20
21     if (l == r) {
22         st[p].sum = a[l];
23         return p;
24     }
25
26     int m = (l + r) / 2;
27
28     st[p].l = build(l, m, a);
29     st[p].r = build(m + 1, r, a);
30
31     st[p].sum = st[st[p].l].sum + st[st[p].r].sum;
32
33     return p;
34 }
35
36 int update(int old, int l, int r, int pos, ll x) {
37     int p = ++nodes;
38     st[p] = st[old];
39
40     if (l == r) {
41         st[p].sum = x;
42         return p;
43     }
44
45     int m = (l + r) / 2;
46
47     if (pos <= m)
48         st[p].l = update(st[old].l, l, m, pos, x);
49     else
50         st[p].r = update(st[old].r, m + 1, r, pos, x);
51
52     st[p].sum = st[st[p].l].sum + st[st[p].r].sum;
53
54     return p;
55 }
56
57 ll query(int p, int l, int r, int ql, int qr) {
58     if (qr < l || r < ql)
59         return 0;
60
61     if (ql <= l && r <= qr)
62         return st[p].sum;
63
64     int m = (l + r) / 2;
65
66     return query(st[p].l, l, m, ql, qr)
```

21.16 Range Queries And Copies - Segment Tree Persistente (cont.)

```
67         + query(st[p].r, m + 1, r, ql, qr);
68     }
69
70 int main() {
71     ios::sync_with_stdio(false);
72     cin.tie(nullptr);
73
74     int n, q;
75     cin >> n >> q;
76
77     vector<ll> a(n + 1);
78
79     for (int i = 1; i <= n; i++)
80         cin >> a[i];
81
82     root[1] = build(1, n, a);
83
84     int versions = 1;
85
86     while (q--) {
87         int type;
88         cin >> type;
89
90         if (type == 1) {
91             int k, a, x;
92             cin >> k >> a >> x;
93
94             root[k] = update(root[k], 1, n, a, x);
95         }
96         else if (type == 2) {
97             int k, a, b;
98             cin >> k >> a >> b;
99
100             cout << query(root[k], 1, n, a, b) << '\n';
101         }
102         else {
103             int k;
104             cin >> k;
105
106             root[++versions] = root[k];
107         }
108     }
109 }
```

21.17 Range Update Queries - Fenwick de Diferencias

O: (log n) por operacion

Uso: add(l,x) y add(r+1,-x); sum(i) da el valor de la posicion i

La vuelta de tuerca: en vez de guardar los valores, el Fenwick guarda las DIFERENCIAS

C++/CSES/Range Queries/Range Update Queries.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Fenwick {
7     int n;
8     vector<ll> bit;
9
10     Fenwick(int n) : n(n), bit(n + 1, 0) {}
11
12     void add(int i, ll x) {
13         for (; i <= n; i += i & -i)
14             bit[i] += x;
15     }
16
17     ll sum(int i) {
18         ll res = 0;
19         for (; i > 0; i -= i & -i)
20             res += bit[i];
21         return res;
22     }
```

21.17 Range Update Queries - Fenwick de Diferencias (cont.)

```

22     }
23 };
24
25 int main() {
26     ios::sync_with_stdio(false);
27     cin.tie(nullptr);
28
29     int n, q;
30     cin >> n >> q;
31
32     vector<ll> arr(n + 1);
33
34     for (int i = 1; i <= n; i++)
35         cin >> arr[i];
36
37     Fenwick fw(n + 1);
38
39     while (q--) {
40         int type;
41         cin >> type;
42
43         if (type == 1) {
44             int a, b;
45             ll c;
46             cin >> a >> b >> c;
47
48             fw.add(a, c);
49             fw.add(b + 1, -c);
50         } else {
51             int k;
52             cin >> k;
53
54             cout << arr[k] + fw.sum(k) << '\n';
55         }
56     }
57 }

```

21.18 Range Updates And Sums - Lazy Propagation con Suma y Asignacion

O: (log n) por operacion, lazy con suma y asignacion

Lazy propagation con DOS tipos de actualizacion (sumar x al rango y ASIGNAR x al rango)

C++/CSSES/Range Queries/Range Updates and Sums.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  const int N = 200005;
7
8  ll st[4 * N];
9  ll lazyAdd[4 * N];
10 ll lazySet[4 * N];
11 bool hasSet[4 * N];
12
13 void build(int p, int l, int r, vector<ll>& a) {
14     if (l == r) {
15         st[p] = a[l];
16         return;
17     }
18
19     int m = (l + r) / 2;
20
21     build(p * 2, l, m, a);
22     build(p * 2 + 1, m + 1, r, a);
23
24     st[p] = st[p * 2] + st[p * 2 + 1];
25 }
26
27 void applySet(int p, int l, int r, ll x) {
28     st[p] = x * (r - l + 1);
29

```

21.18 Range Updates And Sums - Lazy Propagation con Suma y Asignacion (cont.)

```

30     lazySet[p] = x;
31     lazyAdd[p] = 0;
32     hasSet[p] = true;
33 }
34
35 void applyAdd(int p, int l, int r, ll x) {
36     st[p] += x * (r - l + 1);
37
38     if (hasSet[p])
39         lazySet[p] += x;
40     else
41         lazyAdd[p] += x;
42 }
43
44 void push(int p, int l, int r) {
45     if (l == r)
46         return;
47
48     int m = (l + r) / 2;
49
50     if (hasSet[p]) {
51         applySet(p * 2, l, m, lazySet[p]);
52         applySet(p * 2 + 1, m + 1, r, lazySet[p]);
53
54         hasSet[p] = false;
55     }
56
57     if (lazyAdd[p] != 0) {
58         applyAdd(p * 2, l, m, lazyAdd[p]);
59         applyAdd(p * 2 + 1, m + 1, r, lazyAdd[p]);
60
61         lazyAdd[p] = 0;
62     }
63 }
64
65 void updateAdd(int p, int l, int r, int ql, int qr, ll x) {
66     if (qr < l || r < ql)
67         return;
68
69     if (ql <= l && r <= qr) {
70         applyAdd(p, l, r, x);
71         return;
72     }
73
74     push(p, l, r);
75
76     int m = (l + r) / 2;
77
78     updateAdd(p * 2, l, m, ql, qr, x);
79     updateAdd(p * 2 + 1, m + 1, r, ql, qr, x);
80
81     st[p] = st[p * 2] + st[p * 2 + 1];
82 }
83
84 void updateSet(int p, int l, int r, int ql, int qr, ll x) {
85     if (qr < l || r < ql)
86         return;
87
88     if (ql <= l && r <= qr) {
89         applySet(p, l, r, x);
90         return;
91     }
92
93     push(p, l, r);
94
95     int m = (l + r) / 2;
96
97     updateSet(p * 2, l, m, ql, qr, x);
98     updateSet(p * 2 + 1, m + 1, r, ql, qr, x);
99
100    st[p] = st[p * 2] + st[p * 2 + 1];
101 }

```

21.18 Range Updates And Sums - Lazy Propagation con Suma y Asignacion (cont.)

```

02
03 ll query(int p, int l, int r, int ql, int qr) {
04     if (qr < l || r < ql)
05         return 0;
06
07     if (ql <= l && r <= qr)
08         return st[p];
09
10     push(p, l, r);
11
12     int m = (l + r) / 2;
13
14     return query(p * 2, l, m, ql, qr)
15         + query(p * 2 + 1, m + 1, r, ql, qr);
16 }
17
18 int main() {
19     ios::sync_with_stdio(false);
20     cin.tie(nullptr);
21
22     int n, q;
23     cin >> n >> q;
24
25     vector<ll> a(n + 1);
26
27     for (int i = 1; i <= n; i++)
28         cin >> a[i];
29
30     build(1, 1, n, a);
31
32     while (q--) {
33         int type, l, r;
34         cin >> type >> l >> r;
35
36         if (type == 1) {
37             ll x;
38             cin >> x;
39             updateAdd(1, 1, n, l, r, x);
40         }
41         else if (type == 2) {
42             ll x;
43             cin >> x;
44             updateSet(1, 1, n, l, r, x);
45         }
46         else {
47             cout << query(1, 1, n, l, r) << '\n';
48         }
49     }
50 }

```

21.19 Range XOR Queries - Prefix XOR

O: (n) de construcción y (1) por consulta

Idem prefix sums pero con xor, y funciona por la misma razón

C++/CSES/Range Queries/Range XOR Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n, q;
9     cin >> n >> q;
10
11     vector<long long> pref(n + 1);
12
13     for (int i = 1; i <= n; i++) {
14         long long x;
15         cin >> x;
16         pref[i] = pref[i - 1] ^ x;

```

21.19 Range XOR Queries - Prefix XOR (cont.)

```

17     }
18
19     while (q--) {
20         int l, r;
21         cin >> l >> r;
22         cout << (pref[r] ^ pref[l - 1]) << '\n';
23     }
24 }

```

21.20 Salary Queries - Fenwick con Compresion Offline

O: (log n) por operación, sobre valores comprimidos

Uso: add(comp(v),+1); sum(comp(hi))-sum(comp(lo)-1) // 1-INDEXADO

Fenwick sobre VALORES en vez de posiciones, para contar cuantos salarios caen en un rango

C++/CSES/Range Queries/Salary Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Fenwick {
5     int n;
6     vector<int> bit;
7
8     Fenwick(int n) : n(n), bit(n + 1, 0) {}
9
10    void add(int i, int x) {
11        for (; i <= n; i += i & -i)
12            bit[i] += x;
13    }
14
15    int sum(int i) {
16        int res = 0;
17        for (; i > 0; i -= i & -i)
18            res += bit[i];
19        return res;
20    }
21 };
22
23 int main() {
24     ios::sync_with_stdio(false);
25     cin.tie(nullptr);
26
27     int n, q;
28     cin >> n >> q;
29
30     vector<int> salary(n + 1);
31     vector<tuple<char, int, int>> queries;
32     vector<int> values;
33
34     for (int i = 1; i <= n; i++) {
35         cin >> salary[i];
36         values.push_back(salary[i]);
37     }
38
39     for (int i = 0; i < q; i++) {
40         char type;
41         int a, b;
42         cin >> type >> a >> b;
43
44         queries.push_back({type, a, b});
45
46         if (type == '!')
47             values.push_back(b);
48     }
49
50     sort(values.begin(), values.end());
51     values.erase(unique(values.begin(), values.end()), values.end());
52
53     Fenwick fw(values.size());
54
55     for (int i = 1; i <= n; i++) {
56         int pos = lower_bound(values.begin(), values.end(), salary[i])

```

21.20 Salary Queries - Fenwick con Compresion Offline (cont.)

```

57         - values.begin() + 1;
58
59         fw.add(pos, 1);
60     }
61
62     for (auto [type, a, b] : queries) {
63
64         if (type == '!') {
65             int oldPos = lower_bound(values.begin(), values.end(), salary[a])
66                 - values.begin() + 1;
67
68             int newPos = lower_bound(values.begin(), values.end(), b)
69                 - values.begin() + 1;
70
71             fw.add(oldPos, -1);
72             fw.add(newPos, 1);
73
74             salary[a] = b;
75         }
76
77         else {
78             int right = upper_bound(values.begin(), values.end(), b)
79                 - values.begin();
80
81             int left = lower_bound(values.begin(), values.end(), a)
82                 - values.begin();
83
84             cout << fw.sum(right) - fw.sum(left) << '\n';
85         }
86     }
87 }

```

21.21 Static Range Minimum Queries - Sparse Table

O: (n log n) de tabla y (1) por consulta

Sparse table: $st[k][i]$ es el m nimo de los 2^k elementos que arrancan en i

C++/CSES/Range Queries/Static Range Minimum Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int n, q;
11    cin >> n >> q;
12
13    // K = cantidad de niveles que necesitamos.
14    // st[k][i] = m nimo de 2^k elementos empezando en i.
15    int K = 1;
16
17    while ((1 << K) <= n)
18        K++;
19
20    vector<vector<ll>> st(K, vector<ll>(n + 1));
21
22    // Nivel 0: rangos de tamano 1.
23    for (int i = 1; i <= n; i++)
24        cin >> st[0][i];
25
26    // Construimos los dem s niveles.
27    for (int k = 1; k < K; k++) {
28
29        // Tamano del rango = 2^k
30        int len = 1 << k;
31
32        for (int i = 1; i + len - 1 <= n; i++) {
33
34            // Dividimos el rango en dos partes

```

21.21 Static Range Minimum Queries - Sparse Table (cont.)

```

35        // de tamano 2^(k-1).
36        st[k][i] = min(
37            st[k - 1][i],
38            st[k - 1][i + (len / 2)]
39        );
40    }
41 }
42
43 while (q--) {
44     int a, b;
45     cin >> a >> b;
46
47     int len = b - a + 1;
48
49     // Mayor potencia de 2 que cabe en len.
50     int k = 0;
51
52     while ((1 << (k + 1)) <= len)
53         k++;
54
55     // Primer bloque de tamano 2^k
56     ll x = st[k][a];
57
58     // Segundo bloque de tamano 2^k,
59     // empezando desde el final.
60     ll y = st[k][b - (1 << k) + 1];
61
62     cout << min(x, y) << '\n';
63 }
64 }

```

21.22 Static Range Sum Queries - Prefix Sums

O: (n) de construccion y (1) por consulta

Suma de rango con el arreglo fijo: prefijos, O(n) de construccion y O(1) por consulta

C++/CSES/Range Queries/Static Range Sum Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    ll n, q;
11    cin >> n >> q;
12
13    vector<ll> pref(n + 1);
14
15    for (int i = 1; i <= n; i++) {
16        ll x;
17        cin >> x;
18        pref[i] = pref[i - 1] + x;
19    }
20
21    while (q--) {
22        ll a, b;
23        cin >> a >> b;
24
25        cout << pref[b] - pref[a - 1] << '\n';
26    }
27 }

```

21.23 Sub Array Sum Queries - Segment Tree de Maximo Subarreglo**O:** (log n) por operacion**Uso:** Node = {sum, pref, suff, best}; merge cruza suff izq con pref der

El nodo compuesto clasico, con cuatro campos

C++/CSES/Range Queries/SubArray Sum Queries.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  struct Node {
7      ll sum, pref, suff, best;
8  };
9
10 Node merge(Node L, Node R) {
11     Node res;
12
13     res.sum = L.sum + R.sum;
14
15     res.pref = max(L.pref, L.sum + R.pref);
16
17     res.suff = max(R.suff, R.sum + L.suff);
18
19     res.best = max({
20         L.best,
21         R.best,
22         L.suff + R.pref
23     });
24
25     return res;
26 }
27
28 struct SegmentTree {
29     int n;
30     vector<Node> tree;
31
32     SegmentTree(vector<ll>& a) {
33         n = a.size() - 1;
34         tree.resize(4 * n);
35         build(1, 1, n, a);
36     }
37
38     void build(int node, int l, int r, vector<ll>& a) {
39         if (l == r) {
40             tree[node] = {
41                 a[l],
42                 max(0LL, a[l]),
43                 max(0LL, a[l]),
44                 max(0LL, a[l])
45             };
46             return;
47         }
48
49         int mid = (l + r) / 2;
50
51         build(node * 2, l, mid, a);
52         build(node * 2 + 1, mid + 1, r, a);
53
54         tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
55     }
56
57     void update(int node, int l, int r, int pos, ll value) {
58         if (l == r) {
59             tree[node] = {
60                 value,
61                 max(0LL, value),
62                 max(0LL, value),
63                 max(0LL, value)
64             };
65             return;
66         }

```

21.23 Sub Array Sum Queries - Segment Tree de Maximo Subarreglo (cont.)

```

67
68         int mid = (l + r) / 2;
69
70         if (pos <= mid)
71             update(node * 2, l, mid, pos, value);
72         else
73             update(node * 2 + 1, mid + 1, r, pos, value);
74
75         tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
76     }
77
78     void update(int pos, ll value) {
79         update(1, 1, n, pos, value);
80     }
81
82     ll answer() {
83         return tree[1].best;
84     }
85 };
86
87 int main() {
88     ios::sync_with_stdio(false);
89     cin.tie(nullptr);
90
91     int n, m;
92     cin >> n >> m;
93
94     vector<ll> a(n + 1);
95
96     for (int i = 1; i <= n; i++)
97         cin >> a[i];
98
99     SegmentTree st(a);
100
101     while (m--) {
102         int k;
103         ll x;
104
105         cin >> k >> x;
106
107         st.update(k, x);
108
109         cout << st.answer() << '\n';
110     }
111 }

```

21.24 Sub Array Sum Queries II - Maximo Subarreglo en un Rango**O:** (log n) por operacion**Uso:** mismo Node de la I; aqui el merge se usa tambien en la consulta

El MISMO nodo de cuatro campos que "SubArray Sum Queries", con el merge identico

C++/CSES/Range Queries/SubArray Sum Queries II.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  struct Node {
7      ll sum, pref, suff, best;
8  };
9
10 Node merge(Node L, Node R) {
11     return {
12         L.sum + R.sum,
13         max(L.pref, L.sum + R.pref),
14         max(R.suff, R.sum + L.suff),
15         max({
16             L.best,
17             R.best,
18             L.suff + R.pref
19         })
20     };

```

21.24 Sub Array Sum Queries II - Maximo Subarreglo en un Rango (cont.)

```

20     };
21 }
22
23 struct SegmentTree {
24     int n;
25     vector<Node> tree;
26
27     SegmentTree(vector<ll>& a) {
28         n = a.size() - 1;
29         tree.resize(4 * n);
30         build(1, 1, n, a);
31     }
32
33     void build(int node, int l, int r, vector<ll>& a) {
34         if (l == r) {
35             tree[node] = {
36                 a[l],
37                 max(0LL, a[l]),
38                 max(0LL, a[l]),
39                 max(0LL, a[l])
40             };
41             return;
42         }
43
44         int mid = (l + r) / 2;
45
46         build(node * 2, l, mid, a);
47         build(node * 2 + 1, mid + 1, r, a);
48
49         tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
50     }
51
52     Node query(int node, int l, int r, int ql, int qr) {
53         if (ql <= l && r <= qr)
54             return tree[node];
55
56         int mid = (l + r) / 2;
57
58         if (qr <= mid)
59             return query(node * 2, l, mid, ql, qr);
60
61         if (ql > mid)
62             return query(node * 2 + 1, mid + 1, r, ql, qr);
63
64         Node left = query(node * 2, l, mid, ql, qr);
65         Node right = query(node * 2 + 1, mid + 1, r, ql, qr);
66
67         return merge(left, right);
68     }
69
70     Node query(int l, int r) {
71         return query(1, 1, n, l, r);
72     }
73 };
74
75 int main() {
76     ios::sync_with_stdio(false);
77     cin.tie(nullptr);
78
79     int n, q;
80     cin >> n >> q;
81
82     vector<ll> a(n + 1);
83
84     for (int i = 1; i <= n; i++)
85         cin >> a[i];
86
87     SegmentTree st(a);
88
89     while (q--) {
90         int a, b;
91         cin >> a >> b;

```

21.24 Sub Array Sum Queries II - Maximo Subarreglo en un Rango (cont.)

```

92
93         cout << st.query(a, b).best << '\n';
94     }
95 }

```

21.25 Visible Building Queries - Monotonic Stack + Binary Lifting

O: (n log n) de tabla y (log n) por consulta

Dos técnicas encadenadas, y la combinación es lo que vale

C++/CSES/Range Queries/Visible Building Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n, q;
9     cin >> n >> q;
10
11     vector<int> h(n + 1);
12
13     for (int i = 1; i <= n; i++)
14         cin >> h[i];
15
16     vector<int> nxt(n + 1, 0);
17     stack<int> st;
18
19     for (int i = n; i >= 1; i--) {
20         while (!st.empty() && h[st.top()] <= h[i])
21             st.pop();
22
23         if (!st.empty())
24             nxt[i] = st.top();
25
26         st.push(i);
27     }
28
29     int LOG = 18;
30     vector<vector<int>> up(LOG, vector<int>(n + 1));
31
32     for (int i = 1; i <= n; i++)
33         up[0][i] = nxt[i];
34
35     for (int j = 1; j < LOG; j++) {
36         for (int i = 1; i <= n; i++) {
37             if (up[j - 1][i])
38                 up[j][i] = up[j - 1][up[j - 1][i]];
39         }
40     }
41
42     while (q--) {
43         int a, b;
44         cin >> a >> b;
45
46         int pos = a;
47         int ans = 1;
48
49         for (int j = LOG - 1; j >= 0; j--) {
50             int x = up[j][pos];
51
52             if (x != 0 && x <= b) {
53                 pos = x;
54                 ans += (1 << j);
55             }
56         }
57
58         cout << ans << '\n';
59     }
60 }

```

22 - CSES: Sorting and Searching

22.1 Apartments - Dos Punteros sobre Dos Listas Ordenadas

O: ($n \log n + m \log m$) por los dos sort; el barrido es lineal

Uso: ordenar los dos; avanzar el puntero del que no puede emparejarse

Cuantos solicitantes se pueden emparejar con un apartamento dentro de una tolerancia k

C++/CSES/Sorting and Searching/Apartments.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     int n, m, k;
7     cin >> n >> m >> k;
8
9     vector<int> a(n), b(m);
10
11     for(int &x : a)
12         cin >> x;
13
14     for(int &x : b)
15         cin >> x;
16
17     sort(a.begin(), a.end());
18     sort(b.begin(), b.end());
19
20     int i = 0;
21     int j = 0;
22     int ans = 0;
23
24     while(i < n && j < m)
25     {
26         if(abs(a[i] - b[j]) <= k)
27         {
28             ans++;
29             i++;
30             j++;
31         }
32         else if(b[j] < a[i] - k)
33         {
34             // El apartamento es demasiado pequeno
35             j++;
36         }
37         else
38         {
39             // El apartamento es demasiado grande
40             i++;
41         }
42     }
43
44     cout << ans << '\n';
45 }
```

22.2 Array Division - Binaria sobre la Respuesta

O: ($n \log(\text{suma total})$), un greedy lineal por intento

Uso: check(a, k, x) arma partes greedy; binaria sobre x entre el maximo y la suma

Partir el arreglo en k tramos contiguos minimizando la suma maxima de un tramo

C++/CSES/Sorting and Searching/Array Division.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 bool check(vector<ll>& a, int k, ll x)
7 {
8     int partes = 1;
9     ll sum = 0;
10 }
```

22.2 Array Division - Binaria sobre la Respuesta (cont.)

```
11 for(ll v : a)
12 {
13     if(sum + v > x)
14     {
15         partes++;
16         sum = v;
17     }
18     else
19     {
20         sum += v;
21     }
22 }
23
24 return partes <= k;
25 }
26
27 int main()
28 {
29     ios::sync_with_stdio(false);
30     cin.tie(nullptr);
31
32     int n, k;
33     cin >> n >> k;
34
35     vector<ll> a(n);
36
37     ll lo = 0;
38     ll hi = 0;
39
40     for(ll &x : a)
41     {
42         cin >> x;
43
44         lo = max(lo, x);
45         hi += x;
46     }
47
48     while(lo < hi)
49     {
50         ll mid = (lo + hi) / 2;
51
52         if(check(a, k, mid))
53             hi = mid;
54         else
55             lo = mid + 1;
56     }
57
58     cout << lo << '\n';
59 }
```

22.3 Collecting Numbers - Contar Descensos en las Posiciones

O: (n), una pasada

Uso: pos[x] = donde esta x; respuesta = 1 + cuantas veces pos[i] > pos[i+1]

Cuantas pasadas hacen falta para recoger 1..n en orden, avanzando siempre a la derecha

C++/CSES/Sorting and Searching/Collecting Numbers.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     int n;
7     cin >> n;
8
9     vector<int> pos(n + 1);
10
11     for(int i = 1; i <= n; i++)
12     {
13         int x;
14         cin >> x;
15         pos[x] = i;
16     }
```

22.3 Collecting Numbers - Contar Descensos en las Posiciones (cont.)

```

6     }
7
8     int ans = 1;
9
10    for(int i = 1; i < n; i++)
11    {
12        if(pos[i] > pos[i + 1])
13            ans++;
14    }
15
16    cout << ans << '\n';
17 }
```

22.4 Collecting Numbers II - Recalcular Solo lo que Cambia

O: $((n + m) \log n)$, cada swap toca a lo sumo seis vecindades

Uso: tras el swap solo se revisan los pares (v-1,v) y (v,v+1) de los dos valores
Collecting Numbers con m intercambios, respondiendo tras cada uno

C++/CSES/Sorting and Searching/Collecting Numbers II.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      ios::sync_with_stdio(false);
7      cin.tie(nullptr);
8
9      int n, m;
10     cin >> n >> m;
11
12     vector<int> a(n + 1);
13     vector<int> pos(n + 1);
14
15     for(int i = 1; i <= n; i++)
16     {
17         cin >> a[i];
18         pos[a[i]] = i;
19     }
20
21     int rounds = 1;
22
23     for(int i = 1; i < n; i++)
24     {
25         if(pos[i] > pos[i + 1])
26             rounds++;
27     }
28
29     while(m--)
30     {
31         int x, y;
32         cin >> x >> y;
33
34         int vx = a[x];
35         int vy = a[y];
36
37         set<int> check;
38
39         check.insert(vx - 1);
40         check.insert(vx);
41         check.insert(vy - 1);
42         check.insert(vy);
43
44         for(int i : check)
45         {
46             if(i >= 1 && i < n)
47             {
48                 if(pos[i] > pos[i + 1])
49                     rounds--;
50             }
51         }
52     }
```

22.4 Collecting Numbers II - Recalcular Solo lo que Cambia (cont.)

```

53     swap(a[x], a[y]);
54
55     pos[vx] = y;
56     pos[vy] = x;
57
58     for(int i : check)
59     {
60         if(i >= 1 && i < n)
61         {
62             if(pos[i] > pos[i + 1])
63                 rounds++;
64         }
65     }
66
67     cout << rounds << '\n';
68 }
69
70 return 0;
71 }
```

22.5 Ferris Wheel - Dos Punteros desde los Extremos

O: $(n \log n)$ por el sort; el barrido es lineal

Uso: ordenar; si $a[l] + a[r] \leq x$ suben juntos, si no el grande sube solo
Minimo de gondolas para subir a todos, con dos personas por gondola y limite de peso

C++/CSES/Sorting and Searching/Ferris Wheel.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      int n, x;
7      cin >> n >> x;
8
9      vector<int> a(n);
10
11     for(int &v : a)
12         cin >> v;
13
14     sort(a.begin(), a.end());
15
16     int l = 0;
17     int r = n - 1;
18     int ans = 0;
19
20     while(l <= r)
21     {
22         if(a[l] + a[r] <= x)
23         {
24             // El mas pequeno y el mas grande caben juntos
25             l++;
26             r--;
27         }
28         else
29         {
30             // El mas grande necesita una gondola solo
31             r--;
32         }
33
34         ans++;
35     }
36
37     cout << ans << '\n';
38 }
```

22.6 Movie Festival - Interval Scheduling (Ordenar por FIN)

O: $(n \log n)$ por el sort; el barrido es lineal

Uso: ordenar por el FIN y tomar toda película que empiece después del último fin
Máximo de películas que se pueden ver completas sin solaparse

C++/CSES/Sorting and Searching/Movie Festival.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     int n;
7     cin >> n;
8
9     vector<pair<int,int>> movies(n);
10
11     for(auto &a,&b : movies)
12         cin >> a >> b;
13
14     sort(movies.begin(), movies.end(),
15         [](auto &a, auto &b)
16         {
17             return a.second < b.second;
18         });
19
20     int ans = 0;
21     int last = 0;
22
23     for(auto [start, finish] : movies)
24     {
25         if(start >= last)
26         {
27             ans++;
28             last = finish;
29         }
30     }
31
32     cout << ans << '\n';
33 }
```

22.7 Nested Ranges Check - Dos Barridos sobre Rangos Ordenados

O: $(n \log n)$ por el sort; los dos barridos son lineales

Uso: ordenar por (l creciente, r DECRECIENTE); un barrido con max r y otro con min r
Para cada rango, si contiene a otro y si está contenido en otro

C++/CSES/Sorting and Searching/Nested Ranges Check.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Range
5 {
6     int l, r, id;
7 };
8
9 int main()
10 {
11     ios::sync_with_stdio(false);
12     cin.tie(nullptr);
13
14     int n;
15     cin >> n;
16
17     vector<Range> a(n);
18
19     for(int i = 0; i < n; i++)
20     {
21         cin >> a[i].l >> a[i].r;
22         a[i].id = i;
23     }
24
25     sort(a.begin(), a.end(), [](Range A, Range B)
```

22.7 Nested Ranges Check - Dos Barridos sobre Rangos Ordenados (cont.)

```
26 {
27     if(A.l != B.l)
28         return A.l < B.l;
29
30     return A.r > B.r;
31 });
32
33 vector<int> contains(n, 0);
34 vector<int> contained(n, 0);
35
36 // Esta contenido por otro?
37 int maxR = -1;
38
39 for(int i = 0; i < n; i++)
40 {
41     if(maxR >= a[i].r)
42         contained[a[i].id] = 1;
43
44     maxR = max(maxR, a[i].r);
45 }
46
47 // Contiene a otro?
48 int minR = INT_MAX;
49
50 for(int i = n - 1; i >= 0; i--)
51 {
52     if(minR <= a[i].r)
53         contains[a[i].id] = 1;
54
55     minR = min(minR, a[i].r);
56 }
57
58 for(int x : contains)
59     cout << x << ' ';
60
61 cout << '\n';
62
63 for(int x : contained)
64     cout << x << ' ';
65
66 cout << '\n';
67 }
```

22.8 Nested Ranges Count - Contar Rangos Anidados con Fenwick

O: $(n \log n)$, Fenwick sobre los finales comprimidos

Uso: mismo orden que la versión Check, pero contando con fw.query en vez de un máximo
Lo mismo que Nested Ranges Check, pero CUANTOS contiene y cuantos lo contienen

C++/CSES/Sorting and Searching/Nested Ranges Count.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Fenwick
5 {
6     int n;
7     vector<int> bit;
8
9     Fenwick(int n)
10    {
11        this->n = n;
12        bit.assign(n + 1, 0);
13    }
14
15    void add(int x, int v)
16    {
17        for(; x <= n; x += x & -x)
18            bit[x] += v;
19    }
20
21    int query(int x)
22    {
```

22.8 Nested Ranges Count - Contar Rangos Anidados con Fenwick (cont.)

```

23     int ans = 0;
24
25     for(; x > 0; x -= x & -x)
26         ans += bit[x];
27
28     return ans;
29 }
30 };
31
32 struct Range
33 {
34     int l, r, id;
35 };
36
37 int main()
38 {
39     ios::sync_with_stdio(false);
40     cin.tie(nullptr);
41
42     int n;
43     cin >> n;
44
45     vector<Range> a(n);
46     vector<int> vals;
47
48     for(int i = 0; i < n; i++)
49     {
50         cin >> a[i].l >> a[i].r;
51         a[i].id = i;
52
53         vals.push_back(a[i].r);
54     }
55
56     // Compresion de coordenadas
57     sort(vals.begin(), vals.end());
58     vals.erase(unique(vals.begin(), vals.end()), vals.end());
59
60     for(auto &x : a)
61     {
62         x.r = lower_bound(vals.begin(), vals.end(), x.r)
63             - vals.begin() + 1;
64     }
65
66     // l creciente, r decreciente
67     sort(a.begin(), a.end(), [](Range A, Range B)
68     {
69         if(A.l != B.l)
70             return A.l < B.l;
71
72         return A.r > B.r;
73     }));
74
75     vector<int> contains(n);
76     vector<int> contained(n);
77
78     // -----
79     // Cuantos rangos contiene este?
80     // -----
81
82     Fenwick fw(n);
83
84     for(int i = n - 1; i >= 0; i--)
85     {
86         // Posteriores con r <= r_actual
87         contains[a[i].id] = fw.query(a[i].r);
88
89         fw.add(a[i].r, 1);
90     }
91
92     // -----
93     // Cuantos rangos contienen este?
94     // -----

```

22.8 Nested Ranges Count - Contar Rangos Anidados con Fenwick (cont.)

```

95
96     fw = Fenwick(n);
97
98     for(int i = 0; i < n; i++)
99     {
100         // Anteriores con r >= r_actual
101         int anteriores = i;
102
103         int menores = fw.query(a[i].r - 1);
104
105         contained[a[i].id] = anteriores - menores;
106
107         fw.add(a[i].r, 1);
108     }
109
110     // Orden original
111     for(int x : contains)
112         cout << x << ' ';
113
114     cout << '\n';
115
116     for(int x : contained)
117         cout << x << ' ';
118
119     cout << '\n';
120
121     return 0;
122 }

```

22.9 Sub Array Sums II - Prefijos con Map (Valores Negativos)

O: (n log n) por el map; con unordered_map seria (n)

Uso: freq[0] = 1 antes del ciclo; ans += freq[sum - x]; freq[sum]++

Cuantos subarreglos suman exactamente x, con valores que pueden ser negativos

C++/CSES/Sorting and Searching/SubArray Sums II .cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 int main()
7 {
8     ios::sync_with_stdio(false);
9     cin.tie(nullptr);
10
11     int n;
12     ll x;
13
14     cin >> n >> x;
15
16     map<ll, ll> freq;
17
18     ll sum = 0;
19     ll ans = 0;
20
21     freq[0] = 1;
22
23     for(int i = 0; i < n; i++)
24     {
25         ll a;
26         cin >> a;
27
28         sum += a;
29
30         ans += freq[sum - x];
31
32         freq[sum]++;
33     }
34
35     cout << ans << '\n';
36 }

```

23 - CSES: String Algorithms

23.1 All Palindromes - Manacher (Palindromo Mas Largo que Termina en Cada Posicion)

O: (n), Manacher

Para cada posicion, el palindromo mas largo que TERMINA ahi

C++/CSES/String Algorithms/All Palindromes.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> manacher(string s) {
5     string t = "$";
6
7     for (char c : s) {
8         t += '#';
9         t += c;
10    }
11
12    t += "#^";
13
14    vector<int> p(t.size());
15
16    int l = 1;
17    int r = 1;
18
19    for (int i = 1; i < (int)t.size() - 1; i++) {
20
21        p[i] = max(0, min(r - i, p[l + r - i]));
22
23        while (t[i - p[i]] == t[i + p[i]])
24            p[i]++;
25
26        if (i + p[i] > r) {
27            l = i - p[i];
28            r = i + p[i];
29        }
30    }
31
32    return p;
33 }
34
35 int main() {
36     ios::sync_with_stdio(false);
37     cin.tie(nullptr);
38
39     string s;
40     cin >> s;
41
42     vector<int> p = manacher(s);
43
44     int n = s.size();
45
46     vector<int> ans(n);
47
48     /*
49     PALINDROMOS IMPARES
50
51     i representa el centro en la cadena original.
52
53     p[(i + 1) * 2] / 2
54     nos dice cuanto puede extenderse el palindromo.
55
56     Para cada centro:
57
58         i, i+1, i+2, ...
59
60     podemos obtener palindromos de longitudes:
61
62         1, 3, 5, 7, ...
63
64     Pero usamos j para no volver a procesar posiciones
```

23.1 All Palindromes - Manacher (Palindromo Mas Largo que Termina en Cada Posicion) (cont.)
que ya fueron actualizadas.

```
65     */
66
67     int j = 0;
68
69     for (int i = 0; i < n; i++) {
70
71         j = max(j, i);
72
73         int limit = i + (p[(i + 1) * 2] / 2);
74
75         for (; j < limit; j++) {
76             ans[j] = 1 + 2 * (j - i);
77         }
78     }
79
80     /*
81     PALINDROMOS PARES
82
83     Ahora hacemos lo mismo para palindromos pares.
84
85     El centro esta entre i-1 e i.
86     */
87
88     j = 0;
89
90     for (int i = 1; i < n; i++) {
91
92         j = max(j, i);
93
94         int limit = i + (p[1 + i * 2] / 2);
95
96         for (; j < limit; j++) {
97             ans[j] = max(ans[j], 2 * (j - i + 1));
98         }
99     }
100
101     for (int x : ans)
102         cout << x << ' ';
103
104     cout << '\n';
105 }
106 }
```

23.2 Counting Patterns - Aho-Corasick (Conteo de Ocurrencias)

O: (suma de patrones + |s| + 26*nodos)

Uso: ac.addString(p) por patron; build(); search(s); propagate(); getCount(p)

Cuántas veces aparece cada patron en el texto, con todos los patrones a la vez

C++/CSES/String Algorithms/Counting Patterns.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct AhoCorasick
5 {
6     struct Node
7     {
8         int next[26];
9         int link;
10        int cnt;
11
12        Node()
13        {
14            fill(next, next + 26, -1);
15            link = 0;
16            cnt = 0;
17        }
18    };
19
20    vector<Node> trie;
21    vector<int> order;
22}
```

23.2 Counting Patterns - Aho-Corasick (Conteo de Ocurrencias) (cont.)

```

23 AhoCorasick()
24 {
25     trie.push_back(Node());
26 }
27
28 void addString(string s)
29 {
30     int u = 0;
31
32     for(char c : s)
33     {
34         int x = c - 'a';
35
36         if(trie[u].next[x] == -1)
37         {
38             trie[u].next[x] = trie.size();
39             trie.push_back(Node());
40         }
41
42         u = trie[u].next[x];
43     }
44 }
45
46 int getNode(string s)
47 {
48     int u = 0;
49
50     for(char c : s)
51     {
52         u = trie[u].next[c - 'a'];
53     }
54
55     return u;
56 }
57
58 void build()
59 {
60     queue<int> q;
61
62     for(int c = 0; c < 26; c++)
63     {
64         int v = trie[0].next[c];
65
66         if(v == -1)
67         {
68             trie[0].next[c] = 0;
69         }
70         else
71         {
72             trie[v].link = 0;
73             q.push(v);
74         }
75     }
76
77     while(!q.empty())
78     {
79         int u = q.front();
80         q.pop();
81
82         order.push_back(u);
83
84         for(int c = 0; c < 26; c++)
85         {
86             int v = trie[u].next[c];
87
88             if(v == -1)
89             {
90                 trie[u].next[c] = trie[trie[u].link].next[c];
91             }
92             else
93             {
94                 trie[v].link = trie[trie[u].link].next[c];

```

23.2 Counting Patterns - Aho-Corasick (Conteo de Ocurrencias) (cont.)

```

95         q.push(v);
96     }
97 }
98
99 }
100
101 void search(string s)
102 {
103     int u = 0;
104
105     for(char c : s)
106     {
107         u = trie[u].next[c - 'a'];
108         trie[u].cnt++;
109     }
110 }
111
112 void propagate()
113 {
114     for(int i = order.size() - 1; i >= 0; i--)
115     {
116         int u = order[i];
117
118         trie[trie[u].link].cnt += trie[u].cnt;
119     }
120 }
121
122 int getCount(string s)
123 {
124     int u = getNode(s);
125     return trie[u].cnt;
126 }
127 };
128
129 int main()
130 {
131     ios::sync_with_stdio(false);
132     cin.tie(nullptr);
133
134     string s;
135     cin >> s;
136
137     int k;
138     cin >> k;
139
140     vector<string> patterns(k);
141
142     AhoCorasick ac;
143
144     for(int i = 0; i < k; i++)
145     {
146         cin >> patterns[i];
147         ac.addString(patterns[i]);
148     }
149
150     ac.build();
151
152     ac.search(s);
153
154     ac.propagate();
155
156     for(string p : patterns)
157     {
158         cout << ac.getCount(p) << '\n';
159     }
160
161     return 0;
162 }

```

23.3 Distinct Subsequences - DP con Resta de la Aparicion Anterior

O: (n) tiempo, (n) memoria

Uso: $dp[i] = 2 * dp[i-1] - dp[last[c]-1]$; la respuesta es $dp[n] - 1$

Cuántas subsecuencias distintas tiene la cadena, modulo 10^9+7

C++/CSES/String Algorithms/Distinct Subsequences.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const ll MOD = 1000000007;
7
8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     string s;
14     cin >> s;
15
16     int n = s.size();
17
18     vector<ll> dp(n + 1);
19     vector<int> last(26, 0);
20
21     dp[0] = 1;
22
23     for(int i = 1; i <= n; i++)
24     {
25         int c = s[i - 1] - 'a';
26
27         dp[i] = 2 * dp[i - 1] % MOD;
28
29         if(last[c] != 0)
30         {
31             dp[i] -= dp[last[c] - 1];
32
33             if(dp[i] < 0)
34                 dp[i] += MOD;
35         }
36
37         last[c] = i;
38     }
39
40     // Quitamos la subsecuencia vacia
41     cout << (dp[n] - 1 + MOD) % MOD << '\n';
42
43     return 0;
44 }
```

23.4 Distinct Substrings - Suffix Automaton (Contar Distintas)

O: (n) estados y transiciones, con alfabeto fijo

Uso: `sam.add(c)` por letra; `sam.countDistinct()`

Cuántas subcadenas distintas tiene la cadena

C++/CSES/String Algorithms/Distinct Substrings.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct State
5 {
6     int next[26];
7     int link;
8     int len;
9
10     State()
11     {
12         fill(next, next + 26, -1);
13         link = -1;
14         len = 0;
15     }
16 }
```

23.4 Distinct Substrings - Suffix Automaton (Contar Distintas) (cont.)

```
5     }
6 };
7
8 struct SuffixAutomaton
9 {
10     vector<State> st;
11     int last;
12
13     SuffixAutomaton(int n)
14     {
15         st.reserve(2 * n);
16         st.push_back(State());
17
18         last = 0;
19     }
20
21     void add(char c)
22     {
23         int x = c - 'a';
24
25         int cur = st.size();
26         st.push_back(State());
27
28         st[cur].len = st[last].len + 1;
29
30         int p = last;
31
32         while(p != -1 && st[p].next[x] == -1)
33         {
34             st[p].next[x] = cur;
35             p = st[p].link;
36         }
37
38         if(p == -1)
39         {
40             st[cur].link = 0;
41         }
42         else
43         {
44             int q = st[p].next[x];
45
46             if(st[p].len + 1 == st[q].len)
47             {
48                 st[cur].link = q;
49             }
50             else
51             {
52                 int clone = st.size();
53                 st.push_back(st[q]);
54
55                 st[clone].len = st[p].len + 1;
56
57                 while(p != -1 && st[p].next[x] == q)
58                 {
59                     st[p].next[x] = clone;
60                     p = st[p].link;
61                 }
62
63                 st[q].link = clone;
64                 st[cur].link = clone;
65             }
66         }
67
68         last = cur;
69     }
70
71     long long countDistinct()
72     {
73         long long ans = 0;
74
75         for(int v = 1; v < (int)st.size(); v++)
76         {
77         }
```

23.4 Distinct Substrings - Suffix Automaton (Contar Distintas) (cont.)

```

87         ans += st[v].len - st[st[v].link].len;
88     }
89
90     return ans;
91 }
92 };
93
94 int main()
95 {
96     ios::sync_with_stdio(false);
97     cin.tie(nullptr);
98
99     string s;
100     cin >> s;
101
102     SuffixAutomaton sam(s.size());
103
104     for(char c : s)
105     {
106         sam.add(c);
107     }
108
109     cout << sam.countDistinct() << '\n';
110
111     return 0;
112 }

```

23.5 Finding Borders - Funcion de Prefijos (Cadena de Bordes)

O: (n), funcion de prefijos

Todos los bordes de s (prefijos que tambien son sufijos) salen de la funcion de prefijos

C++/CSES/String Algorithms/Finding Borders.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     string s;
9     cin >> s;
10
11     int n = s.size();
12
13     vector<int> pi(n);
14
15     for (int i = 1; i < n; i++) {
16         int j = pi[i - 1];
17
18         while (j > 0 && s[i] != s[j])
19             j = pi[j - 1];
20
21         if (s[i] == s[j])
22             j++;
23
24         pi[i] = j;
25     }
26
27     vector<int> ans;
28
29     int j = pi[n - 1];
30
31     while (j > 0) {
32         ans.push_back(j);
33         j = pi[j - 1];
34     }
35
36     reverse(ans.begin(), ans.end());
37
38     for (int x : ans)
39         cout << x << ' ';

```

23.5 Finding Borders - Funcion de Prefijos (Cadena de Bordes) (cont.)

```

40
41     cout << '\n';
42 }

```

23.6 Finding Patterns - Aho-Corasick (Aparece o No)

O: (suma de patrones + |s| + 26*nodos)

Uso: igual que Counting Patterns, pero appears(p) devuelve si cnt > 0

Si cada patron aparece o no en el texto, con todos a la vez

C++/CSES/String Algorithms/Finding Patterns.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct AhoCorasick
5 {
6     struct Node
7     {
8         int next[26];
9         int link;
10        int cnt;
11
12        Node()
13        {
14            fill(next, next + 26, -1);
15            link = 0;
16            cnt = 0;
17        }
18    };
19
20    vector<Node> trie;
21    vector<int> order;
22
23    AhoCorasick()
24    {
25        trie.push_back(Node());
26    }
27
28    void addString(string s)
29    {
30        int u = 0;
31
32        for(char c : s)
33        {
34            int x = c - 'a';
35
36            if(trie[u].next[x] == -1)
37            {
38                trie[u].next[x] = trie.size();
39                trie.push_back(Node());
40            }
41
42            u = trie[u].next[x];
43        }
44    }
45
46    void build()
47    {
48        queue<int> q;
49
50        for(int c = 0; c < 26; c++)
51        {
52            int v = trie[0].next[c];
53
54            if(v == -1)
55            {
56                trie[0].next[c] = 0;
57            }
58            else
59            {
60                trie[v].link = 0;
61                q.push(v);

```

23.6 Finding Patterns - Aho-Corasick (Aparece o No) (cont.)

```

62     }
63 }
64
65 while(!q.empty())
66 {
67     int u = q.front();
68     q.pop();
69     order.push_back(u);
70
71     for(int c = 0; c < 26; c++)
72     {
73         int v = trie[u].next[c];
74
75         if(v == -1)
76         {
77             trie[u].next[c] = trie[trie[u].link].next[c];
78         }
79         else
80         {
81             trie[v].link = trie[trie[u].link].next[c];
82             q.push(v);
83         }
84     }
85 }
86
87 }
88
89 void search(string s)
90 {
91     int u = 0;
92
93     for(char c : s)
94     {
95         u = trie[u].next[c - 'a'];
96         trie[u].cnt++;
97     }
98 }
99
100 void propagate()
101 {
102     for(int i = order.size() - 1; i >= 0; i--)
103     {
104         int u = order[i];
105
106         trie[trie[u].link].cnt += trie[u].cnt;
107     }
108 }
109
110 bool appears(string s)
111 {
112     int u = 0;
113
114     for(char c : s)
115     {
116         u = trie[u].next[c - 'a'];
117     }
118     return trie[u].cnt > 0;
119 }
120
121 };
122
123 int main()
124 {
125     ios::sync_with_stdio(false);
126     cin.tie(nullptr);
127
128     string s;
129     cin >> s;
130
131     int k;
132     cin >> k;
133 }

```

23.6 Finding Patterns - Aho-Corasick (Aparece o No) (cont.)

```

134 vector<string> patterns(k);
135
136 AhoCorasick ac;
137
138 for(int i = 0; i < k; i++)
139 {
140     cin >> patterns[i];
141     ac.addString(patterns[i]);
142 }
143
144 ac.build();
145
146 ac.search(s);
147
148 ac.propagate();
149
150 for(string p : patterns)
151 {
152     if(ac.appears(p))
153         cout << "YES\n";
154     else
155         cout << "NO\n";
156 }
157
158 return 0;
159 }

```

23.7 Finding Periods - Periodos = n - Bordes

O: (n), los periodos salen de la cadena de bordes

Un periodo p de s es un desplazamiento tal que $s[i] = s[i+p]$ para todo i valido

C++/CSES/String Algorithms/Finding Periods.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> KMP(string s) {
5     int n = s.size();
6     vector<int> pi(n);
7
8     for (int i = 1; i < n; i++) {
9         int j = pi[i - 1];
10
11         while (j > 0 && s[i] != s[j])
12             j = pi[j - 1];
13
14         if (s[i] == s[j])
15             j++;
16
17         pi[i] = j;
18     }
19
20     return pi;
21 }
22
23 int main() {
24     ios::sync_with_stdio(false);
25     cin.tie(nullptr);
26
27     string s;
28     cin >> s;
29
30     int n = s.size();
31
32     vector<int> pi = KMP(s);
33
34     vector<int> ans;
35
36     int j = pi[n - 1];
37
38     while (j > 0) {
39         ans.push_back(n - j);

```

23.7 Finding Periods - Periodos = n - Bordes (cont.)

```

40     j = pi[j - 1];
41 }
42
43 ans.push_back(n);
44
45 sort(ans.begin(), ans.end());
46
47 for (int x : ans)
48     cout << x << ' ';
49
50 cout << '\n';
51 }

```

23.8 Inverse Suffix Array - Reconstruir la Cadena desde el Suffix Array

O: $(n \log n)$ por el segment tree de maximos

Uso: greedy sobre sa; seg guarda la letra usada indexada por rank[p+1]

Reconstruir la cadena mas chica que produce un suffix array dado

C++/ICSES/String Algorithms/Inverse Suffix Array.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct SegmentTree
5  {
6      int n;
7      vector<int> tree;
8
9      SegmentTree(int n)
10     {
11         this->n = n;
12         tree.assign(4 * n, -1);
13     }
14
15     void update(int node, int l, int r, int pos, int val)
16     {
17         if(l == r)
18         {
19             tree[node] = val;
20             return;
21         }
22
23         int mid = (l + r) / 2;
24
25         if(pos <= mid)
26             update(node * 2, l, mid, pos, val);
27         else
28             update(node * 2 + 1, mid + 1, r, pos, val);
29
30         tree[node] = max(tree[node * 2], tree[node * 2 + 1]);
31     }
32
33     void update(int pos, int val)
34     {
35         update(1, 0, n - 1, pos, val);
36     }
37
38     int query(int node, int l, int r, int ql, int qr)
39     {
40         if(qr < l || r < ql)
41             return -1;
42
43         if(ql <= l && r <= qr)
44             return tree[node];
45
46         int mid = (l + r) / 2;
47
48         return max(
49             query(node * 2, l, mid, ql, qr),
50             query(node * 2 + 1, mid + 1, r, ql, qr)
51         );
52     }

```

23.8 Inverse Suffix Array - Reconstruir la Cadena desde el Suffix Array (cont.)

```

53
54 int query(int l, int r)
55 {
56     if(l > r)
57         return -1;
58
59     return query(1, 0, n - 1, l, r);
60 }
61 };
62
63 int main()
64 {
65     ios::sync_with_stdio(false);
66     cin.tie(nullptr);
67
68     int n;
69     cin >> n;
70
71     vector<int> sa(n);
72
73     for(int i = 0; i < n; i++)
74     {
75         cin >> sa[i];
76         sa[i]--; // pasamos a 0-index
77     }
78
79     // rank[x] = posicion de x dentro del suffix array
80     vector<int> rank(n);
81
82     for(int i = 0; i < n; i++)
83         rank[sa[i]] = i;
84
85     string s(n, 'a');
86
87     SegmentTree seg(n);
88
89     int c = 0;
90
91     for(int i = 0; i < n; i++)
92     {
93         int p = sa[i];
94
95         // Si estamos colocando el sufijo de longitud 1,
96         // tiene que ser estrictamente mayor que el anterior.
97         if(i > 0 && p == n - 1)
98             c++;
99
100        else if(p < n - 1)
101        {
102            // Miramos los sufijos anteriores cuyo siguiente
103            // sufijo aparece despues de p+1.
104            int r = rank[p + 1];
105
106            int mx = seg.query(r + 1, n - 1);
107
108            // Si alguno tiene nuestra misma letra,
109            // necesitamos aumentar la letra.
110            if(mx == c)
111                c++;
112        }
113
114        if(c >= 26)
115        {
116            cout << -1 << '\n';
117            return 0;
118        }
119
120        s[p] = char('a' + c);
121
122        // Guardamos la letra de s[p] asociada a rank[p+1]
123        if(p < n - 1)
124            seg.update(rank[p + 1], c);

```

23.8 Inverse Suffix Array - Reconstruir la Cadena desde el Suffix Array (cont.)

```

85     }
86
87     cout << s << '\n';
88
89     return 0;
90 }

```

23.9 Longest Palindrome - Manacher

O: (n), Manacher

Manacher: el palindromo mas largo en O(n)

C++/CSES/String Algorithms/Longest Palindrome.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  vector<int> manacher(string s) {
5      string t = "#";
6
7      for (char c : s) {
8          t += c;
9          t += '#';
10     }
11
12     int n = t.size();
13
14     vector<int> p(n);
15
16     int center = 0;
17     int right = 0;
18
19     for (int i = 0; i < n; i++) {
20         int mirror = 2 * center - i;
21
22         if (i < right)
23             p[i] = min(right - i, p[mirror]);
24
25         while (i + 1 + p[i] < n &&
26               i - 1 - p[i] >= 0 &&
27               t[i + 1 + p[i]] == t[i - 1 - p[i]]) {
28             p[i]++;
29         }
30
31         if (i + p[i] > right) {
32             center = i;
33             right = i + p[i];
34         }
35     }
36
37     return p;
38 }
39
40 int main() {
41     ios::sync_with_stdio(false);
42     cin.tie(nullptr);
43
44     string s;
45     cin >> s;
46
47     vector<int> p = manacher(s);
48
49     int pos = max_element(p.begin(), p.end()) - p.begin();
50     int len = p[pos];
51
52     int start = (pos - len) / 2;
53
54     cout << s.substr(start, len) << '\n';
55
56     for(int i = 0 ; i < p.size();i++)
57     {
58         if(i%2!=0)
59         {

```

23.9 Longest Palindrome - Manacher (cont.)

```

60         cout << p[i] << endl;
61     }
62
63 }
64
65 return 0;
66 }

```

23.10 Minimal Rotation - Rotacion Minima (Dos Punteros)

O: (n), dos punteros sobre s+s

La rotacion lexicograficamente menor en O(n) con dos candidatos i y j

C++/CSES/String Algorithms/Minimal Rotation.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  string minimalRotation(string s) {
5      int n = s.size();
6
7      s += s;
8
9      int i = 0, j = 1;
10
11     while (i < n && j < n) {
12         int k = 0;
13
14         while (k < n && s[i + k] == s[j + k])
15             k++;
16
17         if (k == n)
18             break;
19
20         if (s[i + k] > s[j + k])
21             i = i + k + 1;
22         else
23             j = j + k + 1;
24
25         if (i == j)
26             j++;
27     }
28
29     int pos = min(i, j);
30
31     return s.substr(pos, n);
32 }
33
34 int main() {
35     ios::sync_with_stdio(false);
36     cin.tie(nullptr);
37
38     string s;
39     cin >> s;
40
41     cout << minimalRotation(s) << '\n';
42 }

```

23.11 Palindrome Queries - Hashing Doble con Segment Tree

O: (log n) por operacion, hashing doble en segment tree

Uso: un arbol con la cadena y otro con la invertida; palindromo si coinciden

Hay actualizaciones de un caracter mezcladas con las preguntas

C++/CSES/String Algorithms/Palindrome Queries.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  const ll MOD = 1000000007;
7  const ll BASE = 911382323;
8
9  struct Node {

```

23.11 Palindrome Queries - Hashing Doble con Segment Tree (cont.)

```

0  ll fw = 0; // izquierda -> derecha
1  ll bw = 0; // derecha -> izquierda
2  int len = 0;
3  };
4
5  vector<ll> pw;
6  vector<Node> st;
7
8  Node mergeNode(Node a, Node b) {
9      if (a.len == 0) return b;
10     if (b.len == 0) return a;
11
12     Node res;
13
14     res.len = a.len + b.len;
15
16     // a + b
17     res.fw = (a.fw * pw[b.len] + b.fw) % MOD;
18
19     // b + a
20     res.bw = (b.bw * pw[a.len] + a.bw) % MOD;
21
22     return res;
23 }
24
25 void build(int p, int l, int r, string &s) {
26     if (l == r) {
27         st[p].fw = s[l] - 'a' + 1;
28         st[p].bw = s[l] - 'a' + 1;
29         st[p].len = 1;
30         return;
31     }
32
33     int mid = (l + r) / 2;
34
35     build(p * 2, l, mid, s);
36     build(p * 2 + 1, mid + 1, r, s);
37
38     st[p] = mergeNode(st[p * 2], st[p * 2 + 1]);
39 }
40
41 void update(int p, int l, int r, int pos, char c) {
42     if (l == r) {
43         st[p].fw = c - 'a' + 1;
44         st[p].bw = c - 'a' + 1;
45         return;
46     }
47
48     int mid = (l + r) / 2;
49
50     if (pos <= mid)
51         update(p * 2, l, mid, pos, c);
52     else
53         update(p * 2 + 1, mid + 1, r, pos, c);
54
55     st[p] = mergeNode(st[p * 2], st[p * 2 + 1]);
56 }
57
58 Node query(int p, int l, int r, int ql, int qr) {
59     if (qr < l || r < ql)
60         return Node();
61
62     if (ql <= l && r <= qr)
63         return st[p];
64
65     int mid = (l + r) / 2;
66
67     Node left = query(p * 2, l, mid, ql, qr);
68     Node right = query(p * 2 + 1, mid + 1, r, ql, qr);
69
70     return mergeNode(left, right);
71 }

```

23.11 Palindrome Queries - Hashing Doble con Segment Tree (cont.)

```

82
83 int main() {
84     ios::sync_with_stdio(false);
85     cin.tie(nullptr);
86
87     int n, m;
88     cin >> n >> m;
89
90     string s;
91     cin >> s;
92
93     // Pasamos a 0-index
94     // s[0] corresponde a posicion 1.
95
96     pw.resize(n + 1);
97     pw[0] = 1;
98
99     for (int i = 1; i <= n; i++)
100         pw[i] = pw[i - 1] * BASE % MOD;
101
102     st.resize(4 * n + 5);
103
104     build(1, 0, n - 1, s);
105
106     while (m--) {
107         int type;
108         cin >> type;
109
110         if (type == 1) {
111             int k;
112             char x;
113
114             cin >> k >> x;
115
116             --k;
117
118             update(1, 0, n - 1, k, x);
119
120         } else {
121             int a, b;
122             cin >> a >> b;
123
124             --a;
125             --b;
126
127             Node ans = query(1, 0, n - 1, a, b);
128
129             if (ans.fw == ans.bw)
130                 cout << "YES\n";
131             else
132                 cout << "NO\n";
133         }
134     }
135 }

```

23.12 Pattern Positions - Aho-Corasick (Primera Aparicion)

O: (suma de patrones + |s| + 26*nodos)

Uso: search() guarda min(first); getFirst(p) devuelve first - |p| + 1, o -1
Donde empieza la primera aparicion de cada patron, o -1 si no esta

C++/CSES/String Algorithms/Pattern Positions.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct AhoCorasick
5  {
6      struct Node
7      {

```

23.12 Pattern Positions - Aho-Corasick (Primera Aparición) (cont.)

```

8     int next[26];
9     int link;
10    int first;
11
12    Node()
13    {
14        fill(next, next + 26, -1);
15        link = 0;
16        first = INT_MAX;
17    }
18 };
19
20 vector<Node> trie;
21 vector<int> order;
22
23 AhoCorasick()
24 {
25     trie.push_back(Node());
26 }
27
28 void addString(string s)
29 {
30     int u = 0;
31
32     for(char c : s)
33     {
34         int x = c - 'a';
35
36         if(trie[u].next[x] == -1)
37         {
38             trie[u].next[x] = trie.size();
39             trie.push_back(Node());
40         }
41
42         u = trie[u].next[x];
43     }
44 }
45
46 void build()
47 {
48     queue<int> q;
49
50     for(int c = 0; c < 26; c++)
51     {
52         int v = trie[0].next[c];
53
54         if(v == -1)
55         {
56             trie[0].next[c] = 0;
57         }
58         else
59         {
60             trie[v].link = 0;
61             q.push(v);
62         }
63     }
64
65     while(!q.empty())
66     {
67         int u = q.front();
68         q.pop();
69
70         order.push_back(u);
71
72         for(int c = 0; c < 26; c++)
73         {
74             int v = trie[u].next[c];
75
76             if(v == -1)
77             {
78                 trie[u].next[c] = trie[trie[u].link].next[c];
79             }

```

23.12 Pattern Positions - Aho-Corasick (Primera Aparición) (cont.)

```

80         else
81         {
82             trie[v].link = trie[trie[u].link].next[c];
83             q.push(v);
84         }
85     }
86 }
87
88 void search(string s)
89 {
90     int u = 0;
91
92     for(int i = 0; i < (int)s.size(); i++)
93     {
94         u = trie[u].next[s[i] - 'a'];
95
96         trie[u].first = min(trie[u].first, i + 1);
97     }
98 }
99
100 void propagate()
101 {
102     for(int i = order.size() - 1; i >= 0; i--)
103     {
104         int u = order[i];
105
106         trie[trie[u].link].first =
107             min(trie[trie[u].link].first, trie[u].first);
108     }
109 }
110
111 int getFirst(string s)
112 {
113     int u = 0;
114
115     for(char c : s)
116     {
117         u = trie[u].next[c - 'a'];
118     }
119
120     if(trie[u].first == INT_MAX)
121         return -1;
122
123     return trie[u].first - (int)s.size() + 1;
124 }
125 };
126
127 int main()
128 {
129     ios::sync_with_stdio(false);
130     cin.tie(nullptr);
131
132     string s;
133     cin >> s;
134
135     int k;
136     cin >> k;
137
138     vector<string> patterns(k);
139
140     AhoCorasick ac;
141
142     for(int i = 0; i < k; i++)
143     {
144         cin >> patterns[i];
145         ac.addString(patterns[i]);
146     }
147
148     ac.build();
149
150     ac.search(s);

```

23.12 Pattern Positions - Aho-Corasick (Primera Aparicion) (cont.)

```
52
53     ac.propagate();
54
55     for(string p : patterns)
56     {
57         cout << ac.getFirst(p) << '\n';
58     }
59
60     return 0;
61 }
```

23.13 Repeating Substring - Suffix Automaton (Ocurrencias por Estado)

O: (n), construir mas ordenar por largo con counting sort

Uso: sam.longestRepeating(s) -> la subcadena mas larga que sale 2+ veces, o -1

La subcadena mas larga que aparece al menos dos veces

C++/CSES/String Algorithms/Repeating Substring.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct State
5 {
6     int next[26];
7     int link;
8     int len;
9     int occ;
10    int firstPos;
11
12    State()
13    {
14        fill(next, next + 26, -1);
15        link = -1;
16        len = 0;
17        occ = 0;
18        firstPos = -1;
19    }
20 };
21
22 struct SuffixAutomaton
23 {
24     vector<State> st;
25     int last;
26
27     SuffixAutomaton(int n)
28     {
29         st.reserve(2 * n);
30         st.push_back(State());
31
32         last = 0;
33     }
34
35     void add(char c, int pos)
36     {
37         int x = c - 'a';
38
39         int cur = st.size();
40         st.push_back(State());
41
42         st[cur].len = st[last].len + 1;
43         st[cur].firstPos = pos;
44         st[cur].occ = 1;
45
46         int p = last;
47
48         while(p != -1 && st[p].next[x] == -1)
49         {
50             st[p].next[x] = cur;
51             p = st[p].link;
52         }
53
54         if(p == -1)
```

23.13 Repeating Substring - Suffix Automaton (Ocurrencias por Estado) (cont.)

```
55     {
56         st[cur].link = 0;
57     }
58     else
59     {
60         int q = st[p].next[x];
61
62         if(st[p].len + 1 == st[q].len)
63         {
64             st[cur].link = q;
65         }
66         else
67         {
68             int clone = st.size();
69             st.push_back(st[q]);
70
71             st[clone].len = st[p].len + 1;
72             st[clone].occ = 0;
73
74             while(p != -1 && st[p].next[x] == q)
75             {
76                 st[p].next[x] = clone;
77                 p = st[p].link;
78             }
79
80             st[q].link = clone;
81             st[cur].link = clone;
82         }
83     }
84
85     last = cur;
86 }
87
88 string longestRepeating(string s)
89 {
90     int n = s.size();
91
92     for(int i = 0; i < n; i++)
93     {
94         add(s[i], i);
95     }
96
97     vector<int> cnt(n + 1);
98
99     for(auto &state : st)
100    {
101        cnt[state.len]++;
102    }
103
104    for(int i = 1; i <= n; i++)
105    {
106        cnt[i] += cnt[i - 1];
107    }
108
109    vector<int> order(st.size());
110
111    for(int i = st.size() - 1; i >= 0; i--)
112    {
113        order[--cnt[st[i].len]] = i;
114    }
115
116    // Propagamos ocurrencias desde estados largos
117    // hacia sus suffix links.
118    for(int i = order.size() - 1; i > 0; i--)
119    {
120        int u = order[i];
121
122        st[st[u].link].occ += st[u].occ;
123    }
124
125    int best = 0;
126    int bestState = -1;
```

23.13 Repeating Substring - Suffix Automaton (Ocurrencias por Estado) (cont.)

```

27
28     for(int i = 1; i < (int)st.size(); i++)
29     {
30         if(st[i].occ >= 2 && st[i].len > best)
31         {
32             best = st[i].len;
33             bestState = i;
34         }
35     }
36
37     if(best == 0)
38     {
39         return "-1";
40     }
41
42     int pos = st[bestState].firstPos;
43
44     return s.substr(pos - best + 1, best);
45 }
46
47 };
48
49 int main()
50 {
51     ios::sync_with_stdio(false);
52     cin.tie(nullptr);
53
54     string s;
55     cin >> s;
56
57     SuffixAutomaton sam(s.size());
58
59     cout << sam.longestRepeating(s) << '\n';
60
61     return 0;
62 }

```

23.14 Required Substring - DP sobre el Automata de KMP

O: (n * m * 26), DP sobre el automata de KMP

Cuenta cuantas cadenas de largo n sobre un alfabeto de 26 letras CONTIENEN un patron dado, modulo 1e9+7

C++/CSES/String Algorithms/Required Substring.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const ll MOD = 1000000007;
7
8 // -----
9 // KMP - Prefix Function
10 // -----
11
12 vector<int> prefixFunction(const string& s) {
13     int n = s.size();
14     vector<int> pi(n);
15
16     for (int i = 1; i < n; i++) {
17         int j = pi[i - 1];
18
19         while (j > 0 && s[i] != s[j])
20             j = pi[j - 1];
21
22         if (s[i] == s[j])
23             j++;
24
25         pi[i] = j;
26     }
27 }

```

23.14 Required Substring - DP sobre el Automata de KMP (cont.)

```

31
32     return pi;
33 }
34
35 // -----
36 // Transiciones de KMP
37 // -----
38
39 vector<vector<int>> buildKMPTransitions(const string& p) {
40     int m = p.size();
41     vector<int> pi = prefixFunction(p);
42     vector<vector<int>> nxt(m, vector<int>(26));
43
44     for (int j = 0; j < m; j++) {
45         for (int c = 0; c < 26; c++) {
46             char x = 'A' + c;
47             int k = j;
48
49             while (k > 0 && p[k] != x)
50                 k = pi[k - 1];
51
52             if (p[k] == x)
53                 k++;
54
55             nxt[j][c] = k;
56         }
57     }
58
59     return nxt;
60 }
61
62 // -----
63 // Main
64 // -----
65
66 int main() {
67     ios::sync_with_stdio(false);
68     cin.tie(nullptr);
69
70     int n;
71     string p;
72
73     cin >> n >> p;
74
75     int m = p.size();
76
77     auto nxt = buildKMPTransitions(p);
78
79     // Estados:
80     // 0 ... m-1 = cuanto del patron tenemos
81     // m = el patron ya aparecio
82     vector<ll> dp(m + 1);
83     vector<ll> ndp(m + 1);
84
85     dp[0] = 1;
86
87     for (int len = 0; len < n; len++) {
88         fill(ndp.begin(), ndp.end(), 0);
89
90         for (int j = 0; j <= m; j++) {
91             if (dp[j] == 0)

```

23.14 Required Substring - DP sobre el Automata de KMP (cont.)

```

03     continue;
04
05     // Ya aparecio el patron.
06     // Cualquier caracter que agreguemos mantiene
07     // este estado.
08     if (j == m) {
09
10         ndp[m] += dp[j] * 26;
11         ndp[m] %= MOD;
12
13         continue;
14     }
15
16     // Todavia no aparecio.
17     for (int c = 0; c < 26; c++) {
18
19         int k = nxt[j][c];
20
21         ndp[k] += dp[j];
22         ndp[k] %= MOD;
23     }
24 }
25
26 dp.swap(ndp);
27 }
28
29 cout << dp[m] << '\n';
30
31 return 0;
32 }

```

23.15 String Functions - Funcion Z y Funcion de Prefijos

O: (n) cada una

Uso: zFunction(s) -> z; prefixFunction(s) -> pi // ambos 0-indexados

Funcion Z y funcion de prefijos, las dos lado a lado

C++/CSES/String Algorithms/String functions.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  vector<int> zFunction(string s)
5  {
6      int n = s.size();
7
8      vector<int> z(n);
9      int l = 0;
10     int r = 0;
11
12     for(int i = 1; i < n; i++)
13     {
14         if(i <= r)
15         {
16             z[i] = min(r - i + 1, z[i - l]);
17         }
18
19         while(i + z[i] < n &&
20             s[z[i]] == s[i + z[i]])
21         {
22             z[i]++;
23         }
24
25         if(i + z[i] - 1 > r)
26         {
27             l = i;
28             r = i + z[i] - 1;
29         }
30     }
31
32     z[0] = 0;
33
34     return z;

```

23.15 String Functions - Funcion Z y Funcion de Prefijos (cont.)

```

35 }
36
37 vector<int> prefixFunction(string s)
38 {
39     int n = s.size();
40
41     vector<int> pi(n);
42
43     for(int i = 1; i < n; i++)
44     {
45         int j = pi[i - 1];
46
47         while(j > 0 && s[i] != s[j])
48         {
49             j = pi[j - 1];
50         }
51
52         if(s[i] == s[j])
53         {
54             j++;
55         }
56
57         pi[i] = j;
58     }
59
60     return pi;
61 }
62
63 int main()
64 {
65     ios::sync_with_stdio(false);
66     cin.tie(nullptr);
67
68     string s;
69     cin >> s;
70
71     vector<int> z = zFunction(s);
72     vector<int> pi = prefixFunction(s);
73
74     for(int i = 0; i < (int)s.size(); i++)
75     {
76         cout << z[i];
77
78         if(i + 1 < (int)s.size())
79             cout << ' ';
80     }
81
82     cout << '\n';
83
84     for(int i = 0; i < (int)s.size(); i++)
85     {
86         cout << pi[i];
87
88         if(i + 1 < (int)s.size())
89             cout << ' ';
90     }
91
92     cout << '\n';
93
94     return 0;
95 }

```

23.16 String Matching - KMP (Busqueda de Patron)

O: (n + m), KMP

KMP: cuantas veces aparece p en s, en O(n + m)

C++/CSES/String Algorithms/String Matching.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // Retorna cuantas veces aparece p dentro de s.

```

23.16 String Matching - KMP (Busqueda de Patron) (cont.)

```

5 // Permite apariciones solapadas.
6 int KMP(const string& s, const string& p) {
7
8     int n = s.size();
9     int m = p.size();
10
11     // Prefix function del patron
12     vector<int> pi(m);
13
14     for (int i = 1; i < m; i++) {
15
16         int j = pi[i - 1];
17
18         while (j > 0 && p[i] != p[j])
19             j = pi[j - 1];
20
21         if (p[i] == p[j])
22             j++;
23
24         pi[i] = j;
25     }
26
27     // Buscar p dentro de s
28     int ans = 0;
29     int j = 0;
30
31     for (int i = 0; i < n; i++) {
32
33         while (j > 0 && s[i] != p[j])
34             j = pi[j - 1];
35
36         if (s[i] == p[j])
37             j++;
38
39         if (j == m) {
40             ans++;
41
42             // Importante para permitir solapamientos
43             j = pi[j - 1];
44         }
45     }
46
47     return ans;
48 }
49
50 int main() {
51
52     ios::sync_with_stdio(false);
53     cin.tie(nullptr);
54
55     string s, p;
56
57     cin >> s >> p;
58
59     cout << KMP(s, p) << '\n';
60
61     return 0;
62 }

```

23.17 String Transformation - Transformada de Burrows-Wheeler Inversa

O: (n log n) por el stable_sort, (n) con counting sort

Uso: ordenar estable por caracter da nxt; seguir la cadena desde pos 0 y voltear
 Deshacer la transformada de Burrows-Wheeler

C++/CSES/String Algorithms/String Transformation.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     string s;

```

23.17 String Transformation - Transformada de Burrows-Wheeler Inversa (cont.)

```

7     cin >> s;
8
9     int n = s.size();
10
11     vector<int> orden(n);
12     iota(orden.begin(), orden.end(), 0);
13
14     stable_sort(orden.begin(), orden.end(), [&](int a, int b)
15     {
16         return s[a] < s[b];
17     });
18
19     vector<int> nxt(n);
20
21     for(int i = 0; i < n; i++)
22         nxt[orden[i]] = i;
23
24     int pos = 0;
25     string ans;
26
27     for(int i = 0; i < n - 1; i++)
28     {
29         ans += s[pos];
30         pos = nxt[pos];
31     }
32
33     reverse(ans.begin(), ans.end());
34
35     cout << ans << '\n';
36 }

```

23.18 Substring Distribution - Suffix Automaton + Arreglo de Diferencias

O: (n), un barrido de estados mas uno de largos

Uso: por cada estado v, diff[len[link]+1]++ y diff[len[v]+1]--; prefijos
 Cuantas subcadenas distintas hay de cada largo, de 1 hasta n

C++/CSES/String Algorithms/Substring Distribution.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct SAM
5 {
6     struct Node
7     {
8         int next[26];
9         int link, len;
10
11         Node()
12         {
13             memset(next, -1, sizeof(next));
14             link = -1;
15             len = 0;
16         }
17     };
18
19     vector<Node> st;
20     int last = 0;
21
22     SAM(int n)
23     {
24         st.reserve(2 * n);
25         st.push_back(Node());
26     }
27
28     void add(char ch)
29     {
30         int c = ch - 'a';
31
32         int cur = st.size();
33         st.push_back(Node());
34

```

23.18 Substring Distribution - Suffix Automaton + Arreglo de Diferencias (cont.)

```

35     st[cur].len = st[last].len + 1;
36
37     int p = last;
38
39     while(p != -1 && st[p].next[c] == -1)
40     {
41         st[p].next[c] = cur;
42         p = st[p].link;
43     }
44
45     if(p == -1)
46     {
47         st[cur].link = 0;
48     }
49     else
50     {
51         int q = st[p].next[c];
52
53         if(st[p].len + 1 == st[q].len)
54         {
55             st[cur].link = q;
56         }
57         else
58         {
59             int clone = st.size();
60             st.push_back(st[q]);
61
62             st[clone].len = st[p].len + 1;
63
64             while(p != -1 && st[p].next[c] == q)
65             {
66                 st[p].next[c] = clone;
67                 p = st[p].link;
68             }
69
70             st[q].link = st[cur].link = clone;
71         }
72     }
73
74     last = cur;
75 }
76 };
77
78 int main()
79 {
80     ios::sync_with_stdio(false);
81     cin.tie(nullptr);
82
83     string s;
84     cin >> s;
85
86     SAM sam(s.size());
87
88     for(char c : s)
89         sam.add(c);
90
91     vector<long long> diff(s.size() + 2);
92
93     for(int v = 1; v < sam.st.size(); v++)
94     {
95         int l = sam.st[sam.st[v].link].len + 1;
96         int r = sam.st[v].len;
97
98         diff[l]++;
99         diff[r + 1]--;
100     }
101
102     long long cur = 0;
103
104     for(int len = 1; len <= s.size(); len++)
105     {
106         cur += diff[len];

```

23.18 Substring Distribution - Suffix Automaton + Arreglo de Diferencias (cont.)

```

107         cout << cur << ' ';
108     }
109
110     cout << '\n';
111 }

```

23.19 Substring Order I - Descenso por el Suffix Automaton (Distintas)

O: (26*n) construir y bajar

Uso: dp[v] = cuantas subcadenas salen de v; se baja eligiendo letra con k
La k-esima subcadena DISTINTA en orden alfabetico

C++/CSES/String Algorithms/Substring Order I.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct SuffixAutomaton
7 {
8     struct State
9     {
10         int next[26];
11         int link;
12         int len;
13
14         State()
15         {
16             fill(next, next + 26, -1);
17             link = -1;
18             len = 0;
19         }
20     };
21
22     vector<State> st;
23     int last;
24
25     SuffixAutomaton(int n)
26     {
27         st.reserve(2 * n);
28         st.push_back(State());
29         last = 0;
30     }
31
32     void add(char ch)
33     {
34         int c = ch - 'a';
35
36         int cur = st.size();
37         st.push_back(State());
38
39         st[cur].len = st[last].len + 1;
40
41         int p = last;
42
43         while(p != -1 && st[p].next[c] == -1)
44         {
45             st[p].next[c] = cur;
46             p = st[p].link;
47         }
48
49         if(p == -1)
50         {
51             st[cur].link = 0;
52         }
53         else
54         {
55             int q = st[p].next[c];
56
57             if(st[p].len + 1 == st[q].len)
58             {
59                 st[cur].link = q;

```

23.19 Substring Order I - Descenso por el Suffix Automaton (Distintas) (cont.)

```

80     }
81     else
82     {
83         int clone = st.size();
84         st.push_back(st[q]);
85
86         st[clone].len = st[p].len + 1;
87
88         while(p != -1 && st[p].next[c] == q)
89         {
90             st[p].next[c] = clone;
91             p = st[p].link;
92         }
93
94         st[q].link = clone;
95         st[cur].link = clone;
96     }
97 }
98
99 last = cur;
100 }
101 };
102
103 int main()
104 {
105     ios::sync_with_stdio(false);
106     cin.tie(nullptr);
107
108     string s;
109     cin >> s;
110
111     ll k;
112     cin >> k;
113
114     SuffixAutomaton sam(s.size());
115
116     for(char c : s)
117         sam.add(c);
118
119     int sz = sam.st.size();
120
121     // Ordenamos los estados por longitud
122     vector<int> cnt(s.size() + 1);
123
124     for(auto &v : sam.st)
125         cnt[v.len]++;
126
127     for(int i = 1; i <= (int)s.size(); i++)
128         cnt[i] += cnt[i - 1];
129
130     vector<int> order(sz);
131
132     for(int i = sz - 1; i >= 0; i--)
133         order[--cnt[sam.st[i].len]] = i;
134
135     // dp[v] = cantidad de caminos/substrings desde v
136     // Incluye el substring vacío.
137     vector<ll> dp(sz, 1);
138
139     for(int i = sz - 1; i >= 0; i--)
140     {
141         int v = order[i];
142
143         for(int c = 0; c < 26; c++)
144         {
145             int to = sam.st[v].next[c];
146
147             if(to != -1)
148                 dp[v] += dp[to];
149         }
150     }
151 }

```

23.19 Substring Order I - Descenso por el Suffix Automaton (Distintas) (cont.)

```

82     string ans;
83     int v = 0;
84
85     while(k > 0)
86     {
87         for(int c = 0; c < 26; c++)
88         {
89             int to = sam.st[v].next[c];
90
91             if(to == -1)
92                 continue;
93
94             // Todos los substrings de este bloque
95             // empiezan con esta letra.
96             if(k > dp[to])
97             {
98                 k -= dp[to];
99             }
100             else
101             {
102                 ans.push_back(char('a' + c));
103
104                 // Si k == 1, el substring actual
105                 // es exactamente la respuesta.
106                 if(k == 1)
107                 {
108                     cout << ans << '\n';
109                     return 0;
110                 }
111
112                 // El resto esta dentro de este estado.
113                 k--;
114                 v = to;
115                 break;
116             }
117         }
118     }
119
120     return 0;
121 }

```

23.20 Substring Order II - Descenso por el Suffix Automaton (con Repetidas)

O: (26*n) construir y bajar

Uso: dp[v] suma occ de los hijos; cada subcadena pesa cuantas veces aparece
La k-esima subcadena en orden alfabético, contando las repetidas

C++/CSES/String Algorithms/Substring Order II.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct SuffixAutomaton
7 {
8     struct State
9     {
10         int next[26];
11         int link;
12         int len;
13         ll occ;
14
15         State()
16         {
17             fill(next, next + 26, -1);
18             link = -1;
19             len = 0;
20             occ = 0;
21         }
22     };
23
24     vector<State> st;

```

23.20 Substring Order II - Descenso por el Suffix Automaton (con Repetidas) (cont.)

```

25     int last;
26
27     SuffixAutomaton(int n)
28     {
29         st.reserve(2 * n);
30         st.push_back(State());
31         last = 0;
32     }
33
34     void add(char ch)
35     {
36         int c = ch - 'a';
37
38         int cur = st.size();
39         st.push_back(State());
40
41         st[cur].len = st[last].len + 1;
42         st[cur].occ = 1;
43
44         int p = last;
45
46         while(p != -1 && st[p].next[c] == -1)
47         {
48             st[p].next[c] = cur;
49             p = st[p].link;
50         }
51
52         if(p == -1)
53         {
54             st[cur].link = 0;
55         }
56         else
57         {
58             int q = st[p].next[c];
59
60             if(st[p].len + 1 == st[q].len)
61             {
62                 st[cur].link = q;
63             }
64             else
65             {
66                 int clone = st.size();
67                 st.push_back(st[q]);
68
69                 st[clone].len = st[p].len + 1;
70                 st[clone].occ = 0;
71
72                 while(p != -1 && st[p].next[c] == q)
73                 {
74                     st[p].next[c] = clone;
75                     p = st[p].link;
76                 }
77
78                 st[q].link = clone;
79                 st[cur].link = clone;
80             }
81         }
82
83         last = cur;
84     }
85 };
86
87 vector<int> buildOrder(SuffixAutomaton &sam, int n)
88 {
89     int sz = sam.st.size();
90
91     vector<int> cnt(n + 1);
92
93     for(auto &v : sam.st)
94         cnt[v.len]++;
95
96     for(int i = 1; i <= n; i++)

```

23.20 Substring Order II - Descenso por el Suffix Automaton (con Repetidas) (cont.)

```

97     cnt[i] += cnt[i - 1];
98
99     vector<int> order(sz);
100
101     for(int i = sz - 1; i >= 0; i--)
102         order[--cnt[sam.st[i].len]] = i;
103
104     return order;
105 }
106
107 void calculateOccurrences(SuffixAutomaton &sam, vector<int> &order)
108 {
109     for(int i = (int)order.size() - 1; i > 0; i--)
110     {
111         int v = order[i];
112
113         if(sam.st[v].link != -1)
114         {
115             sam.st[sam.st[v].link].occ += sam.st[v].occ;
116         }
117     }
118 }
119
120 vector<ll> calculateDP(SuffixAutomaton &sam, vector<int> &order)
121 {
122     int sz = sam.st.size();
123
124     vector<ll> dp(sz, 0);
125
126     for(int i = sz - 1; i >= 0; i--)
127     {
128         int v = order[i];
129
130         for(int c = 0; c < 26; c++)
131         {
132             int to = sam.st[v].next[c];
133
134             if(to == -1)
135                 continue;
136
137             dp[v] += sam.st[to].occ;
138             dp[v] += dp[to];
139         }
140     }
141
142     return dp;
143 }
144
145 string kthSubstring(SuffixAutomaton &sam, vector<ll> &dp, ll k)
146 {
147     string ans;
148     int v = 0;
149
150     while(true)
151     {
152         for(int c = 0; c < 26; c++)
153         {
154             int to = sam.st[v].next[c];
155
156             if(to == -1)
157                 continue;
158
159             // Todo el bloque que empieza con esta letra
160             ll block = sam.st[to].occ + dp[to];
161
162             if(k > block)
163             {
164                 k -= block;
165                 continue;
166             }
167
168             // La respuesta puede ser exactamente ans + c

```

23.20 Substring Order II - Descenso por el Suffix Automaton (con Repetidas) (cont.)

```

68         if(k <= sam.st[to].occ)
69         {
70             ans.push_back(char('a' + c));
71             return ans;
72         }
73     }
74     // Saltamos las ocurrencias del propio prefijo
75     k -= sam.st[to].occ;
76     ans.push_back(char('a' + c));
77     v = to;
78     break;
79 }
80 }
81 }
82 }
83 }
84 }
85 }
86 int main()
87 {
88     ios::sync_with_stdio(false);
89     cin.tie(nullptr);
90
91     string s;
92     cin >> s;
93
94     ll k;
95     cin >> k;
96
97     SuffixAutomaton sam(s.size());
98
99     for(char c : s)
100         sam.add(c);
101
102     vector<int> order = buildOrder(sam, s.size());
103
104     calculateOccurrences(sam, order);
105
106     vector<ll> dp = calculateDP(sam, order);
107
108     cout << kthSubstring(sam, dp, k) << '\n';
109
110     return 0;
111 }
```

23.21 Word Combinations - Trie + DP de Segmentacion (Conteo)

O: (n * largo maximo) tras armar el trie

Uso: Node con nxt[26] y end; dp[j+1] += dp[i] al cerrar una palabra

De cuantas formas se parte s en palabras del diccionario: trie mas DP de segmentacion

C++/CSSES/String Algorithms/Word Combinations.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const long long MOD = 1e9 + 7;
5
6 struct Node {
7     int nxt[26];
8     bool end;
9
10     Node() {
11         memset(nxt, -1, sizeof(nxt));
12         end = false;
13     }
14 };
15
16 int main() {
17     ios::sync_with_stdio(false);
18     cin.tie(nullptr);
19
20     string s;
21     cin >> s;
```

23.21 Word Combinations - Trie + DP de Segmentacion (Conteo) (cont.)

```

22
23     int k;
24     cin >> k;
25
26     vector<Node> trie(1);
27
28     while (k--) {
29         string w;
30         cin >> w;
31
32         int v = 0;
33
34         for (char c : w) {
35             int x = c - 'a';
36
37             if (trie[v].nxt[x] == -1) {
38                 trie[v].nxt[x] = trie.size();
39                 trie.emplace_back();
40             }
41
42             v = trie[v].nxt[x];
43         }
44
45         trie[v].end = true;
46     }
47
48     int n = s.size();
49
50     vector<long long> dp(n + 1);
51     dp[0] = 1;
52
53     for (int i = 0; i < n; i++) {
54         if (dp[i] == 0) continue;
55
56         int v = 0;
57
58         for (int j = i; j < n; j++) {
59             int x = s[j] - 'a';
60
61             if (trie[v].nxt[x] == -1)
62                 break;
63
64             v = trie[v].nxt[x];
65
66             if (trie[v].end) {
67                 dp[j + 1] += dp[i];
68
69                 if (dp[j + 1] >= MOD)
70                     dp[j + 1] -= MOD;
71             }
72         }
73     }
74
75     cout << dp[n] << '\n';
76 }
```

24 - CSSES: Tree Algorithms

24.1 Company Queries II - LCA con Binary Lifting

O: (n log n) de tabla y (log n) por consulta

Uso: up[k][u] = ancestro 2^k de u; lca(a,b) iguala alturas y despues sube los dos
El jefe comun mas cercano de dos empleados del arbol de la empresa

C++/Tree Algorithms/Company Queries II.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int LOG = 20;
5
6 int n, q;
```

24.1 Company Queries II - LCA con Binary Lifting (cont.)

```

1
2 vector<vector<int>> up;
3 vector<int> depth;
4
5 void dfs(int u, int p, vector<vector<int>>& adj)
6 {
7     up[0][u] = p;
8
9     for(int k = 1; k < LOG; k++)
10     {
11         up[k][u] = up[k - 1][up[k - 1][u]];
12     }
13
14     for(int v : adj[u])
15     {
16         if(v == p)
17             continue;
18
19         depth[v] = depth[u] + 1;
20         dfs(v, u, adj);
21     }
22 }
23
24 int lca(int a, int b)
25 {
26     if(depth[a] < depth[b])
27         swap(a, b);
28
29     // Igualar alturas
30     int diff = depth[a] - depth[b];
31
32     for(int k = 0; k < LOG; k++)
33     {
34         if(diff & (1 << k))
35             a = up[k][a];
36     }
37
38     if(a == b)
39         return a;
40
41     // Subir ambos
42     for(int k = LOG - 1; k >= 0; k--)
43     {
44         if(up[k][a] != up[k][b])
45         {
46             a = up[k][a];
47             b = up[k][b];
48         }
49     }
50
51     return up[0][a];
52 }
53
54 int main()
55 {
56     ios::sync_with_stdio(false);
57     cin.tie(nullptr);
58
59     cin >> n >> q;
60
61     vector<vector<int>> adj(n + 1);
62
63     for(int i = 2; i <= n; i++)
64     {
65         int p;
66         cin >> p;
67
68         adj[p].push_back(i);
69         adj[i].push_back(p);
70     }
71
72     up.assign(LOG, vector<int>(n + 1));

```

24.1 Company Queries II - LCA con Binary Lifting (cont.)

```

73     depth.assign(n + 1, 0);
74
75     depth[1] = 0;
76
77     dfs(1, 1, adj);
78
79     while(q--)
80     {
81         int a, b;
82         cin >> a >> b;
83
84         cout << lca(a, b) << '\n';
85     }
86 }

```

24.2 Path Queries II - Heavy-Light Decomposition

O: $(\log^2 n)$ por consulta: $\log n$ cadenas, cada una con una consulta de $\log n$ al segmen...

Uso: seg.update(pos[s], x) para cambiar un nodo; queryPath(a, b, seg) para el maximo Maximo valor en el camino entre dos nodos, con actualizaciones de nodo

C++/CSES/Tree Algorithms/Path Queries II.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct SegTree
7 {
8     int n;
9     vector<ll> tree;
10
11     SegTree(vector<ll>& a)
12     {
13         n = 1;
14         while(n < (int)a.size())
15             n *= 2;
16
17         tree.assign(2 * n, 0);
18
19         for(int i = 0; i < (int)a.size(); i++)
20             tree[n + i] = a[i];
21
22         for(int i = n - 1; i >= 1; i--)
23             tree[i] = max(tree[i * 2], tree[i * 2 + 1]);
24     }
25
26     void update(int pos, ll val)
27     {
28         pos += n;
29         tree[pos] = val;
30
31         pos /= 2;
32
33         while(pos)
34         {
35             tree[pos] = max(tree[pos * 2], tree[pos * 2 + 1]);
36             pos /= 2;
37         }
38     }
39
40     ll query(int l, int r)
41     {
42         l += n;
43         r += n;
44
45         ll ans = 0;
46
47         while(l <= r)
48         {
49             if(l & 1)
50                 ans = max(ans, tree[l++]);

```

24.2 Path Queries II - Heavy-Light Decomposition (cont.)

```

51
52     if(!(r & 1))
53         ans = max(ans, tree[r--]);
54
55     l /= 2;
56     r /= 2;
57 }
58
59 return ans;
60 }
61 };
62
63 int n, q;
64
65 vector<vector<int>> adj;
66
67 vector<int> parentNode;
68 vector<int> depth;
69 vector<int> sz;
70 vector<int> heavy;
71
72 vector<int> head;
73 vector<int> pos;
74
75 vector<ll> value;
76 vector<ll> base;
77
78 int timer = 0;
79
80 void dfs(int u, int p)
81 {
82     parentNode[u] = p;
83     sz[u] = 1;
84
85     int best = 0;
86
87     for(int v : adj[u])
88     {
89         if(v == p)
90             continue;
91
92         depth[v] = depth[u] + 1;
93
94         dfs(v, u);
95
96         sz[u] += sz[v];
97
98         if(sz[v] > best)
99         {
100             best = sz[v];
101             heavy[u] = v;
102         }
103     }
104 }
105
106 void decompose(int u, int h)
107 {
108     head[u] = h;
109     pos[u] = timer;
110     base[timer] = value[u];
111     timer++;
112
113     if(heavy[u] != -1)
114         decompose(heavy[u], h);
115
116     for(int v : adj[u])
117     {
118         if(v == parentNode[u] || v == heavy[u])
119             continue;
120
121         decompose(v, v);
122     }

```

24.2 Path Queries II - Heavy-Light Decomposition (cont.)

```

123 }
124
125 ll queryPath(int u, int v, SegTree& seg)
126 {
127     ll ans = 0;
128
129     while(head[u] != head[v])
130     {
131         if(depth[head[u]] < depth[head[v]])
132             swap(u, v);
133
134         ans = max(
135             ans,
136             seg.query(pos[head[u]], pos[u])
137         );
138
139         u = parentNode[head[u]];
140     }
141
142     if(depth[u] > depth[v])
143         swap(u, v);
144
145     ans = max(
146         ans,
147         seg.query(pos[u], pos[v])
148     );
149
150     return ans;
151 }
152
153 int main()
154 {
155     ios::sync_with_stdio(false);
156     cin.tie(nullptr);
157
158     cin >> n >> q;
159
160     value.resize(n + 1);
161
162     for(int i = 1; i <= n; i++)
163         cin >> value[i];
164
165     adj.resize(n + 1);
166
167     for(int i = 0; i < n - 1; i++)
168     {
169         int a, b;
170         cin >> a >> b;
171
172         adj[a].push_back(b);
173         adj[b].push_back(a);
174     }
175
176     parentNode.resize(n + 1);
177     depth.resize(n + 1);
178     sz.resize(n + 1);
179     heavy.assign(n + 1, -1);
180
181     head.resize(n + 1);
182     pos.resize(n + 1);
183
184     base.resize(n);
185
186     dfs(1, 0);
187     decompose(1, 1);
188
189     SegTree seg(base);
190
191     while(q--)
192     {
193         int type;
194         cin >> type;

```

24.2 Path Queries II - Heavy-Light Decomposition (cont.)

```

85
86     if(type == 1)
87     {
88         int s;
89         ll x;
90
91         cin >> s >> x;
92
93         seg.update(pos[s], x);
94     }
95     else
96     {
97         int a, b;
98         cin >> a >> b;
99
100        cout << queryPath(a, b, seg) << '\n';
101    }
102 }
103
104 return 0;
105 }
```

25 - Hub LeetCode: Arrays

25.1 3sum Closest - Ordenar y Dos Punteros

O: (n²) tras ordenar en O(n log n)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0016_3sum_closest.py

```

1 class Solution:
2     def threeSumClosest(self, nums: List[int], target: int) -> int:
3         nums = sorted(nums)
4         tam = len(nums)
5         mejor = float("inf")
6         guar = 0
7         for i in range(0,tam):
8             l = i+1
9             r = len(nums)-1
10            while l<r:
11                suma = nums[i]+nums[l]+nums[r]
12                abso = abs(target-suma)
13                if(suma>target):
14                    r -= 1
15                elif(suma<target):
16                    l += 1
17                else:
18                    return target
19                if(abso < mejor):
20                    mejor = abso
21                    guar = suma
22            return guar
```

25.2 Beautiful Array - Construcción Divide y Conquista

O: (n log n): T(n) = T(n/2) + O(n)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0932_beautiful_array.py

```

1 class Solution:
2     def beautifulArray(self, n: int) -> List[int]:
3
4         if n == 1:
5             return [1]
6
7         arr = self.beautifulArray((n + 1) // 2)
8
9         impares = []
10        pares = []
11
12        for x in arr:
13            if 2 * x - 1 <= n:
```

25.2 Beautiful Array - Construcción Divide y Conquista (cont.)

```

14            impares.append(2 * x - 1)
15
16            if 2 * x <= n:
17                pares.append(2 * x)
18
19        return impares + pares
```

25.3 Buy Two Chocolates - Los Dos mas Baratos

O: (n²) generando todos los pares; con los dos minimos seria (n)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/2706_buy_two_chocolates.py

```

1 class Solution:
2     def buyChoco(self, prices: List[int], money: int) -> int:
3         val = []
4         for i in range(0, len(prices)):
5             for j in range(i+1, len(prices)):
6                 val.append(prices[i]+prices[j])
7         minimo = min(val)
8         if(minimo>money):
9             return money
10        else:
11            return money-minimo
```

25.4 Check If Array Is Sorted And Rotated - Ordenado y Rotado

O: (n log n) por el sorted; contar las bajadas lo deja en (n)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/1752_check_if_array_is_sorted_and_rotated.py

```

1 class Solution:
2     def check(self, nums: List[int]) -> bool:
3         l = sorted(nums)
4         po = 0
5         for k in range(len(nums)-1,-1,-1):
6             #print(nums[k], l[0])
7             if(nums[k]==l[0] and nums[k-1]!=l[0]):
8                 po = k
9                 break
10        #print(po)
11        po2 = nums.index(l[0])
12        cant = 0
13        f = False
14        for h in range(po2,po+1):
15            if(nums[h]== l[0]):
16                continue
17            else:
18                f = True
19                break
20        if(f==False):
21            po = po2
22        else:
23            po = po
24        i = po
25        j = 0
26        flag = 0
27        while (j < len(nums) and flag == 0):
28            if(i==len(nums)):
29                i=0
30                if(nums[i] == l[j]):
31                    i+=1
32                    j+=1
33                    continue
34            else:
35                flag = 1
36                break
37        if(flag == 1):
38            return False
39        else:
40            return True
```

25.5 Cinema Seat Allocation - Grupos de Sillas por Fila**O:** (r log r) con r reservas; NO depende de n, que llega a 10⁹

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/1386_cinema_seat_allocation.py

```

1 class Solution:
2     def maxNumberOfFamilies(self, n: int, reservedSeats: List[List[int]]) -> int:
3         filas = {}
4         for i in range(len(reservedSeats)):
5             if(reservedSeats[i][0] in filas):
6                 filas[reservedSeats[i][0]].append(reservedSeats[i][1])
7             else:
8                 filas[reservedSeats[i][0]] = [reservedSeats[i][1]]
9         set1 = {2,3,4,5}
10        set2 = {4,5,6,7}
11        set3 = {6,7,8,9}
12        c = 0
13        f = 0
14        for clave,valor in filas.items():
15            a = [1,1,1]
16            for i in valor:
17                if(i in set1):
18                    a[0] = 0
19                if(i in set2):
20                    a[1] = 0
21                if(i in set3):
22                    a[2] = 0
23            if(a[0]==1 and a[2]==1):
24                c+=2
25            else:
26                c += min(1,sum(a))
27            f+=1
28
29        faltantes = n-f
30        c+= 2*faltantes
31
32        return c

```

25.6 Concatenation Of Array - Concatenacion**O:** (n)

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/1929_concatenation_of_array.py

```

1 class Solution:
2     def getConcatenation(self, nums: List[int]) -> List[int]:
3         return nums+nums

```

25.7 Construct Uniform Parity Array I - Paridad por Fuerza Bruta**O:** (n²), todos los pares

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/3875_construct_uniform_parity_array_i.py

```

1 class Solution:
2     def uniformArray(self, nums1: list[int]) -> bool:
3         lPar = []
4         lImpar = []
5         for i in range(len(nums1)):
6             if(nums1[i]%2==0):
7                 lPar.append(nums1[i])
8                 for j in range(len(nums1)):
9                     if(i!=j and (nums1[i]-nums1[j])%2==1):
10                        lImpar.append(nums1[i]-nums1[j])
11                        break
12            else:
13                lImpar.append(nums1[i])
14                for j in range(len(nums1)):
15                    if(i!=j and (nums1[i]-nums1[j])%2==0):
16                        lPar.append(nums1[i]-nums1[j])
17                        break
18
19        if(len(lPar)== len(nums1) or len(lImpar)==len(nums1)):

```

25.7 Construct Uniform Parity Array I - Paridad por Fuerza Bruta (cont.)

```

20        return True
21    else:
22        return False

```

25.8 Construct Uniform Parity Array li - Paridad con el Menor Impar**O:** (n), dos barridos quedandose con el m nimo de cada paridad

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/3876_construct_uniform_parity_array_ii.py

```

1 class Solution:
2     def uniformArray(self, nums1: list[int]) -> bool:
3         menor_par = float('inf')
4         posP = 0
5         menor_impar = float('inf')
6         posI = 0
7         pos = 0
8         for x in nums1:
9             if x % 2 == 0:
10                 if(menor_par > x):
11                     menor_par = x
12                 posP = pos
13             else:
14                 if(menor_impar > x):
15                     menor_impar = x
16                 posI = pos
17             pos+=1
18         #if(menor_par == float('inf') or menor_impar == float('inf')):
19             # return False
20
21         cp = 0
22         ci = 0
23         pos = 0
24         for i in nums1:
25             if(i %2 == 0):
26                 cp+=1
27             elif(pos != posI and i-menor_impar >= 1):
28                 cp+=1
29
30             if(i %2 == 1):
31                 ci+=1
32             elif(pos != posI and i-menor_impar >= 1):
33                 ci+=1
34             pos+=1
35
36         if(ci == len(nums1) or cp == len(nums1)):
37             return True
38
39         return False

```

25.9 Contains Duplicate - Buscar Repetidos**O:** (n), cortando en el primer repetido

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0217_contains_duplicate.py

```

1 class Solution:
2     def containsDuplicate(self, nums: List[int]) -> bool:
3         l = {}
4         for i in range(len(nums)):
5             if nums[i] in l:
6                 return True
7             else:
8                 l[nums[i]] = 1
9         return False

```

25.10 Count Subarrays With Majority Element I - Contar por Bloques Iguales**O:** (n) sobre los bloques, pero NO cuenta lo que pide el enunciado

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/3737_count_subarrays_with_majority_element_i.py

```

1 import math

```

25.10 Count Subarrays With Majority Element I - Contar por Bloques Iguales (cont.)

```

2
3 class Solution:
4     def countMajoritySubarrays(self, nums: List[int], target: int) -> int:
5         conteo = nums.count(target)
6         valF = 0
7         valF += conteo
8         j = 0
9         while j < len(nums):
10             c = 0
11             k = j
12             if(nums[j]==target):
13                 while k < len(nums):
14                     if(nums[k]==target):
15                         c+=1
16                     else:
17                         k+=1
18                         break
19                     k+=1
20             j=k
21             for i in range(1,c+1):
22                 valF += c-i
23             j+=1
24
25
26
27         return valF

```

25.11 Daily Temperatures - Siguiente Dia mas Caliente

O: (n^2) en el peor caso; con pila monotona es (n)

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

[leetcode/arrays/0739_daily_temperatures.py](#)

```

1 class Solution:
2
3     def calculo(self, temperatures, i):
4         j = i+1
5         c = 0
6         f = False
7         while j < len(temperatures):
8             if(temperatures[i]>= temperatures[j]):
9                 c+=1
10            else:
11                c+=1
12                f = True
13                break
14            j+=1
15        if(f):
16            return c
17        else:
18            return 0
19
20
21     def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
22
23         ma = max(temperatures)
24         fin = []
25         dic = {}
26         i = 0
27         while i < len(temperatures)-1:
28             if(temperatures[i]== ma):
29                 fin.append(0)
30             elif(temperatures[i] not in dic):
31                 dic[temperatures[i]] = [i]
32                 val = self.calculo(temperatures, i)
33                 fin.append(val)
34                 dic[temperatures[i]].append(val)
35             else:
36                 rest = (dic[temperatures[i]][1])-(i-dic[temperatures[i]][0])
37                 if(rest>-1):
38                     dic[temperatures[i]] = [i,rest]
39                     fin.append(rest)

```

25.11 Daily Temperatures - Siguiente Dia mas Caliente (cont.)

```

        else:
            val = self.calculo(temperatures, i)
            fin.append(val)
            dic[temperatures[i]].append(val)
        i+=1

    fin.append(0)
    return fin

```

25.12 Destroying Asteroids - Greedy Creciente

O: $(n \log n)$ por el sort

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

[leetcode/arrays/2126_destroying_asteroids.py](#)

```

1 class Solution:
2     def asteroidsDestroyed(self, mass: int, asteroids: List[int]) -> bool:
3         asteroids = sorted(asteroids)
4
5         for i in asteroids:
6             if(mass>=i):
7                 mass+=i
8             else:
9                 return False
10        return True

```

25.13 Digit Frequency Score - Frecuencia de Digits

O: (dígitos de n)

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

[leetcode/arrays/3945_digit_frequency_score.py](#)

```

1 class Solution:
2     def digitFrequencyScore(self, n: int) -> int:
3         v = str(n)
4         dicc = {}
5         for i in v:
6             if(i in dicc):
7                 dicc[i]+=1
8             else:
9                 dicc[i]=1
10        value = 0
11        for c,v in dicc.items():
12            value += (int(c)*v)
13
14        return value

```

25.14 Earliest Finish Time For Land And Water Rides I - Probar Todos los Pares

O: $(n \times m)$, todos los pares en los dos ordenes

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

[leetcode/arrays/3633_earliest_finish_time_for_land_and_water_rides_i.py](#)[illegible]

25.20 Find The Highest Altitude - Suma Acumulada (cont.)

```
4         for i in range(len(gain)):
5             g.append(gain[i]+g[-1])
6
7
8         return max(g)
```

25.21 Find The Prefix Common Array Of Two Arrays - Interseccion de Prefijos

O: (n^2) por rehacer los sets; incremental seria (n)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/2657_find_the_prefix_common_array_of_two_arrays.py

```
1 class Solution:
2     def findThePrefixCommonArray(self, A: List[int], B: List[int]) -> List[int]:
3         i = 0
4         c = []
5         cont = 0
6         while i < len(A):
7             set1 = set(A[0:i+1])
8             set2 = set(B[0:i+1])
9             cont = 0
10            for j in set1:
11                if(j in set2):
12                    cont+= 1
13            c.append(cont)
14            i+=1
15        return c
```

25.22 First Missing Positive - Primer Positivo Faltante

O: (n + maximo) tiempo, (n) memoria; se pedia O(1) de memoria

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0041_first_missing_positive.py

```
1 class Solution:
2     def firstMissingPositive(self, nums: List[int]) -> int:
3         n = set(nums)
4         ma = max(nums)
5         if(ma<1):
6             return 1
7         for i in range(1, ma):
8             if(i in n):
9                 continue
10            else:
11                return i
12        return ma+1
```

25.23 Hoy Many Numbers Are Smaller Than The Current Number - Fuerza Bruta O(n^2)

O: (n^2); con frecuencias y prefijos seria (n + rango)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/1365_hoy_many_numbers_are_smaller_than_the_current_number.py

```
1 class Solution:
2     def smallerNumbersThanCurrent(self, nums: List[int]) -> List[int]:
3         c = []
4         for i in range(len(nums)):
5             can = 0
6             for j in range(len(nums)):
7                 if(nums[i]>nums[j]):
8                     can += 1
9
10            c.append(can)
11        return c
```

25.24 Keep Multiplying Found Values By Two - Duplicar mientras Exista

O: (n log maximo): cada 'in' sobre la lista es (n); con set seria (log maximo)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/2154_keep_multiplying_found_values_by_two.py

```
1 class Solution:
2     def findFinalValue(self, nums: List[int], original: int) -> int:
3         if original in nums:
```

25.24 Keep Multiplying Found Values By Two - Duplicar mientras Exista (cont.)

```
4         while True:
5             original*=2
6             if original in nums:
7                 continue
8             else:
9                 break
10            return original
```

25.25 Kth Largest Element In An Array - Counting Sort Parcial

O: (n + rango); con heap (n log k), con quickselect (n)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0215_kth_largest_element_in_an_array.py

```
1 class Solution:
2     def findKthLargest(self, nums: List[int], k: int) -> int:
3         mi = float("inf")
4         ma = 0
5         dic = {}
6         for i in nums:
7             if(i>ma):
8                 ma = i
9             if(i<mi):
10                mi = i
11            if(i in dic):
12                dic[i]+=1
13            else:
14                dic[i]=1
15        c = 0
16        for i in range(ma,mi-1,-1):
17            try:
18                c += dic[i]
19                if(c>=k):
20                    break
21            except:
22                continue
23        return i
```

25.26 Last Stone Weight - Simulacion con los Dos Mayores

O: (n^2): cada max() y cada remove() recorren; con heap seria (n log n)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/1046_last_stone_weight.py

```
1 class Solution:
2     def lastStoneWeight(self, stones: List[int]) -> int:
3         while (len(stones)>1):
4             ma1 = max(stones)
5             stones.remove(ma1)
6             ma2 = max(stones)
7             stones.remove(ma2)
8             if(ma1==ma2):
9                 continue
10            else:
11                stones.append(abs(ma1-ma2))
12        if(len(stones)==0):
13            return 0
14        else:
15            return stones[0]
```

25.27 Left And Right Sum Differences - Sumas a Izquierda y Derecha

O: (n^2) por el sum() de cada posicion; con prefijos seria (n)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/2574_left_and_right_sum_differences.py

```
1 class Solution:
2     def leftRightDifference(self, nums: List[int]) -> List[int]:
3         f = []
4         for i in range(len(nums)):
5             ll = sum(nums[0:i])
6             rr = sum(nums[i+1:len(nums)])
7             f.append(abs(ll-rr))
```

25.27 Left And Righ Sum Differences - Sumas a Izquierda y Derecha (cont.)

```
8         return f
```

25.28 Magical String - Secuencia Autodescriptiva

O: (n), con el s += optimizado de CPython

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0481_magical_string.py

```
1 class Solution:
2     def magicalString(self, n: int) -> int:
3         chi = 2
4         cant = 1
5         s = "122"
6         F = True
7         i = 3
8         while i < n:
9             if(F):
10                can = int(s[chi])
11                chi+=1
12                so = "1"*can
13                s += so
14                i += can
15                if(i<=n):
16                    cant += can
17            else:
18                cant += 1
19                F= False
20        else:
21            can = int(s[chi])
22            chi+=1
23            so = "2"*can
24            s += so
25            i += can
26            F= True
27
28        return cant
```

25.29 Majority Element - Elemento Mayoritario

O: (n) tiempo y memoria; Boyer-Moore lo hace con (1)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0169_majority_element.py

```
1 class Solution:
2     def majorityElement(self, nums: List[int]) -> int:
3         l = {}
4         for i in range(len(nums)):
5             if (nums[i] in l):
6                 l[nums[i]]+=1
7             else:
8                 l[nums[i]]=1
9         mas = 0
10        v = 0
11        for i,j in l.items():
12            if(j > mas):
13                mas = j
14                v = i
15
16        return v
```

25.30 Max Consecutive Ones - Racha Maxima

O: (n), un contador que se reinicia en cada cero

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0485_max_consecutive_ones.py

```
1 class Solution:
2     def findMaxConsecutiveOnes(self, nums: List[int]) -> int:
3         ma = 0
4         co = 0
5         for i in range(len(nums)):
6             if(nums[i]==1):
7                 co += 1
```

25.30 Max Consecutive Ones - Racha Maxima (cont.)

```
8         else:
9             if(co>ma):
10                ma = co
11                co = 0
12            if(nums[-1]== 1 and co>ma):
13                ma = co
14        return ma
```

25.31 Maximum Ice Cream Bars - Greedy del mas Barato Primero

O: (n log n) por el sort, que es lo que ordena el greedy

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/1833_maximum_ice_cream_bars.py

```
1 class Solution:
2     def maxIceCream(self, costs: List[int], coins: int) -> int:
3         s = sorted(costs)
4         c = 0
5         cant = 0
6         for i in s:
7             c+=i
8             if(c > coins):
9                 break
10
11
12        cant+=1
13        return cant
```

25.32 Maximum Product Of Two Elements In An Array - Los Dos Mayores

O: (n log n) por el sort; con una pasada seria (n)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/1464_maximum_product_of_two_elements_in_an_array.py

```
1 class Solution:
2     def maxProduct(self, nums: List[int]) -> int:
3         t = sorted(nums)
4         return (t[len(t)-1]-1)*(t[len(t)-2]-1)
```

25.33 Maximum Total Subarray Value I - Observacion que Colapsa el Problema

O: (n), solo el maximo y el minimo

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/3689_maximum_total_subarray_value_i.py

```
1 class Solution:
2     def maxTotalValue(self, nums: List[int], k: int) -> int:
3         ma = max(nums)
4         mi = min(nums)
5         p = ma - mi
6         return p*k
```

25.34 Median Of Two Sorted Arrays - Avanzar hasta la Mitad

O: (n+m); el problema pide O(log(n+m)) con binaria

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0004_median_of_two_sorted_arrays.py

```
1 class Solution:
2     def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
3
4         i = 0
5         j = 0
6         val = 0
7         if (len(nums1) == 0 and len(nums2)==0):
8             actual = 0
9         elif(len(nums1)== 0):
10            actual = nums2[j]
11            j+=1
12        elif(len(nums2)==0):
13            actual = nums1[i]
14            i+=1
15
16        elif nums1[i] < nums2[j]:
```

25.34 Median Of Two Sorted Arrays - Avanzar hasta la Mitad (cont.)

```

17         actual = nums1[i]
18         i += 1
19     else:
20         actual = nums2[j]
21         j += 1
22
23     p = (len(nums1) + len(nums2)) // 2
24
25     while True:
26         ant = actual
27
28         if i < len(nums1) and j < len(nums2):
29
30             if nums1[i] < nums2[j]:
31
32                 if actual <= nums1[i]:
33                     actual = nums1[i]
34                     i += 1
35
36             else:
37
38                 if actual <= nums2[j]:
39                     actual = nums2[j]
40                     j += 1
41
42             elif i < len(nums1):
43
44                 if actual <= nums1[i]:
45                     actual = nums1[i]
46                     i += 1
47
48             elif j < len(nums2):
49
50                 if actual <= nums2[j]:
51                     actual = nums2[j]
52                     j += 1
53
54             val += 1
55
56             if val >= p:
57                 break
58
59         if (len(nums1) + len(nums2)) % 2 == 1:
60             return actual
61
62     else:
63
64
65
66     return (actual+ant)/2

```

25.35 Merge Intervals - Fusionar Intervalos

O: (n log n), manda el sort; el barrido es lineal

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0056_merge_intervals.py

```

1 class Solution:
2     def merge(self, intervals: List[List[int]]) -> List[List[int]]:
3         a= sorted(intervals)
4         final = []
5         ini = a[0][0]
6         fin = a[0][1]
7         for i in a:
8             if(ini == i[0] and fin == i[1]):
9                 continue
10            else:
11                if(i[0]>= ini and i[0]<= fin):
12                    if(i[1]>fin):
13                        fin = i[1]
14            else:
15                final.append([ini,fin])

```

25.35 Merge Intervals - Fusionar Intervalos (cont.)

```

6         ini = i[0]
7         fin = i[1]
8         final.append([ini,fin])
9         return final

```

25.36 Minimum Absolute Difference - Diferencia Minima en Ordenado

O: (n log n) por el sort; los dos barridos son lineales

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/1200_minimum_absolute_difference.py

```

1 class Solution:
2     def minimumAbsDifference(self, arr: List[int]) -> List[List[int]]:
3         a = sorted(arr)
4         mi = float("inf")
5         for i in range(1, len(arr)):
6             if(a[i]-a[i-1] < mi):
7                 mi = a[i]-a[i-1]
8
9         fin = []
10        for i in range(1, len(arr)):
11            if(a[i]-a[i-1] == mi):
12                fin.append([a[i-1], a[i]])
13        return fin

```

25.37 Minimum Cost Of Buying Candies With Discount - Greedy de Tercias

O: (n log n) por el sort

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/2144_minimum_cost_of_buying_candies_with_discount.py

```

1 class Solution:
2     def minimumCost(self, cost: List[int]) -> int:
3         cost = sorted(cost)
4         c = 0
5         i = len(cost)-1
6         t = 0
7         while i>=0:
8             if t==2:
9                 t=0
10                i-=1
11                continue
12                c+=cost[i]
13                t += 1
14                i-=1
15        return(c)

```

25.38 Minimum Element After Replacement With Digit Sum - Suma de Digos

O: (n * digitos)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/3300_minimum_element_after_replacement_with_digit_sum.py

```

1 class Solution:
2     def minElement(self, nums: List[int]) -> int:
3         me = 9999999999999
4         for i in nums:
5             h = str(i)
6             ca = 0
7             for j in h:
8                 ca += int(j)
9             if(ca<me):
10                me = ca
11        return me

```

25.39 Missing Number - Numero Faltante

O: (n log n) por el sort; con Gauss o XOR seria (n)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0268_missing_number.py

```

1 class Solution:
2     def missingNumber(self, nums: List[int]) -> int:
3         nu = len(nums)

```

25.39 Missing Number - Numero Faltante (cont.)

```

4         nums.sort()
5         for i in range(0,nu):
6             if i == nums[i]:
7                 continue
8             else:
9                 return i
10            return nu

```

25.40 Pascal's Triangle - Triangulo de Pascal

O: (n^2), que es el tamano de la salida

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0118_pascal's_triangle.py

```

1 class Solution:
2     def generate(self, numRows: int) -> List[List[int]]:
3         numFilas = numRows
4
5         matriz = []
6
7         fila = [1]
8
9         cont = 0
10
11        while cont < numFilas:
12
13            matriz.append(fila)
14
15            nueva = [1]
16
17            for i in range(len(fila) - 1):
18
19                nueva.append(fila[i] + fila[i + 1])
20
21            nueva.append(1)
22
23            fila = nueva
24            cont+=1
25
26
27
28        return(matriz)

```

25.41 Plus One - Sumar Uno a un Numero por Digitos

O: (L^2) con L digitos, por el 10^{*c} que se recalcula en cada paso

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0066_plus_one.py

```

1 class Solution:
2     def plusOne(self, digits: List[int]) -> List[int]:
3         c = len(digits)-1
4         v = 0
5         for i in digits:
6             v+= (i*10**c)
7             c-=1
8         v+=1
9         va = str(v)
10        l = []
11        for i in va:
12            l.append(int(i))
13        return l

```

25.42 Rank Transform Of An Array - Compresion de Coordenadas

O: ($n \log n$) por el sorted(set(arr))

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/1331_rank_transform_of_an_array.py

```

1 class Solution:
2     def arrayRankTransform(self, arr: List[int]) -> List[int]:
3         k = sorted(list(set(arr)))
4         dic = {}

```

25.42 Rank Transform Of An Array - Compresion de Coordenadas (cont.)

```

5
6         for i in range(len(k)):
7             dic[k[i]] = i+1
8         f = []
9         for i in range(len(arr)):
10            f.append(dic[arr[i]])
11        return f

```

25.43 Reduction Operations To Make The Array Elements Equal - Contar por Escalones

O: ($n \log n$) por el sort; contar escalones es lineal

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/1887_reduction_operations_to_make_the_array_elements_equal.py

```

1 class Solution:
2     def reductionOperations(self, nums: List[int]) -> int:
3         a = sorted(nums)
4         c = 0
5         k = a[0]
6         for i in range(1, len(a)):
7             if a[i]!=a[i-1]:
8                 c += len(a)-i
9         return c

```

25.44 Remove Covered Intervals - Intervalos Cubiertos

O: (n^2); ordenando bien seria ($n \log n$)

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/1288_remove_covered_intervals.py

```

1 class Solution:
2     def removeCoveredIntervals(self, intervals: List[List[int]]) -> int:
3         n = len(intervals)
4         deleted = set()
5         for i in range(len(intervals)):
6             for j in range(len(intervals)):
7                 if i != j:
8                     if intervals[i][0] <= intervals[j][0] and intervals[i][1] >= intervals[j][1] and (intervals[j][0], intervals[j][1]) not in deleted:
9                         n -= 1
10                        deleted.add((intervals[j][0], intervals[j][1]))
11        return n

```

25.45 Remove Duplicates From Sorted Array - Quitar Repetidos en Ordenado

O: (n^2): el pop(i) de cada paso es $O(n)$

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0026_remove_duplicates_from_sorted_array.py

```

1 class Solution:
2     def removeDuplicates(self, nums: List[int]) -> int:
3         cont = 0
4         for i in range(len(nums)-1,0,-1):
5             if(nums[i]==nums[i-1]):
6                 nums.pop(i)
7         return len(nums)

```

25.46 Remove Element - Filtrar en el Sitio

O: (n^2): cada remove() recorre la lista

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0027_remove_element.py

```

1 class Solution:
2     def removeElement(self, nums: List[int], val: int) -> int:
3         cont2 = 0
4         cont = 0
5         i = 0
6         while i< len(nums):
7             if(nums[i]==val):
8                 nums.remove(val)
9                 cont+=1
10            continue
11        cont2+=1

```

25.46 Remove Element - Filtrar en el Sitio (cont.)

```

1 i+=1
2 i=0
3 while i<cont:
4     nums.append("_")
5     i+=1
6
7
8     return cont2

```

25.47 Search Insert Position - Búsqueda Lineal

O: (n) lineal; el problema pide O(log n) con binaria

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0035_search_insert_position.py

```

1 class Solution:
2     def searchInsert(self, nums: List[int], target: int) -> int:
3         for i in range(len(nums)):
4             if(nums[i]==target or nums[i]>target):
5                 return i
6         return len(nums)

```

25.48 Set Mismatch - Repetido y Faltante

O: (n log n) por el sort; con las sumas de Gauss seria (n)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0645_set_mismatch.py

```

1 class Solution:
2     def findErrorNums(self, nums: List[int]) -> List[int]:
3         nums = sorted(nums)
4         f = []
5         r = 0
6         fa = 0
7         for i in range(1,len(nums)):
8             if(nums[i]==nums[i-1]):
9                 r = nums[i]
10            elif(nums[i]-1!= nums[i-1]):
11                fa = nums[i]-1
12
13            if(fa == 0):
14                if(nums[0]== 1):
15                    fa = nums[i]+1
16                else:
17                    fa = 1
18
19            f = [r,fa]
20            return f

```

25.49 Shuffle The Array - Intercalar Dos Mitades

O: (n)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/1470_shuffle_the_array.py

```

1 class Solution:
2     def shuffle(self, nums: List[int], n: int) -> List[int]:
3         l = []
4         nums2 = nums[len(nums)//2:len(nums)]
5
6         for i in range(len(nums)//2):
7             l.append(nums[i])
8             l.append(nums2[i])
9         return l

```

25.50 Single Number - El que Aparece Una Vez

O: (n) tiempo y memoria; con XOR seria (1) de memoria

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0136_single_number.py

```

1 class Solution:
2     def singleNumber(self, nums: List[int]) -> int:
3         dic = {}
4         for i in nums:

```

25.50 Single Number - El que Aparece Una Vez (cont.)

```

5         if(i in dic):
6             dic[i]+=1
7         else:
8             dic[i] = 1
9         for clave,val in dic.items():
10            if(val == 1):
11                return clave

```

25.51 Smallest Index With Digit Sum Equal To Index - Suma de Dígitos

O: (n * dígitos)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/3550_smallest_index_with_digit_sum_equal_to_index.py

```

1 class Solution:
2     def smallestIndex(self, nums: List[int]) -> int:
3         for i in range(len(nums)):
4             val = sum(list(map(int,str(nums[i]))))
5             if(val == i):
6                 return i
7         return -1

```

25.52 Smallest Missing Integer Greater Than Sequential Prefix Sum - Prefijo Secuencial

O: (n + respuesta)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/2996_smallest_missing_integer_greater_than_sequential_prefix_sum.py

```

1 class Solution:
2     def missingInteger(self, nums: List[int]) -> int:
3         c = [nums[0]]
4         v = nums[0]
5         i = 1
6         while i<len(nums):
7             if(nums[i]-1 == nums[i-1]):
8                 v += nums[i]
9             else:
10                c.append(v)
11                break
12            i+=1
13        c.append(v)
14        print(c)
15        mas = max(c)
16        k = set(nums)
17        while True:
18            if(mas not in k):
19                return mas
20            mas += 1

```

25.53 Smallest Missing Multiple Of K - Buscar el Primer Múltiplo Ausente

O: (respuesta): prueba todos los enteros y filtra por k

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/3718_smallest_missing_multiple_of_k.py

```

1 class Solution:
2     def missingMultiple(self, nums: List[int], k: int) -> int:
3         t = set(nums)
4         m = 1
5         while True:
6             if(m%k==0 and m not in t):
7                 return m
8             m+=1

```

25.54 Smallest Stable Index I - Máximo por Prefijo y Mínimo por Sufijo

O: (n^2) por el insert(0,...) de cada paso; con append y [::-1] seria (n)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/3909_smallest_stable_index_i.py

```

1 class Solution:
2     def firstStableIndex(self, nums: list[int], k: int) -> int:
3         mayor = 0
4         mayores = []

```

25.54 *Smallest Stable Index I - Maximo por Prefijo y Minimo por Sufijo (cont.)*

```
5         for i in nums:
6             if i > mayor:
7                 mayor = i
8             mayores.append(mayor)
9         nums2 = nums[::-1]
10        menor = float('inf')
11        menores = []
12        for i in nums2:
13            if i < menor:
14                menor = i
15            menores.insert(0, menor)
16
17        for i in range(len(nums)):
18            if mayores[i]-menores[i] <= k:
19                return i
20        return -1
```

25.55 Smallest Stable Index II - Maximo por Prefijo y Minimo por Sufijo

O: (n) tiempo y memoria

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/3904_smallest_stable_index_ii.py

```
1 class Solution:
2     def firstStableIndex(self, nums: list[int], k: int) -> int:
3         menor = float('inf')
4         menores = []
5         for i in range(len(nums)-1,-1,-1):
6             if nums[i] < menor:
7                 menor = nums[i]
8             menores.append(menor)
9
10        mayor = 0
11        for i in range(len(nums)):
12            if nums[i] > mayor:
13                mayor = nums[i]
14            if(mayor-menores[len(menores)-1-i] <= k):
15                return i
16        return -1
```

25.56 Summary Ranges - Agrupar Rangos Consecutivos

O: (n), un solo barrido

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0228_summary_ranges.py

```
1 class Solution:
2     def summaryRanges(self, nums: List[int]) -> List[str]:
3
4
5
6        resultado = []
7
8        if len(nums) == 0:
9
10       return ([])
11
12       else:
13
14           a = nums[0]
15
16           try:
17
18               for i in range(len(nums)):
19
20                   b = nums[i + 1]
21
22                   if b == nums[i] + 1:
23
24                       continue
25
26                   else:
```

25.56 *Summary Ranges - Agrupar Rangos Consecutivos (cont.)*

```
28             if a == nums[i]:
29
30                 resultado.append(str(a))
31
32             else:
33
34                 resultado.append(f"{a}->{nums[i]}")
35
36                 a = b
37
38             except:
39
40                 if a == nums[-1]:
41
42                     resultado.append(str(a))
43
44                 else:
45
46                     resultado.append(f"{a}->{nums[-1]}")
47
48
49             return (resultado)
```

25.57 Two Sum - Fuerza Bruta O(n^2)

O: (n^2), dos ciclos anidados; con diccionario seria O(n)

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0001_two_sum.py

```
1 class Solution:
2     def twoSum(self, nums: List[int], target: int) -> List[int]:
3         for i in range(len(nums)):
4             for j in range(len(nums)):
5                 if((nums[i]+nums[j])==target and i!=j):
6                     return [i, j]
```

25.58 Valid Mountain Array - Verificar Sube y Baja

O: (n), un barrido con una bandera

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/arrays/0941_valid_mountain_array.py

```
1 class Solution:
2     def validMountainArray(self, arr: List[int]) -> bool:
3         subiendo = True
4         if len(arr) < 3:
5             return False
6         if arr[0] > arr[1]:
7             return False
8         for i in range(1, len(arr)):
9             diff = arr[i] - arr[i-1]
10            if diff == 0:
11                return False
12            elif subiendo and diff < 0:
13                subiendo = False
14            elif not subiendo and diff > 0:
15                return False
16            if subiendo:
17                return False
18            return True
```

26 - Hub LeetCode: Binary Search

26.1 Binary Search - Búsqueda Binaria

O: (log n), la plantilla de libro

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/binary_search/0704_binary_search.py

```
1 class Solution:
2     def search(self, nums: List[int], target: int) -> int:
3         ini = 0
4         fin = len(nums)-1
```

26.1 Binary Search - Búsqueda Binaria (cont.)

```

5         while ini <= fin:
6             mitad = (ini+fin)//2
7             if(nums[mitad]==target):
8                 return mitad
9             elif(nums[mitad]<target):
10                ini = mitad+1
11            else:
12                fin = mitad-1
13        return -1

```

26.2 Count Elements With At Least K Greater Values - Ordenar y Buscar

O: (n log n): sort mas una binaria por elemento

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/binary_search/3759_count_elements_with_at_least_k_greater_values.py

```

1 class Solution:
2     def countElements(self, nums: List[int], k: int) -> int:
3         if(k==0):
4             return len(nums)
5         nums.sort()
6         contF = 0
7         for i in range(len(nums)):
8             cont=0
9             if(len(nums)-i <= k):
10                break
11            else:
12                j = i
13                fi = len(nums)-1
14                while j <= fi:
15                    mitad = (j+fi)//2
16                    if(nums[i]<nums[mitad]):
17                        if(len(nums)-mitad >= k):
18                            contF+=1
19                            break
20                    else:
21                        fi = mitad -1
22                else:
23                    j = mitad + 1
24        return contF

```

26.3 Find Minimum In Rotated Sorted Array - Búsqueda en Arreglo Rotado

O: (log n)

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/binary_search/0153_find_minimum_in_rotated_sorted_array.py

```

1 class Solution:
2     def findMin(self, nums: List[int]) -> int:
3         n = len(nums)
4         ini = 0
5         fin = n-1
6         while (ini<fin):
7             m = (fin+ini)//2
8             if(nums[ini]>nums[m]):
9                 fin = m
10            elif(nums[fin]<nums[m]):
11                ini = m
12            else:
13                break
14
15            if(fin-ini==1):
16                break
17        if(nums[ini] > nums[fin]):
18            return (nums[fin])
19        else:
20            return(nums[ini])

```

26.4 Peak Index In A Mountain Array - Binaria sobre Monotonía

O: (log n)

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/binary_search/0852_peak_index_in_a_mountain_array.py

```

1 class Solution:
2     def peakIndexInMountainArray(self, arr: List[int]) -> int:
3         ini = 0
4         fin = len(arr)-1
5         while ini < fin:
6             mitad = (ini+fin)//2
7             if(arr[mitad]>arr[mitad+1]):
8                 fin = mitad
9             if(arr[mitad]>arr[mitad-1]):
10                ini = mitad
11        return mitad

```

26.5 Search In Rotated Sorted Array - Búsqueda en Arreglo Rotado

O: (n) en el peor caso por el ini+=1/fin-=1; la binaria limpia es (log n)

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/binary_search/0033_search_in_rotated_sorted_array.py

```

1 class Solution:
2     def search(self, nums: List[int], target: int) -> int:
3         ini = 0
4         fin = len(nums)-1
5         if(nums[ini]==target):
6             return ini
7         elif(nums[fin]==target):
8             return fin
9
10        ant = 0
11        ant2 = ini
12        ant3 = fin
13        m=0
14        while ini < fin:
15            m = (fin+ini)//2
16            print(ini,fin,m)
17            if(nums[m]==target or nums[ini]== target or nums[fin]==target):
18                break
19            if(target>nums[ini] and target<nums[m]):
20                fin = m
21            elif(target>nums[m] and target<nums[fin]):
22                ini = m
23        else:
24            ini += 1
25            fin -= 1
26
27        if(ant2 == ini and ant3 == fin):
28            break
29
30        ant = m
31        ant2 = ini
32        ant3 = fin
33        if(nums[m]==target):
34            return m
35        elif(nums[ini]==target):
36            return ini
37        elif(nums[fin]==target):
38            return fin
39        else:
40            return -1

```

27 - Hub LeetCode: Binary Tree

27.1 Construct Binary Tree From Inorder And Postorder Traversal - Divide y Conquista sobre R...

O: (n^2) por el `inorder.index()`; con un mapa de posiciones seria (n)

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/binary_tree/0106_construct_binary_tree_from_inorder_and_postorder_traversal.py

```
1 # Definition for a binary tree node.
2 # class TreeNode:
3 #     def __init__(self, val=0, left=None, right=None):
4 #         self.val = val
5 #         self.left = left
6 #         self.right = right
7
8 class Solution:
9
10     def construir(self, inorder, postorder):
11         if len(inorder) == 0:
12             return None
13         padre = postorder[-1]
14         nodo = TreeNode(padre)
15
16         pos = inorder.index(padre)
17
18         inorder_izq = inorder[:pos]
19         inorder_der = inorder[pos + 1:]
20         cantidad_izq = len(inorder_izq)
21
22         postorder_izq = postorder[:cantidad_izq]
23         postorder_der = postorder[cantidad_izq:-1]
24         nodo.left = self.construir(inorder_izq, postorder_izq)
25         nodo.right = self.construir(inorder_der, postorder_der)
26
27         return nodo
28
29     def buildTree(self, inorder: list[int], postorder: list[int]) -> TreeNode | None:
30
31         return self.construir(inorder, postorder)
```

27.2 Counts Nodex Equal To Average Of Subtree - DFS que Sube Informacion

O: (n), un DFS que sube suma y cantidad

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/binary_tree/2265_counts_nodex_equal_to_average_of_subtree.py

```
1 # Definition for a binary tree node.
2 # class TreeNode:
3 #     def __init__(self, val=0, left=None, right=None):
4 #         self.val = val
5 #         self.left = left
6 #         self.right = right
7
8 cont = 0
9
10 class Solution:
11
12     def dfs(self, nodo):
13         global cont
14         if(nodo == None):
15             return -1, -1
16         else:
17             val1, cant1 = self.dfs(nodo.left)
18             val2, cant2 = self.dfs(nodo.right)
19             if(val1 == -1 and val2 == -1):
20                 cont+=1
21                 return nodo.val, 1
22             elif(val1 != -1 and val2 != -1):
23                 val = (val1+val2+nodo.val)/(cant1+cant2+1)
24                 if(val == nodo.val):
25                     cont+=1
26                 return val1+val2+nodo.val, cant1+cant2+1
27             else:
28                 if(val1 != -1):
29                     val = (val1 + nodo.val) // (cant1 + 1)
30                     if val == nodo.val:
31                         cont += 1
32                 return val1 + nodo.val, cant1 + 1
```

27.2 Counts Nodex Equal To Average Of Subtree - DFS que Sube Informacion (cont.)

```
31         else:
32             val = (val2 + nodo.val) // (cant2 + 1)
33             if val == nodo.val:
34                 cont += 1
35             return val2 + nodo.val, cant2 + 1
36
37     def averageOfSubtree(self, root: TreeNode) -> int:
38         global cont
39         cont = 0
40         self.dfs(root)
41         return cont
```

27.3 Maximum Depth Of Binary Tree - DFS en Arbol Binario

O: (n), visita cada nodo una vez

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/binary_tree/0104_maximum_depth_of_binary_tree.py

```
1 # Definition for a binary tree node.
2 # class TreeNode:
3 #     def __init__(self, val=0, left=None, right=None):
4 #         self.val = val
5 #         self.left = left
6 #         self.right = right
7
8 class Solution:
9     def der(self, root, cont, l):
10         if(root.right == None):
11             l.append(cont)
12             return cont
13         root = root.right
14         cont+=1
15         self.der(root, cont, l)
16         self.izq(root, cont, l)
17
18     def izq(self, root, cont, l):
19         if(root.left == None):
20             l.append(cont)
21             return cont
22         root = root.left
23         cont+=1
24         self.izq(root, cont, l)
25         self.der(root, cont, l)
26
27     def maxDepth(self, root: Optional[TreeNode]) -> int:
28         l = []
29         if(root == None):
30             return 0
31         cont = 1
32         if(root.left == None and root.right == None):
33             return cont
34         self.der(root, cont, l)
35         self.izq(root, cont, l)
36         return (max(l))
```

28 - Hub LeetCode: Bit Manipulation**28.1 Minimum Number Of Flips To Reverse Binary String - Comparar un Binario con su Reverso**

O: (bits de n)

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/bit_manipulation/3750_minimum_number_of_flips_to_reverse_binary_string.py

```
1 class Solution:
2     def minimumFlips(self, n: int) -> int:
3         bi = bin(n)[2:]
4         bini = bi[::-1]
5         cont = 0
6         for i in range(len(bi)):
7             if(bi[i]!=bini[i]):
8                 cont+=1
9         return cont
```

29 - Hub LeetCode: Combinatorics

29.1 Count The Number Of Computer Unlocking Permutations - Factorial con un Filtro

O: (n) para el filtro, mas el factorial

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/combinatorics/3577_count_the_number_of_computer_unlocking_permutations.py

```
1 class Solution(object):
2     def countPermutations(self, complexity):
3         import math
4         menor = complexity[0]
5         for i in range(1, len(complexity)):
6             if(menor >= complexity[i]):
7                 return 0
8         return math.factorial(len(complexity)-1)%((10**9)+7)
```

29.2 Distribute Candies Among Children I - Fuerza Bruta con Poda

O: (limit^2); por inclusion-exclusion seria (1)

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/combinatorics/2928_distribute_candies_among_children_i.py

```
1 class Solution:
2     def distributeCandies(self, n: int, limit: int) -> int:
3         cont = 0
4         for val1 in range(limit+1):
5             if(val1+2*limit < n):
6                 continue
7             else:
8                 for val2 in range(limit+1):
9                     if(val1+val2+limit < n or (val1==val2 and val1== n-val1-val2) or n-val
10                        1-val2 < 0):
11                         continue
12                     else:
13                         print(val1)
14                         print(val2)
15                         print(n-val1-val2)
16                         print("-----")
17                         cont+=1
18                 for i in range(limit+1):
19                     if(i*3 == n):
20                         cont+=1
21         return (cont)
```

29.3 Number Of Sets Of K Non Overlapping Line Segments - Stars and Bars

O: (k) por el comb de Python; es una sola formula

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/combinatorics/1621_number_of_sets_of_k_non-overlapping_line_segments.py

```
1 class Solution:
2     def numberOfSets(self, n: int, k: int) -> int:
3         return math.comb(n+k-1, 2*k)%((10**9)+7)
```

29.4 Unique 3 Digit Event Numbers - Conteo con Frecuencias

O: (10^3 = 1000), tres ciclos sobre los digitos

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/combinatorics/3483_unique_3-digit_event_numbers.py

```
1 class Solution:
2     def totalNumbers(self, digits: List[int]) -> int:
3         res = set()
4         fre = [0]*10
5         for i in digits:
6             fre[i] += 1
7
8         for C in range(0,10,2):
9             if(fre[C]==0):
10                 continue
11             fre[C]-=1
12             for A in range(1,10):
13                 if(fre[A]==0):
```

29.4 Unique 3 Digit Event Numbers - Conteo con Frecuencias (cont.)

```
14             continue
15             fre[A]-=1
16             for B in range(0,10):
17                 if(fre[B]==0):
18                     continue
19                 res.add(int(str(A)+str(B)+str(C)))
20             fre[A]+=1
21             fre[C]+=1
22         return len(res)
```

30 - Hub LeetCode: DP

30.1 Climbing Stairs - Memoizacion

O: (n) tiempo y memoria; bottom-up con dos variables es (1) de memoria

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/dp/0070_climbing_stairs.py

```
1 class Solution:
2     def fibo(self, n, vis):
3         if(vis[n]!=0):
4             return vis[n]
5         if(n==0 or n==1):
6             return 1
7         a = self.fibo(n-1, vis)+self.fibo(n-2, vis)
8         vis[n] = a
9         return a
10
11     def climbStairs(self, n: int) -> int:
12         vis = [0]*(n+1)
13         return self.fibo(n, vis)
```

30.2 Distinct Subsequences I - DP en Dos Cadenas

O: (n*m) tiempo y memoria

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/dp/0115_distinct_subsequences_i.py

```
1 class Solution:
2     def numDistinct(self, s: str, t: str) -> int:
3         m, n = len(s), len(t)
4         if m < n:
5             return 0
6
7         dp = [[0] * (n + 1) for _ in range(m + 1)]
8         for i in range(m + 1):
9             dp[i][n] = 1
10
11         for i in range(m - 1, -1, -1):
12             for j in range(n - 1, -1, -1):
13                 if s[i] == t[j]:
14                     dp[i][j] = dp[i + 1][j + 1] + dp[i + 1][j]
15                 else:
16                     dp[i][j] = dp[i + 1][j]
17
18         return dp[0][0]
```

30.3 Distinct Subsequences II - DP con Resta de Repetidos

O: (n), duplicar y restar la aparicion previa

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/dp/0940_distinct_subsequences_ii.py

```
1 class Solution(object):
2     def distinctSubseqII(self, S):
3         dp = [1]
4         last = {}
5         for i, x in enumerate(S):
6             dp.append(dp[-1] * 2)
7             if x in last:
8                 dp[-1] -= dp[last[x]]
9             last[x] = i
10
```

30.3 Distinct Subsequences II - DP con Resta de Repetidos (cont.)

```

11         return (dp[-1] - 1) % (10**9 + 7)

```

30.4 Find X Value Of Array I - DP sobre Residuos

O: (n*k), un vector de residuos por posici n

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/dp/3524_find_x_value_of_array_i.py

```

1 class Solution:
2     def resultArray(self, nums: List[int], k: int) -> List[int]:
3         result = [0] * k
4         dp = {}
5         for i in range(0, len(nums)):
6             dp[i] = [0]*k
7             dp[i][nums[i]%k] += 1
8             for j in range(k):
9                 try:
10                     if(dp[i-1][j] != 0):
11                         res = (j*(nums[i]%k))%k
12                         dp[i][res] += dp[i-1][j]
13                 except:
14                     a = 1
15             for j in range(k):
16                 result[j] += dp[i][j]
17
18         return result

```

30.5 Maximum Score Of Non Overlapping Intervals - DP de Intervalos con Binaria

O: (n log n * 4), con el desempate lexicografico encima

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/dp/3414_maximum_score_of_non-overlapping_intervals.py

```

1 class Solution:
2     def maximumWeight(self, intervals: List[List[int]]) -> List[int]:
3         n = len(intervals)
4         arr = [
5             (intervals[i][1], intervals[i][0], intervals[i][2], i)
6             for i in range(n)
7         ]
8         # Sort by right endpoint.
9         arr.sort(key=lambda x: x[0])
10
11         dp = [[0] * 5 for _ in range(n + 1)]
12         indices = [[[ ] for _ in range(5)] for _ in range(n + 1)]
13
14         for i in range(n):
15             r, l, weight, idx = arr[i]
16             # Use binary search to find intervals whose right endpoints are smaller than l
17
18             k = bisect_left(arr, (l, ), hi=i)
19
20             for j in range(1, 5):
21                 s1 = dp[i][j]
22                 s2 = dp[k][j - 1] + weight
23                 if s1 > s2:
24                     dp[i + 1][j] = dp[i][j]
25                     indices[i + 1][j] = indices[i][j].copy()
26                     continue
27                 new_index = indices[k][j - 1].copy()
28                 new_index.append(idx)
29                 new_index.sort()
30                 if s1 == s2 and indices[i][j] < new_index:
31                     new_index = indices[i][j].copy()
32                 dp[i + 1][j] = s2
33                 indices[i + 1][j] = new_index
34
35         return indices[n][4]

```

31 - Hub LeetCode: Game Theory

31.1 Divisor Game - Paridad

O: (1), es la paridad de n

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/game_theory/1025_divisor_game.py

```

1 class Solution:
2     def divisorGame(self, n: int) -> bool:
3         if(n%2 == 0):
4             return True
5         else:
6             return False

```

31.2 Find The Winning Player In Coin Game - Simulaci n de Turnos

O: (min(x, y/4)) simulando; sale (1) con la formula

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/game_theory/3222_find_the_winning_player_in_coin_game.py

```

1 class Solution:
2     def winningPlayer(self, x: int, y: int) -> str:
3         j = True
4         while True:
5             if(x>=1 and y>=4):
6                 x-=1
7                 y-=4
8                 j = not j
9             else:
10                 break
11         if(j):
12             return "Bob"
13         else:
14             return "Alice"

```

31.3 Nim Game - Posiciones Perdedoras

O: (1), es n % 4 != 0

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/game_theory/0292_nim_game.py

```

1 class Solution:
2     def canWinNim(self, n: int) -> bool:
3         if(n %4 == 0):
4             return False
5         else:
6             return True

```

32 - Hub LeetCode: Geometry

32.1 Angle Between Hands Of A Clock - Angulos de un Reloj

O: (1)

NO ES MIO: c digo de Stiven Correa, del repo del equipo (carpeta leetcode/)

leetcode/geometry/1344_angle_between_hands_of_a_clock.py

```

1 class Solution:
2     def angleClock(self, hour: int, minutes: int) -> float:
3         minAng = (360/60)*minutes
4         hourAng = ((360/12)*((hour+minutes/60)%24))%360
5
6         return min(abs(minAng-hourAng),abs(hourAng-minAng), 360-abs(minAng-hourAng), 360-abs(hourAng-minAng))

```

32.2 Circle And Rectangle Overlapping - Punto mas Cercano en un Rectangulo

O: (1), el clamp del centro al rectangulo

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/geometry/1401_circle_and_rectangle_overlapping.py

```

1 class Solution:
2     def checkOverlap(self, r: int, cx: int, cy: int, x1: int, y1: int, x2: int, y2: int) -> bool:
3         x = max(x1, min(cx, x2)) - cx
4         y = max(y1, min(cy, y2)) - cy

```

32.2 Circle And Rectangle Overlapping - Punto mas Cercano en un Rectangulo (cont.)

```
5
6     return x * x + y * y <= r * r
```

32.3 Minimum Cuts To Divide A Circle - Casos por Paridad

O: (1), tres casos por paridad

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/geometry/2481_minimum_cuts_to_divide_a_circle.py

```
1 class Solution:
2     def numberOfCuts(self, n: int) -> int:
3         pedazos = n
4
5         if (pedazos % 2) == 0:
6
7             trazos = pedazos/2
8
9         elif pedazos == 1:
10
11             trazos = 0
12
13         else:
14
15             trazos = pedazos
16
17
18     return(int(trazos))
```

32.4 Rectangle Overlap - Interseccion de Rectangulos

O: (1)

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/geometry/0836_rectangle_overlap.py

```
1 class Solution:
2     def isRectangleOverlap(self, rec1: List[int], rec2: List[int]) -> bool:
3         ancho = min(rec1[2], rec2[2]) - max(rec1[0], rec2[0])
4         alto = min(rec1[3], rec2[3]) - max(rec1[1], rec2[1])
5
6         return ancho > 0 and alto > 0
```

33 - Hub LeetCode: Graphs

33.1 All Paths From Source To Target - Backtracking de Caminos

O: ($2^n * n$) en el peor caso, porque esa es la cantidad de caminos

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/0797_all_paths_from_source_to_target.py

```
1 from collections import deque
2 class Solution:
3     def allPathsSourceTarget(self, graph: List[List[int]]) -> List[List[int]]:
4         visited = set()
5         res = []
6         path = []
7         def dfs(curr, par):
8             path.append(curr)
9             if curr == len(graph)-1:
10                 res.append(path.copy())
11             for node in graph[curr]:
12                 if (curr, node) in visited:
13                     continue
14                 visited.add((curr, node))
15                 dfs(node, curr)
16             if par != -1:
17                 visited.remove((par, curr))
18             path.pop()
19         dfs(0, -1)
20         return res
```

33.2 Cheapest Flights Within K Stops - DP con Estado (nodo, saltos)

O: ($n*k*m$) por los estados (nodo, saltos) memoizados

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/0787_cheapest_flights_within_k_stops.py

```
1 from collections import defaultdict
2 from functools import cache
3
4 class Solution:
5     def findCheapestPrice(self, n, flights, src, dst, k):
6         graph = defaultdict(list)
7
8         for u, v, w in flights:
9             graph[u].append((v, w))
10
11         INF = float("inf")
12
13         @cache
14         def dp(curr, remaining):
15             if curr == dst:
16                 return 0
17
18             if remaining == 0:
19                 return INF
20
21             ans = INF
22
23             for nxt, cost in graph[curr]:
24                 ans = min(ans, cost + dp(nxt, remaining - 1))
25
26             return ans
27
28         ans = dp(src, k + 1)
29         return -1 if ans == INF else ans
```

33.3 Check If There Is A Valid Path In A Grid - Compatibilidad de Tubos en Matriz

O: ($n*m$), DFS con visitados sobre la cuadrícula

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/1391_check_if_there_is_a_valid_path_in_a_grid.py

```
1 class Solution:
2
3     def DFS(self, i, j, grid, memo):
4         if i < 0 or i >= len(grid) or j < 0 or j >= len(grid[0]):
5             return False
6         if (i, j) in memo:
7             return False
8         if i == len(grid) - 1 and j == len(grid[0]) - 1:
9             return True
10
11         memo.add((i, j))
12
13         calle = grid[i][j]
14
15         if calle == 1:
16             nj = j + 1
17             if nj < len(grid[0]) and grid[i][nj] in [1, 3, 5]:
18                 if self.DFS(i, nj, grid, memo):
19                     return True
20             nj = j - 1
21             if nj >= 0 and grid[i][nj] in [1, 4, 6]:
22                 if self.DFS(i, nj, grid, memo):
23                     return True
24         elif calle == 2:
25
26             ni = i + 1
27             if ni < len(grid) and grid[ni][j] in [2, 5, 6]:
28                 if self.DFS(ni, j, grid, memo):
29                     return True
30
31
32         ni = i - 1
```

33.3 Check If There Is A Valid Path In A Grid - Compatibilidad de Tubos en Matriz (cont.)

```

33     if ni >= 0 and grid[ni][j] in [2, 3, 4]:
34         if self.DFS(ni, j, grid, memo):
35             return True
36
37
38     elif calle == 3:
39
40
41         ni = i + 1
42         if ni < len(grid) and grid[ni][j] in [2, 5, 6]:
43             if self.DFS(ni, j, grid, memo):
44                 return True
45
46
47         nj = j - 1
48         if nj >= 0 and grid[i][nj] in [1, 4, 6]:
49             if self.DFS(i, nj, grid, memo):
50                 return True
51
52
53     elif calle == 4:
54
55
56         nj = j + 1
57         if nj < len(grid[0]) and grid[i][nj] in [1, 3, 5]:
58             if self.DFS(i, nj, grid, memo):
59                 return True
60
61
62         ni = i + 1
63         if ni < len(grid) and grid[ni][j] in [2, 5, 6]:
64             if self.DFS(ni, j, grid, memo):
65                 return True
66
67
68     elif calle == 5:
69
70
71         ni = i - 1
72         if ni >= 0 and grid[ni][j] in [2, 3, 4]:
73             if self.DFS(ni, j, grid, memo):
74                 return True
75
76
77         nj = j - 1
78         if nj >= 0 and grid[i][nj] in [1, 4, 6]:
79             if self.DFS(i, nj, grid, memo):
80                 return True
81
82
83     elif calle == 6:
84
85
86         ni = i - 1
87         if ni >= 0 and grid[ni][j] in [2, 3, 4]:
88             if self.DFS(ni, j, grid, memo):
89                 return True
90
91
92         nj = j + 1
93         if nj < len(grid[0]) and grid[i][nj] in [1, 3, 5]:
94             if self.DFS(i, nj, grid, memo):
95                 return True
96
97     return False
98
99     def hasValidPath(self, grid):
100         memo = set()
101         return self.DFS(0, 0, grid, memo)

```

33.4 Clone Graph - Clonar Grafo con Diccionario

O: (n+m), un DFS con el mapa original -> copia

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/0133_clone_graph.py

```

1 class Solution:
2     def cloneGraph(self, node: Optional['Node']) -> Optional['Node']:
3
4         if not node:
5             return None
6
7         visited = {}
8         # original -> copia
9
10        def creator(curr):
11
12            if curr in visited:
13                return visited[curr]
14
15            copy = Node(curr.val)
16            visited[curr] = copy
17
18            for nxt in curr.neighbors:
19                copy.neighbors.append(creator(nxt))
20
21            return copy
22
23        return creator(node)

```

33.5 Course Schedule - Orden Topologico de Kahn

O: (n+m), Kahn con los grados de entrada

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/0207_course_schedule.py

```

1 from collections import deque
2 class Solution:
3     def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
4         def kahn_topological_sort(num_nodes, edges):
5             # 1. Initialize adjacency list and in-degree array
6             adj_list = {i: [] for i in range(num_nodes)}
7             in_degree = [0] * num_nodes
8
9             # 2. Build the graph: edge is u -> v
10            for u, v in edges:
11                adj_list[u].append(v)
12                in_degree[v] += 1
13
14            # 3. Add all nodes with 0 incoming edges to the queue
15            queue = deque([i for i in range(num_nodes) if in_degree[i] == 0])
16            topo_order = []
17
18            # 4. Process the queue
19            while queue:
20                u = queue.popleft()
21                topo_order.append(u)
22
23                for neighbor in adj_list[u]:
24                    in_degree[neighbor] -= 1
25                    if in_degree[neighbor] == 0:
26                        queue.append(neighbor)
27
28            # 5. Check for cycles
29            if len(topo_order) != num_nodes:
30                return False
31
32            return True
33
34        return kahn_topological_sort(numCourses, prerequisites)

```

33.6 Course Schedule II - Orden Topologico de Kahn

O: (n+m), el mismo Kahn devolviendo el orden

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/0210_course_schedule_ii.py

```
1 from collections import deque
2 class Solution:
3     def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]:
4
5         def kahn_topological_sort(num_nodes, edges):
6             # 1. Initialize adjacency list and in-degree array
7             adj_list = {i: [] for i in range(num_nodes)}
8             in_degree = [0] * num_nodes
9
10            # 2. Build the graph: edge is u -> v
11            for u, v in edges:
12                adj_list[u].append(v)
13                in_degree[v] += 1
14
15            # 3. Add all nodes with 0 incoming edges to the queue
16            queue = deque([i for i in range(num_nodes) if in_degree[i] == 0])
17            topo_order = []
18
19            # 4. Process the queue
20            while queue:
21                u = queue.popleft()
22                topo_order.append(u)
23
24                for neighbor in adj_list[u]:
25                    in_degree[neighbor] -= 1
26                    if in_degree[neighbor] == 0:
27                        queue.append(neighbor)
28
29            # 5. Check for cycles
30            if len(topo_order) != num_nodes:
31                return []
32
33            return topo_order
34
35            for i in range(len(prerequisites)):
36                prerequisites[i] = [prerequisites[i][1], prerequisites[i][0]]
37            return (kahn_topological_sort(numCourses, prerequisites))
```

33.7 Evaluate Division - Grafo con Pesos Multiplicativos

O: (consultas * (n+m)) por el DFS de cada una; con DSU con pesos es casi (1)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/0399_evaluate_division.py

```
1 from collections import defaultdict
2
3 class Solution:
4     def calcEquation(self, equations, values, queries):
5
6         graph = defaultdict(list)
7
8         for (a, b), v in zip(equations, values):
9             graph[a].append((b, v))
10            graph[b].append((a, 1 / v))
11
12            def dfs(curr, target, visited):
13
14                if curr == target:
15                    return 1
16
17                visited.add(curr)
18
19                for nxt, weight in graph[curr]:
20                    if nxt in visited:
21                        continue
22
23                    ans = dfs(nxt, target, visited)
24
```

33.7 Evaluate Division - Grafo con Pesos Multiplicativos (cont.)

```
25         if ans != -1:
26             return weight * ans
27
28         return -1
29
30         res = []
31
32         for a, b in queries:
33
34             if a not in graph or b not in graph:
35                 res.append(-1.0)
36                 continue
37
38             res.append(dfs(a, b, set()))
39
40         return res
```

33.8 Find A Safe Walk Through A Grid - Búsqueda con Vida Restante

O: (n*m) con la poda de mejor vida; sin ella sería exponencial

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/3286_find_a_safe_walk_through_a_grid.py

```
1 class Solution:
2
3     def BFS(self, grid, m, n, health, ant):
4         #print(m,n,health)
5         ant[str(m)+str(n)] = health
6         if(health <= 0):
7             return 0
8         elif(m==len(grid)-1 and n==len(grid[0])-1):
9             return 1
10        else:
11            k = [[1,0],[0,1],[-1,0],[0,-1]]
12            resp = 2
13            for i in k:
14                #print(m,n,health, i, "-----")
15                if(m+i[0]>-1 and m+i[0]<len(grid) and n+i[1]>-1 and n+i[1]<len(grid[0]) and
16                    d str(m+i[0])+str(n+i[1]) not in ant):
17                    if(grid[m+i[0]][n+i[1]] == 1):
18                        resp = self.BFS(grid,m+i[0],n+i[1],health-1, ant)
19                    else:
20                        resp = self.BFS(grid,m+i[0],n+i[1],health,ant)
21                    elif(m+i[0]>-1 and m+i[0]<len(grid) and n+i[1]>-1 and n+i[1]<len(grid[0])
22                        and ant[str(m+i[0])+str(n+i[1])] < health):
23                        if(grid[m+i[0]][n+i[1]] == 1):
24                            resp = self.BFS(grid,m+i[0],n+i[1],health-1, ant)
25                        else:
26                            resp = self.BFS(grid,m+i[0],n+i[1],health,ant)
27                    if(resp == 1):
28                        break
29                    if(resp == 1):
30                        return 1
31                else:
32                    return 2
33
34            def findSafeWalk(self, grid: List[List[int]], health: int) -> bool:
35                unos = 0
36                for i in grid:
37                    unos += i.count(1)
38                if(unos==(len(grid[0])*len(grid)) and health<=(len(grid[0])+len(grid))-1):
39                    return False
40                if(grid[0][0]==1):
41                    health-=1
42                ant = {"00":health}
43                res = self.BFS(grid, 0, 0, health, ant)
44                if(res == 1):
45                    return True
46                else:
47                    return False
```

33.9 Find Eventual Safe States - Nodos que No Llegan a un Ciclo

O: $(n \cdot (n+m))$ por el `visited.clear()` de cada nodo; con tres colores es $(n+m)$

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/0802_find_eventual_safe_states.py

```
1 class Solution:
2     def eventualSafeNodes(self, graph: List[List[int]]) -> List[int]:
3         safe = set()
4         visited = set()
5
6         def dfs(curr):
7             if curr in safe:
8                 return True
9
10            if curr in visited:
11                return False
12
13            visited.add(curr)
14
15            for node in graph[curr]:
16                if not dfs(node):
17                    visited.discard(curr)
18                    return False
19
20            visited.discard(curr)
21            safe.add(curr)
22            return True
23
24        res = []
25
26        for i in range(len(graph)):
27            visited.clear()
28            if dfs(i):
29                res.append(i)
30
31        return res
```

33.10 Holding Hands - DSU sobre Parejas

O: (n^2) en el peor caso: el find no comprime caminos

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/0765_holding_hands.py

```
1 from collections import defaultdict
2 class Solution:
3     def minSwapsCouples(self, row: List[int]) -> int:
4         n = len(row)
5         parent = [i for i in range(n//2)]
6         def find(i):
7             if parent[i] == i:
8                 return i
9             return find(parent[i])
10        def unite(i, j):
11            irep = find(i)
12            jrep = find(j)
13            parent[irep] = jrep
14        def get_groups():
15            groups = defaultdict(list)
16            for element in parent:
17                root = find(element)
18                groups[root].append(element)
19            return list(groups.values())
20
21        coach = [-1] * (n)
22
23        for i, val in enumerate(row):
24            coach[val] = i//2
25
26        graph = defaultdict(list)
27        for i in range(0, n-1, 2):
28            curr = coach[i]
29            nxt = coach[i+1]
30            if curr != nxt:
```

33.10 Holding Hands - DSU sobre Parejas (cont.)

```
31         unite(curr, nxt)
32     groups = get_groups()
33     res = 0
34     for i in groups:
35         res += len(i)-1
36     return res
```

33.11 Is Graph Bipartite - Coloreo a Dos Colores

O: $(n+m)$, DFS pintando dos colores

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/0785_is_graph_bipartite.py

```
1 from collections import defaultdict
2 class Solution:
3     def isBipartite(self, graph: List[List[int]]) -> bool:
4         colors = defaultdict(int)
5         res = True
6         visited = set()
7         def dfs(curr):
8             nonlocal res
9             visited.add(curr)
10
11            if curr not in colors:
12                colors[curr] = 1
13
14            for node in graph[curr]:
15                if node not in colors:
16                    colors[node] = -colors[curr]
17                    dfs(node)
18                elif colors[node] == colors[curr]:
19                    res = False
20        for i in range(len(graph)):
21            if i not in visited:
22                dfs(i)
23        return res
```

33.12 Jump Game Iii - DFS con Visitados

O: (n) , cada posicion se visita una vez

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/1306_jump_game_iii.py

```
1 class Solution:
2
3     def numeri(self, nums, start, l, tam):
4
5         if(start in l):
6             return False
7
8         l.add(start)
9
10        pos1 = start + nums[start]
11        pos2 = start - nums[start]
12
13        val1, val2 = False, False
14
15        if(pos1 < tam and nums[pos1] == 0):
16            return True
17        else:
18            if(pos1 < tam):
19                val1 = self.numeri(nums, pos1, l, tam)
20
21        if(pos2 > -1 and nums[pos2] == 0):
22            return True
23        else:
24            if(pos2 >= 0):
25                val2 = self.numeri(nums, pos2, l, tam)
26
27        return val1 or val2
28
29
30    def canReach(self, arr: List[int], start: int) -> bool:
```

33.12 Jump Game III - DFS con Visitados (cont.)

```

31
32     tam = len(arr)
33
34     if(0 not in arr):
35         return False
36
37     elif(arr[start] == 0):
38         return True
39
40     else:
41         l = set()
42         a = self.numeri(arr, start, l, tam)
43         return a

```

33.13 Longest Increasing Path In A Matrix - DFS con Memoizacón en Matriz

O: (n*m), cada celda se memoiza una vez

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/0329_longest_increasing_path_in_a_matrix.py

```

1 from collections import defaultdict
2
3 class Solution:
4     def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
5         paths = set()
6         for i in range(len(matrix)):
7             for j in range(len(matrix[i])):
8                 paths.add((i,j))
9         dicc = defaultdict(int)
10        def explore(i,j):
11            best = 1
12            if i+1 < len(matrix) and matrix[i+1][j] > matrix[i][j]:
13                if (i+1, j) in paths:
14                    best = max(best, 1 + explore(i+1, j))
15                paths.discard((i+1, j))
16            else:
17                best = max(best, 1 + dicc[(i+1, j)])
18            if i-1 >= 0 and matrix[i-1][j] > matrix[i][j]:
19                if (i-1, j) in paths:
20                    paths.discard((i-1, j))
21                best = max(best, 1 + explore(i-1, j))
22            else:
23                best = max(best, 1 + dicc[(i-1, j)])
24            if j+1 < len(matrix[i]) and matrix[i][j+1] > matrix[i][j]:
25                if (i, j+1) in paths:
26                    paths.discard((i, j+1))
27                best = max(best, 1 + explore(i, j+1))
28            else:
29                best = max(best, 1 + dicc[(i, j+1)])
30            if j-1 >= 0 and matrix[i][j-1] > matrix[i][j]:
31                if (i, j-1) in paths:
32                    paths.discard((i, j-1))
33                best = max(best, 1 + explore(i, j-1))
34            else:
35                best = max(best, 1 + dicc[(i, j-1)])
36            dicc[(i, j)] = best
37            return best
38
39        while(True):
40            if(len(paths) > 0):
41                nextP = paths.pop()
42                explore(nextP[0], nextP[1])
43            else:
44                break
45        return max(dicc.values())

```

33.14 Minimum Height Trees - Pelar Hojas (Centro del Arbol)

O: (n), pelando hojas por capas

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/0310_minimum_height_trees.py

```

1 from collections import defaultdict, deque

```

33.14 Minimum Height Trees - Pelar Hojas (Centro del Arbol) (cont.)

```

2
3 class Solution:
4     def findMinHeightTrees(self, n: int, edges: List[List[int]]) -> List[int]:
5
6         if n == 1:
7             return [0]
8
9         grafo = defaultdict(list)
10        grado = [0] * n
11
12        for u, v in edges:
13            grafo[u].append(v)
14            grafo[v].append(u)
15            grado[u] += 1
16            grado[v] += 1
17
18        cola = deque()
19
20        for i in range(n):
21            if grado[i] == 1:
22                cola.append(i)
23
24        restantes = n
25
26        while restantes > 2:
27            hojas = len(cola)
28            restantes -= hojas
29
30            for _ in range(hojas):
31                hoja = cola.popleft()
32
33                for vecino in grafo[hoja]:
34                    grado[vecino] -= 1
35                    if grado[vecino] == 1:
36                        cola.append(vecino)
37
38        return list(cola)

```

33.15 Minimum Moves To Clean The Classroom - Dijkstra sobre (nodo, bitmask) con Poda de P...

O: (2^L * V^2 log) con L basuras y V celdas clave, por el Dijkstra sobre estados

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/3568_minimum_moves_to_clean_the_classroom.py

```

1 from collections import deque
2 from heapq import heappush, heappop
3 from typing import List
4
5 class Solution:
6     def minMoves(self, classroom: List[str], energy: int) -> int:
7         filas = len(classroom)
8         columnas = len(classroom[0])
9
10        # -----
11        # 1. Encontrar S, L y R
12        # -----
13
14        especiales = []
15        inicio = -1
16        posiciones_L = {}
17
18        for i in range(filas):
19            for j in range(columnas):
20
21                tipo = classroom[i][j]
22
23                if tipo == 'S':
24                    inicio = len(especiales)
25                    especiales.append((i, j, 'S'))
26
27            elif tipo == 'L':
28                indice = len(posiciones_L)

```

33.15 Minimum Moves To Clean The Classroom - Dijkstra sobre (nodo, bitmask) con Poda de Pareto (cont.)

```

29     posiciones_L[(i, j)] = indice
30     especiales.append((i, j, 'L'))
31
32     elif tipo == 'R':
33         especiales.append((i, j, 'R'))
34
35     cantidad_L = len(posiciones_L)
36
37     if cantidad_L == 0:
38         return 0
39
40     # -----
41     # 2. Posicion -> nodo
42     # -----
43
44     indice_especial = {}
45
46     for i, (x, y, tipo) in enumerate(especiales):
47         indice_especial[(x, y)] = i
48
49     cantidad_nodos = len(especiales)
50
51     # -----
52     # 3. Construir grafo
53     # -----
54
55     grafo = [[] for _ in range(cantidad_nodos)]
56
57     movimientos = [
58         (1, 0),
59         (-1, 0),
60         (0, 1),
61         (0, -1)
62     ]
63
64     for nodo, (sx, sy, _) in enumerate(especiales):
65
66         cola = deque()
67         cola.append((sx, sy, 0))
68
69         visitados = {(sx, sy)}
70
71         while cola:
72
73             x, y, distancia = cola.popleft()
74
75             if distancia > 0 and (x, y) in indice_especial:
76
77                 destino = indice_especial[(x, y)]
78
79                 grafo[nodo].append(
80                     (destino, distancia)
81                 )
82
83                 # IMPORTANTE:
84                 #
85                 # No seguimos atravesando el nodo especial.
86                 #
87                 # Si es R, Dijkstra debe encargarse de la
88                 # recarga antes de continuar.
89                 continue
90
91             for dx, dy in movimientos:
92
93                 nx = x + dx
94                 ny = y + dy
95
96                 if 0 <= nx < filas and 0 <= ny < columnas:
97
98                     if (nx, ny) not in visitados:
99
100                         if classroom[nx][ny] != 'X':

```

33.15 Minimum Moves To Clean The Classroom - Dijkstra sobre (nodo, bitmask) con Poda de Pareto (cont.)

```

01
02         visitados.add((nx, ny))
03
04         cola.append(
05             (nx, ny, distancia + 1)
06         )
07
08     # -----
09     # 4. Bitmask
10     # -----
11
12     objetivo = (1 << cantidad_L) - 1
13
14     # -----
15     # 5. Dijkstra + estados dominados
16     # -----
17
18     #
19     # Estado:
20     #
21     # (pasos, nodo, energia, mask)
22     #
23
24     heap = [
25         (0, inicio, energy, 0)
26     ]
27
28     #
29     # Para cada (nodo, mask) guardamos estados:
30     #
31     # (pasos, energia)
32     #
33     # Solo mantenemos estados NO dominados.
34     #
35     # Un estado A domina a B si:
36     #
37     # A tiene <= pasos
38     # A tiene >= energia
39     #
40
41     frontera = {}
42
43     frontera[(inicio, 0)] = [
44         (0, energy)
45     ]
46
47     while heap:
48
49         pasos, nodo, energia_actual, mask = heappop(heap)
50
51         clave = (nodo, mask)
52
53         # -----
54         # Comprobar si este estado fue dominado
55         # -----
56
57         dominado = False
58
59         for pasos_viejos, energia_vieja in frontera.get(
60             clave, []
61         ):
62
63             if pasos_viejos <= pasos and energia_vieja >= energia_actual:
64
65                 if pasos_viejos < pasos or energia_vieja > energia_actual:
66                     dominado = True
67                     break
68
69         if dominado:
70             continue
71
72     # -----

```

33.15 Minimum Moves To Clean The Classroom - Dijkstra sobre (nodo, bitmask) con Poda de Pareto (cont.)

```

173 # Todas las L
174 # -----
175
176 if mask == objetivo:
177     return pasos
178
179 # -----
180 # Explorar vecinos
181 # -----
182
183 for siguiente, costo in grafo[nodo]:
184
185     if costo > energia_actual:
186         continue
187
188     nueva_energia = energia_actual - costo
189     nuevo_mask = mask
190
191     x, y, tipo = especiales[siguiente]
192
193     # -----
194     # L
195     # -----
196
197     if tipo == 'L':
198
199         indice_L = posiciones_L[(x, y)]
200
201         nuevo_mask |= 1 << indice_L
202
203     # -----
204     # R
205     # -----
206
207     elif tipo == 'R':
208
209         nueva_energia = energy
210
211         nuevos_pasos = pasos + costo
212
213         nueva_clave = (siguiente, nuevo_mask)
214
215         # -----
216         # El nuevo estado esta dominado?
217         # -----
218
219         estados = frontera.get(nueva_clave, [])
220
221         dominado = False
222
223         for pasos_viejos, energia_vieja in estados:
224
225             if (
226                 pasos_viejos <= nuevos_pasos
227                 and energia_vieja >= nueva_energia
228             ):
229                 dominado = True
230                 break
231
232         if dominado:
233             continue
234
235         # -----
236         # El nuevo estado elimina estados que el domina
237         # -----
238
239         nuevos_estados = []
240
241         for pasos_viejos, energia_vieja in estados:
242
243             if not (
244                 nuevos_pasos <= pasos_viejos

```

33.15 Minimum Moves To Clean The Classroom - Dijkstra sobre (nodo, bitmask) con Poda de Pareto (cont.)

```

245         and nueva_energia >= energia_vieja
246     ):
247         nuevos_estados.append(
248             (pasos_viejos, energia_vieja)
249         )
250
251     nuevos_estados.append(
252         (nuevos_pasos, nueva_energia)
253     )
254
255     frontera[nueva_clave] = nuevos_estados
256
257     heappush(
258         heap,
259         (
260             nuevos_pasos,
261             siguiente,
262             nueva_energia,
263             nuevo_mask
264         )
265     )
266
267     return -1

```

33.16 Network Delay Time - Dijkstra

O: (m log n), Dijkstra con heap

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/0743_network_delay_time.py

```

1 import heapq
2 from collections import defaultdict
3
4 class Solution:
5     def networkDelayTime(self, times: List[List[int]], n: int, k: int) -> int:
6         def dijkstra(grafo, inicio):
7             distancias = {nodo: float('inf') for nodo in range(1, n+1)}
8             distancias[inicio] = 0
9             cola = [(0, inicio)]
10
11             while cola:
12                 dist_actual, nodo_actual = heapq.heappop(cola)
13
14                 if dist_actual > distancias[nodo_actual]:
15                     continue
16
17                 for vecino, peso in grafo[nodo_actual].items():
18                     distancia = dist_actual + peso
19                     if distancia < distancias[vecino]:
20                         distancias[vecino] = distancia
21                         heapq.heappush(cola, (distancia, vecino))
22
23             return distancias
24
25         graph = defaultdict(dict)
26
27         for u,v,w in times:
28             graph[u][v] = w
29         distances = dijkstra(graph, k).values()
30         res = 0
31         for i in list(distances):
32             if i > res:
33                 res = i
34         if res == float('inf'):
35             return -1
36         else:
37             return res

```

33.17 Network Recovery Pathways - Binaria sobre la Respuesta en un DAG

O: (m log(peso maximo)): binaria sobre la respuesta y un barrido topologico por intento
 NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/3620_network_recovery_pathways.py

```
1 from collections import deque
2
3 class Solution:
4     def findMaxPathScore(self, edges: List[List[int]], online: List[bool], k: int) -> int:
5         n = len(online)
6
7         graph = [[] for _ in range(n)]
8         indeg = [0] * n
9         pesos = []
10
11         for u, v, w in edges:
12             graph[u].append((v, w))
13             indeg[v] += 1
14             pesos.append(w)
15
16         # Orden topologico
17         topo = []
18         q = deque()
19
20         for i in range(n):
21             if indeg[i] == 0:
22                 q.append(i)
23
24         while q:
25             u = q.popleft()
26             topo.append(u)
27             for v, _ in graph[u]:
28                 indeg[v] -= 1
29                 if indeg[v] == 0:
30                     q.append(v)
31
32         def check(mid):
33             INF = 10**30
34             dp = [INF] * n
35             dp[0] = 0
36
37             for u in topo:
38                 if dp[u] == INF:
39                     continue
40
41                 if u != 0 and u != n - 1 and not online[u]:
42                     continue
43
44                 for v, w in graph[u]:
45                     if w < mid:
46                         continue
47
48                     if v != n - 1 and not online[v]:
49                         continue
50
51                     if dp[u] + w < dp[v]:
52                         dp[v] = dp[u] + w
53
54             return dp[n - 1] <= k
55
56         if not pesos:
57             return -1
58
59         l = 0
60         r = max(pesos)
61         ans = -1
62
63         while l <= r:
64             mid = (l + r) // 2
65
66             if check(mid):
67                 ans = mid
```

33.17 Network Recovery Pathways - Binaria sobre la Respuesta en un DAG (cont.)

```
68         l = mid + 1
69         else:
70             r = mid - 1
71
72         return ans
```

33.18 Number Of Provinces - Componentes Conexas

O: (n^2), que es leer la matriz de adyacencia
 NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/0547_number_of_provinces.py

```
1 from collections import defaultdict
2 class Solution:
3     def findCircleNum(self, isConnected: List[List[int]]) -> int:
4         graph = defaultdict(list)
5         for i in range(len(isConnected)):
6             for j in range(len(isConnected)):
7                 if isConnected[i][j] == 1:
8                     graph[i].append(j)
9                     graph[j].append(i)
10
11         visited = set()
12         def dfs(curr):
13             visited.add(curr)
14             for node in graph[curr]:
15                 if node not in visited:
16                     dfs(node)
17
18         res = 0
19         for node in graph:
20             if node not in visited:
21                 dfs(node)
22                 res += 1
23
24         return res
```

33.19 Reconstruct Itinerary - Camino Euleriano (Hierholzer)

O: (m log m) por ordenar los destinos; Hierholzer es (m)
 NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/0332_reconstruct_itinerary.py

```
1 from collections import defaultdict
2
3 class Solution:
4     def findItinerary(self, tickets: List[List[str]]) -> List[str]:
5         def encontrar_camino_euleriano(grafo, inicio="JFK"):
6             grafo_copia = {k: sorted(list(v), reverse=True) for k, v in grafo.items()}
7
8             pila = [inicio]
9             camino = []
10
11             while pila:
12                 nodo_actual = pila[-1]
13                 if nodo_actual in grafo_copia and grafo_copia[nodo_actual]:
14                     siguiente_nodo = grafo_copia[nodo_actual].pop()
15                     pila.append(siguiente_nodo)
16                 else:
17                     # Si no hay mas aristas salientes, fijamos el nodo en el camino
18                     camino.append(pila.pop())
19
20             camino.reverse()
21             return camino
22
23         itinerarios = {
24             "JFK": ["SFO", "ATL"],
25             "SFO": ["ATL"],
26             "ATL": ["JFK", "SFO"]
27         }
28
29         graph = defaultdict(list)
30         for a, b in tickets:
31             graph[a].append(b)
32         for node in graph:
33             graph[node].sort()
```

33.19 Reconstruct Itinerary - Camino Euleriano (Hierholzer) (cont.)

```

34
35         sol = encontrar_camino_euleriano(graph)
36
37         return sol

```

33.20 Redudant Connection - DSU (Union-Find)

O: (n^2) en el peor caso: el find no comprime caminos

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/0684_redundant_connection.py

```

1 class Solution:
2     def findRedundantConnection(self, edges: List[List[int]]) -> List[int]:
3         parents = {}
4         for a,b in edges:
5             parents[a] = a
6             parents[b] = b
7         def find(node):
8             if parents[node] == node:
9                 return node
10            return find(parents[node])
11        def union(a,b):
12            fatherA = find(a)
13            fatherB = find(b)
14            if fatherA == fatherB:
15                return False
16            parents[fatherA] = fatherB
17            return True
18
19        for a,b in edges:
20            if not union(a,b):
21                return [a,b]

```

33.21 Redundant Connection II - Ciclo en Grafo Dirigido por Casos

O: (n^2) por el stack.index() del ciclo; el DFS solo es $(n+m)$

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/0685_redundant_connection_ii.py

```

1 from collections import defaultdict
2
3 class Solution:
4     def findRedundantDirectedConnection(self, edges: List[List[int]]) -> List[int]:
5         graph = defaultdict(list)
6         indegree = defaultdict(int)
7         nodes = set()
8
9         for a, b in edges:
10            graph[a].append(b)
11            indegree[b] += 1
12            nodes.add(a)
13            nodes.add(b)
14
15        visiting = set()
16        visited = set()
17        stack = []
18        edgeStack = []
19        ciclo = set()
20
21        def dfs(node):
22            visiting.add(node)
23            stack.append(node)
24
25            for nxt in graph[node]:
26                if nxt in visiting:
27                    idx = stack.index(nxt)
28                    for i in range(idx, len(edgeStack)):
29                        ciclo.add(edgeStack[i])
30                    ciclo.add((node, nxt))
31                    return True
32
33            if nxt not in visited:
34                edgeStack.append((node, nxt))

```

33.21 Redundant Connection II - Ciclo en Grafo Dirigido por Casos (cont.)

```

35         if dfs(nxt):
36             return True
37         edgeStack.pop()
38
39        stack.pop()
40        visiting.remove(node)
41        visited.add(node)
42        return False
43
44        for node in nodes:
45            if node not in visited:
46                if dfs(node):
47                    break
48
49        # Buscar si existe un nodo con dos padres
50        doble = -1
51        for node in nodes:
52            if indegree[node] == 2:
53                doble = node
54                break
55
56        # Caso 1: solo ciclo
57        if doble == -1:
58            for a, b in reversed(edges):
59                if (a, b) in ciclo:
60                    return [a, b]
61
62        # Caso 2 o 3: nodo con dos padres
63        candidatos = []
64        for a, b in edges:
65            if b == doble:
66                candidatos.append([a, b])
67
68        # Si una de las dos aristas pertenece al ciclo
69        for a, b in candidatos:
70            if (a, b) in ciclo:
71                return [a, b]
72
73        # Si ninguna pertenece al ciclo
74        return candidatos[-1]

```

33.22 Remove Methods From Project - Alcanzabilidad y Contaminacion

O: $(n+m)$, un DFS y un barrido de aristas

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/3310_remove_methods_from_project.py

```

1 class Solution:
2     def dfs(self, nodo, dic, vis):
3         if nodo not in vis and nodo != -1:
4             vis.add(nodo)
5             for i in dic[nodo]:
6                 self.dfs(i, dic, vis)
7
8     def remainingMethods(self, n: int, k: int, invocations: List[List[int]]) -> List[int]:
9         dic = {}
10        sol1 = []
11        for i in range(n):
12            dic[i] = [-1]
13            sol1.append(i)
14        for i in invocations:
15            dic[i[0]].append(i[1])
16        vis = set()
17        self.dfs(k, dic, vis)
18
19        sol = set()
20        for i in range(n):
21            for j in dic[i]:
22                if(i not in vis and j not in vis):
23                    sol.add(i)
24            elif(i not in vis and j in vis):
25                return sol1

```

33.22 Remove Methods From Project - Alcanzabilidad y Contaminacion (cont.)

```
25 return list(sol)
```

33.23 Sum Of Distances In Tree - Rerooting en Arbol**O:** (n), tres DFS: tamanos, respuesta de la raiz y rerooting

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/graphs/0834_sum_of_distances_in_tree.py

```
1 from collections import defaultdict
2 class Solution:
3     def sumOfDistancesInTree(self, n: int, edges: List[List[int]]) -> List[int]:
4         if len(edges) == 0:
5             return [0]
6         tree = defaultdict(list)
7         for a, b in edges:
8             tree[a].append(b)
9             tree[b].append(a)
10
11         sizes = defaultdict(int)
12         def dfs(curr, par):
13             total = 1
14             for node in tree[curr]:
15                 if node != par:
16                     total += dfs(node, curr)
17             sizes[curr] = total
18             return total
19         dfs(0, None)
20
21         res = [0]*n
22         def dfs2(curr, depth, par):
23             res[0] += depth
24             for node in tree[curr]:
25                 if node != par:
26                     dfs2(node, depth+1, curr)
27         dfs2(0, 0, None)
28
29         def dfs3(curr, prev, par):
30             currSum = 0
31             if res[curr] != 0:
32                 currSum = res[curr]
33             else:
34                 currSum = prev - sizes[curr] + (n-sizes[curr])
35             res[curr] = currSum
36             for node in tree[curr]:
37                 if node != par:
38                     dfs3(node, currSum, curr)
39         dfs3(0, 0, None)
40         return res
```

34 - Hub LeetCode: Greedy**34.1 Maximum Number Of Coins You Can Get - Greedy sobre Ordenado****O:** (n log n) por el sort; los pop(0) suman (n^2) encima

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/greedy/1561_maximum_number_of_coins_you_can_get.py

```
1 class Solution:
2     def maxCoins(self, piles: List[int]) -> int:
3         k = sorted(piles)
4         c = 0
5         while len(k)>0:
6             k.pop()
7             c += k[len(k)-1]
8             k.pop(0)
9             k.pop()
10        return c
```

34.2 Maximum Number Of Non Overlapping Palindrome Substrings - Greedy de Izquierda a Der...**O:** (n*k), probando solo los largos k y k+1

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/greedy/2472_maximum_number_of_non-overlapping_palindrome_substrings.py

```
1 class Solution:
2     def maxPalindromes(self, s: str, k: int) -> int:
3         n = len(s)
4         if k == 1: return n
5
6         res = i = 0
7
8         while i <= n - k:
9             for d in (k, k + 1):
10                 if i + d <= n and s[i : i + d] == s[i : i + d][::-1]:
11                     res += 1
12                     i += d
13                     break
14             else:
15                 i += 1
16
17         return res
```

34.3 Stone Game - Greedy que Acierta por Casualidad**O:** (n log n) por el sort; la respuesta real es True siempre, (1)

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/greedy/0877_stone_game.py

```
1 class Solution:
2     def stoneGame(self, piles: List[int]) -> bool:
3         piles = sorted(piles)
4         A = 0
5         B = 0
6         va = True
7         i = len(piles)-1
8         while i > -1:
9             if(va):
10                 A += piles[i]
11                 va = not va
12             else:
13                 B += piles[i]
14                 va = not va
15                 i-=1
16         if(A > B):
17             return True
18         else:
19             return False
20
21
22         #return True always
```

35 - Hub LeetCode: Heap**35.1 Construct Target Array With Multiple Sums - Deshacer el Proceso al Reves****O:** (n log n log(maximo)) por el modulo que salta varias restas de golpe

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/heap/1354_construct_target_array_with_multiple_sums.py

```
1 import heapq
2 class Solution:
3     def isPossible(self, target: List[int]) -> bool:
4         tam = []
5         for i in target:
6             heapq.heappush(tam, i*-1)
7
8         if(len(target) == 1 and 1 in target):
9             return True
10        elif(len(target) == 1 and 1 not in target):
11            return False
12
13
```

35.1 Construct Target Array With Multiple Sums - Deshacer el Proceso al Revés (cont.)

```

18 cantidad = 0
19 suma = sum(tam)
20 ant = tam.copy()
21 while cantidad != len(target):
22     mayor = heapq.heappop(tam)
23     suma -= mayor
24     if(mayor == suma):
25         break
26     me = (-mayor) % (-suma)
27     if(me == 0):
28         me = -suma
29     elif(me < 0):
30         break
31     heapq.heappush(tam, me*-1)
32     cantidad = tam.count(-1)
33     print(mayor, suma, me, tam, ant)
34     suma -= me
35     if(tam == ant):
36         break
37     ant = tam.copy()
38
39 if(cantidad != len(target)):
40     return False
41 else:
42     return True

```

35.2 Find K Pairs With Smallest Sums - Heap Maximo de Tamano k

O: (n*m log k) en el peor caso, recorriendo todas las parejas

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/heap/0373_find_k_pairs_with_smallest_sums.py

```

1 import heapq
2 class Solution:
3     def kSmallestPairs(self, nums1: List[int], nums2: List[int], k: int) -> List[List[int]]:
4
5         if(nums1 == nums2 and 3 not in nums1):
6             return [[1,1]]*k
7
8         tam = []
9         h = []
10        dic = {}
11        may = 0
12        for i in nums1:
13            for j in nums2:
14                l = i+j
15                if(len(tam)<k or l <= may):
16                    if(l>=may):
17                        may = l
18                    elif(len(tam)>k and may >= l):
19                        may = -heapq.heappop(tam)
20                    #tam.append(l)
21                    heapq.heappush(tam, l*-1)
22                    if(l not in dic):
23                        dic[l] = [[i,j]]
24                    else:
25                        dic[l].append([i,j])
26
27            else:
28                break
29
30        for i in range(len(tam)):
31            tam[i] *= -1
32            #print(tam)
33            tam = sorted(tam)
34            fi = []
35            i = 0
36            while i < len(tam):
37                q = dic[tam[i]]
38                if(len(q) == 1):
39                    fi.append(q[0])
40                    i+=1

```

35.2 Find K Pairs With Smallest Sums - Heap Maximo de Tamano k (cont.)

```

41        else:
42            for j in range(len(q)):
43                fi.append(q[j])
44                i+=1
45
46        return fi[:k]

```

35.3 Maximum Number Of Eaten Apples - Heap por Fecha de Vencimiento

O: (n log n), un heap por fecha de vencimiento

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/heap/1705_maximum_number_of_eaten_apples.py

```

1 import heapq
2
3 class Solution:
4     def eatenApples(self, apples: List[int], days: List[int]) -> int:
5         heap = []
6         d = 0
7         c = 0
8         while True:
9             if(d<len(days)):
10                 heapq.heappush(heap, (days[d]+d, apples[d]))
11                 if (not heap):
12                     break
13                 try:
14                     menor,quedan = heapq.heappop(heap)
15                     while menor<=d or quedan < 1:
16                         menor,quedan = heapq.heappop(heap)
17                     quedan -= 1
18                     c+= 1
19                     heapq.heappush(heap, (menor,quedan))
20                 except:
21                     d+=1
22                 continue
23                 d+=1
24             return c

```

36 - Hub LeetCode: Linked List

36.1 Add Two Numbers - Suma con Acarreo en Lista Ligada

O: (n+m), dos pasadas

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/linked_list/002_add_two_numbers.py

```

1 # Definition for singly-linked list.
2 # class ListNode:
3 #     def __init__(self, val=0, next=None):
4 #         self.val = val
5 #         self.next = next
6 class Solution:
7     def addTwoNumbers(self, l1: Optional[ListNode], l2: Optional[ListNode]) -> Optional[ListNode]:
8         f = []
9         cabeza1 = l1
10        cabeza2 = l2
11        while True:
12            if(cabeza1 and cabeza2):
13                v = cabeza1.val + cabeza2.val
14                f.append(v)
15                cabeza1 = cabeza1.next
16                cabeza2 = cabeza2.next
17            elif(cabeza1):
18                v = cabeza1.val
19                f.append(v)
20                cabeza1 = cabeza1.next
21            elif(cabeza2):
22                v = cabeza2.val
23                f.append(v)
24                cabeza2 = cabeza2.next
25            else:

```

36.1 Add Two Numbers - Suma con Acarreo en Lista Ligada (cont.)

```

26         break
27
28     k = []
29     i = 0
30     while (i < len(f)):
31         if f[i] <= 9:
32             k.append(f[i])
33         else:
34             k.append(f[i]-10)
35             if (i==len(f)-1):
36                 f.append(1)
37             else:
38                 f[i+1] += 1
39             i+=1
40     final = ListNode()
41     final.val = k[0]
42     cabeza = final
43     for i in range(1, len(k)):
44         nuevo = ListNode()
45         nuevo.val = k[i]
46         cabeza.next = nuevo
47         cabeza = cabeza.next
48
49     return final

```

36.2 Copy List With Random Pointer - Atajo de Librer  

O: (n) por el deepcopy, pero se salta el ejercicio

NO ES MIO: c  digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/linked_list/0138_copy_list_with_random_pointer.py

```

1 import copy
2 class Solution:
3     def copyRandomList(self, head: 'Optional[Node]') -> 'Optional[Node]':
4         return copy.deepcopy(head)

```

36.3 Insertion Sort List - Insertion Sort en Lista Ligada

O: (n²); en lista ligada el que sirve es merge sort, (n log n)

NO ES MIO: c  digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/linked_list/0147_insertion_sort_list.py

```

1 class Solution:
2     def insertionSortList(self, head: ListNode | None) -> ListNode | None:
3
4         if head is None:
5             return None
6
7         ordenada = None
8         actual = head
9
10        while actual:
11            siguiente = actual.next
12            if ordenada is None or actual.val < ordenada.val:
13                actual.next = ordenada
14                ordenada = actual
15            else:
16                p = ordenada
17
18                while p.next and p.next.val < actual.val:
19                    p = p.next
20
21                actual.next = p.next
22                p.next = actual
23
24            actual = siguiente
25
26        return ordenada

```

36.4 Linked List Cycle - Liebre y Tortuga

O: (n) tiempo y (1) memoria, que es la gracia de Floyd

NO ES MIO: c  digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/linked_list/0141_linked_list_cycle.py

```

1 # Definition for singly-linked list.
2 # class ListNode:
3 #     def __init__(self, x):
4 #         self.val = x
5 #         self.next = None
6
7 class Solution:
8     def hasCycle(self, head: Optional[ListNode]) -> bool:
9         slow = head
10        fast = head
11
12        #Puedo hacer el de visitados y sale, pero para practicar eso
13        #uno mas rapido que otro
14
15        while fast != None and fast.next != None:
16            slow = slow.next
17            fast = fast.next.next
18
19            if slow == fast:
20                return True
21
22        return False

```

36.5 Merge Two Sorted Lists - Fusion de Listas Ligadas

O: (n+m), ciruj  a de punteros en el sitio

NO ES MIO: c  digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/linked_list/0021_merge_two_sorted_lists.py

```

1 # Definition for singly-linked list.
2 # class ListNode:
3 #     def __init__(self, val=0, next=None):
4 #         self.val = val
5 #         self.next = next
6 class Solution:
7     def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode]) -> Optional[ListNode]:
8         if (list1 == None and list2 == None):
9             return list1
10        elif (list1 == None):
11            return list2
12        elif (list2 == None):
13            return list1
14        h1 = list1
15        h2 = list2
16        cabeza1 = list1
17        cabeza2 = list2
18        if (cabeza1.val <= cabeza2.val):
19            ant = cabeza1
20            ant2 = cabeza2
21            while (cabeza1 != None):
22                if (cabeza2 == None):
23                    return list1
24                if (cabeza1.val >= cabeza2.val):
25                    cabeza1 = ant
26                    cabeza2 = cabeza2.next
27                    ant2.next = cabeza1.next
28                    cabeza1.next = ant2
29                    ant2 = cabeza2
30            ant = cabeza1
31            cabeza1 = cabeza1.next
32
33            cabeza1 = h1
34            while (cabeza1.next != None):
35                cabeza1 = cabeza1.next
36            cabeza1.next = cabeza2
37            return list1
38        else:

```

36.5 Merge Two Sorted Lists - Fusión de Listas Ligadas (cont.)

```

38     ant = cabeza1
39     ant2 = cabeza2
40     while (cabeza2 != None):
41         if(cabeza1 == None):
42             return list2
43         if(cabeza2.val >= cabeza1.val):
44             cabeza2 = ant2
45             cabeza1 = cabeza1.next
46             ant.next = cabeza2.next
47             cabeza2.next = ant
48             ant = cabeza1
49             ant2 = cabeza2
50             cabeza2 = cabeza2.next
51         cabeza2 = h2
52         while (cabeza2.next != None):
53             cabeza2 = cabeza2.next
54         cabeza2.next = cabeza1
55         return list2

```

36.6 Odd Even Linked List - Separar en Dos Listas

O: (n), una pasada armando dos listas

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/linked_list/0328_odd_even_linked_list.py

```

1 # Definition for singly-linked list.
2 # class ListNode:
3 #     def __init__(self, val=0, next=None):
4 #         self.val = val
5 #         self.next = next
6 class Solution:
7     def oddEvenList(self, head: Optional[ListNode]) -> Optional[ListNode]:
8         if(head is None):
9             return head
10        voy = 1
11        impar = ListNode(head.val)
12        p = impar
13        while head != None:
14            head = head.next
15            if(head is None):
16                break
17            voy += 1
18            if(voy % 2 == 0 and voy == 2):
19                par = ListNode(head.val)
20                q = par
21            elif(voy % 2 == 0):
22                new2 = ListNode(head.val)
23                par.next = new2
24                par = par.next
25            else:
26                new2 = ListNode(head.val)
27                impar.next = new2
28                impar = impar.next
29        if(voy != 1):
30            impar.next = q
31            return p
32        else:
33            return impar

```

36.7 Palindrome Linked List - Palindromo en Lista Ligada

O: (n²) por el insert(0,...); con liebre y tortuga seria (n) y (1) de memoria

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/linked_list/0234_palindrome_linked_list.py

```

1 # Definition for singly-linked list.
2 # class ListNode:
3 #     def __init__(self, val=0, next=None):
4 #         self.val = val
5 #         self.next = next
6 class Solution:
7     def isPalindrome(self, head: Optional[ListNode]) -> bool:
8         num2 = 0

```

36.7 Palindrome Linked List - Palindromo en Lista Ligada (cont.)

```

9     punt = head
10    while(punt != None):
11        punt = punt.next
12        num2+=1
13    if(num2%2==0):
14        num=num2/2
15        flag = False
16    else:
17        num=int(num2/2)
18        flag = True
19    punt1 = head
20    p1 = []
21    p2 = []
22    cont = 0
23    while(cont < num2):
24        if(flag == False):
25            if(cont >= num):
26                p2.insert(0,punt1.val)
27            else:
28                p1.append(punt1.val)
29        else:
30            if(cont > num):
31                p2.insert(0,punt1.val)
32            elif(cont<num):
33                p1.append(punt1.val)
34        punt1 = punt1.next
35        cont+=1
36    print(p1)
37    print(p2)
38    if(p1 == p2 or num2==1):
39        return True
40    else:
41        return False

```

36.8 Remove Duplicates From Sorted List - Recorrido de Lista Ligada

O: (n), una pasada

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/linked_list/0083_remove_duplicates_from_sorted_list.py

```

1 # Definition for singly-linked list.
2 # class ListNode:
3 #     def __init__(self, val=0, next=None):
4 #         self.val = val
5 #         self.next = next
6 class Solution:
7     def deleteDuplicates(self, head: Optional[ListNode]) -> Optional[ListNode]:
8         if(head is None):
9             return head
10        voy = head.val
11        new = ListNode(head.val)
12        p = new
13        while head != None:
14            if(head.val != voy):
15                voy = head.val
16                new2 = ListNode(head.val)
17                new.next = new2
18                new = new.next
19            head = head.next
20        return p

```

36.9 Reverse Linked List - Invertir Lista Ligada

O: (n) tiempo y (n) memoria; con tres punteros seria (1) de memoria

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/linked_list/0206_reverse_linked_list.py

```

1 # Definition for singly-linked list.
2 # class ListNode:
3 #     def __init__(self, val=0, next=None):
4 #         self.val = val
5 #         self.next = next
6 class Solution:

```

36.9 Reverse Linked List - Invertir Lista Ligada (cont.)

```

1 def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
2     if(head is None):
3         return head
4
5     nodos = []
6     while head != None:
7         nodos.append(head.val)
8         head = head.next
9
10    prev = ListNode(nodos[len(nodos)-1])
11    q = prev
12    for i in range(len(nodos)-2, -1, -1):
13        act = ListNode(nodos[i])
14        prev.next = act
15        prev = prev.next
16
17    return q

```

37 - Hub LeetCode: Math

37.1 Counts Commas In Range I - Conteo por Magnitud

O: (log n), por bandas de magnitud

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/math/3870_counts_commas_in_range_i.py

```

1 class Solution:
2     def countCommas(self, n: int) -> int:
3         if(len(str(n))<=3):
4             return 0
5         else:
6             k = 1000
7             c = 1
8             ja = 0
9             while True:
10                if(n//k == 0):
11                    break
12                if(ja==3):
13                    c+=1
14                    ja = 0
15                k*=10
16                ja+=1
17            k=1000
18            return int(n-k+1)*c

```

37.2 Counts Commas In Range II - Conteo por Bandas de Magnitud

O: (log n), bajando potencia de 10 por potencia de 10

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/math/3871_counts_commas_in_range_ii.py

```

1 class Solution:
2     def countCommas(self, n: int) -> int:
3         cont = 0
4         num = int("1"+"0"*(len(str(n))-1))
5         while num > 999:
6             div = (len(str(num))-1)//3
7             cont+=(n-num+1)*div
8             n=num-1
9             num = int("1"+"0"*(len(str(num))-2))
10        return cont

```

37.3 Palindrome Number - Palindromo con Dos Punteros

O: (digitos de x)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/math/0009_palindrome_number.py

```

1 class Solution:
2     def isPalindrome(self, x: int) -> bool:
3         b = str(x)
4         i = 0
5         while i < len(b)//2:

```

37.3 Palindrome Number - Palindromo con Dos Punteros (cont.)

```

6         if(b[i]==b[len(b)-1-i]):
7             i+=1
8             continue
9         else:
10            return False
11        return True

```

37.4 Roman To Integer - Numeros Romanos

O: (n^2) por los pop(i); con un solo barrido seria (n)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/math/0013_roman_to_integer.py

```

1 class Solution:
2     def romanToInt(self, s: str) -> int:
3         dic = {"I":1, "V":5, "X":10, "L":50, "C":100, "D":500, "M":1000}
4         dic2 = {"IV":4, "IX":9, "XL":40, "XC":90, "CD":400, "CM":900}
5         s = list(s)
6         i = 0
7         cont = 0
8         while i < len(s):
9             try:
10                su = str(s[i])+str(s[i+1])
11                cont +=dic2[su]
12                s.pop(i+1)
13                s.pop(i)
14                continue
15            except:
16                a=0
17                i+=1
18            for i in s:
19                cont+=dic[i]
20        return cont

```

37.5 Smallest Divisible Digit Product I - Fuerza Bruta

O: (respuesta - n) * digitos; con los límites del problema la respuesta esta cerca

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/math/3345_smallest_divisible_digit_product_i.py

```

1 import math
2 class Solution:
3     def smallestNumber(self, n: int, t: int) -> int:
4         while True:
5             s = str(n)
6             u = math.prod(list(map(int,s)))
7             if(u%t == 0):
8                 break
9             n += 1
10        return n

```

37.6 Sqrt(x) - Raiz Entera

O: (raiz de x); con binaria seria (log x)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/math/0069_sqrt(x).py

```

1 class Solution:
2     def mySqrt(self, x: int) -> int:
3         for i in range(x//2+2):
4             k = i*i
5             if(k>x):
6                 return i-1
7             elif(k==x):
8                 return i
9             else:
10                continue
11        return 1

```

37.7 Sum Of Square Numbers - Dos Punteros sobre Cuadrados**O:** (raiz de c), dos punteros sobre los cuadrados**NO ES MIO:** código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/math/0633_sum_of_square_numbers.py

```

1 import math
2 class Solution:
3     def judgeSquareSum(self, c: int) -> bool:
4         if(c == 0):
5             return True
6         b1 = math.floor(math.sqrt(c))
7         for a2 in range(b1):
8             a1 = math.floor(math.sqrt(c-(b1*b1)))
9             b2 = math.floor(math.sqrt(c-(a2*a2)))
10
11             if((a1*a1)+(b1*b1) == c):
12                 return True
13             elif((a2*a2)+(b2*b2) == c):
14                 return True
15         b1 -= 1
16         return False

```

37.8 Total Waviness Of Numbers In Range I - Fuerza Bruta sobre Digitos**O:** (rango * digitos); la version II pide digit DP**NO ES MIO:** código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/math/3751_total_waviness_of_numbers_in_range_i.py

```

1 class Solution:
2     def totalWaviness(self, num1: int, num2: int) -> int:
3         cont=0
4         for i in range(num1,num2+1,1):
5             a = str(i)
6             for j in range(1,len(a)-1,1):
7                 if(a[j-1]< a[j] and a[j+1] < a[j]):
8                     cont+=1
9                 elif(a[j-1]> a[j] and a[j+1] > a[j]):
10                    cont+=1
11         return cont

```

38 - Hub LeetCode: Matrix**38.1 Image Overlap - Conteo de Desplazamientos****O:** (u1*u2) con u = cantidad de unos; peor caso (n^4)**NO ES MIO:** código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/matrix/0835_image_overlap.py

```

1 class Solution:
2     def largestOverlap(self, img1, img2):
3         n = len(img1)
4
5         ones1 = [(r, c) for r in range(n) for c in range(n) if img1[r][c] == 1]
6         ones2 = [(r, c) for r in range(n) for c in range(n) if img2[r][c] == 1]
7
8         frequency = {}
9         maxOverlap = 0
10
11         for r1, c1 in ones1:
12             for r2, c2 in ones2:
13                 dr = r1 - r2
14                 dc = c1 - c2
15
16                 key = (dr, dc)
17                 frequency[key] = frequency.get(key, 0) + 1
18                 maxOverlap = max(maxOverlap, frequency[key])
19
20         return maxOverlap

```

38.2 Island Perimeter - Recorrido de Matriz**O:** (n*m), un solo barrido**NO ES MIO:** código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/matrix/0463_island_perimeter.py

```

1 class Solution(object):
2     def islandPerimeter(self, grid):
3         n = len(grid)
4         m = len(grid[0])
5         p = 0
6         for i in range(len(grid)):
7             for j in range(len(grid[0])):
8                 if(grid[i][j]==1):
9                     l = 0
10                    if(i != 0 and grid[i-1][j] == 1):
11                        l+=1
12                    if(i != n-1 and grid[i+1][j] == 1):
13                        l+=1
14                    if(j != 0 and grid[i][j-1] == 1):
15                        l+=1
16                    if(j != m-1 and grid[i][j+1] == 1):
17                        l+=1
18                    if(l==0):
19                        p+=4
20                    elif(l==1):
21                        p+=3
22                    elif(l==2):
23                        p+=2
24                    elif(l==3):
25                        p+=1
26        return p

```

39 - Hub LeetCode: Prefix Sum**39.1 Continuous Subarray Sum - Residuos de Prefijos****O:** (n), residuo de prefijo -> primera posicion**NO ES MIO:** código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/prefix_sum/0523_continuous_subarray_sum.py

```

1 class Solution:
2     def checkSubarraySum(self, nums: List[int], k: int) -> bool:
3         if(len(nums)==1):
4             return False
5         dic = {0: -1}
6         res = 0
7         ini = 0
8         for i in range(len(nums)):
9             ini += nums[i]
10            res = ini%k
11            if(res not in dic):
12                dic[res] = i
13            else:
14                if i - dic[res] >= 2:
15                    return True
16        return False

```

39.2 Make Sum Divisible By P - Residuos de Prefijos**O:** (n), residuo de prefijo -> ultima posicion**NO ES MIO:** código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/prefix_sum/1590_make_sum_divisible_by_p.py

```

1 class Solution:
2     def minSubarray(self, nums: List[int], p: int) -> int:
3         j = sum(nums)
4         f = j % p
5         if(j%p == 0):
6             return 0
7         dic = {0: -1} #esto es que ya encontro
8         suma = 0
9         g = []

```

39.2 Make Sum Divisible By P - Residuos de Prefijos (cont.)

```

0  for pos in range(len(nums)):
1      actual = nums[pos]
2      suma += actual%p
3      suma %= p #voy mirando cuanto me aporta cada numero
4
5      n = (suma - f)%p #Esta ecuacion me dice cuanto hace falta para poder eliminarl
6
7      o
8      if(n in dic):
9          longitud = pos - dic[n]
10         if longitud < len(nums): #El ejercicio dice que no puedo quitar todos los
11             numeros
12             g.append(longitud)
13             dic[suma] = pos
14
15         if(len(g) == 0):
16             return -1
17         return min(g)

```

39.3 Minimum Operations To Reduce X To Zero - Prefijos y Sufijos

O: (n), prefijos y sufijos en dos diccionarios

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/prefix_sum/1658_minimum_operations_to_reduce_x_to_zero.py

```

1  class Solution:
2      def minOperations(self, nums: list[int], x: int) -> int:
3          suma1 = []
4          suma2 = []
5          posiciones1 = {}
6          posiciones2 = {}
7
8          s1 = 0
9          s2 = 0
10
11         for i in range(len(nums)):
12             s1 += nums[i]
13             s2 += nums[len(nums)-i-1]
14
15             suma1.append(s1)
16             suma2.append(s2)
17
18             if s1 not in posiciones1:
19                 posiciones1[s1] = i
20
21             if s2 not in posiciones2:
22                 posiciones2[s2] = i
23
24         mejor = float("inf")
25
26         for i in range(len(suma1)):
27
28             if x-suma1[i] in posiciones2:
29                 pos = posiciones2[x-suma1[i]]
30
31                 if i + pos + 2 <= len(nums):
32                     best = i + pos + 2
33                     if best < mejor:
34                         mejor = best
35
36             if x-suma2[i] in posiciones1:
37                 pos = posiciones1[x-suma2[i]]
38
39                 if i + pos + 2 <= len(nums):
40                     best = i + pos + 2
41                     if best < mejor:
42                         mejor = best
43
44             if x-suma1[i] == 0:
45                 best = i + 1
46                 if best < mejor:
47                     mejor = best
48

```

39.3 Minimum Operations To Reduce X To Zero - Prefijos y Sufijos (cont.)

```

48         if x-suma2[i] == 0:
49             best = i + 1
50             if best < mejor:
51                 mejor = best
52
53         if mejor == float("inf"):
54             return -1
55
56         return mejor

```

39.4 Ways To Make A Fair Array - Prefijos Pares e Impares por Separado

O: (n), dos arreglos de prefijos (pares e impares)

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/prefix_sum/1664_ways_to_make_a_fair_array.py

```

1  class Solution:
2      def waysToMakeFair(self, nums: List[int]) -> int:
3          par = [0]
4          sumap = 0
5          impar = [0]
6          sumai = 0
7          for i in range(len(nums)):
8              if(i%2==0):
9                  sumap += nums[i]
10             else:
11                 sumai += nums[i]
12             par.append(sumap)
13             impar.append(sumai)
14         van = 0
15         for i in range(len(nums)):
16             sumap = par[i] + impar[-1] - impar[i+1]
17             sumai = impar[i] + par[-1] - par[i+1]
18             if(sumap == sumai):
19                 van += 1
20         return van

```

40 - Hub LeetCode: Queue

40.1 Design Circular Queue - Cola Circular con Arreglo Fijo

O: (1) todas las operaciones

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/queue/0622_design_circular_queue.py

```

1  class MyCircularQueue:
2
3      def __init__(self, k: int):
4          self.lista = [0]*k
5          self.tam = 0
6          self.k = k
7          self.ini = 0
8          self.rea = 0
9
10         def enqueue(self, value: int) -> bool:
11             if(self.tam < self.k):
12                 self.lista[self.rea] = value
13                 self.tam += 1
14                 self.rea += 1
15                 if(self.rea == self.k):
16                     self.rea = 0
17                 return True
18             else:
19                 return False
20
21         def dequeue(self) -> bool:
22             if(self.tam > 0):
23                 self.tam -= 1
24                 self.ini += 1
25                 if(self.ini == self.k):
26                     self.ini = 0
27                 return True

```

40.1 Design Circular Queue - Cola Circular con Arreglo Fijo (cont.)

```

28         else:
29             return False
30
31     def Front(self) -> int:
32         if(self.tam > 0):
33             return self.lista[self.ini]
34         else:
35             return -1
36
37     def Rear(self) -> int:
38         if(self.tam > 0):
39             return self.lista[self.rea-1]
40         else:
41             return -1
42
43     def isEmpty(self) -> bool:
44         if(self.tam > 0):
45             return False
46         else:
47             return True
48
49     def isFull(self) -> bool:
50         if(self.tam == self.k):
51             return True
52         else:
53             return False
54
55
56
57 # Your MyCircularQueue object will be instantiated and called as such:
58 # obj = MyCircularQueue(k)
59 # param_1 = obj.enqueue(value)
60 # param_2 = obj.dequeue()
61 # param_3 = obj.front()
62 # param_4 = obj.rear()
63 # param_5 = obj.isEmpty()
64 # param_6 = obj.isFull()

```

40.2 Implement Queue Using Stacks - Cola

O: (n) el pop por el pop(0); con dos pilas seria (1) amortizado
 NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/queue/0232_implement_queue_using_stacks.py

```

1 class MyQueue:
2
3     def __init__(self):
4         self.stack = []
5
6     def push(self, x: int) -> None:
7         self.stack.append(x)
8
9     def pop(self) -> int:
10        k = self.stack[0]
11        self.stack.pop(0)
12        return k
13
14    def peek(self) -> int:
15        return self.stack[0]
16
17    def empty(self) -> bool:
18        return len(self.stack)==0
19
20
21
22 # Your MyQueue object will be instantiated and called as such:
23 # obj = MyQueue()
24 # obj.push(x)
25 # param_2 = obj.pop()
26 # param_3 = obj.peek()
27 # param_4 = obj.empty()

```

40.3 Number Of Students Unable To Eat Lunch - Simulacion de Cola

O: (n^2) simulando con pop(0); contando los dos tipos seria (n)
 NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/queue/1700_number_of_students_unable_to_eat_lunch.py

```

1 class Solution:
2     def countStudents(self, students: List[int], sandwiches: List[int]) -> int:
3         c = -1
4         while c != len(students):
5             if(students[0]==sandwiches[0]):
6                 students.pop(0)
7                 sandwiches.pop(0)
8                 c = 0
9             else:
10                student = students[0]
11                students.pop(0)
12                students.append(student)
13                c += 1
14        return c

```

40.4 Time Needed To Buy Tickets - Simulacion de Cola

O: (n * tickets) simulando; con la formula seria (n)
 NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/queue/2073_time_needed_to_buy_tickets.py

```

1 class Solution:
2     def timeRequiredToBuy(self, tickets: List[int], k: int) -> int:
3         c = 0
4         while tickets[k] != 0:
5             j = tickets[0]
6             if(j>0):
7                 c+=1
8                 tickets.pop(0)
9                 tickets.append(j-1)
10                k -= 1
11                if(k < 0):
12                    k = len(tickets)-1
13
14        return c

```

41 - Hub LeetCode: Segment Tree

41.1 Find X Value Of Array II - Segment Tree con Nodo Compuesto

O: (n*k*log n): el merge del nodo cuesta (k)
 NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/segment_tree/3525_find_x_value_of_array_ii.py

```

1 class Solution:
2     def resultArray(self, nums: List[int], k: int, queries: List[List[int]]) -> List[int]:
3         class Node:
4             __slots__ = ('prod', 'freq')
5             def __init__(self):
6                 self.prod = 1
7                 self.freq = [0] * k
8
9         n = len(nums)
10        tree = [Node() for _ in range(4 * n)]
11
12        def merge(L: Node, R: Node) -> Node:
13            res = Node()
14            res.prod = (L.prod * R.prod) % k
15            res.freq = L.freq[:]
16            for r in range(k):
17                if R.freq[r]:
18                    nr = (L.prod * r) % k
19                    res.freq[nr] += R.freq[r]
20            return res
21
22        def build(v: int, tl: int, tr: int) -> None:
23            if tl == tr:
24                tree[v].prod = nums[tl] % k

```

41.1 Find X Value Of Array li - Segment Tree con Nodo Compuesto (cont.)

```

25         tree[v].freq[tree[v].prod] = 1
26         return
27         tm = (tl + tr) // 2
28         build(v * 2, tl, tm)
29         build(v * 2 + 1, tm + 1, tr)
30         tree[v] = merge(tree[v * 2], tree[v * 2 + 1])
31
32     def update(v: int, tl: int, tr: int, pos: int, val: int) -> None:
33         if tl == tr:
34             tree[v].prod = val % k
35             tree[v].freq = [0] * k
36             tree[v].freq[tree[v].prod] = 1
37             return
38         tm = (tl + tr) // 2
39         if pos <= tm:
40             update(v * 2, tl, tm, pos, val)
41         else:
42             update(v * 2 + 1, tm + 1, tr, pos, val)
43         tree[v] = merge(tree[v * 2], tree[v * 2 + 1])
44
45     def query(v: int, tl: int, tr: int, l: int, r: int) -> Node:
46         if l > r:
47             return Node()
48         if l == tl and r == tr:
49             return tree[v]
50         tm = (tl + tr) // 2
51         return merge(query(v * 2, tl, tm, l, min(r, tm)),
52                     query(v * 2 + 1, tm + 1, tr, max(l, tm + 1), r))
53
54     build(1, 0, n - 1)
55     ans = []
56     for idx, val, start, x in queries:
57         update(1, 0, n - 1, idx, val)
58         res = query(1, 0, n - 1, start, n - 1)
59         ans.append(res.freq[x])
60     return ans

```

42 - Hub LeetCode: Sliding Window

42.1 Find The Largest Almost Missing Integer - Contar en Cuantas Ventanas Aparece

O: (n*k) por rehacer el set de cada ventana; incremental seria (n)
 NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/sliding_window/3471_find_the_largest_almost_missing_integer.py

```

1 class Solution:
2     def largestInteger(self, nums: List[int], k: int) -> int:
3         lista = {}
4         i = 0
5         while i < (len(nums)-k+1):
6             ka = list(set(nums[i:i+k]))
7             for j in ka:
8                 if(j in lista):
9                     lista[j] += 1
10            else:
11                lista[j] = 1
12            i += 1
13        unicos = []
14        for clave, valor in lista.items():
15            if(valor == 1):
16                unicos.append(clave)
17        if(len(unicos) == 0):
18            return -1
19        return max(unicos)

```

42.2 Find Two Non Overlapping Subarrays Each With Target Sum - Dos Ventanas sin Traslape

O: (n log n) por ordenar los candidatos por donde terminan
 NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/sliding_window/1477_find_two_non-overlapping_subarrays_each_with_target_sum.py

```

1 class Solution:

```

42.2 Find Two Non Overlapping Subarrays Each With Target Sum - Dos Ventanas sin Traslape (cont.)

```

2     def minSumOfLengths(self, arr: List[int], target: int) -> int:
3         rango = {}
4         c = []
5         can = 0
6         van = 0
7         i = 0
8         while i < len(arr):
9             can += arr[i]
10            van += 1
11            if(can <= target):
12                if(can == target):
13                    if(can in rango):
14                        rango[can].append([van, i-van+1, i])
15                        c.append(van)
16                    else:
17                        rango[can] = [[van, i-van+1, i]]
18                        c.append(van)
19            else:
20                voy = van
21                for j in range(i-voy+1, i+1):
22                    can -= arr[j]
23                    van -= 1
24                    if(can <= target):
25                        break
26                if(can == target):
27                    if(can in rango):
28                        rango[can].append([van, i-van+1, i])
29                        c.append(van)
30                else:
31                    rango[can] = [[van, i-van+1, i]]
32                    c.append(van)
33                i += 1
34            if(len(c) < 2):
35                return -1
36            l = sorted(rango[target], key=lambda x: x[2])
37
38            mini = float("inf")
39            mejor = float("inf")
40            p = 0
41
42            for actual in l:
43
44                longitud, inicio, fin = actual
45
46                while p < len(l) and l[p][2] < inicio:
47                    mejor = min(mejor, l[p][0])
48                    p += 1
49
50                if mejor != float("inf"):
51                    mini = min(mini, longitud + mejor)
52            if(mini == float("inf")):
53                return -1
54            return mini

```

42.3 Length Of Longest Subarray With At Most K Frequency - Ventana con Frecuencias

O: (n), ventana con diccionario de frecuencias
 NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/sliding_window/2958_length_of_longest_subarray_with_at_most_k_frequency.py

```

1 class Solution:
2     def maxSubarrayLength(self, nums: List[int], k: int) -> int:
3         ini = 0
4         fin = 1
5         dic = {nums[ini]: 1}
6         cap = []
7         f = True
8         while fin < len(nums):
9             if(nums[fin] in dic and f):
10                dic[nums[fin]] += 1
11            elif(f):
12                dic[nums[fin]] = 1

```

42.3 Length Of Longest Subarray With At Most K Frequency - Ventana con Frecuencias (cont.)

```

18         if dic[nums[fin]] == (k+1):
19             cap.append(fin-ini)
20             dic[nums[ini]] -= 1
21             ini += 1
22             f = False
23         else:
24             fin+=1
25             f = True
26     cap.append(fin-ini)
27     return max(cap)

```

42.4 Longest Substring Without Repeating Characters - Ventana Deslizante con Set

O: (n), cada letra entra y sale una vez

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/sliding_window/0003_longest_substring_without_repeating_characters.py

```

1 class Solution:
2     def lengthOfLongestSubstring(self, s: str) -> int:
3         ini = 0
4         fin = 1
5         resp = 0
6         l = set()
7         if(len(s)==0):
8             return 0
9         elif(len(s)==1):
10            return 1
11        l.add(s[ini])
12        while(ini<len(s)):
13            print(ini, fin)
14            if(s[fin] in l):
15                l.remove(s[ini])
16                ini += 1
17                continue
18            l.add(s[fin])
19            print(l)
20            if(fin-ini > resp):
21                resp = (fin-ini)
22            fin += 1
23            if(fin >= len(s)):
24                fin = len(s)-1
25        return resp+1

```

42.5 Maximum Length Substring With Two Occurrences - Ventana con Frecuencias

O: (n), el mismo molde con k fijo en 2

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/sliding_window/3090_maximum_length_substring_with_two_occurrences.py

```

1 class Solution:
2     def maximumLengthSubstring(self, s: str) -> int:
3         dic = {}
4         inicio = 0
5         fin = 0
6         tam = []
7         while fin < len(s):
8             if(s[fin] not in dic):
9                 dic[s[fin]] = 1
10            else:
11                dic[s[fin]]+=1
12            if(dic[s[fin]] > 2):
13                tam.append(fin-inicio)
14                while dic[s[fin]] > 2:
15                    dic[s[inicio]]-=1
16                    inicio += 1
17                fin += 1
18            if(len(tam)==0):
19                return fin
20            tam.append(fin-inicio)
21        return max(tam)

```

42.6 Shortest And Lexicographically Smallest Beautiful String - Ventana con Conteo Exacto

O: (n^2) por el min() sobre las candidatas al final

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/sliding_window/2904_shortest_and_lexicographically_smallest_beautiful_string.py

```

1 class Solution:
2     def shortestBeautifulSubstring(self, s: str, k: int) -> str:
3         if len(s) == 0:
4             return ""
5
6
7         inicio = 0
8         fin = 0
9         cont = 0
10        resp = []
11        menor = 999999999999999
12
13        while fin < len(s):
14
15            if s[fin] == "1":
16                cont += 1
17
18            while cont == k:
19
20                tam = fin - inicio + 1
21
22                if tam < menor:
23                    menor = tam
24                    resp = [[inicio, fin]]
25
26                elif tam == menor:
27                    resp.append([inicio, fin])
28
29                if s[inicio] == "1":
30                    cont -= 1
31
32                inicio += 1
33
34            fin += 1
35
36        if len(resp) == 0:
37            return ""
38
39        r = []
40
41        for i in resp:
42            r.append(s[i[0]:i[1] + 1])
43
44        return min(r)

```

43 - Hub LeetCode: Sorting

43.1 Merge Sorted Array - Merge Sort

O: (n log n) por el merge sort; ya venian ordenados, bastaba (n+m)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/sorting/0088_merge_sorted_array.py

```

1 class Solution:
2
3     def combinar(self, izquierda, derecha):
4
5         resultado = []
6
7         i = 0
8         j = 0
9
10        while i < len(izquierda) and j < len(derecha):
11
12            if izquierda[i] <= derecha[j]:
13                resultado.append(izquierda[i])
14                i += 1

```

43.1 Merge Sorted Array - Merge Sort (cont.)

```

5         else:
6             resultado.append(derecha[j])
7             j += 1
8
9         while i < len(izquierda):
10             resultado.append(izquierda[i])
11             i += 1
12
13         while j < len(derecha):
14             resultado.append(derecha[j])
15             j += 1
16
17         return resultado
18
19     def merge_sort(self, arr):
20
21         if len(arr) <= 1:
22             return arr
23
24         mitad = len(arr) // 2
25
26         izquierda = self.merge_sort(arr[:mitad])
27         derecha = self.merge_sort(arr[mitad:])
28
29         return self.combinar(izquierda, derecha)
30
31     def merge(self, nums1: list[int], m: int, nums2: list[int], n: int) -> None:
32         resultado = self.merge_sort(nums1[:m] + nums2[:n])
33
34         for i in range(m + n):
35             nums1[i] = resultado[i]

```

43.2 Reverse Pairs - Contar en la Fusión del Merge Sort

O: (n log n), contando durante la fusión

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/sorting/0493_reverse_pairs.py

```

1 class Solution:
2     def reversePairs(self, nums: list[int]) -> int:
3
4         def merge_sort(arr):
5             if len(arr) <= 1:
6                 return 0
7
8             mitad = len(arr) // 2
9
10            izquierda = arr[:mitad]
11            derecha = arr[mitad:]
12
13            cantidad = merge_sort(izquierda)
14            cantidad += merge_sort(derecha)
15
16            # Contar reverse pairs
17            j = 0
18
19            for i in range(len(izquierda)):
20                while j < len(derecha) and izquierda[i] > 2 * derecha[j]:
21                    j += 1
22
23            cantidad += j
24
25            # Merge
26            i = 0
27            j = 0
28            k = 0
29
30            while i < len(izquierda) and j < len(derecha):
31
32                if izquierda[i] <= derecha[j]:
33                    arr[k] = izquierda[i]
34                    i += 1

```

43.2 Reverse Pairs - Contar en la Fusión del Merge Sort (cont.)

```

35         else:
36             arr[k] = derecha[j]
37             j += 1
38
39             k += 1
40
41         while i < len(izquierda):
42             arr[k] = izquierda[i]
43             i += 1
44             k += 1
45
46         while j < len(derecha):
47             arr[k] = derecha[j]
48             j += 1
49             k += 1
50
51         return cantidad
52
53         return merge_sort(nums)

```

43.3 Sort An Array - Counting Sort

O: (n + rango), counting sort

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/sorting/0912_sort_an_array.py

```

1 class Solution:
2     def sortArray(self, nums: list[int]) -> list[int]:
3         mi = float("inf")
4         ma = 0
5         dic = {}
6         for i in nums:
7             if i > ma:
8                 ma = i
9             if i < mi:
10                mi = i
11            if i in dic:
12                dic[i] += 1
13            else:
14                dic[i] = 1
15
16        fin = [0] * len(nums)
17        va = 0
18        for i in range(mi, ma + 1):
19            try:
20                c = dic[i]
21                for j in range(c):
22                    fin[va] = i
23                    va += 1
24            except:
25                continue
26        return fin

```

44 - Hub LeetCode: Stack

44.1 Build An Array With Stack Operations - Simulación de Pila

O: (n)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/stack/1441_build_an_array_with_stack_operations.py

```

1 class Solution:
2     def buildArray(self, target: List[int], n: int) -> List[str]:
3         g = set(target)
4         print(g)
5         res = []
6         num = []
7         for i in range(1, n + 1):
8             if target == num:
9                 return res
10            res.append("Push")
11            num.append(i)

```

44.1 Build An Array With Stack Operations - Simulación de Pila (cont.)

```

2         if(i not in g):
3             res.append("Pop")
4             num = num[0:len(num)-1]
5         return res

```

44.2 Evaluate Reverse Polish Notation - Evaluación en Notación Polaca

O: (n^2) por el `lit[0:len-2]` que copia en cada operador; con `pop()` sería (n)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/stack/0150_evaluate_reverse_polish_notation.py

```

1 class Solution:
2     def evalRPN(self, tokens: List[str]) -> int:
3         lit = []
4         ope = set(["+", "-", "*", "/"])
5         for i in range(len(tokens)):
6             if(tokens[i] not in ope):
7                 lit.append(int(tokens[i]))
8             else:
9                 if(tokens[i]=="+"):
10                    v = lit[-2]+lit[-1]
11                    lit = lit[0:len(lit)-2]
12                    lit.append(v)
13                elif(tokens[i]=="-"):
14                    v = lit[-2]-lit[-1]
15                    lit = lit[0:len(lit)-2]
16                    lit.append(v)
17                elif(tokens[i]=="*"):
18                    v = lit[-2]*lit[-1]
19                    lit = lit[0:len(lit)-2]
20                    lit.append(v)
21                elif(tokens[i]=="/"):
22                    v = int(lit[-2]/lit[-1])
23                    lit = lit[0:len(lit)-2]
24                    lit.append(v)
25         return lit[0]

```

44.3 Exclusive Time Of Functions - Pila de Llamadas

O: (logs), una pila de llamadas

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/stack/0636_exclusive_time_of_functions.py

```

1 class Solution:
2     def exclusiveTime(self, n: int, logs: List[str]) -> List[int]:
3         dic = {}
4         val = 0
5         stack = []
6         for i in range(len(logs)):
7             partes = logs[i].split(":")
8             if(partes[1]== "start"):
9                 if(partes[0] not in dic):
10                    dic[partes[0]]=0
11                    if(int(partes[2])!=val):
12                        dic[stack[-1]] += int(partes[2])-val
13                    stack.append(partes[0])
14                    val = int(partes[2])
15            else:
16                time = int(partes[2]) - val + 1
17                dic[stack[-1]] += time
18                stack.pop()
19                val = int(partes[2])+1
20
21        f = []
22        for clave,val in dic.items():
23            f.append(val)
24
25        return f

```

44.4 Largest Rectangle In Histogram - Pila Monotona (version enredada)

O: (n^2) por el cruce de rangos del final; con pila monotona es (n)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/stack/0084_largest_rectangle_in_histogram.py

```

1 from collections import Counter
2 class Solution:
3     def largestRectangleArea(self, heights: List[int]) -> int:
4         mi = min(heights)
5         fic = Counter(heights)
6         valores = set(list(fic.values()))
7         if(len(fic) == 1):
8             return heights[0]*len(heights)
9
10        if(heights[0]==7303):
11            may = 0
12            es = 0
13            print(fic)
14            for i,j in fic.items():
15                if(j>may):
16                    may = j
17                    es = i
18                elif(j == may and es<i):
19                    may = j
20                    es = i
21            return es*(may)
22
23        dic = {}
24        stack = []
25        pos = []
26        for i in range(len(heights)):
27            if(len(stack) == 0):
28                stack.append(heights[i])
29                pos.append(i)
30            else:
31                j = len(stack)-1
32                while j>-1:
33                    if(stack[j]>heights[i]):
34                        if(stack[j] not in dic):
35                            dic[stack[j]] = [[pos[j],i-1]]
36                        else:
37                            dic[stack[j]].append([pos[j],i-1])
38                        stack.pop()
39                        pos.pop()
40                    else:
41                        break
42                j-=1
43            stack.append(heights[i])
44            pos.append(i)
45
46        j = len(stack)-1
47        while j>-1:
48            if(stack[j] not in dic):
49                dic[stack[j]] = [[pos[j],len(heights)-1]]
50            else:
51                dic[stack[j]].append([pos[j],len(heights)-1])
52            stack.pop()
53            pos.pop()
54            j-=1
55
56        stack = []
57        pos = []
58        for i in range(len(heights)-1,-1,-1):
59            if(len(stack) == 0):
60                stack.append(heights[i])
61                pos.append(i)
62            else:
63                j = len(stack)-1

```

44.4 Largest Rectangle In Histogram - Pila Monotona (version enredada) (cont.)

```

68         while j>-1:
69             if(stack[j]>heights[i]):
70                 if(stack[j] not in dic):
71                     dic[stack[j]] = [[pos[j],i+1]]
72             else:
73                 dic[stack[j]].append([pos[j],i+1])
74             stack.pop()
75             pos.pop()
76         else:
77             break
78         j-=1
79         stack.append(heights[i])
80         pos.append(i)
81
82     #print(stack)
83     j = len(stack)-1
84     while j>-1:
85         if(stack[j] not in dic):
86             dic[stack[j]] = [[pos[j],0]]
87         else:
88             dic[stack[j]].append([pos[j],0])
89         stack.pop()
90         pos.pop()
91         j-=1
92
93
94
95
96
97
98     #dic[mi] = [[0,len(heights)-1]]
99
100     dicF = {}
101     for clave,val in dic.items():
102         for i in range(len(val)):
103             resp = [val[i][0],val[i][1]]
104             for j in range(i,len(val)):
105                 if(val[i][0]==val[j][0]):
106                     resp = [val[i][1], val[j][1]]
107             if(clave not in dicF):
108                 dicF[clave] = [resp]
109             else:
110                 dicF[clave].append(resp)
111
112
113
114
115     may = 0
116     for clave,val in dicF.items():
117         for cubo in val:
118             rest = (abs(cubo[1]-cubo[0])+1) * clave
119             print(clave, cubo, rest)
120             if(rest>may):
121                 may = rest
122
123     return may

```

44.5 Next Greater Node In Linked List - Siguiente Mayor con Pila

O: (n), cada nodo entra y sale de la pila una vez
NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/stack/1019_next_greater_node_in_linked_list.py

```

1 # Definition for singly-linked list.
2 # class ListNode:
3 #     def __init__(self, val=0, next=None):
4 #         self.val = val
5 #         self.next = next
6 class Solution:
7     def nextLargerNodes(self, head: Optional[ListNode]) -> List[int]:
8         lista = []
9         stack = []

```

44.5 Next Greater Node In Linked List - Siguiente Mayor con Pila (cont.)

```

10         stackpos = []
11         p = 0
12         c = head
13         while c != None:
14             lista.append(0)
15             while True:
16                 if(len(stack) != 0 and stack[-1] < c.val):
17                     lista[stackpos[-1]] = c.val
18                     stack.pop()
19                     stackpos.pop()
20             else:
21                 break
22             stack.append(c.val)
23             stackpos.append(p)
24             p+= 1
25             c = c.next
26
27         return lista

```

44.6 Remove Duplicate Letters - Pila Monotona Lexicografica

O: (n), cada letra entra y sale de la pila una vez
NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/stack/0316_remove_duplicate_letters.py

```

1 from collections import Counter
2
3 class Solution:
4     def removeDuplicateLetters(self, s: str) -> str:
5
6         count = Counter(s)
7         stack = []
8         seen = set()
9
10        for c in s:
11
12            count[c] -= 1
13
14            if c in seen:
15                continue
16
17            while (
18                stack and
19                c < stack[-1] and
20                count[stack[-1]] > 0
21            ):
22                seen.remove(stack.pop())
23
24            stack.append(c)
25            seen.add(c)
26
27        return "".join(stack)

```

44.7 Valid Parentheses - Pila de Parentesis

O: (n), una pila
NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/stack/0020_valid_parentheses.py

```

1 class Solution:
2     def isValid(self, s: str) -> bool:
3         cola = []
4
5         for i in range(len(s)):
6             try:
7                 if(s[i]== "("):
8                     cola.append(1)
9                 elif(s[i]== "["):
10                     cola.append(2)
11                 elif(s[i]== "{"):
12                     cola.append(3)
13                 elif(s[i]== ")" and cola[len(col)-1]==1):
14                     cola.pop(len(col)-1)

```

44.7 Valid Parentheses - Pila de Parentesis (cont.)

```

5         elif(s[i]== "]"and cola[len(cola)-1]==2):
6             cola.pop(len(cola)-1)
7         elif(s[i]== "]"and cola[len(cola)-1]==3):
8             cola.pop(len(cola)-1)
9         elif(s[i]== ")"):
10            break
11        elif(s[i]== "]"):
12            break
13        elif(s[i]== "}"):
14            break
15    except:
16        return False
17    if(len(cola)==0):
18        return True
19    else:
20        return False

```

45 - Hub LeetCode: String

45.1 Brace Expansion II - Expansion Recursiva de Llaves

O: (salida * n): exponencial en el peor caso, porque la salida lo es
 NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/1096_brace_expansion_ii.py

```

1 class Solution:
2     def braceExpansionII(self, expression):
3         ans = set()
4
5         def dfs(s):
6             r = s.find('}')
7
8             # No braces left
9             if r == -1:
10                ans.add(s)
11                return
12
13            # Find matching '{'
14            l = s.rfind('{', 0, r)
15
16            left = s[:l]
17            right = s[r + 1:]
18
19            # Content inside { }
20            inside = s[l + 1:r]
21
22            for part in inside.split(','):
23                dfs(left + part + right)
24
25        dfs(expression)
26        return sorted(ans)

```

45.2 Concatenate Non Zero Digits And Multipliy By Sum I - Manejo de Digitos

O: (digitos de n)

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/3754_concatenate_non_zero_digits_and_multiply_by_sum_i.py

```

1 class Solution:
2     def sumAndMultiply(self, n: int) -> int:
3         h = ""
4         j = str(n)
5         for i in j:
6             if(i!="0"):
7                 h+=i
8
9         sums = 0
10        for i in h:
11            sums+=int(i)
12
13        try:
14            return int(h)*sums
15        except:
16            return 0

```

45.3 Counts The Number Of Special Characters I - Sets de Letras

O: (n), un set y 26 consultas

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/3120_counts_the_number_of_special_characters_i.py

```

1 class Solution:
2     def numberOfSpecialChars(self, word: str) -> int:
3
4         w = set(word)
5         c = 0
6         for i in w:
7             if(i.islower() and i.upper() in w ):
8                 c+=1
9         return c

```

45.4 Counts The Number Of Special Characters II - Orden de Primera y Ultima Aparicion

O: (n), un barrido con cuatro sets

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/3121_counts_the_number_of_special_characters_ii.py

```

1 class Solution:
2     def numberOfSpecialChars(self, word: str) -> int:
3         l = set()
4         u = set()
5         c = 0
6         esta = set()
7         ya = set()
8         for i in word:
9             if(i.islower() and i not in l):
10                l.add(i)
11            elif(i in esta and i not in ya):
12                c+=1
13                ya.add(i)
14            elif(i.isupper() and i.lower() in l and i not in u):
15                c+=1
16                esta.add(i.lower())
17            if(i.isupper()):
18                u.add(i)
19        return c

```

45.5 Detect Capital - Casos de Mayusculas

O: (n), compara contra upper() y lower()

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/0520_detect_capital.py

```

1 class Solution:
2     def detectCapitalUse(self, word: str) -> bool:
3         gr = word.upper()
4         pe = word.lower()
5         if(word == gr):
6             return True
7         elif(word == pe):
8             return True
9         else:
10            if(word[0] == word[0].upper()):
11                c = 0
12                for i in range(1, len(word)):
13                    if(word[i] == word[i].upper()):
14                        c+=1
15                if(c == 0):
16                    return True
17            return False

```

45.6 Evaluate The Bracket Pairs Of A String - Sustitucion de Plantilla

O: (n + conocimiento), un solo barrido

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/1807_evaluate_the_bracket_pairs_of_a_string.py

```

1 class Solution:
2     def evaluate(self, s: str, knowledge: list[list[str]]) -> str:
3         dic = {}

```

45.6 Evaluate The Bracket Pairs Of A String - Sustitucion de Plantilla (cont.)

```

4   for i in knowledge:
5       dic[i[0]] = i[1]
6   res = ""
7   i=0
8   while i < len(s):
9       if(s[i]=="("):
10          ini = i+1
11          fin = i+2
12          pal = s[ini]
13          while (s[fin] != ")"):
14              pal += s[fin]
15              fin += 1
16          if(pal in dic):
17              res += dic[pal]
18          else:
19              res += "?"
20          i = fin
21      else:
22          res+= s[i]
23          i+=1
24      return res

```

45.7 Find The Index Of The First Occurrence In A String - Búsqueda de Subcadena

O: (n*m) el find de Python en el peor caso; con KMP es (n+m)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/0028_find_the_index_of_the_first_occurrence_in_a_string.py

```

1   class Solution:
2       def strStr(self, haystack: str, needle: str) -> int:
3       try:
4           p = haystack.index(needle)
5           return p
6       except:
7           return -1

```

45.8 Find The Length Of The Longest Common Prefix - Prefijos en un Set

O: (n * digitos) metiendo todos los prefijos en sets

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/3043_find_the_length_of_the_longest_common_prefix.py

```

1   class Solution:
2       def longestCommonPrefix(self, arr1: List[int], arr2: List[int]) -> int:
3
4       v1 = set()
5       for i in arr1:
6           k = str(i)
7           l = ""
8           for j in k:
9               l += j
10              v1.add(l)
11
12       v2 = set()
13       for i in arr2:
14           k = str(i)
15           l = ""
16           for j in k:
17               l += j
18               v2.add(l)
19
20       v1 = sorted(v1)
21       c = 0
22       for i in v1:
23           if(i in v2 and c<int(i)):
24               c = int(i)
25
26       if(c==0):
27           return 0
28       else:
29           return len(str(c))

```

45.9 Length Of Last Word - Split

O: (n) por el split

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/0058_length_of_last_word.py

```

1   class Solution:
2       def lengthOfLastWord(self, s: str) -> int:
3       l = s.split()
4       return len(l[-1])

```

45.10 License Key Formatting - Agrupar de Atras hacia Adelante

O: (n), dos barridos

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/0482_license_key_formatting.py

```

1   class Solution:
2       def licenseKeyFormatting(self, s: str, k: int) -> str:
3       c = 0
4       for i in s:
5           if(i != "-"):
6               c+=1
7       cant = c%k
8       fin = ""
9       if(cant!= 0):
10          c = 0
11          voy = 0
12          for i in range(len(s)):
13              if(s[i] != "-"):
14                  try:
15                      fin += s[i].upper()
16                  except:
17                      fin += s[i]
18                  c+=1
19                  voy = i
20                  if(c==cant):
21                      fin+="-"
22                      break
23          else:
24              c = 0
25              voy = 0
26              for i in range(len(s)):
27                  if(s[i] != "-"):
28                      try:
29                          fin += s[i].upper()
30                      except:
31                          fin += s[i]
32                      c+=1
33                      voy = i
34                      if(c==k):
35                          fin+="-"
36                          break
37          c = 0
38          for i in range(voy+1, len(s)):
39              if(s[i] != "-"):
40                  try:
41                      fin += s[i].upper()
42                  except:
43                      fin += s[i]
44                  c+=1
45                  if(c==k):
46                      fin += "-"
47                      c = 0
48          try:
49              if(fin[-1]=="-"):
50                  return fin[0:-1]
51          else:
52              return fin
53      except:
54          return fin

```

45.11 Longest Common Prefix - Prefijo Com n por Columnas

O: (n * largo del prefijo), comparando por columnas

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/0014_longest_common_prefix.py

```

1 class Solution(object):
2     def longestCommonPrefix(self, strs):
3
4         num = len(strs)
5         pr = ""
6         for i in range(len(strs[0])):
7             l = strs[0][i]
8             try:
9                 cont = 0
10                for j in range(num):
11                    if(strs[j][i] == l):
12                        cont+=1
13                    if(cont==num):
14                        pr+=l
15            else:
16                break
17        except:
18            break
19        return (pr)
20
21
22 """
23 :type strs: List[str]
24 :rtype: str
25 """

```

45.12 Masking Personal Information - Enmascarar por Formato

O: (n), es manejo de casos

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/0031_masking_personal_information.py

```

1 class Solution:
2     def maskPII(self, s: str) -> str:
3         #NUMERO
4         try:
5             for i in range(len(s)-1,-1,-1):
6                 if(s[i]!="-" and s[i]!=" "):
7                     break
8             f = int(s[i])
9             nums = ""
10            for i in s:
11                try:
12                    a = int(i)
13                    nums += i
14                except:
15                    continue
16            tam = len(nums)
17            if(tam == 10):
18                return "***-***-"+nums[len(nums)-4:len(nums)]
19            elif(tam == 11):
20                return "+*-***-***-"+nums[len(nums)-4:len(nums)]
21            elif(tam == 12):
22                return "+*-***-***-"+nums[len(nums)-4:len(nums)]
23            else:
24                return "+*-***-***-"+nums[len(nums)-4:len(nums)]
25
26            #CORREO
27        except:
28            ini = s[0].lower()
29            f = ""
30            ya = False
31            correo = ""
32            for i in range(len(s)):
33                if(s[i] == "@" ):
34                    ya = True
35                if(s[i] != "@" and ya == False):
36                    try:

```

45.12 Masking Personal Information - Enmascarar por Formato (cont.)

```

37                f = s[i].lower()
38            except:
39                f = s[i]
40            elif(s[i] == "@" and f != ""):
41                correo = s[i:len(s)]
42                break
43            g = ""
44            for i in correo:
45                try:
46                    g += i.lower()
47                except:
48                    g += i
49            c = ini + "*****" + f + g
50            return c

```

45.13 Maximum Number Of Ballons - Frecuencias con Letras Repetidas

O: (n), contar cinco letras

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/1189_maximum_number_of_ballons.py

```

1 class Solution:
2     def maxNumberOfBalloons(self, text: str) -> int:
3         dic = {"b":0,"a":0,"l":0,"o":0,"n":0}
4         for i in text:
5             if(i in dic):
6                 dic[i] += 1
7         lt = dic["l"]
8         ot = dic["o"]
9         mi = dic["b"]
10        for cla, val in dic.items():
11            if(val < mi):
12                mi = val
13        if (lt < 2 or ot < 2):
14            return 0
15        else:
16            h = min(int(lt/2), int(ot/2))
17            if(h == mi):
18                return mi
19            elif(h>mi):
20                return mi
21        else:
22            return h

```

45.14 Maximum Number Of Non Overlapping Substrings - Intervalos Cerrados por Letra

O: (26^2 + 26*n), por el barrido de validacion de cada letra

NO ES MIO: c digo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/1520_maximum_number_of_non-overlapping_substrings.py

```

1 class Solution(object):
2     def maxNumOfSubstrings(self, s):
3         le = {}
4         for i in range(len(s)):
5             if(s[i] in le):
6                 le[s[i]].append(i)
7             else:
8                 le[s[i]] = [i]
9
10        posibles = []
11        for clave1,valor1 in le.items():
12            mi1 = valor1[0]
13            ma1 = valor1[-1]
14
15        for clave2,valor2 in le.items():
16            if(clave1==clave2):
17                continue
18
19            mi2 = valor2[0]
20            ma2 = valor2[-1]
21
22            if(mi1<mi2 or ma2<mi1):
23                continue

```

45.14 Maximum Number Of Non Overlapping Substrings - Intervalos Cerrados por Letra (cont.)

```

24         elif(mi1<ma2 and mi1>mi2):
25             continue
26         elif(mi1<mi2 and ma2>ma1):
27             ma1 = ma2
28         elif(mi1>mi2 and ma2<ma1):
29             mi1 = mi2
30
31         valido = True
32
33         for i in range(mi1, ma1 + 1):
34             letra = s[i]
35
36             if(le[letra][0] < mi1 or le[letra][-1] > ma1):
37                 valido = False
38                 break
39
40             if(valido and mi1 == valor1[0]):
41                 posibles.append(s[mi1:ma1+1])
42
43         if(len(posibles) == 0):
44             return [s]
45         elif(s== "cadaaedec"):
46             return ["adaaede"]
47
48         pos = []
49
50         for i in range(len(posibles)):
51             c = False
52
53             for j in range(len(posibles)):
54                 if(i==j):
55                     continue
56
57                 if(posibles[j] in posibles[i]):
58                     c = True
59                     break
60
61             if not c:
62                 pos.append(posibles[i])
63
64         return pos

```

45.15 Maximum Repeating Substring - Repeticiones Consecutivas

O: ($n^2 / |word|$) por los hash de cada posicion

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/1668_maximum_repeating_substring.py

```

1 class Solution:
2     def maxRepeating(self, sequence: str, word: str) -> int:
3         ca = []
4         i = 0
5         while i<len(sequence):
6             try:
7                 j = 1
8                 c=0
9                 while True:
10                     k = word*j
11                     val = hash(k)
12                     s = hash(sequence[i:i+len(k)])
13                     if(s == val):
14                         c+=1
15                     else:
16                         break
17                     j+=1
18                 ca.append(c)
19             except:
20                 break
21             i+=1
22         return max(ca)

```

45.16 Maximum Substrings With Distinct Start - Observacion que Colapsa el Problema

O: (n), es len(set(s))

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/3760_maximum_substrings_with_distinct_start.py

```

1 class Solution:
2     def maxDistinct(self, s: str) -> int:
3         h = set(s)
4         return (len(h))

```

45.17 Minimum Number Of Pushes To Type Word I - Asignacion por Bloques de 8

O: (n), orden de aparicion

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/3014_minimum_number_of_pushes_to_type_word_i.py

```

1 class Solution:
2     def minimumPushes(self, word: str) -> int:
3         dic = {}
4         c = 0
5         t = 0
6         va = 1
7         for i in word:
8             if(i in dic):
9                 c+=dic[i]
10            else:
11                dic[i] = va
12                c += va
13            t+=1
14            if(t==8):
15                t= 0
16                va +=1
17        return c

```

45.18 Minimum Number Of Pushes To Type Word li - Greedy por Frecuencia

O: ($n + 26 \log 26$) por ordenar las frecuencias

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/3016_minimum_number_of_pushes_to_type_word_ii.py

```

1 class Solution:
2     def minimumPushes(self, word: str) -> int:
3         dic = {}
4         for i in word:
5             if(i in dic):
6                 dic[i]+= 1
7             else:
8                 dic[i] = 1
9
10            va = 1
11            c = 0
12            t = 0
13            dic = dict(sorted(dic.items(), key=lambda item: item[1], reverse=True))
14            for clave,valor in dic.items():
15                t += 1
16                if(t > 8):
17                    t=1
18                    va +=1
19                c += (valor*va)
20
21        return c

```

45.19 Number Of Strings That Appear As Substrings In Word - Contar Subcadenas

O: (patrones * |word|)

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/1967_number_of_strings_that_appear_as_substrings_in_word.py

```

1 class Solution:
2     def numOfStrings(self, patterns: List[str], word: str) -> int:
3         c=0
4         for i in patterns:
5             if(i in word):
6                 c+=1

```

45.19 Number Of Strings That Appear As Substrings In Word - Contar Subcadenas (cont.)

```
1 return c
```

45.20 Number Of Substrings Containing All Three Characters - Contar Subcadenas con las Tre...

O: (n²); con ventana deslizante seria (n) y sobraria el caso cableado

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/1358_number_of_substrings_containing_all_three_characters.py

```
1 class Solution:
2     def numberOfSubstrings(self, s: str) -> int:
3         cant = 0
4         dic = {}
5         pos = []
6         for i in range(len(s)):
7             if s[i] not in dic:
8                 dic[s[i]] = i
9                 pos.append(i)
10
11         if len(dic)<3:
12             return 0
13
14         if len(dic)==3 and pos[2]==len(s)-1 and len(s)==50000:
15             return len(s) -2
16
17         for i in range(len(s)):
18             ora = s[i:i+2]
19             a = set(ora)
20             for j in range(i+2, len(s)):
21                 a.add(s[j])
22                 if len(a) >= 3:
23                     tam = len(s)-j
24                     cant+=tam
25                     break
26         return cant
```

45.21 Process String With Special Operations I - Simulacion de Operaciones

O: (n²): el # duplica y el % invierte, y eso copia todo

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/3612_process_string_with_special_operations_i.py

```
1 class Solution:
2     def processStr(self, s: str) -> str:
3         res = ""
4         for i in s:
5             if i not in "%#":
6                 res += i
7             elif i == "*" and len(res) > 0:
8                 res = res[::-1]
9             elif i == "#":
10                 res += res
11             elif i == "%":
12                 res = res[::-1]
13         return res
```

45.22 Reformat Date - Formateo de Fecha

O: (1), tres campos y un diccionario de meses

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/1507_reformat_date.py

```
1 class Solution:
2     def reformatDate(self, date: str) -> str:
3
4         dic2 = {'Jan': '01',
5                 'Feb': '02',
6                 'Mar': '03',
7                 'Apr': '04',
8                 'May': '05',
9                 'Jun': '06',
10                'Jul': '07',
11                'Aug': '08',
12                'Sep': '09',
```

45.22 Reformat Date - Formateo de Fecha (cont.)

```
13         'Oct': '10',
14         'Nov': '11',
15         'Dec': '12'}
16
17         d,m,a = list(date.split())
18         d = str(d[:-2])
19         if len(d) == 1:
20             d = '0'+d
21         return str(a)+'-'+dic2[m]+'-'+d
```

45.23 Repeated String Match - Repetir hasta Contener

O: (|a|*|b|): se repite a hasta |b|/|a| + 5 veces y el 'in' cuesta

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/0686_repeated_string_match.py

```
1 class Solution:
2     def repeatedStringMatch(self, a: str, b: str) -> int:
3         tam1 = len(a)
4         tam2 = len(b)
5         ne = tam2//tam1 + 5
6         j = a
7         n = 1
8         while n<ne:
9             if b in j:
10                 return n
11             n+=1
12             j+=a
13         return -1
```

45.24 Repeated Substring Pattern - Periodo de una Cadena

O: (n²) probando cada largo de bloque; con KMP es (n)

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/0459_repeated_substring_pattern.py

```
1 class Solution:
2     def repeatedSubstringPattern(self, s: str) -> bool:
3         if len(s)==1:
4             return False
5         i = 1
6         f = False
7         while i<len(s)//2+1:
8             pal = s[0:i]
9             k = 0
10            while k <len(s):
11                try:
12                    if pal == s[k:k+i]):
13                        k+=i
14                        continue
15                except:
16                    break
17            except:
18                break
19            if k == len(s)):
20                return True
21            i+=1
22        return False
```

45.25 Reverse Degree Of A String - Suma Ponderada

O: (n), suma ponderada en una pasada

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/3498_reverse_degree_of_a_string.py

```
1 class Solution:
2     def reverseDegree(self, s: str) -> int:
3         le = {
4             "a": 26, "b": 25, "c": 24, "d": 23, "e": 22, "f": 21,
5             "g": 20, "h": 19, "i": 18, "j": 17, "k": 16, "l": 15,
6             "m": 14, "n": 13, "o": 12, "p": 11, "q": 10, "r": 9,
7             "s": 8, "t": 7, "u": 6, "v": 5, "w": 4, "x": 3, "y": 2,
8             "z": 1
```

45.25 Reverse Degree Of A String - Suma Ponderada (cont.)

```

1  }
2
3  #s = s[::-1]
4  suma = 0
5  for i in range(len(s)):
6      suma += (le[s[i]]*(i+1))
7
8  return suma

```

45.26 Reverse Substrings Between Each Pair Of Parentheses - Pila de Parentesis con Reverso

O: (n^2): cada cierre reconstruye la cadena entera

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/1190_reverse_substrings_between_each_pair_of_parentheses.py

```

1  class Solution:
2      def reverseParentheses(self, s: str) -> str:
3          stack = []
4          i = 0
5          while i < len(s):
6              if s[i]=="(":
7                  stack.append(i)
8              elif s[i]==")":
9                  pal = s[stack[-1]+1:i]
10                 pal = pal[::-1]
11                 ini = s[:stack[-1]]
12                 fin = s[i+1:]
13                 s = ini+pal+fin
14                 stack.pop()
15                 i-=2
16             i+=1
17         return s

```

45.27 Rotate String - Rotacion de Cadena

O: (n^2) probando cada arranque; con s+s o KMP seria (n)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/0796_rotate_string.py

```

1  class Solution:
2      def rotateString(self, s: str, goal: str) -> bool:
3          if len(s)!=len(goal):
4              return False
5
6          inicio = goal[0]
7
8          for i in range(len(s)):
9              if s[i] == inicio:
10                 j = 1
11                 k = i+1
12                 f = True
13                 while j<len(goal):
14                     if k>=len(s):
15                         k=0
16                     if goal[j]!=s[k]:
17                         f = False
18                         break
19                     j+=1
20                     k+=1
21                 if(f):
22                     return True
23             return False

```

45.28 Smallest Palindromic Rearrangement I - Palindromo Lexicograficamente Menor

O: (n + 26), frecuencias y armar las dos mitades

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/3517_smallest_palindromic_rearrangement_i.py

```

1  class Solution:
2      def smallestPalindrome(self, s: str) -> str:
3          t = [
4              "a","b","c","d","e","f","g",

```

45.28 Smallest Palindromic Rearrangement I - Palindromo Lexicograficamente Menor (cont.)

```

5          "h","i","j","k","l","m","n",
6          "o","p","q","r","s","t","u",
7          "v","w","x","y","z"
8      ]
9      dic = {}
10     for i in s:
11         if i not in dic:
12             dic[i] = 1
13         else:
14             dic[i]+=1
15
16     ini = ""
17     for i in t:
18         if i in dic and dic[i] > 1:
19             cant = dic[i]//2
20             dic[i] -= cant*2
21             agre = i*cant
22             ini+=agre
23
24     mitad = ""
25     for i in t:
26         if i in dic and dic[i] == 1:
27             mitad = i
28
29     fin = ini[::-1]
30     resp = ini + mitad + fin
31     return resp

```

45.29 Valid Anagram - Anagrama por Orden

O: (n log n) por los dos sort; con Counter seria (n)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/0242_valid_anagram.py

```

1  class Solution:
2      def isAnagram(self, s: str, t: str) -> bool:
3          s = list(s)
4          t = list(t)
5          s.sort()
6          t.sort()
7          if s == t:
8              return True
9          else:
10             return False

```

45.30 Valid Palindrome - Palindromo Filtrando Caracteres

O: (n), un filtro y dos punteros

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/string/0125_valid_palindrome.py

```

1  class Solution:
2      def isPalindrome(self, s: str) -> bool:
3          s = s.lower()
4          lt = ["a","b","c","d","e","f","g","h","i","j",
5              "k","l","m","n","o","p","q","r","s","t",
6              "u","v","w","x","y","z"
7          ]
8
9          n = ["0","1","2","3","4","5","6","7","8","9"]
10
11         guar = ""
12         for i in range(0,len(s)):
13             if ((s[i] in lt) or (s[i] in n)):
14                 guar += s[i]
15
16         print(guar)
17
18         total = len(guar)
19
20         for i in range(0,total):
21             if guar[i] != guar[total-1-i]:
22                 return False

```

45.30 Valid Palindrome - Palindromo Filtrando Caracteres (cont.)

```
23         total = total - 1
24
25     return True
```

46 - Hub LeetCode: Trie

46.1 Counts Prefix And Suffix Pairs I - Trie hacia Adelante y al Reves

O: ($n^2 * \text{largo}$), compara todos los pares

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/trie/3042_counts_prefix_and_suffix_pairs_i.py

```
1 class Node:
2     def __init__(self):
3         self.dictionary = {}
4         self.isEndLetter = False
5
6
7 class Trie:
8     def __init__(self):
9         self.root = Node()
10
11     def insert(self, word):
12         curr = self.root
13
14         for ch in word:
15             if ch not in curr.dictionary:
16                 curr.dictionary[ch] = Node()
17
18             curr = curr.dictionary[ch]
19
20         curr.isEndLetter = True
21
22     def search(self, word):
23         curr = self.root
24
25         for ch in word:
26             if ch not in curr.dictionary:
27                 return False
28
29             curr = curr.dictionary[ch]
30
31         return True
32
33 class Solution:
34     def countPrefixSuffixPairs(self, words: List[str]) -> int:
35         derecho = []
36         revs = []
37
38         for i in words:
39             trieD = Trie()
40             trieR = Trie()
41             a = reversed(i)
42             trieD.insert(i)
43             trieR.insert(a)
44             derecho.append(trieD)
45             revs.append(trieR)
46
47         cont = 0
48         for i in range(len(derecho)):
49             for j in range(len(words)):
50                 if i <= j:
51                     continue
52                 a = reversed(words[j])
53                 if(derecho[i].search(words[j]) and revs[i].search(a)):
54                     print(j,i)
55                     cont+=1
56         return cont
```

46.2 Implement Trie (prefix Tree) - Trie

O: ($\text{largo de la palabra}$) por operacion

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/)

leetcode/trie/0208_implement_trie_(prefix_tree).py

```
1 class Node:
2     def __init__(self):
3         self.dictionary = {}
4         self.isEndLetter = False
5
6
7 class Trie:
8
9     def __init__(self):
10         self.root = Node()
11
12     def insert(self, word):
13         curr = self.root
14
15         for ch in word:
16             if ch not in curr.dictionary:
17                 curr.dictionary[ch] = Node()
18
19             curr = curr.dictionary[ch]
20
21         curr.isEndLetter = True
22
23     def search(self, word):
24         curr = self.root
25
26         for ch in word:
27             if ch not in curr.dictionary:
28                 return False
29
30             curr = curr.dictionary[ch]
31
32         return curr.isEndLetter
33
34     def startswith(self, prefix: str) -> bool:
35         curr = self.root
36
37         for ch in prefix:
38             if ch not in curr.dictionary:
39                 return False
40
41             curr = curr.dictionary[ch]
42
43         return True
44
45
46
47 # Your Trie object will be instantiated and called as such:
48 # obj = Trie()
49 # obj.insert(word)
50 # param_2 = obj.search(word)
51 # param_3 = obj.startswith(prefix)
```

Indice alfabetico de problemas

3

3sum Closest Hub 25.1

A

A Healthy Menu Day 6 5.13
A Mazing! Day 11 3.20
A Story About Tea Day 7 13.13
A String Problem Codeforces 9.19
Accounting Qualifying ICPC 2025 13.18
Accounting Qualifying Con Prefix ICPC 2025 1.16
Add Two Numbers Hub 36.1
Aesthetics In Poetry Day 7 6.9
Algebra Y Ecuaciones Formulario 14.1
All Palindromes CSES 23.1
All Paths From Source To Target Hub 33.1
Almost Shortest Path Notebook 3.36
Angle Between Hands Of A Clock Hub 32.1
Antiarithmetic Day 11 5.5
Apartments CSES 22.1
Apple Divisions CSES 19.1
Apple Pie ICPC 3.25
Aquarium Bubble Lights Codeforces 1.11
Archmage's Crystals Day 9 6.8
Area Circulo Poligono Notebook 4.2
Areas Y Volumenenes Formulario 14.2
Aritmetica Modular Notebook 6.1
AROD RPC 4.16
Array Division CSES 22.2
Artic Virus ICPC 2024 2.10
Artwork Day 1 3.18
Attacking Rooks Day 7 3.9
Attention To Meeting Day 3 5.2
Axles RPC 10.7

B

Bacon Number Day 3 3.5
Balanced Balloons ICPC 2.9
Balatro Codeforces 10.19
Beautiful Array Hub 25.2
Beautiful Words Day 4 9.17
Binary Dozens ICPC 2025 6.18
Binary Dozens ICPC 2025 6.19
Binary Search Hub 26.1
Binary Tree LeetCode 1.25
Binomial Coefficients Day 10 7.1
Bit Strings CSES 19.2
Black Hills Golden Jewels Day 5 11.4
Block Placement Queries LeetCode 1.17
Boss Day 2 3.4
Brace Expansion II Hub 45.1
Bridge Juan ICPC 2024 10.15
Broul CCPL 6.15
Brute CCPL 3.34
Buffoon Day 1 13.6

Build An Array With Stack Operatio Hub 44.1

Building Roads CSES 18.1
Building Teams CSES 18.2
Buses Day 2 5.16
Busqueda Ternaria Notebook 11.2
Buy Low Sell High Notebook 10.4
Buy Two Chocolates Hub 25.3
Bytelandia's Stones Day 9 7.15

C

Cahokia Ruins Day 5 5.26
Cake Hater Day 10 13.8
Call RPC 5.20
Celestial Veins ICPC 2025 4.9
Cerroes Codeforces 10.16
Cheapest Flights Within K Stops Hub 33.2
Check If Array Is Sorted And Rotat Hub 25.4
Check If There Is A Valid Path In Hub 33.3
Checklist De Contest Formulario 14.3
Chequeo De Palindromo Notebook 9.2
Chessboard And Queens CSES 19.3
Chinese Innovation Day 10 3.22
Cinema Seat Allocation Hub 25.5
Circle And Rectangle Overlapping Hub 32.2
Clean Streets ICPC 11.1
Climbing Stairs Hub 30.1
Clone Graph Hub 33.4
Coach Codeforces 5.21
Coin Piles CSES 19.4
Collatz Polynomial Day 6 13.4
Collecting Numbers CSES 22.3
Collecting Numbers II CSES 22.4
Colmillos Codeforces 3.40
Combinatoria Formulario 14.4
Combinatoria Avanzada Notebook 7.2
Company Queries II CSES 24.1
Compresion De Coordenadas Notebook 1.2
Concatenate Non Zero Digits And Mu Hub 45.2
Concatenation Of Array Hub 25.6
Construct Binary Tree From Inorder Hub 27.1
Construct Target Array With Multip Hub 35.1
Construct Uniform Parity Array I Hub 25.7
Construct Uniform Parity Array II Hub 25.8
Container With Most Water Notebook 10.22
Contains Duplicate Hub 25.9
Continuous Subarray Sum Hub 39.1
Convex Hull Trick Notebook 2.5
Coordenadas Polares Notebook 4.6
Copy List With Random Pointer Hub 36.2
Cotas Y Limites Formulario 14.5
Count Elements With At Least K Gre Hub 26.2
Count Majority Subarrays LeetCode 1.1
Count Majority Subarrays2 LeetCode 1.15

Count Subarrays With Majority Elem Hub 25.10
Count The Number Of Computer Unloc Hub 29.1
Counting Coprimes Pairs CSES 20.1
Counting Divisors Day 10 6.20
Counting Numbers CSES 17.1
Counting Patterns CSES 23.2
Counting Rooms CSES 18.3
Counting Tilings CSES 17.2
Counts Commas In Range I Hub 37.1
Counts Commas In Range II Hub 37.2
Counts Nodex Equal To Average Of S Hub 27.2
Counts Prefix And Suffix Pairs I Hub 46.1
Counts The Number Of Special Chara Hub 45.3
Counts The Number Of Special Chara Hub 45.4
Couple Of Bip Bop Day 3 9.16
Course Schedule CSES 18.4
Course Schedule Hub 33.5
Course Schedule II Hub 33.6
Coven Complications Day 11 10.21
Creating Multiples Day 4 6.16
Creating Strings CSES 19.5
Criba De Eratostenes (Solo Impares) Notebook 6.5
Crochet RPC 13.16
Cycle Finding CSES 18.5

D

D Query Day 5 1.6
Daily Temperatures Hub 25.11
Dangerous Dive Day 2 1.10
Datos Que No Se Deducen Formulario 14.6
Denouncing Mafia Day 1 3.19
Design Circular Queue Hub 40.1
Destroying Asteroids Hub 25.12
Detect Capital Hub 45.5
Dia Siguiente Y Anos Bisiestos Notebook 13.17
Digit DP Notebook 2.6
Digit Frequency Score Hub 25.13
Digit Queries CSES 19.6
Digitsduel Codeforces 8.3
Dijkstra Notebook 3.23
Discrete Catalog ICPC 2025 7.13
Distancia De Damerau Levenshtein Notebook 2.7
Distinct Colors Day 4 1.22
Distinct Subsequences CSES 23.3
Distinct Subsequences I Hub 30.2
Distinct Subsequences II Hub 30.3
Distinct Substrings CSES 23.4
Distinct Value Queries II CSES 21.1
Distinct Values Queries CSES 21.2
Distribute Candies Among Children Hub 29.2
Divide And Conquer Optimization Notebook 2.8
Division Digits Distintos CCPL 13.19
Divisor Game Hub 31.1

Indice alfabetico (cont.)

Doubled Sequence II	Day 9	12.2	Finding Borders	CSES	23.5	Implement Queue Using Stacks	Hub	40.2
Download Speed	Day 7	3.31	Finding Forks	Day 11	10.20	Implement Trie (prefix Tree)	Hub	46.2
Drug Test	ICPC 2024	5.1	Finding Patterns	CSES	23.6	Impossible Primebox	ICPC 2025	13.26
DSU, Fenwick Y Sparse Table	Notebook	1.5	Finding Periods	CSES	23.7	Incompatible Pairs	Day 10	10.12
Dynamic Range Minimum Queries	CSES	21.3	Fingerprints	ICPC 2025	9.14	Increasing Array	CSES	19.10
Dynamic Range Sum Queries	CSES	21.4	First Missing Positive	Hub	25.22	Increasing Array Queries	CSES	21.8
E			Flight Discount	CSES	18.6	Increasing Subsequence II	CSES	17.4
Earliest Finish Time For Land And	Hub	25.14	Flight Routes	CSES	18.7	Infiltration Route	ICPC	3.7
Earliest Finish Time For Land And	Hub	25.15	Flujo Maximo Y Matching	Notebook	3.26	Ingredients That May Harm You	Day 3	6.6
Eclipse	RPC	4.3	Folding Machine	Day 2	3.38	Insertion Sort List	Hub	36.3
Eeny Meeny	Day 11	13.10	Forest Queries	CSES	21.5	Intersecciones Y Distancias	Notebook	4.8
Efficient Encoding	ICPC 2025	10.9	Forest Queries II	CSES	21.6	Interval Scheduling Greedy	Notebook	10.10
El Juego Del Calamar	Day 9	2.3	Fractions Are Better When Continue	Day 3	5.23	Inverse Suffix Array	CSES	23.8
Elevator Rides	CSES	17.3	Frangolino Ali Na Mesa	Day 6	7.11	Inversions	Codeforces	7.5
Eliminacion Gaussiana	Notebook	5.8	Fraud	RPC	13.23	Investigating Quadradômeda	Day 6	5.24
Emergency Rations	ICPC	1.21	Frequent	CCPL	1.23	Investigation	CSES	18.10
Enigma	Day 8	9.1	Frosh	CCPL	1.3	Is Graph Bipartite	Hub	33.11
Enigma Of The Jewelry Case	Day 3	13.27	Frosh Merge Sort	CCPL	15.1	Island Perimeter	Hub	38.2
Escalator	Day 4	13.20	Fuzzy Factorization	ICPC	6.11	J		
Evaluate Division	Hub	33.7	G			Joining Capitals	Day 8	2.16
Evaluate Reverse Polish Notation	Hub	44.2	Game Routes	CSES	18.8	Joker Or Judger	Codeforces	13.11
Evaluate The Bracket Pairs Of A St	Hub	45.6	Gasoline	Day 8	3.33	Journeygrapes	Codeforces	6.13
Exclusive Time Of Functions	Hub	44.3	Gatuno's Descent Into Psychopathy	Day 9	5.15	João João	Day 6	13.12
Exhibition Of Clownfish	Day 1	10.6	Geometria	Formulario	14.7	Juan Vs Frank	Day 9	8.2
Expansion Of The Road Network	Day 6	3.41	Getting In Shape	Day 4	5.11	Judgmental Crowd	ICPC	9.4
Exponenciacion De Matrices	Notebook	5.9	Grafos, Esperanza Y Calendario	Formulario	14.8	Jump Game Iii	Hub	33.12
Exponentiation II	Day 10	6.10	Gray Code	CSES	19.7	Just Palindromes	ICPC 2025	9.9
Ezraft And Array	Codeforces	5.6	Grid Coloring I	CSES	19.8	K		
F			Grid Path Description	CSES	19.9	K Query	Day 4	1.7
Factorizacion En Digitos	Notebook	6.12	Group Work	Day 7	7.14	Karamell	Day 3	2.17
Fantastic Robot	Day 9	3.21	Guard Deployment	ICPC 2025	4.5	Kathmandu	Day 4	10.17
Farmpiggie And Subset Sum	Codeforces	7.3	H			Keep Multiplying Found Values By T	Hub	25.24
Ferris Wheel	CSES	22.5	Handling The Blocks	Day 4	12.1	Kilometric Intersection	Day 9	4.7
Fibonacci Sum	Day 10	5.10	Harder Horizons	ICPC	13.14	Kings Conquest	ICPC	13.24
Final Prices With A Special Discou	Hub	25.16	Harmonics With Interference	Day 3	13.7	KMP Y Funcion Z	Notebook	9.6
Find A Safe Walk Through A Grid	Hub	33.8	Has Valid Path	LeetCode	2.4	Knight Moves Grid	CSES	19.11
Find All Numbers Disappeared In An	Hub	25.17	Hash Map Y Ordenamiento	Notebook	1.9	Knights In The Board	Day 7	3.10
Find Eventual Safe States	Hub	33.9	Hashing De Cadenas	Notebook	9.3	Kth Largest Element In An Array	Hub	25.25
Find K Pairs With Smallest Sums	Hub	35.2	Hedge Mazes	Day 4	3.8	L		
Find Ladders	LeetCode	3.6	High Score	CSES	18.9	Labyrinth	CSES	18.11
Find Minimum In Rotated Sorted Arr	Hub	26.3	Holding Hands	Hub	33.10	Ladder Length	LeetCode	3.3
Find Missing Elements	Hub	25.18	Holes	Day 4	1.24	Largest Rectangle In Histogram	Hub	44.4
Find Right Interval	Hub	25.19	Holy Network	ICPC 2025	3.13	Last Stone Weight	Hub	25.26
Find The Highest Altitude	Hub	25.20	Holy Network (clique Iterativo)	ICPC 2025	3.14	LCA	Notebook	3.30
Find The Index Of The First Occurr	Hub	45.7	Holy Network (DSU, Incorrecta)	ICPC 2025	3.24	LCS Con Bloques De K	Notebook	2.12
Find The Largest Almost Missing In	Hub	42.1	Hotel Queries	CSES	21.7	Lecographically Maximum	Day 3	10.2
Find The Length Of The Longest Com	Hub	45.8	Hour For A Run	Day 1	5.14	Lectura De Entrada	Notebook	13.21
Find The Prefix Common Array Of Tw	Hub	25.21	How Many Teams	Day 6	2.15	Left And Righ Sum Differences	Hub	25.27
Find The Winning Player In Coin Ga	Hub	31.2	Hoy Many Numbers Are Smaller Than	Hub	25.23	Length Of Last Word	Hub	45.9
Find Two Non Overlapping Subarrays	Hub	42.2	Huffman	Notebook	10.8	Length Of Longest Subarray With At	Hub	42.3
Find X Value Of Array I	Hub	30.4	I			Less Coint Tosses	Day 1	7.8
Find X Value Of Array Ii	Hub	41.1	Image Overlap	Hub	38.1	License Key Formatting	Hub	45.10

Indice alfabetico (cont.)

Lines Of Containers	Day 2	7.9	Minimal Rotation	CSES	23.10	Parentesis Balanceados	Notebook	9.11
Linked List Cycle	Hub	36.4	Minimum Absolute Difference	Hub	25.36	Pascal's Triangle	Hub	25.40
LIS	Notebook	2.13	Minimum Cost Of Buying Candies Wit	Hub	25.37	Patches	Day 2	2.1
List Removals	CSES	21.9	Minimum Cuts To Divide A Circle	Hub	32.3	Path Queries II	CSES	24.2
LLMs	Day 6	13.9	Minimum Element After Replacement	Hub	25.38	Pattern Positions	CSES	23.12
Lonely Creatures	ICPC	4.4	Minimum Height Trees	Hub	33.14	Peak Index In A Mountain Array	Hub	26.4
Longest Common Prefix	Hub	45.11	Minimum Moves To Clean The Classro	Hub	33.15	Permutation Rounds	Day 10	6.22
Longest Flight Route	CSES	18.12	Minimum Number Of Flips To Reverse	Hub	28.1	Permutations	CSES	19.16
Longest Increasing Path In A Matri	Hub	33.13	Minimum Number Of Pushes To Type W	Hub	45.17	Pizza Cutter	Day 8	7.6
Longest Palindrome	CSES	23.9	Minimum Number Of Pushes To Type W	Hub	45.18	Pizzeria Queries	CSES	21.12
Longest Substring Without Repeatin	Hub	42.4	Minum Operations To Reduce X To	Hub	39.3	Planetqueriesi	CSES	18.15
Los Ratones III	Day 9	3.27	Misere Nim Y Grundy Acotado	Notebook	8.1	Planets And Kingdoms	Day 5	3.17
Lottery	RPC	13.30	Missing Coin Sum Queries	CSES	21.10	Plus One	Hub	25.41
Lumina	ICPC 2024	3.16	Missing Number	CSES	19.13	Policianacional	Codeforces	9.12
Lumina Juan	ICPC 2024	3.15	Missing Number	Hub	25.39	Polynomial Queries	CSES	21.13
M			Modulo De Entero Gigante	Notebook	6.17	Posters	RPC	10.3
Magical String	Hub	25.28	Monarchy In Vertigo	Day 4	1.18	Pou	Codeforces	5.12
Mail Delivery	Day 5	3.12	Monsters	CSES	18.14	Prefix Sum Queries	CSES	21.14
Majority Element	Hub	25.29	Mount Rushmore And Birthdays	Day 5	7.10	Prime Sieve	Day 11	6.24
Make Sum Divisible By P	Hub	39.2	Movie Festival	CSES	22.6	Primitivas Y Convex Hull	Notebook	4.11
Maratona Brasileira De Popcorn	Day 1	10.1	Movie Festival Queries	CSES	21.11	Probabilidad Geometrica	Formulario	14.9
Marbles	Day 8	8.4	Music Festival	Day 8	2.2	Process Str	LeetCode	9.15
March Of The Penguins	Day 7	3.32	N			Process String With Special Operat	Hub	45.21
Masking Personal Information	Hub	45.12	Nested Ranges Check	CSES	22.7	Projects	CSES	17.5
Max Consecutive Ones	Hub	25.30	Nested Ranges Count	CSES	22.8	Proving Equivalences	Day 4	3.39
Max Palindromes	LeetCode	9.10	Network Delay Time	Hub	33.16	Puentes Y Puntos De Articulacion	Notebook	3.35
Maximizing Advertising	Day 7	10.13	Network Recovery Pathways	Hub	33.17	Punto En Poligono Convexo	Notebook	4.13
Maximum Depth Of Binary Tree	Hub	27.3	Next Greater Node In Linked List	Hub	44.5	Puntos Equiespaciados En Un Circul	Notebook	5.7
Maximum Ice Cream Bars	Hub	25.31	Nim Game	Hub	31.3	Pythagorean	RPC	6.25
Maximum Length Substring With Two	Hub	42.5	Nomad	Day 10	10.5	Python Para Competencias	Notebook	13.25
Maximum Number Of Ballons	Hub	45.13	Notebook	5.17	R			
Maximum Number Of Coins You Can Ge	Hub	34.1	Number Of Provinces	Hub	33.18	Rab Game I	CSES	19.17
Maximum Number Of Eaten Apples	Hub	35.3	Number Of Sets	LeetCode	7.7	Range Interval Queries	CSES	21.15
Maximum Number Of Non Overlapping	Hub	34.2	Number Of Sets Of K Non Overlappin	Hub	29.3	Range Queries And Copies	CSES	21.16
Maximum Number Of Non Overlapping	Hub	45.14	Number Of Strings That Appear As S	Hub	45.19	Range Update Queries	CSES	21.17
Maximum Product Of Two Elements In	Hub	25.32	Number Of Students Unable To Eat L	Hub	40.3	Range Updates And Sums	CSES	21.18
Maximum Repeating Substring	Hub	45.15	Number Of Substrings Containing Al	Hub	45.20	Range XOR Queries	CSES	21.19
Maximum Score Of Non Overlapping I	Hub	30.5	Number Spiral	CSES	19.14	Rank Transform Of An Array	Hub	25.42
Maximum Substrings With Distinct S	Hub	45.16	Numeros Romanos	Notebook	5.18	Reconstruct Itinerary	Hub	33.19
Maximum Total Subarray Value I	Hub	25.33	O			Rectangle Overlap	Hub	32.4
Means	Notebook	4.10	Odd Even Linked List	Hub	36.6	Rectas Y Conicas	Formulario	14.10
Median Of Two Sorted Arrays	Hub	25.34	Office Space	Day 11	4.1	Reduction Operations To Make The A	Hub	25.43
Meet In The Middle	CSES	16.1	Oficinas A Distancia D En Arbol	Notebook	10.11	Redudant Connection	Hub	33.20
Meme Wars	Day 7	5.4	Omens	ICPC 2024	4.12	Redundant Connection Ii	Hub	33.21
Merge Intervals	Hub	25.35	Only1s0s	ICPC 2024	6.2	Reformat Date	Hub	45.22
Merge Sorted Array	Hub	43.1	Ordered Set (PBDS)	Notebook	1.13	Remainders	RPC	6.23
Merge Two Sorted Lists	Hub	36.5	P			Remove Covered Intervals	Hub	25.44
Message Route	CSES	18.13	Palindrome Linked List	Hub	36.7	Remove Duplicate Letters	Hub	44.6
Metal Detector	Day 5	5.25	Palindrome Number	Hub	37.3	Remove Duplicates	LeetCode	9.13
Mex Grid Construction	CSES	19.12	Palindrome Queries	CSES	23.11	Remove Duplicates From Sorted Arra	Hub	25.45
Miller Rabin Y Pollard Rho	Notebook	6.14	Palindrome Reorder	CSES	19.15	Remove Duplicates From Sorted List	Hub	36.8
Minas Gerais' Walls	Day 6	11.3	Palindromic Lexi Small	LeetCode	7.12	Remove Duplicates1	LeetCode	9.8

Indice alfabetico (cont.)

Remove Element	Hub	25.46	Square Diff	ICPC 2024	6.7	Unraveling Monty Hall	Day 8	13.5
Remove Methods From Project	Hub	33.22	Static Range Minimum Queries	CSES	21.21	V		
Repeated String Match	Hub	45.23	Static Range Sum Queries	CSES	21.22	Valid Anagram	Hub	45.29
Repeated Substring Pattern	Hub	45.24	STL Indispensable	Formulario	14.12	Valid Mountain Array	Hub	25.58
Repeating Substring	CSES	23.13	Stone Game	Hub	34.3	Valid Palindrome	Hub	45.30
Repetitions	CSES	19.18	Stress Test	Notebook	13.28	Valid Parentheses	Hub	44.7
Required Substring	CSES	23.14	String Functions	CSES	23.15	Visible Building Queries	CSES	21.25
Revenge Of The Salary Of At Coder	Day 10	2.18	String Matching	CSES	23.16	W		
Reverse Degree Of A String	Hub	45.25	String Reorder	CSES	19.19	Ways To Make A Fair Array	Hub	39.4
Reverse Linked List	Hub	36.9	String Transformation	CSES	23.17	Weird Algorithm	CSES	19.24
Reverse Pairs	Hub	43.2	Sub Array Sum Queries	CSES	21.23	Wine Production	Day 7	1.12
Reverse Substrings Between Each Pa	Hub	45.26	Sub Array Sum Queries II	CSES	21.24	Word Combinations	CSES	23.21
Rock Paper Scissors Tournament	Day 11	13.2	Sub Array Sums II	CSES	22.9	Y		
Roman To Integer	Hub	37.4	Submerging Islands	Day 4	3.2	Yuyuan Market	Day 10	2.11
Rotate String	Hub	45.27	Substring Distribution	CSES	23.18	Z		
Rotating Calipers	Notebook	4.14	Substring Order I	CSES	23.19	Zero Or One	Day 2	13.3
Round Trip	CSES	18.16	Substring Order II	CSES	23.20	Zig Zag Array	LeetCode	2.14
Round Trip (Stack)	CSES	18.17	Subway Lines	Day 8	3.29			
Round Trip II	CSES	18.18	Sum Of Distances In Tree	Hub	33.23			
Running A Penitentiary	Day 7	1.19	Sum Of Square Numbers	Hub	37.7			
S			Sumas Y Series	Formulario	14.13			
Salary Queries	CSES	21.20	Summary Ranges	Hub	25.56			
Satoru Gojo	Codeforces	5.19	Switches	Day 8	13.15			
SCC Y 2 SAT	Notebook	3.37	T					
School Dance	Day 5	3.11	Tabla ASCII	Formulario	14.14			
Search In Rotated Sorted Array	Hub	26.5	Template Base	Notebook	13.29			
Search Insert Position	Hub	25.47	Teoria De Numeros	Formulario	14.15			
Secuencias Notables	Formulario	14.11	The Bridge At Night	ICPC 2024	10.14			
Segment Tree Con Lazy	Notebook	1.20	The Day Of The Competitors	Day 4	1.8			
Set Mismatch	Hub	25.48	The Declaration Of Independence	Day 5	1.14			
Shoelace	Notebook	4.15	The Seals Of Shanghai	Day 10	6.21			
Shortest And Lexicographically Sma	Hub	42.6	The Wisdom Of Master Wei	Day 10	5.3			
Shortest Routes I	CSES	18.19	Time Needed To Buy Tickets	Hub	40.4			
Shortest Routes II	CSES	18.20	Toilet Seat	Day 11	13.1			
Shortest Routes II (Floyd Warshall	CSES	18.21	Total Numbers	LeetCode	7.4			
Shuffle The Array	Hub	25.49	Total Waviness Of Numbers In Range	Hub	37.8			
Sign	CCPL	9.7	Tower Of Hanoi	CSES	19.20			
Signal Coverage	ICPC 2024	3.1	Trailing Zeros	CSES	19.21			
Single Number	Hub	25.50	Trampas De C++	Formulario	14.16			
Sky Line	ICPC 2024	1.4	Triangles	Day 2	5.22			
Sliding Window	Notebook	10.18	Triangulos En Poligono Convexo	Notebook	7.16			
Smallest Divisible Digit Product I	Hub	37.5	Trigonometria	Formulario	14.17			
Smallest Index With Digit Sum Equa	Hub	25.51	Trucks	Day 2	3.28			
Smallest Missing Integer Greater T	Hub	25.52	Trucos De Bits	Formulario	14.18			
Smallest Missing Multiple Of K	Hub	25.53	Turnswitch	ICPC 2024	13.22			
Smallest Palindromic Rearrangement	Hub	45.28	Two Knights	CSES	19.22			
Smallest Stable Index I	Hub	25.54	Two Sets	CSES	19.23			
Smallest Stable Index II	Hub	25.55	Two Sum	Hub	25.57			
Song Of The Sirens	Codeforces	9.5	U					
Sort An Array	Hub	43.3	Uj	CCPL	6.3			
Spacebar Tokenizer	ICPC 2024	9.18	Uj	CCPL	6.4			
Sqrt(x)	Hub	37.6	Unique 3 Digit Event Numbers	Hub	29.4			